FoliaTrace - complete first-party source listing

97 code files from the accompanying allow-listed source archive.

The archive also contains data, evaluation reports, README and AI disclosure.

Third-party runtimes, model binaries, credentials, caches and saved user records are excluded.

Non-ASCII code points are shown as reversible Python-style Unicode escapes.

Long lines wrap visually; the original file bytes and hashes remain in source.zip.

AI-assisted development by the entrant with OpenAI Codex and Anthropic Claude.

Prior EthnoHack work is disclosed in README; this listing includes the complete packaged implementation.

Source archive SHA-256: c7bdb1217e79785d11d9ff63c9276e4707c5df01ce797140dec5f214a338c498

app.py
SHA-256: 1a4a624fbbb70e556be556a581e6c9fbf3d90b03babf3bbd8f3a5780f7497e21

```
    1 import uvicorn
    2
    3 if __name__ == "__main__":
    4     uvicorn.run("src.api.app:app", host="127.0.0.1", port=8000)
```

frontend/app.js
SHA-256: 55b6a87cc315376357975a115dff3dddcd2f849f72625da80bed237d94360d4d

```
import {initHandoffs} from './handoff.js';
import {initConsultation} from './consultation.js';
import {initQuestionReview} from './question-review.js';
import {publicationText} from './publication-text.js';
const $ = (s) => document.querySelector(s);
let data,
  health,
  passages = [],
  custom = false,
  selected = null,
  graphFocus = null,
  recorded = false,
  analysisSerial = 0,
  requestedSourceName = null;
let readingResumeActive=false,restoringReading=false;
const RESUME_READING_KEY='foliatrace.resume.v1';
const identityComparisons=new Map();
const statementReadingTargets=new Map();
const studyComparisons=new Map();

// Plain-language meaning of every machine state shown to a reader.
const STATUS = {
  RESOLVED_MODERN_NAME: ["ok", "Matches a modern scientific name", "A modern checklist accepts this name. That still does not prove what the historical author meant."],
  PROVISIONAL: ["warn", "One candidate, via a common name", "Only one species in this snapshot carries this name. Species outside the snapshot may share it."],
  AMBIGUOUS: ["bad", "Ambiguous: several species", "Several species share this name. Evidence is kept separate for each candidate."],
  GENUS_ONLY: ["warn", "Genus only", "The name points to a whole genus. Each species in the snapshot is a separate branch."],
  OPEN_WORLD_AMBIGUOUS: ["warn", "Other regional species share this name", "The snapshot holds one species, but GBIF lists other species of this genus with the same common name and records in Great Britain."],
  UNRESOLVED: ["neutral", "Not resolved", "No candidate found. That is not evidence that the plant is unknown."],
};
const RATING = { STRONG: ["ok", "Strong fit"], MODERATE: ["warn", "Moderate fit"], WEAK: ["neutral", "Weak fit"], NOT_ASSESSABLE: ["neutral", "Not assessable"] };
const MATRIX_STATUS = { MATCH: ["ok", "fits"], PARTIAL: ["warn", "partly fits"], CONFLICT: ["bad", "conflicts"] };
const RESEARCH_STATE = {
  HUMAN_FINDINGS_FAVOURABLE: ["ok", "Human studies report benefit"],
  HUMAN_FINDINGS_EARLY: ["warn", "Early human findings, too few to rely on"],
  HUMAN_FINDINGS_MIXED: ["warn", "Human findings mixed or inconclusive"],
  HUMAN_NO_BENEFIT_REPORTED: ["bad", "Human studies report no benefit"],
  HUMAN_STUDIED_RESULT_UNCLEAR: ["neutral", "Studied in humans, result unclear"],
  PRECLINICAL_ONLY: ["warn", "Laboratory studies only"],
  SPECIES_NAMED_NO_TESTED_RESULT: ["neutral", "Mentioned, not the study subject"],
  COMMON_NAME_ONLY: ["neutral", "Common name only; species unclear"],
  NO_STUDY_RETRIEVED: ["neutral", "No study retrieved"],
};
const REASONS = {
  OPEN_WORLD_CHECK_NOT_AVAILABLE: "Regional name-collision check unavailable for this taxon",
  EXACT_ACCEPTED_NAME: "Exact accepted scientific name",
  EXACT_SYNONYM: "Exact scientific synonym",
  EXACT_INPUT_SYNONYM_MATCH: "Exact scientific synonym",
  EXACT_SYNONYM_HOMONYM: "Same wording is another species' accepted name (homonym)",
  NAMED_IN_SOURCE_FIRST_SENTENCE: "The source names this species in the next sentence",
  OPENING_SOURCE_COMMON_NAME: "The opening botanical subject uses a specific common name for this candidate",
  EXACT_VERNACULAR: "Exact common-name match",
  RECORDED_LOCAL_NAME: "Local name in a published list",
  EXACT_HISTORICAL_LEXICON: "Listed for this historical name",
```

```
  GBIF_EXACT_COMMON_NAME: "GBIF lists this common name",
  GBIF_GENUS_MEMBER: "A species of the named genus",
  GBIF_EXACT_SCIENTIFIC_NAME: "Exact GBIF scientific name",
  FUZZY_NAME: "Spelling-tolerant name match",
  ABBREVIATED_GENUS: "Abbreviated genus with matching epithet",
  GENUS_VERNACULAR_ONLY: "Common name used for the whole genus",
  GENUS_CONTEXT_ONLY: "Genus name only",
  OPEN_WORLD_CONGENERS_IN_REGION: "Other species of this genus share the name",
  RANGE_MATCH: "Recorded in the source region",
  RANGE_CONFLICT_REQUIRES_REVIEW: "Not recorded in the source region",
  NO_VERIFIED_RANGE_IN_SNAPSHOT: "No verified range in snapshot",
};
const SENSITIVITY = {
  CONCLUSION_CHANGES_WITH_IDENTITY: ["Evidence availability and its limits depend on which plant this is.", false],
  EVIDENCE_SET_CHANGES_WITH_IDENTITY: ["Different candidates reach different papers.", false],
  STABLE_ACROSS_CANDIDATES: ["Every candidate leads to the same evidence.", true],
  SINGLE_CANDIDATE_IN_SNAPSHOT: ["One candidate in this snapshot.", true],
  OPEN_WORLD_ALTERNATIVES_NOT_RETRIEVED: ["Other species share this name; their evidence was not retrieved.", false],
  NO_CANDIDATE: ["No candidate in this snapshot.", true],
};
const BRANCH_STATE = {
  NO_EVIDENCE_IN_SNAPSHOT: "No evidence in snapshot",
  COMMON_NAME_ONLY_EVIDENCE: "Only papers that use the common name",
  SPECIES_SPECIFIC_BUT_INSUFFICIENT: "Species named, but cannot answer the question",
  SPECIES_SPECIFIC_FINDINGS: "Species-specific findings",
  COMPARABILITY_NOT_ASSESSED: "Saved before screening existed; re-run to screen",
};
const BLOCKING = {
  identity: "species not established",
  population: "population not human or not stated",
  design: "design cannot test efficacy",
  comparator: "comparator missing or not established",
  outcome: "no result for the condition",
  preparation: "preparation differs or is not established",
  intervention: "isolated compound, not the plant",
  NONE: "every automated check passed",
  NONE_UNRESOLVED: "no check failed, some unresolved",
  NOT_ASSESSED_LEGACY_RECORD: "not screened (saved before checks existed)",
};
// No blocking check is not the same as passing: partial or unknown checks stay visible.
function blockingKey(e) {
  const c = e.comparability;
  if (!c) return "NOT_ASSESSED_LEGACY_RECORD";
  if (c.first_blocking_check) return c.first_blocking_check;
  return (c.checks || []).some((x) => x.status !== "PASS") ? "NONE_UNRESOLVED" : "NONE";
}
const CHECK_LABEL = { identity: "Species", population: "Population", design: "Design", comparator: "Comparator", outcome: "Outcome", preparation: "Preparation",
intervention: "Intervention" };
const CASES = [
  ["CAMOMILE.", "Camomile", "compare possible plant identities"],
  ["MINT.", "Mint", "evidence changes with identity"],
  ["SAGE.", "Sage \xb7 1897", "follow leaf tea into a controlled study", {collection:true,sourceStudy:{paperId:'PMC13203699',name:'Salvia officinalis'}}],
  ["FEVERFEW OR FEATHERFEW.", "Feverfew", "follow its recorded preparations"],
  ["THYME.", "Thyme", "see why the species matters"],
  ["FEVERFEW.", "Feverfew \xb7 1897", "untangle changes of plant in the source"],
];

function el(tag, text, cls) {
```

```
  const e = document.createElement(tag);
  if (text !== undefined && text !== null) e.textContent = String(text);
  if (cls) e.className = cls;
  return e;
}
function link(text, url) {
  const a = el("a", text);
  try {
    const u = new URL(url, location.origin);
    if (["https:", "http:"].includes(u.protocol)) {
      a.href = u.href;
      a.target = "_blank";
      a.rel = "noopener noreferrer";
    }
  } catch {}
  return a;
}
function pill(text, tone = "neutral") {
  return el("span", text, "pill " + tone);
}
function human(value) {
  return String(value ?? "").replaceAll("_", " ").toLowerCase();
}
function currentLiterature(name) {
  return link('Find current papers \u2197', 'https://pubmed.ncbi.nlm.nih.gov/?term='+encodeURIComponent('"'+name+'"'));
}
function option(value, label) {
  const o = el("option", label);
  o.value = value;
  return o;
}
function status(text, error = false) {
  $("#status").textContent = text;
  $("#status").className = error ? "error" : "";
  if (error) $("#status").hidden = false;
}
async function api(path, options) {
  const r = await fetch(path, options);
  if (!r.ok) {
    const e = await r.json().catch(() => ({ detail: "Request failed" }));
    throw new Error(typeof e.detail === "string" ? e.detail : JSON.stringify(e.detail));
  }
  return r.json();
}
let benchmarkRequest;
function benchmarks() {
  if (!benchmarkRequest) benchmarkRequest = api('/api/benchmarks').catch(error => { benchmarkRequest = null; throw error; });
  return benchmarkRequest;
}
// Europe PMC titles keep inline markup such as <i>Genus species</i>; strip it for display only.
function cleanTitle(t) {
  return publicationText(t);
}
function plural(n, word) {
  return `${n} ${word}${n === 1 ? "" : "s"}`;
}
function relationTone(r) {
  return r === "SUPPORTS" ? "ok" : r === "CONTRADICTS" || r === "MIXED" ? "bad" : "neutral";
```

```
}
function group() {
  return data?.identity_counterfactuals.find((g) => g.key === selected);
}
function taxonName(id) {
  for (const g of data.identity_counterfactuals) for (const b of g.branches) if (b.taxon_id === id) return b.name;
  return id;
}

/* ---------- Views and tabs ---------- */
let researchReady=false,researchLoading=null;
async function ensureResearch(){
  if(researchReady)return;
  if(!researchLoading)researchLoading=(async()=>{
    passages=await api('/api/passages');
    for(const id of ['compare-a','compare-b'])$('#'+id).replaceChildren(...passages.map(p=>option(p.id,`${p.collection||'Culpeper 1850'} \xb7 ${p.title}`)));
    $('#compare-a').value=passages.find(p=>p.title==='CAMOMILE.'&&!p.collection)?.id||passages[0]?.id;
    $('#compare-b').value=passages.find(p=>p.collection&&/CHAMOM|CAMOM/.test(p.title))?.id||passages[1]?.id;
    $('#passage').replaceChildren();
    for(const label of [...new Set(passages.map(p=>p.collection||'Culpeper 1850'))]){const
group=el('optgroup');group.label=label;group.append(...passages.filter(p=>(p.collection||'Culpeper
1850')===label).map(p=>option(p.id,p.title)));$('#passage').append(group);}
    renderCases();renderResearchTrails();researchReady=true;
  })().catch(error=>{researchLoading=null;throw error;});
  return researchLoading;
}
function showView(name,questionInitial=null) {
  if(!['consultation','question'].includes(name)&&!researchReady)return ensureResearch().then(()=>showView(name,questionInitial)).catch(error=>status('Could not load
 the research library: '+error.message,true));
  rememberReading();
  for (const b of document.querySelectorAll(".view-tab")) {
    if (b.dataset.view === name) b.setAttribute("aria-current", "page");
    else b.removeAttribute("aria-current");
  }
  for (const v of ["workspace", "map", "graph", "evaluation", "chemistry", "library", "handoffs", "consultation", "question"]) $("#view-" + v).hidden = v !== name;
  history.replaceState(null,'','#'+name);
  const tools=$('.research-tools');if(tools){tools.hidden=!location.pathname.startsWith('/research');tools.open=!['consultation','question'].includes(name);}
  if(health)$('#literature-availability').hidden=name==='consultation'||health.abstracts_available>=health.papers;
  $(".source-bar").hidden = name !== "workspace";
  $("#status").hidden = name !== "workspace";
  if(!restoringReading&&data&&['workspace','graph'].includes(name))readingResumeActive=true;
  if (name === "graph" && data) renderGraph();
  if (name === "chemistry") return loadChemistry();
  if (name === "map") return loadEvidenceMap();
  if (name === "library") loadLibrary();
  if (name === 'handoffs') return handoffs.open();
  if (name === 'consultation') return consultation.open();
  if (name === 'question') return questionReview.open(questionInitial);
  if (name === 'evaluation') return loadMetrics();
}

let librarySnapshot, libraryLimit = 6, rankedLibrary = null, librarySearchText = '', libraryFocus = null, librarySearchSerial = 0, libraryScrollY = 0;
let librarySaved = new Set(), librarySavedOnly = false, libraryVisibleRows = [], libraryNotes = {};
let libraryBeforeSaved = null;
let sourceFocusOrigin = 'library';
let sourceEvidenceFocus = null;
let libraryReadingEntries = new Map(), libraryReadingMigrations = new Set();
function savedReading(){
```

```
  if(recorded)return null;
  try{const saved=JSON.parse(localStorage.getItem(RESUME_READING_KEY)||'null');return saved?.version===1&&/^[a-f0-9]{24}$/.test(saved.runId)?saved:null;}catch{return null;}
}
function updateResumeControl(){const control=$('#resume-reading');if(control)control.hidden=recorded||!savedReading();}
function rememberReading(){
  if(recorded||restoringReading||!readingResumeActive||!data?.run_id||($('#view-workspace').hidden&&$('#view-graph').hidden))return;
  const focused=libraryFocus?.source_id===data.source.id?libraryFocus.statement:null;
  const previous=savedReading(),activeView=$('#view-workspace').hidden?$('#view-graph'):$('#view-workspace'),bounds=activeView.getBoundingClientRect();
  const filters=Object.fromEntries(['branch','study-type','relation','year'].map(id=>[id,$('#'+id).value]));
  const tab=$('[role="tab"][aria-selected="true"]')?.id||'tab-identity';
  const keepPosition=!(bounds.top<window.innerHeight-80&&bounds.bottom>80)&&previous?.runId===data.run_id&&previous.name===selected&&previous.tab===tab&&previous.width===window.innerWidth&&JSON.stringify(previous.filters)===JSON.stringify(filters);
  const saved={version:1,runId:data.run_id,name:selected,tab,filters,
    identityPair:identityComparisons.get(data.run_id+'::'+selected)||null,
    statementTarget:statementReadingTargets.get(data.run_id+'::'+selected)??'',
    studyComparison:studyComparisons.get(studyComparisonScope())||[],
    scrollY:keepPosition?previous.scrollY:window.scrollY,width:window.innerWidth,sourceScroll:$('#original').scrollTop,
    sourceFocus:focused?{start:focused.start,end:focused.end}:null,
    evidenceFocus:sourceFocusOrigin==='evidence'?sourceEvidenceFocus:null,
    papers:[...$('#evidence').querySelectorAll('.evidence-item')].map(card=>({paper:card.dataset.paper,taxon:card.dataset.taxon,condition:card.dataset.condition,hidden:card.hidden,source:!!card.querySelector(':scope > .study-source-comparison')?.open,context:!!card.querySelector(':scope > .study-context')?.open,topics:!!card.querySelector(':scope > .linked-conditions')?.open})),
    trialReadings:[...$('#evidence').querySelectorAll('.trial-readings')].map(panel=>({id:panel.dataset.readingSet,open:panel.open,contexts:[...panel.querySelectorAll('.trial-reading-context[open]')].map(context=>context.closest('.trial-reading-card').dataset.pmid)}))};
  try{localStorage.setItem(RESUME_READING_KEY,JSON.stringify(saved));updateResumeControl();}catch{}
}
async function resumeReading(saved,{navigate=true,position=true}={}){
  invalidatePendingAnalysis();
  const serial=analysisSerial;
  status('Reopening your saved analysis\u2026');
  const result=await api('/api/analyses/'+saved.runId);
  if(serial!==analysisSerial)return null;
  restoringReading=true;
  try{
    data=result;data.researcher_reviews||=[];requestedSourceName=data.configuration.requested_name||null;
    const passage=passages.find(p=>p.id===data.source.id);custom=!passage;
    $('#passage').value=passage?.id||'';$('#input').value=data.input;
    if(custom)$('#custom-input').open=true;
    if([...$('#analysis-ranking').options].some(o=>o.value===data.configuration.retrieval))$('#analysis-ranking').value=data.configuration.retrieval;
    libraryFocus=null;sourceEvidenceFocus=null;sourceFocusOrigin='resume';
    const chars=Array.from(data.input),focus=saved.sourceFocus;
    if(focus&&Number.isInteger(focus.start)&&Number.isInteger(focus.end)&&focus.start>=0&&focus.end>focus.start&&focus.end<=chars.length){
      libraryFocus={source_id:data.source.id,statement:{start:focus.start,end:focus.end,text:chars.slice(focus.start,focus.end).join('')}};
      const target=saved.evidenceFocus;
      if(target&&data.evidence.some(e=>e.paper.id===target.paperId&&e.taxon_id===target.taxonId)&&data.identity_counterfactuals.some(g=>g.key===target.nameKey)){sourceEvidenceFocus=target;sourceFocusOrigin='evidence';}
    }
    const resumeName=data.identity_counterfactuals.find(g=>g.key===saved.name);
    if(resumeName&&(saved.statementTarget===''||data.historical_claims.some(claim=>claim.id===saved.statementTarget&&resumeName.claim_ids.includes(claim.id))))statementReadingTargets.set(identityPairKey(resumeName),saved.statementTarget);
    if(resumeName&&Array.isArray(saved.identityPair)&&saved.identityPair.length===2&&saved.identityPair[0]!==saved.identityPair[1]&&saved.identityPair.every(id=>resumeName.branches.some(branch=>branch.taxon_id===id)))identityComparisons.set(identityPairKey(resumeName),saved.identityPair);
    render();if(resumeName)selectName(saved.name);
    if(resumeName&&Array.isArray(saved.studyComparison)){
      const pool=studyComparisonPool();studyComparisons.set(studyComparisonScope(),[...new Set(saved.studyComparison)].filter(key=>pool.has(key)).slice(0,3));
```

```
    }
    for(const [id,value] of Object.entries(saved.filters||{}))if(['branch','study-type','relation','year'].includes(id)){
      const control=$('#'+id);if(control.tagName!=='SELECT'||[...control.options].some(o=>o.value===value))control.value=value;
    }
    renderEvidence();
    for(const item of Array.isArray(saved.papers)?saved.papers:[]){
      const card=[...$('#evidence').querySelectorAll('.evidence-item')].find(card=>card.dataset.paper===item.paper&&card.dataset.taxon===item.taxon&&card.dataset.condition===item.condition);
      if(!card)continue;card.hidden=item.hidden===true;
      for(const [selector,key] of [['.study-source-comparison','source'],['.study-context','context'],['.linked-conditions','topics']]){const panel=card.querySelector(':scope > '+selector);if(panel)panel.open=item[key]===true;}
    }
    for(const section of $('#evidence').querySelectorAll('.branch-group')){
      const more=section.querySelector(':scope > .more'),remaining=section.querySelectorAll(':scope > .evidence-item[hidden]').length;
      if(more){if(remaining)more.textContent='Read '+remaining+' more publications';else more.remove();}
    }
    for(const item of Array.isArray(saved.trialReadings)?saved.trialReadings:[]){
      const panel=[...$('#evidence').querySelectorAll('.trial-readings')].find(panel=>panel.dataset.readingSet===item.id);if(!panel)continue;panel.open=item.open===true;
      for(const context of panel.querySelectorAll('.trial-reading-context'))context.open=Array.isArray(item.contexts)&&item.contexts.includes(context.closest('.trial-reading-card').dataset.pmid);
    }
    showTab(['tab-identity','tab-evidence','tab-review'].includes(saved.tab)?saved.tab:'tab-identity');
    if(navigate)showView('workspace');
    readingResumeActive=true;updateResumeControl();
    status('Resumed your saved analysis and reading context. Analyze again when you want results from the current data.');
    if(position)requestAnimationFrame(()=>requestAnimationFrame(()=>{if($('#view-workspace').hidden&&$('#view-graph').hidden)return;if(Number.isFinite(saved.sourceScroll))$('#original').scrollTop=Math.max(0,saved.sourceScroll);if(saved.width===window.innerWidth&&Number.isFinite(saved.scrollY))window.scrollTo({top:Math.max(0,saved.scrollY),behavior:'auto'});else focusNameInspector();}));
    return true;
  }finally{restoringReading=false;}
}
window.addEventListener('pagehide',rememberReading);
let rememberReadingTimer;
function scheduleRememberReading(){clearTimeout(rememberReadingTimer);rememberReadingTimer=setTimeout(rememberReading,250);}
window.addEventListener('scroll',scheduleRememberReading,{passive:true});
document.addEventListener('toggle',scheduleRememberReading,true);
$('#resume-reading').addEventListener('click',async()=>{const saved=savedReading();if(!saved)return;const control=$('#resume-reading');control.disabled=true;try{await resumeReading(saved);}catch(error){status('Could not reopen that saved reading: '+error.message,true);}finally{control.disabled=false;}});
const READING_STORE = 'foliatrace.reading.v2';
const libraryComparison = new Map();
const libraryExpanded = new Set();
const ROLE_LABEL = {THERAPEUTIC_USE_RECORDED:'Recorded use', ADVERSE_EFFECT_RECORDED:'Possible harmful effect', CONTEXT_ONLY_OR_UNCLEAR:'Context or unclear'};
function statementRoleLabel(record){
  const role=record.statement_role,scope=record.scope??record.relation_scope;
  if(scope==='PARAGRAPH_CONTEXT_UNCERTAIN_RELATION'&&['THERAPEUTIC_USE_RECORDED','ADVERSE_EFFECT_RECORDED'].includes(role))return 'Statement \xb7 subject unresolved';
  if(scope==='MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION'&&['THERAPEUTIC_USE_RECORDED','ADVERSE_EFFECT_RECORDED'].includes(role))return 'Statement \xb7 plant attribution unresolved';
  return ROLE_LABEL[role]||'Statement';
}
function sourceLabel(value) { return value.replace(/\[\d+\]\s*/g,'').replace(/\.$/,'').trim(); }
function readingKey(row){return JSON.stringify([row.source_id,row.collection,row.date,row.url,row.statement.start,row.statement.end,row.statement.text]);}
function persistLibraryReading(record){
  for(const row of record?[record]:librarySnapshot.records){
    const key=readingKey(row),saved=librarySaved.has(row.id),note=libraryNotes[row.id]||'';
    if(saved||note)libraryReadingEntries.set(key,{key,saved,note});else libraryReadingEntries.delete(key);
```

```
  }
  localStorage.setItem(READING_STORE,JSON.stringify({version:2,entries:[...libraryReadingEntries.values()],migrated_snapshots:[...libraryReadingMigrations]}));
}
function restoreLibraryReading(){
  try{
    const stored=JSON.parse(localStorage.getItem(READING_STORE)||'{}');
    if(stored.version===2&&Array.isArray(stored.entries)){
      libraryReadingEntries=new Map(stored.entries.filter(e=>typeof e.key==='string'&&typeof
e.note==='string').map(e=>[e.key,{key:e.key,saved:e.saved===true,note:e.note}]));
      libraryReadingMigrations=new Set((stored.migrated_snapshots||[]).filter(s=>typeof s==='string'));
    }
  }catch{}
  const rows=new Map(librarySnapshot.records.map(row=>[row.id,row]));
  for(const migration of [{snapshot_sha256:librarySnapshot.snapshot_sha256,id_map:null},...(librarySnapshot.reading_migrations||[])]){
    if(libraryReadingMigrations.has(migration.snapshot_sha256))continue;
    try{
      const saved=JSON.parse(localStorage.getItem('foliatrace.saved.'+migration.snapshot_sha256)||'[]');
      const notes=JSON.parse(localStorage.getItem('foliatrace.notes.'+migration.snapshot_sha256)||'{}');
      const savedIds=new Set(Array.isArray(saved)?saved:[]),noteMap=notes&&typeof notes==='object'&&!Array.isArray(notes)?notes:{};
      for(const oldId of new Set([...savedIds,...Object.keys(noteMap)])){
        const row=rows.get(migration.id_map?migration.id_map[oldId]:oldId);if(!row)continue;
        const key=readingKey(row);if(libraryReadingEntries.has(key))continue;
        libraryReadingEntries.set(key,{key,saved:savedIds.has(oldId),note:typeof noteMap[oldId]==='string'?noteMap[oldId]:''});
      }
      libraryReadingMigrations.add(migration.snapshot_sha256);
    }catch{}
  }
  librarySaved=new Set();libraryNotes={};
  for(const row of librarySnapshot.records){const
entry=libraryReadingEntries.get(readingKey(row));if(entry?.saved)librarySaved.add(row.id);if(entry?.note)libraryNotes[row.id]=entry.note;}
}
function storeLibrarySaved() {
  try{persistLibraryReading();}catch{$('#knowledge-search-status').textContent='Browser storage is unavailable. Export Reading notes to keep this reading list.';}
  updateSavedControl();
}
function updateSavedControl(){
  $('#library-saved').textContent=librarySavedOnly?'\u2190 Back to results':'Saved statements'+(librarySaved.size?' ('+librarySaved.size+')':'');
  $('#library-saved').setAttribute('aria-pressed',String(librarySavedOnly));
}
async function openLibraryRecord(row, origin = 'library') {
  sourceFocusOrigin = origin;
  libraryFocus = row;
  if (origin === 'library') libraryScrollY = window.scrollY;
  choosePassage(row.source_id); showView('workspace'); await analyze();
  if(data?.source.id!==row.source_id)return;
  const claim = data?.source.id === row.source_id && data.historical_claims.find(c=>c.support.start === row.statement.start);
  const target = claim && data.identity_counterfactuals.find(g=>g.claim_ids.includes(claim.id));
  if(target){statementReadingTargets.set(identityPairKey(target),claim.id);selectName(target.key);}
  $('#source-focus')?.scrollIntoView({behavior:'smooth',block:'nearest'});
}
function saveStatementButton(row) {
  const button = el('button', librarySaved.has(row.id) ? 'Saved \u2713' : 'Save statement', 'quiet');
  button.setAttribute('aria-pressed', String(librarySaved.has(row.id)));
  button.addEventListener('click', () => {
    librarySaved.has(row.id) ? librarySaved.delete(row.id) : librarySaved.add(row.id);
    storeLibrarySaved();
    button.textContent = librarySaved.has(row.id) ? 'Saved \u2713' : 'Save statement';
    button.setAttribute('aria-pressed', String(librarySaved.has(row.id)));
```

```
    if (librarySavedOnly) loadLibrary();
  });
  return button;
}
function libraryAttribution(row) {
  const block=el('div',undefined,'statement-attribution');
  const label=row.scope==='PARAGRAPH_CONTEXT_UNCERTAIN_RELATION'?'Chapter index (subject unresolved): ':row.scope==='SOURCE_SECTION_HEADING'?'Chapter plant: ':'Plant wording: ';
  block.append(el('p',label+[...new Set(row.names.map(n=>n.text))].join(', '),'hint'));
  const note=statementQualifier({relation_scope:row.scope,negated:row.negated});
  if(note)block.append(el('p',note,'hint'));
  return block;
}
function readingNoteEditor(row) {
  const label=el('label','Your research note','reading-note'),input=el('textarea');
  input.rows=3;input.maxLength=4000;input.value=libraryNotes[row.id]||'';
  input.placeholder='Your interpretation, a question, or a citation to follow up\u2026';
  const state=el('span','Stored in this browser; included in Reading notes.','hint');state.setAttribute('aria-live','polite');
  input.addEventListener('input',()=>{
    const value=input.value.trim();if(value)libraryNotes[row.id]=value;else delete libraryNotes[row.id];
    try{persistLibraryReading(row);state.textContent='Saved in this browser; included in Reading notes.';}
    catch{state.textContent='Browser storage is unavailable. Export Reading notes or a backup to keep this note.';}
  });
  label.append(input,state);return label;
}
function readingBackup() {
  // Include every saved statement and note, independently of the current filters.
  try{persistLibraryReading();}catch{} // In-memory notes can still be downloaded when storage is full.
  return {schema:'foliatrace.reading-backup',version:1,exported_at:new Date().toISOString(),entries:[...libraryReadingEntries.values()]};
}
function parseReadingBackup(value) {
  if(value?.schema!=='foliatrace.reading-backup'||value.version!==1||!Array.isArray(value.entries)||value.entries.length>20000)throw new Error('Choose a FoliaTrace reading backup JSON file.');
  const entries=new Map();
  for(const entry of value.entries){
    if(!entry||typeof entry.key!=='string'||typeof entry.saved!=='boolean'||typeof entry.note!=='string'||entry.note.length>4000)throw new Error('The backup contains an invalid reading entry. No notes were imported.');
    let anchor;try{anchor=JSON.parse(entry.key);}catch{throw new Error('The backup contains an invalid source anchor. No notes were imported.');}
    if(!Array.isArray(anchor)||anchor.length!==7||typeof anchor[0]!=='string'||typeof anchor[6]!=='string'||!Number.isInteger(anchor[4])||!Number.isInteger(anchor[5])||anchor[4]<0||anchor[5]<=anchor[4]||anchor[5]-anchor[4]!==Array.from(anchor[6]).length||anchor.slice(1,4).some(field=>field!==null&&!['string','number'].includes(typeof field)))throw new Error('The backup contains an invalid source anchor. No notes were imported.');
    const key=JSON.stringify(anchor),previous=entries.get(key);
    if(previous&&(previous.note!==entry.note||previous.saved!==entry.saved))throw new Error('The backup contains conflicting copies of one entry. No notes were imported.');
    entries.set(key,{key,saved:entry.saved,note:entry.note});
  }
  return [...entries.values()];
}
function restoreReadingBackup(value) {
  const incoming=parseReadingBackup(value),merged=new Map(libraryReadingEntries);
  let conflicts=0;
  for(const entry of incoming){
    const previous=merged.get(entry.key),conflict=!!(previous?.note&&entry.note&&previous.note!==entry.note);
    if(conflict)conflicts++;
    merged.set(entry.key,{...entry,saved:entry.saved||previous?.saved===true,note:previous?.note||entry.note});
  }
  // Commit only after the whole file is validated and storage succeeds.
```

```
  localStorage.setItem(READING_STORE,JSON.stringify({version:2,entries:[...merged.values()],migrated_snapshots:[...libraryReadingMigrations]}));
  libraryReadingEntries=merged;
  restoreLibraryReading();
  const active=new Set(librarySnapshot.records.map(readingKey)),matched=incoming.filter(entry=>active.has(entry.key)).length;
  return {entries:incoming.length,matched,unavailable:incoming.length-matched,conflicts};
}
function openSourceStudyReading(sourceStudy){
  const branch=data.identity_counterfactuals.flatMap(name=>name.branches).find(branch=>branch.name===sourceStudy.name);
  const study=branch&&data.evidence.find(e=>e.taxon_id===branch.taxon_id&&e.paper.id===sourceStudy.paperId&&e.historical_contexts?.some(context=>context.route.some(r
oute=>/gargl/i.test(route.text))));
  const name=study&&data.identity_counterfactuals.find(name=>name.branches.some(branch=>branch.evidence_ids.includes(study.id)));
  if(!study||!name)return false;
  const context=study.historical_contexts.find(context=>context.route.some(route=>/gargl/i.test(route.text)));
  statementReadingTargets.set(identityPairKey(name),context.claim_id);selectName(name.key);openBranchPublication(study.taxon_id,study.paper.id);rememberReading();ret
urn true;
}
function renderResearchTrails() {
  const box=$('#research-trails');box.replaceChildren();
  const trails=[
    {title:'One name. Which plant?',source:'CAMOMILE.',collection:false,copy:'Choose two possible identities and read the changed publications. Then compare a
historical swelling / joint-pain statement with a modern mouthwash study: the shared topic does not make the indications equivalent.',tab:'tab-
identity',action:'Explore Camomile identities',indicationStudy:{paperId:'PMC13203699',taxonId:'gbif:8370958'}},
    {title:'Does the study answer the source?',source:'MINT.',collection:false,copy:'Read a Mint statement beside a combined-oil trial, or compare three placebo-
controlled peppermint-oil trials. Inspect how formulation and primary endpoints change the question.',tab:'tab-evidence',action:'Read Mint
trial',readingSet:'reading-set:peppermint-ibs-trials'},
    {title:'What carries forward into a modern study?',source:'SAGE.',collection:true,copy:'Fernie records Sage leaf tea with honey as a gargle. Read the Sage arm of
 a modern controlled mouthwash trial: inspect its aqueous preparation, measured outcomes and reported findings, alongside what the old statement leaves
unspecified.',tab:'tab-evidence',action:'Follow Sage tea into the study',sourceStudy:{paperId:'PMC13203699',name:'Salvia officinalis'}},
    {title:'What does a molecule assay show?',view:'chemistry',copy:'Follow parthenolide\u2019s reported assays and publication mentions. See the molecular target
and original document, with the limits of what an assay can establish.',action:'Inspect parthenolide assays'},
  ];
  for(const trail of trails){
    const passage=passages.find(p=>p.title===trail.source&&Boolean(p.collection)===trail.collection);if(!passage&&!trail.view)continue;
    const card=el('article',undefined,'research-trail');card.append(el('h4',trail.title),el('p',trail.copy));
    const go=el('button',trail.action+' \u2192','text-action');
    const openTrail=async(mode='study')=>{
      if(trail.view==='chemistry'){await showView('chemistry');const
compound=chemistrySnapshot?.mappings.find(c=>/parthenolide/i.test(c.name));if(compound){$('#chemical-compound').value=String(compound.cid);await
loadChemistry();}$('#view-chemistry').scrollIntoView({behavior:'smooth',block:'start'});return;}
      libraryFocus=null;choosePassage(passage.id);showView('workspace');await analyze();
      if(data?.source.id===passage.id){
        if(trail.sourceStudy){
          if(openSourceStudyReading(trail.sourceStudy))return;
          status('No active source-statement link to this study is available in this analysis. Inspect the current source records instead.',true);
        }
        if(mode==='indication'&&trail.indicationStudy){
          const study=data.evidence.find(e=>e.paper.id===trail.indicationStudy.paperId&&e.taxon_id===trail.indicationStudy.taxonId);
          const name=study&&data.identity_counterfactuals.find(g=>g.branches.some(b=>b.evidence_ids.includes(study.id)));
          if(study&&name){const statement=study.historical_contexts?.find(context=>context.preparation?.length)||study.historical_contexts?.[0];statementReadingTarge
ts.set(identityPairKey(name),statement?.claim_id||'');selectName(name.key);openBranchPublication(study.taxon_id,study.paper.id);return;}
          status('This source analysis has no active matching study; inspect the available publications instead.',true);
        }
        if(mode==='trials'&&trail.readingSet){openTrialReadingSet(trail.readingSet);return;}
        if(trail.tab==='tab-evidence'){
          const study=data.evidence.find(e=>e.paper.methods_contexts?.length);
          const name=study&&data.identity_counterfactuals.find(g=>g.branches.some(b=>b.evidence_ids.includes(study.id)));
          if(study&&name){selectName(name.key);openBranchPublication(study.taxon_id,study.paper.id);return;}
        }
```

```
        showTab(trail.tab,true);$('#'+$('#'+trail.tab).getAttribute('aria-controls')).scrollIntoView({behavior:'smooth',block:'start'});
      }
    };
    go.addEventListener('click',()=>openTrail());card.append(go);
    if(trail.indicationStudy){const indication=el('button','Compare historical and study indications \u2192','text-
action');indication.addEventListener('click',()=>openTrail('indication'));card.append(indication);}
    if(trail.readingSet){const compare=el('button','Compare controlled Mint trials \u2192','text-
action');compare.addEventListener('click',()=>openTrail('trials'));card.append(compare);}
    box.append(card);
  }
}
function cancelLibrarySearch() {
  librarySearchSerial++;
  $('#knowledge-search').disabled = false;
  $('#knowledge-search').textContent = 'Search knowledge';
  $('#knowledge-search-status').textContent = '';
}
function applyLibraryFilters() {
  libraryLimit = 6;
  if ($('#library-query').value.trim()) searchLibrary();
  else { rankedLibrary = null; librarySearchText = ''; loadLibrary(); }
}
function renderLibraryComparison() {
  const tray=$('#library-comparison'); tray.hidden=!libraryComparison.size; tray.replaceChildren();
  if(!libraryComparison.size)return;
  const text=el('div'); text.append(el('strong',libraryComparison.size===2?'Ready to compare':'Choose one more
chapter'),el('p',[...libraryComparison.values()].map(r=>sourceLabel(r.source_title)).join(' + '),'hint'));
  const go=el('button','Compare sources'); go.disabled=libraryComparison.size!==2;
  go.addEventListener('click',()=>{const
ids=[...libraryComparison.keys()];$('#compare-a').value=ids[0];$('#compare-b').value=ids[1];showView('workspace');$('#comparison-box').open=true;$('#compare-
run').click();$('#comparison-box').scrollIntoView({behavior:'smooth',block:'start'});});
  const clear=el('button','Clear','quiet');clear.addEventListener('click',()=>{libraryComparison.clear();loadLibrary();});tray.append(text,go,clear);
}
async function loadLibrary() {
  const box=$('#library-records');
  try {
    if(!librarySnapshot){
      librarySnapshot=await api('/api/research/corpus');
      $('#library-collection').append(...Object.keys(librarySnapshot.collections).map(s=>option(s,s)));
      $('#library-condition').append(...Object.keys(librarySnapshot.conditions).sort().map(s=>option(s,human(s.replace('condition:','')))));
      restoreLibraryReading();
      $('#library-summary').replaceChildren(...[[Object.values(librarySnapshot.collections).reduce((a,b)=>a+b,0),'historical
chapters'],[Object.keys(librarySnapshot.collections).length,'books to explore'],[librarySnapshot.records.length,'source statements']].map(([value,label])=>{const
s=el('div',undefined,'library-stat');s.append(el('strong',String(value)),el('span',label));return s;}));
      storeLibrarySaved();
    }
    const collection=$('#library-collection').value,condition=$('#library-condition').value,role=$('#library-role').value,query=librarySearchText.toLowerCase();
    const rows=(rankedLibrary||librarySnapshot.records).filter(r=>(!librarySavedOnly||librarySaved.has(r.id))&&(!role||r.statement_role===role)&&(!collection||r.coll
ection===collection)&&(!condition||r.conditions.some(c=>c.id===condition))&&(rankedLibrary||!query||[r.statement.text,r.source_title,...r.names.map(n=>n.text)].join(
' ').toLowerCase().includes(query)));
    libraryVisibleRows=rows;
    const groups=new Map();for(const r of rows){if(!groups.has(r.source_id))groups.set(r.source_id,[]);groups.get(r.source_id).push(r);}
    let chapters=[...groups.values()];
    if(!rankedLibrary&&!collection){const books=[...new
Set(chapters.map(g=>g[0].collection))].map(book=>chapters.filter(g=>g[0].collection===book));chapters=[];for(let i=0;books.some(b=>i<b.length);i++)for(const book of
books)if(book[i])chapters.push(book[i]);}
    $('#library-count').textContent=plural(rows.length,'statement')+' in '+plural(chapters.length,'chapter')+(librarySavedOnly?' \xb7 your reading list':'');
    box.replaceChildren();
```

```
    if(!chapters.length){const empty=el('div',undefined,'library-empty');empty.append(el('h3',librarySavedOnly?'Your reading list starts here.':'No statements match these filters.'),el('p',librarySavedOnly?'Save a statement while you browse to collect it here.':'Try another topic, include both books, or search by meaning.'));const reset=el('button','Browse recorded uses','quiet');reset.addEventListener('click',resetLibrary);empty.append(reset);box.append(empty);}
    for(const group of chapters.slice(0,libraryLimit)){
      const r=group[0],card=el('article',undefined,'knowledge-card'),head=el('div',undefined,'knowledge-card-head');
      const title=el('div');title.append(el('p',r.collection,'kicker historical'),el('h3',sourceLabel(r.source_title)));
      head.append(title,pill(statementRoleLabel(r),r.statement_role==='ADVERSE_EFFECT_RECORDED'?'bad':'neutral'));card.append(head);
      const topics=el('div',undefined,'knowledge-topics');for(const term of [...new Map(group.flatMap(row=>row.conditions).map(c=>[c.id,c])).values()].slice(0,4)){const topic=el('button',term.label,'topic-chip');topic.addEventListener('click',()=>{$('#library-condition').value=term.id;applyLibraryFilters();});topics.append(topic);}card.append(topics);
      const quote=el('blockquote',r.statement.text);card.append(quote);
      if(r.statement.text.length>240){const full=el('button','Show full statement','text-action');full.setAttribute('aria-expanded','false');full.addEventListener('click',()=>{const on=quote.classList.toggle('full-quote');full.textContent=on?'Show shorter preview':'Show full statement';full.setAttribute('aria-expanded',String(on));});card.append(full);}
      card.append(libraryAttribution(r));
      const preparations=[...new Set(group.flatMap(row=>row.preparations.map(p=>p.text.toLowerCase())))].slice(0,4);
      if(preparations.length)card.append(el('p',(group.length>1?'Preparations across matching statements: ':'Preparation wording: ')+preparations.join(' \xb7 '),'hint'));
      if(group.length>1){const more=el('details',undefined,'knowledge-more');more.open=libraryExpanded.has(r.source_id);more.addEventListener('toggle',()=>{more.open?libraryExpanded.add(r.source_id):libraryExpanded.delete(r.source_id);});more.append(el('summary',(group.length-1)+' more matching statement'+(group.length>2?'s':'')));for(const row of group.slice(1)){const item=el('div',undefined,'knowledge-extra');item.append(pill(statementRoleLabel(row),row.statement_role==='ADVERSE_EFFECT_RECORDED'?'bad':'neutral'),el('blockquote',row.statement.text),libraryAttribution(row));const actions=el('div',undefined,'knowledge-actions');const open=el('button','Read in context','text-action');open.addEventListener('click',()=>openLibraryRecord(row));if(passages.some(p=>p.id===row.source_id))actions.append(open);actions.append(saveStatementButton(row));item.append(actions);if(librarySavedOnly||libraryNotes[row.id])item.append(readingNoteEditor(row));more.append(item);}card.append(more);}
      const actions=el('div',undefined,'knowledge-actions');
      if(passages.some(p=>p.id===r.source_id)){const open=el('button','Read in workspace \u2192');open.addEventListener('click',()=>openLibraryRecord(r));actions.append(open);}else actions.append(link('Read original source \u2197',r.url));
      actions.append(saveStatementButton(r));card.append(actions);
      if(librarySavedOnly||libraryNotes[r.id])card.append(readingNoteEditor(r));
      const foot=el('div',undefined,'knowledge-card-foot');foot.append(link('Original book \u2197',r.url));
      if(passages.some(p=>p.id===r.source_id)){const compare=el('label',undefined,'compare-choice'),check=el('input');check.type='checkbox';check.checked=libraryComparison.has(r.source_id);check.disabled=libraryComparison.size>=2&&!check.checked;check.addEventListener('change',()=>{check.checked?libraryComparison.set(r.source_id,r):libraryComparison.delete(r.source_id);loadLibrary();});compare.append(check,document.createTextNode('Compare chapter'));foot.append(compare);}card.append(foot);box.append(card);
    }
    $('#library-more').hidden=chapters.length<=libraryLimit;renderLibraryComparison();
  }catch(error){box.replaceChildren(el('p',error.message,'error'));}
}
function resetLibrary(){cancelLibrarySearch();librarySavedOnly=false;libraryBeforeSaved=null;updateSavedControl();librarySearchText='';rankedLibrary=null;libraryLimit=6;$('#library-query').value='';$('#library-collection').value='';$('#library-condition').value='';$('#library-role').value='THERAPEUTIC_USE_RECORDED';loadLibrary();}
for(const id of ['library-collection','library-condition','library-role'])$('#'+id).addEventListener('change',applyLibraryFilters);
$('#knowledge-method').addEventListener('change',()=>{if($('#library-query').value.trim())searchLibrary();});
$('#library-query').addEventListener('keydown',event=>{if(event.key==='Enter'){event.preventDefault();$('#knowledge-search').click();}});
$('#library-query').addEventListener('input',()=>{cancelLibrarySearch();if(!$('#library-query').value.trim()){librarySearchText='';rankedLibrary=null;libraryLimit=6;loadLibrary();}else $('#knowledge-search-status').textContent='Press Enter to search the source statements.';});
$('#library-more').addEventListener('click',()=>{libraryLimit+=6;loadLibrary();});
$('#library-saved').addEventListener('click',async()=>{
  let returnScroll=null;
  if(!librarySavedOnly){
    libraryBeforeSaved={query:$('#library-query').value,text:librarySearchText,ranked:rankedLibrary,limit:libraryLimit,scroll:window.scrollY,status:$('#knowledge-search-status').textContent,fields:Object.fromEntries(['library-role','library-collection','library-condition','knowledge-method'].map(id=>[id,$('#'+id).value]))};
    cancelLibrarySearch();librarySavedOnly=true;rankedLibrary=null;librarySearchText='';$('#library-query').value='';libraryLimit=6;
    for(const id of ['library-role','library-collection','library-condition'])$('#'+id).value='';
    $('#knowledge-search-status').textContent='Your saved statements. Filters apply to this reading list.';
  }else{
```

```
    cancelLibrarySearch();librarySavedOnly=false;
    if(libraryBeforeSaved){
      const previous=libraryBeforeSaved;$('#library-query').value=previous.query;librarySearchText=previous.text;rankedLibrary=previous.ranked;libraryLimit=previous.limit;
      for(const [id,value] of Object.entries(previous.fields))$('#'+id).value=value;
      $('#knowledge-search-status').textContent=previous.status;returnScroll=previous.scroll;
    }
    libraryBeforeSaved=null;
  }
  updateSavedControl();await loadLibrary();
  requestAnimationFrame(()=>{if(returnScroll!==null)window.scrollTo({top:returnScroll,behavior:'auto'});else $('.library-results-bar').scrollIntoView({block:'start',behavior:'smooth'});});
});
for(const [label,query] of [['Sleep','What was recorded for sleeplessness?'],['Headaches','Preparations used to relieve headache'],['Wounds','Leaves or decoctions applied to wounds'],['Digestion','Plants recorded for indigestion']]){const b=el('button',label,'topic-chip');b.addEventListener('click',()=>{$('#library-query').value=recorded?label.toLowerCase():query;$('#knowledge-search').click();});$('#library-examples').append(b);}
async function searchLibrary(){
  const button=$('#knowledge-search'),status=$('#knowledge-search-status'),query=$('#library-query').value.trim(),serial=++librarySearchSerial;
  if(!query){resetLibrary();return;}librarySearchText=query;libraryLimit=6;
  if(recorded){rankedLibrary=null;loadLibrary();status.textContent='Searching wording in the recorded library.';return;}
  button.disabled=true;button.textContent='Searching\u2026';status.textContent='Finding relevant passages across the historical books\u2026';
  try{const result=await api('/api/research/knowledge-search?'+new URLSearchParams({q:query,method:$('#knowledge-method').value,limit:'40',collection:$('#library-collection').value,condition:$('#library-condition').value,role:$('#library-role').value}));if(serial!==librarySearchSerial)return;rankedLibrary=result.results.map(x=>x.record);status.textContent='Best matching source statements for \u201c'+query+'\u201d. Open a chapter to inspect its context.';await loadLibrary();}catch(error){if(serial===librarySearchSerial)status.textContent=error.message;}finally{if(serial===librarySearchSerial){button.disabled=false;button.textContent='Search knowledge';}}
}
$('#knowledge-search').addEventListener('click',searchLibrary);
$('#library-export-results').addEventListener('click',()=>{
  const csvValue=value=>{let text=String(value??'');if(/^[=+@-]/.test(text.trimStart()))text="'"+text;return '"'+text.replaceAll('"','""')+'"';};
  const rows=[['source_id','book','chapter','year','statement','statement_type','plant_wording','preparations','attribution_scope','negation_flag','researcher_note','source_url','licence'],...libraryVisibleRows.map(r=>[r.source_id,r.collection,r.source_title,r.date,r.statement.text,r.statement_role,r.names.map(n=>n.text).join('; '),r.preparations.map(p=>p.text).join('; '),r.scope,r.negated,libraryNotes[r.id]||'',r.url,r.licence])];
  offerExport(rows.map(r=>r.map(csvValue).join(',')).join('\r\n'),'foliatrace-selected-statements.csv','text/csv;charset=utf-8','Selected historical statements (CSV)');
});

$('#library-export-notes').addEventListener('click',()=>{
  const lines=['# FoliaTrace reading notes','',librarySavedOnly?'Saved historical statements.':'Historical statements from the current search and filters.','These are historical records, not treatment recommendations.',''];
  for(const r of libraryVisibleRows){
    lines.push('## '+sourceLabel(r.source_title),'',r.collection+' \xb7 '+r.date+' \xb7 '+statementRoleLabel(r),'',...r.statement.text.split('\n').map(line=>'> '+line),'','Plant wording: '+r.names.map(n=>n.text).join(', '),'Preparations mentioned: '+(r.preparations.map(p=>p.text).join(', ')||'not stated'),statementQualifier({relation_scope:r.scope,negated:r.negated}),'Source: '+r.url,'Rights: '+r.licence,'Original span: '+r.statement.start+'\u2013'+r.statement.end,'');
    if(libraryNotes[r.id])lines.push('### Researcher note (not source text)','',libraryNotes[r.id],'');
  }
  offerExport(lines.join('\n'),'foliatrace-reading-notes.md','text/markdown;charset=utf-8','Reading notes');
});
$('#library-backup').addEventListener('click',async()=>{
  const message=$('#library-backup-status');
  try{if(!librarySnapshot)await loadLibrary();if(!librarySnapshot)throw new Error('The library is not available. Try again when it has loaded.');
    const backup=readingBackup();offerExport(JSON.stringify(backup,null,2),'foliatrace-reading-backup.json','application/json','Saved reading backup');
    message.textContent=`Backup ready to download: ${backup.entries.length} entries, including entries outside the current filters.`;
  }catch(error){message.textContent=error.message;}
});
$('#library-restore').addEventListener('change',async event=>{
  const file=event.target.files[0],message=$('#library-backup-status');if(!file)return;
```

```
  try{if(file.size>50000000)throw new Error('Reading backups must be smaller than 50 MB.');
    if(!librarySnapshot)await loadLibrary();if(!librarySnapshot)throw new Error('The library is not available. Try again when it has loaded.');
    const result=restoreReadingBackup(JSON.parse(await file.text()));
    message.textContent=`Restored ${result.matched} matching entries. ${result.unavailable} entries belong to sources unavailable in this library; they remain in your backup data. ${result.conflicts} note conflicts: existing notes were kept; the incoming notes remain in your original backup file.`;
    updateSavedControl();await loadLibrary();
  }catch(error){message.textContent=`Could not restore: ${error.message}`;}
  finally{event.target.value='';}
});

let chemistrySnapshot;
async function loadChemistry() {
  const box = $("#chemical-records");
  try {
    if (!chemistrySnapshot) {
      chemistrySnapshot = await api('/api/research/chemistry');
      const choices = chemistrySnapshot.mappings || [];
      $("#chemical-compound").replaceChildren(...choices.map((c) => option(String(c.cid), `${c.name} \xb7 ${c.chembl_id}`)));
    }
    const cid = Number($("#chemical-compound").value), snapshot = chemistrySnapshot;
    box.replaceChildren();
    const mapping = snapshot.mappings.find((m) => m.cid === cid);
    if (!mapping) { box.append(el("p", "No verified chemical mappings installed.", "empty")); return; }
    box.append(el("p", `Exact InChIKey join: ${mapping.inchi_key} \xb7 ${snapshot.release.chembl_db_version} \xb7 ${snapshot.licence}`, "hint"));
    const rows = snapshot.records.filter((r) => r.cid === cid);
    box.append(el("h3", `${rows.length} reported assay measurements`));
    for (const r of rows) {
      const item = el("details", undefined, "reading-record");
      item.append(el("summary", `${r.target.pref_name} \xb7 ${r.standard_type} ${r.standard_relation} ${r.standard_value} ${r.standard_units}`), el("p", `${r.target.organism} \xb7 ${human(r.target.target_type)} \xb7 assay ${r.assay_id}`), el("blockquote", r.assay_description), link("ChEMBL measurement \u2197", r.url), document.createTextNode(" \xb7 "), link("Original document record \u2197", 'https://www.ebi.ac.uk/chembl/explore/document/' + r.document_id), el("p", `${r.licence} \xb7 ${r.scope}`, "hint"));
      box.append(item);
    }
    box.append(el("h3", "Publications in this snapshot mentioning the compound"));
    const papers = snapshot.publications[String(cid)] || [];
    if (!papers.length) box.append(el("p", "No publication co-mention in the bundled abstracts.", "empty"));
    for (const p of papers) {
      const item = el("details", undefined, "reading-record");
      item.append(el("summary", cleanTitle(p.title)), el("blockquote", p.supporting_text), link(p.paper_id + ' \u2197', p.url), el("p", `${p.licence} \xb7 ${p.scope}`, "hint"));
      box.append(item);
    }
    box.append(el("p", `${snapshot.selection} ${snapshot.attribution}`, "hint"));
    if (snapshot.failures.length) box.append(el("p", `${snapshot.failures.length} compound mappings unavailable; they were not guessed.`, "hint"));
  } catch (error) { box.replaceChildren(el("p", error.message, "error")); }
}

$("#chemical-compound").addEventListener('change', loadChemistry);
$("#verify-name-form").addEventListener('submit', async (event) => {
  event.preventDefault();
  const box = $("#name-check-result"), button = $("#verify-name-form button");
  button.disabled = true;
  try {
    const response = await api('/api/taxonomy/verify-scientific?' + new URLSearchParams({name: $("#verify-scientific-name").value.trim()}));
    box.replaceChildren(el("p", response.scope, "hint"));
    if (!response.results.length) box.append(el("p", "No scientific-name match returned.", "empty"));
    for (const c of response.results) box.append(link(`${c.currentCanonicalSimple || c.matchedName} \xb7 ${c.dataSourceTitleShort}`, c.outlink), el("p",
```

```
    `${c.taxonomicStatus || 'status not supplied'} \xb7 ${response.match_type || 'match type not supplied'}`, "hint"));
  } catch (error) { box.replaceChildren(el("p", error.message, "error")); }
  finally { button.disabled = false; }
});
function statementQualifier(claim) {
  const notes = [];
  if (claim.relation_scope === 'MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION') notes.push('Several plants are named; the use is not assigned to a single species.');
  if (claim.relation_scope === 'PARAGRAPH_CONTEXT_UNCERTAIN_RELATION') notes.push('A nearby sentence names another plant, so the subject of this sentence is uncertain. Literature is shown for the plant of this chapter; check the source before applying it.');
  if (claim.relation_scope === 'SOURCE_SECTION_HEADING') notes.push('Plant inferred from the chapter heading, not named in this sentence.');
  if (claim.relation_scope === 'PRECEDING_SENTENCE_SUBJECT') notes.push('Plant carried over from the previous sentence.');
  if (claim.relation_scope === 'PRECEDING_NAME_LIST') notes.push('Plant taken from the list of names that opens this entry.');
  if (claim.practice === 'RITUAL_OR_AMULET') notes.push('Reports an amulet or ritual practice.');
  if (claim.practice === 'VETERINARY') notes.push('Reports use on animals.');
  if (claim.practice === 'HOMOEOPATHIC') notes.push('Reports a homoeopathic claim.');
  if (claim.practice === 'NON_PLANT_REMEDY') notes.push('The remedy here is not a plant, so no plant literature is linked.');
  if (claim.negated) notes.push('Contains negation; read the original wording.');
  return notes.join(' ');
}
function comparisonTopics(result) {
  const topics = new Map();
  for (const [index, item] of result.sources.entries()) {
    for (const claim of item.claims.filter(c => c.statement_role === 'THERAPEUTIC_USE_RECORDED')) {
      for (const term of new Map(claim.conditions.map(c=>[c.id,c])).values()) {
        if (!topics.has(term.id)) topics.set(term.id, {id:term.id,label:term.label || human(term.id.replace('condition:','')),claims:result.sources.map(()=>[])});
        topics.get(term.id).claims[index].push(claim);
      }
    }
  }
  return [...topics.values()].sort((a,b) => b.claims.filter(c=>c.length).length-a.claims.filter(c=>c.length).length || a.label.localeCompare(b.label));
}
function offerExport(text,filename,type,title){
  const dialog=el('dialog',undefined,'export-preview'),head=el('div',undefined,'export-preview-head'),close=el('button','Close','quiet');
  const heading=el('h2',title);heading.id='export-preview-title';dialog.setAttribute('aria-labelledby',heading.id);
  close.type='button';close.addEventListener('click',()=>dialog.close());head.append(heading,close);
  const content=el('textarea');content.readOnly=true;content.value=text.replace(/^\uFEFF/,'');content.setAttribute('aria-label',title+' export text');
  const actions=el('div',undefined,'row'),download=el('button','Download file'),copy=el('button','Copy text','quiet'),message=el('p',undefined,'hint');
  message.setAttribute('role','status');
  download.type='button';download.addEventListener('click',()=>{
    const payload=type.startsWith('text/csv')&&!text.startsWith('\uFEFF')?'\uFEFF'+text:text;
    const url=URL.createObjectURL(new Blob([payload],{type})),anchor=el('a');anchor.href=url;anchor.download=filename;anchor.click();setTimeout(()=>URL.revokeObjectURL(url),1000);
    message.textContent='Download requested. You can also copy the text below.';
  });
  copy.type='button';copy.addEventListener('click',async()=>{
    try{await navigator.clipboard.writeText(content.value);message.textContent='Copied to clipboard.';}
    catch{content.focus();content.select();message.textContent='Select the text and copy it to keep this export.';}
  });
  actions.append(download,copy);dialog.append(head,el('p','Review the source references and your notes before saving or sharing.','hint'),content,actions,message);
  dialog.addEventListener('close',()=>dialog.remove());document.body.append(dialog);dialog.showModal();
}
function downloadNote(text, filename) {
  offerExport(text,filename,'text/markdown;charset=utf-8','Source comparison notes');
}
function exportComparison(result, topics) {
  const lines=['# FoliaTrace source comparison','','Topic links are extracted reading aids. Shared terms do not prove the same historical plant identity or an effective treatment.'];
  for(const item of result.sources)lines.push('', '## '+sourceLabel(item.source.title),item.source.context || '',String(item.source.date || 'Date
```

```
    unknown'),item.source.url,'Rights: '+item.source.licence);
  706   for(const topic of topics){
  707     lines.push('','## '+topic.label);
  708     result.sources.forEach((item,index)=>{
  709       lines.push('','### '+sourceLabel(item.source.title));
  710       if(!topic.claims[index].length)lines.push('No recorded-use statement extracted for this topic in this chapter. This does not establish absence.');
  711       for(const claim of topic.claims[index])lines.push('',...claim.support.text.split('\n').map(line=>'> '+line),'Preparations:
    '+(claim.preparation.map(p=>p.text).join(', ')||'not stated'),statementQualifier(claim),'Source span: '+claim.support.start+'\u2013'+claim.support.end);
  712     });
  713   }
  714   lines.push('','## Candidate identities','These are conditional research links.');
  715   for(const item of result.sources)for(const name of item.names)lines.push(sourceLabel(item.source.title)+' \u2014 '+name.label+':
    '+(name.branches.map(b=>b.name).join('; ')||'unresolved')+' ('+(STATUS[name.status]?.[1]||human(name.status))+')');
  716   for(const item of result.sources){const
    other=item.claims.filter(c=>c.statement_role!=='THERAPEUTIC_USE_RECORDED'||!c.conditions.length);if(other.length){lines.push('','## Additional context \u2014
    '+sourceLabel(item.source.title));for(const claim of other)lines.push('',statementRoleLabel(claim),...claim.support.text.split('\n').map(line=>'>
    '+line),statementQualifier(claim),'Source span: '+claim.support.start+'\u2013'+claim.support.end);}}
  717   downloadNote(lines.join('\n'),'foliatrace-source-comparison.md');
  718 }
  719 function comparisonQuotation(item,claim) {
  720   const fragment=el('div',undefined,'comparison-quotation');
  721   fragment.append(el('blockquote',claim.support.text));
  722   const facts=[['Parts',claim.part],['Preparations',claim.preparation]].map(([label,values])=>label+': '+(values.map(v=>v.text).join(', ')||'not stated'));
  723   fragment.append(el('p',facts.join(' \xb7 '),'hint'));
  724   const qualifier=statementQualifier(claim);if(qualifier)fragment.append(el('p',qualifier,'hint'));
  725   const open=el('button','Read in original context \u2192','text-action');
  726   open.addEventListener('click',()=>openLibraryRecord({source_id:item.source.id,statement:claim.support},'comparison'));
  727   fragment.append(open);return fragment;
  728 }
  729 function renderComparison(result,box) {
  730   const topics=comparisonTopics(result),shared=topics.filter(t=>t.claims.every(c=>c.length));
  731   box.replaceChildren(el('h3','What do these sources record?'));
  732   const intro=el('p',`${shared.length} shared topic${shared.length===1?'':'s'} \xb7 ${topics.length-shared.length} topic${topics.length-shared.length===1?'':'s'}
    extracted in only one chapter.`, 'comparison-intro');
  733   box.append(intro,el('p','A blank match means no use was extracted here. Read each quotation to see which plant, preparation and wording it actually
    concerns.','hint'));
  734   const actions=el('div',undefined,'row comparison-actions'),exportButton=el('button','Download comparison notes','quiet');
  735   exportButton.addEventListener('click',()=>exportComparison(result,topics));actions.append(exportButton);box.append(actions);
  736   const wrap=el('div',undefined,'table-wrap comparison-matrix'),table=el('table'),caption=el('caption','Recorded uses by topic; expand a cell to read its source
    quotations.');table.append(caption);
  737   const thead=el('thead'),head=el('tr'),topicHead=el('th','Topic');topicHead.scope='col';head.append(topicHead);
  738   for(const item of result.sources){const
    th=el('th');th.scope='col';th.append(el('strong',sourceLabel(item.source.title)),el('p',[item.source.context,item.source.date].filter(Boolean).join(' \xb7
    '),'hint'),link('Original book \u2197',item.source.url));head.append(th);}thead.append(head);table.append(thead);
  739   const tbody=el('tbody'),rows=[];
  740   for(const topic of topics){
  741     const row=el('tr'),label=el('th');label.scope='row';label.append(el('span',topic.label),pill(topic.claims.every(c=>c.length)?'Both chapters':'One
    chapter'));row.append(label);
  742     result.sources.forEach((item,index)=>{
  743       const cell=el('td'),claims=topic.claims[index];
  744       if(!claims.length)cell.append(el('p','No recorded use extracted','comparison-absence'));
  745       else {
  746         const details=el('details'),summary=el('summary',plural(claims.length,'statement'));
  747         const preparations=[...new Set(claims.flatMap(c=>c.preparation.map(p=>p.text.toLowerCase())))];
  748         if(preparations.length)summary.append(el('span',preparations.join(', '),'comparison-preparations'));
  749         details.append(summary);for(const claim of claims)details.append(comparisonQuotation(item,claim));cell.append(details);
  750       }
  751       row.append(cell);
```

```
    });
    rows.push({row,shared:topic.claims.every(c=>c.length)});tbody.append(row);
  }
  table.append(tbody);wrap.append(table);
  if(topics.length){const toggle=el('button','Show shared topics only','quiet');toggle.disabled=!shared.length;toggle.setAttribute('aria-pressed','false');toggle.addEventListener('click',()=>{const on=toggle.getAttribute('aria-pressed')!=='true';toggle.setAttribute('aria-pressed',String(on));toggle.textContent=on?'Show all topics':'Show shared topics only';for(const item of rows)item.row.hidden=on&&!item.shared;});actions.prepend(toggle);box.append(wrap);}
  else box.append(el('p','No therapeutic-use topic was extracted in these chapters. Their context statements remain available below.','hint'));
  const identities=el('details',undefined,'comparison-context');identities.append(el('summary','Which plant identities could connect these chapters?'));
  const linked=result.candidate_links.filter(c=>result.shared_candidate_ids.includes(c.id));
  identities.append(el('p',linked.length?'Shared candidates: '+linked.map(c=>c.name).join('; ')+'.':'No shared candidate in this snapshot.','hint'));
  for(const item of result.sources){identities.append(el('h4',sourceLabel(item.source.title)));for(const name of item.names)identities.append(el('p',name.label+': '+(name.branches.map(b=>b.name).join('; ')||'unresolved')),el('p',STATUS[name.status]?.[1]||human(name.status),'hint'));}
  identities.append(el('p','These candidates are hypotheses, not a verified historical identity match.','hint'));box.append(identities);
  const additional=el('details',undefined,'comparison-context');additional.append(el('summary','Harmful effects, other context and unclassified uses'));
  for(const item of result.sources){
    const other=item.claims.filter(c=>c.statement_role!=='THERAPEUTIC_USE_RECORDED'||!c.conditions.length);
    additional.append(el('h4',sourceLabel(item.source.title)));
    if(!other.length)additional.append(el('p','No additional statements extracted.','hint'));
    for(const claim of other){additional.append(pill(statementRoleLabel(claim),claim.statement_role==='ADVERSE_EFFECT_RECORDED'?'bad':'neutral'),comparisonQuotation(item,claim));}
  }
  box.append(additional);
}
$('#compare-run').addEventListener('click',async()=>{
  const button=$('#compare-run'),box=$('#comparison-result'),ids=[$('#compare-a').value,$('#compare-b').value];
  if(ids[0]===ids[1]){box.replaceChildren(el('p','Choose two different chapters to compare.','hint'));return;}
  button.disabled=true;box.replaceChildren(el('p','Reading the two source records\u2026','hint'));
  try{const result=await api('/api/research/compare-sources',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({passage_ids:ids})});renderComparison(result,box);}
  catch(error){box.replaceChildren(el('p',error.message,'error'));}
  finally{button.disabled=false;}
});
function showTab(id, focus = false) {
  for (const t of document.querySelectorAll('[role="tab"]')) {
    const on = t.id === id;
    t.setAttribute("aria-selected", String(on));
    t.tabIndex = on ? 0 : -1;
    $("#" + t.getAttribute("aria-controls")).hidden = !on;
    if (on && focus) t.focus();
  }
}

/* ---------- Source ---------- */
function mentionGroup(mentionId) {
  return data.identity_counterfactuals.find((g) => g.mention_ids.includes(mentionId));
}
function renderSource() {
  const requested=data.configuration.requested_name;
  $('#source-name-input').value=requested||'';
  $('#source-name-reset').hidden=!requested;
  $('#source-name-state').textContent=requested?'Researcher-selected wording: \u201c'+requested+'\u201d. Exact source occurrences are marked; botanical identity remains a separate question.':'';
  if(requested)$('#inspect-source-name').open=true;
  $("#source-title").textContent = data.source.title || "Untitled text";
  const meta = $("#source-meta");
  meta.replaceChildren();
```

```
  const parts = [data.source.context, data.source.date ? `Edition ${data.source.date}` : "Edition date not established", human(data.source.licence)];
  meta.append(document.createTextNode(parts.filter(Boolean).join(" \xb7 ") + " "));
  if (data.source.url) meta.append(link("Source \u2197", data.source.url));
  const box = $("#original");
  let focus = $('#source-focus');
  if(!focus){focus=el('aside',undefined,'source-focus');focus.id='source-focus';box.before(focus);}
  focus.replaceChildren();focus.hidden=!libraryFocus||libraryFocus.source_id!==data.source.id;
  if(!focus.hidden){
    focus.append(el('p','The statement you opened','kicker historical'),el('blockquote',libraryFocus.statement.text));
    const back=el('button',sourceFocusOrigin==='map'?'\u2190 Back to the research map':sourceFocusOrigin==='comparison'?'\u2190 Back to your comparison':sourceFocusOrigin==='evidence'?'\u2190 Back to this study':sourceFocusOrigin==='resume'?'\u2190 Back to reading overview':'\u2190 Back to your results','quiet');
    back.addEventListener('click',()=>{
      if(sourceFocusOrigin==='evidence'&&sourceEvidenceFocus){
        const target=sourceEvidenceFocus;selectName(target.nameKey);openBranchPublication(target.taxonId,target.paperId);
      }else if(sourceFocusOrigin==='map'){
        showView('map');requestAnimationFrame(()=>window.scrollTo({top:mapScrollY,behavior:'auto'}));
      }else if(sourceFocusOrigin==='resume'){
        focusNameInspector();
      }else if(sourceFocusOrigin==='comparison'){
        showView('workspace');$('#comparison-box').open=true;$('#comparison-result').scrollIntoView({behavior:'smooth',block:'start'});
      }else{showView('library');requestAnimationFrame(()=>window.scrollTo({top:libraryScrollY,behavior:'auto'}));}
    });
    const close=el('button','Clear statement focus','text-action');close.addEventListener('click',()=>{libraryFocus=null;renderSource();});
    const actions=el('div',undefined,'row');actions.append(back);
    if(sourceFocusOrigin!=='evidence'){
      const inspect=el('button','Inspect name & studies','quiet');inspect.type='button';
      inspect.addEventListener('click',()=>{showTab('tab-identity');focusNameInspector();});actions.append(inspect);
    }
    actions.append(close);focus.append(actions);
  }
  box.replaceChildren();
  const chars = Array.from(data.input);
  const activeSpan=!focus.hidden?libraryFocus.statement:null;
  function appendOriginalText(start,end) {
    if(!activeSpan||end<=activeSpan.start||start>=activeSpan.end){box.append(document.createTextNode(chars.slice(start,end).join('')));return;}
    const from=Math.max(start,activeSpan.start),to=Math.min(end,activeSpan.end);
    box.append(document.createTextNode(chars.slice(start,from).join('')),el('mark',chars.slice(from,to).join(''),'statement-highlight'),document.createTextNode(chars.slice(to,end).join('')));
  }
  let last = 0;
  for (const m of data.mentions) {
    if (m.start < last) continue;
    appendOriginalText(last,m.start);
    const g = mentionGroup(m.id);
    const b = el("button", chars.slice(m.start, m.end).join(""), "mention" + (g && g.key === selected ? " active" : ""));
    b.type = "button";
    b.dataset.group = g?.key || "";
    b.setAttribute("aria-label", `${m.text}: inspect this name`);
    b.addEventListener("click", () => {if(g){selectName(g.key);focusNameInspector();}});
    if(activeSpan&&m.start<activeSpan.end&&m.end>activeSpan.start)b.classList.add('statement-highlight');
    box.append(b);
    last = m.end;
  }
  appendOriginalText(last,chars.length);
  if(activeSpan)requestAnimationFrame(()=>{const mark=box.querySelector('.statement-highlight');if(mark)box.scrollTop+=mark.getBoundingClientRect().top-box.getBoundingClientRect().top-32;});
}
```

```
function renderClaims() {
  const g = group();
  const claims = data.historical_claims.filter((c) => !g || g.claim_ids.includes(c.id));
  $("#claims-summary").textContent = g ? `Source statements linked to \u201c${g.label}\u201d (${claims.length})` : `Source statements (${claims.length})`;
  const box = $("#claims");
  box.replaceChildren();
  if (!claims.length) box.append(el("p", "No plant\u2013use statement extracted for this name. Fields absent from the text stay unknown.", "hint"));
  for (const c of claims) {
    const row = el("div", undefined, "claim");
    row.append(pill(c.statement_role?statementRoleLabel(c):human('LEGACY_ROLE_NOT_SCREENED'), c.statement_role === 'ADVERSE_EFFECT_RECORDED' ? 'bad' : 'neutral'));
    const facts = el("div", undefined, "facts");
    const fact = (label, values) => {
      const s = el("span");
      s.append(el("b", label + ": "), document.createTextNode(values.length ? [...new Set(values)].join(", ") : "not stated"));
      facts.append(s);
    };
    fact("Condition terms", c.conditions.map((x) => x.original));
    fact("Part", c.part.map((x) => x.text));
    fact("Preparation", c.preparation.map((x) => x.text));
    row.append(el("blockquote", c.support.text), facts);
    const qualifier=statementQualifier(c);if(qualifier)row.append(el('p',qualifier,'hint'));
    if(g){const focus=el('button','Research this statement \u2192','text-action');focus.type='button';focus.addEventListener('click',()=>{
      statementReadingTargets.set(identityPairKey(g),c.id);
      for(const id of ['branch','study-type','relation','year'])$('#'+id).value='';
      renderEvidence();showTab('tab-evidence',true);$('.statement-reading').scrollIntoView({block:'start',behavior:'smooth'});rememberReading();
    });row.append(focus);}
    box.append(row);
  }
}

/* ---------- Names ---------- */
function renderNames() {
  const box = $("#names");
  box.replaceChildren();
  const groups = data.identity_counterfactuals;
  if (!groups.length) box.append(el("p", "No plant name recognised in this text. Try the Latin name, or enter the wording under \u201cInspect a name the extractor missed\u201d.", "empty"));
  for (const g of groups) {
    const b = el("button", g.label, "chip");
    b.type = "button";
    b.dataset.key = g.key;
    b.setAttribute("aria-pressed", String(g.key === selected));
    b.append(el("span", `\xd7${g.occurrences.length} \xb7 ${STATUS[g.status]?.[1] ?? human(g.status)}`, "count"));
    b.addEventListener("click", () => selectName(g.key));
    box.append(b);
  }
  const unlisted = data.local_names_not_in_any_list || [];
  if (unlisted.length) box.append(el("p", `Not in any open name list, so left unidentified: ${unlisted.slice(0, 8).join(", ")}${unlisted.length > 8 ? ` and ${unlisted.length - 8} more` : ""}.`, "hint"));
}
function selectName(key) {
  selected = key;
  for (const b of document.querySelectorAll("#names .chip")) b.setAttribute("aria-pressed", String(b.dataset.key === key));
  for (const b of document.querySelectorAll("#original .mention")) b.classList.toggle("active", b.dataset.group === key);
  renderInspector();
  renderClaims();
}
function focusNameInspector(){
```

```
  const heading=$('#name-title');heading.tabIndex=-1;heading.focus({preventScroll:true});
  $('.inspector-pane').scrollIntoView({behavior:'smooth',block:'start'});
}
const DIRECTION = { SUPPORTS: ["ok", "Reports benefit"], CONTRADICTS: ["bad", "Reports no benefit"], MIXED: ["warn", "Mixed result"], INSUFFICIENT: ["neutral", "Inconclusive"], NO_RESULT_STATEMENT: ["neutral", "No result sentence read"] };
function designName(types) {
  const set = new Set(types || []);
  return set.has("META_ANALYSIS") ? "Meta-analysis" : set.has("SYSTEMATIC_REVIEW") ? "Systematic review" : set.has("CLINICAL_TRIAL") ? "Clinical trial" : set.has("OBSERVATIONAL_HUMAN") ? "Observational study" : set.has("REVIEW") ? "Review" : "Study";
}
function resultCounts(r) {
  return [r.favourable ? `${r.favourable} favourable` : "", r.no_benefit ? `${r.no_benefit} no benefit` : "", r.mixed ? `${r.mixed} mixed or inconclusive` : ""].filter(Boolean).join(" \xb7 ");
}
function assessmentBasis(r) {
  const basis = r.assessment_basis;
  if (!basis) return null;
  const box = el('details', undefined, 'assessment-basis');
  box.append(el('summary', 'Why this summary?'));
  const stated = (basis.records || []).filter(s => s.direction && s.direction !== 'NO_RESULT_STATEMENT');
  const explanations = {
    CURRENT_SYNTHESES: `${stated.length} retrieved reviews with an attributable result inform this summary. Conflicting or uncertain review findings remain mixed. Reviews can share trials.`,
    INDIVIDUAL_PLANT_STUDIES: `${r.isolated_plant_studies ?? r.human_studies} human studies examine this plant without another named plant ingredient. Observational findings and small trial sets remain preliminary.`,
    COMBINATION_ONLY: 'The human studies test combinations. Their results do not isolate this plant.',
    PRECLINICAL_ONLY: 'The available attributable results are from laboratory or animal studies.',
    NO_ELIGIBLE_HUMAN_RESULT: 'No attributable human result was found in the retrieved records.'
  };
  box.append(el('p', explanations[basis.kind] || human(basis.kind), 'hint'));
  if (r.human_studies > r.isolated_plant_studies && basis.kind !== 'COMBINATION_ONLY') box.append(el('p', `${r.human_studies - r.isolated_plant_studies} combination studies remain available below, but do not decide the isolated-plant summary.`, 'hint'));
  for (const s of basis.records || []) {
    const [tone, label] = DIRECTION[s.direction] || DIRECTION.NO_RESULT_STATEMENT;
    const line = el('p', undefined, 'digest-meta');
    line.append(pill(label, tone), document.createTextNode(` ${s.year || 'Year unknown'} \xb7 `), link(cleanTitle(s.title), s.url));
    box.append(line);
  }
  if (stated.length) box.append(el('p', 'Compare the preparations, populations and measured outcomes in the studies. This summary is not a pooled effect estimate or a risk-of-bias assessment.', 'hint'));
  return box;
}
function studyList(r) {
  const box = el("details", undefined, "study-list");
  const shown = r.studies?.length || 0;
  box.append(el("summary", `Read ${shown === r.human_studies ? plural(shown, "study").replace("studys", "studies") : `${shown} of ${r.human_studies} studies`}`));
  const basis = assessmentBasis(r);
  if (basis) box.append(basis);
  for (const s of r.studies || []) {
    const [tone, label] = DIRECTION[s.direction] || DIRECTION.NO_RESULT_STATEMENT;
    const row = el("div", undefined, "study-row");
    const head = el("p");
    head.append(pill(label, tone), el("span", ` ${designName(s.design)}${s.year ? " \xb7 " + s.year : ""}${s.combination ? " \xb7 with other ingredients" : ""}${s.identity === "SCIENTIFIC_NAME" ? "" : " \xb7 species taken from the common name"} `, "digest-meta"), link(cleanTitle(s.title).slice(0, 110) + (s.title.length > 110 ? "\u2026" : "") + " \u2197", s.url));
    row.append(head);
    if (s.finding_statements?.length) {
      for (const statement of s.finding_statements) {
```

```
        row.append(el('p', human(statement.direction), 'quote-label'), el('blockquote', statement.supporting_text));
      }
    } else if (s.finding) row.append(el("blockquote", s.finding));
    box.append(row);
  }
  return box;
}
function renderResearchSummary(g) {
  const box = $("#research-summary");
  box.replaceChildren();
  if (!g || !g.branches.length) return;
  const ids = new Set(g.branches.map((b) => b.taxon_id)), rows = (data.research_digest || []).filter((r) => ids.has(r.taxon_id) &&
g.conditions.includes(r.condition_id));
  const matrix = data.resolutions.find((r) => g.mention_ids.includes(r.mention_id))?.identity_matrix;
  const leading = matrix?.candidates.find((c) => c.taxon_id === matrix.leading);
  const identity = leading ? `Best fit so far: ${leading.name} (${(RATING[leading.rating] || [])[1]?.toLowerCase()}). ` : g.branches.length > 1 ? "The candidates are
 not separated by the source wording. " : `Candidate: ${g.branches[0].name}. `;
  const withHuman = rows.filter((r) => r.human_studies).length, topics = new Set(rows.map((r) => r.topic));
  box.append(el("p", identity + (rows.length ? `${plural(topics.size, "recorded topic")} reached modern literature, ${withHuman} with human studies.` : "No study
retrieved for its recorded uses."), "summary-line"));
  const listed = data.resolutions.find((r) => g.mention_ids.includes(r.mention_id) && r.list_agreement?.names_agreeing > 1)?.list_agreement;
  if (listed) box.append(el("p", `${listed.names_agreeing} of the ${listed.names_listed} names the author lists point to this plant in published name lists. A list
of names is evidence for reading, not an identification.`, "hint"));
  const record = data.resolutions.flatMap((r) => (g.mention_ids.includes(r.mention_id) ? r.candidates : [])).find((c) => c.name_record)?.name_record;
  if (record) box.append(el("p", `Name recorded by ${record.source === "Duke" ? "Dr. Duke (USDA)" : record.source}: ${record.printed}${record.where ? ", " +
record.where : ""}.`, "hint"));
  if (!rows.length) {
    const known = g.branches.map((b) => data.plant_literature?.[b.taxon_id]).find((row) => row?.publications);
    if (known) {
      const note = el("p", `On other topics: ${plural(known.publications, "publication")} name ${known.name}, ${known.human_studies} human. `, "hint");
      for (const row of known.leading.slice(0, 2)) { note.append(link(cleanTitle(row.title).slice(0, 70) + "\u2026 \u2197", row.url), document.createTextNode(" "));
}
      box.append(note);
    }
  }
  if (rows.length) {
    const list = el("ul", undefined, "digest");
    for (const r of rows.slice(0, Math.min(8, Math.max(4, withHuman)))) {
      const [tone, label] = RESEARCH_STATE[r.state] || ["neutral", human(r.state)];
      const li = el("li"), counts = resultCounts(r);
      li.append(pill(label, tone), el("span", ` ${r.topic}`, "digest-topic"), el("span", ` \xb7 ${r.taxon}${r.human_studies ? " \xb7 " + plural(r.human_studies,
"human study").replace("studys", "studies") : ""}${counts ? " \xb7 " + counts : ""}${r.combination_only ? " \xb7 only tested with other ingredients" : ""}`, "digest-
meta"));
      if (r.independent_use) li.append(el("span", ` \xb7 reported elsewhere: ${independentLine(r.independent_use)}`, "digest-meta"));
      if (r.recorded_poisonous) li.append(document.createTextNode(" "), pill("Recorded as poisonous", "bad"));
      if (r.regulatory) li.append(el("span", ` \xb7 ${regulatoryLine(r.regulatory)}`, "digest-meta"));
      if (r.studies?.length) li.append(studyList(r));
      else {
        const basis = assessmentBasis(r);
        if (basis) li.append(basis);
        if (r.best_study) { li.append(document.createTextNode(" ")); li.append(link((r.best_study.year || "") + " study \u2197", r.best_study.url)); }
      }
      list.append(li);
    }
    box.append(list);
  }
}
function renderIdentityMatrix(g) {
```

```
  const box = $("#identity-matrix");
  box.replaceChildren();
  const matrix = g && data.resolutions.find((r) => g.mention_ids.includes(r.mention_id))?.identity_matrix;
  if (!matrix?.candidates.length) return;
  const section = el("section", undefined, "identity-matrix");
  section.append(el("h3", "Which candidate fits what the author wrote?"), el("p", matrix.candidates.length > 1 ? (matrix.separation === "CLEAR" ? "The source wording separates these candidates. Read the rows behind the rating." : "The source wording does not clearly separate these candidates.") : "One candidate, checked against the source wording.", "hint"));
  if(matrix.reading_order)section.append(el('p',matrix.reading_order,'hint'));
  for (const c of matrix.candidates) {
    const card = el("details", undefined, "matrix-candidate");
    card.open = c.taxon_id === matrix.leading || matrix.candidates.length === 1;
    const head = el("summary"), [tone, label] = RATING[c.rating] || ["neutral", c.rating];
    head.append(el("i", c.name), document.createTextNode(" "), pill(label, tone), el("span", ` score ${c.score}`, "digest-meta"));
    card.append(head);
    if (!c.rows.length) card.append(el("p", "The source gives no comparable description for this candidate.", "hint"));
    for (const row of c.rows) {
      const [rowTone, word] = MATRIX_STATUS[row.status] || ["neutral", human(row.status)];
      const item = el("div", undefined, "matrix-row");
      item.append(el("strong", row.feature + " "), pill(`${word} ${row.points > 0 ? "+" : ""}${row.points}`, rowTone));
      if (row.values?.length) item.append(el("span", " " + row.values.join(" / "), "digest-meta"));
      if (row.historical) item.append(el("blockquote", row.historical.text));
      if (row.modern?.text) item.append(el("p", row.modern.text, "hint"));
      card.append(item);
    }
    if (c.description_source) { const note = el("p", undefined, "hint"); note.append(document.createTextNode("Modern description: "), link(c.description_source.title + " \u2197", c.description_source.url), document.createTextNode(" \xb7 " + c.description_source.licence)); card.append(note); }
    section.append(card);
  }
  section.append(el("p", "Scoring adapted from Lardos et al. 2023: +2 fits, \u22122 conflicts, Strong 5 or more, Moderate 3\u20134, Weak 2 or less. It ranks hypotheses for reading; it does not identify the plant.", "hint"));
  box.append(section);
}
function renderInspector() {
  const g = group();
  $("#name-title").textContent = g ? `\u201c${g.label}\u201d` : "No name selected";
  const line = $("#name-status");
  line.replaceChildren();
  if (g) {
    const [tone, label, explain] = STATUS[g.status] || ["neutral", human(g.status), ""];
    line.append(pill(label, tone), document.createTextNode(explain));
    if(data.mentions.some(m=>g.mention_ids.includes(m.id)&&m.method==='RESEARCHER_SELECTED_NAME'))line.append(pill('Researcher-selected wording'));
    if(g.source_names?.length)line.append(pill('This source calls it '+g.source_names.map(n=>'\u201c'+n.name+'\u201d').join(', ')),document.createTextNode(' The author pairs this local name with the scientific name; the pairing is theirs, not a name-list match.'));
  }
  renderResearchSummary(g);
  renderIdentityMatrix(g);
  renderIdentity();
  setupFilters();
  renderEvidence();
  renderBrief();
  renderReviews();
}

/* ---------- Research hand-off ---------- */
function renderBrief() {
  const next = $("#research-next"), box = $("#research-brief");
  next.replaceChildren();
```

```
  box.replaceChildren();
  const brief = data.research_brief;
  const handoff=el('button','Record an evidence handoff','quiet');handoff.type='button';handoff.addEventListener('click',()=>showView('handoffs'));box.append(handoff);
  const name = brief?.names.find((n) => n.key === selected);
  if (!name) {
    box.append(el("p", "No research brief for this name. Re-run an older analysis, or check the source manually when no name was recognised.", "empty"));
    return;
  }
  next.append(el("strong", "Next research step"), el("p", name.next_actions[0].text));
  const open = el("button", "Open research brief", "quiet");
  open.type = "button";
  open.addEventListener("click", () => showTab("tab-review", true));
  next.append(open);
  box.append(el("h3", `Research brief: \u201c${name.label}\u201d`), el("p", "A hand-off for checking this source and planning your next search. Review judgements annotate the automated results; they do not establish efficacy.", "hint"));
  if (name.identity_review_state === "CONFLICTING_ACCEPTS") box.append(el("p", "More than one candidate accepted. Reconcile these judgements before choosing a plant.", "error"));
  const tasks = el("ol", undefined, "research-tasks");
  for (const action of name.next_actions) {
    const task = el("li", action.text);
    if(action.kind==='SEARCH_BEYOND_SNAPSHOT'){
      const searches=el('div',undefined,'research-search-links');
      for(const branch of group()?.branches||[]){const row=el('div');row.append(el('i',branch.name),document.createTextNode(' \xb7 '),currentLiterature(branch.name));searches.append(row);}
      task.append(searches);
    }
    if (action.evidence_ids.length) {
      const inspect = el("button", `Inspect ${plural(action.evidence_ids.length, "record")}`, "quiet task-records");
      inspect.type = "button";
      inspect.addEventListener("click", () => {
        let first;
        for (const item of box.querySelectorAll(".reading-record")) {
          if (JSON.parse(item.dataset.records).some((id) => action.evidence_ids.includes(id))) {
            item.open = true;
            if (item.parentElement.tagName === "DETAILS") item.parentElement.open = true;
            first ||= item;
          }
        }
        first?.scrollIntoView?.({ block: "nearest" });
      });
      task.append(inspect);
    }
    tasks.append(task);
  }
  box.append(tasks);
  const claims = el("details");
  claims.append(el("summary", `Source statements (${name.source_claims.length})`));
  for (const c of name.source_claims) {
    claims.append(pill(statementRoleLabel(c),c.statement_role==='ADVERSE_EFFECT_RECORDED'?'bad':'neutral'),el("blockquote", c.text));
    const qualifier=statementQualifier(c);if(qualifier)claims.append(el('p',qualifier,'hint'));
  }
  box.append(claims, el("h4", "Reading and verification queue"), el("p", brief.reading_order, "hint"));
  const queue = el("div", undefined, "reading-queue");
  if (!name.reading_list.length) queue.append(el("p", "No records retrieved in this snapshot. Search beyond it before drawing a conclusion.", "empty"));
  const publications = new Map();
  for (const r of name.reading_list) {
    if (!publications.has(r.paper.id)) publications.set(r.paper.id, []);
```

```
    publications.get(r.paper.id).push(r);
  }
  const rest = el("details", undefined, "more-reading");
  if (publications.size > 5) rest.append(el("summary", `Show ${publications.size - 5} more publications`));
  let index = 0;
  for (const records of publications.values()) {
    const p = records[0].paper;
    const row = el("details", undefined, "reading-record");
    row.dataset.records = JSON.stringify(records.map((r) => r.evidence_id));
    row.append(el("summary", `${cleanTitle(p.title)} \xb7 ${p.year || "year unknown"}`));
    row.append(link("Read original publication \u2197", p.url), el("p", `${p.id} \xb7 ${p.licence}`, "hint"));
    const compareSource=el('button','Compare this study with the source','quiet');
    compareSource.type='button';compareSource.addEventListener('click',()=>openBranchPublication(records[0].taxon_id,p.id));row.append(compareSource);
    if(records[0].reported_results?.length){
      row.append(el('h4','Author-reported results \xb7 study intervention and population'));
      for(const context of records[0].reported_results)row.append(el('blockquote',context.text),link('Original Results section \u2197',context.url));
    }
    for (const r of records) {
      const scope = el("div", undefined, "reading-scope");
      scope.append(el("p", `${r.taxon_name} \xb7 ${human(r.condition_id.replace("condition:", ""))} \xb7 ${r.evidence_id}`, "hint"));
      scope.append(el("p", `Species check: ${human(r.species_check)}. Final relation: ${human(r.final_relation)}.`));
      scope.append(el("p", `Failed checks: ${r.failed_checks.join(", ") || "none recorded"}. Unresolved: ${r.unresolved_checks.join(", ") || "none recorded"}. Screening: ${human(r.screening)}.`, "hint"));
      if (r.review) scope.append(el("p", `Researcher review: ${human(r.review.decision)} \u2014 ${r.review.reason}`));
      row.append(scope);
    }
    (index++ < 5 ? queue : rest).append(row);
  }
  if (publications.size > 5) queue.append(rest);
  box.append(queue);
}

/* ---------- Identity ---------- */
function historicalIdentityMarkdown(g){
  const references=data.historical_identity_references?.[g.key]||[];
  const lines=['# FoliaTrace historical name reading','','Current source: '+data.source.title,'Current source URL: '+(data.source.url||'not supplied'),'Historical name: '+g.label,'Analysis: '+data.run_id,'Snapshot: '+data.snapshot_sha256,'','The sources below share an exact normalized chapter heading, allowing camomile/chamomile and featherfew/feverfew. A later author\u2019s scientific-name wording does not settle an earlier author\u2019s plant identity. Candidate ranking and evidence relations are unchanged.'];
  for(const reference of references){
    const source=reference.source;lines.push('','## '+source.title+' \xb7 '+(source.collection||'Culpeper 1850'),'Date: '+(source.date||'not established'),'Source: '+source.url,'Rights: '+source.licence,'Source ID: '+source.id,'Text SHA256: '+reference.text_sha256);
    if(!reference.excerpts.length)lines.push('No current candidate scientific-name spelling was found in this matching chapter. Read its full text before concluding anything about its identification.');
    for(const excerpt of reference.excerpts){
      lines.push('',excerpt.text,'Source input span: '+excerpt.start+'\u2013'+excerpt.end);
      for(const name of excerpt.names)lines.push('Source wording: '+name.text+'; exact '+name.alias_kind+' spelling for '+name.candidate_name+' ('+name.candidate_id+') in the bundled taxonomy. The source may distinguish or reject this name; read the complete quote.');
    }
  }
  return lines.join('\n');
}
function renderHistoricalIdentityReferences(g){
  const box=$('#historical-identity-references');box.replaceChildren();
  if(!g)return;
  const references=data.historical_identity_references?.[g.key];
  if(!references?.length)return;
  const panel=el('section',undefined,'historical-name-reading');
```

```
  panel.append(el('h3','What another historical source calls this plant'),el('p','Read the other author\u2019s name wording and distinctions before applying a modern identity. A matching chapter title does not prove the two sources mean the same plant. A name can occur as a contrast with another plant.','hint'));
  for(const reference of references){
    const source=reference.source,card=el('article',undefined,'historical-name-source');
    card.append(el('h4',source.title+' \xb7 '+(source.collection||'Culpeper 1850')),el('p',(source.date||'date unknown')+' \xb7 '+source.licence,'hint'),link('Original historical source \u2197',source.url));
    const full=el('details',undefined,'historical-name-chapter');full.append(el('summary','Read this chapter without leaving your analysis'));
    const chapter=el('div',undefined,'historical-name-original');chapter.tabIndex=0;chapter.setAttribute('role','region');chapter.setAttribute('aria-label',source.title+' '+(source.collection||'Culpeper 1850')+' source context');
    const chars=Array.from(reference.input);
    const showContext=excerpt=>{
      chapter.replaceChildren();
      if(!excerpt){chapter.append(document.createTextNode(reference.input));return;}
      const mark=el('mark',excerpt.text,'statement-highlight');chapter.append(document.createTextNode(chars.slice(0,excerpt.start).join('')),mark,document.createTextNode(chars.slice(excerpt.end).join('')));
      full.open=true;requestAnimationFrame(()=>{chapter.scrollTop+=mark.getBoundingClientRect().top-chapter.getBoundingClientRect().top-24;chapter.focus({preventScroll:true});});
    };
    showContext(null);full.append(chapter);
    for(const excerpt of reference.excerpts){
      const leading=excerpt.start>0&&chars.slice(Math.max(0,excerpt.start-2),excerpt.start).join('')!=='\n\n';
      const trailing=excerpt.end<chars.length&&chars.slice(excerpt.end,excerpt.end+2).join('')!=='\n\n';
      const quote=el('section',undefined,'historical-name-quote');quote.append(el('p','Excerpt \xb7 open the chapter to read the surrounding context','hint'),el('blockquote',(leading?'\u2026 ':'')+excerpt.text+(trailing?' \u2026':'')));
      const names=new Map(excerpt.names.map(name=>[name.text+'::'+name.candidate_id,name]));
      for(const name of names.values())quote.append(el('p','Source names \u201c'+name.text+'\u201d \xb7 '+(name.alias_kind==='ACCEPTED_NAME'?'accepted scientific spelling':'stored scientific synonym')+' for '+name.candidate_name+' in the bundled taxonomy.','historical-name-match'));
      const read=el('button','Read this wording in the chapter','text-action');read.type='button';read.addEventListener('click',()=>{showContext(excerpt);full.scrollIntoView({behavior:'smooth',block:'nearest'});});quote.append(read);card.append(quote);
    }
    if(!reference.excerpts.length)card.append(el('p','This chapter shares the heading, but no current candidate\u2019s scientific-name spelling was found. Its full text remains available below.','hint'));
    card.append(full);panel.append(card);
  }
  const exportButton=el('button','Export this historical name reading','quiet');exportButton.type='button';exportButton.addEventListener('click',()=>offerExport(historicalIdentityMarkdown(g),'foliatrace-historical-name-reading.md','text/markdown;charset=utf-8','Historical name reading'));panel.append(exportButton);box.append(panel);
}
function identityPairKey(g){return data.run_id+'::'+g.key;}
function identityPairRows(g,a,b){
  const evidence=groupEvidence();
  const read=(branch,paperId)=>{
    const records=evidence.filter(e=>e.taxon_id===branch.taxon_id&&e.paper.id===paperId);
    const record=records.find(e=>e.comparability?.checks?.some(c=>c.check==='identity'&&c.status==='PASS'))||records[0];
    const check=record?.comparability?.checks?.find(c=>c.check==='identity');
    return {record,check,state:!record?'NOT_RETRIEVED':check?.status||'NOT_SCREENED'};
  };
  return [...new Set([...a.papers,...b.papers])].map(id=>{
    const left=read(a,id),right=read(b,id);
    return {id,left,right,paper:(left.record||right.record).paper,changed:left.state!==right.state};
  }).sort((x,y)=>Number(y.changed)-Number(x.changed)||Number(y.paper.year||0)-Number(x.paper.year||0)||x.id.localeCompare(y.id));
}
const IDENTITY_READING={PASS:'Species wording found',FAIL:'Species not established',PARTIAL:'Species partly established',UNKNOWN:'Species not established',NOT_SCREENED:'Not screened',NOT_RETRIEVED:'Not retrieved for this candidate'};
function identityPairMarkdown(g,a,b,rows){
  const lines=['# FoliaTrace identity hypothesis comparison','','Historical name: '+g.label,'Source: '+data.source.title,'Source URL: '+(data.source.url||'not supplied'),'Analysis: '+data.run_id,'Snapshot: '+data.snapshot_sha256,'', 'Hypothesis A: '+a.name+' ('+a.taxon_id+')','Hypothesis B: '+b.name+' ('+b.taxon_id+')','','Changing a reading hypothesis does not resolve the historical identity. Retrieval and species wording are separate from intervention
```

```
    applicability or treatment effects. Differences apply only to this analysis and bundled literature.','','## Historical statements'];
1214   for(const claim of data.historical_claims.filter(c=>g.claim_ids.includes(c.id)))lines.push('',claim.support.text,'Statement: '+claim.id+'; scope:
    '+claim.relation_scope);
1215   lines.push('','## Publication comparison');
1216   for(const row of rows){
1217     lines.push('','### '+cleanTitle(row.paper.title),row.paper.url,row.paper.id+' \xb7 '+(row.paper.year||'year unknown')+' \xb7 '+row.paper.licence);
1218     for(const [branch,result] of [[a,row.left],[b,row.right]]){
1219       lines.push('',branch.name+': '+(IDENTITY_READING[result.state]||human(result.state)));
1220       if(result.check?.span?.text)lines.push('Species wording: '+result.check.span.text);
1221       if(result.check?.detail)lines.push('Screening detail: '+result.check.detail);
1222     }
1223   }
1224   return lines.join('\n');
1225 }
1226 function identityPairComparison(g){
1227   const panel=el('section',undefined,'identity-pair');
1228   panel.append(el('h4','Compare two identity hypotheses'),el('p','Choose the plants you want to compare. The source stays the same; inspect how retrieval and species
     wording change.','hint'));
1229   const key=identityPairKey(g),saved=identityComparisons.get(key);
1230   let a=g.branches.find(b=>b.taxon_id===saved?.[0])||g.branches[0];
1231   let b=g.branches.find(b=>b.taxon_id===saved?.[1]&&b.taxon_id!==a.taxon_id)||g.branches.find(b=>b.taxon_id!==a.taxon_id);
1232   const controls=el('div',undefined,'identity-pair-controls'),leftLabel=el('label','If this name means'),rightLabel=el('label','Compare
    with'),left=el('select'),right=el('select');
1233   left.id='identity-hypothesis-a';right.id='identity-hypothesis-b';
1234   leftLabel.htmlFor=left.id;rightLabel.htmlFor=right.id;
1235   for(const branch of g.branches){left.append(option(branch.taxon_id,branch.name));right.append(option(branch.taxon_id,branch.name));}
1236   left.value=a.taxon_id;right.value=b.taxon_id;leftLabel.append(left);rightLabel.append(right);controls.append(leftLabel,rightLabel);panel.append(controls);
1237   const output=el('div',undefined,'identity-pair-output');panel.append(output);
1238   const draw=()=>{
1239     identityComparisons.set(key,[a.taxon_id,b.taxon_id]);output.replaceChildren();
1240     const rows=identityPairRows(g,a,b),changed=rows.filter(row=>row.changed),shared=rows.filter(row=>row.left.record&&row.right.record).length;
1241     output.append(el('p',changed.length+' publication'+(changed.length===1?' changes':'s change')+' in retrieval or species wording \xb7 '+shared+' retrieved for
    both.','identity-pair-result'));
1242     const actions=el('div',undefined,'row'),exportButton=el('button','Export this hypothesis comparison','quiet');exportButton.type='button';
1243     exportButton.addEventListener('click',()=>offerExport(identityPairMarkdown(g,a,b,rows),'foliatrace-identity-
    comparison.md','text/markdown;charset=utf-8','Identity hypothesis comparison'));actions.append(exportButton);output.append(actions);
1244     if(!rows.length){output.append(el('p','Neither candidate retrieved publications for these historical topics in this snapshot. This does not establish that
    studies do not exist.','empty'));return;}
1245     if(!changed.length)output.append(el('p','These two candidates have the same retrieval and species-screening states here. Their historical identity and study
    applicability still need separate reading.','hint'));
1246     const renderPaper=row=>{
1247       const card=el('details',undefined,'identity-pair-paper');card.dataset.paper=row.id;
1248       card.append(el('summary',cleanTitle(row.paper.title)),el('p',(row.changed?'Changes between these hypotheses':'Same retrieval and species-screening state')+'
    \xb7 '+(row.paper.year||'year unknown'),'hint'));
1249       const design=(row.left.record||row.right.record).study.types;card.append(el('p',design.map(human).join(', ')||'Study design not
    established','hint'),link('Original publication \u2197',row.paper.url));
1250       const grid=el('div',undefined,'identity-pair-grid');
1251       for(const [branch,result] of [[a,row.left],[b,row.right]]){
1252         const cell=el('section');cell.append(el('h5',branch.name),el('p',IDENTITY_READING[result.state]||human(result.state),result.state==='PASS'?'identity-wording-
    found':'hint'));
1253         if(result.check?.span?.text)cell.append(el('blockquote',result.check.span.text));
1254         if(result.check?.detail)cell.append(el('p',result.check.detail,'hint'));
1255         if(result.record){const inspect=el('button','Read study and source comparison','quiet');inspect.type='button';inspect.addEventListener('click',()=>openBranch
    Publication(branch.taxon_id,row.id,{returnToIdentityComparison:true}));cell.append(inspect);}
1256         grid.append(cell);
1257       }
1258       card.append(grid);return card;
1259     };
```

```
    for(const row of changed.slice(0,5))output.append(renderPaper(row));
    if(changed.length>5){const more=el('details',undefined,'identity-pair-more');more.append(el('summary','Read '+(changed.length-5)+' more changed publications'));for(const row of changed.slice(5))more.append(renderPaper(row));output.append(more);}
    const unchanged=rows.filter(row=>!row.changed);
    if(unchanged.length){const same=el('details',undefined,'identity-pair-more');same.append(el('summary','Read '+unchanged.length+' publications with unchanged screening'));for(const row of unchanged)same.append(renderPaper(row));output.append(same);}
    output.append(el('p','Species wording can occur in a review or a multi-ingredient study. It does not establish an experimental species, the historical identity or a treatment effect.','hint'));
  };
  left.addEventListener('change',()=>{const previous=a;a=g.branches.find(branch=>branch.taxon_id===left.value);if(a.taxon_id===b.taxon_id){b=previous;right.value=b.taxon_id;}draw();rememberReading();});
  right.addEventListener('change',()=>{const previous=b;b=g.branches.find(branch=>branch.taxon_id===right.value);if(a.taxon_id===b.taxon_id){a=previous;left.value=a.taxon_id;}draw();rememberReading();});
  draw();return panel;
}
function renderWhatIf(g) {
  const box = $("#whatif");
  box.replaceChildren();
  if (!g || !g.branches.length) return;
  const [headline, stable] = SENSITIVITY[g.sensitivity] || [human(g.sensitivity), true];
  const card = el("div", undefined, "whatif" + (stable ? " stable" : ""));
  card.append(el("h3", g.branches.length > 1 ? "If it is the other plant, what changes?" : "What a single answer would hide"), el("p", headline));
  const s = g.single_answer_view;
  if (g.branches.length > 1 || g.open_world.length) {
    const parts = [`A tool that returns one species would show ${s.name}${s.tie_broken_arbitrarily ? ", chosen only by sort order because the top candidates tie" : ""}.`];
    if (g.branches.length > 1) {
      const others = g.branches.filter((b) => b.taxon_id !== s.taxon_id).map((b) => b.name).join(" or ");
      if (s.misattributed_papers.length) parts.push(`${s.misattributed_papers.length} of the ${plural(s.shown_papers, "paper")} it lists actually name ${others}; it reaches them only through the shared common name.`);
      if (s.hidden_papers.length) parts.push(`It would drop ${plural(s.hidden_papers.length, "paper")} that only ${others} reaches.`);
      parts.push(`Its verdict would be \u201c${BRANCH_STATE[s.would_state].toLowerCase()}\u201d.`);
    } else parts.push(`It would not mention the ${plural(g.open_world.length, "other regional species")} listed below.`);
    const reconstruction=el('details',undefined,'identity-reconstruction');reconstruction.append(el('summary','What an automatic first choice would hide'),el('p',parts.join(' ')));card.append(reconstruction);
  }
  if (g.branches.length > 1) {
    card.append(identityPairComparison(g));
    const wrap = el("div", undefined, "table-wrap"),
      table = el("table"),
      head = el("tr");
    head.append(el("th", "Candidate"));
    for (const b of g.branches) {
      const th = el("th", b.name);
      th.style.fontStyle = "italic";
      head.append(th);
    }
    table.append(head);
    const row = (label, fn, cls) => {
      const tr = el("tr");
      tr.append(el("th", label));
      for (const b of g.branches) tr.append(el("td", fn(b), cls));
      table.append(tr);
    };
    row("Papers retrieved", (b) => b.papers.length, "num");
    row("Name the species", (b) => b.species_specific_papers.length, "num");
    row("Study designs", (b) => Object.entries(b.designs).map(([k, v]) => `${human(k)} ${v}`).join(", ") || "\u2014");
    row("Main reason it can't answer", (b) => {
```

```
      const top = Object.entries(b.blocking_checks).sort((x, y) => y[1] - x[1])[0];
      return top ? `${BLOCKING[top[0]] || human(top[0])} (${top[1]})` : "\u2014";
    });
    row("Branch conclusion", (b) => BRANCH_STATE[b.state] || human(b.state));
    wrap.append(table);
    const allCandidates=el('details',undefined,'identity-all-candidates');allCandidates.append(el('summary','Summary across all '+g.branches.length+' candidates'),wrap);card.append(allCandidates);
    const differences=el('details',undefined,'whatif-publications');
    differences.append(el('summary','Read the publications behind these differences'));
    for(const section of [
      {title:'Publications a single-species answer would omit',ids:s.hidden_papers},
      {title:'Shared-name results that name another candidate',ids:s.misattributed_papers},
    ]){
      if(!section.ids.length)continue;
      differences.append(el('h4',section.title));
      for(const paperId of section.ids){
        const matches=groupEvidence().filter(e=>e.paper.id===paperId&&e.taxon_id!==s.taxon_id);
        const record=matches.find(e=>e.comparability?.checks?.some(c=>c.check==='identity'&&c.status==='PASS'))||matches[0];
        if(!record)continue;
        const item=el('div',undefined,'reading-record');
        item.append(link(cleanTitle(record.paper.title),record.paper.url),el('p',taxonName(record.taxon_id)+' \xb7 '+(record.paper.year||'year unknown'),'hint'));
        const inspectStudy=el('button','Inspect species, abstract and applicability','quiet');
        inspectStudy.type='button';
        inspectStudy.addEventListener('click',()=>openBranchPublication(record.taxon_id,paperId));
        item.append(inspectStudy);differences.append(item);
      }
    }
    if(s.hidden_papers.length||s.misattributed_papers.length){
      differences.append(el('p','These are differences within this analysis and its literature snapshot. A species mention does not identify the historical plant or establish efficacy.','hint'));
      card.append(differences);
    }
  }
  box.append(card);
}
function renderIdentity() {
  const g = group();
  renderHistoricalIdentityReferences(g);
  renderWhatIf(g);
  const box = $("#identities");
  box.replaceChildren();
  if (!g || !g.branches.length) {
    box.append(el("p", "No modern candidate found. The name stays unresolved; nothing downstream is attached to it.", "empty"));
    if(g){const explore=el('button','Find historical cross-references','quiet');explore.addEventListener('click',()=>{showView('library');$('#library-query').value=g.label;searchLibrary();});box.append(explore);}
  }
  for (const b of g?.branches || []) {
    const c = data.resolutions.find((r) => g.mention_ids.includes(r.mention_id))?.candidates.find((x) => x.taxon_id === b.taxon_id);
    const card = el("div", undefined, "candidate");
    card.append(el("h3", b.name), el("p", `${c?.scientific_name ?? b.name} \xb7 ${c?.family ?? ""}`, "family"));
    const why = el("ul", undefined, "why");
    for (const r of b.reasons) why.append(el("li", REASONS[r] || human(r)));
    const alias = c?.matched_alias;
    const said = alias?.lang && alias.lang !== "en" ? ` (${languageName(alias.lang)})` : "";
    const recorded = alias?.sources?.length ? `, a local name recorded beside this identification in ${alias.sources.length === 1 ? "1 field study" : `${alias.sources.length} field studies`}` : "";
    if (alias?.name) why.append(el("li", alias.short_form_of ? `Matched \u201c${alias.name}\u201d${said}, the everyday short form of the recorded name \u201c${alias.short_form_of}\u201d` : `Matched \u201c${alias.name}\u201d${said}${recorded}`));
```

```
    card.append(why, el("p", `Ranking feature ${b.score.toFixed(2)} \u2014 orders candidates, not a probability.`, "hint"));
    const links = el("p", undefined, "hint");
    if (alias?.url) links.append(link(alias.sources?.length ? "Field study \u2197" : "Taxonomy record \u2197", alias.url));
    links.append(document.createTextNode(` \xb7 ${b.papers.length} papers on this branch`));
    card.append(links);
    card.append(currentLiterature(b.name));
    const actions = el("div", undefined, "actions");
    actions.append(reviewForm("identity", `${g.key}::${b.taxon_id}`, `Is \u201c${g.label}\u201d in this source ${b.name}?`));
    card.append(actions);
    box.append(card);
  }
  const ow = $("#open-world");
  ow.replaceChildren();
  if (g?.open_world.length) {
    const box2 = el("div", undefined, "openworld");
    box2.append(el("strong", "Also called this, outside the snapshot"), el("p", "GBIF lists these species of the same genus with a matching English name and at least 100 records in Great Britain. No evidence was retrieved for them.", "hint"));
    const ul = el("ul");
    for (const a of g.open_world) {
      const li = el("li");
      const i = el("i", a.name);
      const literature=currentLiterature(a.name);
      li.append(i, document.createTextNode(` \u2014 \u201c${a.shared_vernacular.join("\u201d, \u201c")}\u201d, ${a.region_records.toLocaleString()} GB records `));
      li.append(link("GBIF \u2197", `https://www.gbif.org/species/${a.gbif_key}`));
      ul.append(li);
      li.append(document.createTextNode(' \xb7 '),literature);
    }
    box2.append(ul);
    ow.append(box2);
  }
}

/* ---------- Evidence ---------- */
function groupEvidence() {
  const g = group();
  if (!g) return [];
  const ids = new Set(g.branches.flatMap((b) => b.evidence_ids));
  return data.evidence.filter((e) => ids.has(e.id));
}
function statementReadingTarget(g=group()){
  const id=g&&statementReadingTargets.get(identityPairKey(g));
  return id?data.historical_claims.find(claim=>claim.id===id&&g.claim_ids.includes(claim.id)):null;
}
function statementReadingMarkdown(claim,rows){
  const g=group(),lines=['# FoliaTrace statement reading','','Source: '+data.source.title,'Source URL: '+(data.source.url||'not supplied'),'Source date: '+(data.source.date||'not established'),'Source licence: '+data.source.licence,'Analysis: '+data.run_id,'Snapshot: '+data.snapshot_sha256,'','## Selected historical statement','',claim.support.text,'','Plant wording being investigated: '+g.label,'Statement: '+claim.id+'; scope: '+claim.relation_scope,'','This list contains only stored publication links to this statement, after the current candidate/design/relation/year filters. The links are research leads, not diagnostic equivalence, confirmed historical identities or treatment-effect conclusions.','', 'Reading filters: '+['branch','study-type','relation','year'].map(id=>id+': '+($('#'+id).value||'all')).join('; '),'','## Candidate identities'];
  for(const branch of g.branches){
    lines.push('- '+branch.name+' ('+branch.taxon_id+')');
    const review=data.researcher_reviews?.find(review=>review.target_kind==='identity'&&review.target_id===g.key+'::'+branch.taxon_id);
    if(review)lines.push('Stored identity judgement: '+review.decision+'; '+review.reason+'; '+review.reviewer+'; '+review.created_at);
  }
  lines.push('','## Publication reading list');
  const publications=new Map();
  for(const row of rows){const key=row.taxon_id+'::'+row.paper.id;if(!publications.has(key))publications.set(key,[]);publications.get(key).push(row);}
```

```
  for(const records of publications.values()){
    const row=records[0];
    const context=row.historical_contexts.find(context=>context.claim_id===claim.id),indication=context?.indication_comparison;
    lines.push('','### '+taxonName(row.taxon_id)+' \u2014 '+cleanTitle(row.paper.title),row.paper.url,row.paper.id+' \xb7 '+(row.paper.year||'year unknown')+' \xb7 '+row.paper.licence,'Design: '+row.study.types.map(human).join(', '));
    lines.push('Authors: '+row.paper.authors,'DOI: '+(row.paper.doi||'not supplied')+'; PMID: '+(row.paper.pmid||'not supplied'));
    if(context?.preparation_comparison)lines.push('Preparation comparison: '+context.preparation_comparison.detail);
    if(indication){lines.push('Indication reading: '+indication.detail);if(indication.study_aim)lines.push('Original abstract aim/objective wording: '+indication.study_aim.text);}
    for(const result of row.paper.reported_results||[])lines.push('Author-reported Results for the study intervention and population ('+result.licence+'): '+result.text,result.url);
    for(const record of records){
      lines.push('Topic '+record.condition_id+': automated relation '+record.relation.relation+'; failed checks: '+(record.comparability?.checks?.filter(check=>check.status==='FAIL').map(check=>check.check).join(', ')||'none recorded'));
      const review=data.researcher_reviews?.find(review=>review.target_kind==='evidence'&&review.target_id===record.id);
      if(review)lines.push('Stored evidence judgement: '+review.decision+'; '+review.reason+'; '+review.reviewer+'; '+review.created_at);
    }
  }
  if(!rows.length)lines.push('No stored publication links match this selected statement and the current filters. This does not establish that relevant studies do not exist.');
  return lines.join('\n');
}
function renderStatementReading(g,target,rows){
  const select=$('#evidence-source-statement'),box=$('#statement-reading-context'),jump=$('#statement-reading-jump');
  jump.replaceChildren();jump.hidden=!target;
  if(target){const back=el('button','\u2191 Your selected source statement','text-action');back.type='button';back.addEventListener('click',()=>{$('.statement-reading').scrollIntoView({block:'start',behavior:'auto'});select.focus({preventScroll:true});});jump.append(back,el('span',target.support.text.slice(0,85)+(target.support.text.length>85?'\u2026':''),'hint'));}
  select.replaceChildren(option('','All linked statements in this chapter'));
  for(const claim of data.historical_claims.filter(claim=>g?.claim_ids.includes(claim.id))){
    const text=claim.support.text,wording=[...new Set(claim.conditions.map(condition=>condition.original))].join(', ');
    select.append(option(claim.id,(wording?wording+' \xb7 ':'')+text.slice(0,110)+(text.length>110?'\u2026':'')));
  }
  select.disabled=!g;select.value=target?.id||'';box.replaceChildren();
  if(!target){box.append(el('p','Chapter overview: the list includes all recorded-use topics linked to this plant name. Choose one statement to keep its research question separate.','hint'));return;}
  box.append(pill(statementRoleLabel(target),target.statement_role==='ADVERSE_EFFECT_RECORDED'?'bad':'neutral'),el('blockquote',target.support.text));
  const describe=field=>[...new Set((target[field]||[]).map(span=>span.text))].join(', ')||'not stated';
  box.append(el('p','Part: '+describe('part')+' \xb7 Preparation: '+describe('preparation')+' \xb7 Route: '+describe('route'),'hint'));
  const qualifier=statementQualifier(target);if(qualifier)box.append(el('p',qualifier,'hint'));
  const actions=el('div',undefined,'row'),whole=el('button','Show all linked statements','quiet'),exportButton=el('button','Export this statement\u2019s reading list','quiet');
  whole.type='button';whole.addEventListener('click',()=>{statementReadingTargets.set(identityPairKey(g),'');renderEvidence();rememberReading();});
  exportButton.type='button';exportButton.addEventListener('click',()=>offerExport(statementReadingMarkdown(target,rows),'foliatrace-statement-reading.md','text/markdown;charset=utf-8','Selected statement reading'));
  actions.append(whole,exportButton);box.append(actions,el('p','This scope selects existing statement links. The identity comparison and full-analysis exports retain the chapter overview.','hint'));
}
$('#evidence-source-statement').addEventListener('change',()=>{if(!group())return;statementReadingTargets.set(identityPairKey(group()),$('#evidence-source-statement').value);renderEvidence();rememberReading();});
function setupFilters() {
  const g = group();
  const b = $("#branch"),
    s = $("#study-type");
  b.replaceChildren(option("", "All candidates"));
  for (const br of g?.branches || []) b.append(option(br.taxon_id, br.name));
  s.replaceChildren(option("", "All designs"));
  for (const t of [...new Set(groupEvidence().flatMap((e) => e.study.types))].sort()) s.append(option(t, human(t)));
```

```
  $("#relation").value = "";
  $("#year").value = "";
}
function openBranchPublication(taxonId,paperId,{returnToIdentityComparison=false}={}){
  const target=statementReadingTarget();
  if(target&&!groupEvidence().some(e=>e.taxon_id===taxonId&&e.paper.id===paperId&&(e.historical_contexts||[]).some(context=>context.claim_id===target.id))){
    statementReadingTargets.set(identityPairKey(group()),'');
    status('Opened this publication in the chapter overview: it has no stored link to the previously selected statement.');
  }
  $('#branch').value=taxonId;
  for(const id of ['study-type','relation','year'])$('#'+id).value='';
  renderEvidence();showTab('tab-evidence',true);
  const card=[...$('#evidence').querySelectorAll('.evidence-item')].find(item=>item.dataset.paper===paperId&&item.dataset.taxon===taxonId);
  if(!card)return;
  card.hidden=false;
  const section=card.closest('.branch-group'),more=section?.querySelector(':scope > button.more');
  if(more){const remaining=section.querySelectorAll(':scope > .evidence-item[hidden]').length;if(remaining)more.textContent='Read '+remaining+' more publications';else more.remove();}
  const sourceComparison=card.querySelector(':scope > .study-source-comparison');if(sourceComparison)sourceComparison.open=true;
  const context=card.querySelector(':scope > .study-context');if(context)context.open=true;
  if(returnToIdentityComparison){
    const back=el('button','\u2190 Back to these identity hypotheses','quiet');back.type='button';back.className+=' back-to-identity-comparison';
    back.addEventListener('click',()=>{
      showTab('tab-identity',true);
      const original=[...$('#whatif').querySelectorAll('.identity-pair-paper')].find(item=>item.dataset.paper===paperId);
      const target=original||$('#whatif .identity-pair');if(target){if(original)original.open=true;target.tabIndex=-1;target.focus({preventScroll:true});target.scrollIntoView({block:'start',behavior:'smooth'});}
    });card.prepend(back);
  }
  card.tabIndex=-1;card.focus({preventScroll:true});card.scrollIntoView({block:'start',behavior:'smooth'});
}
const TRIAL_READING_FIELDS=[['population','Participants'],['design','Design'],['intervention','Formulation'],['comparator','Comparator'],['follow_up','Follow-up'],['primary_outcome','Primary endpoint'],['primary_result','Primary finding'],['other_results','Other findings'],['reading_question','Reading question']];
function trialReadingMarkdown(readingSet){
  const lines=['# '+readingSet.title,'',readingSet.takeaway,'',readingSet.selection,'',readingSet.annotation.method,readingSet.annotation.licence,readingSet.annotation.rights_scope,'',readingSet.link_basis];
  for(const study of readingSet.studies){
    const publication=study.publication;
    lines.push('','## '+study.label,'',publication.title,publication.authors,'PMID '+publication.pmid+' \xb7 DOI '+publication.doi,publication.url,'');
    for(const [key,label] of TRIAL_READING_FIELDS)lines.push(label+': '+study[key],'');
  }
  lines.push('Reading-set ID: '+readingSet.id,'Metadata source: '+readingSet.studies[0].publication.provenance.url,'Factual note source: '+readingSet.note_source.path+' \xb7 '+readingSet.note_source.sha256);
  return lines.join('\n');
}
function trialReadingComparison(readingSet){
  const panel=el('details',undefined,'trial-readings');panel.dataset.readingSet=readingSet.id;
  panel.append(el('summary','Compare controlled trials \xb7 '+plural(readingSet.studies.length,'selected reading')),
    el('h3',readingSet.title),el('p','Compare the primary endpoints and results. Open each trial for its formulation, participants and secondary findings.'),el('p','AI-assisted factual notes \xb7 linked original studies \xb7 separate from automated relation labels','hint'));
  const grid=el('div',undefined,'trial-reading-grid');
  for(const study of readingSet.studies){
    const card=el('article',undefined,'trial-reading-card');card.dataset.pmid=study.pmid;
    card.append(el('h4',study.label),link('Original study \u2197',study.publication.url),el('p',study.follow_up+' \xb7 '+study.comparator,'hint'));
    const fields=el('dl'),contextFields=el('dl'),context=el('details',undefined,'trial-reading-context');
    context.append(el('summary','Formulation, population and other findings'),el('p',cleanTitle(study.publication.title),'hint'));
    for(const [key,label] of TRIAL_READING_FIELDS){
      const target=['primary_outcome','primary_result'].includes(key)?fields:contextFields;
```

```
      target.append(el('dt',label),el('dd',study[key],key==='primary_result'?'primary-reading-result':undefined));
    }
    context.append(contextFields);card.append(fields,context);grid.append(card);
  }
  panel.append(grid,el('p',readingSet.takeaway));
  const provenance=el('details',undefined,'reading-note-provenance');provenance.append(el('summary','Reading-note sources and scope'),el('p',readingSet.selection),el('p',readingSet.link_basis),el('p',readingSet.annotation.method),el('p',readingSet.annotation.licence),el('p',readingSet.annotation.rights_scope));
  for(const study of readingSet.studies)provenance.append(el('p',study.publication.authors+' \xb7 PMID '+study.pmid+' \xb7 DOI '+study.publication.doi,'hint'),link('Indexed source record \u2197',study.publication.url));
  panel.append(provenance);
  const actions=el('div',undefined,'row'),markdown=el('button','Export trial comparison','quiet'),csv=el('button','Comparison CSV','quiet');
  markdown.type='button';markdown.addEventListener('click',()=>offerExport(trialReadingMarkdown(readingSet),'foliatrace-trial-comparison.md','text/markdown;charset=utf-8',readingSet.title));
  csv.type='button';csv.addEventListener('click',()=>{
    const rows=[['study','title','pmid','doi','source_url',...TRIAL_READING_FIELDS.map(([key])=>key),'annotation_method','annotation_licence']];
    for(const study of readingSet.studies)rows.push([study.label,study.publication.title,study.pmid,study.publication.doi,study.publication.url,...TRIAL_READING_FIELDS.map(([key])=>study[key]),readingSet.annotation.method,readingSet.annotation.licence]);
    offerExport(rows.map(row=>row.map(csvValue).join(',')).join('\r\n'),'foliatrace-trial-comparison.csv','text/csv;charset=utf-8','Controlled-trial reading comparison (CSV)');
  });
  actions.append(markdown,csv);panel.append(actions);return panel;
}
function openTrialReadingSet(id){
  const readingSet=(data.study_reading_sets||[]).find(item=>item.id===id);
  const name=readingSet&&data.identity_counterfactuals.find(g=>g.branches.some(b=>readingSet.taxon_ids.includes(b.taxon_id))&&g.conditions.some(c=>readingSet.condition_ids.includes(c)));
  if(!name){status('This analysis has no matching trial reading set. Analyze the source again to obtain the current readings.',true);return;}
  statementReadingTargets.set(identityPairKey(name),'');selectName(name.key);$('#branch').value=readingSet.taxon_ids.find(id=>name.branches.some(b=>b.taxon_id===id))||'';
  for(const filter of ['study-type','relation','year'])$('#'+filter).value='';
  renderEvidence();showTab('tab-evidence',true);
  const panel=[...$('#evidence').querySelectorAll('.trial-readings')].find(item=>item.dataset.readingSet===id);
  if(panel){panel.open=true;panel.tabIndex=-1;panel.focus({preventScroll:true});panel.scrollIntoView({block:'start',behavior:'smooth'});}
}
function studyComparisonScope(){
  return (group()?identityPairKey(group()):'none')+'::'+(statementReadingTarget()?.id||'chapter');
}
function studyComparisonPool(){
  const target=statementReadingTarget(),pool=new Map();
  for(const e of groupEvidence()){
    if(target&&!(e.historical_contexts||[]).some(c=>c.claim_id===target.id))continue;
    const key=JSON.stringify([e.taxon_id,e.paper.id]);
    if(!pool.has(key))pool.set(key,{key,records:[]});
    pool.get(key).records.push(e);
  }
  return pool;
}
function selectedStudyComparisons(){
  const pool=studyComparisonPool();
  return (studyComparisons.get(studyComparisonScope())||[]).map(key=>pool.get(key)).filter(Boolean);
}
function toggleStudyComparison(e){
  const key=JSON.stringify([e.taxon_id,e.paper.id]),scope=studyComparisonScope(),selected=selectedStudyComparisons().map(item=>item.key);
  if(selected.includes(key))selected.splice(selected.indexOf(key),1);
  else if(selected.length<3)selected.push(key);
  else {status('Three studies are selected. Remove one before adding another.',true);return;}
  studyComparisons.set(scope,selected);renderStudyComparisonSelection();rememberReading();
}
```

```
function renderStudyComparisonSelection(){
  const box=$('#study-comparison-selection'),selected=selectedStudyComparisons(),keys=new Set(selected.map(item=>item.key));
  box.replaceChildren();box.hidden=!selected.length;
  if(selected.length){
    box.append(el('h3','Your study comparison'),el('p','Choose two or three study / candidate records for this source question. Changing a filter keeps your selection; a different source statement has its own selection.','hint'));
    const list=el('ul');
    for(const item of selected){
      const e=item.records[0],row=el('li'),remove=el('button','Remove','text-action');
      row.append(el('span',cleanTitle(e.paper.title)+' \xb7 '+taxonName(e.taxon_id)));
      remove.type='button';remove.setAttribute('aria-label','Remove '+cleanTitle(e.paper.title)+' for '+taxonName(e.taxon_id));
      remove.addEventListener('click',()=>{toggleStudyComparison(e);$('#study-comparison-selection').querySelector('button')?.focus();});row.append(remove);list.append(row);
    }
    const actions=el('div',undefined,'row'),open=el('button',selected.length>1?'Compare selected studies ('+selected.length+')':'Choose one more study'),clear=el('button','Clear selection','quiet');
    open.type='button';open.disabled=selected.length<2;open.addEventListener('click',openStudyComparison);
    clear.type='button';clear.addEventListener('click',()=>{studyComparisons.delete(studyComparisonScope());renderStudyComparisonSelection();rememberReading();$('#modern-question').tabIndex=-1;$('#modern-question').focus();});
    actions.append(open,clear);box.append(list,actions);
  }
  for(const button of document.querySelectorAll('[data-study-choice]')){
    const included=keys.has(button.dataset.studyChoice);button.textContent=included?'Remove from comparison':'Select for comparison';button.setAttribute('aria-pressed',String(included));
    button.disabled=!included&&selected.length>=3;
  }
  for(const button of document.querySelectorAll('[data-open-study-comparison]')){button.hidden=selected.length<2;button.textContent='Compare selected ('+selected.length+')';}
}
function studyMeasurementReadings(e){
  const primary=/\b(?:co[- ]?)?primary\s+(?:clinical\s+)?(?:outcomes?|end\s*points?|efficacy\s+(?:variables?|parameters?|measures?))\b/i;
  const secondary=/\bsecondary\s+(?:clinical\s+)?(?:outcomes?|end\s*points?|efficacy\s+(?:variables?|parameters?|measures?))\b/i;
  return (e.paper.methods_contexts||[]).filter(context=>context.source_kind!=='FULL_TEXT_METHODS_TABLE').filter(context=>{
    const explicit=primary.test(context.text)||secondary.test(context.text)||/\boutcome measures?\b/i.test(context.text);
    const measurementSection=/assessments?|outcomes?|measurements?|biophysical parameters|data collection/i.test(context.section);
    return explicit||measurementSection&&/\b(?:record(?:ed)?|assess(?:ed|ment)?|measur(?:ed|ements?)|evaluat(?:ed|ion)|analy[sz]ed|rating scales?|questionnaires?)\b/i.test(context.text);
  }).map(context=>{
    const priorities=[primary.test(context.text)?'primary':null,secondary.test(context.text)?'secondary':null].filter(Boolean);
    return {context,priority:priorities.length?'Authors explicitly use '+priorities.join(' and ')+' outcome wording in this excerpt.':'Primary/secondary priority is not stated in this excerpt.'};
  });
}
function studyMeasurementPanel(e,readings){
  const panel=el('details',undefined,'study-outcome-definitions');
  panel.append(el('summary',readings.length?'What was measured \xb7 '+plural(readings.length,'source excerpt'):'What was measured \xb7 read the original study'));
  if(!readings.length){
    panel.append(el('p','No measurement definition has been isolated in the stored Methods excerpts. This does not mean the study had no outcomes.'),link('Read original study \u2197',e.paper.url));return panel;
  }
  panel.append(el('p','Original Methods wording. Outcomes may belong to different parts of a study; a measurement or endpoint is not a finding.','hint'));
  for(const reading of readings){
    const {context,priority}=reading,section=el('section');
    section.append(el('p',context.section,'quote-label'),el('p',priority,'hint'),el('blockquote',context.text),link('Original Methods \u2197',context.url),el('p',context.id+' \xb7 '+context.licence,'hint'));panel.append(section);
  }
  return panel;
}
```

```
function studyComparisonFields(item){
  const records=item.records,e=records[0],paper=e.paper,study=e.study,target=statementReadingTarget();
  const measurements=studyMeasurementReadings(e);
  const unique=values=>[...new Set(values.filter(Boolean))];
  const quote=span=>{
    if(!span)return 'Not established in the stored source excerpts.';
    const context=(paper.methods_contexts||[]).find(c=>c.id===span.source_id);
    return '\u201c'+span.text+'\u201d \u2014 '+(context?context.section+' \xb7 '+context.id:human(span.source_kind||'ABSTRACT'))+(Number.isInteger(span.start)?' \xb7 characters '+span.start+'\u2013'+span.end:'')+(context?.url?' \xb7 '+context.url:'');
  };
  const forms=study.preparation_attribution?.forms||[],comparator=e.comparability?.checks.find(c=>c.check==='comparator');
  const contexts=[...new Map(records.flatMap(record=>(record.historical_contexts||[]).filter(c=>!target||c.claim_id===target.id)).map(c=>[c.claim_id,c])).values()];
  const findings=unique((paper.reported_results||[]).map(c=>c.section+' \xb7 '+c.licence+'\n\u201c'+c.text+'\u201d\n'+c.url));
  if(!findings.length)for(const record of records){const finding=record.relation.reported_finding;if(finding?.direction!=='NO_RESULT_STATEMENT')for(const statement of finding?.statements?.length?finding.statements:[finding])if(statement?.supporting_text)findings.push('Original abstract finding ('+human(statement.direction)+')\n\u201c'+statement.supporting_text+'\u201d');}
  const formulation=unique(forms.map(form=>quote({...form,text:form.context?.text||form.text,start:form.context?.start??form.start,end:form.context?.end??form.end})));
  if(!formulation.length)formulation.push(study.pico_source_spans?.intervention_preparation?quote(study.pico_source_spans.intervention_preparation):'No preparation wording has been assigned to this candidate in the stored excerpts.');
  const methodIds=new Set([...(study.preparation_attribution?.methods_context_ids||[]),...(e.comparability?.checks||[]).map(c=>c.span?.source_id)].filter(Boolean));
  for(const context of paper.methods_contexts||[])if(methodIds.has(context.id)&&/intervention|preparation/i.test(context.section)&&context.source_kind!=='FULL_TEXT_METHODS_TABLE')formulation.push(context.section+' \xb7 original wording\n\u201c'+context.text+'\u201d\n'+context.url+' \xb7 '+context.licence+' \xb7 '+context.id);
  formulation.push('Candidate / preparation association: '+human(study.preparation_attribution?.status||'NOT_RECORDED')+'. Lexical association requires intervention review.');
  const historical=unique(contexts.flatMap(c=>['Historical statement \xb7 '+c.claim_id+'\n\u201c'+c.support.text+'\u201d',c.preparation_comparison?.detail,c.indication_comparison?.detail]));
  return [
    {label:'Candidate being investigated',paragraphs:[taxonName(e.taxon_id)+' \xb7 '+e.taxon_id]},
    {label:'Study design',paragraphs:[study.types.map(human).join(', '),'Automated classification \xb7 '+human(study.population_status)+'. Risk of bias has not been appraised.']},
    {label:'Population wording',paragraphs:[quote(study.pico_source_spans?.population),...(study.sample_size_support?['Sample-size wording: '+(typeof study.sample_size_support==='string'?study.sample_size_support:quote(study.sample_size_support))]:[])]},
    {label:'Candidate-linked preparation in context',paragraphs:formulation},
    {label:'Comparator wording',paragraphs:[study.pico_source_spans?.comparator?quote(study.pico_source_spans.comparator):comparator?.span?.source_kind?.startsWith('FULL_TEXT_METHODS')?'Design wording in Methods: '+quote(comparator.span):'No comparator wording was established in the stored excerpts.',...(comparator?[comparator.detail]:[])]},
    {label:'Measurement definitions and outcome priorities',paragraphs:measurements.length?measurements.map(({context,priority})=>priority+'\n'+context.section+' \xb7 original Methods\n\u201c'+context.text+'\u201d\n'+context.url+' \xb7 '+context.licence+' \xb7 '+context.id):['No measurement definition has been isolated in the stored Methods excerpts. Read the original study; missing extraction is not absence of an outcome.']},
    {label:'Reported findings',paragraphs:[...(unique(findings).length?unique(findings):['No explicit result statement was extracted in the stored excerpts. Open the original study.']),...unique(records.flatMap(record=>record.relation.reported_finding?.comparison_notes||[]))]},
    {label:'Link to the historical question',paragraphs:historical.length?historical:['No statement context stored for this record.']},
    {label:'Reading limits by topic',paragraphs:records.map(record=>human(record.condition_id.replace('condition:',''))+' \xb7 '+(record.comparability?.checks||[]).map(c=>(CHECK_LABEL[c.check]||c.check)+': '+human(c.status)).join('; '))},
    {label:'Publication and rights',paragraphs:[paper.authors||'Authors not supplied',paper.id+' \xb7 '+(paper.year||'year not supplied')+' \xb7 '+paper.licence,paper.url,...(paper.doi?['DOI https://doi.org/'+paper.doi]:[])]}
  ];
}
function studyComparisonMarkdown(items){
  const g=group(),target=statementReadingTarget(),lines=['# FoliaTrace selected-study comparison','','Source: '+data.source.title,data.source.url,'Source date: '+data.source.date+' \xb7 '+data.source.licence,'Analysis: '+data.run_id,'Snapshot: '+data.snapshot_sha256,'Plant wording: '+g.label,'Reading scope: '+(target?target.id+' \xb7 '+target.support.text:'Chapter overview \u2014 these records may link to different historical statements.'),'','Reader-selected records, not a systematic review or pooled effect estimate. The same publication may appear under different candidate hypotheses; those columns are not independent studies. Source wording and rule-based checks require critical reading.'];
  lines.push('Historical-name resolution: '+human(g.status)+'. Modern candidate names do not establish the historical referent.');
  for(const item of items){
    const e=item.records[0];lines.push('','## '+cleanTitle(e.paper.title),'Stored links: '+item.records.map(record=>record.id).join(', '));
```

```
    lines.push('Identity scope: '+(e.identity_qualifier||'Conditional on the candidate identity being investigated'));
    for(const field of studyComparisonFields(item))lines.push('','### '+field.label,...field.paragraphs.map(p=>p+'\n'));
    const reviews=(data.researcher_reviews||[]).filter(review=>review.target_kind==='evidence'&&item.records.some(record=>record.id===review.target_id)||review.target_kind==='identity'&&review.target_id===g.key+'::'+e.taxon_id);
    for(const review of reviews)lines.push('Stored '+review.target_kind+' judgement ('+review.target_id+'): '+review.decision+' \xb7 '+review.reason+' \xb7 '+review.reviewer+' \xb7 '+review.created_at);
  }
  lines.push('','## Original study excerpts','One source appendix per publication, even when several candidate hypotheses use it. These are the stored licensed excerpts, not full articles.');
  for(const paper of new Map(items.map(item=>[item.records[0].paper.id,item.records[0].paper])).values()){
    lines.push('','### '+paper.id+' \xb7 '+cleanTitle(paper.title),paper.authors||'Authors not supplied',paper.url);
    if(paper.abstract_rights==='LOCAL_CACHE_NOT_REDISTRIBUTED')lines.push('Abstract not copied into this export: rights remain with the publisher. Read it at the link above.');
    else lines.push('Abstract rights: '+paper.licence,'','#### Original abstract','',paper.abstract);
    for(const context of paper.methods_contexts||[])lines.push('','#### '+context.section,'Original source: '+context.url,'Excerpt: '+context.id+' \xb7 '+context.licence,'Text SHA256: '+context.text_sha256,'Document SHA256: '+context.document_sha256,context.text_format||'Whitespace-normalized JATS paragraph; no inferred wording.','',context.text);
  }
  return lines.join('\n');
}
function openStudyComparison(){
  const items=selectedStudyComparisons();if(items.length<2)return;
  const dialog=el('dialog',undefined,'study-comparison-dialog'),head=el('div',undefined,'study-comparison-heading'),heading=el('h2','Compare selected studies'),close=el('button','Close','quiet');
  heading.id='study-comparison-title';dialog.setAttribute('aria-labelledby',heading.id);close.type='button';close.autofocus=true;close.addEventListener('click',()=>dialog.close());head.append(heading,close);
  const target=statementReadingTarget(),intro=el('section',undefined,'study-comparison-question');intro.append(el('strong',data.source.title+' \xb7 '+group().label));
  intro.append(target?el('blockquote',target.support.text):el('p','Chapter overview: these records may address different source statements. Compare their historical links below.'));
  intro.append(el('p','Reader-selected research leads. Compare interventions and outcomes before interpreting differences. Columns for the same publication under different candidate hypotheses are not independent studies.','hint'));
  const scroller=el('div',undefined,'study-comparison-scroll');scroller.tabIndex=0;scroller.setAttribute('role','region');scroller.setAttribute('aria-label','Study comparison table; scroll horizontally to read every column');
  const table=el('table'),caption=el('caption','Selected study records \xb7 original wording and reading limits'),thead=el('thead'),tr=el('tr');
  table.style.minWidth=(150+items.length*300)+'px';
  const corner=el('th','Reading question');corner.scope='col';tr.append(corner);
  for(const item of items){const e=item.records[0],th=el('th');th.scope='col';th.append(link(cleanTitle(e.paper.title),e.paper.url),el('p',taxonName(e.taxon_id)+' \xb7 '+(e.paper.year||'year unknown'),'hint'));tr.append(th);}
  thead.append(tr);const tbody=el('tbody'),fields=items.map(studyComparisonFields);
  fields[0].forEach((field,index)=>{
    const row=el('tr'),label=el('th',field.label);label.scope='row';row.append(label);
    for(const column of fields){const cell=el('td');for(const paragraph of column[index].paragraphs)cell.append(el('p',paragraph));row.append(cell);}tbody.append(row);
  });
  const sourceRow=el('tr'),sourceLabel=el('th','Read original excerpts');sourceLabel.scope='row';sourceRow.append(sourceLabel);
  for(const item of items){
    const e=item.records[0],cell=el('td'),details=el('details');details.append(el('summary','Full abstract and available Methods'),el('p',e.paper.abstract,'abstract'));
    for(const context of e.paper.methods_contexts||[])details.append(el('h4',context.section),el('blockquote',context.text),link('Original Methods \u2197',context.url),el('p',context.id+' \xb7 '+context.licence,'hint'));
    cell.append(details);sourceRow.append(cell);
  }
  tbody.append(sourceRow);table.append(caption,thead,tbody);scroller.append(table);
  const actions=el('div',undefined,'row'),exportButton=el('button','Export this comparison','quiet');exportButton.type='button';
  exportButton.addEventListener('click',()=>offerExport(studyComparisonMarkdown(items),'foliatrace-selected-study-comparison.md','text/markdown;charset=utf-8','Selected-study comparison'));actions.append(exportButton);
  dialog.append(head,intro,el('p','Scroll across the table to read each selected study.','hint'),scroller,actions);
```

```
  dialog.addEventListener('close',()=>dialog.remove());document.body.append(dialog);dialog.showModal();
}
function renderEvidence() {
  const g = group();
  const branch = $("#branch").value,
    type = $("#study-type").value,
    relation = $("#relation").value,
    year = Number($("#year").value) || 0;
  const target=statementReadingTarget(g);
  const all = groupEvidence().filter(e=>!target||(e.historical_contexts||[]).some(context=>context.claim_id===target.id));
  const rows = all.filter((e) => (!branch || e.taxon_id === branch) && (!type || e.study.types.includes(type)) && (!relation ||
(e.relation.reported_finding?.direction||'NO_RESULT_STATEMENT') === relation) && (!year || Number(e.paper.year) >= year));
  renderStatementReading(g,target,rows);
  const topicIds=target?[...new Set(target.conditions.map(condition=>condition.id))]:(g?.conditions||[]);
  const conditions = topicIds.map((c) => human(c.replace("condition:", "")));
  $("#modern-question").textContent = !g?.branches.length?'No candidate, so no modern question is asked.':target?'Reading question: do the studies\u2019 species,
preparations and measured outcomes address this exact source statement? Check each candidate separately.':`Modern question, asked separately for each candidate: does
 a specified preparation help ${conditions.join(", ") || "the recorded conditions"} in humans? A historical use is not evidence that it does.`;
  $('#evidence-count').textContent=new Set(rows.map(e=>e.paper.id)).size+' publications \xb7 '+rows.length+(target?' links to this statement':' links to historical
topics');
  const blocking = $("#blocking");
  blocking.replaceChildren();
  if (rows.length) {
    const counts = {};
    for (const e of rows) {
      const k = blockingKey(e);
      counts[k] = (counts[k] || 0) + 1;
    }
    blocking.append(el("span", "Why records stop short:"));
    for (const [k, n] of Object.entries(counts).sort((a, b) => b[1] - a[1])) blocking.append(el("span", `${BLOCKING[k] || human(k)} ${n}`));
  }
  const conflicts = $("#conflicts");
  conflicts.replaceChildren();
  for (const c of data.conflicts.filter((c) => rows.some(e=>e.taxon_id===c.taxon_id&&e.condition_id===c.condition_id) && ["MIXED",
"POTENTIAL_CONFLICT_REQUIRES_COMPARABILITY_REVIEW"].includes(c.state)))
    conflicts.append(el("div", `${taxonName(c.taxon_id)} \xb7 ${human(c.condition_id.replace("condition:", ""))}: ${human(c.state)}. ${c.limitation}`, "conflict"));
  const box = $("#evidence");
  box.replaceChildren();
  if(!type&&!relation&&!year){
    for(const readingSet of data.study_reading_sets||[]){
      if(g?.branches.some(b=>readingSet.taxon_ids.includes(b.taxon_id))&&(!branch||readingSet.taxon_ids.includes(branch))&&topicIds.some(c=>readingSet.condition_ids.
includes(c))&&(!target||all.length))box.append(trialReadingComparison(readingSet));
    }
  }
  if (!rows.length)
    box.append(el("p", all.length ? "No record matches these filters." : target?"No stored publication links to this exact statement in this analysis. Statements
outside recorded uses do not start a literature question. Switch to the chapter overview or analyze again if this saved record lacks statement links.":"No evidence
for this name in the local snapshot. That is not evidence of no effect.", "empty"));
  // Keep candidate identities separate, while letting the researcher read each paper once.
  for (const b of g?.branches || []) {
    const mine=rows.filter(e=>e.taxon_id===b.taxon_id);
    if(!mine.length)continue;
    const papers=new Map();for(const e of mine){if(!papers.has(e.paper.id))papers.set(e.paper.id,[]);papers.get(e.paper.id).push(e);}
    const section=el('section',undefined,'branch-group'),head=el('h3');
    head.append(el('i',b.name),document.createTextNode(' \xb7 '+papers.size+' publication'+(papers.size===1?'':'s')));section.append(head);
    [...papers.values()].forEach((links,i)=>{
      const card=evidenceCard(links[0]);
      if(links.length>1){const related=el('details',undefined,'linked-conditions');const topics=[...new
```

```
Set(links.slice(1).map(e=>human(e.condition_id.replace('condition:',''))))];related.append(el('summary','Also linked to '+topics.join(', ')));for(const e of links.slice(1))related.append(evidenceCard(e));card.append(related);}
      if(i>=3)card.hidden=true;section.append(card);
    });
    if(papers.size>3){const more=el('button','Read '+(papers.size-3)+' more publications','quiet more');more.addEventListener('click',()=>{for(const card of section.querySelectorAll(':scope > .evidence-item[hidden]'))card.hidden=false;more.remove();});section.append(more);}box.append(section);
  }
  renderStudyComparisonSelection();
}

function sourceStudyComparison(e){
  const box=el('details',undefined,'study-source-comparison');
  const target=statementReadingTarget(),contexts=(e.historical_contexts||[]).filter(context=>!target||context.claim_id===target.id),study=e.study,attribution=study.preparation_attribution;
  box.append(el('summary','Compare historical wording and study intervention'+(contexts.length?' \xb7 '+plural(contexts.length,'statement'):'')));
  if(!contexts.length){
    box.append(el('p','This saved analysis has no statement-level comparison. Analyze the source again to link exact original statements and candidate-specific preparation wording.','hint'));
    return box;
  }
  box.open=!target&&contexts.length===1;
  box.append(el('p','The topic mapping is a search aid. The original condition wording is not treated as an equivalent modern diagnosis.','hint'));
  const grid=el('div',undefined,'source-study-grid'),source=el('section'),modern=el('section');
  source.append(el('h4','Historical source'));
  const prioritized=[...contexts].sort((a,b)=>{
    const priority=context=>(libraryFocus?.source_id===data.source.id&&libraryFocus.statement.start===context.support.start?4:0)+(context.preparation.length?1:0);
    return priority(b)-priority(a);
  });
  const additional=el('details',undefined,'more-source-statements');
  additional.append(el('summary',plural(contexts.length-1,'more historical statement')+' linked to this topic'));
  for(const [index,context] of prioritized.entries()){
    const item=el('div',undefined,'source-study-statement');
    item.append(el('blockquote',context.support.text));
    const wording=(context.condition_wording||[]).map(c=>c.original).filter(Boolean);
    const describe=field=>[...new Set((context[field]||[]).map(x=>x.text))].join(', ')||'not stated';
    item.append(el('p','Original topic wording: '+[...new Set(wording)].join(', '),'hint'),el('p','Part: '+describe('part')+' \xb7 Preparation: '+describe('preparation')+' \xb7 Route: '+describe('route'),'hint'));
    const qualifier=statementQualifier(context);if(qualifier)item.append(el('p',qualifier,'hint'));
    const comparison=context.preparation_comparison;
    if(comparison)item.append(el('p','Preparation comparison: '+comparison.detail,'source-study-assessment'));
    const indication=context.indication_comparison;
    if(indication&&(indication.historical_wording.length||indication.study_wording.length)){
      const wording=spans=>[...new Map(spans.map(span=>[span.text.toLowerCase(),span.text])).values()].join(', ')||'Not established by this word screen';
      const reading=el('details',undefined,'indication-comparison');
      reading.append(el('summary',indication.status==='DIFFERENT_EXPLICIT_SITE_WORDING'?'Different site words \xb7 compare the actual indications':'Compare indication and site wording'));
      reading.open=indication.status==='DIFFERENT_EXPLICIT_SITE_WORDING';
      const fields=el('dl',undefined,'study-wording');fields.append(el('dt','Words in this historical statement'),el('dd',wording(indication.historical_wording)),el('dt',indication.study_aim?'Words in the study title / aim':'Words in the study title / abstract'),el('dd',wording(indication.study_wording)));
      reading.append(fields,el('p',indication.detail,'source-study-assessment'));
      if(indication.study_aim)reading.append(el('p','Stated study aim \xb7 original abstract wording','quote-label'),el('blockquote',indication.study_aim.text),link('Original publication \u2197',e.paper.url));
      else reading.append(el('p','Study title: '+cleanTitle(e.paper.title)),el('p','No explicit aim sentence was found; the site words also include abstract context.','hint'));
      const scope=el('details',undefined,'indication-screen-scope');scope.append(el('summary','How these words were compared'),el('p',indication.scope),el('p','Literal word groups, not an anatomical ontology, diagnosis or outcome-equivalence decision. Negation and grammatical relations still require source reading.','hint'));
```

```
      reading.append(scope);item.append(reading);
    }
    const read=el('button','Read this statement in the full source','text-action');
    read.type='button';
    read.addEventListener('click',()=>{
      sourceEvidenceFocus={nameKey:selected,taxonId:e.taxon_id,paperId:e.paper.id};
      sourceFocusOrigin='evidence';libraryFocus={source_id:data.source.id,statement:context.support};renderSource();
      requestAnimationFrame(()=>{
        $('#source-focus')?.scrollIntoView({behavior:'auto',block:'start'});
        $('#original').focus({preventScroll:true});
      });
    });
    item.append(read);(index===0?source:additional).append(item);
  }
  if(contexts.length>1)source.append(additional);
  modern.append(el('h4','Modern study'),el('p','Candidate being checked: '+taxonName(e.taxon_id),'hint'));
  const fields=el('dl',undefined,'study-wording');
  fields.append(el('dt','Study title / question'),el('dd',cleanTitle(e.paper.title)));
  for(const [label,key] of [['Population wording','population'],['Preparation wording','intervention_preparation'],['Comparator wording','comparator']]){
    const comparison=e.comparability?.checks.find(c=>c.check==='comparator');
    const value=key==='comparator'&&comparison?.span?.source_kind==='FULL_TEXT_METHODS'?'Methods comparison / design wording:
\u201c'+comparison.span.text+'\u201d':study.pico_source_spans?.[key]?.text||'Not established in the abstract';
    fields.append(el('dt',label),el('dd',value));
  }
  modern.append(fields);
  const methodIds=new Set((e.comparability?.checks||[]).map(c=>c.span?.source_kind?.startsWith('FULL_TEXT_METHODS')?c.span.source_id:null).filter(Boolean));
  for(const span of study.preparation_attribution?.forms||[])if(span.source_id)methodIds.add(span.source_id);
  for(const id of study.preparation_attribution?.methods_context_ids||[])methodIds.add(id);
  const methods=(e.paper.methods_contexts||[]).filter(context=>methodIds.has(context.id));
  if(methods.length){
    const excerpts=el('details',undefined,'study-methods-source');
    excerpts.append(el('summary','Read the Methods behind the species, design and preparation checks'));
    for(const context of methods)excerpts.append(el('h4',context.section),el('blockquote',context.text),link('Original Methods section
\u2197',context.url),el('p',context.licence+' \xb7 '+(context.source_kind==='FULL_TEXT_METHODS_TABLE'?'Original table cells transcribed with |
separators':'Whitespace-normalized JATS paragraph')+' \xb7 '+context.id,'hint'));
    modern.append(excerpts);
    const comparator=e.comparability.checks.find(c=>c.check==='comparator');
    if(comparator?.span?.source_kind==='FULL_TEXT_METHODS')modern.append(el('p',comparator.detail,'source-study-assessment'));
  }
  const otherMethods=(e.paper.methods_contexts||[]).filter(context=>!methodIds.has(context.id));
  if(otherMethods.length){
    const excerpts=el('details',undefined,'study-methods-source');excerpts.append(el('summary','Read '+plural(otherMethods.length,'other stored Methods excerpt')));
    for(const context of otherMethods)excerpts.append(el('h4',context.section),el('blockquote',context.text),link('Original Methods section
\u2197',context.url),el('p',context.licence+' \xb7 '+context.id,'hint'));
    modern.append(excerpts);
  }
  if(attribution){
    if(attribution.scientific_grounding_scope==='FULL_TEXT_METHODS')modern.append(el('p','The scientific name is grounded in the Methods excerpt. The abstract\u2019s
 nearby preparation wording still requires intervention review.','hint'));
    if(attribution.status==='COMMON_NAME_ONLY_PREPARATION_WORDING')modern.append(el('p','Preparation wording is near a common plant name. The study species remains
unconfirmed for this candidate.','hint'));
    if(attribution.status==='MULTIPLE_CANDIDATE_FORMS')modern.append(el('p','Several preparation forms are linked to this species. A single intervention has not been
 established.','hint'));
    else if(attribution.status==='CANDIDATE_PREPARATION_UNRESOLVED')modern.append(el('p','The abstract\u2019s preparation wording is not assigned to this candidate
species.','hint'));
    if(attribution.forms.length){
      const form=attribution.forms.find(f=>f.start===study.pico_source_spans?.intervention_preparation?.start)||attribution.forms[0];
      modern.append(el('p',form.source_kind==='FULL_TEXT_METHODS_TABLE'?'Candidate formulation row \xb7 original Methods table:':'Candidate-linked wording in
```

```
context:','quote-label'),el('blockquote',form.context.text));
    }
    if(attribution.forms.length>1||attribution.other_preparation_wording.length){
      const signals=el('details');signals.append(el('summary','Inspect all preparation wording'));
      for(const form of attribution.forms)signals.append(el('p','Near '+form.candidate_wording+': '+form.text),el('blockquote',form.context.text));
      for(const form of attribution.other_preparation_wording){signals.append(el('p','Other wording, not assigned to this candidate:
'+form.text,'hint'));if(form.context)signals.append(el('blockquote',form.context.text));}
      modern.append(signals);
    }
    modern.append(el('p','Nearby wording is a lexical reading aid. It does not establish the administered intervention, isolate an ingredient\u2019s effect, or show
equivalent dose and preparation.','hint'));
  }else modern.append(el('p','This older analysis did not assign preparation wording to a candidate species.','hint'));
  grid.append(source,modern);box.append(grid);return box;
}

function evidenceCard(e) {
  const card = el("article", undefined, "evidence-item");
  card.dataset.paper=e.paper.id;card.dataset.taxon=e.taxon_id;card.dataset.condition=e.condition_id;
  const top = el("div", undefined, "row");
  top.append(pill(e.relation.relation==='INSUFFICIENT'?(e.comparability?.first_blocking_check?'Limited applicability':'Applicability
unresolved'):human(e.relation.relation), relationTone(e.relation.relation)), el("span", `${human(e.condition_id.replace("condition:", ""))} \xb7
${e.study.types.map(human).join(", ")}`, "hint"));
  const title = el("h3");
  title.append(link(cleanTitle(e.paper.title), e.paper.url));
  card.append(top, title, el('p', [e.paper.year,e.paper.licence].filter(Boolean).join(' \xb7 '), 'meta'));
  const comparisonActions=el('div',undefined,'row study-comparison-actions'),choose=el('button','Select for comparison','quiet'),compare=el('button','Compare
selected','quiet');
  choose.type='button';choose.dataset.studyChoice=JSON.stringify([e.taxon_id,e.paper.id]);choose.setAttribute('aria-label','Select or remove
'+cleanTitle(e.paper.title)+' for '+taxonName(e.taxon_id)+' in the study comparison');choose.addEventListener('click',()=>toggleStudyComparison(e));
  compare.type='button';compare.dataset.openStudyComparison='';compare.hidden=true;compare.addEventListener('click',openStudyComparison);comparisonActions.append(cho
ose,compare);card.append(comparisonActions);
  const measurements=studyMeasurementReadings(e);if(measurements.length)card.append(studyMeasurementPanel(e,measurements));
  if(e.paper.reported_results?.length){
    const results=el('section',undefined,'reported-results');
    results.append(el('h4','What the study reports'));
    for(const context of e.paper.reported_results){
      results.append(el('blockquote',context.text),link('Original Results section \u2197',context.url),el('p',context.licence+' \xb7 '+context.section,'hint'));
    }
    const uncontrolled=e.comparability?.checks.some(c=>c.check==='comparator'&&c.status==='FAIL'&&c.span?.source_kind==='FULL_TEXT_METHODS');
    results.append(el('p',uncontrolled?'These are reported changes during an uncontrolled study. They do not isolate a treatment effect.':'These results concern this
 study\u2019s intervention and population. Compare them with the source statement below.','hint'));
    for(const note of e.relation.reported_finding?.comparison_notes||[])results.append(el('p',note,'source-study-assessment'));
    card.append(results);
  }
  const checks = el("ul", undefined, "checks");
  for (const c of e.comparability?.checks || []) {
    const li = el("li", `${CHECK_LABEL[c.check] || c.check}`, c.status);
    li.title = c.detail;
    li.setAttribute("aria-label", `${CHECK_LABEL[c.check] || c.check}: ${human(c.status)}. ${c.detail}`);
    checks.append(li);
  }
  card.append(checks);
  card.append(sourceStudyComparison(e));
  const open = (e.comparability?.checks || []).filter((c) => c.status === "PARTIAL" || c.status === "UNKNOWN");
  if (!e.comparability?.first_blocking_check && open.length)
    card.append(el('p', 'Still unclear: '+open.map(c=>(CHECK_LABEL[c.check]||c.check).toLowerCase()).join(', ')+'. Read the full study before carrying this finding
over to the historical preparation.', 'hint'));
  if (e.relation.pre_comparability_relation)
```

```
    card.append(el('p', 'This finding cannot be carried over directly: the '+(CHECK_LABEL[e.comparability.first_blocking_check]?.toLowerCase()||e.comparability.first_blocking_check)+' is different or remains unclear.', 'hint'));
  const finding = e.relation.reported_finding;
  if(finding?.candidate_scope_note)card.append(el('p',finding.candidate_scope_note,'hint'));
  if (finding && finding.direction !== "NO_RESULT_STATEMENT") card.append(el("p", finding.comparison_notes?.length?'A treatment-versus-comparator direction could not be established from these statements. Read the comparison notes and source findings.':`Automated abstract reading: ${human(finding.direction)}. Read the quoted findings for the study's own population and preparation.`, "hint"));
  // Say what the quoted sentence is: a result statement, or only a mention of the condition.
  const isResult = finding && finding.direction !== "NO_RESULT_STATEMENT" && finding.supporting_text === e.relation.supporting_text;
  if(!e.paper.reported_results?.length){
    if (finding?.statements?.length) {
      card.append(el('p', 'Endpoint-specific result statements:', 'quote-label'));
      for (const statement of finding.statements) card.append(el('p',human(statement.direction),'quote-label'),el('blockquote',statement.supporting_text));
    } else {
      if (e.relation.supporting_text) card.append(el("p", isResult ? "Result sentence:" : "Sentence that mentions the condition (not a reported result):", "quote-label"));
      card.append(el("blockquote", e.relation.supporting_text || "No sentence in the abstract mentions the condition."));
    }
  }
  const d = el("details",undefined,"study-context");
  d.append(el('summary', 'Study context, source and full abstract'), el('p', e.relation.rationale),el('p', 'Publication '+e.paper.id+(e.paper.pmid?' \xb7 PMID '+e.paper.pmid:''), 'hint'));
  if(finding?.unassigned_statements?.length){
    const unassigned=el('div',undefined,'source-study-assessment');
    unassigned.append(el('strong','Findings not assigned to this candidate'));
    for(const statement of finding.unassigned_statements)unassigned.append(el('blockquote',statement.supporting_text));
    unassigned.append(el('p','These source findings may concern another intervention. They do not contribute a direction for this candidate.','hint'));
    d.append(unassigned);
  }
  const list = el("ul", undefined, "why");
  for (const c of e.comparability?.checks || []) list.append(el("li", `${CHECK_LABEL[c.check] || c.check} \u2014 ${human(c.status)}: ${c.detail}`));
  d.append(list, el("p", "Question: " + e.modern_hypothesis.question), el("p", `Citation: ${human(e.citation.status)}. The identifier and quoted sentence match the stored record; that does not prove the paper supports the claim.`, "hint"));
  if (e.paper.doi) d.append(link("DOI " + e.paper.doi, "https://doi.org/" + e.paper.doi));
  d.append(el("p", "Authors: " + (e.paper.authors || "not supplied"), "hint"));
  if (e.study.local_model?.used) d.append(el("p", `Local design classifier proposes \u201c${human(e.study.local_model.label)}\u201d. A proposal for review, not a verified label.`, "hint"));
  d.append(el("p", e.paper.abstract, "abstract"));
  if (!recorded) d.append(outcomeForm(e));
  if (e.safety_signals.length) {
    d.append(el("strong", "Safety or interaction wording in the abstract"));
    for (const s of e.safety_signals) d.append(el("blockquote", s.supporting_text));
    d.append(el("p", "Source wording only; not a safety assessment for any person.", "hint"));
  }
  if (e.compound_co_mentions.length) {
    const p = el("p", "Compounds mentioned (not proof the plant contains them): ", "hint");
    for (const c of e.compound_co_mentions) p.append(link(`${c.name} \xb7 CID ${c.cid}`, c.url), document.createTextNode(" "));
    d.append(p);
    const leads = (data.compound_assay_leads || []).filter((a) => a.linked_evidence.some((x) => x.evidence_id === e.id));
    if (leads.length) {
      const chemistry = el("details", undefined, "chemistry");
      chemistry.append(el("summary", `Compound assay leads (${leads.length})`), el("p", "These molecules are mentioned in the publication. The database join does not show that this plant or historical preparation contains them. Assay measurements are not clinical effects.", "hint"));
      for (const a of leads) {
        const item = el("div", undefined, "reading-scope");
        item.append(el("strong", `${a.compound} \u2192 ${a.target.pref_name}`), el("p", `${a.standard_type} ${a.standard_relation} ${a.standard_value} ${a.standard_units} \xb7 ${a.target.organism} \xb7 ${human(a.target.target_type)}`), el("p", a.assay_description, "hint"), link("ChEMBL assay record \u2197", a.url), el("p", `${a.document_id} \xb7 ${a.assay_id} \xb7 ${a.licence}`, "hint"));
```

```
        chemistry.append(item);
      }
      d.append(chemistry);
    }
  }
  const actions = el("div", undefined, "row");
  const raw = el("button", "Show provenance record", "quiet");
  raw.type = "button";
  raw.addEventListener("click", () => {
    inspect(e, "Publication evidence");
    showView("graph");
  });
  actions.append(raw);
  if (!recorded) {
    const v = el("button", "Re-check metadata at the source", "quiet");
    v.type = "button";
    v.addEventListener("click", async () => {
      v.disabled = true;
      try {
        const r = await api(`/api/citations/${e.paper.id}/verify-live`);
        status(`Source metadata ${r.metadata_match ? "matches" : "does NOT match"} the stored record for ${e.paper.id}. Entailment is not checked by this lookup.`,
    !r.metadata_match);
      } catch (error) {
        status(error.message, true);
      } finally {
        v.disabled = false;
      }
    });
    actions.append(v);
  }
  d.append(actions, reviewForm("evidence", e.id, "Does this record bear on the modern question?"));
  card.append(d);
  return card;
}

/* ---------- Reviews ---------- */
function outcomeForm(e) {
  const d = el("details", undefined, "outcome-tool");
  d.append(el("summary", "Explore a reported outcome with the local model"), el("p", "Specify the study's actual comparison. The model proposes increased, decreased
    or no significant difference for that outcome. It does not change the evidence verdict.", "hint"));
  const form = el("form"), fields = {};
  for (const [key, label, value] of [["intervention", "Intervention studied", ""], ["comparator", "Comparator", ""], ["outcome", "Outcome measured", ""],
    ["evidence", "Exact evidence from this abstract", e.relation.supporting_text || ""]]) {
    const l = el("label", label), input = el(key === "evidence" ? "textarea" : "input");
    input.required = true;
    input.maxLength = key === "evidence" ? 12000 : 500;
    input.minLength = key === "evidence" ? 15 : 2;
    input.value = value;
    if (key === "evidence") input.rows = 4;
    l.append(input);
    form.append(l);
    fields[key] = input;
  }
  const button = el("button", "Propose outcome direction");
  button.type = "submit";
  const result = el("p", "", "hint");
  result.setAttribute("aria-live", "polite");
  form.addEventListener("submit", async (event) => {
```

```
    event.preventDefault();
    button.disabled = true;
    try {
      const response = await api('/api/research/outcome-direction', {method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({run_id: data.run_id, paper_id: e.paper.id, ...Object.fromEntries(Object.entries(fields).map(([key, input]) => [key, input.value.trim()]))})});
      result.textContent = `${response.direction ? "Model proposal: " + human(response.direction) + ". " : ""}${response.scope}`;
    } catch (error) { result.textContent = error.message; }
    finally { button.disabled = false; }
  });
  form.append(button, result);
  d.append(form);
  return d;
}

function reviewFor(kind, id) {
  return (data.researcher_reviews || []).find((r) => r.target_kind === kind && r.target_id === id);
}
function reviewForm(kind, id, question) {
  const wrap = el("div", undefined, "review-form");
  const existing = reviewFor(kind, id);
  if (existing) {
    const p = el("p", undefined, "review-saved");
    p.append(pill("Your review: " + human(existing.decision), existing.decision === "ACCEPT" ? "ok" : existing.decision === "REJECT" ? "bad" : "warn"), document.createTextNode(existing.reason));
    wrap.append(p);
    if(existing.carried_forward)wrap.append(el('p','Retained from an earlier analysis of this exact source and research question.','hint'));
  }
  if (recorded) {
    if (!existing) wrap.append(el("p", "Reviews can be recorded in the local application.", "hint"));
    return wrap;
  }
  const d = el("details");
  d.append(el("summary", existing ? "Change review" : "Record your judgement"));
  const fs = el("fieldset");
  fs.append(el("legend", question));
  const name = `review-${kind}-${id}`.replace(/[^\w-]/g, "_");
  for (const [value, label] of [["ACCEPT", "Accept"], ["REJECT", "Reject"], ["INSUFFICIENT_INFORMATION", "Can't tell"]]) {
    const l = el("label", undefined, "choice");
    const r = el("input");
    r.type = "radio";
    r.name = name;
    r.value = value;
    r.checked = existing?.decision === value;
    l.append(r, document.createTextNode(label));
    fs.append(l);
  }
  const reasonLabel = el("label", "Reason (kept with exports)");
  const reason = el("input");
  reason.type = "text";
  reason.maxLength = 1000;
  reason.value = existing?.reason || "";
  reasonLabel.append(reason);
  const save = el("button", "Save review");
  save.type = "button";
  save.addEventListener("click", async () => {
    const decision = fs.querySelector("input:checked")?.value;
    if (!decision || reason.value.trim().length < 3) {
      status("Choose a decision and give a short reason.", true);
```

```
      return;
    }
    save.disabled = true;
    const reviewRunId=data.run_id;
    try {
      const r = await api(`/api/analyses/${reviewRunId}/reviews`, { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ target_kind: kind, target_id: id, decision, reason: reason.value.trim() }) });
      if(data?.run_id!==reviewRunId)return;
      data.researcher_reviews = r.reviews;
      data.research_brief = r.research_brief;
      status("Review saved. It is stored apart from the automated output.");
      renderInspector();
    } catch (error) {
      status(error.message, true);
      save.disabled = false;
    }
  });
  fs.append(reasonLabel, save);
  d.append(fs);
  wrap.append(d);
  return wrap;
}
function renderReviews() {
  const box = $("#reviews");
  box.replaceChildren();
  const reviews = data.researcher_reviews || [];
  if (!reviews.length) box.append(el("p", "No judgements recorded for this analysis yet. Record them from the Identity and Evidence tabs.", "empty"));
  for (const r of reviews) {
    const row = el("div", undefined, "review-row");
    const target = r.target_kind === "identity" ? `Identity: \u201c${r.target_id.split("::")[0]}\u201d \u2192 ${taxonName(r.target_id.split("::")[1])}` : `Evidence: ${cleanTitle(data.evidence.find((e) => e.id === r.target_id)?.paper.title) || r.target_id}`;
    const left = el("div");
    left.append(pill(human(r.decision), r.decision === "ACCEPT" ? "ok" : r.decision === "REJECT" ? "bad" : "warn"), el("strong", target), el("p", r.reason, "hint"), el("p", `${r.reviewer} \xb7 ${r.created_at.slice(0, 16).replace("T", " ")} UTC`, "hint"));
    if(r.carried_forward)left.append(el('p','Retained from your earlier analysis of the same source and target.','hint'));
    row.append(left);
    if (!recorded) {
      const del = el("button", "Remove judgement", "quiet");
      del.title='Remove this judgement for the same source and target across evidence rankings.';
      del.type = "button";
      del.addEventListener("click", async () => {
        const reviewRunId=data.run_id;
        try {
          const res = await api(`/api/analyses/${reviewRunId}/reviews/${r.target_kind}/${encodeURIComponent(r.target_id)}`, { method: "DELETE" });
          if(data?.run_id!==reviewRunId)return;
          data.researcher_reviews = res.reviews;
          data.research_brief = res.research_brief;
          renderInspector();
        } catch (error) {
          status(error.message, true);
        }
      });
      row.append(del);
    }
    box.append(row);
  }
  const x = $("#export-links");
  x.replaceChildren();
```

```
  for (const [f, label] of [["brief", "Research brief (Markdown)"], ["json", "Full record (JSON)"], ["csv", "Evidence table (CSV)"], ["bibtex", "Citations (BibTeX)"], ["graph", "Graph (JSON)"]]) {
    const a = el("a", label);
    a.href = `/api/exports/${data.run_id}?format=${f}`;
    a.setAttribute("download", "");
    const sourceTitle=data.source.title,runId=data.run_id;
    a.addEventListener('click',async event=>{
      event.preventDefault();
      try{const response=await fetch(a.href);if(!response.ok)throw new Error('Export is unavailable');const text=await response.text();const extension={brief:'md',json:'json',csv:'csv',bibtex:'bib',graph:'json'}[f];offerExport(text,'foliatrace-'+runId+'.'+extension,response.headers.get('Content-Type')||'text/plain;charset=utf-8',label+' \xb7 '+sourceTitle);}
      catch(error){status(error.message,true);}
    });
    x.append(a);
  }
}

/* ---------- Graph ---------- */
const colors = {
  HistoricalSource: "#e3cc9c",
  PlantMention: "#efe0bd",
  HistoricalClaim: "#f1e6d0",
  Condition: "#eedcae",
  PlantPart: "#efe6cf",
  Preparation: "#efe6cf",
  Route: "#efe6cf",
  ModernTaxon: "#c4dfc6",
  ModernHypothesis: "#dbe8d8",
  Publication: "#c6d7ea",
  Compound: "#dad0ea",
  SafetySignal: "#f1cfc8",
};
function svgEl(tag, attrs = {}) {
  const e = document.createElementNS("http://www.w3.org/2000/svg", tag);
  for (const [k, v] of Object.entries(attrs)) e.setAttribute(k, v);
  return e;
}
function inspect(value, title = "Source provenance") {
  const box = $("#inspector");
  box.replaceChildren(el("h3", title));
  if (value.url || value.paper?.url) box.append(link("Original source \u2197", value.url || value.paper.url));
  box.append(el("pre", JSON.stringify(value, null, 2)));
}
async function renderGraph() {
  if (!data) return;
  const legend = $("#legend");
  legend.replaceChildren();
  for (const [k, c] of Object.entries(colors).filter(([k]) => data.graph.nodes.some((n) => n.kind === k))) {
    const s = el("span");
    const i = el("i");
    i.style.background = c;
    s.append(i, document.createTextNode(k.replace(/([a-z])([A-Z])/g, "$1 $2")));
    legend.append(s);
  }
  const params = new URLSearchParams({ limit: "80" });
  if (graphFocus) params.set("focus", graphFocus);
  if ($("#edge-filter").value) params.set("predicate", $("#edge-filter").value);
  let g;
```

```
  try {
    g = await api(`/api/graph/${data.run_id}?${params}`);
  } catch (error) {
    status(error.message, true);
    return;
  }
  const svg = $("#graph");
  svg.replaceChildren();
  const nodes = g.nodes.slice(0, 45),
    ids = new Set(nodes.map((n) => n.id)),
    positions = new Map();
  [["HistoricalSource", "PlantMention"], ["HistoricalClaim", "Condition", "PlantPart", "Preparation", "Route"], ["ModernTaxon", "ModernHypothesis"], ["Publication", "Compound", "SafetySignal", "AssayMeasurement", "MolecularTarget"]].forEach((kinds, col) => {
    const ns = nodes.filter((n) => kinds.includes(n.kind));
    ns.forEach((n, i) => positions.set(n.id, [col * 235 + 14, 32 + i * Math.min(440 / Math.max(ns.length, 1), 70)]));
  });
  const edges = g.edges.filter((e) => ids.has(e.subject) && ids.has(e.object));
  for (const edge of edges) {
    const a = positions.get(edge.subject),
      b = positions.get(edge.object);
    if (!a || !b) continue;
    const line = svgEl("line", { x1: a[0] + 88, y1: a[1] + 18, x2: b[0] + 88, y2: b[1] + 18, tabindex: "0", role: "button", "aria-label": human(edge.predicate) + "; inspect source" }),
      action = () => inspect(edge.provenance, human(edge.predicate));
    line.addEventListener("click", action);
    line.addEventListener("keydown", (event) => {
      if (["Enter", " "].includes(event.key)) {
        event.preventDefault();
        action();
      }
    });
    svg.append(line);
  }
  for (const n of nodes) {
    const pos = positions.get(n.id);
    if (!pos) continue;
    const node = svgEl("g", { class: "node", tabindex: "0", role: "button", "aria-label": n.kind + ": " + n.label });
    node.append(svgEl("rect", { x: pos[0], y: pos[1], width: 180, height: 37, rx: 5, fill: colors[n.kind] || "#e5e5e5" }));
    const text = svgEl("text", { x: pos[0] + 8, y: pos[1] + 23 });
    text.textContent = n.label.slice(0, 25) + (n.label.length > 25 ? "\u2026" : "");
    node.append(text);
    const action = () => {
      graphFocus = n.id;
      inspect(n, n.kind);
      renderGraph();
    };
    node.addEventListener("click", action);
    node.addEventListener("keydown", (event) => {
      if (["Enter", " "].includes(event.key)) {
        event.preventDefault();
        action();
      }
    });
    svg.append(node);
  }
  const list = $("#graph-list");
  list.replaceChildren(el("p", `Showing ${nodes.length} nodes and ${edges.length} lines from a bounded neighbourhood.`, "hint"));
  for (const edge of g.edges) {
```

```
    const b = el("button", `${edge.subject} \u2014 ${human(edge.predicate)} \u2014 ${edge.object}`);
    b.type = "button";
    b.addEventListener("click", () => inspect(edge.provenance, human(edge.predicate)));
    list.append(b);
  }
}
function setupGraphFilter() {
  const ef = $("#edge-filter");
  ef.replaceChildren(option("", "All relations"));
  for (const p of [...new Set(data.graph.edges.map((e) => e.predicate))].sort()) ef.append(option(p, human(p)));
}

/* ---------- Evaluation ---------- */
const TIER = {
  SOURCE_ASSERTED_REFERENCE: ["warn", "Historical author's own Latin name; not expert gold"],
  EXTERNAL_HUMAN_GOLD: ["ok", "Published human labels for a narrow clinical task"],
  SILVER: ["neutral", "Database-derived labels"],
  SILVER_SOURCE_RECORDS: ["neutral", "Consistency with stored source records"],
  WEAK: ["neutral", "Weak labels from title matching or headings"],
  SYNTHETIC: ["neutral", "Constructed perturbations"],
  BEHAVIOURAL_COUNT: ["neutral", "Counts of system behaviour, not accuracy"],
};
function productEvaluation(report, known, books, outside) {
  const section = el("section", undefined, "metric-group");
  const c = report.coverage, t = report.identity_transfer, s = report.statement_audit;
  section.append(el("h3", "Whole-product measurements"), el("p", report.scope, "hint"));
  const wrap = el("div", undefined, "table-wrap"), table = el("table"), head = el("tr");
  head.append(el("th", "Measure"), el("th", "Result"));
  table.append(head);
  const rows = [
    ["Chapters whose plant has a modern candidate", `${c.chapters.subject_resolved} of ${c.chapters.total}`],
    ["Chapters whose plant reaches modern papers", `${c.chapters.subject_has_papers} of ${c.chapters.total}`],
    ["Chapters whose plant reaches a species-named human study", `${c.chapters.subject_has_human_study} of ${c.chapters.total}`],
    ["Author-asserted identity references: selected Latin clue masked, chapter context retained (development rerun)", `${(100 *
t.without_historical_lexicon.top1_of_all).toFixed(1)}% without the lexicon, ${(100 * t.with_historical_lexicon.top1_of_all).toFixed(1)}% with it; common-name lookup
baseline ${(100 * t.baseline_gbif_vernacular.top1_of_all).toFixed(1)}%. This is not expert gold, a name-only test, or a mask of every scientific name in the
chapter.`],
    ["Identity recovery at genus level (first answer)", `${(100 * t.with_historical_lexicon.genus_top1_of_all).toFixed(1)}%`],
    ["Outside texts, development panel: plants recovered", `${report.external_texts_final.plants_found} of ${report.external_texts_final.plants_expected} across
${report.external_texts_final.passages} passages`],
  ];
  if (known?.by_split?.held_out) {
    const h = known.by_split.held_out;
    rows.push([`Known plant and topic findings, held-out pairs (n = ${h.pairs})`, `human study found ${(100 * h.found_human_study).toFixed(0)}%, direction compatible
 ${(100 * h.compatible).toFixed(0)}%, opposite ${(100 * h.opposite).toFixed(0)}%`]);
  }
  const blind = known?.blind_cochrane_first_run, again = known?.blind_cochrane;
  if (blind) rows.push([`Cochrane conclusions, one blind run with each review hidden (n = ${blind.pairs})`, `human study found ${(100 *
blind.found_human_study).toFixed(0)}%, direction compatible ${(100 * blind.compatible).toFixed(0)}%, exact ${(100 * blind.exact).toFixed(0)}%, opposite ${(100 *
blind.opposite).toFixed(0)}%${again ? `; after the fix it prompted, no longer blind: exact ${(100 * again.exact).toFixed(0)}%, opposite ${(100 *
again.opposite).toFixed(0)}%` : ""}`]);
  const records = outside?.independent_records;
  if (records) rows.push([`Duke database concordance on the displayed map cohort (${records.pairs_read_as_use} pairs; not held out)`, `${(100 *
records.agreement_read_as_use).toFixed(1)}%, against ${(100 * records.agreement_shuffled_mean).toFixed(1)}% shuffled (permutation p ${records.permutation_p <= 0.0001
 ? "< 0.001" : "= " + records.permutation_p}); context-only diagnostic ${typeof records.agreement_read_as_context_only === 'number' ? (100 *
records.agreement_read_as_context_only).toFixed(1) + '%' : 'not measured'}. Duke also informs candidate ranking.`]);
  const committee = outside?.regulator_agreement;
  if (committee) rows.push([`EU herbal committee: does the reading separate well-established from traditional use? (${committee.pairs} pairs, labels read from
```

```
    monographs by rule)`, `AUC ${committee.auc} (95% interval ${committee.auc_95_interval.join(" to ")}), human study found ${(100 *
    committee.well_established_human_study_found).toFixed(0)}% against ${(100 * committee.traditional_human_study_found).toFixed(0)}%. The one blind run gave AUC
    ${committee.first_blind_run.auc}; the later figure follows fixes it prompted`]);
2255   const folk = outside?.folklore_blind;
2256   if (folk) rows.push([`Folklore never opened during development, Folkard 1884 (${folk.entries} entries, one blind run)`, `plant found ${(100 *
    folk.heading_plant_found).toFixed(0)}%; ${(100 * folk.agreement).toFixed(1)}% of ${folk.pairs} use pairs are in Duke's records against ${(100 *
    folk.agreement_shuffled_mean).toFixed(1)}% shuffled`]);
2257   if (outside?.independent_records?.constituent_signal) { const signals = outside.independent_records; rows.push(["Descriptive checks of ranking inputs against Duke
    records", `related species ${signals.related_species_signal.observed} against ${signals.related_species_signal.shuffled_mean} shuffled; constituents
    ${signals.constituent_signal.observed} against ${signals.constituent_signal.shuffled_mean}. These checks involve data also used by ranking; they are not held-out
    validation. Constituents carry no priority weight.`]); }
2258   const review = outside?.expert_review;
2259   if (review) rows.push([`Human expert review: ${review.reviewer}`, `use statements ${(100 * review.use_statements.agreement).toFixed(0)}% (kappa
    ${review.use_statements.cohen_kappa}), plant identity ${(100 * review.plant_identity.agreement).toFixed(0)}%, study direction ${(100 *
    review.study_direction.agreement).toFixed(0)}%`]);
2260   const hausa = books?.tiers?.blind_round3?.["dalziel-1916-hausa"];
2261   if (hausa) rows.push([`Language no rule was tuned on, Hausa vocabulary: species found (n = ${hausa.as_printed.entries}, one blind run)`, `as printed ${(100 *
    hausa.as_printed.reference_among_candidates).toFixed(0)}%; from the Hausa name alone ${(100 * hausa.latin_names_hidden.reference_among_candidates).toFixed(0)}%,
    because no open list holds most of these names`]);
2262   const revisited = books?.tiers?.revisited_after_blind || {};
2263   for (const [label, book] of Object.entries(books?.tiers?.blind || {})) {
2264     const hidden = revisited[label]?.latin_names_hidden;
2265     rows.push([`Book not used in development, ${label}: species found (n = ${book.as_printed.entries})`, `as printed ${(100 *
    book.as_printed.reference_among_candidates).toFixed(0)}% on the first blind run${hidden ? `; Latin names hidden ${(100 *
    hidden.reference_among_candidates).toFixed(0)}% on a later rerun, no longer blind` : ""}`]);
2266   }
2267   for (const [tier, label, unit] of [["blind_round4", "Waring 1897, India: twelve Indian languages per entry", "Indian names"], ["blind_round5", "Flora of the Sudan
    1929: the Arabic name selected by the reader", "the Arabic name"], ["blind_round6", "Henkel 1906, United States: English common names", "common names"]]) {
2268     const first = Object.values(books?.tiers?.[tier] || {})[0];
2269     const later = Object.values(books?.tiers?.[tier.replace("blind", "revisited")] || {})[0];
2270     if (first) rows.push([`Book never opened before its run, ${label} (n = ${first.as_printed.entries})`, `species found as printed ${(100 *
    first.as_printed.reference_among_candidates).toFixed(0)}%; from ${unit} alone ${(100 * first.latin_names_hidden.reference_among_candidates).toFixed(0)}% on the one
    blind run${later ? `, ${(100 * later.latin_names_hidden.reference_among_candidates).toFixed(0)}% after its misses were fixed (no longer blind)` : ""}`]);
2271   }
2272   const foresightReport = outside?.lead_foresight;
2273   const foresight = foresightReport?.protocol_version === 'SOURCE_ONLY_ALL_HUMAN_STUDIES_V2' ? foresightReport.cutoffs?.["2010"] : null;
2274   if (foresight) {
2275     const benefit = foresight.benefit_reported_after;
2276     const interval = Array.isArray(benefit.auc_95_interval) ? benefit.auc_95_interval.join(' to ') : 'not estimable';
2277     const percent = value => value == null ? 'not estimable' : (100*value).toFixed(0)+'%';
2278     rows.push([`Retrospective source-only ranking: dated historical support, no retrieved human study dated before 2010 (n = ${foresight.eligible_pairs}); all
    retrieved human studies checked`, `top tenth: ${percent(benefit.top_decile_rate)} later had a retrieved human study reporting benefit, against
    ${percent(benefit.overall_rate)} overall; AUC ${benefit.auc_history_score ?? 'not estimable'} (95% interval ${interval}),
    ${benefit.auc_history_within_publication_bands ?? 'not estimable'} within bands of retrieved prior publication volume (permutation p
    ${benefit.permutation_p_within_bands ?? 'not estimable'}). Retrieved prior publication volume alone gives ${benefit.auc_prior_publications_only ?? 'not estimable'}.
    No retrieved earlier study does not prove a use was never tested. Retrospective association, not prospective validation or proof of benefit.`]);
2279   }
2280   const firstTemporal = foresightReport?.first_run?.cutoffs?.['2010']?.benefit_reported_after;
2281   if(firstTemporal)rows.push(['Superseded exploratory temporal result, retained for audit',`Original AUC ${firstTemporal.auc_history_score}. Its eligibility used the
     six-study display subset, and auxiliary historical features were not filtered by publication date. It is not the current validation result.`]);
2282   const pasted = outside?.free_text_blind;
2283   if (pasted) {
2284     const [first, current] = [pasted.first_run, pasted.current];
2285     const share = ([hit, total]) => `${hit} of ${total}`;
2286     rows.push([`Text a reviewer pastes in, round 8: ${pasted.texts} short texts in Albanian, Bulgarian, Turkish, Portuguese, Romanian, Spanish, Croatian, 1850s
    German, 1650s English and Balkan field notes; answers written before the run`, `one blind run: right plant leads for ${share(first.names_leading)} names, use read
    for ${share(first.uses)}, ${first.other_leading_names.length} words wrongly read as plants. After the fixes it prompted (a development figure):
    ${share(current.names_leading)}, ${share(current.uses)}, ${current.other_leading_names.length} wrong`]);
```

```
  }
  const fresh = outside?.free_text_blind_round9;
  if (fresh) {
    const [first, current] = [fresh.first_run, fresh.current];
    const share = ([hit, total]) => `${hit} of ${total}`;
    const later = JSON.stringify(current.names_leading) !== JSON.stringify(first.names_leading) || JSON.stringify(current.uses) !== JSON.stringify(first.uses) ? ` After later fixes (a development figure): ${share(current.names_leading)}, ${share(current.uses)}, ${current.other_leading_names.length} wrong` : "";
    rows.push([`Round 9, new languages after the round 8 fixes: ${fresh.texts} texts in Macedonian, Slovenian, Italian, French, Polish, Hungarian, Greek, Czech, Dutch, Serbian, Albanian local names in English field notes, and 1597 English; answers written before the run`, `one blind run: right plant leads for ${share(first.names_leading)} names, use read for ${share(first.uses)}, ${first.other_leading_names.length} words wrongly read as plants.${later}`]);
  }
  const tenth = outside?.free_text_blind_round10;
  if (tenth?.first_run) {
    const [first, current] = [tenth.first_run, tenth.current];
    const share = ([hit, total]) => `${hit} of ${total}`;
    const later = JSON.stringify(current.names_leading) !== JSON.stringify(first.names_leading) || JSON.stringify(current.uses) !== JSON.stringify(first.uses) ? ` After later fixes (a development figure): ${share(current.names_leading)}, ${share(current.uses)}, ${current.other_leading_names.length} wrong` : "";
    rows.push([`Round 10, written before any of its texts was run: ${tenth.texts} texts in Kosovo Albanian, Macedonian, Serbian Cyrillic, Slovenian, Polish, Hungarian, German, Italian, English field notes, 1700s English, Spanish, Turkish, Greek and Romanian`, `one blind run: right plant leads for ${share(first.names_leading)} names, use read for ${share(first.uses)}, ${first.other_leading_names.length} words wrongly read as plants.${later}`]);
  }
  if (s) rows.push(["Statement screen, held-out sample (single AI annotator)", `use precision ${(100 * s.use_precision).toFixed(1)}%, use recall ${(100 * s.use_recall).toFixed(1)}%, topic ${(100 * s.topic_correct).toFixed(0)}%, plant name ${(100 * s.plant_name_correct).toFixed(0)}%, n = ${s.n}`]);
  const audit = outside?.book_statement_audit;
  if (audit) rows.push(["Statement screen on outside books (single AI annotator)", `use precision ${(100 * audit.precision).toFixed(1)}%, estimated recall ${(100 * audit.recall_estimated).toFixed(0)}%, plant name ${(100 * audit.plant_name_correct).toFixed(0)}%, n = 60`]);
  for (const [label, value] of rows) { const tr = el("tr"); tr.append(el("td", label), el("td", value)); table.append(tr); }
  wrap.append(table);
  section.append(wrap);
  return section;
}
function languageName(code) {
  try { return new Intl.DisplayNames(["en"], { type: "language" }).of(code) || code; } catch { return code; }
}
async function loadMetrics() {
  const box = $("#metrics");
  try {
    const m = await benchmarks();
    if (m.product_evaluation) box.append(productEvaluation(m.product_evaluation, m.known_answers, m.unseen_books, m));
    if (m.retrieval_external) {
      const r = m.retrieval_external, section = el('section', undefined, 'metric-group retrieval-metric');
      section.append(el('h3', 'Independent biomedical retrieval benchmark'), el('p', `SciFact: ${r.documents} abstracts, ${r.queries} claims with published human evidence labels. Both supporting and contradicting papers count as relevant.`, 'hint'));
      const wrap = el('div', undefined, 'table-wrap'), table = el('table'), head = el('tr');
      head.append(el('th', 'Method'), el('th', 'Recall@5'), el('th', 'Recall@10'), el('th', 'nDCG@10'), el('th', 'MRR@10'));
      table.append(head);
      for (const [method, values] of Object.entries(r.metrics)) {
        const row = el('tr');
        row.append(el('td', human(method)), ...['recall@5', 'recall@10', 'ndcg@10', 'mrr@10'].map((key) => el('td', values[key].toFixed(3))));
        table.append(row);
      }
      wrap.append(table);
      section.append(wrap, el('p', `Cluster-bootstrap 95% interval for reranking minus BM25 nDCG@10: ${r.rerank_minus_bm25_ndcg10_cluster_bootstrap.interval95.map((n) => n.toFixed(3)).join(' to ')}. ${r.scope}`, 'hint'));
      box.append(section);
    }
    if (m.outcome_model) {
      const learned = el("section", undefined, "metric-group outcome-metric");
```

```
      learned.append(el("h3", "Supervised outcome direction"), el("p", `Published human annotations \xb7 ${m.outcome_model.test_articles} disjoint test articles \xb7 ${m.outcome_model.test.selected_supervised.n} prompts. Given the annotated evidence and comparison; not end-to-end extraction.`, "hint"));
      const wrap = el("div", undefined, "table-wrap"), table = el("table"), head = el("tr");
      head.append(el("th", "Method"), el("th", "Macro-F1"), el("th", "Accuracy, all"), el("th", "Accuracy, answered"), el("th", "Coverage"));
      table.append(head);
      for (const [method, values] of Object.entries(m.outcome_model.test)) {
        const tr = el("tr");
        tr.append(el("td", human(method)), el("td", values.macro_f1.toFixed(3)), el("td", (100 * values.accuracy).toFixed(1) + "%"), el("td", values.accuracy_when_answered == null ? '\u2014' : (100 * values.accuracy_when_answered).toFixed(1) + "%"), el("td", (100 * values.coverage).toFixed(1) + "%"));
        table.append(tr);
      }
      wrap.append(table);
      learned.append(wrap, el("p", `Article-bootstrap 95% interval for macro-F1 gain over lexical rules: ${m.outcome_model.delta_macro_f1_vs_lexical_article_bootstrap_95.map((n) => n.toFixed(3)).join(" to ")}. Validation-only parameter selection. Rules answer fewer questions but are more accurate on those they answer; the learned model's broader coverage is not higher conditional precision.`, "hint"));
      // Keep the main historical-identity results first in the page.
      box.append(learned);
    }
    if (m.status === "NOT_RUN") {
      box.append(el("p", "The benchmark has not been run. No score is asserted.", "empty"));
      return;
    }
    // Group by task and tier: one task can have rows from different label tiers and sample sizes.
    const groups = new Map();
    const tiersOf = new Map();
    for (const row of m.summary ?? []) {
      const key = row.task + "\u0000" + row.tier;
      if (!groups.has(key)) groups.set(key, []);
      groups.get(key).push(row);
      tiersOf.set(row.task, (tiersOf.get(row.task) || new Set()).add(row.tier));
    }
    // Lead with the identity results the tool exists for; the report's own order is chronological.
    const first = (task) => (task.startsWith("Historical identity transfer") ? 0 : 1);
    const archived = [...groups.values()].filter(rows => /Identity propagation|silver/i.test(rows[0].task + ' ' + rows[0].tier));
    const archive = el('details', undefined, 'metric-group');
    archive.append(el('summary', 'Archived component checks \xb7 earlier build'));
    archive.append(el('p', 'These frozen lookup and silver-label checks describe an earlier component snapshot. They do not measure the current product or expert-validated historical identification.', 'hint'));
    for (const rows of [...groups.values()].sort((a, b) => first(a[0].task) - first(b[0].task))) {
      const sec = el("section", undefined, "metric-group");
      const { task, tier } = rows[0];
      const [tone, meaning] = TIER[tier] || ["neutral", human(tier)];
      sec.append(el("h3", tiersOf.get(task).size > 1 ? `${task} \xb7 ${human(tier).toLowerCase()} cases` : task));
      const p = el("p", undefined, "hint");
      p.append(pill(human(tier), tone), document.createTextNode(meaning + ` \xb7 n = ${rows[0].n}`));
      sec.append(p);
      const wrap = el("div", undefined, "table-wrap"),
        table = el("table"),
        head = el("tr");
      head.append(el("th", "Method"), el("th", "Result"));
      table.append(head);
      for (const r of rows) {
        const tr = el("tr");
        tr.append(el("td", r.method), el("td", r.result));
        table.append(tr);
      }
      wrap.append(table);
      sec.append(wrap);
```

```
      (archived.includes(rows) ? archive : box).append(sec);
    }
    if (archived.length) box.append(archive);
    if (box.querySelector('.outcome-metric')) box.append(box.querySelector('.outcome-metric'));
    if (box.querySelector('.retrieval-metric')) box.append(box.querySelector('.retrieval-metric'));
    box.append(el("p", (m.limitations || []).join(" "), "hint"));
  } catch (error) {
    box.append(el("p", "Evaluation report unavailable: " + error.message, "empty"));
  }
}

/* ---------- Research map ---------- */
let evidenceMap = null, mapLimit = 40, mapScrollY = 0;
const openMapLeads = new Set();
const openMapSourceTrails = new Set();
function mapRows() {
  const query = $("#map-query").value.trim().toLowerCase(), state = $("#map-state").value, order = $("#map-order").value;
  const rows = evidenceMap.rows.filter((r) => (!state || r.state === state) && (!query || (r.taxon + " " + r.topic + " " + r.headings.join("
")).toLowerCase().includes(query)));
  const key = { lead: (r) => [-r.lead_score, -r.historical_statements], places: (r) => [-(r.independent_use?.places || 0), -(r.independent_use?.records || 0)],
statements: (r) => [-r.historical_statements, -r.human_studies], human: (r) => [-r.human_studies, -r.historical_statements] }[order];
  return rows.sort((a, b) => { const x = key(a), y = key(b); return x[0] - y[0] || x[1] - y[1]; });
}
function independentLine(reach) {
  if (!reach) return "";
  return `${plural(reach.records, "record")}${reach.places ? ", " + plural(reach.places, "place") : ""}`;
}
function independentCell(reach) {
  const cell = el("td");
  if (!reach) { cell.append(el("span", "\u2014", "digest-meta")); return cell; }
  cell.append(link(independentLine(reach) + " \u2197", reach.source.url));
  if (reach.place_names.length) cell.append(el("span", reach.place_names.slice(0, 4).join(", "), "digest-meta map-heading"));
  return cell;
}
const REGULATORY = { WELL_ESTABLISHED: "well-established use, which needs clinical evidence", TRADITIONAL: "traditional use, resting on long use rather than trials"
};
function regulatoryLine(row) {
  return row ? `EMA herbal committee: ${REGULATORY[row.class]}` : "";
}
function nextQuestion(r) {
  if (r.direct_statements === 0) return `Resolve the source attribution first: these passages have an uncertain or shared plant subject. They do not raise the
reading priority for ${r.taxon}.`;
  const noun = (r.preparations || []).find((p) =>
/^(?:infusion|decoction|tincture|syrup|tea|oil|poultice|ointment|juice|powder|extract|plaster|wash|gargle|lotion)$/.test(p));
  const prep = noun ? `${/^[aeiou]/.test(noun) ? "an" : "a"} ${noun} of ${r.taxon}` : r.taxon;
  const opening = r.recorded_poisonous ? `Establish safe exposure first: ${r.taxon} is recorded as poisonous. ` : "";
  const asks = {
    PRECLINICAL_ONLY: `Laboratory work exists but no human study was retrieved. Does ${prep} change ${r.topic} in a controlled human study?`,
    HUMAN_FINDINGS_EARLY: `Do larger controlled trials confirm the early findings for ${r.taxon} and ${r.topic}?`,
    HUMAN_FINDINGS_MIXED: `Which preparation, dose or population explains the disagreement for ${r.taxon} and ${r.topic}?`,
    HUMAN_NO_BENEFIT_REPORTED: `Was the preparation tested the one the sources describe? Reported benefit for ${r.topic} is absent, so ${prep} is the open
question.`,
    HUMAN_FINDINGS_FAVOURABLE: `Does the tested preparation match the historical one, ${prep}?`,
  };
  return opening + (asks[r.state] || `No tested result was retrieved. Is there laboratory evidence that ${prep} acts on ${r.topic}?`);
}
function leadText(r) {
  const lines = [`${r.taxon}: ${r.topic}`, `Identity assumption: the map's primary candidate, not a confirmed historical identification.`, `Modern research:
```

```
${(RESEARCH_STATE[r.state] || [0, human(r.state)])[1]} (${r.human_studies} human, ${r.preclinical_studies} laboratory)`];
  if (r.priority_components) lines.push(`Reading priority ${r.lead_score}: ${r.direct_statements} eligible source statements; ${r.uncertain_statements} uncertain links add no weight.`, priorityExplanation(r));
  for (const q of r.statements || r.quotes || []) {
    lines.push(`${q.book}, ${sourceLabel(q.heading)}: "${q.text}"`);
    if(q.attribution) lines.push(`Source link: ${human(q.attribution)}; ${q.source_id}/${q.claim_id}; span ${q.statement.start}-${q.statement.end}; ${q.source_url}; ${q.licence}`);
    if(q.identity_candidates?.length>1) lines.push(`Candidate identities retained: ${q.identity_candidates.map(c=>c.name).join('; ')}.`);
  }
  const references=r.studies?.length?r.studies:r.best_study?[r.best_study]:[];
  if (r.assessment_basis) {
    lines.push(`Summary basis: ${human(r.assessment_basis.kind)}. ${r.assessment_basis.scope}`);
    for (const s of r.assessment_basis.records || []) lines.push(`Summary source: ${s.title} (${s.year || 'year unknown'}); ${s.paper_id}; ${s.direction}; ${s.url}`);
  }
  for(const study of references) {
    lines.push(`Research to read: ${study.title} (${study.year||'year unavailable'}). ${study.paper_id}; ${study.url}; ${(study.design||[]).join(', ')}; reported direction ${study.direction||'not stated'}; identity ${study.identity||'not established'}.`);
    for(const statement of study.finding_statements||[])lines.push(`Abstract finding (${statement.direction}; span ${statement.start}-${statement.end}): \u201c${statement.supporting_text}\u201d`);
  }
  if(r.combination_only)lines.push('Study scope: the listed studies use combinations; the findings do not isolate this plant.');
  if (r.independent_use) lines.push(`Recorded elsewhere: ${independentLine(r.independent_use)}: ${r.independent_use.place_names.join(", ")}. Dr. Duke's databases, ${r.independent_use.source.url}`);
  if (r.relatives) lines.push(`Related species with this use: ${r.relatives.names.join(", ")} (${plural(r.relatives.species, "species").replace("speciess", "species")}, ${plural(r.relatives.places, "place")})`);
  if (r.constituents) lines.push(`Constituents to inspect (no ranking weight): ${r.constituents.rarest.map((c) => `${c.chemical} (${c.activity})`).join(", ")}`);
  if (r.regulatory) lines.push(`${regulatoryLine(r.regulatory)}. ${r.regulatory.url}`);
  if (r.recorded_poisonous) lines.push("Safety: recorded as poisonous.");
  lines.push(`Next question: ${nextQuestion(r)}`, "A recorded use is not evidence that a treatment works.");
  return lines.join("\n");
}
function priorityExplanation(r) {
  const p=r.priority_components;
  return p?`(${p.source_statements} source statements + ${p.additional_books} additional-book points + ${p.independent_places} independent-place points + ${p.related_species} related-species points) \xd7 ${p.research_state_weight} research-state weight \xf7 ${p.poison_divisor} safety divisor. Uncertain source links add zero.`:'Priority was calculated by an earlier build; its source attribution was not screened.';
}
async function openMapStatement(r,q) {
  if(!q?.statement){choosePassage(r.passages[0].id);showView('workspace');await analyze();return;}
  mapScrollY=window.scrollY;
  await openLibraryRecord({id:q.source_id+':'+q.claim_id,source_id:q.source_id,statement:q.statement},'map');
  if(data?.source.id!==q.source_id)return;
  const claim=data.historical_claims.find(c=>c.support.start===q.statement.start&&c.support.end===q.statement.end);
  const target=claim&&data.identity_counterfactuals.find(g=>g.claim_ids.includes(claim.id)&&g.branches.some(b=>b.taxon_id===r.taxon_id));
  if(target){statementReadingTargets.set(identityPairKey(target),claim.id);selectName(target.key);}
}
function leadRow(r, columns) {
  const row = el("tr", undefined, "lead-row"), cell = el("td"), box = el("div", undefined, "lead-brief");
  cell.colSpan = columns;
  const part = (title, ...content) => { const block = el("div"); block.append(el("h4", title), ...content); box.append(block); };
  if(r.priority_components) part('Why this lead is ranked here',el('p',`Reading priority ${r.lead_score} \xb7 ${r.direct_statements} eligible statements \xb7 ${r.uncertain_statements} uncertain links`),el('p',priorityExplanation(r),'hint'),el('p',`Assuming ${r.taxon}. Candidate identity remains conditional.`,'hint'));
  const sourceStatement = (q) => {
    const box=el('div'),quote=el("blockquote", q.text);
    quote.append(el("span", `${q.book}, ${sourceLabel(q.heading)}`, "digest-meta map-heading"));box.append(quote);
    if(q.attribution)box.append(pill(q.eligible_for_priority?(q.attribution==='EXPLICIT_SINGLE_PLANT'?'Plant named in this statement':'Single chapter subject'):'Uncertain subject \xb7 no priority weight',q.eligible_for_priority?'neutral':'warn'));
```

```
    if(q.identity_candidates?.length>1)box.append(el('p','Identity hypotheses: '+q.identity_candidates.map(c=>c.name).join(' \xb7 '),'hint'));
    if(q.statement){const read=el('button','Read this source statement \u2192','text-action');read.type='button';read.addEventListener('click',()=>openMapStatement(r,q));box.append(read);}
    return box;
  };
  if (r.quotes?.length) part("From the books", ...r.quotes.map(sourceStatement));
  const shown=new Set((r.quotes||[]).map(q=>q.source_id+':'+q.claim_id));
  const remaining=(r.statements||[]).filter(q=>!shown.has(q.source_id+':'+q.claim_id));
  if(remaining.length){const more=el('details',undefined,'map-source-statements'),key=r.taxon_id+'::'+r.condition_id;more.open=openMapSourceTrails.has(key);more.addEventListener('toggle',()=>{more.open?openMapSourceTrails.add(key):openMapSourceTrails.delete(key);});more.append(el('summary',`${remaining.length} more source statement${remaining.length===1?'':'s'} behind this reading`),...remaining.map(sourceStatement));part('Complete source trail',more);}
  if (r.independent_use) part("Recorded elsewhere", el("p", `${independentLine(r.independent_use)}: ${r.independent_use.place_names.join(", ")}.`), el("p", `Recorded as ${r.independent_use.activities.join(", ")}.`, "digest-meta"));
  if (r.relatives) part("Related species", el("p", `${r.relatives.names.join(", ")}: ${r.relatives.species > 1 ? r.relatives.species + " species" : "1 species"} of this genus recorded for this use in ${plural(r.relatives.places, "place")}.`));
  if (r.constituents) part("Constituents", el("p", r.constituents.rarest.map((c) => `${c.chemical} (${c.activity.toLowerCase()})`).join(", ") + "."), el("p", `${plural(r.constituents.count, "distinctive chemical")} with a recorded activity, in any assay and dose. Random plant and topic pairs reach one about as often as real uses, so read this as chemistry to inspect, not as support.`, "digest-meta"));
  if (r.regulatory) { const line = el("p", regulatoryLine(r.regulatory) + " "); line.append(link("monograph \u2197", r.regulatory.url)); part("Regulators", line, el("p", r.regulatory.indication, "digest-meta")); }
  const next = [el("p", nextQuestion(r))], copy = el("button", "Copy this lead", "quiet");
  copy.type = "button";
  copy.addEventListener("click", async () => {
    try { await navigator.clipboard.writeText(leadText(r)); copy.textContent = "Copied"; }
    catch { copy.textContent = "Copy failed"; }
  });
  part("Next question", ...next, copy);
  cell.append(box);
  row.append(cell);
  return row;
}
function renderEvidenceMap() {
  const rows = mapRows(), table = $("#map-table");
  $("#map-count").textContent = `${rows.length} plant and topic pairs`;
  const head = el("tr");
  for (const label of ["Plant", "Topic", "Historical record", "Recorded elsewhere", "Modern research", "Key study"]) head.append(el("th", label));
  table.replaceChildren(head);
  for (const r of rows.slice(0, mapLimit)) {
    const leadKey=r.taxon_id+'::'+r.condition_id;
    const tr = el("tr"), plant = el("td"), open = el("button", r.taxon, "text-action");
    open.type = "button";
    open.addEventListener("click", () => openMapStatement(r,r.quotes?.[0]));
    const more = el("button", "Open lead", "text-action lead-toggle");
    more.type = "button";
    more.setAttribute("aria-expanded", String(openMapLeads.has(leadKey)));
    plant.append(open, el("span", r.headings.slice(0, 2).map(sourceLabel).join(" \xb7 "), "digest-meta map-heading"), more);
    more.addEventListener("click", () => {
      const shown = tr.nextElementSibling?.classList.contains("lead-row");
      if (shown) tr.nextElementSibling.remove(); else tr.after(leadRow(r, 6));
      if(shown)openMapLeads.delete(leadKey);else openMapLeads.add(leadKey);
      more.setAttribute("aria-expanded", String(!shown));
      more.textContent = shown ? "Open lead" : "Close lead";
    });
    if (r.recorded_poisonous) plant.append(pill("Recorded as poisonous", "bad"));
    const [tone, label] = RESEARCH_STATE[r.state] || ["neutral", human(r.state)];
    const research = el("td");
    research.append(pill(label, tone));
    const counts = resultCounts(r);
```

```
    if (r.human_studies || r.preclinical_studies) research.append(el("span", ` ${r.human_studies} human \xb7 ${r.preclinical_studies} laboratory${counts ? " \xb7 " + counts : ""}${r.combination_only ? " \xb7 only with other ingredients" : ""}`, "digest-meta"));
    const best = el("td");
    if (r.studies?.length) best.append(studyList(r));
    else {
      const basis = assessmentBasis(r);
      if (basis) best.append(basis);
      if (r.best_study) { best.append(link(cleanTitle(r.best_study.title).slice(0, 90) + (r.best_study.title.length > 90 ? "\u2026" : "") + " \u2197", r.best_study.url)); if (r.best_study.year) best.append(el("span", " " + r.best_study.year, "digest-meta")); }
      else if (!basis) best.append(el("span", "\u2014", "digest-meta"));
    }
    const historical=el('td',`${plural(r.historical_statements, "statement")} \xb7 ${r.books.length === 2 ? "both books" : r.books[0]}`);
    if(r.direct_statements!==undefined)historical.append(el('span',`${r.direct_statements} eligible \xb7 ${r.uncertain_statements} uncertain`,'digest-meta map-heading'));
    tr.append(plant, el("td", r.topic), historical, independentCell(r.independent_use), research, best);
    table.append(tr);
    if(openMapLeads.has(leadKey)){tr.after(leadRow(r,6));more.textContent='Close lead';}
  }
  $("#map-more").hidden = rows.length <= mapLimit;
}
function mapTemporalSummary(map, report) {
  const box=el('section',undefined,'hint map-foresight');
  box.append(el('h3','What this ranking has demonstrated'));
  const current=report?.protocol_version==='SOURCE_ONLY_ALL_HUMAN_STUDIES_V2'&&report.code_fingerprint===map.code_fingerprint&&report.snapshot_sha256===map.snapshot_sha256&&report.pairs_in_map===map.totals.pairs&&map.build_status!=='PREVIOUS_BUILD';
  const cohort=current?report.cutoffs?.['2010']:null,reading=cohort?.benefit_reported_after;
  if(!reading){box.append(el('p','A matching source-only retrospective evaluation is unavailable for this map. These priorities guide source reading; predictive performance is not established here.'));return box;}
  const metric=value=>Number.isFinite(value)?value.toFixed(3):'not estimable';
  box.append(el('p',`Retrospective 2010 split: ${cohort.eligible_pairs} attributable plant\u2013use pairs with dated historical support and no earlier human study retrieved in this collection. Missing retrieval does not establish that a use was never tested.`));
  box.append(el('p',`Later retrieved human studies reporting benefit: history score AUC ${metric(reading.auc_history_score)}; prior-publication-count baseline AUC ${metric(reading.auc_prior_publications_only)}.`));
  if(Number.isFinite(reading.auc_history_score)&&Number.isFinite(reading.auc_prior_publications_only))box.append(el('p',reading.auc_history_score<=reading.auc_prior_publications_only?'The history score did not outperform the publication-count baseline in this analysis.':'The history score exceeded the publication-count baseline in this retrospective analysis; prospective performance has not been established.'));
  box.append(el('p','This source-only retrospective score differs from the displayed reading priority, which also considers research state and safety. It is not prospective validation, a treatment-effect estimate, or evidence that a lead works.'));
  return box;
}
let evidenceMapLoading;
function loadEvidenceMap() {
  if(!evidenceMapLoading)evidenceMapLoading=populateEvidenceMap().finally(()=>{evidenceMapLoading=null;});
  return evidenceMapLoading;
}
async function populateEvidenceMap() {
  if (!evidenceMap) {
    try { evidenceMap = await api("/api/research/evidence-map"); }
    catch (error) { $("#map-count").textContent = error.message; return; }
    const states = $("#map-state");
    for (const [key, value] of Object.entries(RESEARCH_STATE)) if (evidenceMap.counts[key]) states.append(option(key, `${value[1]} (${evidenceMap.counts[key]})`));
    const summary = $("#map-summary");
    if (evidenceMap.build_warning) summary.before(el('p', evidenceMap.build_warning, 'error'));
    for (const [number, label] of [[evidenceMap.totals.pairs, "plant and topic pairs"], [evidenceMap.totals.with_human_studies, "with human studies"], [evidenceMap.totals.leads, "research leads"], [evidenceMap.totals.plants, "plants"]]) { const tile = el("div", undefined, "map-tile"); tile.append(el("strong", number), el("span", label)); summary.append(tile); }
    const report=await benchmarks().then(result=>result.lead_foresight).catch(()=>null);
    summary.after(mapTemporalSummary(evidenceMap,report));
```

```
    for (const id of ["#map-query", "#map-state", "#map-order"]) $(id).addEventListener("input", () => { mapLimit = 40; renderEvidenceMap(); });
    $("#map-more").addEventListener("click", () => { mapLimit += 60; renderEvidenceMap(); });
  }
  renderEvidenceMap();
}

/* ---------- Analysis ---------- */
function render() {
  if(!$('#view-handoffs').hidden)handoffs.open();
  const groups = data.identity_counterfactuals;
  const manual=new Set(data.mentions.filter(m=>m.method==='RESEARCHER_SELECTED_NAME').map(m=>m.id));
  const announced=new Set(data.mentions.filter(m=>m.method==='SOURCE_HEADING_SUBJECT').map(m=>m.id));
  const preferred = groups.find(g=>g.mention_ids.some(id=>manual.has(id))) || groups.find(g=>g.branches.length&&g.mention_ids.some(id=>announced.has(id))) || groups.find((g) => g.branches.length > 1 || g.open_world.length) || groups.find((g) => g.branches.length) || groups[0];
  selected = preferred?.key ?? null;
  renderSource();
  renderNames();
  renderInspector();
  renderClaims();
  setupGraphFilter();
  graphFocus = null;
  if (!$("#view-graph").hidden) renderGraph();
  $("#limitations").replaceChildren(...data.limitations.map((x) => el("li", x)));
  $("#run-info").textContent = `Run ${data.run_id} \xb7 pipeline ${data.pipeline_version} \xb7 ${data.performance.latency_ms} ms${data.cache_hit ? " \xb7 cached" : ""}`;
  $('#run-info').textContent='Local research workspace';
  for (const b of document.querySelectorAll(".case")) b.setAttribute("aria-pressed", String(!custom && b.dataset.passage === $("#passage").value));
}
async function analyze(options={}) {
  const b = $("#analyze"),serial=++analysisSerial;
  b.disabled = true;
  status(recorded ? "Loading the recorded pipeline output\u2026" : "Extracting names, building identity branches and screening evidence\u2026");
  try {
    const request=custom?{text:$('#input').value,retrieval:$('#analysis-ranking').value}:{passage_id:$('#passage').value,retrieval:$('#analysis-ranking').value};
    request.name_to_inspect=requestedSourceName;
    const result = await api("/api/analyses", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(request),
    });
    if(serial!==analysisSerial)return;
    if(libraryFocus){const span=libraryFocus.statement;if(libraryFocus.source_id!==result.source.id||Array.from(result.input).slice(span.start,span.end).join('')!==span.text){libraryFocus=null;sourceEvidenceFocus=null;}}
    data=result;
    data.researcher_reviews ||= [];
    render();
    showTab("tab-identity");
    readingResumeActive=options.remember!==false;
    rememberReading();
    const onDemand=data.literature_access?.plants_read_on_demand?.length||0,offline=data.literature_access?.status==='NETWORK_UNAVAILABLE';
    status(new Set(data.evidence.map(e=>e.paper.id)).size+' publications connected to '+plural(data.identity_counterfactuals.length,'plant name')+(data.language&&data.language!=='en'?' \xb7 source language: '+languageName(data.language):'')+(onDemand?' \xb7 '+plural(onDemand,'plant')+' read from PubMed just now':'')+(offline?' \xb7 offline: plants outside the snapshot could not be looked up':'')+'.');
    return result;
  } catch (error) {
    if(serial===analysisSerial)status(error.message, true);
  } finally {
    if(serial===analysisSerial)b.disabled = false;
```

```
  }
}
function invalidatePendingAnalysis() {
  analysisSerial++;
  $('#analyze').disabled=false;
  status('Input changed. Analyze to read this version; the previous analysis is still shown.');
}
function choosePassage(id) {
  invalidatePendingAnalysis();
  requestedSourceName=null;
  $("#passage").value = id;
  const p = passages.find((x) => x.id === id);
  if (p) {
    $("#input").value = p.title + "\n\n" + p.text;
    custom = false;
  }
}
let searchSerial=0;
async function search() {
  const q = $("#search").value.trim();
  const serial=++searchSerial;
  if (q.length < 2) return;
  const box = $("#search-results");
  try {
    const r = await api("/api/search?" + new URLSearchParams({ q }));
    if(serial!==searchSerial||q!==$('#search').value.trim())return;
    box.replaceChildren(el("p", `${r.results.length} results (at most 100)`, "hint"));
    const kinds = { HistoricalSource: "Historical section", ModernTaxon: "Modern taxon", Publication: "Publication", Compound: "Compound" };
    for (const item of r.results) {
      if (item.kind === "HistoricalSource" && passages.some((p) => p.id === item.id)) {
        const b = el("button", undefined, "result");
        b.type = "button";
        b.append(pill(kinds[item.kind]), document.createTextNode(item.label));
        b.addEventListener("click", () => {
          choosePassage(item.id);
          analyze();
        });
        box.append(b);
      } else {
        const row = el("div", undefined, "result");
        row.append(pill(kinds[item.kind] || item.kind), link(cleanTitle(item.label), item.url));
        box.append(row);
      }
    }
  } catch (error) {
    if(serial===searchSerial&&q===$('#search').value.trim())status(error.message, true);
  }
}
function renderCases() {
  const box = $("#cases");
  box.replaceChildren();
  for (const [title, label, why, settings={}] of CASES) {
    const p = passages.find((x) => x.title === title&&(settings.collection===undefined||Boolean(x.collection)===settings.collection));
    if (!p) continue;
    const b = el("button", label, "case");
    b.type = "button";
    b.dataset.passage = p.id;
    b.setAttribute("aria-pressed", "false");
```

```
    b.append(el("small", why));
    b.addEventListener("click", async () => {
      showView('workspace');libraryFocus=null;sourceEvidenceFocus=null;
      choosePassage(p.id);
      const result=await analyze();
      if(result?.source.id===p.id&&settings.sourceStudy&&!openSourceStudyReading(settings.sourceStudy))status('This analysis has no active matching study link. Read the source and available studies instead.',true);
    });
    box.append(b);
  }
}

$("#analyze").addEventListener("click", analyze);
$('#inspect-source-name-form').addEventListener('submit',async event=>{
  event.preventDefault();const button=$('#inspect-source-name-form button[type="submit"]'),previous=requestedSourceName;
  requestedSourceName=$('#source-name-input').value.trim();button.disabled=true;
  $('#source-name-state').textContent='Finding the exact wording in this source\u2026';
  try{const pending=analyze(),serial=analysisSerial,result=await pending;if(!result&&serial===analysisSerial){requestedSourceName=previous;$('#source-name-state').textContent='Could not inspect this name. Check the source spelling and the message above.';}}
  finally{button.disabled=false;}
});
$('#source-name-reset').addEventListener('click',()=>{requestedSourceName=null;analyze();});
$("#passage").addEventListener("change", () => choosePassage($("#passage").value));
$("#input").addEventListener("input", () => {
  invalidatePendingAnalysis();
  custom = true;
  requestedSourceName=null;
});
$('#analysis-ranking').addEventListener('change',invalidatePendingAnalysis);
$('#search').addEventListener('input',()=>{searchSerial++;$('#search-results').replaceChildren();});
$("#search-button").addEventListener("click", search);
$("#search").addEventListener("keydown", (e) => {
  if (e.key === "Enter") search();
});
for (const id of ["branch", "study-type", "relation", "year"]) $("#" + id).addEventListener("change", () => data && renderEvidence());
$("#edge-filter").addEventListener("change", renderGraph);
$("#graph-reset").addEventListener("click", () => {
  graphFocus = null;
  renderGraph();
});
for (const b of document.querySelectorAll(".view-tab")) b.addEventListener("click", () => showView(b.dataset.view));
const tabs = [...document.querySelectorAll('[role="tab"]')];
for (const t of tabs) {
  t.addEventListener("click", () => showTab(t.id));
  t.addEventListener("keydown", (e) => {
    const i = tabs.indexOf(t);
    const next = e.key === "ArrowRight" ? tabs[(i + 1) % tabs.length] : e.key === "ArrowLeft" ? tabs[(i - 1 + tabs.length) % tabs.length] : e.key === "Home" ? tabs[0] : e.key === "End" ? tabs.at(-1) : null;
    if (next) {
      e.preventDefault();
      showTab(next.id, true);
    }
  });
}
$("#text-file").addEventListener("change", async (e) => {
  const file = e.target.files[0];
  if (!file) return;
  const form = new FormData();
```

```
  form.append("file", file);
  const serial=++analysisSerial;
  try {
    const result = await api("/api/uploads/text", { method: "POST", body: form });
    if(serial!==analysisSerial)return;
    data=result;
    libraryFocus=null;sourceEvidenceFocus=null;
    requestedSourceName=null;
    data.researcher_reviews ||= [];
    custom = true;
    $("#input").value = data.input;
    render();
    readingResumeActive=true;rememberReading();
    status("Uploaded text analysed. Its historical provenance is not verified.");
  } catch (error) {
    if(serial===analysisSerial)status(error.message, true);
  } finally {
    if(serial===analysisSerial)$('#analyze').disabled=false;
  }
});
const handoffs=initHandoffs({getAnalysis:()=>recorded?null:data,api,el,link,showView});
const questionReview=initQuestionReview({api,el,link,onAttach:(context,reviewId)=>consultation.attach(context,reviewId),onReturn:async()=>{await
showView('consultation');consultation.focusSaved();}});
const consultation=initConsultation({api,el,link,isOffline:()=>health?.offline!==false,openQuestion:initial=>showView('question',initial)});
$('#care-start-own').addEventListener('click',()=>{const editor=$('#consultation-editor');editor.open=true;const
input=editor.querySelector('[name="product_name"]');input?.scrollIntoView({block:'center',behavior:'smooth'});input?.focus({preventScroll:true});});
$('#start-source-check').addEventListener('click',()=>{showView('workspace');$('#custom-input').open=true;$('#input').focus();});
$('#open-evidence-handoffs').addEventListener('click',()=>showView('handoffs'));
async function init() {
  const requestedView=location.hash.slice(1);
  const initialView=['workspace','library','map','chemistry','graph','evaluation','handoffs','consultation','question'].includes(requestedView)?requestedView:(locati
on.pathname.startsWith('/research')?'library':'consultation');
  try {
    // Draft entry does not wait for model, health or the historical library.
    const opening=showView(initialView);
    health = await api("/api/health");
    recorded = health.mode === "READ_ONLY_RECORDED_PIPELINE";
    const badge = $("#system-state");
    badge.textContent = recorded ? "Recorded outputs \xb7 read-only" : health.offline ? "Offline workspace" : "Local workspace";
    badge.className = "mode" + (recorded ? " recorded" : "");
    badge.title = recorded ? "These results were computed by the local pipeline and recorded. This page runs no new analysis." : `Snapshot: ${health.passages}
historical sections, ${health.taxa} taxa, ${health.papers} literature records; ${health.abstracts_available ?? health.papers} abstracts available locally.`;
    if (recorded) {
      $("#analysis-ranking-label").hidden = true;
      $("#custom-input").hidden = true;
      $("#external-name-check").hidden = true;
      $('#inspect-source-name').hidden=true;
      $("#knowledge-search").hidden = false;
      $("#knowledge-method-label").hidden = true;
      $("#analyze").textContent = "Load record";
    }
    if (!recorded) {
      const semantic = Boolean(health.encoder_installed), reranking = semantic && Boolean(health.reranker_installed);
      for (const choice of $('#knowledge-method').options) if (choice.value !== 'bm25') choice.disabled = !semantic;
      $('#analysis-ranking').querySelector('option[value="rerank"]').disabled = !reranking;
      if (reranking) { $('#knowledge-method').append(option('rerank', 'Best match')); $('#knowledge-method').value='rerank'; }
      else $('#knowledge-method').value='bm25';
    }
```

```
    if (!recorded && health.abstracts_available < health.papers) {
      const availability = $('#literature-availability');
      availability.textContent = `${health.abstracts_available.toLocaleString()} abstracts available locally \xb7 additional records have citation metadata only`;
      availability.hidden = initialView==='consultation';
    }
    await opening;
    const saved=savedReading();let resumed=false,resumeError=null;
    if(saved&&!custom&&!['consultation','question'].includes(initialView)){try{resumed=await resumeReading(saved,{navigate:false,position:['workspace','graph'].includes(initialView)});}catch(error){resumeError=error.message;}}
    // A reader may have started typing while the collection was still loading.
    if(resumed===false&&!custom&&!['consultation','question'].includes(initialView)){
      const p=passages.find(p=>p.title===CASES[0][0])||passages[0];
      if(p){choosePassage(p.id);await analyze({remember:false});}
      if(!saved&&data&&['workspace','graph'].includes(initialView)){readingResumeActive=true;rememberReading();}
    }
    if(resumeError)status('Your saved analysis is unavailable here: '+resumeError+'. Choose a source to start a new reading.',true);
    updateResumeControl();
    // Research evaluation is loaded only when that view is opened.
  } catch (error) {
    status("Startup failed: " + error.message, true);
    $("#system-state").textContent = "Service unavailable";
    $("#system-state").className = "mode error";
  }
}
init();
```

frontend/care-tools.js
SHA-256: be9e9bff78a0939df45ef41762d19ad5aadab1a8b62826e86bd5cd59e01dfe7b

```
// Task tools use saved source versions; editable drafts never become evidence.
const STATUS = {PASS:'Text signal found', PARTIAL:'Needs reading', UNKNOWN:'Not established', FAIL:'Scope conflict'};
const SOURCE = {PERSON_REPORTED:'Person or caregiver', PACKAGING_REPORTED:'Packaging, reported by recorder', PROFESSIONAL_NOTE:'Professional note'};

export function clarificationPanel({record,item,api,el,onSave}) {
  const panel=el('section',undefined,'clarification-panel');
  const history=(record.packet.clarification_history||[]).filter(n=>n.item_id===item.reported.item_id);
  const latest=new Map(history.map(n=>[n.field,n]));
  const answered=item.clarification_questions.filter(q=>latest.get(q.field)?.status==='ANSWER_RECORDED').length;
  panel.append(el('h4','Make the next conversation easier'),el('p',answered+' of '+item.clarification_questions.length+' questions have a recorded answer. Original reported details stay unchanged.','hint'));
  for(const task of item.clarification_questions){
    const note=latest.get(task.field),row=el('details',undefined,'clarification-task');
    row.append(el('summary',(note?.status==='ANSWER_RECORDED'?'Answer recorded \xb7 ':note?'Ask pharmacist \xb7 ':'To clarify \xb7 ')+task.question));
    if(note){row.append(el('p',note.note,'clarification-answer'),el('p',(SOURCE[note.source]||note.source)+' \xb7 '+note.recorded_by+' \xb7 '+new Date(note.created_at).toLocaleString(),'hint'));}
    if(record.origin==='LOCAL_CONSULTATION_PREPARATION'){
      const form=el('form');
      const field=(title,name,type='input')=>{const label=el('label',title),input=el(type);input.name=name;label.append(input);form.append(label);return input;};
      const status=field('What happens next?','status','select');
      for(const [value,text] of [['ANSWER_RECORDED','Record an answer, with its source'],['ASK_PROFESSIONAL','Keep a question for the pharmacist']]){const o=el('option',text);o.value=value;status.append(o);}
      status.value=note?.status||'ASK_PROFESSIONAL';
      const text=field('Answer or question to bring','note','textarea');text.rows=3;text.required=true;text.minLength=5;text.maxLength=2000;text.value=note?.note||'';
      const source=field('Where did this information come from?','source','select');
      for(const [value,label] of Object.entries(SOURCE)){const o=el('option',label);o.value=value;source.append(o);}source.value=note?.source||'PERSON_REPORTED';
      const by=field('Recorded by (a role is enough)','recorded_by');by.required=true;by.maxLength=80;by.value=note?.recorded_by||'Patient or caregiver';
      const save=el('button','Save note as a new snapshot','quiet');save.type='submit';const feedback=el('p','','hint');feedback.setAttribute('role','status');form.append(save,feedback);
      form.onsubmit=async event=>{event.preventDefault();save.disabled=true;try{await onSave({item_id:item.reported.item_id,field:task.field,status:status.value,note:text.value,source:source.value,recorded_by:by.value},record);}
        catch(error){feedback.textContent='Not saved: '+error.message+'. Your note is kept here.';}finally{save.disabled=false;}};
      row.append(form);
    }
    panel.append(row);
  }
  if(history.length){const audit=el('details');audit.append(el('summary','Earlier clarification notes ('+history.length+')'));for(const n of history)audit.append(el('p',n.field+' \xb7 '+n.note),el('p',(SOURCE[n.source]||n.source)+' \xb7 '+n.recorded_by+' \xb7 '+n.created_at,'hint'));panel.append(audit);}
  return panel;
}

export function sharePanel({record,api,el}) {
  const panel=el('details',undefined,'selected-sharing');panel.append(el('summary','Choose what to share with your pharmacist'));
  panel.append(el('p','Nothing is selected automatically. Preview the exact information before downloading. Omitted information does not mean \u201cnone\u201d.','hint'));
  const form=el('form'),products=el('fieldset');products.append(el('legend','1 \xb7 Select products'));
  const checkbox=(parent,label,name,value)=>{const wrap=el('label',undefined,'share-choice'),input=el('input');input.type='checkbox';input.name=name;if(value)input.value=value;wrap.append(input,el('span',label));parent.append(wrap);return input;};
  for(const item of record.packet.products)checkbox(products,item.reported.name,'item_ids',item.reported.item_id);
  form.append(products);const extra=el('fieldset');extra.append(el('legend','2 \xb7 Add only what you want included'));
  for(const [name,title] of [['include_medicines','Other medicines and their reported status'],['include_allergies','Allergies / previous reactions and their reported status'],['include_questions','Your additional questions and missing-context prompts'],['include_clarifications','Recorded clarification answers and their sources'],['include_reading','Attached professional reading decisions and reasons']])checkbox(extra,title,name);
  form.append(extra);
```

```
  const preview=el('button','Preview selected handoff');preview.type='submit';preview.disabled=true;
  const message=el('p','Select at least one product.','hint');message.setAttribute('role','status');
  const output=el('div',undefined,'share-preview');output.hidden=true;
  const downloads=el('div',undefined,'share-downloads');downloads.hidden=true;let generation=0,urls=[];
  const invalidate=()=>{generation++;for(const u of urls)URL.revokeObjectURL(u);urls=[];output.replaceChildren();output.hidden=true;downloads.replaceChildren();downl
oads.hidden=true;preview.disabled=!form.querySelector('[name="item_ids"]:checked');message.textContent=preview.disabled?'Select at least one product.':'Selection
changed. Generate a fresh preview before sharing.';};
  form.addEventListener('change',invalidate);
  form.onsubmit=async event=>{event.preventDefault();const at=++generation,data=new FormData(form),selection={item_ids:data.getAll('item_ids')};for(const input of
form.querySelectorAll('input[type="checkbox"]'))if(input.name!=='item_ids')selection[input.name]=input.checked;preview.disabled=true;message.textContent='Preparing
only your selected information\u2026';
    try{const result=await api('/api/consultations/'+record.id+'/share-preview',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify(selection)});if(at!==generation)return;
      const p=result.projection;output.replaceChildren(el('h3','Your sharing preview'),el('p',p.sharing_scope,'hint'),el('p',p.scope,'hint'));
      for(const [key,title] of [['other_medicines','Other medicines'],['allergies','Allergies / previous
reactions']])if(p[key])output.append(el('h4',title),el('p',p[key].status),el('p',p[key].details||'No details recorded'));
      if('questions' in p){output.append(el('h4','Questions to discuss'),el('p',p.questions||'No additional question recorded'));for(const q of
p.next_questions||[])output.append(el('p',q));}
      for(const row of p.products){const card=el('article',undefined,'share-preview-product');card.append(el('h4',row.name),el('p',row.use),el('p','Brand:
'+(row.brand||'Not provided')));
        for(const [key,value] of Object.entries(row.actual_use))card.append(el('p',key.replaceAll('_',' ')+': '+value));
        card.append(el('p','Stated form: '+row.stated_preparation),el('p','Reported reason: '+row.reported_reason));
        if(row.label){card.append(el('p','NIH label '+row.label.id+' \xb7 bottle match not established'));for(const ing of
row.ingredients)card.append(el('p',(ing.nested?'Within blend: ':'')+ing.name+': '+(ing.amounts.join('; ')||'Individual amount not
provided')),el('p',ing.forms),el('p',ing.notes));const other=row.other_ingredients||{};card.append(el('p','Other declared ingredients:
'+(other.text||(other.ingredients||[]).map(i=>i.name).join('; ')||'Not provided')));}
        else card.append(el('p',row.source_text||'No original wording recorded'));
        const questions=el('ul');for(const q of row.clarifications)questions.append(el('li',q));card.append(questions);
        for(const note of row.clarification_notes||[])card.append(el('p',note.field+' \xb7 '+note.note),el('p',(SOURCE[note.source]||note.source)+' \xb7
'+note.recorded_by+' \xb7 '+note.created_at,'hint'));
        for(const r of row.reviews||[]){card.append(el('h4','Professional reading \xb7 not product
approval'),el('p',r.question),el('p',r.coverage_note,'hint'),el('p',r.unreviewed_records+' candidate/study records have no recorded human
decision.'));if(!r.decisions.length)card.append(el('p','No human reading decision recorded.'));for(const d of
r.decisions)card.append(el('p',d.title),el('p',d.decision+': '+d.reason),el('p','Recorded by: '+d.reviewer));}output.append(card);
      }
      const omitted=[['include_medicines','other medicines'],['include_allergies','allergies'],['include_questions','additional
questions'],['include_clarifications','clarification notes'],['include_reading','professional readings']].filter(([key])=>!selection[key]).map(([,name])=>name);
      message.textContent=p.products.length+' selected product(s). Excluded: '+(omitted.join(', ')||'no optional sections')+'. This file contains health
information.';
      output.hidden=false;downloads.replaceChildren();for(const [text,type,content,filename] of [['Open selected handoff to
print','text/html',result.html,'foliatrace-selected-handoff.html'],['Download selected data','application/json',JSON.stringify(p,null,2),'foliatrace-selected-
handoff.json']]){const a=el('a',text,'button-link');const u=URL.createObjectURL(new
Blob([content],{type}));urls.push(u);a.href=u;if(type==='text/html'){a.target='_blank';a.rel='noopener noreferrer';}else
a.download=filename;downloads.append(a);}downloads.hidden=false;
    }catch(error){if(at===generation)message.textContent='Preview unavailable: '+error.message+'. Nothing has been
shared.';}finally{if(at===generation)preview.disabled=false;}
  };
  form.append(preview,message);panel.append(form,output,downloads);return panel;
}

export function bridgePanel({context,record,row,api,el,link}) {
  const panel=el('section',undefined,'product-study-bridge');
  const button=el('button','Compare this study with the saved product','quiet');button.type='button';
  const content=el('div');panel.append(button,content);
  button.onclick=async()=>{button.disabled=true;content.replaceChildren(el('p','Checking the exact product and study versions\u2026','hint'));
    try{const params=new
URLSearchParams({review_id:record.id,item_id:context.item_id,record_id:row.id});if(context.ingredient_key)params.set('ingredient_key',context.ingredient_key);
      const result=await api('/api/consultations/'+context.consultation_id+'/study-bridge?'+params);
```

```
      content.replaceChildren(el('h4',result.product_name+' \u2194 '+result.candidate),el('p',result.scope,'hint'));
      if(result.source_label)content.append(link('Selected NIH label '+result.source_label.id+' \u2197',result.source_label.url));
      const scroll=el('div',undefined,'bridge-scroll');scroll.tabIndex=0;scroll.setAttribute('role','region');scroll.setAttribute('aria-label','Product and study comparison; scroll horizontally if needed');
      const table=el('table',undefined,'bridge-table'),caption=el('caption','What is known on each side');table.append(caption);const head=el('thead'),tr=el('tr');for(const title of ['Check','Saved product / report','Question and study source']){const th=el('th',title);th.scope='col';tr.append(th);}head.append(tr);table.append(head);const body=el('tbody');
      for(const d of result.dimensions){
        const tr=el('tr'),title=el('th',d.label);title.scope='row';
        const product=el('td',d.product),study=el('td');
        const requirement={human:'Human participants',any:'No specific requirement',single_plant:'This plant alone',controlled:'A comparison group',placebo:'A placebo comparison'}[d.question_requirement]||String(d.question_requirement).replaceAll('_',' ');
        study.append(el('span',STATUS[d.status]||d.status,'bridge-status '+d.status.toLowerCase()),el('p','Requested: '+requirement));
        const why=el('details',undefined,'bridge-reason');why.open=d.status==='FAIL';
        why.append(el('summary','Reason and source'),el('p',d.detail));
        if(d.caution&&d.caution!==d.detail)why.append(el('p',d.caution,'hint'));
        for(const span of d.spans){why.append(el('blockquote',span.text));if(span.url)why.append(link('Inspect source \u2197',span.url));}
        study.append(why);tr.append(title,product,study);body.append(tr);
      }
      table.append(body);scroll.append(table);content.append(scroll);button.textContent='Product comparison loaded';
    }catch(error){content.replaceChildren(el('p','Comparison unavailable: '+error.message+'. No product match has been inferred.','error'));button.disabled=false;}
  };
  return panel;
}
```

frontend/care.css
SHA-256: a5fd17096d6aff5b8f86d21a25b8d35d466aaf12334e6e5be834b9a14a66aeb3

```
/* The consultation product: quiet hierarchy, source comparisons and deliberate sharing. */
.care-view{font:17px/1.7 var(--sans)}
.brand-row,.views{padding-inline:0}.brand-row{flex-wrap:wrap}
body:has(.care-view:not([hidden]))>main{padding:0}
.topbar{padding:22px max(24px,calc((100vw - 1280px)/2));background:var(--surface);border-bottom:1px solid var(--line)}
.brand-row{align-items:center;gap:22px}.brand{display:inline-flex;align-items:center;gap:12px;font-family:var(--sans);font-size:23px;letter-spacing:-.7px;text-decoration:none;color:var(--ink)}
.brand-symbol{display:grid;place-items:center;width:38px;height:38px;border-radius:50% 50% 50% 4px;background:var(--button);color:var(--button-ink);font:italic 30px var(--serif)}
.topbar .tagline{font-size:12px;letter-spacing:.045em}.views{margin-top:18px;gap:8px}.views .view-tab{border-radius:30px;padding:10px 20px;font-size:14px}.views .view-tab[aria-current=page]{background:var(--button);color:var(--button-ink);border-color:var(--button)}
.care-view{max-width:1280px;margin:0 auto;padding:0 24px}.care-hero{display:grid;grid-template-columns:minmax(0,1.55fr) minmax(290px,1fr);gap:64px;padding:56px 0 38px;align-items:center}
.care-heading .kicker{font-size:11px;letter-spacing:.12em;font-weight:650;line-height:1.8}.care-heading h1{max-width:24ch;font:clamp(38px,4.5vw,61px)/1.07 var(--serif);font-weight:400;letter-spacing:-1.9px;margin:16px 0 22px}.care-heading h1 em{color:var(--identity);font-weight:400}
.care-hero .library-lede{font-size:17px;line-height:1.8;max-width:55ch}.hero-principles{display:flex;flex-wrap:wrap;gap:8px 18px;margin-top:24px;color:var(--muted);font-size:12px}.hero-principles span:before{content:'\2713';color:var(--identity);margin-right:6px}
.hero-note{position:relative;padding:30px 32px;background:var(--surface);border:1px solid var(--line);border-radius:6px 6px 42px 6px;box-shadow:8px 10px 0 var(--surface-2);transform:rotate(1deg)}
.note-number{font:10px var(--mono);letter-spacing:.12em;color:var(--muted)}.hero-note ol{list-style:none;margin:22px 0;padding:0;counter-reset:care}.hero-note li{counter-increment:care;display:grid;grid-template-columns:30px 1fr;gap:3px 12px;margin:20px 0}.hero-note li:before{content:'0' counter(care);grid-row:span 2;font:12px var(--mono);color:var(--identity);padding-top:4px}.hero-note strong{font-size:16px;font-weight:600}.hero-note li span{font-size:13px;line-height:1.6;color:var(--muted)}.hero-note>p{border-top:1px solid var(--line);padding-top:18px;font:italic 17px/1.5 var(--serif)}
.care-start{padding:22px 0;gap:16px}.care-start strong{font-size:16px}.care-start p{font-size:13px;color:var(--muted);margin:5px 0 0}.care-start .row{gap:10px}.care-start button{font-size:13px}.care-scope{font-size:12px;margin:16px 0}
.care-layout{grid-template-columns:minmax(0,1fr) 265px;gap:28px}.care-view .handoff-card{background:var(--surface);border:1px solid var(--line);border-radius:12px;box-shadow:none;padding:28px}.care-view .care-saved{background:transparent;border:none;border-left:1px solid var(--line);border-radius:0;padding:8px 0 8px 24px;margin-top:28px}.care-saved h2{font:23px/1.2 var(--serif)}
.care-steps{gap:0;border:1px solid var(--line);border-radius:7px;overflow:hidden}.care-steps button{border:0;border-radius:0;background:var(--surface-2);color:var(--muted);font-size:13px;min-height:48px}.care-steps button+button{border-left:1px solid var(--line)}.care-steps [aria-current=step]{background:var(--button);color:var(--button-ink)}
.care-view .care-form-heading h2{font:29px/1.2 var(--serif)}.care-view input,.care-view textarea,.care-view select{border-radius:6px;background:var(--surface);border:1px solid var(--line-strong);padding:11px 12px}.care-view button,.care-view .button-link{border-radius:6px}.care-view button:focus-visible,.care-view input:focus-visible,.care-view select:focus-visible,.care-view textarea:focus-visible,summary:focus-visible{outline:3px solid var(--focus);outline-offset:3px}
.care-view .consultation-product{border-color:var(--line);border-radius:8px;padding:22px;background:var(--surface)}.care-view .consultation-product legend{font-size:13px;letter-spacing:.02em}.care-output-actions{gap:8px}.care-view .consultation-summary{padding:18px 0;border-block:1px solid var(--line);gap:10px 24px}.consultation-summary strong{font:25px var(--serif)}
.consultation-product-reading{padding:22px 0}.consultation-product-reading>h3{font-family:var(--serif)}.consultation-facts{background:var(--surface-2);border-radius:8px;padding:18px;font-size:14px;gap:9px 20px}.consultation-facts dd{font-weight:500}.consultation-product-reading .label-ingredient{border-left-width:2px}
.clarification-panel{margin:24px 0}.clarification-panel>h4{font:22px var(--serif)}.clarification-task{border-top:1px solid var(--line);padding:7px 0}.clarification-task>summary{font-size:14px;line-height:1.6;font-weight:500}.clarification-task form{background:var(--surface-2);padding:16px;border-radius:8px;margin:10px 0}.clarification-answer{padding:14px;border-left:3px solid var(--identity);background:var(--surface-2)}
.selected-sharing{padding:18px 20px;margin:24px 0;border:1px solid var(--line-strong);border-radius:8px;background:var(--surface-2)}.selected-sharing>summary{font-weight:650}.selected-sharing fieldset{max-height:none!important;overflow:visible!important;background:var(--surface);padding:16px;border-radius:6px}.selected-sharing legend{font-size:13px;font-weight:650;padding:0 8px}.care-view .share-choice{display:flex;align-items:flex-start;gap:10px;cursor:pointer;padding:6px 0;min-height:44px}.care-view .share-choice input{width:20px;height:20px;min-height:20px;flex:0 0 20px;margin:1px 0 0;accent-color:var(--identity)}
.share-preview{padding:20px;margin-top:20px;background:var(--surface);border:1px solid var(--line);border-radius:8px}.share-preview h3{font:25px var(--serif)}.share-preview-product{border-top:1px solid var(--line);padding:16px 0;overflow-wrap:anywhere}.share-preview-product h4{font:22px var(--serif)}.share-downloads{display:flex;gap:10px;flex-wrap:wrap;margin-top:18px}.share-downloads .button-link{padding:12px 15px;background:var(--surface);font-size:14px}
.question-starters{display:flex;gap:8px;flex-wrap:wrap;margin:12px 0 22px}.question-starters>p{flex-basis:100%;margin:0}.question-starters button{font-size:12px;padding:8px 12px;min-height:38px;border-radius:30px}
.product-study-bridge{margin:16px 0;padding:16px;border:1px solid var(--line-strong);border-radius:8px;background:var(--surface-2)}.product-study-bridge h4{font-size:18px;line-height:1.5}.bridge-scroll{overflow:auto;max-width:100%;margin-top:16px}.bridge-table{width:100%;border-collapse:collapse;font-size:13px;table-layout:fixed;min-width:540px}.bridge-table caption{text-align:left;margin-bottom:10px;font-weight:600}.bridge-table th,.bridge-table td{text-align:left;vertical-
```

```
align:top;border:1px solid var(--line);padding:12px;background:var(--surface);overflow-wrap:anywhere}.bridge-table th:first-child{width:19%}.bridge-table th:nth-child(2){width:28%}.bridge-table th{font-weight:600}.bridge-table td p{font-size:12px;line-height:1.65}.bridge-status{display:inline-block;padding:4px 7px;font-size:11px;border:1px solid var(--line-strong);border-radius:4px}.bridge-status.fail{color:var(--fail);border-color:var(--fail)}.bridge-status.unknown,.bridge-status.partial{color:var(--warn)}.bridge-table blockquote{font-size:12px}
.care-footer{max-width:1280px;margin:36px auto;padding:24px;font-size:12px}.care-view [hidden],.share-downloads[hidden]{display:none!important}
@media(max-width:1000px){.care-hero{gap:30px;grid-template-columns:minmax(0,1.5fr) minmax(240px,1fr)}.hero-note{padding:22px}.care-layout{grid-template-columns:minmax(0,1fr)}.care-view .care-saved{border-left:0;border-top:1px solid var(--line);padding:24px 0}.care-heading h1{font-size:48px}.topbar .tagline{display:none}}
@media(max-width:640px){.topbar{padding:16px 20px}.brand-row{gap:12px}.brand{font-size:20px}.topbar .mode{font-size:10px;max-width:150px}.views{flex-wrap:wrap}.views .view-tab{font-size:12px;padding:8px 12px}.care-view{padding:0 18px}.care-hero{grid-template-columns:1fr;gap:26px;padding-top:34px}.care-heading h1{font-size:41px;letter-spacing:-1.2px}.care-hero .library-lede{font-size:15px}.hero-note{transform:none;box-shadow:5px 6px 0 var(--surface-2)}.hero-note ol{display:grid;grid-template-columns:1fr;gap:8px;margin:10px 0}.hero-note li{margin:8px 0}.care-view .handoff-card{padding:19px}.care-start .row{display:grid;grid-template-columns:1fr 1fr;align-items:stretch;width:100%}.care-field-grid,.consultation-fields{grid-template-columns:1fr}.care-view .consultation-facts{grid-template-columns:1fr}.consultation-facts dt{font-size:12px}.consultation-facts dd{margin-bottom:8px}.care-output-actions{display:grid;grid-template-columns:1fr}.care-view .care-form-heading h2{font-size:25px}.selected-sharing{padding:14px}.care-view .consultation-product{padding:14px}.care-heading .kicker{font-size:10px}.hero-principles{gap:8px 13px}.care-view .question-study{padding:12px}.care-view .question-study>summary{font-size:16px}}
@media(prefers-reduced-motion:reduce){*{scroll-behavior:auto!important;animation:none!important;transition:none!important}}
@media print{.topbar,.care-start,.care-editor,.care-saved,.care-scope,.care-hero,.care-footer,.care-output-actions,.selected-sharing form{display:none!important}.care-layout{display:block}.care-view{padding:0}.care-view .handoff-card{border:0;padding:0}.care-view button{display:none!important}}

/* Readability is the default, including secondary information. */
.care-hero{position:relative;gap:42px;padding-top:40px;padding-bottom:32px}
.care-hero-copy{position:relative;z-index:1}.care-hero-copy:before{content:'';position:absolute;width:330px;height:330px;border-radius:50%;background:radial-gradient(circle,#e5edda 0,transparent 69%);left:-110px;top:-40px;z-index:-1;pointer-events:none}
.care-illustration{display:block;width:100%;max-width:330px;height:150px;margin:14px auto 0}.hero-note{background:linear-gradient(145deg,var(--surface),var(--surface-2));transform:rotate(1deg);border-radius:16px 16px 44px 16px}.hero-note ol{margin:10px 0 16px}.hero-note li{margin:14px 0}.hero-note strong{font-size:18px}.hero-note li span{font-size:16px;line-height:1.55}.hero-note>p{font-size:19px}.note-number{font:16px/1.4 var(--serif);letter-spacing:0;color:var(--ink)}
.hero-actions{display:flex;align-items:center;gap:18px;flex-wrap:wrap;margin-top:24px}.hero-actions button{padding:16px 23px;font-size:17px;font-weight:600;display:flex;align-items:center;gap:25px;background:var(--button);color:var(--button-ink);border-color:var(--button);border-radius:7px;box-shadow:0 4px 0 #183f31}.hero-actions button span{font-size:23px;line-height:1}.hero-actions>span{font-size:15px;color:var(--muted)}
.care-heading .kicker{font-size:13px;letter-spacing:.055em}.hero-principles{font-size:14px;gap:8px 16px}.care-hero .library-lede{font-size:19px;line-height:1.7}.care-heading .library-lede{font-size:18px}
.topbar .tagline{font-size:14px}.views .view-tab{font-size:16px}.care-start strong{font-size:18px}.care-start p,.care-start button,.care-scope{font-size:15px}.care-view .hint{font-size:15px;line-height:1.7}.care-view label{font-size:16px}.care-view input,.care-view textarea,.care-view select{font-size:17px;min-height:48px}.care-view button,.care-view .button-link{font-size:16px;min-height:48px}.care-view summary{font-size:16px;line-height:1.6}.care-view .care-steps button{font-size:16px}.care-view .pill{font-size:14px}.care-view .consultation-summary{font-size:16px}.care-view .consultation-facts{font-size:16px}.care-view .care-saved-item p{font-size:14px}.care-view .care-saved h2{font-size:25px}.care-view .handoff-list-item{font-size:16px}
.care-view .question-check>strong,.care-view .question-check>p,.care-view .question-finding strong{font-size:16px}.care-view .question-study>summary{font-size:22px}.care-view .question-criteria legend,.care-view .consultation-product legend{font-size:16px}.care-view .question-starters button{font-size:15px;min-height:44px}.clarification-task>summary{font-size:16px}.care-view .selected-sharing legend{font-size:15px}.care-view .share-choice input{min-height:20px}.bridge-table{font-size:15px;min-width:620px}.bridge-table td p,.bridge-table blockquote{font-size:15px}.bridge-status{font-size:13px}.care-view .bridge-table .hint{font-size:14px}.reading-progress{padding:18px 20px;background:var(--surface-2);border-left:4px solid var(--identity);border-radius:6px;margin:20px 0}.reading-progress>strong{font-size:18px}.care-view .reading-progress p{margin:6px 0}
@media(max-width:640px){.care-view{font-size:17px}.care-heading h1{font-size:42px}.care-heading .kicker{font-size:12px}.care-hero .library-lede{font-size:18px}.hero-actions{gap:12px}.hero-actions button{width:100%;justify-content:space-between}.hero-principles{font-size:14px}.hero-note{transform:none;padding:22px}.care-illustration{height:130px}.views .view-tab{font-size:14px}.care-view .care-steps button{font-size:14px}.care-view .consultation-facts dt{font-size:14px}.care-view .question-study>summary{font-size:19px}.care-start .row{grid-template-columns:1fr}.care-view .share-choice{font-size:16px}}
.care-view .pill{white-space:normal;max-width:100%;overflow-wrap:anywhere;display:inline-block;line-height:1.6}
.topbar .mode{font-size:13px;white-space:normal;line-height:1.5}
.evidence-metrics{display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:12px;margin:24px 0 12px}.evidence-metrics>div{display:flex;flex-direction:column-reverse;align-items:flex-start;gap:8px;padding:18px;background:var(--surface);border:1px solid var(--line);border-radius:8px}.evidence-metrics dt{font-size:15px;line-height:1.45;color:var(--muted)}.evidence-metrics dd{margin:0;font:38px/1 var(--serif);color:var(--identity)}.care-view .question-overview h3{font-size:25px;line-height:1.3;margin:8px 0 14px}.care-view .question-finding h4{font-size:22px;line-height:1.4}.reading-selection{margin:12px 0}.reading-selection summary{font-weight:500}.care-view .question-overview .question-finding{padding:22px;margin-top:20px}
@media(max-width:640px){.evidence-metrics{gap:6px}.evidence-metrics>div{padding:12px 9px}.evidence-metrics dd{font-size:29px}.evidence-metrics dt{font-size:14px}.care-view .question-overview h3{font-size:23px}.care-view .question-overview .question-finding{padding:16px}}
```

frontend/consultation.js
SHA-256: 6d4e5dca029dbd46f62a3c6be9f53168b023675b874b52c4b060bef175d95c89

```
import {clarificationPanel,sharePanel} from './care-tools.js';
export function initConsultation({api,el,link,openQuestion,isOffline=()=>true}) {
  const $=s=>document.querySelector(s),key='foliatrace.consultation.draft.v1';
  let step=1;
  const atStep=(node,steps)=>{node.dataset.careStep=String(steps);return node;};
  function showStep(next){step=next;for(const node of form.querySelectorAll('[data-care-step]'))node.hidden=!node.dataset.careStep.split(',').includes(String(step));for(const button of form.querySelectorAll('[data-step-go]')){button.setAttribute('aria-current',Number(button.dataset.stepGo)===step?'step':'false');}form.querySelector('.care-step-heading').textContent=['1 \xb7 Find or enter your product','2 \xb7 Tell us how you use it','3 \xb7 Prepare your pharmacist handoff'][step-1];form.querySelector('.care-step-back').hidden=step===1;form.querySelector('.care-step-next').hidden=step===3;}
  function nextStep(){const inputs=[...form.querySelectorAll('input,textarea,select')].filter(input=>input.required&&input.closest('[data-care-step]')&&!input.closest('[data-care-step]').hidden);for(const input of inputs)if(!input.reportValidity()){reveal(input);return;}showStep(Math.min(3,step+1));reveal(form.querySelector('.care-step-heading'));}
  let form,products=[],active=null,dirty=false,listSerial=0,draftSerial=0,saveSerial=0,selectionSerial=0,undoDraft=null;
  async function saveClarification(answer,record){
    if(dirty||active?.id!==record.id)throw new Error('Save your changed draft or reopen this saved version before recording a clarification');
    const at=draftSerial,selected=selectionSerial;
    const next=await api('/api/consultations/'+record.id+'/clarifications',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(answer)});
    if(at!==draftSerial||selected!==selectionSerial||active?.id!==record.id){await refresh();throw new Error('The note was saved in a separate snapshot; your current draft is preserved. Open the saved snapshot to continue');}
    renderPacket(next);dirty=false;const task=[...$('#consultation-packet').querySelectorAll('.clarification-task')].find(node=>node.querySelector('summary').textContent.includes(answer.field==='identity'?'ingredient label':answer.field));if(task){task.open=true;reveal(task.querySelector('summary'));}else reveal($('#consultation-packet'));message('Clarification saved with its source. Original fields and earlier snapshots are preserved.');await refresh();
  }
  const message=(text,bad=false)=>{$('#consultation-status').textContent=text;$('#consultation-status').className=bad?'error':'hint';};
  function reveal(target){for(let node=target.parentElement;node;node=node.parentElement)if(node.tagName==='DETAILS')node.open=true;target.scrollIntoView({block:'start',behavior:'auto'});target.focus({preventScroll:true});}
  function readSaved(){if(!dirty)$('#consultation-editor').open=false;reveal($('#consultation-packet'));}
  let savedItems=[],savedLimit=10;
  function filterSaved(){const term=$('#consultation-saved-filter').value.trim().toLocaleLowerCase();let count=0;for(const row of savedItems){const match=row.text.includes(term);row.node.hidden=!match||count>=savedLimit;if(match)count++;}const empty=$('#consultation-saved-empty');if(empty){empty.textContent=count?Math.min(count,savedLimit)+' of '+count+' saved records shown':term?'No saved visit label matches. Your draft and other records are kept.':'No saved preparations yet. Start with your draft.';}const more=$('#consultation-saved-more');if(more)more.hidden=count<=savedLimit;}
  $('#consultation-saved-filter').oninput=()=>{savedLimit=10;filterSaved();};
  $('#consultation-open-saved').onclick=()=>reveal($('#consultation-saved-filter'));
  function field(parent,label,name,value='',type='input'){
    const wrap=el('label',label),input=el(type);input.name=name;input.value=value;input.maxLength=type==='textarea'?5000:200;if(type==='textarea')input.rows=3;wrap.append(input);parent.append(wrap);return input;
  }
  function choices(parent,label,name,options,value){const input=field(parent,label,name,value,'select');for(const [v,text] of options){const opt=el('option',text);opt.value=v;input.append(opt);}input.value=value;return input;}
  function labelAmount(q){return q.amount_status==='NOT_PROVIDED'?'Individual amount not provided':(q.operator&&q.operator!=='='?q.operator+' ':'')+q.quantity+' '+q.unit+' per '+(q.servingSizeQuantity??'stated')+' '+(q.servingSizeUnit||'label serving');}
  function labelForms(row){return (row.forms||[]).map(f=>[f.prefix,f.name].filter(Boolean).join(' ')).join('; ');}
  function labelServings(raw){const rows=raw.servingSizes;if(!Array.isArray(rows)||!rows.length)return 'Not stated in this record';return rows.map(row=>{if(!row||typeof row!=='object')return 'Serving details not stated';const min=row.minQuantity,max=row.maxQuantity,amount=min==null?(max==null?'Quantity not stated':'up to '+max):max==null||min===max?String(min):min+'\u2013'+max;return amount+' '+(row.unit||'(unit not stated)');}).join('; ');}
  function current(){const data=Object.fromEntries(new FormData(form));return {visit_label:data.visit_label,prepared_by:data.prepared_by,other_medicines_status:data.other_medicines_status,other_medicines:data.other_medicines,allergies_status:data.allergies_status,allergies:data.allergies,questions_for_professional:data.questions_for_professional,products:products.map(item=>({...Object.fromEntries(Object.entries(item.fields).map(([k,v])=>[k,v.value])),...(item.label?{label_snapshot:item.label}:{})}))};}
  function remember(){draftSerial++;dirty=true;try{localStorage.setItem(key,JSON.stringify(current()));}catch{message('Browser draft storage is unavailable. Keep this page open and save to the local service before leaving.',true);return;}if(active)message('Draft changed. The saved record below keeps its earlier wording. Save a new snapshot before attaching new evidence.');}
```

```
  function keepUndo(){if(!form)return;undoDraft=current();$('#consultation-undo-draft').hidden=false;try{localStorage.setItem(key+'.undo',JSON.stringify(undoDraft));}catch{}}
  $('#consultation-undo-draft').onclick=()=>{if(!undoDraft)return;const restore=undoDraft;undoDraft=current();build(restore);remember();message('Previous draft restored. The saved snapshots remain unchanged.');try{localStorage.setItem(key+'.undo',JSON.stringify(undoDraft));}catch{}};
  function addProduct(value={}){
    const box=el('fieldset',undefined,'consultation-product'),legend=el('legend','Product '+(products.length+1)+(value.name?' \xb7 '+value.name:''));box.append(legend);
    const fields={};fields.item_id=el('input');fields.item_id.type='hidden';fields.item_id.value=value.item_id||crypto.randomUUID().replaceAll('-','');box.append(fields.item_id);
    const namePart=atStep(el('div'),1);box.append(namePart);
    fields.name=field(namePart,'Product name as written or reported (required)','product_name',value.name||'');fields.name.required=true;fields.name.minLength=2;
    fields.name.addEventListener('input',()=>{legend.textContent='Product '+(products.findIndex(p=>p.fields===fields)+1)+(fields.name.value?' \xb7 '+fields.name.value:'');});
    const sourceBox=atStep(el('details',undefined,'consultation-label-source'),1);sourceBox.append(el('summary','Label wording, brand and product form'));
    fields.source_kind=choices(sourceBox,'Where did this information come from?','source_kind',[['PERSON_REPORTED','Person or caregiver reported it'],['PRODUCT_LABEL','Copied from the product label']],value.source_kind||'PERSON_REPORTED');
    const usePart=atStep(el('div',undefined,'care-use-questions'),2);box.append(usePart);
    fields.use_status=choices(usePart,'Is the product actually being used?','use_status',[['NOT_ESTABLISHED','Not established yet'],['REPORTED_CURRENT_USE','The person reports using it now'],['REPORTED_PAST_USE','The person reports past use']],value.use_status||'NOT_ESTABLISHED');
    fields.source_text=field(sourceBox,'Exact ingredient label or original note','source_text',value.source_text||'','textarea');
    const grid=el('div',undefined,'consultation-fields');usePart.append(grid);
    for(const [name,label] of [['dose','How much do you take each time?'],['frequency','How often do you take it?']])fields[name]=field(grid,label,name,value[name]||'');
    for(const [name,label] of [['preparation','Form stated on label'],['brand','Brand / manufacturer']])fields[name]=field(sourceBox===box?grid:sourceBox,label,name,value[name]||'');
    const more=el('details',undefined,'care-advanced');more.append(el('summary','More use details, if known'));usePart.append(more);for(const [name,label] of [['route','How it is used'],['last_taken','Last used']])fields[name]=field(more,label,name,value[name]||'');
    fields.reason_for_use=field(more,'What the person says they use it for','reason_for_use',value.reason_for_use||'');
    const advanced=el('details');advanced.append(el('summary','Check a specific plant name inside the label'));fields.plant_wording=field(advanced,'Exact plant wording to look up','plant_wording',value.plant_wording||'');advanced.append(el('p','Use wording that occurs in the product name or recorded label. Name candidates do not establish the actual ingredients.','hint'));sourceBox.append(advanced);if(sourceBox!==box)box.append(sourceBox);
    usePart.append(el('p','Not sure? Leave it blank and bring the packaging.','hint'));
    const entry={box,fields,label:value.label_snapshot||null};
    if(entry.label){
      const info=el('div',undefined,'label-attachment');info.append(el('p','NIH label '+entry.label.label_id+' \xb7 source snapshot attached; bottle match not established. Label directions have not filled actual use.','pill warn'));
      for(const key of ['name','brand','source_text'])fields[key].readOnly=true;fields.source_kind.disabled=true;
      const detach=el('button','Detach label snapshot to edit copied wording','quiet');detach.type='button';detach.onclick=()=>{entry.label=null;for(const key of ['name','brand','source_text'])fields[key].readOnly=false;fields.source_kind.disabled=false;if(sourceBox!==box)sourceBox.open=true;fields.source_text.focus();info.remove();remember();};info.append(detach);box.append(info);
    }
    const remove=el('button','Remove this draft item','quiet');remove.type='button';remove.onclick=()=>{keepUndo();products=products.filter(p=>p.fields!==fields);box.remove();products.forEach((p,i)=>p.box.querySelector('legend').textContent='Product '+(i+1));remember();message('Draft item removed. Undo can restore it; saved snapshots remain.');};box.append(remove);$('#consultation-products').append(box);products.push(entry);return fields;
  }
  function labelLookup(parent){
    const lookup=atStep(el('details',undefined,'label-lookup'),1);lookup.open=false;lookup.append(el('summary','Search a product label by name or barcode'));
    lookup.append(el('p','Use the product name, brand or barcode only. Live lookup sends this separate search to NIH; consultation details are not sent. Compare the version with the actual bottle.','hint'));
    const mode=choices(lookup,'Label source','label_lookup_mode',[['live','Live NIH database \xb7 US products'],['reference','Six fixed reference labels \xb7 offline examples']],isOffline()?'reference':'live');
    const market=choices(lookup,'Market status in NIH records','label_lookup_market',[['all','All versions, including off-market labels'],['on','On-market records'],['off','Off-market records']], 'all');
    const query=field(lookup,'Product, brand or barcode','label_lookup_query');query.maxLength=120;query.placeholder='e.g. Peppermint Spirits';
    const search=el('button','Find label records','quiet');search.type='button';const status=el('p','','hint');status.setAttribute('role','status');status.setAttribute('aria-live','polite');const results=el('div',undefined,'label-results');lookup.append(search,status,results);parent.append(lookup);
    let requestSerial=0,offset=0,pinned=null;const ownForm=form;
```

```
    const comparison=el('div',undefined,'label-comparison');lookup.append(comparison);
    const cancel=()=>{requestSerial++;offset=0;search.disabled=false;results.replaceChildren();status.textContent='Search changed. Run again to view matching label records.';};query.oninput=cancel;mode.onchange=cancel;market.onchange=cancel;
    query.addEventListener('keydown',event=>{if(event.key==='Enter'){event.preventDefault();search.click();}});
    const find=async(start=0)=>{
      const term=query.value.trim();if(term.length<2){status.textContent='Enter at least two characters. Manual entry remains available below.';return;}
      const serial=++requestSerial,selectedMode=mode.value;offset=start;search.disabled=true;status.textContent='Finding label records\u2026';results.replaceChildren();
      try{
        const response=await api('/api/product-labels?q='+encodeURIComponent(term)+'&mode='+selectedMode+'&offset='+start+'&market='+market.value);if(serial!==requestSerial||form!==ownForm)return;
        status.textContent=response.coverage+(response.source?.retrieved_at?' Cached/acquired at '+response.source.retrieved_at+'.':'')+(response.labels.length?' Choose a record to inspect its version and formula.':' No matching label in this source. Try another wording or enter the label manually below.');
        for(const summary of response.labels){
          const card=el('article',undefined,'label-result');card.append(el('h3',summary.name),el('p',summary.brand+' \xb7 NIH '+summary.label_id+' \xb7 entered '+(summary.entry_date||'not stated')+' \xb7 '+(summary.off_market===1?'off-market record':summary.off_market===0?'on-market record':'market status not stated')));
          card.append(el('p','Barcode: '+(summary.upc||'not stated')+' \xb7 version: '+(summary.version||'not stated')+' \xb7 form: '+(summary.form||'not stated'),'hint'));
          const inspect=el('button','Inspect this label','quiet');inspect.type='button';const detail=el('div');card.append(inspect,detail);results.append(card);
          inspect.onclick=async()=>{inspect.disabled=true;detail.textContent='Loading the original label record\u2026';try{
            const record=await api('/api/product-labels/'+summary.label_id+'?mode='+selectedMode);if(serial!==requestSerial||form!==ownForm)return;
            const raw=record.snapshot.raw;detail.replaceChildren(el('p','This is a database record, not confirmation of the bottle or its contents. Compare the barcode, date and formula before adding.','pill warn'),link('Open original NIH record \u2197','https://dsld.od.nih.gov/label/'+summary.label_id));
            detail.append(el('p','Label serving size: '+labelServings(raw)+'. This is not the amount actually used.','hint'));const provenance=el('details',undefined,'care-advanced');provenance.append(el('summary','Source record and acquisition details'),el('p',record.snapshot.access.replaceAll('_',' ')+' \xb7 acquired '+record.snapshot.retrieved_at,'hint'),el('p','Source SHA256: '+record.snapshot.sha256,'hint'));detail.append(provenance);
            for(const ingredient of record.ingredients){const row=el('div',undefined,'label-ingredient');row.append(el('strong',ingredient.name),el('p',ingredient.parent_key?'Part of '+(record.ingredients.find(r=>r.key===ingredient.parent_key)?.name||'a declared blend'):'Top-level ingredient','hint'));if(ingredient.research_lookup)row.append(el('p','Reading uses the original ingredient wording: '+ingredient.research_lookup+'. The source group remains '+ingredient.lookup+'; species is not established.','hint'));for(const q of ingredient.quantities)row.append(el('p',labelAmount(q)));if(labelForms(ingredient))row.append(el('p','Declared forms: '+labelForms(ingredient)));if(ingredient.notes)row.append(el('blockquote',ingredient.notes));detail.append(row);}
            const other=raw.otheringredients||{};detail.append(el('h4','Other declared ingredients'),el('p',other.text||((other.ingredients||[]).map(i=>i.name).join('; '))||'Not provided in this label record.'));
            const pin=el('button','Use as comparison reference','quiet');pin.type='button';pin.onclick=()=>{pinned=record.snapshot;comparison.replaceChildren(el('p','Comparison reference: '+raw.fullName+' \xb7 NIH '+record.snapshot.label_id+'. Inspect another record and compare its declared ingredients.','hint'));};
            const compare=el('button','Compare with reference','quiet');compare.type='button';compare.onclick=async()=>{if(!pinned){comparison.replaceChildren(el('p','Choose a comparison reference from one inspected label first.','hint'));return;}compare.disabled=true;const left=pinned;try{
              const result=await api('/api/product-labels/compare',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({left,right:record.snapshot})});if(serial!==requestSerial||form!==ownForm||pinned!==left)return;
              comparison.replaceChildren(el('h3','NIH '+left.label_id+' compared with NIH '+record.snapshot.label_id),el('p',result.ingredient_changes.length+' ingredient groups have different declared data. No preferred or clinically equivalent version is inferred.','pill warn'));
              for(const change of result.ingredient_changes){const group=el('section');group.append(el('h4',change.ingredient_path.join(' \u2192 ')));
                for(const [side,title] of [['before','Reference'],['after','Compared record']]){group.append(el('strong',title));if(!change[side].length)group.append(el('p','Not listed in this record.'));for(const row of change[side]){group.append(el('p',row.declared.name),el('p','Declared category: '+row.declared.category),el('blockquote',row.declared.notes||'No ingredient notes'));if(labelForms(row.declared))group.append(el('p','Declared forms: '+labelForms(row.declared)));for(const q of row.declared.quantities)group.append(el('p',labelAmount(q)));}}
                comparison.append(group);
              }
              for(const change of result.label_field_changes){const group=el('details');group.append(el('summary','Changed label field: '+change.field),el('p','Reference: '+JSON.stringify(change.before)),el('p','Compared record: '+JSON.stringify(change.after)));comparison.append(group);}
              for(const limit of result.limits)comparison.append(el('p',limit,'hint'));
            }catch(error){comparison.append(el('p','Comparison unavailable: '+error.message+'. Both original records and your draft remain.','error'));}finally{compare.disabled=false;}};detail.append(pin,compare);
```

```
            const add=el('button','Add this specific label as a separate product','quiet');add.type='button';add.onclick=()=>{if(serial!==requestSerial||form!==ownForm)return;
              // An untouched initial placeholder is replaced; meaningful draft inputs always remain.
              const empty=products.find(p=>!p.label&&Object.entries(p.fields).every(([k,v])=>['item_id','source_kind','use_status'].includes(k)||!v.value.trim()));if(empty){products=products.filter(p=>p!==empty);empty.box.remove();}
              const fields=addProduct({name:raw.fullName,brand:raw.brandName||'',source_kind:'PRODUCT_LABEL',source_text:record.source_text,preparation:record.summary.form||'',label_snapshot:record.snapshot});remember();add.disabled=true;add.textContent='Added this label version';lookup.open=false;showStep(2);reveal(fields.use_status);message('Label version added. Actual amount, frequency and use status remain for the person to report.');};detail.append(add);
          }catch(error){if(serial===requestSerial&&form===ownForm)detail.textContent='Could not load: '+error.message+'. Existing draft remains.';}finally{inspect.disabled=false;}};
        }
        const pages=el('div',undefined,'row');const back=el('button','Previous label matches','quiet');back.type='button';back.disabled=start===0;back.onclick=()=>find(Math.max(0,start-10));const next=el('button','Next label matches','quiet');next.type='button';next.disabled=!response.has_more;next.onclick=()=>find(start+10);pages.append(back,el('span','Matches '+(response.labels.length?start+1:0)+'\u2013'+(start+response.labels.length)),next);results.append(pages);
      }catch(error){if(serial===requestSerial&&form===ownForm)status.textContent='Search unavailable: '+error.message+'. Keep your draft; use the reference labels or enter the product manually.';}finally{if(serial===requestSerial)search.disabled=false;}
    };
    search.onclick=()=>find(0);
  }
  function build(value=null){
    draftSerial++;saveSerial++;selectionSerial++;
    $('#consultation-editor').open=true;
    const box=$('#consultation-compose');box.replaceChildren();form=el('form');products=[];const ownForm=form;
    const v=value||{};
    const heading=el('div',undefined,'care-form-heading');const title=el('h2','1 \xb7 Find or enter your product','care-step-heading');title.tabIndex=-1;heading.append(title,el('p','Keep the bottle nearby. You can leave unknown details blank.','hint'));form.append(heading);const progress=el('nav',undefined,'care-steps');progress.setAttribute('aria-label','Preparation steps');for(const [number,text] of [[1,'Product'],[2,'Actual use'],[3,'Handoff']]){const button=el('button',number+' \xb7 '+text,'quiet');button.type='button';button.dataset.stepGo=number;button.onclick=()=>{if(number>step){nextStep();return;}showStep(number);};progress.append(button);}form.append(progress);
    const meta=atStep(el('div',undefined,'care-field-grid'),3);
    const label=field(meta,'Visit or conversation label (required)','visit_label',v.visit_label||'Pharmacist conversation');label.required=true;label.maxLength=160;label.placeholder='e.g. Friday pharmacy visit \u2014 no patient name needed';
    choices(meta,'Prepared by','prepared_by',[['PATIENT_OR_CARER','Patient or caregiver'],['HEALTHCARE_PROFESSIONAL','Healthcare professional']],v.prepared_by||'PATIENT_OR_CARER');
    const productsHeading=el('p','Search a label above or enter the name below. Bring the packaging to the visit.','hint');atStep(productsHeading,1);form.append(productsHeading);
    labelLookup(form);if(v.products?.some(p=>p.name))form.querySelector('.label-lookup').open=false;
    const rows=atStep(el('div'),'1,2');rows.id='consultation-products';form.append(rows);
    const add=el('button','Add another product','quiet');add.type='button';add.onclick=()=>{const fields=addProduct();showStep(1);remember();reveal(fields.name);};form.append(add);
    atStep(add,'1,2');const others=atStep(el('details',undefined,'care-section care-advanced'),3);others.open=Boolean(v.other_medicines||v.allergies);others.append(el('summary','Other medicines and allergies, if known'),el('p','Keep unknown details blank; record \u201cnone\u201d only when explicitly reported.','hint'));
    choices(others,'Other prescription / over-the-counter medicines','other_medicines_status',[['NOT_PROVIDED','Information not provided yet'],['NONE_REPORTED','The person explicitly reports none'],['LIST_PROVIDED','A list is provided below']],v.other_medicines_status||'NOT_PROVIDED');
    const medicine=field(others,'Exact medicine list, including stated amounts and frequency','other_medicines',v.other_medicines||'','textarea');medicine.maxLength=6000;
    choices(others,'Allergies or previous reactions','allergies_status',[['NOT_PROVIDED','Information not provided yet'],['NONE_REPORTED','The person explicitly reports none'],['DETAILS_PROVIDED','Details are provided below']],v.allergies_status||'NOT_PROVIDED');
    const allergy=field(others,'Reported allergy / reaction details','allergies',v.allergies||'','textarea');allergy.maxLength=2000;form.append(others);
    for(const [statusName,input,provided] of [['other_medicines_status',medicine,'LIST_PROVIDED'],['allergies_status',allergy,'DETAILS_PROVIDED']]){const status=others.querySelector('[name="'+statusName+'"]');const sync=()=>{input.closest('label').hidden=status.value!==provided&&!input.value.trim();};status.addEventListener('change',sync);sync();}
    const finish=atStep(el('section',undefined,'care-section'),3);finish.append(el('h3','Your pharmacist conversation starter'),meta);field(finish,'Questions to bring to the pharmacist or clinician','questions_for_professional',v.questions_for_professional||'','textarea').maxLength=2000;
    const save=el('button','Save and prepare handoff');save.type='submit';const footer=el('div',undefined,'care-form-actions');footer.append(save,el('p','Saves a new
```

```
 local record. Earlier saved versions are kept.','hint'));finish.append(footer);form.append(finish);box.append(form);
130     for(const product of v.products||[{}])addProduct(product);const navigation=el('div',undefined,'care-step-actions');const back=el('button','Back','quiet care-step-back');back.type='button';back.onclick=()=>showStep(Math.max(1,step-1));const next=el('button','Continue','care-step-next');next.type='button';next.onclick=nextStep;navigation.append(back,next);form.append(navigation);showStep(1);
131     const edited=event=>{if(!event.target.closest('.label-lookup'))remember();};form.addEventListener('input',edited);form.addEventListener('change',edited);
132     form.addEventListener('invalid',event=>{const part=event.target.closest('[data-care-step]');if(part)showStep(Number(part.dataset.careStep.split(',')[0]));reveal(event.target);},true);
133     form.addEventListener('submit',async event=>{event.preventDefault();if(step<3){nextStep();return;}if(!products.length){message('Add at least one product. Existing saved records are unchanged.',true);reveal(add);return;}selectionSerial++;const serial=++saveSerial,draftAtSave=draftSerial;save.disabled=true;save.textContent='Saving your preparation\u2026';ownForm.setAttribute('aria-busy','true');message('Keeping your exact wording and checking which details need clarification\u2026');const request=current();try{const result=await api('/api/consultations',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(request)});if(serial!==saveSerial){await refresh();return;}active=result;dirty=draftAtSave!==draftSerial;renderPacket(result);if(!dirty)readSaved();message(dirty?'The earlier input was saved; your newer draft is still in the form. Save again before attaching evidence.':'Saved as a new snapshot. Bring the original products and this preparation record to the professional.');await refresh();}catch(error){if(serial===saveSerial)message('Not saved: '+error.message+'. Your draft remains in this form.',true);}finally{save.disabled=false;save.textContent='Save and prepare handoff';ownForm.removeAttribute('aria-busy');}});
134   }
135   function renderPacket(record){
136     active=record;const p=record.packet,box=$('#consultation-packet');box.hidden=false;box.replaceChildren(el('h2',p.intake.visit_label),el('p',record.origin==='LOCAL_CONSULTATION_PREPARATION'?'Saved consultation preparation \xb7 reported information':'Imported preparation \xb7 source and clinical content unverified','pill warn'));
137     const actions=el('div',undefined,'care-output-actions');const printable=el('button','Choose what to share','care-primary-link');printable.type='button';printable.onclick=()=>{const panel=box.querySelector('.selected-sharing');panel.open=true;reveal(panel.querySelector('summary'));};actions.append(printable);const downloads=el('details',undefined,'care-downloads');downloads.append(el('summary','Full record and backup'),el('p','These files contain the full saved record. Use selective sharing for your pharmacist.','hint'));for(const [format,text] of [['summary','Download concise handoff'],['markdown','Download the conversation record'],['json','Download portable backup']]){const a=el('a',text);a.href='/api/consultations/'+record.id+(format!=='json'?'?format='+format:'');a.download='foliatrace-consultation-'+record.id+(format==='json'?'.json':'.md');downloads.append(a);}actions.append(downloads);const copy=el('button','Use this record as a new draft','quiet');copy.type='button';copy.onclick=()=>{keepUndo();build(record.draft_intake||p.intake);remember();dirty=Boolean(record.draft_intake);showStep(3);reveal(form.querySelector('[name="visit_label"]'));message(record.draft_notice||'Copied into the form. Save creates a separate snapshot; existing records and evidence stay preserved. Undo can restore your previous draft.');};actions.append(copy);const resume=el('button','Return to my current draft','quiet');resume.type='button';resume.onclick=()=>{showStep(step);reveal(form.querySelector('.care-step-heading'));};actions.append(resume);box.append(actions);
138     box.append(el('p','Bring this list and the original packaging to your pharmacist.','hint'));
139     const summary=el('div',undefined,'consultation-summary');summary.append(el('strong',p.summary.products+' '+(p.summary.products===1?'product':'products')),el('span',p.summary.missing_reported_fields+' use details not provided'),el('span',p.summary.products_with_several_candidates+' '+(p.summary.products_with_several_candidates===1?'name has':'names have')+' several possible identities'));box.append(summary);
140     for(const item of p.products){
141       const r=item.reported,card=el('article',undefined,'consultation-product-reading');card.append(el('h3',r.name),el('p',(r.use_status||'NOT_ESTABLISHED').replaceAll('_',' ').toLowerCase(),'pill warn'),el('p',r.source_kind==='PRODUCT_LABEL'?'Copied label wording':'Person-reported wording','hint'),el('blockquote',r.source_text||'No original label or note was provided.'));
142       const facts=el('dl',undefined,'consultation-facts');for(const [name,title] of [['dose','Amount per use'],['frequency','Frequency'],['route','Route'],['last_taken','Last use'],['preparation','Stated form'],['brand','Brand']])facts.append(el('dt',title),el('dd',r[name]||'Not provided'));card.append(facts);
143       if(r.reason_for_use)card.append(el('p','Reported reason: '+r.reason_for_use));
144       card.append(el('h4','Name candidates to check against the product'));
145       if(!item.candidates.length)card.append(el('p','No candidate established in the local name snapshot. Keep the original name and ask for the packaging.','hint'));
146       for(const candidate of item.candidates){const row=el('p');row.append(el('i',candidate.name),document.createTextNode(' \xb7 check against the label'));if(candidate.reference_url)row.append(document.createTextNode(' \xb7 '),link('Name reference \u2197',candidate.reference_url));card.append(row);}
147       card.append(clarificationPanel({record,item,api,el,onSave:saveClarification}));
148       const linked=p.linked_evidence.filter(l=>l.item_id===r.item_id);if(linked.length)card.append(el('p',linked.length+' professional evidence reading(s) attached. These are not safety clearances.','hint'));
149       for(const entry of linked){const button=el('button','Open evidence question: '+entry.review.packet.protocol.question,'quiet');button.type='button';button.onclick=()=>openQuestion({savedRecord:record.origin==='LOCAL_CONSULTATION_PREPARATION'?entry.review:{...entry.review,origin:'IMPORTED_UNVERIFIED_REVIEW'}});card.append(button);}
```

```
      const research=el('button','Professional: review evidence for this product','quiet');research.type='button';research.disabled=!item.candidates.length||record.origin!=='LOCAL_CONSULTATION_PREPARATION';research.onclick=()=>{if(dirty){message('Save the changed draft before opening an evidence review tied to this product.',true);return;}openQuestion({plant_name:item.name_lookup,preparation:r.preparation,source_text:r.source_text,question:'Which human studies address the reported use of '+r.name+'?',context:{consultation_id:record.id,item_id:r.item_id,item_sha256:item.item_sha256}});};card.append(research);box.append(card);
      if(r.label_snapshot){
        research.remove();card.append(el('h4','Label ingredients \xb7 separate evidence questions'),el('p','The full formula is retained. Evidence about one constituent does not establish an effect for the mixture or this product.','hint'));
        for(const ingredient of item.label_ingredients||[]){const row=el('div',undefined,'label-ingredient');row.append(el('strong',ingredient.name),el('p',ingredient.notes||'No ingredient notes provided.','hint'));
          if(ingredient.research_lookup)row.append(el('p','Reading uses the original ingredient wording: '+ingredient.research_lookup+'. The source group remains '+ingredient.lookup+'; species is not established.','hint'));
          for(const q of ingredient.quantities)row.append(el('p',labelAmount(q)));if(labelForms(ingredient))row.append(el('p','Declared forms: '+labelForms(ingredient)));
          const start=el('button','Professional: review '+ingredient.name,'quiet');start.type='button';start.disabled=!ingredient.candidates.length||record.origin!=='LOCAL_CONSULTATION_PREPARATION';start.onclick=()=>{if(dirty){message('Save the changed draft before opening an evidence review tied to this product.',true);return;}openQuestion({plant_name:ingredient.research_lookup||ingredient.lookup,source_text:ingredient.review_source_text,preparation:r.preparation,source_url:r.label_snapshot.source_url,question:'Which human studies address '+ingredient.name+' in the stated preparation?',context:{consultation_id:record.id,item_id:r.item_id,item_sha256:item.item_sha256,ingredient_key:ingredient.key}});};row.append(start);if(!ingredient.candidates.length)row.append(el('p',ingredient.plant_lookup_available?'No local name candidate; keep the original ingredient and check another source.':'This ingredient has no botanical literature lookup in this product. It remains in the formula.','hint'));card.append(row);}
        const other=r.label_snapshot.raw.otheringredients||{};card.append(el('h4','Other declared ingredients'),el('p',other.text||((other.ingredients||[]).map(i=>i.name).join('; '))||'Not provided in this label record.'));
      }
    }
    box.append(sharePanel({record,api,el}));
    box.append(el('h3','Other medicines'),el('p',p.intake.other_medicines_status.replaceAll('_',' ')),el('pre',p.intake.other_medicines||'No list provided.'),el('h3','Allergies / previous reactions'),el('p',p.intake.allergies_status.replaceAll('_',' ')),el('pre',p.intake.allergies||'No details provided.'));
    if(p.intake.questions_for_professional)box.append(el('h3','Questions to bring'),el('p',p.intake.questions_for_professional));
    for(const question of p.next_visit_questions)box.append(el('p',question,'hint'));
    const scope=el('details');scope.append(el('summary','Scope and workflow sources'));for(const limit of p.limits)scope.append(el('p',limit,'hint'));for(const source of p.workflow_sources)scope.append(link(source.title+' \u2197',source.url));box.append(scope);
  }
  async function refresh(){const serial=++listSerial;try{const result=await api('/api/consultations');if(serial!==listSerial)return;const box=$('#consultation-saved');box.replaceChildren();savedItems=[];for(const record of result.preparations){const row=el('div',undefined,'care-saved-item'),button=el('button',record.visit_label,'quiet handoff-list-item');button.type='button';button.onclick=async()=>{const selected=++selectionSerial;saveSerial++;try{const saved=await api('/api/consultations/'+record.id);if(selected!==selectionSerial)return;renderPacket(saved);dirty=false;readSaved();message('Opened the saved snapshot. Your current draft remains above; use the copy button to edit this record.');}catch(error){if(selected===selectionSerial)message(error.message,true);}};const date=new Date(record.saved_at);row.append(button,el('p',(Number.isNaN(date.getTime())?record.saved_at:date.toLocaleString())+' \xb7 '+(record.origin==='LOCAL_CONSULTATION_PREPARATION'?'local record':'unverified import'),'hint'));box.append(row);savedItems.push({node:row,text:record.visit_label.toLocaleLowerCase()});}const empty=el('p','','hint');empty.id='consultation-saved-empty';empty.setAttribute('role','status');box.append(empty);const more=el('button','Show 10 more saved records','quiet');more.type='button';more.id='consultation-saved-more';more.onclick=()=>{const term=$('#consultation-saved-filter').value.trim().toLocaleLowerCase(),first=savedItems.filter(r=>r.text.includes(term))[savedLimit];savedLimit+=10;filterSaved();first?.node.querySelector('button').focus();};box.append(more);filterSaved();}catch(error){message('Could not open saved records: '+error.message,true);}}
  $('#consultation-refresh').onclick=refresh;
  $('#consultation-example').onclick=()=>{try{if(form)localStorage.setItem(key+'.previous',JSON.stringify(current()));}catch{}build({visit_label:'Fictional example - preparation for a pharmacy conversation',products:[{name:'Mint product',plant_wording:'Mint',source_text:'Mint capsules. No botanical species is written in this fictional note.',dose:'one capsule',frequency:'once daily',route:'oral',reason_for_use:'The person says they use it for digestion'}]});dirty=true;$('#consultation-undo-example').hidden=false;message('Fictional example loaded; no real patient or clinical outcome is represented. Your earlier draft can be restored.');};
  $('#consultation-undo-example').onclick=()=>{try{const previous=JSON.parse(localStorage.getItem(key+'.previous')||'null');if(previous){build(previous);remember();message('Previous draft restored.');}}catch(error){message(error.message,true);}};
  $('#consultation-import').addEventListener('change',async event=>{const file=event.target.files[0];if(!file)return;if(file.size>20_000_000){message('The file exceeds 20 MB. Existing records are unchanged.',true);return;}const selected=++selectionSerial;saveSerial++;const body=new FormData();body.append('file',file);try{const record=await api('/api/consultations/import',{method:'POST',body});if(selected!==selectionSerial){await refresh();return;}renderPacket(record);dirty=true;readSaved();message('Imported as a separate unverified preparation. Your current draft and prior records remain.');await refresh();}catch(error){if(selected===selectionSerial)message('Import failed: '+error.message+'. Existing records
```

```
    remain.',true);}finally{event.target.value='';}});
171   return {open(){if(!form){let
    draft=null;try{draft=JSON.parse(localStorage.getItem(key)||'null');undoDraft=JSON.parse(localStorage.getItem(key+'.undo')||'null');$('#consultation-undo-
    draft').hidden=!undoDraft;}catch{}build(draft);}return refresh();},focusSaved(){if(active)readSaved();},async attach(context,reviewId){if(dirty)throw new Error('The
    consultation draft changed; save a current snapshot and open its evidence question first');if(active?.id!==context.consultation_id)throw new Error('This reading
    belongs to an earlier saved snapshot. Open its product record or start a reading from the current snapshot.');const
    draftAtStart=draftSerial,selected=selectionSerial;const record=await api('/api/consultations/'+context.consultation_id+'/evidence',{method:'POST',headers:{'Content-T
    ype':'application/json'},body:JSON.stringify({item_id:context.item_id,review_id:reviewId,...(context.ingredient_key?{ingredient_key:context.ingredient_key}:{})})});i
    f(draftAtStart!==draftSerial||selected!==selectionSerial){await refresh();throw new Error('The evidence was saved as a separate snapshot; your newer draft and
    current record were kept. Open the saved snapshot to continue.');}renderPacket(record);dirty=false;message('Evidence reading attached to a new consultation snapshot.
     No product safety conclusion was generated.');await refresh();return record;}};
172 }
173
```

frontend/demo/ai-worker.js
SHA-256: 1c7d25acf7e3459fd48fc153f26a2f2ddc933196ced9909e73f6468190a2b6a9

```
// On-device language understanding: embeds what a person typed (e.g. "my blood pressure pill") with
// all-MiniLM-L6-v2 and compares it with medicine-group descriptions. Nothing leaves the device.
import {medicineExclusion} from './medicine-language.js';
let runtime, session, tokenizer, groupVecs = null, groupKey = '';
const here = p => new URL(p, self.location).href;
const sha = async buf => [...new Uint8Array(await crypto.subtle.digest('SHA-256', buf))].map(b => b.toString(16).padStart(2, '0')).join('');

async function load() {
  if (session) return;
  const [ort, tok, manifest] = await Promise.all([import(here('inference/ort.wasm.min.mjs')), import(here('inference/tokenizers.mjs')), fetch(here('inference/embed-manifest.json')).then(r => r.json())]);
  const tbytes = await (await fetch(here('inference/embed-tokenizer.json'))).arrayBuffer();
  if (await sha(tbytes) !== manifest.tokenizer_sha256) throw Error('tokenizer checksum mismatch');
  const mbytes = await (await fetch(here('inference/embed-model.onnx'))).arrayBuffer();
  if (await sha(mbytes) !== manifest.model_sha256) throw Error('model checksum mismatch');
  runtime = ort; runtime.env.wasm.numThreads = 1; runtime.env.wasm.wasmPaths = here('inference/');
  tokenizer = new tok.Tokenizer(JSON.parse(new TextDecoder().decode(tbytes)), {});
  session = await runtime.InferenceSession.create(new Uint8Array(mbytes), {executionProviders: ['wasm']});
}

async function embed(text) {
  const ids = [101, ...tokenizer.encode(String(text).slice(0, 400), {add_special_tokens: false}).ids.slice(0, 254), 102];
  // Match Python's pinned tokenizer padding. Dynamic int8 quantisation is sensitive
  // to sequence shape, even when padding is masked from attention and pooling.
  const real = ids.length; while (ids.length < 128) ids.push(0);
  const n = ids.length;
  const feed = {input_ids: new runtime.Tensor('int64', BigInt64Array.from(ids.map(BigInt)), [1, n]),
    attention_mask: new runtime.Tensor('int64', BigInt64Array.from(ids.map((_, i) => i < real ? 1n : 0n)), [1, n]),
    token_type_ids: new runtime.Tensor('int64', new BigInt64Array(n), [1, n])};
  const out = await session.run(Object.fromEntries(session.inputNames.map(k => [k, feed[k]])));
  const h = out[session.outputNames[0]].data, d = h.length / n, v = new Float32Array(d);
  for (let t = 0; t < real; t++) for (let j = 0; j < d; j++) v[j] += h[t * d + j] / real;
  let norm = 0; for (const x of v) norm += x * x; norm = Math.sqrt(norm); for (let j = 0; j < d; j++) v[j] /= norm;
  Object.values(out).forEach(t => t.dispose?.()); Object.values(feed).forEach(t => t.dispose?.());
  return v;
}
const dot = (a, b) => { let s = 0; for (let i = 0; i < a.length; i++) s += a[i] * b[i]; return s; };

let queue = Promise.resolve();
self.onmessage = ({data}) => { queue = queue.then(() => handle(data)); };
async function handle(data) {
  const {id, type} = data;
  try {
    await load();
    if (type === 'classify') {
      const key = JSON.stringify(data.groups);
      if (groupKey !== key) { groupVecs = []; for (const g of data.groups) for (const p of [g.description, ...(g.prototypes || [])]) groupVecs.push([g.id, await embed(p)]); groupKey = key; }
      const results = [];
      for (const text of data.texts) {
        const v = await embed(text), scores = new Map();
        for (const [gid, gv] of groupVecs) scores.set(gid, Math.max(scores.get(gid) ?? -1, dot(v, gv)));
        const ranked = [...scores].sort((a, b) => b[1] - a[1]);
        const [best, score] = ranked[0], margin = score - (ranked[1]?.[1] ?? -1);
        const excluded = medicineExclusion(text);
```

```
        results.push({text, group: !excluded && score >= data.threshold && margin >= (data.margin || 0) ? best : null, best, score: Math.round(score * 1000) / 1000,
margin: Math.round(margin * 1000) / 1000, excluded});
      }
      postMessage({id, type: 'classified', results});
    }
  } catch (e) { postMessage({id, type: 'error', text: e.message}); }
}
```

frontend/demo/check.js
SHA-256: 195185cbdc46582a1ebf399c844833138025d218881e668cc56462d9a4de6299

```
import {indexKnowledge, parseLabelText, fromDsldLabel, analyze, resolveMedicines, norm} from './safety-engine.js';
import {EXAMPLES} from './safety-examples.js';
import {researchQueries} from './research-evidence.js';

const $ = s => document.querySelector(s);
const esc = v => String(v ?? '').replace(/[&<>"']/g, c => ({'&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'}[c]));
const el = (tag, text, cls) => { const n = document.createElement(tag); if (text !== undefined) n.textContent = text; if (cls) n.className = cls; return n; };
const DSLD = 'https://api.ods.od.nih.gov/dsld/v9/';
const STORE = 'foliatrace.check.v2';
let saveAllowed = !location.hash.startsWith('#brief=');

let kb, idx, result = null, ocrWorker = null, ocrPending = false;
const state = {products: [], medicines: [], conditions: [], example: null, aiMeds: {}, reason: ''};

function save() { if (!saveAllowed) return; try { localStorage.setItem(STORE, JSON.stringify({...state, aiVersion: 2})); } catch {} }
function cleanAi(a) {
  const out = {};
  for (const [k, v] of Object.entries(a && typeof a === 'object' ? a : {}).slice(0, 40)) {
    if (typeof k === 'string' && k.length <= 80 && v && typeof v === 'object') out[k] = {group: typeof v.group === 'string' && kb.ai_medicine_groups.groups.some(g => g.id === v.group) ? v.group : null, score: Number(v.score) || 0};
  }
  return out;
}
function load() {
  try {
    const s = JSON.parse(localStorage.getItem(STORE) || 'null');
    if (s && Array.isArray(s.products) && s.products.length <= 12 && Array.isArray(s.medicines) && s.medicines.length <= 40) {
      state.products = s.products.filter(p => p && typeof p.name === 'string' && Array.isArray(p.ingredients) && p.ingredients.length <= 120);
      state.medicines = s.medicines.filter(m => typeof m === 'string' && m.length <= 80);
      state.conditions = (s.conditions || []).filter(c => kb.conditions[c]);
      state.aiMeds = s.aiVersion === 2 ? cleanAi(s.aiMeds) : {}; state.reason = typeof s.reason === 'string' ? s.reason.slice(0, 120) : '';
    }
  } catch {}
}

// ------------------------------------------------------------ on-device AI
// 1. Language understanding (ai-worker.js, all-MiniLM-L6-v2): how a person describes a medicine.
// 2. Research ranking (model-search-worker.js, MiniLM cross-encoder): studies for each herb and reason.
function workerClient(file) {
  let w = null, seq = 0; const waiting = new Map();
  return (msg, onProgress) => {
    if (!w) {
      w = new Worker(new URL(file, location.href), {type: 'module'});
      w.onmessage = ({data}) => { const p = waiting.get(data.id); if (!p) return; if (data.type === 'progress') { p.onProgress?.(data.text); return; } waiting.delete(data.id); data.type === 'error' ? p.reject(Error(data.text)) : p.resolve(data); };
      w.onerror = () => { for (const p of waiting.values()) p.reject(Error('on-device AI could not start')); waiting.clear(); w = null; };
    }
    const id = ++seq; return new Promise((resolve, reject) => { waiting.set(id, {resolve, reject, onProgress}); w.postMessage({...msg, id}); });
  };
}
const understandAI = workerClient('ai-worker.js');
const researchAI = workerClient('model-search-worker.js');
let aiBusy = false, aiPending = Promise.resolve();
const aiInFlight = new Set();
function understandMedicines() {
```

```
  const todo = resolveMedicines(state.medicines, idx).filter(m => !m.known && !m.excluded && !(m.input in state.aiMeds) && !aiInFlight.has(m.input)).map(m => m.input);
  if (!todo.length) return aiPending;
  aiBusy = true; renderMeds();
  todo.forEach(t => aiInFlight.add(t));
  aiPending = aiPending.then(() => understandAI({type: 'classify', texts: todo, threshold: kb.ai_medicine_groups.threshold, margin: kb.ai_medicine_groups.margin,
    groups: kb.ai_medicine_groups.groups}))
    .then(({results}) => { let changed = false; for (const r of results) if (state.medicines.includes(r.text)) { state.aiMeds[r.text] = {group: r.group, score: r.score}; changed = true; } if (changed) save(); })
    .catch(() => { $('#run-hint').textContent = 'Medicine AI is unavailable. Enter the exact name, or keep it as a question for your pharmacist.'; })
    .finally(() => { todo.forEach(t => aiInFlight.delete(t)); aiBusy = aiInFlight.size > 0; renderMeds(); });
  return aiPending;
}

// ---------------------------------------------------------------- rendering
function renderExamples() {
  const box = $('#example-list'); box.replaceChildren();
  for (const ex of EXAMPLES) {
    const b = el('button', undefined, 'example'); b.type = 'button'; b.dataset.example = ex.id;
    const meds = ex.medicines.map(m => m.replace(/\s*\d+\s*mg/i, '')).join(' + ');
    b.innerHTML = `<span class="ex-name">${esc(ex.person)}</span><span class="ex-story">${esc(ex.story)}</span><span class="ex-meta">${esc(meds)} \xb7 ${ex.products.length} product${ex.products.length > 1 ? 's' : ''}</span><span class="ex-go" aria-hidden="true">Run check \u2192</span>`;
    b.setAttribute('aria-label', `Run the check for ${ex.person}: ${ex.story}`);
    b.onclick = () => runExample(ex.id);
    box.append(b);
  }
}

function runExample(id) {
  invalidateBrief();
  const ex = EXAMPLES.find(e => e.id === id);
  state.products = ex.products.map(p => ({...parseLabelText(p.text, p.name), fictional: true}));
  state.medicines = [...ex.medicines]; state.conditions = [...ex.conditions]; state.example = ex.id; state.reason = ex.reason || '';
  $('#reason-input').value = state.reason;
  save(); renderAll(); runCheck(true);
}

function renderProducts() {
  const box = $('#product-list'); box.replaceChildren();
  if (!state.products.length) { box.append(el('p', 'No product yet. Search NIH, paste the label, or photograph it.', 'small empty')); }
  for (const [i, p] of state.products.entries()) {
    const card = el('article', undefined, 'product-card');
    const head = el('div', undefined, 'product-head');
    head.append(el('strong', p.name));
    const src = p.source === 'dsld' ? `NIH label #${p.dsldId}${p.brand ? ' \xb7 ' + p.brand : ''}` : p.source === 'ocr' ? 'Read from your photo' : p.fictional ? 'Fictional example label' : 'Typed label text';
    head.append(el('span', src, 'badge ' + (p.source === 'dsld' ? 'badge-nih' : '')));
    const active = p.ingredients.filter(x => !x.other);
    card.append(head, el('p', active.slice(0, 6).map(x => x.name || x.text).join(' \xb7 ') + (active.length > 6 ? ` \xb7 +${active.length - 6} more` : ''), 'small ingredients-line'));
    const rm = el('button', 'Remove', 'quiet small-btn'); rm.type = 'button'; rm.setAttribute('aria-label', 'Remove ' + p.name);
    rm.onclick = () => { invalidateBrief(); state.products.splice(i, 1); state.example = null; save(); renderAll(); };
    if (p.url) { const a = el('a', 'View NIH record \u2197'); a.href = p.url; a.target = '_blank'; a.rel = 'noopener noreferrer'; a.className = 'small'; card.append(a); }
    card.append(rm);
    box.append(card);
  }
}
```

```

function renderMeds() {
  const box = $('#med-list'); box.replaceChildren();
  const resolved = resolveMedicines(state.medicines, idx, state.aiMeds);
  for (const [i, m] of resolved.entries()) {
    const pending = !m.known && aiBusy && !(m.input in state.aiMeds);
    const li = el('li', undefined, 'chip ' + (m.inferred ? 'ai' : m.known ? 'known' : pending ? 'pending' : 'unknown'));
    li.append(el('span', m.input));
    if (m.inferred || pending) li.append(el('span', 'AI', 'pill pill-ai'));
    li.append(el('span', m.inferred ? `suggests ${m.name} \xb7 confirm` : pending ? 'AI is reading\u2026' : m.known ? (m.corrected ? `read as ${m.name}` :
  (norm(m.input).startsWith(m.name) ? '\u2713' : m.name)) : m.excluded === 'not_current' ? 'confirm if currently taken' : 'not recognised', 'chip-note'));
    const x = el('button', '\xd7', 'chip-x'); x.type = 'button'; x.setAttribute('aria-label', 'Remove ' + m.input);
    x.onclick = () => { invalidateBrief(); state.medicines.splice(i, 1); state.example = null; save(); renderAll(); };
    li.append(x); box.append(li);
  }
}

function renderConditions() {
  const box = $('#condition-list'); box.replaceChildren();
  for (const [id, label] of Object.entries(kb.conditions)) {
    const l = el('label', undefined, 'check'); const c = el('input'); c.type = 'checkbox'; c.value = id; c.checked = state.conditions.includes(id);
    c.onchange = () => { invalidateBrief(); state.conditions = [...box.querySelectorAll('input:checked')].map(x => x.value); state.example = null; save(); };
    l.append(c, el('span', label)); box.append(l);
  }
  if (state.conditions.length) $('#conditions-box').open = true;
}

function renderAll() {
  renderProducts(); renderMeds(); renderConditions();
  for (const b of document.querySelectorAll('.example')) b.setAttribute('aria-pressed', String(b.dataset.example === state.example));
  $('#run-check').disabled = !state.products.length;
  $('#run-hint').textContent = state.products.length ? `${state.products.length} product${state.products.length > 1 ? 's' : ''} \xb7 ${state.medicines.length}
  medicine${state.medicines.length === 1 ? '' : 's'} ready to check.` : 'Add at least one product.';
}

// ------------------------------------------------------------- brief HTML
const LEVEL_TAG = {discuss: 'Discuss before combining', caution: 'Ask about this', note: 'Label & quality'};
const fmtQuote = q => `\u201c${esc(q.text)}\u201d${q.items.length ? `<span class="nih-item">NIH\u2019s list includes: <q>${q.items.map(esc).join('</q>,
  <q>')}</q></span>` : ''}`;

function findingTitle(f) {
  const herb = esc(f.herbName);
  if (f.because === 'medicine') return `${herb} <span class="x">\xd7</span> ${esc(f.medicines[0].input)}`;
  if (f.because === 'always') return `${herb} interacts with many medicines`;
  if (f.because === 'condition') return `${herb} <span class="x">\xb7</span> ${f.conditions.map(c => esc(kb.conditions[c])).join('; ')}`;
  if (f.because === 'variant') return `${herb}: ${esc(f.variant)}`;
  if (f.because === 'label') return `${herb}: what the label may not tell you`;
  return herb;
}
function findingSub(f) {
  const parts = [`In ${f.products.map(esc).join(' and ')}`];
  if (f.because === 'medicine') { const m = f.medicines[0]; if (m.inferred) parts.push(`<span class="pill pill-ai">AI</span> understood \u201c${esc(m.input)}\u201d
  as ${esc(m.label)}`); else if (m.label) parts.push(`${esc(m.corrected ? m.name : m.input)} is ${esc(m.label)}`); else if (m.corrected) parts.push(`read as
  ${esc(m.name)}`); }
  if (f.because === 'always' && f.medicines.length) parts.push(`Also ask about: ${f.medicines.map(m => esc(m.input)).join(', ')}`);
  return parts.join(' \xb7 ');
}
function sourceLine(s) {
```

```
  return `<a href="${esc(s.url)}" target="_blank" rel="noopener noreferrer">NIH \xb7 ${esc(s.title.replace(/\s*\|\s*NCCIH$/, ''))}</a>${s.updated ? ` \xb7 page updated ${esc(s.updated)}` : ''}`;
}
function findingHTML(f, compact = false) {
  return `<article class="finding ${f.level}"><header><span class="tag">${LEVEL_TAG[f.level]}</span><h4>${findingTitle(f)}</h4><p class="small">${findingSub(f)}</p></header>
  ${f.quotes.map(q => `<blockquote class="nih"><p>${fmtQuote(q)}</p></blockquote>`).join('')}
  <p class="nih-source small">${sourceLine(f.source)}</p>${compact ? '' : `<p class="ask"><strong>Ask:</strong> ${esc(f.question)}</p>`}</article>`;
}

function gapHTML(g) {
  if (g.type === 'identity') {
    const cands = g.candidates.map(c => {
      let body;
      if (c.herb && c.findings.length) body = c.findings.map(f => `<div class="if-finding"><p>${fmtQuote(f.quotes[0])}</p><p class="small">${sourceLine(f.source)}</p><p class="ask"><strong>Ask:</strong> ${esc(f.question)}</p></div>`).join('');
      else if (c.herb) body = '<p class="small">No NIH statement for your medicines.</p>';
      else body = '<p class="small">Not covered \u2014 ask your pharmacist.</p>';
      return `<li><p><strong><i>${esc(c.species)}</i></strong> \xb7 ${esc(c.common)}${c.flagged ? ' <span class="pill pill-warn">applies to you</span>' : ''}</p>${body}</li>`;
    }).join('');
    return `<article class="gap identity"><h4>\u201c${esc(g.ingredient)}\u201d \u2014 which ${esc(g.term)}?</h4><p class="small">In ${esc(g.productName)}. This name fits more than one plant.</p><ul class="candidates">${cands}</ul><p class="ask"><strong>Ask:</strong> Which species is the \u201c${esc(g.ingredient)}\u201d in ${esc(g.productName)}?</p></article>`;
  }
  if (g.type === 'blend') return `<li><strong>${g.ingredients.map(esc).join(', ')}</strong> ${g.ingredients.length > 1 ? 'are' : 'is'} inside \u201c${esc(g.blend)}\u201d${g.total ? ` (${esc(g.total)} in total)` : ''}: amount of each not stated.</li>`;
  if (g.type === 'unverified') return `<li><strong>${esc(g.ingredient)}</strong>: not covered \u2014 ask your pharmacist.</li>`;
  if (g.type === 'duplicate') return `<li><strong>${esc(g.herbName)}</strong> is in ${g.products.length} products: ${g.products.map(esc).join(', ')}.</li>`;
  if (g.type === 'unknown_medicine') return `<li>We did not recognise the medicine <strong>${esc(g.input)}</strong> \u2014 tell your pharmacist.</li>`;
  if (g.type === 'inferred_medicine') return `<li><span class="pill pill-ai">AI</span> understood <strong>${esc(g.input)}</strong> as ${esc(g.label)}. Confirm the exact medicine with your pharmacist.</li>`;
  return '';
}

function briefHTML(r, opts = {meds: true, conditions: true, ingredients: true}, forPrint = false, qr = '') {
  // Recompute the export with only the selected personal inputs. Hiding just the
  // list would still disclose excluded medicine names in cards and questions.
  if (forPrint && (!opts.meds || !opts.conditions)) r = analyze({products: state.products,
    medicines: opts.meds ? state.medicines : [], conditions: opts.conditions ? state.conditions : [],
    aiMedicines: opts.meds ? state.aiMeds : {}}, idx);
  const main = r.findings.filter(f => f.level !== 'note'), notes = r.findings.filter(f => f.level === 'note');
  const ident = r.gaps.filter(g => g.type === 'identity'), other = r.gaps.filter(g => g.type !== 'identity');
  let h = '';
  if (forPrint) {
    // qr is an SVG generated locally from our own base64url link; it contains no user-supplied markup.
    h += `<header class="print-head">${qr ? `<div class="print-qr">${qr}<span>Scan to open this brief with live NIH links</span></div>` : ''}<h1>Pharmacist brief</h1><p>Prepared ${esc(new Date().toLocaleDateString(undefined, {year: 'numeric', month: 'long', day: 'numeric'}))} with FoliaTrace \xb7 ${r.counts.discuss} to discuss \xb7 ${r.counts.caution} cautions \xb7 ${r.gaps.length} label questions \xb7 for discussion with a pharmacist or clinician \xb7 not medical advice</p></header>`;
    if (opts.meds) h += `<p><strong>Medicines I take:</strong> ${r.medicines.length ? r.medicines.map(m => esc(m.input) + (m.inferred ? ` (AI: ${esc(m.name)}, to confirm)` : m.known && m.corrected ? ` (read as ${esc(m.name)})` : '')).join(', ') : 'none entered'}</p>`;
    if (opts.conditions && state.reason) h += `<p><strong>I take the supplements for:</strong> ${esc(state.reason)}</p>`;
    if (opts.conditions && r.conditions.length) h += `<p><strong>About me:</strong> ${r.conditions.map(c => esc(kb.conditions[c])).join('; ')}</p>`;
    h += '<h2>Products</h2>' + state.products.map(p => `<div class="print-product"><strong>${esc(p.name)}</strong>${p.brand ? ' \xb7 ' + esc(p.brand) : ''}${p.source === 'dsld' ? ` \xb7 NIH DSLD label #${esc(p.dsldId)}` : p.fictional ? ' \xb7 fictional example' : p.source === 'ocr' ? ' \xb7 text read from photo' : ''}${opts.ingredients ? `<p class="small">${p.ingredients.filter(x => !x.other).map(x => esc(x.text.split(' \xb7 ')[0])).join('; ')}</p>` : ''}</div>`).join('');
  }
```

```
  const inferred = r.medicines.filter(m => m.inferred);
  if (opts.meds && inferred.length) h += `<section class="brief-sec ai-understanding"><h3><span class="pill pill-ai">AI</span> From your words to a medicine group</h3>${inferred.map(m => `<p><strong>\u201c${esc(m.input)}\u201d</strong> \u2192 ${esc(m.name)}</p>`).join('')}<p class="small">A suggested group, not an identified medicine. Confirm the exact name with your pharmacist.</p></section>`;
  h += `<section class="brief-sec"><h3>${main.length ? 'Raise these first' : 'Nothing NIH names for your medicines'}</h3>`;
  h += main.length ? `<div class="findings">${main.map(f => findingHTML(f)).join('')}</div>` : `<p>That doesn\u2019t mean safe \u2014 NIH: \u201c${esc(kb.general_quote)}\u201d</p>`;
  h += '</section>';
  if (ident.length || other.length) {
    h += `<section class="brief-sec"><h3>What the label doesn\u2019t settle</h3>${ident.map(gapHTML).join('')}${other.length ? `<ul class="gap-list">${other.map(gapHTML).join('')}</ul>` : ''}</section>`;
  }
  h += forPrint ? researchPrintHTML(opts.conditions) : '<section class="brief-sec ai-sec" id="ai-research" aria-live="polite"></section>';
  if (notes.length) h += `<section class="brief-sec"><h3>Label &amp;amp; product quality notes</h3><div class="findings">${notes.map(f => findingHTML(f, true)).join('')}</div></section>`;
  const rows = r.checked.map(c => `<tr><th scope="row">${esc(c.ingredient.split(' \xb7 ')[0])}${c.amount ? `<span class="small"> \xb7 ${esc(c.amount)}</span>` : ''}<br><span class="small">${esc(c.productName)}</span></th><td><a href="${esc(c.source.url)}" target="_blank" rel="noopener noreferrer">${esc(c.herbName)}</a>${c.basis === 'label_species' ? `<br><span class="pill">label states <i>${esc(c.species)}</i></span>` : ''}${c.basis === 'fuzzy' ? `<br><span class="pill">read from photo as \u201c${esc(c.read)}\u201d</span>` : ''}${c.identity_note ? `<br><span class="small">${esc(c.identity_note)}</span>` : ''}</td><td>${c.flagged ? 'See above' : 'Nothing for you'}${forPrint ? '' : `<br><button type="button" class="link research" data-herb="${esc(c.herbName)}">Research this herb \u2192</button>`}</td></tr>`).join('');
  const others = (r.unmatched || []).map(u => esc(u.ingredient.split(' \xb7 ')[0]));
  if (rows || others.length) h += `<section class="brief-sec"><h3>Everything we checked</h3>${rows ? `<div class="table-wrap"><table class="checked"><thead><tr><th scope="col">On the label</th><th scope="col">NIH fact sheet</th><th scope="col">For you</th></tr></thead><tbody>${rows}</tbody></table></div>` : ''}${others.length ? `<p class="small unmatched">Not checked: ${others.join('; ')}.</p>` : ''}</section>`;
  h += `<p class="small foot">Quotes from NIH NCCIH fact sheets (public domain), retrieved ${esc(kb.built)}. No statement does not mean safe. Not medical advice.</p>`;
  return h;
}

function summaryHTML(r) {
  const n = (v, label, cls) => `<p class="sum ${cls}"><strong>${v}</strong><span>${label}</span></p>`;
  return n(r.counts.discuss, 'to discuss before combining', 'sum-discuss')
    + n(r.counts.caution, r.counts.caution === 1 ? 'caution for you' : 'cautions for you', 'sum-caution')
    + n(r.gaps.length, r.gaps.length === 1 ? 'label question' : 'label questions', 'sum-gap')
    + n(r.checked.length, (r.checked.length === 1 ? 'herb' : 'herbs') + ' checked', 'sum-checked');
}

// ---------------------------------------------- AI research leads in the brief
let leads = [], leadsToken = 0, checkToken = 0;
function invalidateBrief() {
  ++checkToken; ++leadsToken; ++downloadSeq; leads = []; result = null;
  $('#brief').hidden = true; delete $('#brief').dataset.leads;
  $('#handoff').hidden = true;
  $('#run-check').textContent = 'Build my brief \u2192';
  if (downloadUrl) { URL.revokeObjectURL(downloadUrl); downloadUrl = null; }
  $('#download-brief').removeAttribute('href');
}
const LEAD_MIN = 2.0; // raw cross-encoder logit, calibrated on demo development queries; not a probability
const coreName = n => n.replace(/\s*\/.*$/, '').replace(/\s+(oil|extract|root)$/i, '').trim();
function researchPrintHTML(includePurpose = true) {
  const shown = leads.filter(l => l.rows.length);
  if (!shown.length) return '';
  return `<section class="brief-sec"><h3>Research leads, ranked by on-device AI</h3>${shown.map(l => `<p><strong>${esc(l.herb)}</strong>${includePurpose && l.query ? ` \xb7 ${esc(state.reason)}` : ''}</p><ul>${l.rows.map(r => `<li><a href="${esc(r.url)}">${esc(r.title.replace(/<[^>]*>/g, ''))}</a></li>`).join('')}</ul>`).join('')}<p class="small">Reading leads for the pharmacist, not a verdict on benefit or safety.</p></section>`;
}
function leadHTML(row, lead) {
```

```
  const bottle = (result?.checked || []).filter(c => c.herb === lead.herbId).map(c => `${c.ingredient}${c.amount ? ' \xb7 ' + c.amount : ''} (${c.productName})`).join('; ');
  return `<li><a href="${esc(row.url)}" target="_blank" rel="noopener noreferrer">${esc(row.title.replace(/<[^>]*>/g, ''))}</a><p class="small">AI matched to: ${esc(row.goal)}</p><details class="lead-context"><summary>Compare source wording with your bottle</summary><p><strong>Your label:</strong> ${esc(bottle)}</p>${(row.context || []).map(c => `<h5>${esc(c.label)}</h5>${c.span ? `<blockquote>${esc(c.span.text)}</blockquote>` : '<p class="small">No excerpt identified; read the abstract.</p>'}`).join('')}<p class="small">Literal abstract excerpts \xb7 ${esc(row.licence)}. Text cues only; your pharmacist checks whether the study applies.</p></details></li>`;
}
function renderLeads(status) {
  const box = $('#ai-research'); if (!box) return;
  const rows = leads.map(l => `<article class="lead-group"><h4>${esc(l.herb)}${state.reason ? ` <span class="small">\xb7 for ${esc(state.reason)}</span>` : ''}</h4>${
    l.pending ? '<p class="small">AI is ranking\u2026</p>' : l.rows.length ? `<ol>${l.rows.map(r => leadHTML(r, l)).join('')}</ol>`
    : `<p class="small">${l.error ? 'On-device research is unavailable. Retry by rebuilding the brief.' : 'No closely matching study in the on-device library.'}</p>`}</article>`).join('');
  box.innerHTML = `<h3><span class="pill pill-ai">AI</span> Research leads, ranked on your device</h3><p class="small ai-status">${esc(status)}</p><div class="leads">${rows}</div>`;
}
async function runResearch(r) {
  const token = ++leadsToken;
  const reason = state.reason;
  const herbs = [...new Map(r.checked.map(c => [c.herb, c])).values()].slice(0, 6);
  leads = herbs.map(c => ({herbId: c.herb, herb: coreName(c.herbName), rows: [], pending: true, query: !!state.reason}));
  if (!leads.length) { renderLeads(''); return; }
  renderLeads('AI is ranking the on-device research library\u2026');
  const t0 = performance.now();
  for (const l of leads) {
    const h = idx.herbs.get(l.herbId);
    const names = [l.herb, ...h.latin, ...h.aliases.filter(a => a.length >= 6)].map(norm);
    try {
      const found = new Map(); l.all = [];
      for (const {goal, query} of researchQueries(l.herb, reason)) {
        const res = await researchAI({query, method: 'ai', herbNames: names, patientLeads: true}, () => {});
        if (token !== leadsToken) return;
        l.all.push(...res.rows.map(row => ({id: row.id, title: row.title, score: row.score, goal})));
        for (const row of res.rows) if (row.score >= LEAD_MIN && names.some(n => norm(row.title + ' ' + row.abstract).includes(n)) && row.score > (found.get(row.id)?.score ?? -Infinity)) found.set(row.id, {...row, goal});
      }
      l.rows = [...found.values()].sort((a, b) => b.score - a.score || a.id.localeCompare(b.id)).slice(0, 2);
    } catch { if (token !== leadsToken) return; l.rows = []; l.error = true; }
    l.pending = false;
    renderLeads(`AI is ranking on this device\u2026 ${leads.filter(x => !x.pending).length} of ${leads.length} herbs done`);
  }
  $('#brief').dataset.leads = leads.reduce((n, l) => n + l.rows.length, 0);
  $('#brief').dataset.research = JSON.stringify(leads.map(l => ({herb: l.herb, all: l.all, shown: l.rows.map(r => ({id: r.id, score: r.score, goal: r.goal}))})));
  await updateDownload();
  if (token !== leadsToken) return;
  renderLeads(`${leads.some(l => l.error) ? 'Research incomplete' : 'Ranked'} on this device in ${((performance.now() - t0) / 1000).toFixed(1)} s. Reading leads for the pharmacist. Study populations and preparations may differ from your bottle.`);
}

async function runCheck(scroll = true) {
  if (!state.products.length) return;
  const token = ++checkToken; ++leadsToken; ++downloadSeq; leads = [];
  $('#handoff').hidden = true;
  $('#run-check').disabled = true; $('#run-check').textContent = 'Understanding your medicines\u2026';
  understandMedicines(); await aiPending;
  if (token !== checkToken || !state.products.length) return;
```

```
  result = analyze({products: state.products, medicines: state.medicines, conditions: state.conditions, aiMedicines: state.aiMeds}, idx);
  $('#brief-summary').innerHTML = summaryHTML(result);
  $('#brief-body').innerHTML = briefHTML(result);
  $('#brief-foot').textContent = '';
  $('#brief').hidden = false;
  $('#run-check').disabled = false; $('#run-check').textContent = 'Build my brief \u2192';
  $('#brief').dataset.discuss = result.counts.discuss;
  for (const b of document.querySelectorAll('#brief .research')) b.onclick = () => window.dispatchEvent(new CustomEvent('foliatrace:research', {detail:
  `${b.dataset.herb} clinical trial`}));
  updateDownload();
  if (scroll) { $('#brief').scrollIntoView({block: 'start'}); $('#brief').focus({preventScroll: true}); }
  runResearch(result);
}

let downloadUrl = null;
function shareOpts() { return {meds: $('#share-meds').checked, conditions: $('#share-conditions').checked, ingredients: $('#share-ingredients').checked}; }
function standalone(body) {
  return `<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><title>Pharmacist brief \xb7
  FoliaTrace</title><style>${PRINT_CSS}</style></head><body>${body}</body></html>`;
}
// The QR (which carries everything) is only added when everything is chosen for sharing.
async function printQr() { const o = shareOpts(); if (!(o.meds && o.conditions && o.ingredients)) return ''; try { return await qrSvg(await shareLink()); } catch {
  return ''; } }
let downloadSeq = 0;
async function updateDownload() {
  if (!result) return;
  $('#download-brief').removeAttribute('href');
  const seq = ++downloadSeq; const opts = shareOpts(); const qr = await printQr();
  if (seq !== downloadSeq || !result) return;
  if (downloadUrl) URL.revokeObjectURL(downloadUrl);
  downloadUrl = URL.createObjectURL(new Blob([standalone(briefHTML(result, opts, true, qr))], {type: 'text/html;charset=utf-8'}));
  $('#download-brief').href = downloadUrl;
}
const PRINT_CSS = `.print-qr{float:right;width:132px;text-align:center;font:11px/1.3 system-ui;color:#46574e;margin:0 0 8px 16px}.print-qr
  svg{width:132px;height:132px;display:block}`
 + `body{font:12.5px/1.42 Georgia,serif;color:#1d2f27;max-width:780px;margin:18px auto;padding:0 18px}h1{font-size:24px;margin:0}h2{font-size:15px;border-bottom:1px
  solid #999;margin:12px 0 4px}h3{font-size:14.5px;margin:12px 0 4px}h4{font-size:13.5px;margin:2px 0}p{margin:3px 0}.small,.print-head p{font-
  size:11px;color:#46574e}.finding,.gap{border:1px solid #bbb;border-left:4px solid #888;border-radius:4px;padding:5px 10px;margin:5px 0;break-
  inside:avoid}.finding.discuss{border-left-color:#a8432f}.finding.caution{border-left-color:#b17d1a}.finding header p{margin:0}.tag{font:700 9.5px/1 system-ui;letter-
  spacing:.08em;text-transform:uppercase}blockquote{margin:3px 0;padding:1px 8px;border-left:2px solid #ccc;font-style:italic}.nih-item{display:block;font-
  style:normal;font-size:11px}.nih-source{margin:0}.ask{background:#f3f1e8;padding:3px 8px;border-radius:3px;margin-top:3px}table{border-
  collapse:collapse;width:100%;font-size:11px}th,td{border-top:1px solid #ccc;padding:3px 5px;text-align:left;vertical-align:top}.pill{font-size:10px;border:1px solid
  #999;border-radius:3px;padding:0 3px}ul{padding-left:18px;margin:4px 0}li{margin:2px 0}.candidates>li{border-top:1px dashed #ccc;padding:3px 0}.if-finding{border-
  left:2px solid #b17d1a;padding-left:8px}a{color:#24543e}.foot{margin-top:12px;border-top:1px solid #999;padding-top:6px;font-size:10.5px}.print-product p{margin:0 0
  4px}button{display:none}`;

// ------------------------------------------------ serverless pharmacist link
// The brief travels inside the URL fragment (#brief=\u2026), which browsers never send to a server.
const b64url = bytes => btoa(String.fromCharCode(...bytes)).replace(/\+/g, '-').replace(/\//g, '_').replace(/=+$/, '');
const unb64url = s => Uint8Array.from(atob(s.replace(/-/g, '+').replace(/_/g, '/')), c => c.charCodeAt(0));
async function pipe(bytes, stream) { return new Uint8Array(await new Response(new Blob([bytes]).stream().pipeThrough(stream)).arrayBuffer()); }
async function encodeBrief() {
  const compact = {v: 1, m: state.medicines, c: state.conditions, r: state.reason || undefined,
    a: Object.fromEntries(Object.entries(state.aiMeds).filter(([k, v]) => v.group && state.medicines.includes(k)).map(([k, v]) => [k, [v.group, v.score]])), p:
  state.products.map(p => ({n: p.name, s: p.source, b: p.brand || undefined, d: p.dsldId || undefined, f: p.fictional ? 1 : undefined,
    i: p.ingredients.filter(x => !x.other).map(x => [String(x.text).slice(0, 160), x.name || '', x.amount || '', x.genus || '', x.species || '', x.blend || '',
  x.blendTotal || '', x.category || ''])}))};
  return b64url(await pipe(new TextEncoder().encode(JSON.stringify(compact)), new CompressionStream('deflate-raw')));
```

```
}
async function decodeBrief(token) {
  if (token.length > 6000) throw Error('link too long');
  const data = JSON.parse(new TextDecoder().decode(await pipe(unb64url(token), new DecompressionStream('deflate-raw'))));
  if (data.v !== 1 || !Array.isArray(data.p) || data.p.length > 12) throw Error('unsupported brief');
  const str = (v, n) => typeof v === 'string' ? v.slice(0, n) : '';
  return {
    medicines: (data.m || []).filter(x => typeof x === 'string').slice(0, 40).map(x => x.slice(0, 80)),
    conditions: (data.c || []).filter(c => kb.conditions[c]),
    reason: str(data.r, 120),
    aiMeds: cleanAi(Object.fromEntries(Object.entries(data.a && typeof data.a === 'object' ? data.a : {}).map(([k, v]) => [k, {group: Array.isArray(v) ? v[0] : null,
 score: Array.isArray(v) ? v[1] : 0}]))),
    products: data.p.map(p => ({name: str(p.n, 120) || 'Product', source: ['dsld', 'ocr', 'text'].includes(p.s) ? p.s : 'text', brand: str(p.b, 80), dsldId:
Number.isInteger(p.d) ? p.d : undefined,
      url: Number.isInteger(p.d) ? `https://dsld.od.nih.gov/label/${p.d}` : undefined, fictional: !!p.f,
      ingredients: (p.i || []).slice(0, 120).map(r => ({text: str(r[0], 160), name: str(r[1], 160) || undefined, amount: str(r[2], 40) || null, genus: str(r[3], 40)
|| null, species: str(r[4], 40) || null,
        blend: str(r[5], 120) || null, blendTotal: str(r[6], 40) || null, category: str(r[7], 40) || undefined}))}))};
}
function loadQr() { return window.qrcode ? Promise.resolve() : loadScript('vendor/qrcode.js'); }
async function qrSvg(text) { await loadQr(); const q = window.qrcode(0, 'M'); q.addData(text, 'Byte'); q.make(); return q.createSvgTag({cellSize: 4, margin: 2,
scalable: true, alt: 'QR code that opens this brief'}); }
async function shareLink() { return `${location.origin}${location.pathname}#brief=${await encodeBrief()}`; }
async function showHandoff() {
  const box = $('#handoff'); box.hidden = false;
  try {
    const link = await shareLink();
    $('#handoff-qr').innerHTML = await qrSvg(link);
    $('#handoff-link').value = link;
    $('#handoff-size').textContent = `${link.length.toLocaleString()} characters \xb7 the brief is inside the link itself`;
  } catch (e) { $('#handoff-size').textContent = 'This browser cannot build a link (' + e.message + '). Print or download the brief instead.'; }
  box.scrollIntoView({block: 'nearest'});
}

// ---------------------------------------------------------------------- inputs
function setupTabs() {
  const tabs = [...document.querySelectorAll('[role=tab]')];
  const select = t => { for (const x of tabs) { const on = x === t; x.setAttribute('aria-selected', on); x.tabIndex = on ? 0 : -1; $('#' + x.getAttribute('aria-
controls')).hidden = !on; } };
  for (const [i, t] of tabs.entries()) {
    t.onclick = () => select(t);
    t.onkeydown = e => { if (e.key === 'ArrowRight' || e.key === 'ArrowLeft') { const n = tabs[(i + (e.key === 'ArrowRight' ? 1 : tabs.length - 1)) % tabs.length];
select(n); n.focus(); } };
  }
  return select;
}

async function dsldSearch(q) {
  const st = $('#dsld-status'), list = $('#dsld-results');
  list.replaceChildren(); st.textContent = 'Searching NIH\u2026';
  try {
    const ctl = new AbortController(); const t = setTimeout(() => ctl.abort(), 15000);
    const r = await fetch(DSLD + 'search-filter?' + new URLSearchParams({q, size: '8', from: '0'}), {signal: ctl.signal}); clearTimeout(t);
    if (r.status === 429) throw Error('NIH is limiting requests right now; try again in a minute');
    if (!r.ok) throw Error('NIH returned ' + r.status);
    const data = await r.json(); const hits = data.hits || [];
    st.textContent = hits.length ? `${(data.stats?.count ?? hits.length).toLocaleString()} NIH labels match. Showing the top ${hits.length}. Pick the one that
matches your bottle.` : 'No NIH label matched. Try the brand, or type the label instead.';
```

```
    for (const h of hits) {
      const s = h._source; const li = el('li');
      const botan = (s.allIngredients || []).filter(i => i.category === 'botanical').map(i => i.name);
      const info = el('div'); info.append(el('strong', s.fullName || 'Unnamed label'), el('span', [s.brandName, s.offMarket ? 'off market' : 'on market', s.entryDate ? 'entered ' + s.entryDate.slice(0, 4) : ''].filter(Boolean).join(' \xb7 '), 'small'));
      if (botan.length) info.append(el('span', botan.slice(0, 5).join(', ') + (botan.length > 5 ? '\u2026' : ''), 'small botan'));
      const add = el('button', 'Add', 'quiet small-btn'); add.type = 'button'; add.setAttribute('aria-label', 'Add ' + (s.fullName || 'label') + ' ' + (s.brandName || ''));
      add.onclick = async () => {
        add.disabled = true; add.textContent = 'Loading\u2026';
        try {
          const lr = await fetch(DSLD + 'label/' + encodeURIComponent(h._id)); if (!lr.ok) throw Error(lr.status === 429 ? 'NIH is limiting requests; try again in a minute' : 'label ' + lr.status);
          const p = fromDsldLabel(await lr.json());
          if (!p.ingredients.length) throw Error('this record lists no ingredients');
          invalidateBrief(); state.products.push(p); state.example = null; save(); renderAll();
          st.textContent = `Added \u201c${p.name}\u201d from NIH label #${p.dsldId}.`; add.textContent = 'Added \u2713';
        } catch (e) { add.disabled = false; add.textContent = 'Add'; st.textContent = 'Could not load that label (' + e.message + '). Try another, or type the label.'; }
      };
      li.append(info, add); list.append(li);
    }
  } catch (e) { st.textContent = 'The NIH label service is unavailable right now (' + (e.name === 'AbortError' ? 'timed out' : e.message) + '). You can type or photograph the label instead.'; }
}

function loadScript(src) { return new Promise((res, rej) => { const s = document.createElement('script'); s.src = src; s.onload = res; s.onerror = () => rej(Error('could not load ' + src)); document.head.append(s); }); }
async function prepImage(file) {
  const img = await createImageBitmap(file);
  const scale = Math.min(1, 2200 / Math.max(img.width, img.height));
  const c = document.createElement('canvas'); c.width = Math.round(img.width * scale); c.height = Math.round(img.height * scale);
  const g = c.getContext('2d'); g.filter = 'grayscale(1) contrast(1.35)'; g.drawImage(img, 0, 0, c.width, c.height);
  return c;
}
async function runOcr(file, selectTab) {
  const st = $('#ocr-status');
  if (!file || !/^image\//.test(file.type)) { st.textContent = 'Please choose an image file.'; return; }
  if (file.size > 25e6) { st.textContent = 'That image is over 25 MB. Try a smaller photo.'; return; }
  $('#ocr-review').hidden = false; const prev = $('#ocr-preview'); if (prev.src) URL.revokeObjectURL(prev.src); prev.src = URL.createObjectURL(file);
  try {
    st.textContent = 'Preparing on-device text recognition\u2026';
    if (!window.Tesseract) await loadScript('ocr/tesseract.min.js');
    if (!ocrWorker) ocrWorker = await window.Tesseract.createWorker('eng', 1, {workerPath: new URL('ocr/worker.min.js', location.href).href, corePath: new URL('ocr/', location.href).href, langPath: new URL('ocr/lang', location.href).href, gzip: true, workerBlobURL: false,
      logger: m => { if (m.status) st.textContent = `${m.status[0].toUpperCase() + m.status.slice(1)}\u2026 ${m.progress ? Math.round(m.progress * 100) + '%' : ''}`; }});
    const t0 = performance.now();
    const {data} = await ocrWorker.recognize(await prepImage(file));
    const text = data.text.replace(/\n{3,}/g, '\n\n').trim();
    if (text.length < 4) { st.textContent = 'We could not read text in that photo. Try better light, closer to the Supplement Facts panel, or type it.'; return; }
    selectTab($('#tab-text'));
    const f = $('#text-form'); if (!f.elements.name.value) f.elements.name.value = 'Photographed label'; f.elements.text.value = text; ocrPending = true;
    f.elements.text.focus();
    st.textContent = '';
    $('#text-status')?.remove();
    const note = el('p', `Read in ${((performance.now() - t0) / 1000).toFixed(1)} s. Check the text, then add it.`, 'small'); note.id = 'text-status'; note.setAttribute('role', 'status');
```

```
    f.prepend(note);
  } catch (e) { st.textContent = 'On-device text recognition is unavailable (' + e.message + '). Please type the label instead.'; }
}

// ---------------------------------------------------------------------- boot
async function boot() {
  try {
    const r = await fetch('safety-knowledge.json'); if (!r.ok) throw Error(r.status);
    kb = await r.json(); idx = indexKnowledge(kb);
  } catch (e) { $('#run-hint').textContent = 'The NIH knowledge base could not load. Reload to try again.'; return; }
  load(); renderExamples(); renderAll();
  const quoteCount = kb.herbs.reduce((n, h) => n + h.rules.length + (h.variants || []).length, 0);
  $('#kb-stats').textContent = `${kb.herbs.length} herbs \xb7 ${kb.verified_passages || quoteCount} NIH passages \xb7 ${kb.medicines.length} medicines.`;
  const dl = $('#med-names'); for (const m of kb.medicines) for (const a of [m.name, ...m.aliases]) { const o = el('option'); o.value = a; dl.append(o); }
  const selectTab = setupTabs();
  $('#dsld-form').onsubmit = e => { e.preventDefault(); const q = e.target.elements.q.value.trim(); if (q.length >= 2) dsldSearch(q); };
  $('#text-form').onsubmit = e => {
    e.preventDefault(); const f = e.target; const text = f.elements.text.value.trim();
    if (text.length < 3) { f.elements.text.focus(); return; }
    const p = parseLabelText(text, f.elements.name.value.trim() || 'My product'); if (ocrPending) p.source = 'ocr';
    if (!p.ingredients.length) return;
    invalidateBrief(); state.products.push(p); state.example = null; ocrPending = false; save(); f.reset(); $('#text-status')?.remove(); renderAll();
  };
  $('#photo-input').onchange = e => runOcr(e.target.files[0], selectTab);
  $('#med-form').onsubmit = e => {
    e.preventDefault(); const v = $('#med-input').value.trim();
    invalidateBrief();
    for (const part of v.split(/,|;| and /i).map(s => s.trim()).filter(s => s.length > 1)) if (!state.medicines.some(m => norm(m) === norm(part)) &&
state.medicines.length < 40) state.medicines.push(part);
    $('#med-input').value = ''; state.example = null; save(); renderMeds(); renderAll(); understandMedicines();
  };
  $('#reason-input').value = state.reason;
  $('#reason-input').oninput = e => { invalidateBrief(); state.reason = e.target.value.trim().slice(0, 120); state.example = null; save(); renderAll(); };
  $('#run-check').onclick = () => runCheck(true);
  $('#clear-all').onclick = () => { invalidateBrief(); state.products = []; state.medicines = []; state.conditions = []; state.example = null; state.aiMeds = {};
state.reason = ''; $('#reason-input').value = ''; try { localStorage.removeItem(STORE); } catch {} renderAll(); };
  for (const id of ['share-meds', 'share-conditions', 'share-ingredients']) $('#' + id).onchange = () => { $('#handoff').hidden = true; updateDownload(); };
  $('#print-brief').onclick = async () => { $('#print-root').innerHTML = briefHTML(result, shareOpts(), true, await printQr()); window.print(); };
  $('#show-handoff').onclick = showHandoff;
  $('#copy-link').onclick = async () => { try { await navigator.clipboard.writeText($('#handoff-link').value); $('#copy-link').textContent = 'Copied \u2713'; } catch
 { $('#handoff-link').select(); } setTimeout(() => { $('#copy-link').textContent = 'Copy link'; }, 2500); };
  $('#keep-shared').onclick = () => { saveAllowed = true; save(); $('#shared-banner').hidden = true; history.replaceState(null, '', location.pathname);
$('#check').scrollIntoView({block: 'start'}); };
  window.addEventListener('afterprint', () => { $('#print-root').innerHTML = ''; });
  $('#copy-questions').onclick = async () => {
    const qs = [...result.findings.filter(f => f.level !== 'note').map(f => f.question), ...result.gaps.filter(g => g.type === 'identity').map(g => `Which species is
 the \u201c${g.ingredient}\u201d in ${g.productName}?`)];
    try { await navigator.clipboard.writeText(qs.map((q, i) => `${i + 1}. ${q}`).join('\n')); $('#copy-questions').textContent = 'Copied \u2713'; } catch { $('#copy-
questions').textContent = 'Copy unavailable'; }
    setTimeout(() => { $('#copy-questions').textContent = 'Copy my questions'; }, 2500);
  };
  for (const b of document.querySelectorAll('#dsld-form button,#text-form button,#med-form button')) b.disabled = false;
  if (location.hash.startsWith('#brief=')) {
    try {
      const shared = await decodeBrief(location.hash.slice(7));
      Object.assign(state, shared, {example: null}); $('#reason-input').value = state.reason; renderAll(); await runCheck(true); $('#shared-banner').hidden = false;
    } catch (e) { $('#run-hint').textContent = 'This shared brief link could not be opened (' + e.message + ').'; }
  }
```

```
  if (state.products.length && new URLSearchParams(location.search).has('resume')) runCheck(false);
  const ex = new URLSearchParams(location.search).get('example'); if (ex && EXAMPLES.some(e => e.id === ex)) runExample(ex);
  document.documentElement.dataset.ready = 'true';
}
boot();
```

frontend/demo/demo.css
SHA-256: 7027e6e9960e1d5b91daf361c8ccc39313a772ffa06d576d27358fb38adfc58b

```
1 :root{font-family:Inter,"Segoe UI",Arial,sans-serif;color:#233e33;background:#faf9f3;font-size:18px;line-
  height:1.6;--green:#24543e;--line:#ccd5c9;--muted:#4c6257}*{box-sizing:border-box}body{margin:0}a{color:#24543e;text-underline-
  offset:4px}button,input,textarea{font:inherit}button,.button{background:var(--green);color:#fff;border:1px solid var(--green);border-radius:8px;padding:12px
  20px;cursor:pointer;font-size:17px;font-weight:650;min-height:50px;text-decoration:none;display:inline-flex;align-items:center;justify-content:center;gap:16px;line-
  height:1.45}button:hover,.button:hover{background:#173c2b}button:disabled{background:#e4e8df;border-
  color:#a4b2a4;color:#526456;cursor:default}.quiet{background:transparent;color:var(--green)}.quiet:hover{background:#e8eddf}input,textarea{width:100%;border:1px
  solid #859c88;border-radius:6px;background:#fffefb;padding:11px 13px;color:#233e33;min-
  height:48px}input::placeholder,textarea::placeholder{color:#607165;opacity:1}input[type=checkbox]{width:22px;height:22px;min-height:22px;accent-
  color:var(--green);flex-shrink:0}label{display:block;font-size:16px;font-weight:600;margin:0 0 20px}label>input,label>textarea{margin-top:7px;font-weight:400;font-
  size:17px}.check{display:flex;align-items:center;gap:10px;margin:14px 0;font-weight:500}.check input{margin:0}h1,h2{font-family:Georgia,"Times New Roman",serif;font-
  weight:400;line-height:1.08;letter-spacing:-.045em;margin:16px 0 26px}h1{font-size:clamp(50px,5.9vw,84px)}h2{font-size:clamp(36px,4vw,56px)}h3{font-size:22px;line-
  height:1.3;margin:0 0 18px;font-weight:650;letter-spacing:-.02em}em{color:#597344}p{margin:0 0 18px}.small{font-size:15px;color:var(--muted);line-
  height:1.6}.top{display:flex;align-items:center;justify-content:space-between;gap:24px;margin:0 auto;max-width:1352px;padding:22px 44px;border-bottom:1px solid
  var(--line)}.brand{font-weight:750;letter-spacing:-.045em;font-size:25px;text-decoration:none;white-space:nowrap}.brand>span{font-family:Georgia,serif;font-
  size:42px;font-style:italic;margin-right:7px}nav{display:flex;gap:28px;font-size:16px;font-weight:600;flex-wrap:wrap}nav a{padding:10px 0;text-decoration:none}nav
  a:hover{text-decoration:underline}main{max-width:1264px;padding:0 44px;margin:auto}.hero{display:grid;grid-template-columns:1.45fr 1fr;gap:65px;align-
  items:center;padding:70px 0 60px}.eyebrow{font-size:14px;letter-spacing:.13em;font-weight:700;text-transform:uppercase;color:#566d4a;margin:0 0 14px}.lead{font-
  size:20px;line-height:1.7;max-width:590px}.actions{display:flex;gap:15px;flex-wrap:wrap;align-items:center;margin:24px 0 14px}.text-link{font-size:16px;font-
  weight:600}.journey{border:1px solid #bdcbb7;border-radius:22px 22px 80px 22px;background:#eff1e6;padding:30px 32px}.journey svg{width:100%;height:auto;margin-
  bottom:18px}.journey ol{list-style:none;counter-reset:steps;padding:0;margin:0}.journey li{counter-increment:steps;position:relative;padding:0 0 20px 46px}.journey
  li:before{content:counter(steps);position:absolute;left:0;top:0;border:1px solid #a7b89c;width:29px;height:29px;display:grid;place-items:center;border-
  radius:50%;font-size:15px}.journey strong,.journey span{display:block}.journey strong{font-size:19px}.journey span,.journey p{font-size:16px;color:#496144}.journey
  p{border-top:1px solid #bdcbb7;padding-top:20px;margin:0}.scope{border-left:3px solid #9c773f;padding:18px 22px;background:#f3eee0;font-
  size:16px;color:#5c4a2b}.section{padding:70px 0;border-bottom:1px solid var(--line);scroll-margin-top:20px}.section-head{display:flex;justify-content:space-
  between;align-items:center;gap:20px;margin-bottom:30px}.section-head h2{margin-bottom:17px}.badge{font-size:14px;font-weight:650;padding:7px
  11px;background:#e8eddf;color:#34583c;border-radius:6px;display:inline-block}.card{background:#fffefb;border:1px solid var(--line);border-
  radius:12px;padding:28px;min-width:0}.prep-grid{display:grid;grid-template-columns:1fr 1fr;gap:24px}.fields{display:grid;grid-template-columns:1fr
  1fr;gap:18px}.list-heading{display:flex;gap:12px;justify-content:space-between;align-items:center;margin-bottom:18px}.list-heading h3{margin:0}.list-
  card{background:#f1f3e9;display:flex;flex-direction:column}.product{border-top:1px solid var(--line);padding:20px 0;overflow-wrap:anywhere}.product .check{align-
  items:flex-start;margin:0 0 12px;font-size:19px;font-weight:650}.product p{margin:5px 0;font-size:16px}.product .quiet{font-size:15px;padding:8px 12px;min-
  height:44px;margin:12px 8px 0 0}.share-controls{border-top:1px solid var(--line);padding-top:24px;margin-top:auto;font-size:16px}.share-preview{margin-
  top:24px}.share-preview h4{font-size:20px;margin:20px 0 8px}.recording-note{border-left:3px solid #819369;padding:12px 20px;font-size:16px;background:#f0f2e7}.case-
  tabs{display:flex;gap:10px;flex-wrap:wrap;margin:30px 0 22px}.case-tabs button{background:#fffefb;color:var(--green);font-size:16px}.case-tabs button[aria-
  pressed=true]{background:var(--green);color:white}.case-top{display:grid;grid-template-columns:1.2fr 1fr;gap:28px;padding-bottom:25px;border-bottom:1px solid
  var(--line);margin-bottom:25px}.case-top h3{font-family:Georgia,serif;font-size:32px;font-weight:400}.case-top p{font-size:17px}.case-
  source{background:#f1f3e9;padding:22px;border-radius:8px;overflow-wrap:anywhere}.case-source p{font-size:16px}.case-source ul{padding-left:22px;font-
  size:16px}.metrics{display:grid;grid-template-columns:repeat(3,1fr);gap:14px;margin:24px 0}.metric{border-top:2px solid #9aa98b;padding-top:10px;font-
  size:15px;color:#4c6257}.metric strong{display:block;font-family:Georgia,serif;font-size:35px;font-weight:400;color:#24543e}.study{margin:22px 0 0;border:1px solid
  var(--line);border-radius:10px;background:#fffefb;overflow:hidden}.study-head{padding:24px}.study-head h4{font-size:21px;line-height:1.4;margin:12px 0}.study-head
  p{font-size:16px}.table-wrap{overflow-x:auto}table{border-collapse:collapse;width:100%;font-size:16px;min-width:550px}caption{text-align:left;padding:16px;font-
  size:15px}th,td{padding:15px;text-align:left;vertical-align:top;border-top:1px solid var(--line);line-height:1.55}thead th{background:#eef1e5;font-size:14px}tbody
  th{font-weight:600;width:22%}td:nth-child(2){width:29%}.signal{display:inline-block;font-size:14px;padding:4px 8px;border-
  radius:4px;color:#4d5634;background:#eef0df;margin:0 0 8px}.signal.unknown{color:#6d4f25;background:#f6ecd6}.study details{padding:0 24px 24px;font-size:16px}.study
  summary{margin-top:20px}.source-quote{border-left:3px solid #a3b38f;padding:12px 16px;margin:15px 0;background:#f5f5ed;overflow-wrap:anywhere}.source-quote p{white-
  space:pre-wrap;margin:0 0 10px;font-size:16px}.source-quote a{font-size:15px}.three{display:grid;grid-template-columns:repeat(3,1fr);gap:20px;margin:32px 0}.three
  p{font-size:17px}.number{font-family:Georgia,serif;font-size:36px;color:#769066;display:block;margin-bottom:22px}.methods details{font-size:17px}.methods
  summary{margin-bottom:20px}summary{cursor:pointer;font-size:18px;font-weight:650;min-height:40px}details[open]>summary{margin-bottom:20px}.download-
  section{margin:70px 0;display:grid;grid-template-columns:1.2fr 1fr;gap:42px;align-items:center;padding:40px;background:#e9efdf;border-radius:14px}.download-section
  h2{font-size:40px}.download-section p{font-size:17px}.download-actions{display:flex;flex-direction:column;gap:20px}.download-actions>a:not(.button){font-
  size:16px}footer{border-top:1px solid var(--line);padding:25px 44px;display:flex;align-items:center;justify-content:space-between;gap:20px;font-size:14px;max-
  width:1352px;margin:auto}footer p{margin:0}a:focus-visible,button:focus-visible,input:focus-visible,textarea:focus-visible,summary:focus-visible,[tabindex]:focus-
  visible{outline:3px solid #94702d;outline-offset:4px}.skip{position:absolute;top:-100px;left:20px;background:white;padding:15px;z-
  index:10}.skip:focus{top:15px}[hidden]{display:none!important}.error{color:#8b2c24;font-size:16px}#case-status,#draft-status{font-size:16px}code{overflow-
  wrap:anywhere;font-size:14px}blockquote{margin:0}
2 @media(max-width:950px){.hero{gap:30px}.journey{padding:24px}.top{padding:20px 28px}main{padding:0 28px}.three{grid-template-columns:1fr}.download-
```

```
section{padding:28px}nav{gap:17px}.prep-grid{gap:18px}.card{padding:22px}}
@media(max-width:700px){:root{font-size:17px}.top{padding:15px 20px;align-items:flex-start;flex-direction:column;gap:5px}nav{gap:18px;font-size:15px}main{padding:0 20px}.hero{grid-template-columns:1fr;padding:40px 0;gap:24px}h1{font-size:54px}.lead{font-size:19px}.journey{display:grid;grid-template-columns:100px 1fr;gap:20px;border-radius:15px}.journey svg{align-self:center}.journey li{padding:0 0 13px 40px}.journey span{font-size:15px}.journey p{grid-column:1/-1;padding-top:15px}.section{padding:45px 0}.section-head{align-items:flex-start;flex-direction:column;gap:0}.prep-grid,.case-top,.download-section{grid-template-columns:1fr}.case-top{gap:12px}.fields{grid-template-columns:1fr;gap:0}.metrics{gap:12px}.metric{font-size:14px}.metric strong{font-size:30px}.case-tabs{gap:8px}.case-tabs button{font-size:15px;padding:10px 12px}.download-section{margin:40px 0;padding:24px}.download-section h2{font-size:34px}footer{padding:22px 20px;flex-wrap:wrap;justify-content:flex-start}footer p{flex-basis:100%}.actions .button{width:100%}.study-head{padding:20px}.study details{padding:0 20px 20px}.hero .actions{gap:20px}}
@media(prefers-reduced-motion:no-preference){html{scroll-behavior:smooth}}
@media print{.top,footer,.actions,form,.share-controls,.hero,.scope,.evidence-section,.methods,.download-section{display:none}.section{padding:0}.share-preview{display:block!important}.card{border:none;padding:0}main{max-width:none;padding:0}}
.study>summary{margin:0;padding:22px;font-size:20px;line-height:1.45}.study>summary:hover{background:#eef1e5}.study>.study-head{display:flex;flex-wrap:wrap;gap:15px;align-items:center}.study>.study-head p{flex-basis:100%;margin:0}

/* ---------- v0.8 checker ---------- */
:root{--discuss:#a8432f;--discuss-bg:#fbeee8;--caution:#a87512;--caution-bg:#fbf3df;--note:#5f7366;--note-bg:#f1f3ee}
.hero{padding:56px 0 48px}.hero h1{font-size:clamp(46px,5.2vw,76px)}
.stats{display:grid;grid-template-columns:1fr 1fr;gap:18px;margin-top:30px;max-width:620px}.stats p{border-top:2px solid #9aa98b;padding-top:10px;margin:0;font-size:15px;color:var(--muted)}.stats strong{display:block;font-family:Georgia,serif;font-weight:400;font-size:40px;color:var(--green);line-height:1.1}.stats span{display:block;margin:4px 0}.stats a{font-size:13px}
.people{border:1px solid #bdcbb7;border-radius:22px;background:#eff1e6;padding:26px}.people-title{font-size:30px;margin:0 0 6px}.example-list{display:grid;gap:12px;margin-top:14px}
.example{display:grid;grid-template-columns:1fr auto;gap:2px 14px;text-align:left;background:#fffefb;color:#233e33;border:1px solid var(--line);border-radius:12px;padding:16px 18px;font-weight:400;min-height:0}.example:hover{background:#fff;border-color:var(--green)}.example[aria-pressed=true]{border:2px solid var(--green);padding:15px 17px}
.ex-name{font-weight:750;font-size:18px}.ex-story{grid-column:1;font-size:15px;line-height:1.45}.ex-meta{grid-column:1;font-size:13px;color:var(--muted)}.ex-go{grid-column:2;grid-row:1/4;align-self:center;font-weight:700;color:var(--green);font-size:15px;white-space:nowrap}
.check-section .section-head h2{margin-bottom:0}
.checker{display:grid;grid-template-columns:1.35fr 1fr .85fr;gap:20px;align-items:start}
.step{position:relative;padding-top:26px}.step h3{font-size:24px;margin-bottom:16px}.step-n{position:absolute;top:-16px;left:24px;margin:0;width:34px;height:34px;border-radius:50%;background:var(--green);color:#fff;display:grid;place-items:center;font-weight:700;font-size:16px}
.tabs{display:flex;gap:6px;flex-wrap:wrap;border-bottom:1px solid var(--line);margin-bottom:18px}.tabs button{background:transparent;color:var(--muted);border:none;border-bottom:3px solid transparent;border-radius:0;padding:8px 6px;min-height:44px;font-size:15px}.tabs button[aria-selected=true]{color:var(--green);border-bottom-color:var(--green)}.tabs button:hover{background:transparent;color:var(--green)}
.inline-form label{margin-bottom:6px}.row{display:flex;gap:8px;margin-bottom:8px}.row input{margin:0}.row button{flex-shrink:0}
.dsld-results{list-style:none;padding:0;margin:0;max-height:330px;overflow:auto}.dsld-results li{display:flex;justify-content:space-between;align-items:center;gap:12px;border-top:1px solid var(--line);padding:10px 0}.dsld-results strong,.dsld-results span{display:block}.dsld-results .botan{color:#4d6a3c}
.small-btn{font-size:14px;padding:6px 12px;min-height:40px}
.photo-drop{display:grid;justify-items:center;text-align:center;gap:6px;border:2px dashed #9fb19c;border-radius:12px;padding:26px 18px;cursor:pointer;background:#f6f7f0;font-weight:500}.photo-drop:hover{border-color:var(--green)}.photo-icon{font-size:34px;color:var(--green);line-height:1}
.visually-hidden{position:absolute!important;width:1px;height:1px;overflow:hidden;clip:rect(0 0 0 0);white-space:nowrap}
#ocr-review img{max-width:100%;max-height:220px;border-radius:8px;margin-top:10px;display:block}
.product-list{margin-top:18px;border-top:1px solid var(--line);padding-top:6px}.product-card{border-bottom:1px solid var(--line);padding:12px 0;display:flex;flex-wrap:wrap;gap:4px 12px;align-items:center}.product-head{display:flex;flex-wrap:wrap;gap:8px;align-items:center;flex-basis:100%}.product-head strong{font-size:17px}.ingredients-line{flex-basis:100%;margin:0}.badge-nih{background:#dde7f1;color:#244a6b}.empty{margin:12px 0 0}
.chips{list-style:none;padding:0;margin:8px 0 10px;display:flex;flex-wrap:wrap;gap:8px}.chip{display:flex;align-items:center;gap:6px;border:1px solid var(--line);border-radius:999px;padding:3px 4px 3px 12px;background:#fff;font-size:15px}.chip.known{border-color:#8fae8c}.chip.unknown{border-color:#c9a25a;background:#fdf8ec}.chip-note{font-size:12.5px;color:var(--muted)}.chip-x{background:transparent;color:var(--muted);border:none;min-height:32px;width:32px;padding:0;font-size:20px;border-radius:50%}.chip-x:hover{background:#eef0e6;color:#8b2c24}
.conditions{border:1px solid var(--line);border-radius:10px;padding:10px 14px 4px;margin-top:14px}.conditions legend{font-weight:650;font-size:16px;padding:0 6px}.conditions .check{margin:8px 0;font-size:15px;align-items:flex-start}
.go-card{background:#f1f3e9}.go-card p{font-size:16px}.big{width:100%;font-size:19px;min-height:58px;margin:6px 0 10px}.link{background:none;border:none;color:var(--green);padding:0;min-height:0;font-size:inherit;font-weight:600;text-decoration:underline;text-underline-offset:3px}.link:hover{background:none;color:#173c2b}.privacy{margin-top:14px}
.brief{margin-top:34px;border:2px solid var(--green);border-radius:16px;background:#fffefb;padding:32px;scroll-margin-top:16px}.brief-head{display:flex;justify-content:space-between;gap:20px;align-items:flex-end;flex-wrap:wrap;border-bottom:1px solid var(--line);padding-bottom:18px}.brief-head h2{margin:0;font-
```

```
size:40px}.brief-actions{display:flex;gap:10px;flex-wrap:wrap}.brief-actions .button,.brief-actions button{font-size:15px;min-height:46px;padding:10px 16px}
.summary{display:grid;grid-template-columns:repeat(4,1fr);gap:14px;margin:22px 0 8px}.sum{margin:0;padding:14px 16px;border-radius:10px;background:var(--note-bg);font-size:14.5px;line-height:1.4}.sum strong{display:block;font-family:Georgia,serif;font-weight:400;font-size:44px;line-height:1}.sum-discuss{background:var(--discuss-bg);color:#6e2a1c}.sum-discuss strong{color:var(--discuss)}.sum-caution{background:var(--caution-bg);color:#5f4209}.sum-caution strong{color:var(--caution)}.sum-checked strong{color:var(--green)}
.brief-sec{margin-top:28px}.brief-sec>h3{font-family:Georgia,serif;font-weight:400;font-size:28px;letter-spacing:-.02em;margin-bottom:14px}
.findings{display:grid;grid-template-columns:repeat(auto-fill,minmax(340px,1fr));gap:16px}
.finding{border:1px solid var(--line);border-left:6px solid var(--note);border-radius:10px;padding:16px 18px;background:#fff;min-width:0;overflow-wrap:anywhere}.finding.discuss{border-left-color:var(--discuss);background:linear-gradient(90deg,var(--discuss-bg),#fff 40%)}.finding.caution{border-left-color:var(--caution);background:linear-gradient(90deg,var(--caution-bg),#fff 40%)}
.finding header p{margin:2px 0 10px}.finding h4{font-size:20px;margin:6px 0 2px;line-height:1.3}.x{color:var(--muted);font-weight:400;padding:0 2px}
.tag{display:inline-block;font-size:12px;font-weight:750;letter-spacing:.09em;text-transform:uppercase;color:var(--note)}.discuss .tag{color:var(--discuss)}.caution .tag{color:var(--caution)}
blockquote.nih{border-left:3px solid #b9c6b4;margin:8px 0;padding:4px 0 4px 14px;font-family:Georgia,serif;font-size:16.5px;line-height:1.5;color:#2b3f35}blockquote.nih p{margin:0 0 6px}.nih-item{display:block;font-family:Inter,"Segoe UI",sans-serif;font-size:14px;color:var(--muted);margin-top:4px}.nih-item q{font-weight:650;color:#2b3f35}.nih-source{margin:4px 0 10px}
.ask{background:#f3f1e6;border-radius:8px;padding:10px 12px;font-size:15.5px;margin:8px 0 0}
.gap{border:1px solid var(--line);border-radius:10px;padding:16px 18px;margin-bottom:14px;background:#fff}.gap h4{font-size:19px;margin:0 0 4px}.candidates{list-style:none;padding:0;margin:10px 0}.candidates>li{border-top:1px dashed var(--line);padding:10px 0}.candidates>li>p{margin:0 0 4px}.if-finding{border-left:3px solid var(--caution);padding:2px 0 2px 12px;margin:6px 0;font-size:15.5px}.if-finding p{margin:0 0 4px}
.pill{display:inline-block;font-size:12.5px;border:1px solid #b9c6b4;border-radius:5px;padding:1px 7px;background:#f4f6ef;font-weight:600;color:#3f5a44}.pill-warn{border-color:#d7b36b;background:#fbf3df;color:#6a4a0b}
.gap-list{padding-left:22px;font-size:16px}.gap-list li{margin:8px 0}
table.checked{min-width:520px}table.checked th{width:36%}
.share-choice{border:1px dashed var(--line);border-radius:10px;padding:8px 16px;margin-top:26px;display:flex;flex-wrap:wrap;gap:4px 22px}.share-choice legend{font-size:15px;font-weight:650;padding:0 6px}.share-choice .check{margin:8px 0;font-size:15px}
.foot,.brief-foot{margin-top:18px}
.limits{margin:4px 0 22px}.limits ul{margin:0;padding-left:22px}.limits li{margin:8px 0;font-size:17px}
#print-root{display:none}
@media(max-width:1100px){.checker{grid-template-columns:1fr 1fr}.go-card{grid-column:1/-1}.summary{grid-template-columns:repeat(2,1fr)}}
@media(max-width:700px){.checker{grid-template-columns:1fr;gap:26px}.stats{grid-template-columns:1fr}.brief{padding:20px 16px}.brief-head h2{font-size:32px}.findings{grid-template-columns:1fr}.sum strong{font-size:36px}.people{padding:20px}.row{flex-wrap:wrap}.row button{width:100%}.ex-go{grid-column:1;grid-row:auto;margin-top:6px}}
@media print{body>*:not(#print-root){display:none!important}#print-root{display:block!important;font:13.5px/1.5 Georgia,serif;color:#111}#print-root .finding,#print-root .gap{break-inside:avoid;background:#fff!important}#print-root .findings{display:block}#print-root .finding{margin:8px 0}#print-root .print-head h1{font-size:26px;margin:0}#print-root button{display:none}#print-root a{color:#111}}
.handoff{display:grid;grid-template-columns:220px 1fr;gap:26px;align-items:center;border:1px solid var(--line);background:#f4f6ee;border-radius:12px;padding:22px;margin:20px 0 4px}.handoff-qr svg{width:220px;height:220px;display:block;background:#fff;border-radius:8px}.handoff h3{font-size:22px;margin-bottom:8px}.handoff p{font-size:16px}.handoff label{margin-bottom:6px}.shared-banner{background:#e5eef6;border-left:4px solid #3b6c96;padding:12px 16px;border-radius:6px;margin:18px 0 0;font-size:16px}#print-root .print-qr{float:right;width:132px;text-align:center;font:11px/1.3 system-ui;margin:0 0 8px 16px}#print-root .print-qr svg{width:132px;height:132px;display:block}@media(max-width:700px){.handoff{grid-template-columns:1fr}.handoff-qr svg{width:200px;height:200px;margin:auto}}
.measured{margin:4px 0 22px;background:#f1f3e9}.measure-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:16px;margin:6px 0 12px}.measure-grid p{margin:0;border-top:2px solid #9aa98b;padding-top:10px;font-size:15px;color:var(--muted)}.measure-grid strong{display:block;font-family:Georgia,serif;font-weight:400;font-size:40px;color:var(--green);line-height:1.1}@media(max-width:950px){.measure-grid{grid-template-columns:1fr 1fr}}
.conditions{padding:12px 16px}.conditions summary{font-weight:650;font-size:16px;min-height:0}.conditions[open] summary{margin-bottom:8px}.evidence-box{margin-top:26px}.evidence-box>summary{font-size:20px}.evidence-box>summary .small{font-weight:400;margin-left:8px}#live-results{margin-top:18px}.hero .lead{font-size:22px;max-width:560px}
.pill-ai{background:#2b4f8c;color:#fff;border-color:#2b4f8c;font-size:11px;letter-spacing:.06em;padding:1px 6px;vertical-align:middle}.chip.ai{border-color:#2b4f8c;background:#eef3fb}.chip.pending{border-color:#9fb3d6;background:#f4f7fc}.chip .pill-ai{margin-left:2px}.reason-label{margin-top:14px;font-size:15px}.reason-label input{margin-top:6px;font-weight:400}.four{display:grid;grid-template-columns:repeat(4,1fr);gap:18px;margin:32px 0}.four .card p{font-size:16px}.four .number{font-size:26px}.four .rule{background:#f1f3e9}.measure-grid.g4{grid-template-columns:repeat(4,1fr)}.ai-sec{border:1px solid #c9d6ec;background:#f6f9fe;border-radius:12px;padding:16px 20px}.ai-sec>h3{font-family:Georgia,serif;font-weight:400;font-size:26px;margin:0 0 6px}.leads{display:grid;grid-template-columns:repeat(auto-fill,minmax(300px,1fr));gap:12px;margin-top:10px}.lead-group{background:#fff;border:1px solid #dbe4f3;border-radius:10px;padding:10px 14px}.lead-group h4{margin:0 0 6px;font-size:17px}.lead-group ol{margin:0;padding-left:20px;font-size:15px}.lead-group li{margin:4px 0}@media(max-width:1100px){.four,.measure-grid.g4{grid-template-columns:1fr 1fr}}@media(max-width:700px){.four,.measure-grid.g4{grid-template-columns:1fr}}
.lead-context{margin:10px 0 14px;border-top:1px solid #dbe4f3;padding-top:8px}.lead-context summary{font-size:14px;min-height:0;color:#24513d}.lead-context
```

```
h5{font:700 13px/1.4 system-ui;margin:12px 0 5px}.lead-context blockquote{margin:0;padding:3px 0 3px 10px;border-left:2px solid #b8c9e4;font:14px/1.55
Georgia,serif}.lead-context p{font-size:14px}.ai-understanding{background:#eef3fb;border:1px solid #c9d6ec;border-radius:10px;padding:12px 18px}.ai-understanding
h3{margin-top:0}.ai-understanding p{margin:6px 0}.leads{grid-template-columns:repeat(auto-fit,minmax(min(100%,320px),1fr))}
```

frontend/demo/demo.js
SHA-256: 49f2af31dd7857dc9da6e6a2207555a7f4623b92bfeabb1c1576cb0aabf34a34

```
const $=s=>document.querySelector(s);
fetch('product-metrics.json').then(r=>{if(!r.ok)throw Error();return r.json()}).then(metrics=>{for(const node of document.querySelectorAll('[data-metric]'))if(metrics[node.dataset.metric]!==undefined)node.textContent=metrics[node.dataset.metric];}).catch(()=>{});
const el=(tag,text,cls)=>{const n=document.createElement(tag);if(text!==undefined)n.textContent=text;if(cls)n.className=cls;return n;};
// Recorded pharmacist evidence examples and live on-device research search.
function sourceLink(text,url){const a=el('a',text);try{const u=new URL(url);if(!['https:','http:'].includes(u.protocol))throw Error();a.href=u.href;a.target='_blank';a.rel='noopener noreferrer';}catch{a.removeAttribute('href');}return a;}
const signalLabels={PASS:'Text signal found',PARTIAL:'Needs source reading',UNKNOWN:'Not established',FAIL:'Scope conflict'};
function quotes(parent,spans,licence){for(const span of spans){const block=el('blockquote',undefined,'source-quote');block.append(el('p','\u201c'+span.text+'\u201d'),sourceLink((span.source_kind||'Source')+' \xb7 characters '+span.start+'\u2013'+span.end,span.url),el('p',span.licence||licence||'Rights remain with source','small'));parent.append(block);}}
function renderCase(c,data){for(const b of $('#case-tabs').children)b.setAttribute('aria-pressed',String(b.dataset.case===c.id));
 const box=$('#case-detail');box.replaceChildren();const top=el('div',undefined,'case-top'),question=el('div');question.append(el('h3',c.question));
 const metrics=el('div',undefined,'metrics');for(const [value,label] of [[c.overview.unique_publications,'unique publications found'],[c.overview.endpoint_publications,'with exact outcome wording'],[c.overview.human_trial_signals,'human-trial text signals']]){const m=el('div',undefined,'metric');m.append(el('strong',String(value)),el('span',label));metrics.append(m);}question.append(metrics);

 const source=el('aside',undefined,'case-source');source.append(el('h3',c.reported.name));if(c.ingredient)source.append(el('p','Investigated ingredient: '+c.ingredient));source.append(el('p','Actual amount: '+(c.reported.dose||'Not provided')),el('p','How often: '+(c.reported.frequency||'Not provided')));
 if(c.source_label)source.append(sourceLink('NIH DSLD label #'+c.source_label.id,c.source_label.url));
 const names=el('details');names.append(el('summary',c.candidates.length+' candidate '+(c.candidates.length===1?'identity':'identities')+' retained'));const list=el('ul');for(const candidate of c.candidates)list.append(el('li',candidate.name));names.append(list);source.append(names);top.append(question,source);box.append(top);
 if(!c.studies.length)box.append(el('p','No matching human study in this small open-access library.'));
 for(const [index,s] of c.studies.entries()){const study=el('details',undefined,'study');study.open=index===0;study.append(el('summary',(index+1)+'. '+s.bridge.paper.title.replace(/<[^>]*>/g,'')));const head=el('div',undefined,'study-head');head.append(el('span','Source comparison '+(index+1),'badge'),sourceLink('Open original publication \u2197',s.bridge.paper.url),el('p','Candidate branch: '+s.bridge.candidate+' \xb7 '+s.study_types.join(', '),'small'));study.append(head);
 const wrap=el('div',undefined,'table-wrap'),table=el('table');const thead=el('thead'),tr=el('tr');for(const title of ['Dimension','The product / person','What the study check establishes']){const th=el('th',title);th.scope='col';tr.append(th);}thead.append(tr);table.append(thead);const tbody=el('tbody');
 for(const d of s.bridge.dimensions){const row=el('tr'),key=el('th',d.label);key.scope='row';const result=el('td');result.append(el('span',signalLabels[d.status]||d.status,'signal '+(d.status==='UNKNOWN'?'unknown':'')),el('div',d.detail));row.append(key,el('td',d.product),result);tbody.append(row);}table.append(tbody);wrap.append(table);study.append(wrap);
 const detail=el('details');detail.append(el('summary','Read the exact source excerpts and saved criteria'));

 let count=0;for(const d of s.bridge.dimensions){if(!d.spans.length)continue;detail.append(el('h4',d.label));quotes(detail,d.spans,s.licence);count+=d.spans.length;}if(!count)detail.append(el('p','No quoted source wording for these dimensions. Bibliographic metadata alone does not establish study eligibility.'));
 if(s.abstract){const abstract=el('details');abstract.append(el('summary','Read the bundled abstract'),el('p',s.abstract),el('p','Article rights: '+s.licence,'small'));detail.append(abstract);}study.append(detail);box.append(study);
 }

 $('#case-status').textContent=c.title+' \xb7 recorded example loaded.';
}
async function loadRecordedCases(){try{const response=await fetch('care-cases.json');if(!response.ok)throw Error('Examples are unavailable');const data=await response.json();for(const c of data.cases){const b=el('button',c.title);b.type='button';b.dataset.case=c.id;b.setAttribute('aria-pressed','false');b.onclick=()=>renderCase(c,data);$('#case-tabs').append(b);}renderCase(data.cases[0],data);}catch(error){$('#case-status').textContent='The recorded evidence examples could not be loaded. Your browser-local product list still works. '+error.message;$('#case-status').className='error';}}
loadRecordedCases();

let searchWorker=null,searchId=0,searchRunning=false;
const liveState=(message,bad=false)=>{$('#live-status').textContent=message;$('#live-status').className=bad?'error':'small';};
$('#live-form input').oninput=()=>{searchId++;$('#live-results').hidden=true;liveState('Question changed. Run a new search to update the reading order.');};
$('#live-form').onsubmit=event=>{event.preventDefault();if(searchRunning)return;const query=$('#live-form').elements.query.value.trim();if(query.length<3||query.length>500){liveState('Enter 3\u2013500 characters.',true);return;}const method=event.submitter?.value||'ai';const id=++searchId;searchRunning=true;$('#live-results').hidden=true;for(const b of $('#live-
```

```
form').querySelectorAll('button'))b.disabled=true;liveState('Opening the bundled literature library\u2026');
 const reset=()=>{searchRunning=false;for(const b of $('#live-form').querySelectorAll('button'))b.disabled=false;};
 try{if(!searchWorker)searchWorker=new Worker(new URL('model-search-worker.js',location.href),{type:'module'});
 searchWorker.onmessage=({data})=>{if(data.type!=='progress')reset();if(data.id!==searchId)return;if(data.type==='progress'){liveState(data.text);return;}if(data.type==='error'){liveState(data.text,true);return;}
 const box=$('#live-results');box.replaceChildren(el('h3',(data.method==='ai'?'On-device AI':'Keyword')+' reading order'),el('p',data.query),el('p',data.library_count+' abstracts \xb7 '+data.candidate_count+' candidates \xb7 '+data.seconds.toFixed(1)+' s on this device','small'));
 if(!data.rows.length)box.append(el('p','No match. Try different wording.'));
 for(const [position,row] of data.rows.entries()){const card=el('article',undefined,'card');card.style.marginBottom='18px';card.append(el('span',String(position+1)+' / '+row.id,'badge'),el('h4',row.title.replace(/<[^>]*>/g,'')),sourceLink('Read original source \u2197',row.url));const details=el('details');details.append(el('summary','Read bundled abstract and rights'),el('p',row.abstract),el('p','Article rights: '+row.licence,'small'));card.append(details);box.append(card);}
 if(data.method==='ai'){const compare=el('details',undefined,'card');compare.append(el('summary','Compare the keyword candidate order'));const list=el('ol');for(const paperId of data.baseline.slice(0,5)){const title=data.rows.find(r=>r.id===paperId)?.title?.replace(/<[^>]*>/g,'');list.append(el('li',title||paperId));}compare.append(list);box.append(compare);}
 box.hidden=false;box.dataset.method=data.method;box.dataset.query=data.query;box.dataset.resultIds=JSON.stringify(data.rows.map(r=>r.id));box.dataset.scores=JSON.stringify(data.rows.map(r=>r.score));liveState('Finished on this device. No question was sent to a server.');};
 searchWorker.onerror=()=>{reset();liveState('This browser could not start local inference. Reload to retry; your saved preparation is unchanged.',true);searchWorker?.terminate();searchWorker=null;};searchWorker.postMessage({id,query,method});
 }catch(error){reset();liveState('Local search is unavailable: '+error.message,true);}
};
for(const button of $('#live-form').querySelectorAll('button'))button.disabled=false;

window.addEventListener('foliatrace:research',e=>{const input=$('#live-form').elements.query;input.value=String(e.detail||'').slice(0,500);$('#live-search').scrollIntoView({block:'start'});input.focus({preventScroll:true});liveState('Query filled from your brief. Choose AI or keyword search.');});
```

frontend/demo/index.html
SHA-256: f0e9ad896a8d9ee52a1d77a9a5f2cf7e73eb23e751ffe5ab0eaa6901d2c4d17b

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' 'wasm-unsafe-eval'; worker-src 'self' blob:; connect-src 'self' blob: data: https://api.ods.od.nih.gov; img-src 'self' data: blob:; style-src 'self'; object-src 'none'; base-uri 'none'; form-action 'none'"><meta name="referrer" content="no-referrer"><meta name="description" content="Check a supplement label against your medicines with verbatim NIH statements, and hand your pharmacist a brief. Runs in your browser."><title>FoliaTrace \xb7 Bring the bottle. Start a better conversation.</title><link rel="stylesheet" href="demo.css"><script type="module" src="check.js"></script><script type="module" src="demo.js"></script><link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'%3E%3Ctext y='26' font-size='30'%3E\U0001f33f%3C/text%3E%3C/svg%3E"></head>
<body>
<a class="skip" href="#main">Skip to content</a>
<header class="top"><a class="brand" href="./" aria-label="FoliaTrace home"><span aria-hidden="true">f.</span> FoliaTrace</a><nav aria-label="Main"><a href="#check">Check a bottle</a><a href="#pharmacist">For pharmacists</a><a href="#how">How it works</a><a href="source.zip" download>Source</a></nav></header>
<main id="main"><noscript><p class="scope">FoliaTrace runs in your browser. Please enable JavaScript.</p></noscript>

<section class="hero" aria-labelledby="hero-title">
 <div>
  <h1 id="hero-title">Bring the bottle.<br><em>Start a better conversation.</em></h1>
  <p class="lead">Can\u2019t remember a medicine\u2019s name? Describe it. On-device AI suggests its group and finds research for <strong>your bottle and your goal</strong>. Your pharmacist gets a brief with the exact NIH warnings.</p>
  <div class="actions"><a class="button" href="#check">Check a bottle <span aria-hidden="true">\u2192</span></a><span class="small"><span class="pill pill-ai">AI</span> Language &amp;amp; research AI \xb7 On your device \xb7 Free</span></div>
  <div class="stats">
   <p><strong>57.6%</strong><span>of U.S. adults take supplements</span><a href="https://www.cdc.gov/nchs/products/databriefs/db399.htm" target="_blank" rel="noopener noreferrer">CDC, 2017\u201318 (age 20+)</a></p>
   <p><strong>1 in 3</strong><span>tell a health-care provider</span><a href="https://pmc.ncbi.nlm.nih.gov/articles/PMC2787410/" target="_blank" rel="noopener noreferrer">Mehta et al. 2008</a></p>
  </div>
 </div>
 <aside class="people" aria-labelledby="people-title">
  <h2 id="people-title" class="people-title">Start with a person</h2>
  <p class="small">Fictional people. Real NIH statements.</p>
  <div id="example-list" class="example-list"></div>
 </aside>
</section>

<section id="check" class="section check-section" aria-labelledby="check-title">
 <div class="section-head"><h2 id="check-title">What\u2019s in the bottle?<br><em>What else do you take?</em></h2></div>
 <div class="checker">
  <div class="step card" aria-labelledby="step1-title">
   <p class="step-n">1</p><h3 id="step1-title">The bottle</h3>
   <div class="tabs" role="tablist" aria-label="How to add a product">
    <button type="button" role="tab" id="tab-search" aria-selected="true" aria-controls="pane-search">Search NIH labels</button>
    <button type="button" role="tab" id="tab-text" aria-selected="false" aria-controls="pane-text" tabindex="-1">Type or paste</button>
    <button type="button" role="tab" id="tab-photo" aria-selected="false" aria-controls="pane-photo" tabindex="-1">Photo</button>
   </div>
   <div id="pane-search" role="tabpanel" aria-labelledby="tab-search">
    <form id="dsld-form" class="inline-form"><label for="dsld-q">Product or brand</label><div class="row"><input id="dsld-q" name="q" minlength="2" maxlength="120" placeholder="e.g. ginkgo, St. John's wort" autocomplete="off"><button type="submit" disabled>Search</button></div></form>
    <p class="small">\u2248200,000 U.S. labels from NIH.</p>
    <p id="dsld-status" role="status" class="small"></p>
    <ul id="dsld-results" class="dsld-results"></ul>
   </div>
   <div id="pane-text" role="tabpanel" aria-labelledby="tab-text" hidden>
    <form id="text-form"><label>Product name<input name="name" maxlength="120" placeholder="e.g. Joint support" autocomplete="off"></label><label>Ingredients from
```

```
   the label<textarea name="text" rows="6" maxlength="6000" placeholder="One per line, e.g.&#10;Ginkgo biloba leaf extract 120 mg"></textarea></label><button
   type="submit" disabled>Add product</button></form>
44    </div>
45    <div id="pane-photo" role="tabpanel" aria-labelledby="tab-photo" hidden>
46     <label class="photo-drop" for="photo-input"><span class="photo-icon" aria-hidden="true">\u25ce</span><strong>Photo of the Supplement Facts panel</strong><span
   class="small">Read on this device. Never uploaded.</span></label>
47     <input id="photo-input" type="file" accept="image/*" capture="environment" class="visually-hidden">
48     <p id="ocr-status" role="status" aria-live="polite" class="small"></p>
49     <div id="ocr-review" hidden><img id="ocr-preview" alt="Your label photo"></div>
50    </div>
51    <div id="product-list" class="product-list" aria-live="polite"></div>
52   </div>
53
54   <div class="step card" aria-labelledby="step2-title">
55    <p class="step-n">2</p><h3 id="step2-title">Your medicines</h3>
56    <form id="med-form" class="inline-form"><label for="med-input">Name, brand, typo \u2014 or just describe it</label><div class="row"><input id="med-input"
   list="med-names" maxlength="80" placeholder="e.g. warfarin, Zoloft, my water pill" autocomplete="off"><button type="submit" class="quiet"
   disabled>Add</button></div><datalist id="med-names"></datalist></form>
57    <ul id="med-list" class="chips" aria-live="polite"></ul>
58    <label for="reason-input" class="reason-label">What do you take the supplement for? <span class="small">(optional)</span><input id="reason-input" maxlength="120"
   placeholder="e.g. memory, sleep, joint pain" autocomplete="off"></label>
59    <details id="conditions-box" class="conditions"><summary>About you <span class="small">(optional)</span></summary><div id="condition-list"></div></details>
60   </div>
61
62   <div class="step card go-card" aria-labelledby="step3-title">
63    <p class="step-n">3</p><h3 id="step3-title">Your brief</h3>
64    <p>Your words become medicine questions. Your bottle and goal become a research reading list. Every warning stays in NIH\u2019s words.</p>
65    <button id="run-check" type="button" class="big" disabled>Build my brief <span aria-hidden="true">\u2192</span></button>
66    <p id="run-hint" class="small">Add at least one product.</p>
67    <p class="small privacy">Saved only in this browser. <button id="clear-all" type="button" class="link">Erase</button></p>
68   </div>
69  </div>
70
71  <article id="brief" class="brief" tabindex="-1" hidden aria-labelledby="brief-title">
72   <header class="brief-head"><h2 id="brief-title">Your pharmacist brief</h2><div class="brief-actions"><button id="show-handoff" type="button">Hand to pharmacist
   (QR)</button><button id="print-brief" type="button" class="quiet">Print / PDF</button><a id="download-brief" class="button quiet" download="foliatrace-pharmacist-
   brief.html">Download</a><button id="copy-questions" type="button" class="quiet">Copy my questions</button></div></header>
73   <p id="shared-banner" class="shared-banner" hidden>Opened from a shared link \u2014 nothing was uploaded. <button id="keep-shared" type="button" class="link">Keep
   on this device</button></p>
74   <div id="handoff" class="handoff" hidden><div id="handoff-qr" class="handoff-qr"></div><div><h3>Show this to your pharmacist</h3><p>They scan it and see this brief
    on their screen. The brief lives inside the link \u2014 share it only with them.</p><label for="handoff-link">Link</label><div class="row"><input id="handoff-link"
   readonly><button id="copy-link" type="button" class="quiet">Copy link</button></div><p id="handoff-size" class="small"></p></div></div>
75   <div id="brief-summary" class="summary" role="status"></div>
76   <div id="brief-body"></div>
77   <fieldset class="share-choice"><legend>Include when printing or downloading</legend><label class="check"><input type="checkbox" id="share-meds" checked>My
   medicines</label><label class="check"><input type="checkbox" id="share-conditions" checked>About me</label><label class="check"><input type="checkbox" id="share-
   ingredients" checked>Ingredient lists</label></fieldset>
78   <p class="small brief-foot" id="brief-foot"></p>
79  </article>
80 </section>
81
82 <section id="pharmacist" class="section" aria-labelledby="pharm-title">
83  <div class="section-head"><h2 id="pharm-title">For the pharmacist.<br><em>Research, ranked on your device.</em></h2></div>
84  <div id="live-search">
85   <form id="live-form" class="card"><label>Research question<input name="query" required minlength="3" maxlength="500" value="peppermint oil abdominal pain in
   irritable bowel syndrome" autocomplete="off"></label><div class="actions"><button type="submit" disabled name="method" value="ai">Search with on-device
   AI</button><button type="submit" disabled name="method" value="bm25" class="quiet">Keyword search</button></div><p class="small">A neural reranker over <span data-
   metric="library_count">the bundled</span> open-access abstracts. Your question never leaves this device (\u224839 MB on first use).</p></form>
```

```
  <p id="live-status" role="status" aria-live="polite"></p><div id="live-results" tabindex="-1" hidden></div>
 </div>
 <details id="evidence" class="card evidence-box"><summary>Does a study apply to this bottle? <span class="small">Seven-dimension comparisons from the full app</span></summary><p class="recording-note">Five examples replayed from the full app.</p><div id="case-tabs" class="case-tabs" role="group" aria-label="Choose an example"></div><p id="case-status" role="status">Loading\u2026</p><div id="case-detail"></div></details>
</section>

<section id="how" class="section methods" aria-labelledby="methods-title"><h2 id="methods-title">AI for the hard parts.<br><em>Sources for every warning.</em></h2>
 <div class="four">
  <article class="card"><span class="number"><span class="pill pill-ai">AI</span> 01</span><h3>Describe your medicine.</h3><p>\u201cMy water pill\u201d, \u201cmy cholesterol pill\u201d: a language model suggests a likely medicine group. Uncertain descriptions stay open. You and your pharmacist confirm the exact medicine.</p></article>
  <article class="card"><span class="number"><span class="pill pill-ai">AI</span> 02</span><h3>Find evidence for your goal.</h3><p>A neural model ranks studies for each herb and each reason you entered. Expand a lead to compare the study\u2019s own population, preparation and comparator wording with your bottle.</p></article>
  <article class="card rule"><span class="number">Source</span><h3>Warnings are NIH\u2019s words.</h3><p>No model writes a warning. Every one is an NIH sentence the build verifies word for word. <span id="kb-stats"></span></p></article>
  <article class="card"><span class="number">Photo</span><h3>Start with the label.</h3><p>Search NIH, paste the ingredients, or photograph the panel. On-device text recognition turns the photo into editable text for you to check.</p></article>
 </div>
 <div class="measured card"><h3>Measured, not promised</h3><div class="measure-grid g4"><p><strong data-metric="medicine_wrong">\u2014</strong><span>wrong AI group assignments in <span data-metric="medicine_phrases">the held-out</span> synthetic English phrases</span></p><p><strong data-metric="medicine_recall">\u2014</strong><span>medicine descriptions assigned correctly (<span data-metric="medicine_correct">\u2014</span>/<span data-metric="medicine_drugs">\u2014</span>); the rest stayed unknown</span></p><p><strong data-metric="species_precision">\u2014</strong><span>exact plant named from the label, on <span data-metric="labels_holdout">\u2014</span> unseen NIH labels</span></p><p><strong data-metric="blend_percent">\u2014</strong><span>of botanical ingredients on <span data-metric="labels_total">\u2014</span> real labels had no individual amount</span></p></div><p class="small">Language tests use synthetic phrases, not patient outcomes. No pharmacist review or clinical validation yet. Methods and results ship with the source.</p></div>
 <div class="limits card"><h3>What it won\u2019t do</h3><ul><li>Say a combination is \u201csafe\u201d.</li><li>Replace a pharmacist or give doses.</li><li>Check herbs beyond the 25 it covers \u2014 those are listed as \u201cnot covered\u201d.</li></ul><p class="small"><a href="safety-knowledge.json">Knowledge base</a> \xb7 <a href="DATA-NOTICE.md">Sources</a> \xb7 <a href="one-page.pdf">One-page overview</a> \xb7 <a href="code.pdf">Code PDF</a> \xb7 <a href="source.zip" download>Full source &amp;amp; local app</a></p></div>
</section>
</main><footer><span class="brand">f. FoliaTrace</span><p>DSH Hacks V2 \xb7 Not medical advice</p><a href="DATA-NOTICE.md">Data attribution</a><a href="CODE-LICENSE.txt">Code license</a></footer>
<div id="print-root" aria-hidden="true"></div>
</body></html>
```

frontend/demo/medicine-language.js
SHA-256: eed62ab559ce02a59f85151ac1936547938711172cbbe3139ba1fd278ec3875d

```
// Conservative exclusions before dictionary or semantic inference. A historical,
// negated or supplement-only mention is not evidence of a current medicine.
export function medicineExclusion(text) {
  const s = String(text).toLowerCase().replace(/[\u2019]/g, "'");
  if (/\b(not taking|no longer|stopped|used to|don't take|do not take|never take|without medication|not on|none)\b/.test(s)) return 'not_current';
  if (/\b(herbal|tea|fish oil|vitamin|multivitamin|supplement|probiotic)\b/.test(s)) return 'not_medicine';
  if (/^(the pill|a pill|pill|medicine|medication|tablets?|a small white pill|i cannot remember)$/i.test(s.trim())) return 'too_vague';
  return null;
}
```

frontend/demo/model-search-worker.js
SHA-256: b009558eabf8a07529908afcc93e739e42846de0c120bb81d5b5ff9e0624467a

```
import {encodePair} from './pair-tokenizer.js';
import {sourceContext, excludedFromPatientLeads} from './research-evidence.js';
let runtime, session, tokenizer, library;
const cache = new Map();
const normalized = text => text.toLowerCase().normalize('NFKD').replace(/\p{M}/gu, '').replace(/[\u2018\u2019]/g, "'").replace(/<[^>]*>/g, '').replace(/[^a-z0-9]+/g,
   ' ').trim();
const tokens = text => normalized(text).match(/[a-z]{2,}/g) || [];
async function loadLibrary() {
  if (library) return library;
  const r = await fetch(new URL('browser-library.json', self.location));
  if (!r.ok) throw Error('The bundled literature library is unavailable.');
  const papers = await r.json(), df = new Map(); let total = 0;
  for (const p of papers) {
    p.searchText = ' ' + normalized(p.title + ' ' + p.abstract) + ' ';
    p.words = tokens(p.title + ' ' + p.abstract); p.counts = new Map(); total += p.words.length;
    for (const w of p.words) p.counts.set(w, (p.counts.get(w) || 0) + 1);
    for (const w of p.counts.keys()) df.set(w, (df.get(w) || 0) + 1);
  }
  library = {papers, df, avg: total / papers.length}; return library;
}
function bm25(query, index, herbNames, patientLeads) {
  const words = new Set(tokens(query));
  const names = (herbNames || []).filter(n => typeof n === 'string' && n.length >= 3).map(n => ' ' + normalized(n) + ' ');
  return index.papers.filter(p => (!patientLeads || !excludedFromPatientLeads(p)) && (!names.length || names.some(n => p.searchText.includes(n)))).map(p => {
    let score = 0;
    for (const w of words) {
      const tf = p.counts.get(w) || 0; if (!tf) continue;
      const idf = Math.log(1 + (index.papers.length - (index.df.get(w) || 0) + .5) / ((index.df.get(w) || 0) + .5));
      score += idf * tf * 2.5 / (tf + 1.5 * (.25 + .75 * p.words.length / index.avg));
    }
    return {paper: p, score};
  }).filter(r => r.score > 0).sort((a,b) => b.score-a.score || a.paper.id.localeCompare(b.paper.id)).slice(0,20);
}
const sha = async bytes => [...new Uint8Array(await crypto.subtle.digest('SHA-256', bytes))].map(b=>b.toString(16).padStart(2,'0')).join('');
async function loadModel(id) {
  if (session) return;
  postMessage({id,type:'progress',text:'Loading the local AI runtime and 23 MB model. Your question stays on this device.'});
  const [ort,tok,manifest,tbytes] = await Promise.all([
    import(new URL('inference/ort.wasm.min.mjs',self.location).href), import(new URL('inference/tokenizers.mjs',self.location).href),
    fetch(new URL('inference/manifest.json',self.location)).then(r=>r.json()), fetch(new URL('inference/tokenizer.json',self.location)).then(r=>r.arrayBuffer())]);
  if (await sha(tbytes) !== manifest.tokenizer_sha256) throw Error('Tokenizer checksum mismatch.');
  const response = await fetch(new URL('inference/model.onnx', self.location));
  if (!response.ok) throw Error('Model download failed.');
  const bytes = await response.arrayBuffer();
  if (await sha(bytes) !== manifest.model_sha256) throw Error('Model checksum mismatch.');
  runtime = ort; runtime.env.wasm.numThreads = 1; runtime.env.wasm.wasmPaths = new URL('inference/',self.location).href;
  tokenizer = new tok.Tokenizer(JSON.parse(new TextDecoder().decode(tbytes)),{});
  session = await runtime.InferenceSession.create(new Uint8Array(bytes),{executionProviders:['wasm']});
}
let queue = Promise.resolve();
self.onmessage = ({data}) => { queue = queue.then(() => search(data)); };
async function search({id,query,method,herbNames,patientLeads}) {
  if (typeof query !== 'string' || query.trim().length < 3 || query.length > 500 || !['bm25','ai'].includes(method) || (herbNames && (!Array.isArray(herbNames) ||
 herbNames.length > 100))) {
    postMessage({id,type:'error',text:'Enter a research question of 3\u2013500 characters.'}); return;
```

```
  }
  try {
    const start = performance.now(), index = await loadLibrary();
    const key = JSON.stringify([query,method,herbNames,!!patientLeads]);
    if (cache.has(key)) { postMessage({...cache.get(key),id,seconds:(performance.now()-start)/1000}); return; }
    let rows = bm25(query,index,herbNames,patientLeads), baseline = rows.map(r=>r.paper.id);
    if (method === 'ai' && rows.length) {
      await loadModel(id);
      for (let i=0; i<rows.length; i++) {
        postMessage({id,type:'progress',text:'Prioritising source '+(i+1)+' of '+rows.length+' on your device\u2026'});
        const encoded = encodePair(tokenizer,query,rows[i].paper.title+' '+rows[i].paper.abstract), feed = {};
        for (const name of session.inputNames) feed[name] = new runtime.Tensor('int64',BigInt64Array.from(encoded[name].map(BigInt)),[1,encoded.input_ids.length]);
        let output;
        try {
          output = await session.run(feed); const value = Number(output[session.outputNames[0]].data[0]);
          if (!Number.isFinite(value)) throw Error('Invalid relevance score.'); rows[i].score = value;
        } finally { Object.values(output || {}).forEach(t=>t.dispose()); Object.values(feed).forEach(t=>t.dispose()); }
      }
      rows.sort((a,b)=>b.score-a.score || a.paper.id.localeCompare(b.paper.id));
    }
    const result = {type:'result',query,method,library_count:index.papers.length,candidate_count:rows.length,seconds:(performance.now()-start)/1000,baseline,
      rows:rows.slice(0,5).map(({paper,score})=>({id:paper.id,title:paper.title,url:paper.url,abstract:paper.abstract,licence:paper.licence,score,context:sourceConte
xt(paper)}))};
    if (cache.size >= 100) cache.delete(cache.keys().next().value); cache.set(key,result);
    postMessage({...result,id});
  } catch(error) { postMessage({id,type:'error',text:'Could not run '+(method==='ai'?'local AI':'keyword')+' search: '+error.message+' Your preparation is
unchanged.'}); }
}
```

frontend/demo/pair-tokenizer.js
SHA-256: f91378e22802558963d349cd5d3ff79a1bf5d42fa9ec52b91271224fba43537d

```
// Match the pinned Python reranker's longest-first 384-token pair encoding.
export function encodePair(tokenizer,query,document,maxLength=384){
 const a=tokenizer.encode(query,{add_special_tokens:false}).ids;
 const b=tokenizer.encode(document,{add_special_tokens:false}).ids;
 const available=maxLength-3;
 if(a.length+b.length>available){
  // Rust tokenizers balances lengths in original length order. The originally
  // longer sequence receives the odd token; equal original lengths favour B.
  if(a.length>b.length){const keepB=Math.min(b.length,Math.floor(available/2));a.splice(available-keepB);b.splice(keepB);}
  else{const keepA=Math.min(a.length,Math.floor(available/2));a.splice(keepA);b.splice(available-keepA);}
 }
 return
{input_ids:[101,...a,102,...b,102],attention_mask:Array(a.length+b.length+3).fill(1),token_type_ids:[...Array(a.length+2).fill(0),...Array(b.length+1).fill(1)]};
}
```

frontend/demo/research-evidence.js
SHA-256: 6458b478bfa95434dd2b3d357f0ed37f3bcb5855bb78caad08eb177111df7254

```
// Literal abstract excerpts with stable character offsets. These are text cues,
// not an eligibility decision or a clinical interpretation.
export function researchQueries(herb, reason) {
  const goals = String(reason || '').split(/\s+and\s+|\s*&\s*|[,;|]/i).map(s => s.trim()).filter(Boolean).slice(0, 3);
  return (goals.length ? goals : ['reported health outcomes']).map(goal => ({goal, query: `Clinical studies of ${herb} for ${goal} in people`}));
}
export function excludedFromPatientLeads(paper) {
  // Explicit title cues only. Passing this filter does not prove a human study.
  return /\b(rats?|mice|murine|canine|dogs?|cats?|quails?|rainbow trout|broiler|in vitro|in silico|hepg2|cell
lines?|foliar|irrigation|drought|cultivation|seedlings?|nanoemulsion|nanoparticles?|metal content|phytochemical profiling|protocol)\b/i.test(paper.title);
}
export function sourceContext(paper) {
  const abstract = String(paper.abstract || '');
  const sentences = [...abstract.matchAll(/[^.!?]+(?:[.!?]+|$)/g)].map(m => {
    const start = m.index + m[0].search(/\S/); const text = m[0].trim();
    return {text, start, end: start + text.length};
  }).filter(s => s.text.length >= 20);
  const patterns = [
    ['Population',
/\b(patients?|participants?|subjects?|individuals?|adults?|children|adolescents?|elderly|women|men|volunteers?|rats?|mice|quails?|dogs?|cats?|cells?)\b/i],
    ['Preparation / amount', /\b(extract|capsules?|tablets?|\d+\s*(?:mg|g\/day|ml)|standardized|standardised|oral|topical)\b/i],
    ['Study design / comparator', /\b(randomi[sz]ed|placebo|double.blind|systematic review|meta.analysis|cohort|comparator)\b/i],
  ];
  return patterns.map(([label, pattern]) => ({label, span: sentences.find(s => pattern.test(s.text)) || null}));
}
```

frontend/demo/safety-engine.js
SHA-256: c2e7f57c06739264fc07d925ba0a0ffc8904a68fe38914b6768982252156ff9d

```
// FoliaTrace safety engine. Pure functions, no DOM, no network.
// Matches label wording to NIH (NCCIH) fact-sheet statements and the person's medicines.
// It never chooses between candidate species and never reports "safe".
import {medicineExclusion} from './medicine-language.js';

export const norm = s => String(s ?? '').normalize('NFKD').replace(/[\u0300-\u036f]/g, '')
  .replace(/[\u2018\u2019\u02bc`\xb4]/g, "'").replace(/[\u201c\u201d]/g, '"').replace(/[\u2010-\u2015]/g, '-')
  .replace(/\xd7/g, 'x').toLowerCase().replace(/\s+/g, ' ').trim();

const escapeRe = s => s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
const wordRe = alias => new RegExp('(^|[^a-z0-9])' + escapeRe(norm(alias)).replace(/\\ /g, '[\\s-]+').replace(/'/g, "'?") + '(?=$|[^a-z0-9])', 'i');

export function levenshtein(a, b, max = 3) {
  if (Math.abs(a.length - b.length) > max) return max + 1;
  let prev = Array.from({length: b.length + 1}, (_, i) => i);
  for (let i = 1; i <= a.length; i++) {
    const cur = [i]; let best = i;
    for (let j = 1; j <= b.length; j++) {
      cur[j] = Math.min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (a[i - 1] === b[j - 1] ? 0 : 1));
      best = Math.min(best, cur[j]);
    }
    if (best > max) return max + 1;
    prev = cur;
  }
  return prev[b.length];
}
const tolerance = len => len >= 9 ? 2 : len >= 5 ? 1 : 0;

// ------------------------------------------------------------ knowledge index
export function indexKnowledge(kb) {
  const herbs = new Map(kb.herbs.map(h => [h.id, {...h, res: h.aliases.map(a => ({alias: a, re: wordRe(a)}))}]));
  const meds = new Map();
  for (const m of kb.medicines) for (const a of [m.name, ...m.aliases]) meds.set(norm(a), m);
  const ambiguous = kb.ambiguous.map(a => ({...a, re: new RegExp(a.pattern, 'i'), qre: new RegExp(a.qualified, 'i')}));
  const hints = kb.unverified_hints.map(h => ({hint: h, re: wordRe(h)}));
  return {kb, herbs, meds, ambiguous, hints};
}

// ------------------------------------------------------------------ products
const AMOUNT = /(\d+(?:[.,]\d+)?)\s*(mcg|\xb5g|ug|mg|g|iu|ml|%)\b/i;

/** Parse free label text (typed, pasted or OCR) into ingredient lines. */
export function parseLabelText(text, name = 'Product') {
  const lines = String(text || '').split(/\r?\n|;|\u2022|\xb7/).map(l => l.replace(/\s+/g, ' ').trim()).filter(l => l.length > 1);
  const ingredients = []; let blend = null;
  for (const line of lines) {
    const amount = line.match(AMOUNT);
    const isBlend = /proprietary|blend|complex|matrix|formula\b/i.test(line);
    if (isBlend) {
      const [head, ...rest] = line.split(/:\s*/);
      blend = {name: head.replace(AMOUNT, '').replace(/[\-\u2013\s]+$/, '').trim(), total: amount ? amount[0] : null};
      // Ingredients listed on the same line after a colon belong to the blend.
      const after = rest.join(':');
      if (after) for (const part of after.split(/,(?![^()]*\))/)) {
        const t = part.trim(); const a = t.match(AMOUNT);
```

```
        if (t.length > 1) ingredients.push({text: t, name: t.replace(AMOUNT, '').trim(), amount: a ? a[0] : null, blend: a ? null : blend.name, blendTotal: a ? null : blend.total});
      }
      continue;
    }
    if (/^(other ingredients|inactive|directions|suggested use|warning|caution|serving size|servings per|amount per|% ?daily|supplement facts|keep out|store)/i.test(line)) { blend = null; if (!/^(other ingredients|inactive)/i.test(line)) continue; ingredients.push({text: line, other: true}); continue; }
    for (const part of line.split(/,(?![^()]*\))/)) {
      const t = part.trim(); if (t.length < 2) continue;
      const a = t.match(AMOUNT);
      ingredients.push({text: t, name: t.replace(AMOUNT, '').replace(/\s+/g, ' ').trim() || t, amount: a ? a[0] : null, blend: blend && !a ? blend.name : null, blendTotal: blend && !a ? blend.total : null});
    }
  }
  return {name, source: 'text', ingredients, raw: String(text || '')};
}

/** Convert an NIH DSLD label record into the same product shape. */
export function fromDsldLabel(label) {
  const ingredients = [];
  const walk = (rows, blend) => {
    for (const r of rows || []) {
      const q = (r.quantity || [])[0];
      const amount = q && q.quantity ? `${q.quantity} ${q.unit || ''}`.trim() : null;
      const notes = r.notes || '';
      const genus = (notes.match(/Genus:\s*([A-Za-z]+)/) || [])[1] || null;
      const species = (notes.match(/Species:\s*([a-z][a-z-]+)/) || [])[1] || null;
      const part = (notes.match(/PlantPart:\s*([A-Za-z ,]+?)(?=\s+(?:Genus|Species|Form)|$)/) || [])[1] || null;
      const forms = (r.forms || []).map(f => f.name).filter(Boolean);
      const nested = r.nestedRows && r.nestedRows.length;
      if (nested) { walk(r.nestedRows, {name: r.name, total: amount}); continue; }
      ingredients.push({text: [r.name, notes, forms.join(' ')].filter(Boolean).join(' \xb7 '), name: r.name, category: r.category,
        amount, genus, species, part, blend: blend && !amount ? blend.name : null, blendTotal: blend && !amount ? blend.total : null});
    }
  };
  walk(label.ingredientRows);
  for (const o of (label.otheringredients?.ingredients || [])) ingredients.push({text: o.name, name: o.name, other: true});
  const statements = (label.statements || []).filter(s => /precaution|warning/i.test(s.type || '')).map(s => s.notes).filter(Boolean);
  return {name: label.fullName || 'NIH label', brand: label.brandName || '', source: 'dsld', dsldId: label.id,
    url: `https://dsld.od.nih.gov/label/${label.id}`, version: label.productVersionCode || null, upc: label.upcSku || null,
    offMarket: label.offMarket === 1, entered: (label.events || []).find(e => /entered/i.test(e.type))?.date || null,
    labelWarnings: statements, ingredients};
}

// ------------------------------------------------------------ herb matching
function matchFuzzy(textNorm, alias) {
  const words = textNorm.split(/[^a-z0-9']+/).filter(Boolean), aw = alias.split(' ');
  if (alias.length < 5) return false;
  for (let i = 0; i + aw.length <= words.length; i++) {
    const w = words.slice(i, i + aw.length).join(' ');
    if (Math.abs(w.length - alias.length) <= 2 && levenshtein(w, alias, 2) <= tolerance(alias.length)) return w;
  }
  return false;
}

/** Find NIH-covered herbs, ambiguous names and unverified botanicals in a product. */
export function identify(product, idx, {fuzzy = false} = {}) {
  const found = [];
```

```
  for (const [i, ing] of product.ingredients.entries()) {
    if (ing.other) continue;
    const t = norm(ing.text);
    const hit = new Set();
    // 1. Label-declared genus/species (DSLD) resolves identity directly.
    if (ing.genus) {
      const latin = norm(ing.genus + ' ' + (ing.species || ''));
      for (const h of idx.herbs.values()) {
        if (h.latin.some(l => norm(l).startsWith(latin.trim())) || h.res.some(r => r.alias === latin.trim())) {
          found.push({ingredient: i, herb: h.id, matched: `${ing.genus} ${ing.species || ''}`.trim(), basis: 'label_species'}); hit.add(h.id);
        }
      }
      if (!hit.size && ing.species) {
        found.push({ingredient: i, herb: null, unverified: `${ing.genus} ${ing.species}`, basis: 'label_species'}); continue;
      }
    }
    if (hit.size) continue;
    // 2. Ambiguous common names (never auto-resolved).
    let ambiguousHit = false;
    for (const a of idx.ambiguous) if (a.re.test(t) && !a.qre.test(t)) { found.push({ingredient: i, ambiguous: a.term, matched: a.term, basis: 'name'}); ambiguousHit
   = true; }
    // 3. Specific names/aliases.
    for (const h of idx.herbs.values()) {
      if (hit.has(h.id)) continue;
      let m = h.res.find(r => r.re.test(t));
      let how = 'name';
      if (!m && fuzzy) { for (const r of h.res) { const w = matchFuzzy(t, r.alias); if (w) { m = {alias: r.alias, read: w}; how = 'fuzzy'; break; } } }
      if (m) {
        // Qualified non-covered species such as "Siberian ginseng" must not map to Asian ginseng.
        if (h.id === 'asian-ginseng' && /siberian|eleuthero|american|quinquefolius|indian|brazilian|peruvian/.test(t)) continue;
        found.push({ingredient: i, herb: h.id, matched: m.alias, read: m.read, basis: how}); hit.add(h.id);
      }
    }
    if (hit.size || ambiguousHit) continue;
    // 4. Botanicals outside the verified set.
    const hint = idx.hints.find(h => h.re.test(t));
    if (hint) found.push({ingredient: i, herb: null, unverified: hint.hint, basis: 'name'});
    else if (ing.category === 'botanical') found.push({ingredient: i, herb: null, unverified: ing.name || ing.text, basis: 'label_category'});
    else found.push({ingredient: i, herb: null, unmatched: true});
  }
  return found;
}

// ------------------------------------------------------------------ medicines
/**
 * Dictionary first (exact, brand, typo). Only if that fails, an on-device AI reading of the phrase
 * (ai[input] = {group, score}, from ai-worker.js) maps it to a medicine group; such a medicine is
 * marked inferred, so every finding and question says the exact medicine still needs confirming.
 */
export function resolveMedicines(inputs, idx, ai = {}) {
  const groups = new Map((idx.kb.ai_medicine_groups?.groups || []).map(g => [g.id, g]));
  return inputs.map(s => String(s).trim()).filter(Boolean).map(input => {
    if (medicineExclusion(input)) return {input, known: false, classes: [], excluded: medicineExclusion(input)};
    const r = resolveOne(input, idx);
    if (r.known) return r;
    const a = ai[input]; const g = a && Number.isFinite(a.score) && a.score >= idx.kb.ai_medicine_groups.threshold && a.group && groups.get(a.group);
    return g ? {input, known: true, inferred: true, name: g.label, group: g.id, classes: g.classes, aiScore: a.score} : r;
  });
```

```
}
function resolveOne(input, idx) {
  {
    const n = norm(input).replace(/\b\d+(\.\d+)?\s*(mg|mcg|g|units?|iu)\b/g, '').replace(/\b(tablet|tablets|daily|once|twice|er|xr|sr|hcl)\b/g, '').replace(/\s+/g, '
 ').trim();
    let m = idx.meds.get(n);
    if (!m) for (const [k, v] of idx.meds) if (k.length > 3 && (n === k || n.startsWith(k + ' ') || n.endsWith(' ' + k) || n.includes(' ' + k + ' '))) { m = v;
break; }
    let corrected = null;
    if (!m && n.length >= 5) {
      let best = 3;
      for (const [k, v] of idx.meds) { if (k.length < 5) continue; const d = levenshtein(n, k, 2); if (d <= tolerance(k.length) && d < best) { best = d; m = v;
corrected = k; } }
    }
    return m ? {input, name: m.name, classes: m.classes, known: true, corrected} : {input, known: false, classes: []};
  }
}

// ------------------------------------------------------------------ analysis
const LEVEL_ORDER = {discuss: 0, caution: 1, note: 2};

/**
 * Findings are grouped: one card per herb x reason (same medicines / conditions),
 * holding one or more verbatim NIH quotes and every product that contains the herb.
 * Statements that apply only if an ambiguous name is a particular species are
 * returned inside that identity gap (gap.candidates[].findings), never in the main list.
 * @param {{products: object[], medicines: string[], conditions: string[], fuzzy?: boolean}} input
 */
export function analyze(input, idx) {
  const {kb} = idx;
  const meds = resolveMedicines(input.medicines || [], idx, input.aiMedicines || {});
  const conditions = new Set(input.conditions || []);
  const findings = [], gaps = [], checked = [], unmatched = [], perHerb = new Map();

  const add = (list, f) => {
    const medKey = (f.medicines || []).map(m => m.input).sort().join(','), condKey = (f.conditions || []).slice().sort().join(',');
    const key = [f.level, f.herb, f.because, medKey, condKey, f.variant || ''].join('|');
    const q = {text: f.quote, items: f.items || []};
    const existing = list.find(x => x.key === key);
    if (existing) {
      const eq = existing.quotes.find(x => x.text === q.text);
      if (eq) { for (const i of q.items) if (!eq.items.includes(i)) eq.items.push(i); } else existing.quotes.push(q);
      if (!existing.products.includes(f.productName)) existing.products.push(f.productName);
      return;
    }
    const {quote, items, productName, ...rest} = f;
    list.push({...rest, key, quotes: [q], products: [productName], medicines: f.medicines || [], conditions: f.conditions || []});
  };

  const applyHerb = (herbId, ctx, list) => {
    const h = idx.herbs.get(herbId); const src = kb.sources[h.source];
    const base = {herb: h.id, herbName: h.name, productName: ctx.productName, source: {url: src.url, title: src.title, updated: src.updated}};
    let flagged = false; const matchedMeds = new Set();
    const text = norm(ctx.ingredientText) + ' ' + norm(ctx.productText);
    for (const v of h.variants || []) if (new RegExp(v.pattern, 'i').test(text)) {
      add(list, {...base, level: v.level || 'note', quote: v.quote, because: 'variant', variant: v.label}); flagged = true;
    }
    for (const r of h.rules) {
```

```
      if ((r.meds || []).length) {
        const hits = meds.filter(m => m.classes.some(c => r.meds.includes(c)));
        // One card per medicine keeps "who" unambiguous in the brief.
        for (const m of hits) {
          const cls = m.classes.find(c => r.meds.includes(c));
          const item = r.items ? m.classes.map(c => r.items[c]).find(Boolean) : null;
          // "metformin is metformin" says nothing: fall back to a descriptive group the medicine belongs to.
          let label = m.inferred ? m.name : kb.class_labels[cls];
          if (!m.inferred && norm(label) === norm(m.name)) label = m.classes.map(c => kb.class_labels[c]).find(l => l && norm(l) !== norm(m.name) && /^an? /.test(l))
     || null;
          add(list, {...base, level: r.level, quote: r.quote, items: item ? [item] : [], because: 'medicine', effect: r.effect,
            medicines: [{input: m.input, name: m.name, label, corrected: m.corrected, inferred: !!m.inferred, aiScore: m.aiScore}]});
          matchedMeds.add(m.input); flagged = true;
        }
      }
      const hitConds = (r.conditions || []).filter(c => conditions.has(c));
      if (hitConds.length) { add(list, {...base, level: r.level === 'discuss' ? 'discuss' : 'caution', quote: r.quote, because: 'condition', conditions: hitConds});
    flagged = true; }
      if (!r.meds && !r.conditions && !r.always && r.level === 'note') add(list, {...base, level: 'note', quote: r.quote, because: 'label'});
    }
    // "Interacts with many medicines" statements: attach the medicines not already covered by a specific statement.
    for (const r of h.rules.filter(r => r.always)) {
      if (r.level !== 'discuss') {
        // Already shown for the person's own condition: don't repeat it as a general caution.
        if (!(r.conditions || []).some(c => conditions.has(c))) add(list, {...base, level: r.level, quote: r.quote, because: 'always', effect: r.effect});
        flagged = true; continue;
      }
      const rest = meds.filter(m => !matchedMeds.has(m.input));
      if (!meds.length) { add(list, {...base, level: 'caution', quote: r.quote, because: 'always', effect: r.effect}); flagged = true; }
      else if (rest.length) { add(list, {...base, level: 'discuss', quote: r.quote, because: 'always', effect: r.effect,
        medicines: rest.map(m => ({input: m.input, name: m.name || m.input, label: 'another medicine you take'}))}); flagged = true; }
    }
    return flagged;
  };

  for (const [pi, product] of (input.products || []).entries()) {
    const productText = product.ingredients.map(i => i.text).join(' ') + ' ' + (product.name || '');
    const found = identify(product, idx, {fuzzy: input.fuzzy || product.source === 'ocr'});
    const seenBlend = new Set();
    for (const f of found) {
      const ing = product.ingredients[f.ingredient];
      const ctx = {productName: product.name, ingredientText: ing.text, productText};
      const label = ing.name || ing.text;
      if (ing.blend && !seenBlend.has(label)) {
        seenBlend.add(label);
        const g = gaps.find(x => x.type === 'blend' && x.product === pi && x.blend === ing.blend);
        if (g) g.ingredients.push(label); else gaps.push({type: 'blend', product: pi, productName: product.name, ingredients: [label], blend: ing.blend, total:
    ing.blendTotal});
      }
      if (f.herb) {
        const flagged = applyHerb(f.herb, ctx, findings);
        const prior = perHerb.get(f.herb) || new Set(); prior.add(pi); perHerb.set(f.herb, prior);
        const herb = idx.herbs.get(f.herb);
        checked.push({product: pi, productName: product.name, ingredient: label, herb: f.herb, herbName: herb.name,
          basis: f.basis, matched: f.matched, read: f.read || null, flagged, amount: ing.amount || null,
          species: ing.genus ? `${ing.genus} ${ing.species || ''}`.trim() : null, identity_note: herb.identity_note || null,
          source: kb.sources[herb.source]});
      } else if (f.ambiguous) {
```

```
        const a = idx.ambiguous.find(x => x.term === f.ambiguous);
        const candidates = a.candidates.map(c => {
          const list = [];
          if (c.herb) applyHerb(c.herb, ctx, list);
          return {...c, findings: list.filter(x => x.level !== 'note'), flagged: list.some(x => x.level !== 'note')};
        });
        gaps.push({type: 'identity', product: pi, productName: product.name, ingredient: label, amount: ing.amount || null, term: a.term, candidates,
          source: a.source_page ? {url: kb.sources[a.source_page].url, quote: a.source_quote} : null});
      } else if (f.unverified) {
        gaps.push({type: 'unverified', product: pi, productName: product.name, ingredient: label, name: f.unverified});
      } else if (f.unmatched) {
        unmatched.push({product: pi, productName: product.name, ingredient: label});
      }
    }
  }
  for (const [herb, products] of perHerb) if (products.size > 1)
    gaps.push({type: 'duplicate', herb, herbName: idx.herbs.get(herb).name, products: [...products].map(p => input.products[p].name)});
  for (const m of meds) if (!m.known) gaps.push({type: 'unknown_medicine', input: m.input});
  for (const m of meds) if (m.inferred) gaps.push({type: 'inferred_medicine', input: m.input, label: m.name, score: m.aiScore});

  findings.sort((a, b) => LEVEL_ORDER[a.level] - LEVEL_ORDER[b.level] || (a.because === 'always') - (b.because === 'always') ||
a.herbName.localeCompare(b.herbName));
  for (const f of findings) f.question = askLine(f, kb);
  for (const g of gaps) for (const c of g.candidates || []) for (const f of c.findings) f.question = askLine(f, kb, c.species);
  const counts = {discuss: findings.filter(f => f.level === 'discuss').length, caution: findings.filter(f => f.level === 'caution').length,
    note: findings.filter(f => f.level === 'note').length, gaps: gaps.length, checked: checked.length,
    identity: gaps.filter(g => g.type === 'identity').length};
  return {findings, gaps, checked, unmatched, medicines: meds, conditions: [...conditions], counts, kbVersion: kb.version};
}

export function askLine(f, kb, ifSpecies = null) {
  const who = f.medicines.map(m => m.input).join(', ');
  const products = f.products.join(' and ');
  const herb = ifSpecies ? `${f.herbName} (if the label's ingredient is ${ifSpecies})` : f.herbName;
  if (f.because === 'medicine' && f.medicines[0]?.inferred) return `I take ${who}, which I think is ${f.medicines[0].label}. Which medicine is it exactly, and is it
OK with the ${herb} in ${products}?`;
  if (f.because === 'medicine') return `I take ${who}. ${products} contains ${herb}. Should I take them together, and is there anything to watch for?`;
  if (f.because === 'always' && who) return `${products} contains ${herb}, which NIH says interacts with many medicines. Could it affect ${who}?`;
  if (f.because === 'condition') return `${f.conditions.map(c => kb.conditions[c]).join('; ')}: is the ${herb} in ${products} appropriate for me?`;
  if (f.because === 'variant') return `${products} uses a \u201c${f.variant.toLowerCase()}\u201d form of ${f.herbName}. Does that change anything for me?`;
  if (f.because === 'label') return `Can we check the quality and contents of the ${f.herbName} in ${products}?`;
  return `Is the ${herb} in ${products} right for me?`;
}
```

frontend/demo/safety-examples.js
SHA-256: 8cfd1c269c2d9e0811a8093a1834d3a775aa45653628ccae980887725dbda85c

```
// Fictional people and fictional labels for demonstration. No real brand or person.
// Medicines are written the way people actually say them; the on-device AI understands the vague ones.
export const EXAMPLES = [
  {
    id: 'mei', person: 'Mei, 72', story: 'Takes warfarin after a heart-valve repair, plus \u201cher water pill\u201d. Her daughter bought her a \u201cmemory & joint\u201d supplement.',
    medicines: ['Warfarin 5 mg', 'her water pill'], conditions: ['surgery'], reason: 'memory and joint pain',
    products: [{name: 'Memory & Joint Support (fictional)', text:
`Supplement Facts
Serving Size 2 Capsules
Ginkgo biloba leaf extract 120 mg
Turmeric root extract (Curcuma longa) 500 mg
Black pepper extract (BioPerine) 5 mg
Proprietary Wellness Blend 300 mg: Garlic bulb, Ginger root, Ginseng root
Other ingredients: cellulose, rice flour`}],
  },
  {
    id: 'sam', person: 'Sam, 29', story: 'Takes sertraline and the pill. Started an over-the-counter \u201cmood\u201d supplement before exams.',
    medicines: ['Sertraline', 'Birth control pill'], conditions: [], reason: 'mood and stress',
    products: [{name: 'Calm Mood Blend (fictional)', text:
`Supplement Facts
St. John's Wort aerial parts extract (0.3% hypericin) 300 mg
Valerian root 200 mg
Chamomile flower 100 mg
Other ingredients: gelatin, magnesium stearate`}],
  },
  {
    id: 'ray', person: 'Ray, 61', story: 'On metformin and \u201cmy cholesterol pill\u201d \u2014 can\u2019t remember its name. Wants to \u201cgo natural\u201d for cholesterol and energy.',
    medicines: ['Metformin', 'my cholesterol pill', 'Lisinopril'], conditions: ['diabetes'], reason: 'cholesterol and energy',
    products: [
      {name: 'Heart Health Formula (fictional)', text:
`Red Yeast Rice 1200 mg
Green Tea Extract (50% EGCG) 250 mg
Other ingredients: vegetable cellulose`},
      {name: 'Daily Energy (fictional)', text:
`Ginseng root powder 400 mg
Green tea leaf extract 100 mg
Goldenseal root 100 mg`}],
  },
];
```

frontend/handoff.js
SHA-256: 39d817a2263bd74f500cc29d5e254202ef9b140245e64b159cadfc65c7d83bb8

```
// Portable reading decisions are distinct from inference and never change its labels.
export function initHandoffs({getAnalysis, api, el, link, showView}) {
  const $ = s => document.querySelector(s);
  let composedRun = null, serial = 0;
  const report = (message, bad=false) => {$('#handoff-list-status').textContent=message;$('#handoff-list-status').className=bad?'error':'hint';};
  function field(form, label, tag, name, maxLength) {
    const wrap=el('label',label), input=el(tag);
    input.name=name;input.required=true;
    if(maxLength)input.maxLength=maxLength;
    if(tag==='textarea')input.rows=3;
    wrap.append(input);form.append(wrap);return input;
  }
  function compose() {
    const data=getAnalysis(),box=$('#handoff-compose');
    if(!data){box.replaceChildren(el('h2','Start with a source'),el('p','Analyze a source in the workspace, check its possible species and studies, then record your reading here.'));const start=el('button','Open workspace');start.onclick=()=>showView('workspace');box.append(start);return;}
    if(data.run_id===composedRun)return;
    composedRun=data.run_id;box.replaceChildren(el('h2','Create a source-scoped handoff'),el('p',data.source.title));
    const form=el('form'),question=field(form,'Research question','textarea','question',600),statement=field(form,'Source statement','select','claim_id');statement.required=false;
    const whole=el('option','Whole source \u2014 no specific statement');whole.value='';statement.append(whole);
    for(const c of data.historical_claims){const opt=el('option',c.support.text);opt.value=c.id;statement.append(opt);}
    const quote=el('blockquote'),assumptions=el('fieldset'),legend=el('legend','Candidate assumptions \u2014 these do not establish the source identity');assumptions.append(legend);form.append(quote,assumptions);
    const decision=field(form,'Reading decision','select','decision');
    for(const [value,text] of [['INSUFFICIENT_INFORMATION','Insufficient information'],['NEXT_READING','Continue to the next reading'],['EXCLUDE_FROM_THIS_QUESTION','Exclude from this research question']]){const opt=el('option',text);opt.value=value;decision.append(opt);}
    const reason=field(form,'Reason and remaining uncertainty','textarea','reason',3000),next=field(form,'Next verification step','textarea','next_step',2000),reviewer=field(form,'Recorded by','input','reviewer',80);reviewer.value='Local researcher';
    const save=el('button','Save evidence handoff');save.type='submit';const state=el('p','','hint');state.setAttribute('role','status');state.setAttribute('aria-live','polite');form.append(save,state);box.append(form);
    const key='foliatrace.handoff.draft.'+data.run_id;
    function candidates(selectedIds=null){
      const c=data.historical_claims.find(c=>c.id===statement.value);quote.textContent=c?c.support.text:'Whole source reading. Select a statement above to restrict the handoff and its studies.';
      assumptions.replaceChildren(legend);const seen=new Set();
      for(const g of data.identity_counterfactuals){if(c&&!g.mention_ids.some(id=>c.mention_ids.includes(id)))continue;for(const b of g.branches){if(seen.has(b.taxon_id))continue;seen.add(b.taxon_id);const label=el('label'),input=el('input');input.type='checkbox';input.name='candidate';input.value=b.taxon_id;input.checked=selectedIds?selectedIds.includes(b.taxon_id):true;label.append(input,document.createTextNode(' '+b.name));assumptions.append(label);}}
      if(!seen.size)assumptions.append(el('p','No candidate for this selection. The unresolved source and your next step can still be saved.','hint'));
    }
    function draft(){return {question:question.value,claim_id:statement.value||null,candidate_ids:[...assumptions.querySelectorAll('input:checked')].map(x=>x.value),decision:decision.value,reason:reason.value,next_step:next.value,reviewer:reviewer.value};}
    function persist(){try{localStorage.setItem(key,JSON.stringify(draft()));}catch{state.textContent='Draft storage unavailable. Keep this page open and save the handoff to this local service before leaving.';}}
    let saved=null;try{saved=JSON.parse(localStorage.getItem(key)||'null');}catch{}
    if(saved){question.value=saved.question||'';if([...statement.options].some(o=>o.value===(saved.claim_id||'')))statement.value=saved.claim_id||'';decision.value=['NEXT_READING','INSUFFICIENT_INFORMATION','EXCLUDE_FROM_THIS_QUESTION'].includes(saved.decision)?saved.decision:'INSUFFICIENT_INFORMATION';reason.value=saved.reason||'';next.value=saved.next_step||'';reviewer.value=saved.reviewer||'Local researcher';}
    candidates(saved?.candidate_ids);statement.addEventListener('change',()=>{candidates();persist();});form.addEventListener('input',persist);form.addEventListener('change',persist);
    form.addEventListener('submit',async event=>{event.preventDefault();save.disabled=true;state.textContent='Saving the source, candidate assumptions and research checks\u2026';try{const record=await api('/api/analyses/'+data.run_id+'/handoffs',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(draft())});state.textContent='Saved. Download JSON for another installation or Markdown for a reading note.';showRecord(record);await refresh();}catch(error){state.textContent='Not saved: '+error.message+'. Your draft remains in this
```

```
form.';}finally{save.disabled=false;}});
  }
  function showRecord(record){
    const box=$('#handoff-detail'),p=record.packet,a=p.assessment;box.hidden=false;box.replaceChildren(el('h2',a.question),el('p',record.origin==='IMPORTED_UNVERIFIED_RECORD'?'Imported record \xb7 author and scientific claims unverified':'Saved local research decision','pill warn'));
    const links=el('div',undefined,'row');for(const [format,label] of [['json','Download portable JSON'],['markdown','Download reading note']]){const anchor=el('a',label);anchor.href='/api/handoffs/'+record.id+(format==='markdown'?'?format=markdown':'');anchor.download='foliatrace-handoff-'+record.id+(format==='json'?'.json':'.md');links.append(anchor);}box.append(links,el('p',a.decision.replaceAll('_',' ')),el('p',a.reason),el('p','Next step: '+a.next_step),el('p','Recorded by '+a.reviewer+' \xb7 '+p.created_at,'hint'),el('h3','Source statement'),el('p',p.source.title),el('blockquote',p.statement?.support.text||p.source_input));
    if(p.source.url)box.append(link('Read source \u2197',p.source.url));box.append(el('p','Rights: '+(p.source.licence||'not established'),'hint'),el('h3','Candidate assumptions'));
    box.append(el('p',p.candidates.map(c=>c.name).join(' \xb7 ')||'No species selected; unresolved source.'));
    box.append(el('h3',p.studies.length+' candidate/study records'),el('p','Automatic checks describe the original analysis. Your recorded question and decision do not rerun or validate those checks.','hint'));
    for(const s of p.studies){const details=el('details'),summary=el('summary',s.paper.title+' \xb7 '+(p.candidates.find(c=>c.taxon_id===s.taxon_id)?.name||s.taxon_id));details.append(summary,link('Original study \u2197',s.paper.url),el('p','Conditional on candidate identity. '+s.relation.relation));if(s.relation.supporting_text)details.append(el('blockquote',s.relation.supporting_text));if(!s.paper.abstract)details.append(el('p','Abstract unavailable; read the original publication.','hint'));for(const check of s.comparability.checks||[])details.append(el('p',check.check+': '+check.status+' \xb7 '+(check.detail||''),'hint'));box.append(details);}
    const methods=el('section');
    for(const s of p.studies){const forms=s.study?.preparation_attribution?.forms||[];if(!forms.length)continue;methods.append(el('h3','Preparation wording \xb7 '+s.paper.title));for(const form of forms){methods.append(el('blockquote',form.context?.text||form.text));if(form.url)methods.append(link('Check preparation at source \u2197',form.url));if(form.licence)methods.append(el('p',form.licence,'hint'));}methods.append(el('p','Candidate-linked wording is a reading lead. It does not establish equivalent formulation, dose or causal effect.','hint'));}
    if(methods.childElementCount)box.append(methods);
    box.append(el('h3','Coverage and limitations'),el('p',p.coverage.publications+' publications included. Unselected candidates and studies remain outside this handoff.'));
    for(const text of p.limitations)box.append(el('p',text,'hint'));
    box.append(el('p','Analysis '+p.analysis_id+' \xb7 snapshot '+p.snapshot_sha256,'hint'),el('p','SHA-256 checks file integrity; it does not prove authorship or scientific correctness.','hint'));
  }
  async function refresh(){const current=++serial;try{const result=await api('/api/handoffs');if(current!==serial)return;const box=$('#handoff-list');box.replaceChildren();if(!result.handoffs.length)box.append(el('p','No saved handoffs yet.','hint'));for(const row of result.handoffs){const button=el('button',row.question,'handoff-list-item quiet');button.type='button';button.onclick=async()=>{try{showRecord(await api('/api/handoffs/'+row.id));report('Opened saved handoff.');}catch(error){report(error.message,true);}};box.append(button,el('p',row.source_title+' \xb7 '+row.saved_at,'hint'));}}catch(error){if(current===serial)report('Could not load saved handoffs: '+error.message,true);}}
  $('#handoff-refresh').addEventListener('click',refresh);
  $('#handoff-import').addEventListener('change',async event=>{const file=event.target.files[0];if(!file)return;if(file.size>20_000_000){report('Handoff exceeds 20 MB; existing records are unchanged.',true);return;}const body=new FormData();body.append('file',file);try{const record=await api('/api/handoffs/import',{method:'POST',body});showRecord(record);report('Imported as a separate unverified record. Existing decisions were kept.');await refresh();}catch(error){report('Import failed: '+error.message+'. Existing records were kept.',true);}finally{event.target.value='';}});
  return {open(){compose();return refresh();},analysisChanged(){composedRun=null;}};
}
```

frontend/index.html
SHA-256: 451665b09dbbfb8af7c2c4365fb52d017526dcc2fa8a9b5787f7b98ee5730d99

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="description" content="Prepare a clearer herbal-product conversation. Keep product labels, actual use, unanswered questions and pharmacist evidence reading together." />
    <title>FoliaTrace \xb7 From the bottle to a better conversation</title>
    <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'%3E%3Cpath fill='%231f5c46' d='M26 5C14 5 6 12 6 22c0 2 .4 3.6 1 5 2-8 8-13 15-15-6 4-10 9-12 16 1 .3 2 .5 3 .5C23 28.500 27 18 26 5z'/%3E%3C/svg%3E" />
    <link rel="stylesheet" href="/assets/style.css?v=20261106-clarity" />
    <link rel="stylesheet" href="/assets/care.css?v=20261112" />
    <script type="module" src="/assets/app.js"></script>
  </head>
  <body>
    <a class="skip" href="#main">Skip to content</a>
    <header class="topbar">
      <div class="brand-row">
        <a class="brand" href="/" aria-label="FoliaTrace home"><span class="brand-symbol" aria-hidden="true">f.</span>FoliaTrace</a>
        <p class="tagline">The bottle. The person. The evidence.</p>
        <span id="system-state" class="mode" title="What this page can do">Connecting\u2026</span>
      </div>
      <nav class="views" aria-label="Views">
        <button class="view-tab" data-view="consultation" aria-current="page">Prepare a conversation</button>
        <button class="view-tab" data-view="question">Read the evidence</button>
        <a class="view-link" href="/check/">Check a bottle against my medicines</a>
        <details class="research-tools" hidden><summary>Source and research tools</summary><div class="research-tools-menu">
        <button class="view-tab" data-view="library">Source library</button>
        <button class="view-tab" data-view="workspace">Workspace</button>
        <button class="view-tab" data-view="handoffs">Evidence handoffs</button>
        <button class="view-tab" data-view="map">Research map</button>
        <button class="view-tab" data-view="graph">Provenance graph</button>
        <button class="view-tab" data-view="chemistry">Compound assays</button>
        <button class="view-tab" data-view="evaluation">Methods &amp;amp; scope</button>
        <a class="view-link" id="api-link" href="/docs">API</a>
        </div></details>
      </nav>
      <p id="literature-availability" class="hint" hidden></p>
    </header>

    <main id="main">
      <section id="view-consultation" class="view knowledge-view care-view" aria-label="Herbal product consultation preparation">
        <header class="library-heading care-heading care-hero"><div class="care-hero-copy"><p class="kicker taxonomy">For patients, caregivers &amp;amp; the pharmacist who listens</p><h1>Bring the bottle.<br><em>Start a better conversation.</em></h1><p class="library-lede">\u201cI take mint capsules.\u201d A name is only the beginning. Bring the label, how you actually use it, and the questions you haven\u2019t answered yet.</p><div class="hero-actions"><button id="care-start-own" type="button">Prepare my product list <span aria-hidden="true">&#8594;</span></button><span>Three steps. Start with one product.</span></div><div class="hero-principles"><span>No account needed</span><span>Your consultation stays local</span><span>Unknown is an honest answer</span></div></div><aside class="hero-note" aria-label="The FoliaTrace workflow"><span class="note-number">One conversation, connected.</span><svg class="care-illustration" viewBox="0 0 360 170" aria-hidden="true"><circle cx="270" cy="70" r="65" fill="#e8eee0"/><path d="M105 110C150 140 150 30 205 65" stroke="#bd8540" stroke-width="3" fill="none" stroke-dasharray="5 7"/><g transform="rotate(-9 78 85)"><rect x="32" y="38" width="90" height="112" rx="18" fill="#dfe9cd" stroke="#315744" stroke-width="2"/><rect x="48" y="18" width="58" height="27" rx="7" fill="#315744"/><rect x="42" y="70" width="70" height="54" rx="4" fill="#fffdf7"/><path d="M65 110C60 89 85 79 96 80C96 100 82 109 65 110M67 108L87 87" stroke="#315744" stroke-width="2" fill="#dfe9cd"/></g><g transform="rotate(7 250 83)"><rect x="199" y="28" width="118" height="125" rx="9" fill="#fffdf7" stroke="#315744" stroke-width="2"/><rect x="214" y="44" width="62" height="7" rx="3" fill="#315744"/><path d="M218 75L223 81L234 68M218 105L223 111L234 98" fill="none" stroke="#315744" stroke-width="3"/><path d="M246 75H297M246 83H282M246 105H297M246 113H276M215 134H297" stroke="#abbca9" stroke-width="3" stroke-linecap="round"/></g><circle cx="166" cy="61" r="19" fill="#f2dbaa"/><text x="166" y="68" text-anchor="middle" fill="#69451e" font-family="Georgia,serif"
```

```
font-size="24">?</text></svg><ol><li><strong>The bottle</strong><span>Keep the exact label and version.</span></li><li><strong>The person</strong><span>Record actual
 use. Leave gaps visible.</span></li><li><strong>The evidence</strong><span>A pharmacist reads, compares and records why.</span></li></ol><p>A clearer handoff, with
its sources still attached.</p></aside></header>
42        <div class="care-start"><div><strong>Start with what you know.</strong><p>No botanical name or complete label required.</p></div><div class="row"><button
id="consultation-example" class="quiet" type="button">Explore a fictional example \u2197</button><button id="consultation-open-saved" class="quiet care-jump"
type="button">Continue a saved preparation</button><button id="consultation-undo-example" class="quiet" type="button" hidden>Restore my earlier
draft</button></div></div>
43        <p class="care-scope">Prepare a saved, versioned conversation with your pharmacist. To compare a label with NIH statements about your medicines, use <a
href="/check/">Check a bottle</a>. Product identity and safety still need professional assessment.</p>
44        <p id="consultation-status" role="status" aria-live="polite"></p>
45        <button id="consultation-undo-draft" class="quiet" type="button" hidden>Undo replacing my draft</button>
46        <div class="care-layout"><div class="care-main"><details id="consultation-editor" class="handoff-card care-editor" open><summary><span>Your draft</span><span
 class="hint">Products, actual use and questions</span></summary><div id="consultation-compose"></div></details><article id="consultation-packet" class="handoff-
card" aria-label="Saved consultation preparation" tabindex="-1" hidden></article></div><aside class="handoff-card care-saved" aria-label="Saved preparation
tools"><h2>Pick up where you left off</h2><p class="hint">The latest 100 records on this installation. Drafts stay in this browser.</p><label>Find a saved
preparation<input id="consultation-saved-filter" type="search" placeholder="Search visit labels" /></label><button id="consultation-refresh" class="quiet"
type="button">Refresh saved records</button><div id="consultation-saved"></div><details class="care-import"><summary>Restore a portable backup</summary><label>Open a
 portable preparation<input id="consultation-import" type="file" accept=".json,application/json" /></label><p class="hint">Imports create separate unverified
records. Exports can contain health information; choose what you share.</p></details></aside></div>
47      </section>
48      <section id="view-question" class="view knowledge-view care-view" aria-label="Professional health evidence review" hidden>
49        <header class="library-heading care-heading"><p class="kicker evidence">The professional reading workspace</p><h1>The same plant?<br><em>The same
preparation?</em></h1><p class="library-lede">A relevant paper is a starting point. Compare its sources with the saved product, keep uncertainty visible, and record
your reading decision.</p><button id="question-open-saved" class="quiet care-jump" type="button">Continue a saved evidence question \u2197</button></header>
50        <div id="question-product-context" class="care-context" hidden><p id="question-context-label"></p><button id="question-return" class="quiet"
type="button">Return to the product preparation</button></div>
51        <p id="question-status" role="status" aria-live="polite"></p>
52        <div class="care-layout"><div class="care-main"><details id="question-editor" class="handoff-card care-editor" open><summary><span>Your evidence
question</span><span class="hint">Question, study criteria and sources</span></summary><div id="question-compose"></div></details><article id="question-result"
class="handoff-card" aria-label="Evidence summary and studies" tabindex="-1" hidden></article></div><aside class="handoff-card care-saved" aria-label="Saved evidence
 tools"><h2>Saved evidence questions</h2><p class="hint">The latest 100 questions. Each run preserves its first result and reading history.</p><label>Find a saved
question<input id="question-saved-filter" type="search" placeholder="Search questions or plant names" /></label><button id="question-refresh" class="quiet"
type="button">Refresh questions</button><div id="question-saved"></div><details class="care-import"><summary>Restore a portable review</summary><label>Open a
portable evidence review<input id="question-import" type="file" accept=".json,application/json" /></label></details></aside></div>
53      </section>
54      <section class="source-bar" aria-label="Choose a source" hidden>
55        <div class="cases" aria-label="Guided cases">
56          <span class="label">Start with a guided case</span>
57          <div id="cases" class="row"></div>
58        </div>
59        <div class="source-pick">
60          <label for="passage">or choose any historical section</label>
61          <div class="row">
62            <select id="passage"><option>Loading sources\u2026</option></select>
63            <button id="analyze">Analyze</button>
64          </div>
65          <label id="analysis-ranking-label">Evidence ranking<select id="analysis-ranking"><option value="bm25">Keywords</option><option value="rerank">Local AI
reranking</option></select></label>
66        </div>
67        <details class="more-input" id="custom-input">
68          <summary>Paste or upload your own text</summary>
69          <label for="input">Text (up to 50,000 characters)</label>
70          <textarea id="input" rows="6" maxlength="50000"></textarea>
71          <div class="row">
72            <label class="file" for="text-file">UTF-8 .txt file</label>
73            <input id="text-file" type="file" accept=".txt,text/plain" />
74          </div>
75          <p class="hint">Any herbal, ethnobotanical study or folklore text, in English, Greek, Albanian, Italian, German, French, Spanish, Portuguese, Croatian,
```

```
    Romanian, Turkish or Latin. Your text stays on this machine: only plant names found in it are looked up in GBIF and PubMed. Rights of your text are not verified. No
    OCR.</p>
 76         </details>
 77         <details class="search" id="search-box">
 78           <summary>Search the snapshot</summary>
 79           <div class="row">
 80             <label class="sr-only" for="search">Search names, conditions, compounds or publications</label>
 81             <input id="search" type="search" placeholder="camomile, headache, salicin\u2026" maxlength="150" />
 82             <button id="search-button" class="quiet">Search</button>
 83           </div>
 84           <div id="search-results" aria-live="polite"></div>
 85         </details>
 86         <details class="source-compare" id="comparison-box">
 87           <summary>Compare historical sources</summary>
 88           <p class="hint">Compare recorded uses and candidate identities across passages. Shared terms do not establish a common historical identity.</p>
 89           <div class="row">
 90             <label>First passage<select id="compare-a"></select></label>
 91             <label>Second passage<select id="compare-b"></select></label>
 92             <button id="compare-run" class="quiet">Compare sources</button>
 93           </div>
 94           <div id="comparison-result" aria-live="polite"></div>
 95         </details>
 96       </section>
 97
 98       <div id="status" role="status" aria-live="polite" hidden>Loading the local research snapshot.</div>
 99
100       <section id="view-workspace" class="view workspace" aria-label="Workspace" hidden>
101         <article class="pane source-pane" aria-labelledby="source-title">
102           <header class="pane-head">
103             <p class="kicker historical">Historical attestation</p>
104             <h1 id="source-title">\u2014</h1>
105             <p id="source-meta" class="meta"></p>
106           </header>
107           <p class="hint">Highlighted words are plant names found in the text. Select one to inspect it.</p>
108           <details id="inspect-source-name" class="inspect-source-name">
109             <summary>Inspect a name the extractor missed</summary>
110             <p class="hint">Enter its exact wording from this source. Your selection stays labelled as researcher input; any botanical identification still needs
    review.</p>
111             <form id="inspect-source-name-form" class="row">
112               <label>Name in this source<input id="source-name-input" required minlength="2" maxlength="120" placeholder="e.g. Sanicle" /></label>
113               <button type="submit" class="quiet">Inspect name</button>
114               <button id="source-name-reset" type="button" class="text-action" hidden>Return to automatic extraction</button>
115             </form>
116             <p id="source-name-state" class="hint" aria-live="polite"></p>
117           </details>
118           <div id="original" class="original" tabindex="0" aria-label="Source text"></div>
119           <details id="claims-box" class="claims">
120             <summary id="claims-summary">Source statements</summary>
121             <div id="claims"></div>
122           </details>
123         </article>
124
125         <article class="pane inspector-pane" aria-labelledby="name-title">
126           <div class="names" role="group" aria-label="Plant names in this text" id="names"></div>
127           <header class="pane-head name-head">
128             <p class="kicker taxonomy">Selected name</p>
129             <h2 id="name-title">\u2014</h2>
130             <p id="name-status" class="status-line"></p>
```

```
131          </header>
132          <div id="research-summary" class="research-summary"></div>
133          <div id="research-next" class="research-next"></div>
134          <div class="tabs" role="tablist" aria-label="Inspect the selected name">
135            <button role="tab" id="tab-identity" aria-controls="panel-identity" aria-selected="true">1 \xb7 Identity</button>
136            <button role="tab" id="tab-evidence" aria-controls="panel-evidence" aria-selected="false" tabindex="-1">2 \xb7 Evidence</button>
137            <button role="tab" id="tab-review" aria-controls="panel-review" aria-selected="false" tabindex="-1">3 \xb7 Research brief</button>
138          </div>
139
140          <section id="panel-identity" role="tabpanel" aria-labelledby="tab-identity" class="tab-panel">
141            <div id="identity-matrix"></div>
142            <div id="whatif"></div>
143            <details class="identity-references-box"><summary>What other sources call this plant</summary><div id="historical-identity-references"></div></details>
144            <div id="identities" class="candidates"></div>
145            <div id="open-world"></div>
146            <details id="external-name-check">
147              <summary>Check a scientific name beyond the snapshot</summary>
148              <p class="hint">Enter a Latin name from the source. This optional GNverifier lookup needs the internet and does not settle what a historical common
    name meant.</p>
149              <form id="verify-name-form" class="row">
150                <label>Scientific name<input id="verify-scientific-name" required minlength="3" maxlength="120" placeholder="Thymus serpyllum" /></label>
151                <button type="submit" class="quiet">Verify name</button>
152              </form>
153              <div id="name-check-result" aria-live="polite"></div>
154            </details>
155          </section>
156
157          <section id="panel-evidence" role="tabpanel" aria-labelledby="tab-evidence" class="tab-panel" hidden>
158            <section class="statement-reading" aria-label="Source statement for this reading">
159              <label for="evidence-source-statement">Research a source statement<select id="evidence-source-statement"><option value="">All linked
    statements</option></select></label>
160              <div id="statement-reading-context"></div>
161            </section>
162            <p class="question" id="modern-question"></p>
163            <section id="study-comparison-selection" class="study-comparison-selection" aria-label="Selected studies for comparison" hidden></section>
164            <div class="filters">
165              <label>Candidate<select id="branch"><option value="">All candidates</option></select></label>
166              <label>Study design<select id="study-type"><option value="">All designs</option></select></label>
167              <label>Study reports<select id="relation">
168                <option value="">Any finding</option>
169                <option value="SUPPORTS">Benefit</option>
170                <option value="CONTRADICTS">No benefit</option>
171                <option value="MIXED">Mixed</option>
172                <option value="INSUFFICIENT">Inconclusive</option>
173                <option value="NO_RESULT_STATEMENT">No result stated</option>
174              </select></label>
175              <label>Since<input id="year" type="number" min="1900" max="2100" placeholder="Any year" /></label>
176            </div>
177            <p id="evidence-count" class="hint"></p>
178            <div id="blocking" class="blocking"></div>
179            <div id="conflicts"></div>
180            <div id="statement-reading-jump" class="statement-reading-jump" hidden></div>
181            <div id="evidence"></div>
182          </section>
183
184          <section id="panel-review" role="tabpanel" aria-labelledby="tab-review" class="tab-panel" hidden>
185            <div id="export-links" class="row" aria-label="Download this analysis"></div>
186            <div id="research-brief"></div>
```

```
            <h3 class="subhead">Researcher judgements</h3>
            <p class="hint">Your judgements stay with the same source and research question when evidence ranking changes. They remain separate from automated output and travel with exports. Changed source or publication wording needs a separate judgement.</p>
            <div id="reviews"></div>
            <h3 class="subhead">Export this analysis</h3>
            <p class="hint">Exports keep candidate identities, identity qualifiers, citations, licences, withdrawal state and limitations.</p>
          </section>
        </article>
      </section>

      <section id="view-map" class="view evaluation research-map" aria-label="Research map" hidden>
        <h2>Where history and modern research meet</h2>
        <p class="hint">Follow each chapter plant from an exact historical statement into modern research. Open a lead to inspect its source attribution, candidate identities and reading-priority calculation. Uncertain or shared plant subjects remain visible and add no priority weight. Independent records come from Dr. Duke's USDA database. A recorded use is not evidence that a treatment works.</p>
        <div class="map-summary" id="map-summary"></div>
        <div class="filters">
          <label>Plant or topic<input id="map-query" type="search" placeholder="feverfew, cough\u2026" maxlength="80" /></label>
          <label>Modern research<select id="map-state"><option value="">All</option></select></label>
          <label>Order<select id="map-order"><option value="lead">Strongest research leads</option><option value="statements">Most historical statements</option><option value="places">Most places reporting it</option><option value="human">Most human studies</option></select></label>
          <a id="map-export" class="button-link" href="/api/research/evidence-map?format=csv">Export CSV</a>
        </div>
        <p id="map-count" class="hint"></p>
        <div class="table-wrap"><table id="map-table"></table></div>
        <button id="map-more" class="quiet library-more" hidden>Show more</button>
      </section>

      <section id="view-graph" class="view graph-view" aria-label="Provenance graph" hidden>
        <div class="graph-head">
          <div>
            <h2>Provenance graph</h2>
            <p class="hint">Select a node to show its neighbourhood. Select a line to see the source behind it.</p>
          </div>
          <div class="row">
            <label for="edge-filter">Relation</label>
            <select id="edge-filter"><option value="">All relations</option></select>
            <button id="graph-reset" class="quiet">Reset focus</button>
          </div>
        </div>
        <div class="graph-layout">
          <div>
            <div class="legend" id="legend" aria-hidden="true"></div>
            <svg id="graph" role="group" aria-label="Source and evidence graph" viewBox="0 0 960 500"></svg>
            <details>
              <summary>Graph as a list</summary>
              <div id="graph-list"></div>
            </details>
          </div>
          <aside id="inspector" class="inspector" aria-live="polite">
            <h3>Provenance</h3>
            <p class="hint">Select a node or a line.</p>
          </aside>
        </div>
      </section>

      <section id="view-handoffs" class="view knowledge-view" aria-label="Evidence handoffs" hidden>
        <header class="library-heading"><p class="kicker evidence">Medicinal-plant evidence review</p><h1>Keep the question with the evidence.</h1><p class="library-
```

```
lede">Record which source statement and possible species you checked, what the studies can answer, and what the next researcher should read.</p></header>
241         <p class="hint">A reading decision is not a treatment recommendation. Imported records retain their author's wording; they are not verified current analysis.</p>
242         <div class="handoff-columns"><section aria-label="Create evidence handoff" class="handoff-card"><div id="handoff-compose"></div></section><section aria-label="Saved evidence handoffs" class="handoff-card"><h2>Saved handoffs</h2><p class="hint">Saved on this machine. Download JSON to continue on another installation. The latest 100 are listed.</p><button id="handoff-refresh" class="quiet">Refresh list</button><label>Open a handoff JSON<input id="handoff-import" type="file" accept=".json,application/json" /></label><p class="hint">Up to 20 MB. Import creates a separate record and keeps existing decisions. Check source and publisher rights before sharing.</p><p id="handoff-list-status" role="status" aria-live="polite"></p><div id="handoff-list"></div></section></div>
243         <article id="handoff-detail" class="handoff-card" aria-label="Selected evidence handoff" hidden></article>
244       </section>
245
246       <section id="view-library" class="view knowledge-view" aria-label="Knowledge library" hidden>
247         <header class="library-heading">
248           <p class="kicker historical">For medicinal-plant evidence researchers</p>
249           <h2>Which plant did the author mean?</h2>
250           <p class="library-lede">Does a modern study answer this recorded plant use? Keep possible species, preparations and study limits separate, then hand off a traceable reading decision.</p>
251           <button id="resume-reading" class="text-action" hidden>Continue your saved reading \u2192</button>
252         </header>
253         <details class="research-trails" aria-labelledby="research-trails-title">
254           <summary id="research-trails-title">Try a guided research case</summary>
255           <div id="research-trails" class="research-trail-grid"></div>
256         </details>
257         <div class="research-task-start"><div><strong>One source \u2192 possible species \u2192 study checks \u2192 saved handoff</strong><p>Start with a source below or bring your own text. Historical uses are research questions, not evidence that a treatment works.</p></div><button id="start-source-check">Check my source text</button><button id="open-evidence-handoffs" class="quiet">Continue a handoff</button></div>
258         <div class="library-toolbar">
259           <label for="library-query">What would you like to explore?</label>
260           <div class="library-search-row"><input id="library-query" type="search" placeholder="A plant, a preparation, or a question about a recorded use\u2026" maxlength="150" /><button id="knowledge-search">Search knowledge</button></div>
261           <div id="library-examples" class="library-examples" aria-label="Suggested searches"></div>
262           <div class="library-filter-row">
263             <label>Book<select id="library-collection"><option value="">Both books</option></select></label>
264             <label>Topic<select id="library-condition"><option value="">All topics</option></select></label>
265             <label>Statement<select id="library-role"><option value="THERAPEUTIC_USE_RECORDED" selected>Recorded uses</option><option value="ADVERSE_EFFECT_RECORDED">Possible harmful effects</option><option value="CONTEXT_ONLY_OR_UNCLEAR">Context or unclear</option><option value="">All statements</option></select></label>
266             <details class="library-options" id="knowledge-method-label"><summary>Search options</summary><label>Match by<select id="knowledge-method"><option value="bm25">Exact wording</option><option value="embedding">Meaning</option><option value="hybrid">Wording and meaning</option></select></label></details>
267           </div>
268           <p id="knowledge-search-status" class="hint" aria-live="polite"></p>
269         </div>
270         <div class="library-results-bar"><p id="library-count" aria-live="polite"></p><div class="row"><button id="library-saved" class="quiet" aria-pressed="false">Saved statements</button><button id="library-export-results" class="quiet">Export CSV</button><button id="library-export-notes" class="quiet">Reading notes</button></div></div>
271         <details class="more-input"><summary>Back up or restore saved reading</summary><p class="hint">Keep all saved statements and research notes across browsers. A backup includes entries outside your current filters; it is separate from analysis judgements.</p><div class="row"><button id="library-backup" class="quiet">Download reading backup</button><label>Restore reading backup <input id="library-restore" type="file" accept=".json,application/json" /></label></div><p id="library-backup-status" class="hint" aria-live="polite"></p></details>
272         <div id="library-records" class="library-grid"><p class="hint" role="status">Opening the source library\u2026</p></div>
273         <button id="library-more" class="quiet library-more" hidden>Explore more chapters</button>
274         <aside id="library-comparison" class="library-comparison" hidden aria-live="polite"></aside>
275         <p class="library-footnote">These are historical records, with automatically identified topics and statement types. A recorded use is not evidence that a treatment works.</p>
276         <details class="library-downloads"><summary>Collection scope and complete source dataset</summary><div id="library-summary" class="library-summary"></div><p id="library-exports" class="row"><a href="/api/research/corpus?format=csv" download>All statements (CSV)</a><a href="/api/research/corpus" download>Full records and provenance (JSON)</a></p></details>
277       </section>
```

```
278
279       <section id="view-chemistry" class="view evaluation" aria-label="Compound assays" hidden>
280         <h2>Compound assays and their sources</h2>
281         <p class="hint">Measured activities of isolated molecules from ChEMBL. A human molecular target is not a human clinical study. A publication co-mention does
    not establish that a historical plant preparation contains the compound.</p>
282         <label for="chemical-compound">Compound<select id="chemical-compound"></select></label>
283         <div id="chemical-records"></div>
284       </section>
285
286       <section id="view-evaluation" class="view evaluation" aria-label="Evaluation" hidden>
287         <h2>Evaluation, with its limits</h2>
288         <p class="hint">Every row states its label tier. Source-asserted references, database-derived silver labels, weak labels and synthetic cases are not expert
    gold, and none of these results establish clinical correctness.</p>
289         <div id="metrics"></div>
290         <details class="limits" open>
291           <summary>Scope and limitations of this analysis</summary>
292           <ul id="limitations"></ul>
293           <p>Research use only. No diagnosis, prescribing or treatment recommendations. Work done before the official event is disclosed as pre-existing.</p>
294         </details>
295       </section>
296     </main><footer class="care-footer"><a href="/research#library">Historical source research workspace</a><span>Separate tools for source and methods
    research</span></footer>
297
298     <footer>
299       <span>FoliaTrace \xb7 prepare a clearer pharmacist conversation</span>
300       <span id="run-info">No analysis yet</span>
301     </footer>
302   </body>
303 </html>
```

frontend/publication-text.js
SHA-256: 4d91f8cb937e3c64d00c19f63cf8f383ffdc1ec9cb04f3e52ce30b4c243a4833

```
// Read publication formatting as inert text. Never insert publisher markup into the UI.
export function publicationText(value) {
  const template = document.createElement('template');
  template.innerHTML = String(value || '');
  template.content.querySelectorAll('script,style').forEach(node => node.remove());
  return template.content.textContent || '';
}
```

frontend/question-review.js
SHA-256: 3af67addf5ecc3ff97f0370e643d5b73ebb914a001a32d3b50383a5411a0fc70

```
import {publicationText} from './publication-text.js';
import {bridgePanel} from './care-tools.js';
const screeningLabels={PRIORITY_FOR_READING:'Ready for closer review',NEEDS_FULL_TEXT:'Details to confirm',OUTSIDE_REQUESTED_SCOPE:'Outside this question'};
export function initQuestionReview({api,el,link,onAttach,onReturn=()=>{}}) {
  const $=s=>document.querySelector(s);let form,options=null,active=null,context=null,activeContext=null,serial=0,listSerial=0,previous=null;
  const state=(text,bad=false)=>{$('#question-status').textContent=text;$('#question-status').className=bad?'error':'hint';};
  function reveal(target){for(let node=target.parentElement;node;node=node.parentElement)if(node.tagName==='DETAILS')node.open=true;target.scrollIntoView({block:'start',behavior:'auto'});target.focus({preventScroll:true});}
  function readSaved(){$('#question-editor').open=false;reveal($('#question-result'));}
  let savedItems=[],savedLimit=10;
  function filterSaved(){const term=$('#question-saved-filter').value.trim().toLocaleLowerCase();let count=0;for(const row of savedItems){const match=row.text.includes(term);row.node.hidden=!match||count>=savedLimit;if(match)count++;}const empty=$('#question-saved-empty');if(empty)empty.textContent=count?Math.min(count,savedLimit)+' of '+count+' saved questions shown':term?'No saved question or plant name matches.':'No saved evidence questions yet.';const more=$('#question-saved-more');if(more)more.hidden=count<=savedLimit;}
  $('#question-saved-filter').oninput=()=>{savedLimit=10;filterSaved();};
  $('#question-open-saved').onclick=()=>reveal($('#question-saved-filter'));
  $('#question-return').onclick=onReturn;
  function cancelRun(){serial++;const run=form?.querySelector('button[type="submit"]');if(run){run.disabled=false;run.textContent='Run and save this evidence question';form.removeAttribute('aria-busy');}return serial;}
  function field(parent,label,name,type='input',value=''){const wrap=el('label',label),input=el(type);input.name=name;input.value=value;input.maxLength=type==='textarea'?10000:800;if(type==='textarea')input.rows=3;wrap.append(input);parent.append(wrap);return input;}
  function choose(parent,label,name,values,selected){const input=field(parent,label,name,'select');for(const [v,text] of values){const option=el('option',text);option.value=v;input.append(option);}input.value=selected;return input;}
  function value(){const v=Object.fromEntries(new FormData(form));return {question:v.question,plant_name:v.plant_name,condition_id:v.retrieval_scope==='topic'?v.condition_id||null:null,retrieval_scope:v.retrieval_scope,source_mode:v.source_mode,reference_pmids:v.source_mode==='local'?[]:v.reference_pmids.split(/[,;\s]+/).filter(Boolean),population:v.population,preparation:v.preparation,comparator:v.comparator,intervention:v.intervention,outcome_terms:v.outcome_terms.split(';').map(v=>v.trim()).filter(Boolean),source_text:v.source_text,source_url:v.source_url,ranking:v.ranking};}
  function build(initial={}){
    serial++;activeContext=null;$('#question-result').hidden=true;
    $('#question-editor').open=true;
    const box=$('#question-compose');box.replaceChildren();form=el('form');const ownForm=form;
    const heading=el('div',undefined,'care-form-heading');heading.append(el('h2','1 \xb7 Define the question'),el('p','Required: a question, an exact plant name and at least one outcome phrase.','hint'));form.append(heading);
    field(form,'The health research question (required)','question','textarea',initial.question||'').required=true;
    const name=field(form,'Exact plant name to investigate (required)','plant_name','input',initial.plant_name||'');name.required=true;name.maxLength=120;
    const starters=el('div',undefined,'question-starters');starters.append(el('p','Choose an editable research starting point','hint'));
    for(const [label,terms] of [['Digestive symptoms',['abdominal pain','irritable bowel syndrome']],['Sleep',['sleep quality','insomnia']],['Nausea',['nausea','vomiting']],['Joint symptoms',['joint pain','osteoarthritis']]]){
      const b=el('button',label,'quiet');b.type='button';b.onclick=()=>{form.elements.question.value='Which human studies of '+(name.value||'this plant')+' report '+terms[0]+'?';form.elements.outcome_terms.value=terms.join('; ');form.dispatchEvent(new Event('input',{bubbles:true}));form.elements.outcome_terms.focus();};starters.append(b);
    }form.append(starters);
    const advanced=el('details',undefined,'care-advanced');advanced.open=initial.retrieval_scope==='topic'||initial.ranking==='bm25';advanced.append(el('summary','Search scope and ranking options'));
    const scope=choose(advanced,'Retrieval scope','retrieval_scope',[['question','Plant and this question \xb7 all local topics'],['topic','Also narrow by a broad topic']],initial.retrieval_scope||'question');
    const topic=choose(advanced,'Broad topic, only used when narrowing above','condition_id',[['','No broad topic selected'],...options.topics.map(v=>[v.id,v.label])],initial.condition_id||'');const syncTopic=()=>{topic.closest('label').hidden=scope.value!=='topic';topic.required=scope.value==='topic';};scope.onchange=syncTopic;syncTopic();
    const criteria=el('fieldset',undefined,'question-criteria care-field-grid');criteria.append(el('legend','2 \xb7 Study criteria'));
    choose(criteria,'Population','population',[['human','Human studies'],['any','Any research population']],initial.population||'human');
    field(criteria,'Requested preparation, if specified','preparation','input',initial.preparation||'').maxLength=120;
    choose(criteria,'Comparator needed','comparator',[['controlled','A comparator is required'],['placebo','A placebo comparison is required'],['any','No comparator requirement']],initial.comparator||'controlled');
    choose(criteria,'Intervention scope','intervention',[['single_plant','Evidence about this plant alone'],['any','Mixtures and single-plant
```

```
studies']],initial.intervention||'single_plant');
37    const outcome=field(criteria,'Exact outcome phrases (required); separate alternatives with semicolons','outcome_terms','input',(initial.outcome_terms||[]).join('; '));outcome.required=true;outcome.placeholder='e.g. abdominal pain; IBS symptom severity';outcome.closest('label').className='care-full-width';
38    criteria.append(el('p','Literal wording is a reading check. A shared phrase still needs appraisal; missing wording may have a synonym or appear in full text.','hint'));form.append(criteria);
39    form.append(el('h3','3 \xb7 Choose sources','care-section-title'));
40    const sources=choose(form,'Where to find studies','source_mode',[['local','Current local snapshot - no network'],['public','Local plus Europe PMC (PubMed-indexed records; live or cached)']],initial.source_mode==='pubmed'?'public':initial.source_mode||'local');
41    const publicPanel=el('div',undefined,'question-public-search');publicPanel.hidden=sources.value==='local';publicPanel.append(el('p','Europe PMC receives the curated plant names and exact outcome phrases above. Use public research terms only. The free-text question, product label, visit, medicine list and allergies are not sent. Up to 20 records per candidate plus ten optional reference IDs; not an exhaustive search.','hint'));
42    field(publicPanel,'Known PubMed IDs, optional; separate with commas','reference_pmids','input',(initial.reference_pmids||[]).join(', '));
43    const preview=el('button','Preview public search queries','quiet');preview.type='button';const plan=el('div');publicPanel.append(preview,plan);form.append(publicPanel);
44    sources.addEventListener('change',()=>{publicPanel.hidden=sources.value==='local';});
45    preview.onclick=async()=>{const at=serial,ownForm=form;preview.disabled=true;try{const result=await api('/api/research/public-search-plan',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(value())});if(at!==serial||form!==ownForm)return;plan.replaceChildren(el('p',result.offline?'Offline: only previously cached public requests can be used.':'These public search queries will be used; the search has not run yet.','hint'));for(const query of result.queries)plan.append(el('strong',query.name),el('pre',query.term));if(!result.queries.length)plan.append(el('p','No candidate identity established. Clarify the scientific name before public lookup.'));}catch(error){if(at===serial&&form===ownForm)plan.replaceChildren(el('p','Could not prepare search: '+error.message,'error'));}finally{preview.disabled=false;}};
46    const source=el('details',undefined,'care-advanced');source.append(el('summary',initial.source_text?'Product label or source claim \xb7 original wording retained':'Keep the product label or source claim'));
47    field(source,'Exact source wording','source_text','textarea',initial.source_text||'');field(source,'Source URL, if available','source_url','input',initial.source_url||'').maxLength=1000;form.append(source);
48    choose(advanced,'Reading priority','ranking',[['auto','Use available local AI for this question'],['bm25','Rank by question wording']],initial.ranking||'auto');form.append(advanced);
49    const run=el('button','Run and save this evidence question');run.type='submit';const footer=el('div',undefined,'care-form-actions');footer.append(run,el('p','Creates a new saved review. Reading priority is not a clinical verdict.','hint'));form.append(footer);box.append(form);
50    const remember=()=>{serial++;run.disabled=false;run.textContent='Run and save this evidence question';ownForm.removeAttribute('aria-busy');plan.replaceChildren();state(active?'Question changed. The result below keeps the earlier question; run this version to update screening.':'Question draft saved in this browser.');try{localStorage.setItem('foliatrace.question.draft.v1',JSON.stringify(value()));}catch{state('Draft storage unavailable. Keep this form open until saved to the local service.',true);}};
51    form.addEventListener('input',remember);form.addEventListener('change',remember);
52    form.addEventListener('invalid',event=>reveal(event.target),true);
53    form.addEventListener('submit',async event=>{event.preventDefault();const current=++serial,requestContext=context?{...context}:null;run.disabled=true;run.textContent='Searching and saving\u2026';ownForm.setAttribute('aria-busy','true');state('Searching and screening each candidate study against your question\u2026');try{const record=await api('/api/research/questions',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(value())});if(current!==serial)return;previous=active;active=record;activeContext=requestContext;render(record);readSaved();const external=record.packet.coverage.public_search;state(external&&external.status!=='BOUNDED_SEARCH_COMPLETED'?'Saved local evidence and any received studies. Public literature lookup was incomplete; check source coverage before drawing a conclusion.':'Saved the first result. Your reading decisions annotate it without changing the automated result.');await refresh();}catch(error){if(current===serial)state('Not saved: '+error.message+'. The draft and previous results remain.',true);}finally{if(current===serial){run.disabled=false;run.textContent='Run and save this evidence question';}ownForm.removeAttribute('aria-busy');}});
54   }
55   function studyCard(record,branch,row){
56    const card=el('details',undefined,'question-study'),paper=row.paper;card.append(el('summary',publicationText(paper.title)),link('Read original study \u2197',paper.url),el('p',branch.name+' \xb7 '+screeningLabels[row.screening.state]||row.screening.state,'pill warn'));
57    const latest=record.decision_history.filter(d=>d.record_id===row.id).at(-1);if(latest)card.append(el('p','Reading decision: '+latest.decision.replaceAll('_',' ')+' \u2014 '+latest.reason));
58    if(activeContext&&active?.id===record.id)card.append(bridgePanel({context:{...activeContext},record,row,api,el,link}));
59    const labels={PASS:'Text signal found',PARTIAL:'Needs reading',UNKNOWN:'Not established',FAIL:'Scope conflict'};
60    for(const check of row.screening.checks){const box=el('div',undefined,'question-check');box.append(el('strong',check.check.replaceAll('_',' ')+' \xb7 '+(labels[check.status]||check.status)),el('p',check.detail));for(const span of check.spans?.length?check.spans:check.span?[check.span]:[]){box.append(el('blockquote',span.text),el('p',(span.source_kind||'PUBLICATION_ABSTRACT').replaceAll('_',' ').toLowerCase()+(Number.isInteger(span.start)?' \xb7 '+span.start+'\u2013'+span.end:''),'hint'));if(span.url)box.append(link('Check quoted source
```

```
\u2197',span.url));}card.append(box);}
61    for(const entry of
row.study.preparation_attribution?.forms||[]){card.append(el('blockquote',entry.context?.text||entry.text));if(entry.url)card.append(link('Check preparation source
\u2197',entry.url));}
62    const abstract=el('details');abstract.append(el('summary',paper.abstract?'Read available abstract':'Abstract unavailable
locally'),el('p',paper.abstract?publicationText(paper.abstract):'Citation metadata only. Open the original study text.'));card.append(abstract,el('p','Rights:
'+paper.licence,'hint'));
63    if(record.origin==='LOCAL_QUESTION_REVIEW'){
64      const review=el('form');review.append(el('h4','Your reading decision'));choose(review,'Decision','decision',[['NEEDS_FULL_TEXT','Need more information or full
text'],['KEEP_FOR_READING','Keep for professional reading'],['EXCLUDE_FROM_QUESTION','Exclude from this question']],latest?.decision||'NEEDS_FULL_TEXT');
65      const reason=field(review,'Reason and uncertainty','reason','textarea',latest?.reason||'');reason.required=true;reason.minLength=5;reason.maxLength=2000;const
who=field(review,'Recorded by','reviewer','input',latest?.reviewer||'Local researcher');who.maxLength=80;
66      const save=el('button','Save reading decision','quiet');save.type='submit';const
status=el('p','','hint');status.setAttribute('role','status');review.append(save,status);
67      review.onsubmit=async event=>{event.preventDefault();save.disabled=true;try{const result=await
api('/api/research/questions/'+record.id+'/decisions',{method:'PUT',headers:{'Content-Type':'application/json'},body:JSON.stringify({record_id:row.id,decision:review
.elements.decision.value,reason:reason.value,reviewer:who.value})});if(active?.id===record.id)active=result;record.decision_history=result.decision_history;updatePro
gress(record);status.textContent='Saved. The first automated result is unchanged; decision history travels in exports.';}catch(error){status.textContent='Not saved:
'+error.message+'. Your text remains.';}finally{save.disabled=false;}};card.append(review);
68    }
69    return card;
70  }
71  function updateProgress(record){
72    const box=document.querySelector('[data-reading-progress="'+record.id+'"]');if(!box)return;
73    const latest=new Map(record.decision_history.map(d=>[d.record_id,d]));const counts={KEEP_FOR_READING:0,EXCLUDE_FROM_QUESTION:0,NEEDS_FULL_TEXT:0};for(const d of
latest.values())counts[d.decision]=(counts[d.decision]||0)+1;
74    box.replaceChildren(el('strong',latest.size+' human reading decision'+(latest.size===1?'':'s')+' recorded'),el('p',counts.KEEP_FOR_READING+' kept for reading
\xb7 '+counts.EXCLUDE_FROM_QUESTION+' excluded from this question \xb7 '+counts.NEEDS_FULL_TEXT+' need more information.','hint'),el('p','Decisions and reasons can
travel back to the saved product. A recorded decision is not a clinical approval.','hint'));
75  }
76  function renderOverview(record,box){
77    const p=record.packet,rows=p.branches.flatMap(b=>b.records.map(row=>({branch:b,row})));
78    // Older saved reviews keep their immutable packet; summarize their stored checks only.
79    const unique=new
Set(rows.map(({row})=>row.paper.pmid?'pmid:'+row.paper.pmid:row.paper.doi?'doi:'+row.paper.doi.toLowerCase().replace('https://doi.org/',''):row.paper.id));
80    const leads=new Map();for(const {branch,row} of
rows){if(row.screening.state==='OUTSIDE_REQUESTED_SCOPE'||!row.screening.checks.some(c=>c.check==='requested_outcome'&&c.spans?.length))continue;const key=row.paper.
pmid||row.paper.doi||row.paper.id;if(!leads.has(key))leads.set(key,{paper_id:row.paper.id,record_id:row.id,taxon_id:branch.taxon_id,title:publicationText(row.paper.t
itle),url:row.paper.url,candidate:branch.name,human_trial_signal:row.study.types.includes('CLINICAL_TRIAL')&&row.screening.checks.some(c=>c.check==='population'&&c.s
tatus==='PASS'),needs_confirmation:row.screening.checks.filter(c=>['UNKNOWN','PARTIAL'].includes(c.status)).map(c=>c.check)});}
81    const overview=p.overview||{unique_publications:unique.size,candidate_records:rows.length,endpoint_publications:leads.size,human_trial_signals:[...leads.values()
].filter(r=>r.human_trial_signal).length,top_studies:[...leads.values()].sort((a,b)=>Number(b.human_trial_signal)-
Number(a.human_trial_signal)).slice(0,3),selected_readings:[]};
82    const summary=el('section',undefined,'question-overview');summary.setAttribute('aria-label','What this search found');
83    if(overview.findings_summary)summary.append(el('h3','Selected trials report different primary findings'),el('p',overview.findings_summary));
84    const metrics=el('dl',undefined,'evidence-metrics');for(const [number,label] of [[overview.endpoint_publications,'With outcome
wording'],[overview.human_trial_signals,'Human-trial signals'],[overview.unique_publications,'Unique publications']]){const
tile=el('div');tile.append(el('dt',label),el('dd',String(number)));metrics.append(tile);}summary.append(metrics,el('p','Checked separately across
'+p.branches.length+' possible plant identities. Signals are not final inclusion decisions.','hint'));
85    summary.append(el('p','These are leads for professional review. Confirm species, preparation and study methods before applying findings to the
product.','hint'));
86    const openStudy=(parent,entry)=>{const
branch=p.branches.find(b=>b.taxon_id===entry.taxon_id),row=branch?.records.find(r=>r.id===entry.record_id);if(row)parent.append(studyCard(record,branch,row));};
87    if(overview.selected_readings?.length){for(const reading of overview.selected_readings){const selected=el('section',undefined,'question-selected-
readings');selected.append(el('h3','Selected trial findings'),el('p',reading.takeaway));const selection=el('details',undefined,'reading-
selection');selection.append(el('summary','Why these trials?'),el('p',reading.selection,'hint'));selected.append(selection);
88      for(const note of reading.studies){const card=el('article',undefined,'question-
finding');card.append(el('h4',note.label),link(publicationText(note.publication.title)+' \u2197',note.publication.url),el('p',note.intervention+' Follow-up:
```

```
'+note.follow_up),el('strong','Primary outcome'),el('p',note.primary_outcome),el('strong','Primary finding'),el('p',note.primary_result));const more=el('details');more.append(el('summary','Population, other findings and source checks'),el('p',note.population),el('p',note.design+' Comparator: '+note.comparator),el('p',note.other_results),el('p',note.reading_question));openStudy(more,note);if(activeContext){const branch=p.branches.find(b=>b.taxon_id===note.taxon_id),row=branch?.records.find(r=>r.id===note.record_id);if(row)card.append(bridgePanel({context:{...activeContext},record,row,api,el,link}));}card.append(more);selected.append(card);}
      const provenance=el('details');provenance.append(el('summary','How these findings were prepared'),el('p',reading.annotation.method),el('p','Notes dated '+reading.annotation.date+'. '+reading.annotation.rights_scope));selected.append(provenance);summary.append(selected);}
    }else if(overview.top_studies.length){summary.append(el('h3','Start with these studies'));for(const entry of overview.top_studies){const card=el('article',undefined,'question-finding');card.append(link(publicationText(entry.title)+' \u2197',entry.url),el('p',entry.candidate+' \xb7 '+(entry.human_trial_signal?'Human-trial signals':'Related publication'),'hint'),el('p','Confirm: '+entry.needs_confirmation.map(v=>v.replaceAll('_',' ')).join(', ')));if(activeContext){const branch=p.branches.find(b=>b.taxon_id===entry.taxon_id),row=branch?.records.find(r=>r.id===entry.record_id);if(row)card.append(bridgePanel({context:{...activeContext},record,row,api,el,link}));}openStudy(card,entry);summary.append(card);}}
    else summary.append(el('p','Try a specific botanical name, broader outcome wording, or the optional public literature search. Missing local text does not establish absence of evidence.'));
    if(overview.candidate_records!==overview.unique_publications)summary.append(el('p','The detailed review keeps '+overview.candidate_records+' plant/study links. A publication appearing under several identities is counted once above.','hint'));
    box.append(summary);
  }
  function render(record,attachContext=activeContext){
    const p=record.packet,box=$('#question-result');box.hidden=false;box.replaceChildren(el('h2',p.protocol.question),el('p',record.origin==='LOCAL_QUESTION_REVIEW'?'Saved professional reading question':'Imported review \xb7 source and claims unverified','pill warn'));
    if(record.origin==='LOCAL_QUESTION_REVIEW'&&options?.code_fingerprint&&p.code_fingerprint!==options.code_fingerprint)box.append(el('p','This first result was saved by an earlier software version. Revise and run as a new question to apply current screening; the original result and decisions are kept.','pill warn'));
    const progress=el('section',undefined,'reading-progress');progress.dataset.readingProgress=record.id;progress.setAttribute('aria-live','polite');box.append(progress);updateProgress(record);
    const links=el('div',undefined,'care-output-actions');const downloads=el('details',undefined,'care-downloads');downloads.append(el('summary','Download this review'));for(const [format,label] of [['markdown','Reading report'],['csv','Study screening table'],['json','Portable review and history']]){const a=el('a',label);a.href='/api/research/questions/'+record.id+(format==='json'?'':'?format='+format);a.download='foliatrace-question-'+record.id+'.'+({markdown:'md',csv:'csv',json:'json'}[format]);downloads.append(a);}links.append(downloads);const edit=el('button','Revise as a new question','quiet');edit.type='button';edit.onclick=()=>{build(p.protocol);reveal(form.querySelector('[name="question"]'));state('The saved result is kept. Rerun to create the revised question.');};links.append(edit);const resume=el('button','Return to my question draft','quiet');resume.type='button';resume.onclick=()=>reveal(form.querySelector('[name="question"]'));links.append(resume);box.append(links);
    if(attachContext&&record.origin==='LOCAL_QUESTION_REVIEW'){const attach=el('button','Attach this reading to the saved product record');attach.type='button';let attached=false;attach.onclick=async()=>{attach.disabled=true;try{await onAttach(attachContext,record.id);attached=true;attach.textContent='Reading attached';$('#question-return').textContent='Open the updated preparation';$('#question-context-label').textContent='Reading attached to a new preparation. Its saved sources and decisions are ready to hand off.';state('Attached to a new consultation snapshot. Return to Consultation prep to read or export it.');}catch(error){state(error.message,true);}finally{attach.disabled=attached;}};box.append(attach,el('p','Save reading decisions before attaching. The preparation keeps the review version attached at that time.','hint'));}
    renderOverview(record,box);
    const external=p.coverage.public_search;
    if(external){const log=el('details',undefined,'question-public-log');log.open=external.status!=='BOUNDED_SEARCH_COMPLETED';log.append(el('summary',(external.provider||'Public literature')+' coverage \xb7 '+external.status.replaceAll('_',' ').toLowerCase()),el('p',external.scope),el('p',external.received_records+' source records received; '+(p.coverage.new_publication_ids?.length||0)+' were not in the local snapshot.','hint'));
      for(const query of external.queries){log.append(el('strong',query.name),el('pre',query.term));for(const search of query.searches)log.append(link('Check public query \u2197','https://europepmc.org/search?query='+encodeURIComponent(query.term)),el('p',search.order+' \xb7 source acquired '+search.source.retrieved_at+' \xb7 '+search.returned_ids.length+' identifiers','hint'));}
      for(const error of external.errors)log.append(el('p',error.stage+': '+error.detail,'error'));log.append(el('p','Retractions flagged by the source are excluded. A missing result or missing abstract is not evidence of no effect.','hint'));box.append(log);
    }
    const protocol=el('details');protocol.append(el('summary','Exact question and criteria used'),el('pre',JSON.stringify(p.protocol,null,2)),el('p','Actual retrieval query: '+p.query_used,'hint'));box.append(protocol);
    if(previous&&previous.id!==record.id){const diff=el('details');diff.open=true;diff.append(el('summary','What changed from the previous question?'));const oldRows=new Map(previous.packet.branches.flatMap(b=>b.records).map(r=>[r.id,r]));let count=0;for(const b of p.branches)for(const r of b.records){const old=oldRows.get(r.id);if(old&&JSON.stringify(old.screening)!==JSON.stringify(r.screening)){count++;diff.append(el('p',b.name+' \xb7 '+publicationText(r.paper.title)+': '+old.screening.state.replaceAll('_',' ')+' \u2192 '+r.screening.state.replaceAll('_',' ')));}}diff.append(el('p',count+' shared candidate/study records changed question-specific checks. Original study results were not rewritten.','hint'));box.append(diff);}
```

```
    if(!p.branches.length)box.append(el('p','No candidate established in the local snapshot. Clarify the original plant wording; this is not evidence of no benefit.','empty'));
    for(const branch of p.branches){
      const section=el('section',undefined,'question-branch');section.append(el('h3',branch.name+' \xb7 possible label identity'),el('p',Object.entries(branch.counts).map(([s,n])=>n+' '+(screeningLabels[s]||s).toLowerCase()).join(' \xb7 '),'hint'));
      const focus=choose(section,'Show studies','reading_focus',[['all','All matched candidate records'],['endpoint','Exact endpoint wording found'],['unread','No human reading decision yet'],['reviewed','Has a recorded human reading decision']], 'all');
      const update=el('button','Refresh reading focus','quiet');update.type='button';const shown=el('p','','hint'),queues=el('div'),cache=new Map();section.append(update,shown,queues);
      function draw(){
        queues.replaceChildren();const read=new Set(record.decision_history.map(d=>d.record_id));const selected=branch.records.filter(r=>focus.value==='all'||focus.value==='endpoint'&&r.screening.checks.some(c=>c.check==='requested_outcome'&&c.spans?.length)||focus.value==='unread'&&!read.has(r.id)||focus.value==='reviewed'&&read.has(r.id));
        shown.textContent=selected.length+' of '+branch.records.length+' candidate/study records shown. Filtering does not exclude records from the saved review or export.';
        if(!selected.length)queues.append(el('p',branch.records.length?'No records match this view. Missing exact wording may have a synonym or appear in full text; change the reading focus.':'No matching study in this bounded search; no efficacy conclusion follows.','hint'));
        for(const state of ['PRIORITY_FOR_READING','NEEDS_FULL_TEXT','OUTSIDE_REQUESTED_SCOPE']){
          const rows=selected.filter(r=>r.screening.state===state);if(!rows.length)continue;const queue=el('details',undefined,'question-queue');queue.open=state==='PRIORITY_FOR_READING';queue.append(el('summary',rows.length+' \xb7 '+screeningLabels[state]));let n=0;const items=el('div'),more=el('button','Show more studies','quiet');more.type='button';const append=()=>{for(const row of rows.slice(n,n+10)){if(!cache.has(row.id))cache.set(row.id,studyCard(record,branch,row));items.append(cache.get(row.id));}n+=10;more.hidden=n>=rows.length;};more.onclick=append;append();queue.append(items,more);queues.append(queue);
        }
      }
      focus.onchange=draw;update.onclick=draw;draw();box.append(section);
    }
    const coverage=el('details');coverage.append(el('summary','Coverage, AI and limits'),el('p',p.coverage.papers_in_snapshot+' local snapshot records; '+(p.coverage.papers_searched||p.coverage.papers_in_snapshot)+' total records searched; '+p.coverage.abstracts_available+' abstracts available in this run.'),el('p',p.coverage.model==='LOCAL_CROSS_ENCODER'?'A local cross-encoder ranked studies for this question. Relevance is not a clinical probability.':'This run used wording ranking; no runtime ranking model was used.'),el('p',p.coverage.retrieval_scope==='question'?'The question searches candidate names across local topics. Missing identity or endpoint wording requires reading; it does not establish irrelevance.':'This run narrows by the selected broad topic.'),el('p',external?'Bounded Europe PMC lookup and local records. All matched records, including scope conflicts, remain in exports.':'Current local snapshot only. All matching records, including scope conflicts, remain in exports.'));for(const limit of p.limitations)coverage.append(el('p',limit,'hint'));box.append(coverage);
  }
  async function refresh(){const listing=++listSerial;try{const result=await api('/api/research/questions');if(listing!==listSerial)return;const box=$('#question-saved');box.replaceChildren();savedItems=[];for(const item of result.reviews){const row=el('div',undefined,'care-saved-item'),button=el('button',item.question,'quiet handoff-list-item');button.type='button';button.onclick=async()=>{const selected=cancelRun();try{const record=await api('/api/research/questions/'+item.id);if(selected!==serial)return;context=null;activeContext=null;previous=null;active=record;$('#question-product-context').hidden=true;render(active);readSaved();state('Opened saved first result and decision history.');}catch(error){if(selected===serial)state(error.message,true);}};const date=new Date(item.saved_at);row.append(button,el('p',item.plant_name+' \xb7 '+(Number.isNaN(date.getTime())?item.saved_at:date.toLocaleString()),'hint'));box.append(row);savedItems.push({node:row,text:(item.question+' '+item.plant_name).toLocaleLowerCase()});}const empty=el('p','','hint');empty.id='question-saved-empty';empty.setAttribute('role','status');box.append(empty);const more=el('button','Show 10 more saved records','quiet');more.type='button';more.id='question-saved-more';more.onclick=()=>{const term=$('#question-saved-filter').value.trim().toLocaleLowerCase(),first=savedItems.filter(r=>r.text.includes(term))[savedLimit];savedLimit+=10;filterSaved();first?.node.querySelector('button').focus();};box.append(more);filterSaved();}catch(error){if(listing===listSerial)state(error.message,true);}}
  $('#question-refresh').onclick=refresh;
  $('#question-import').onchange=async event=>{const file=event.target.files[0];if(!file)return;if(file.size>20_000_000){state('Review exceeds 20 MB; existing records remain.',true);return;}const selected=cancelRun();const body=new FormData();body.append('file',file);try{const record=await api('/api/research/questions/import',{method:'POST',body});if(selected!==serial){await refresh();return;}context=null;activeContext=null;previous=null;active=record;$('#question-product-context').hidden=true;render(active);readSaved();state('Imported as an unverified read-only review. Its original history is preserved.');await refresh();}catch(error){if(selected===serial)state('Import failed: '+error.message+'. Existing records remain.',true);}finally{event.target.value='';}};
  return {async open(initial=null){if(initial)cancelRun();const opening=serial;options ||= await api('/api/research/question-options');if(opening!==serial)return;if(initial){$('#question-product-context').hidden=!(initial.context||initial.savedRecord);$('#question-context-label').textContent=initial.savedRecord?'Reading opened from a saved preparation. Its original sources and history are preserved.':'Reading for a saved product. Record your decisions here, then attach the review to its preparation.';$('#question-return').textContent='Return to the product preparation';}if(initial?.savedRecord){context=null;activeContext=null;previous=null;active=initial.savedRecord;if(!form)build(active.packet.protocol);render(active);readSaved();}else
```

```
    if(initial){context=initial.context||null;active=null;activeContext=null;build(initial);reveal(form.querySelector('[name="question"]'));state(context?'Tied to a
    saved product. Define the health endpoint and criteria before running.':'Define the question before searching.');}else if(!form){let
    draft=null;try{draft=JSON.parse(localStorage.getItem('foliatrace.question.draft.v1')||'null');}catch{}build(draft||{});}return refresh();}};
}
```

frontend/style.css
SHA-256: d4fd7f99e907c8463d07acc0773c470a01b5f17beff2dd184800c6992b6512a1

```
.consultation-intro { padding:1.3rem; border-left:4px solid var(--identity); background:var(--surface-2); border-radius:4px; }
.consultation-intro p { max-width:75ch; }
.consultation-product,.question-branch { margin-block:1.3rem; padding:1.1rem; border:1px solid var(--line); border-radius:8px; min-width:0; }
.handoff-card fieldset.consultation-product,.handoff-card fieldset.question-criteria { max-height:none; overflow:visible; }
.research-tools { align-self:center; padding:.45rem .6rem; }.research-tools > summary { cursor:pointer; color:var(--muted); }
.research-tools-menu { display:flex; flex-wrap:wrap; gap:.4rem; padding-top:.6rem; }
.consultation-fields { display:grid; grid-template-columns:repeat(2,minmax(0,1fr)); gap:0 1rem; }
.consultation-summary { display:flex; flex-wrap:wrap; gap:1rem; padding-block:1rem; border-block:1px solid var(--line); }
.consultation-product-reading { margin-block:1.4rem; padding-block:1rem; border-bottom:1px solid var(--line); }
.consultation-facts { display:grid; grid-template-columns:minmax(6rem,.6fr) minmax(0,1.4fr); gap:.5rem 1rem; }
.consultation-facts dt { color:var(--muted); }.consultation-facts dd { margin:0; overflow-wrap:anywhere; }
.question-study { padding:1rem; margin-block:.7rem; background:var(--surface); border:1px solid var(--line); border-radius:6px; }
.question-study > summary { font-weight:600; line-height:1.5; }.question-check { padding-block:.6rem; border-bottom:1px solid var(--line); }
.handoff-card pre { white-space:pre-wrap; overflow-wrap:anywhere; font:inherit; }

.handoff-columns { display:grid; grid-template-columns: minmax(0, 1.4fr) minmax(0, 1fr); gap:1.25rem; }
.handoff-card, .research-task-start { padding:1.4rem; margin-block:1.25rem; border:1px solid var(--line); background:var(--surface); border-radius:12px; min-width:0;
 }
.research-task-start { display:flex; flex-wrap:wrap; align-items:center; gap:1rem; }
.research-task-start > div { flex:1 1 22rem; }
.research-task-start p { margin-bottom:0; }
.handoff-card label { display:block; margin-block:1rem; }
.handoff-card input:not([type="checkbox"]), .handoff-card textarea, .handoff-card select { display:block; width:100%; box-sizing:border-box; margin-top:.4rem; }
.handoff-card fieldset { max-height:17rem; overflow:auto; border:1px solid var(--line); margin-block:1rem; }
.handoff-card fieldset label { margin-block:.5rem; }
.handoff-card blockquote { overflow-wrap:anywhere; white-space:pre-wrap; }
.handoff-card .hint { overflow-wrap:anywhere; }
.handoff-list-item { display:block; text-align:left; width:100%; }

:root {
  color-scheme: light;
  --bg: #f6f4ee;
  --surface: #fffdf8;
  --surface-2: #efece3;
  --ink: #1d2b27;
  --muted: #4c5c55;
  --line: #d6d0c1;
  --line-strong: #a9a291;
  --historical: #7a4b10;
  --identity: #1f5c46;
  --evidence: #22507a;
  --warn: #855600;
  --fail: #9b2c25;
  --pass: #1f5c46;
  --mark: #e2ebcf;
  --mark-active: #f3d68c;
  --focus: #b4530f;
  --button: #1f5c46;
  --button-ink: #ffffff;
  --shadow: 0 1px 2px rgb(29 43 39 / 0.06);
  --serif: "Iowan Old Style", "Palatino Linotype", Palatino, Georgia, serif;
  --sans: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  --mono: ui-monospace, "Cascadia Mono", Consolas, monospace;
}
@media (prefers-color-scheme: dark) {
```

```
  :root:not([data-theme="light"]) {
    color-scheme: dark;
    --bg: #111614;
    --surface: #181f1c;
    --surface-2: #212a26;
    --ink: #e6ebe7;
    --muted: #a7b3ad;
    --line: #313c37;
    --line-strong: #56645d;
    --historical: #e0b06a;
    --identity: #86cdab;
    --evidence: #93bfe8;
    --warn: #e6b85c;
    --fail: #f19388;
    --pass: #86cdab;
    --mark: #2c4536;
    --mark-active: #6b5420;
    --focus: #f0a35e;
    --button: #86cdab;
    --button-ink: #10211a;
    --shadow: none;
  }
}
:root[data-theme="dark"] {
  color-scheme: dark;
  --bg: #111614;
  --surface: #181f1c;
  --surface-2: #212a26;
  --ink: #e6ebe7;
  --muted: #a7b3ad;
  --line: #313c37;
  --line-strong: #56645d;
  --historical: #e0b06a;
  --identity: #86cdab;
  --evidence: #93bfe8;
  --warn: #e6b85c;
  --fail: #f19388;
  --pass: #86cdab;
  --mark: #2c4536;
  --mark-active: #6b5420;
  --focus: #f0a35e;
  --button: #86cdab;
  --button-ink: #10211a;
  --shadow: none;
}

* {
  box-sizing: border-box;
}

.research-next {
  margin: 0 20px 16px;
  padding: 14px 16px;
  border-left: 3px solid var(--identity);
  background: var(--surface-2);
  overflow-wrap: anywhere;
}
.research-next p { margin: 6px 0 10px; font-size: 0.9rem; }
```

```
.research-tasks { padding-left: 22px; }
.research-tasks li { margin-bottom: 12px; }
.reading-record { padding: 12px 0; border-bottom: 1px solid var(--line); overflow-wrap: anywhere; }
.reading-record summary { cursor: pointer; }
.reading-scope { border-left: 2px solid var(--line); padding-left: 10px; margin-top: 14px; }
.task-records { display: block; margin-top: 6px; }
.more-reading { margin-top: 16px; }
.outcome-tool form { display: grid; gap: 12px; margin: 14px 0; }
.outcome-tool label { display: grid; gap: 5px; }
.outcome-tool textarea { max-width: 100%; width: 100%; }
.chemistry { margin: 12px 0; }
.record-preview { display: block; padding: 6px 0 0 16px; color: var(--muted); font-size: .9rem; line-height: 1.5; font-weight: normal; }

/* A reading library, organised around chapters and the statements a researcher saves. */
.knowledge-view { max-width: 1160px; margin: 24px auto 0; }
.library-heading { padding: 12px 0 24px; }
.library-heading h2 { font: 46px/1.12 var(--serif); margin: 10px 0 14px; letter-spacing: -.025em; }
.library-lede { max-width: 680px; color: var(--muted); font-size: 17px; line-height: 1.65; margin: 0; }
.library-summary { display: flex; flex-wrap: wrap; gap: 14px 36px; margin-top: 24px; }
.library-stat { display: flex; align-items: baseline; gap: 9px; }
.library-stat strong { font: 26px var(--serif); color: var(--historical); }
.library-stat span { color: var(--muted); font-size: 13px; }
.library-toolbar { padding: 22px; border: 1px solid var(--line); border-radius: 12px; background: var(--surface); box-shadow: var(--shadow); }
.research-trails { margin: 0 0 28px; }
.research-trails:not([open]) { margin-bottom: 12px; }
.research-trails > summary { font-size: 14px; cursor: pointer; font-weight: 600; color: var(--identity); }
.research-trails[open] > summary { margin-bottom: 12px; }
.research-trail-grid { display: grid; grid-template-columns: repeat(3,minmax(0,1fr)); gap: 14px; }
.research-trail { display: flex; flex-direction: column; padding: 18px; border: 1px solid var(--line); border-radius: 10px; background: var(--surface); }
.research-trail h4 { font: 20px/1.3 var(--serif); color: var(--identity); margin: 0; }
.research-trail p { font-size: 13px; line-height: 1.7; color: var(--muted); margin: 10px 0 16px; }
.research-trail .text-action { margin-top: auto; font-size: 12px; white-space: normal; text-align: left; }
.statement-attribution { margin: 0 0 12px; }
.statement-attribution p { font-size: 12px; margin: 6px 0; }
.reading-note { display: grid; gap: 7px; font-size: 12px; color: var(--muted); margin-top: 16px; }
.reading-note textarea { width: 100%; min-width: 0; resize: vertical; font: 14px/1.6 var(--sans); }
.reading-note .hint { font-weight: normal; }
.inspect-source-name { padding: 10px 0 16px; font-size: 13px; }
.inspect-source-name summary { cursor: pointer; color: var(--identity); }
.inspect-source-name form { flex-wrap: wrap; align-items: end; }
.inspect-source-name label { display: grid; gap: 6px; }
.inspect-source-name input { max-width: 100%; }

.library-toolbar > label { display: block; font-size: 14px; font-weight: 600; margin-bottom: 8px; }
.library-search-row { display: grid; grid-template-columns: minmax(0,1fr) auto; gap: 10px; }
.library-search-row input { width: 100%; min-width: 0; font-size: 16px; padding: 12px 14px; }
.library-search-row button { padding: 12px 22px; }
.library-examples { display: flex; flex-wrap: wrap; gap: 8px; padding: 12px 0 20px; }
button.topic-chip { min-height: 28px; border-radius: 20px; padding: 4px 11px; font-size: 12px; font-weight: 500; background: var(--surface-2); color: var(--muted);
    border: 1px solid var(--line); }
button.topic-chip:hover { color: var(--ink); border-color: var(--identity); background: var(--mark); }
.library-filter-row { display: grid; grid-template-columns: repeat(3,minmax(0,1fr)) minmax(140px,.7fr); align-items: start; gap: 14px; border-top: 1px solid
    var(--line); padding-top: 16px; }
.library-filter-row label { display: grid; gap: 5px; font-size: 12px; color: var(--muted); }
.library-filter-row select { width: 100%; min-width: 0; font-size: 13px; }
.library-options { font-size: 13px; padding-top: 22px; }
.library-options summary { color: var(--muted); cursor: pointer; }
.library-options label { margin-top: 10px; }
```

```
.research-search-links { margin-top: 10px; font-size: 13px; line-height: 1.8; }
.library-results-bar { display: flex; justify-content: space-between; align-items: center; gap: 12px; margin: 24px 0 14px; }
.library-results-bar > p { margin: 0; font-size: 14px; color: var(--muted); }
.library-results-bar .row { gap: 8px; }
.library-results-bar button { padding: 7px 11px; font-size: 12px; }
#library-saved[aria-pressed="true"], .knowledge-actions button[aria-pressed="true"] { background: var(--mark); border-color: var(--identity); color: var(--ink); }
.library-grid { display: grid; grid-template-columns: repeat(2,minmax(0,1fr)); gap: 18px; }
.knowledge-card { display: flex; flex-direction: column; min-width: 0; padding: 22px; background: var(--surface); border: 1px solid var(--line); border-radius: 12px;
 box-shadow: var(--shadow); }
.knowledge-card-head { display: flex; justify-content: space-between; align-items: start; gap: 12px; }
.knowledge-card-head > div { min-width: 0; }
.knowledge-card .kicker { font-size: 10px; margin: 0 0 7px; }
.knowledge-card h3 { font: 22px/1.25 var(--serif); margin: 0; overflow-wrap: anywhere; }
.knowledge-card-head .pill { font-size: 10px; max-width: 130px; line-height: 1.4; flex-shrink: 0; white-space: normal; }
.knowledge-topics { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 14px; }
.knowledge-card > blockquote { display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 4; overflow: hidden; margin: 18px 0 12px; padding: 0; border:
0; font: 17px/1.65 var(--serif); color: var(--ink); }
.knowledge-card > blockquote.full-quote { display: block; overflow: visible; }
.knowledge-card > .text-action { align-self: start; margin: 0 0 12px; }
.knowledge-card > .hint { font-size: 12px; margin: 0 0 12px; }
.knowledge-more { font-size: 12px; color: var(--muted); margin: 0 0 16px; }
.knowledge-more summary { cursor: pointer; }
.knowledge-extra { padding: 15px 0; border-bottom: 1px solid var(--line); }
.knowledge-extra blockquote { margin: 10px 0; padding: 0; border: 0; font: 16px/1.65 var(--serif); color: var(--ink); }
.knowledge-actions { display: flex; flex-wrap: wrap; gap: 8px; padding-top: 8px; margin-top: auto; }
.knowledge-actions button { font-size: 13px; padding: 9px 13px; }
.knowledge-card-foot { display: flex; flex-wrap: wrap; justify-content: space-between; align-items: center; gap: 10px; margin-top: 18px; padding-top: 14px; border-
top: 1px solid var(--line); font-size: 12px; }
.compare-choice { display: inline-flex; align-items: center; gap: 6px; color: var(--muted); cursor: pointer; }
.compare-choice input { width: 15px; height: 15px; margin: 0; accent-color: var(--identity); }
.library-empty { grid-column: 1 / -1; padding: 48px 24px; border: 1px dashed var(--line-strong); border-radius: 12px; text-align: center; color: var(--muted); }
.library-empty h3 { color: var(--ink); font: 25px var(--serif); }
.library-more { display: block; margin: 24px auto; }
.library-comparison { position: sticky; bottom: 16px; z-index: 10; display: flex; align-items: center; gap: 12px; padding: 16px 20px; margin-top: 24px; border: 1px
solid var(--identity); border-radius: 12px; background: var(--surface); box-shadow: 0 6px 24px rgb(0 0 0 / .15); }
.library-comparison > div { flex: 1; min-width: 0; }
.library-comparison p { margin: 4px 0 0; overflow-wrap: anywhere; }
.library-comparison button { font-size: 13px; }
.library-footnote { color: var(--muted); font-size: 12px; margin-top: 30px; max-width: 780px; line-height: 1.6; }
.library-downloads { color: var(--muted); font-size: 12px; margin: 16px 0 30px; }
.library-downloads summary { cursor: pointer; }
.reported-results { margin: 14px 0; padding: 14px 16px; border-left: 3px solid var(--evidence); background: var(--surface); }
.reported-results h4 { margin: 0 0 8px; }
.reported-results blockquote { margin: 8px 0 12px; }
.trial-readings { border: 1px solid var(--line-strong); border-radius: 8px; margin: 14px 0 24px; padding: 14px 16px; background: var(--surface); }
.trial-readings > summary { cursor: pointer; color: var(--evidence); font-weight: 600; }
.trial-reading-grid { display: flex; flex-wrap: wrap; gap: 14px; margin: 18px 0; }
.trial-reading-card { flex: 1 1 300px; min-width: 0; padding: 14px; border: 1px solid var(--line); border-radius: 6px; }
.trial-reading-card h4 { margin: 0 0 8px; }
.trial-reading-card dt { font: 600 12px var(--sans); margin: 14px 0 5px; color: var(--muted); }
.trial-reading-card dd { margin: 0; font-size: 14px; line-height: 1.6; }
.trial-reading-card .primary-reading-result { padding: 10px; background: var(--surface-2); border-left: 3px solid var(--evidence); }
.trial-reading-context { margin-top: 14px; font-size: 13px; }
.trial-reading-context > summary { cursor: pointer; color: var(--evidence); }
.reading-note-provenance { margin: 14px 0; font-size: 13px; }
.source-focus { margin: 0 20px 16px; border-left: 3px solid var(--historical); padding: 14px 16px; background: var(--surface-2); }
.source-focus .kicker { margin-top: 0; }
.source-focus blockquote { border: 0; padding: 0; margin: 8px 0 12px; font: 16px/1.6 var(--serif); }
```

```
.text-action { border: 0; background: none; padding: 0; color: var(--identity); font-size: 12px; text-decoration: underline; min-height: 28px; }


[hidden] {
  display: none !important;
}
html {
  scroll-padding-top: 16px;
}
body {
  margin: 0;
  background: var(--bg);
  color: var(--ink);
  font: 15px/1.55 var(--sans);
  overflow-wrap: break-word;
}
a {
  color: var(--evidence);
}
button,
input,
select,
textarea {
  font: inherit;
  color: inherit;
}
h1,
h2,
h3 {
  font-family: var(--serif);
  font-weight: 600;
  line-height: 1.2;
  margin: 0;
}
h1 {
  font-size: 26px;
}
h2 {
  font-size: 23px;
}
h3 {
  font-size: 17px;
}
p {
  margin: 0;
}
:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
}
.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
}
```

```
.skip {
  position: absolute;
  left: 16px;
  top: -60px;
  background: var(--surface);
  padding: 10px 14px;
  z-index: 10;
}
.skip:focus {
  top: 10px;
}

/* Controls */
button {
  border: 1px solid var(--button);
  background: var(--button);
  color: var(--button-ink);
  padding: 8px 16px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
  min-height: 38px;
}
button:hover {
  filter: brightness(0.94);
}
button:disabled {
  cursor: progress;
  opacity: 0.6;
}
button.quiet,
.chip,
.view-tab {
  background: transparent;
  color: var(--ink);
  border-color: var(--line-strong);
  font-weight: 500;
}
input,
select,
textarea {
  background: var(--surface);
  border: 1px solid var(--line-strong);
  border-radius: 6px;
  padding: 7px 10px;
  max-width: 100%;
  min-height: 38px;
}
textarea {
  width: 100%;
  resize: vertical;
}
label {
  display: block;
  font-size: 13px;
  color: var(--muted);
}
summary {
```

```
  cursor: pointer;
  font-weight: 600;
}
.row {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
}
.hint {
  font-size: 13px;
  color: var(--muted);
}
.meta {
  font-size: 13px;
  color: var(--muted);
  margin-top: 4px;
}
.kicker {
  font-size: 12px;
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
}
.historical {
  color: var(--historical);
}
.taxonomy {
  color: var(--identity);
}
.modern {
  color: var(--evidence);
}

/* Top bar */
.topbar {
  border-bottom: 1px solid var(--line);
  background: var(--surface);
}
.brand-row,
.views {
  max-width: 1440px;
  margin: 0 auto;
  padding: 12px 24px 0;
  display: flex;
  align-items: center;
  gap: 16px;
}
.brand {
  font: 600 24px/1 var(--serif);
  color: var(--ink);
  text-decoration: none;
}
.tagline {
  color: var(--muted);
  font-size: 14px;
  flex: 1;
  min-width: 0;
```

```
}
.mode {
  font-size: 12px;
  font-weight: 600;
  border: 1px solid currentColor;
  border-radius: 999px;
  padding: 3px 10px;
  color: var(--identity);
  white-space: nowrap;
}
.mode.recorded {
  color: var(--warn);
}
.mode.error {
  color: var(--fail);
}
#literature-availability {
  max-width: 1440px;
  margin: 0 auto;
  padding: 8px 24px 12px;
}
.views {
  padding-top: 8px;
  gap: 4px;
}
.view-tab,
.view-link {
  border: 0;
  border-bottom: 3px solid transparent;
  border-radius: 0;
  padding: 8px 12px;
  min-height: 40px;
  font-size: 14px;
  text-decoration: none;
  color: var(--muted);
}
.view-link {
  margin-left: auto;
  display: inline-flex;
  align-items: center;
}
.view-tab[aria-current="page"] {
  color: var(--ink);
  border-bottom-color: var(--identity);
  font-weight: 600;
}

main {
  max-width: 1440px;
  margin: 0 auto;
  padding: 16px 24px 32px;
}

/* Source bar */
.source-bar {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto auto;
  gap: 12px 20px;
```

```
  align-items: end;
}
.source-bar > * {
  min-width: 0;
}
.cases {
  grid-column: 1 / -1;
}
.source-pick .row {
  flex-wrap: nowrap;
}
.source-pick .row select {
  flex: 1;
  min-width: 0;
  width: 100%;
  max-width: 560px;
}
.more-input > summary,
.search > summary {
  padding: 8px 0;
}
.cases .label {
  display: block;
  font-size: 13px;
  color: var(--muted);
  margin-bottom: 4px;
}
.case {
  max-width: 100%;
  background: var(--surface);
  color: var(--ink);
  border: 1px solid var(--line-strong);
  font-weight: 500;
  text-align: left;
  padding: 6px 12px;
}
.case small {
  display: block;
  color: var(--muted);
  font-size: 12px;
  font-weight: 400;
}
.case[aria-pressed="true"] {
  border-color: var(--identity);
  box-shadow: inset 0 0 0 1px var(--identity);
}
.more-input,
.search {
  grid-column: span 1;
  font-size: 14px;
}
.more-input[open],
.search[open] {
  grid-column: 1 / -1;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 12px;
```

```
}
.more-input textarea {
  margin: 4px 0 8px;
}
.search input {
  flex: 1;
  min-width: 180px;
}
#search-results {
  max-height: 260px;
  overflow: auto;
  margin-top: 8px;
}
.result {
  display: flex;
  gap: 8px;
  align-items: baseline;
  width: 100%;
  padding: 6px 2px;
  border-bottom: 1px solid var(--line);
  font-size: 14px;
  background: none;
  color: var(--ink);
  border-width: 0 0 1px;
  border-radius: 0;
  text-align: left;
  font-weight: 400;
  min-height: 0;
}
#status {
  min-height: 24px;
  margin: 12px 0;
  font-size: 13px;
  color: var(--muted);
}
#status.error {
  color: var(--fail);
  font-weight: 600;
}

/* Workspace */
.workspace {
  display: grid;
  grid-template-columns: minmax(300px, 5fr) minmax(360px, 7fr);
  gap: 20px;
  align-items: start;
}
.pane {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 10px;
  box-shadow: var(--shadow);
  padding: 20px;
  min-width: 0;
}
.pane-head {
  margin-bottom: 12px;
}
```

```
.source-pane {
  position: sticky;
  top: 12px;
  max-height: calc(100vh - 24px);
  display: flex;
  flex-direction: column;
}
.original {
  font: 16px/1.75 var(--serif);
  white-space: pre-wrap;
  overflow: auto;
  margin: 10px 0;
  padding: 4px 14px 4px 0;
  flex: 1;
  min-height: 160px;
}
.original button.mention {
  font: inherit;
  background: var(--mark);
  color: var(--ink);
  border: 0;
  border-bottom: 2px solid var(--identity);
  border-radius: 2px;
  padding: 0 2px;
  min-height: 0;
  font-weight: inherit;
  cursor: pointer;
}
.original button.mention.active {
  background: var(--mark-active);
}
.claims {
  border-top: 1px solid var(--line);
  padding-top: 10px;
  font-size: 14px;
}
.claims[open] {
  overflow: auto;
  max-height: 38vh;
}
.claim {
  padding: 10px 0;
  border-bottom: 1px solid var(--line);
}
.claim blockquote {
  margin: 0 0 6px;
  font: 15px/1.55 var(--serif);
}
.facts {
  display: flex;
  flex-wrap: wrap;
  gap: 4px 14px;
  font-size: 13px;
  color: var(--muted);
}
.facts b {
  color: var(--ink);
  font-weight: 600;
```

```
}

/* Name inspector */
.names {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
  margin-bottom: 14px;
}
.chip {
  border: 1px solid var(--line-strong);
  border-radius: 999px;
  padding: 4px 12px;
  min-height: 34px;
  font-size: 14px;
}
.chip .count {
  color: var(--muted);
  font-size: 12px;
  margin-left: 4px;
}
.chip[aria-pressed="true"] {
  background: var(--identity);
  border-color: var(--identity);
  color: var(--button-ink);
}
.chip[aria-pressed="true"] .count {
  color: inherit;
}
.name-head h2 {
  margin-top: 2px;
}
.status-line {
  margin-top: 6px;
  font-size: 14px;
}
.pill {
  display: inline-block;
  font-size: 12px;
  font-weight: 600;
  border: 1px solid currentColor;
  border-radius: 999px;
  padding: 1px 9px;
  margin-right: 6px;
  white-space: nowrap;
}
.pill.ok {
  color: var(--pass);
}
.pill.warn {
  color: var(--warn);
}
.pill.bad {
  color: var(--fail);
}
.pill.neutral {
  color: var(--muted);
}
```

```
.tabs {
  display: flex;
  gap: 2px;
  border-bottom: 1px solid var(--line);
  margin: 6px 0 16px;
  overflow-x: auto;
}
.tabs [role="tab"] {
  background: transparent;
  color: var(--muted);
  border: 0;
  border-bottom: 3px solid transparent;
  border-radius: 0;
  padding: 8px 12px;
  font-weight: 500;
  white-space: nowrap;
}
.tabs [role="tab"][aria-selected="true"] {
  color: var(--ink);
  border-bottom-color: var(--identity);
  font-weight: 600;
}
.tab-panel > * + * {
  margin-top: 14px;
}

/* What-if comparison */
.whatif {
  border: 1px solid var(--line);
  border-left: 4px solid var(--warn);
  border-radius: 8px;
  padding: 14px 16px;
  background: var(--surface-2);
}
.whatif.stable {
  border-left-color: var(--identity);
}
.whatif h3 {
  margin-bottom: 6px;
}
.whatif p + p {
  margin-top: 6px;
}
.table-wrap {
  overflow-x: auto;
  margin-top: 10px;
}
table {
  border-collapse: collapse;
  width: 100%;
  font-size: 13px;
}
th,
td {
  text-align: left;
  vertical-align: top;
  padding: 7px 8px;
  border-bottom: 1px solid var(--line);
```

```
}
th {
  font-weight: 600;
  color: var(--muted);
}
td.num {
  font-variant-numeric: tabular-nums;
}
.candidates {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
  gap: 12px;
}
.candidate {
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 14px;
  display: flex;
  flex-direction: column;
  gap: 6px;
}
.candidate h3 {
  font-style: italic;
}
.candidate .family {
  font-size: 13px;
  color: var(--muted);
}
.why {
  font-size: 13px;
  margin: 0;
  padding-left: 18px;
}
.candidate .actions {
  margin-top: auto;
  padding-top: 6px;
}
.openworld {
  border: 1px dashed var(--line-strong);
  border-radius: 8px;
  padding: 12px 14px;
  font-size: 14px;
}
.openworld ul {
  margin: 6px 0 0;
  padding-left: 18px;
}

/* Evidence */
.question {
  font: 16px/1.5 var(--serif);
  padding: 10px 14px;
  background: var(--surface-2);
  border-radius: 8px;
}
.filters {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
```

```
  gap: 8px;
}
.filters select,
.filters input {
  width: 100%;
  margin-top: 2px;
  font-size: 14px;
}
.blocking {
  display: flex;
  flex-wrap: wrap;
  gap: 6px 16px;
  font-size: 13px;
  color: var(--muted);
}
.conflict {
  border-left: 4px solid var(--warn);
  background: var(--surface-2);
  padding: 10px 12px;
  border-radius: 6px;
  font-size: 14px;
}
.branch-group + .branch-group {
  margin-top: 22px;
}
.branch-group > h3 {
  font-size: 17px;
  padding-bottom: 6px;
}
.branch-group .more {
  margin-top: 8px;
}
.evidence-item {
  scroll-margin-top: 116px;
  border-top: 1px solid var(--line);
  padding: 14px 0 6px;
}
.evidence-item h3 {
  font-size: 16px;
  margin: 6px 0 4px;
}
.evidence-item h3 a {
  color: var(--ink);
  text-decoration-color: var(--line-strong);
}
.checks {
  list-style: none;
  padding: 0;
  margin: 8px 0;
  display: flex;
  flex-wrap: wrap;
  gap: 4px 14px;
  font-size: 13px;
}
.checks li::before {
  font-weight: 700;
  margin-right: 4px;
}
```

```
.checks .PASS::before {
  content: "\u2713";
  color: var(--pass);
}
.checks .FAIL::before {
  content: "\u2715";
  color: var(--fail);
}
.checks .PARTIAL::before {
  content: "\u2248";
  color: var(--warn);
}
.checks .UNKNOWN::before {
  content: "?";
  color: var(--muted);
}
.evidence-item .quote-label {
  margin: 8px 0 0;
  font-size: 12px;
  color: var(--muted);
}
.evidence-item .quote-label + blockquote {
  margin-top: 2px;
}
.evidence-item blockquote {
  margin: 8px 0;
  padding-left: 12px;
  border-left: 3px solid var(--line-strong);
  font: 14px/1.6 var(--serif);
}
.evidence-item details {
  font-size: 14px;
  margin: 6px 0;
}
.evidence-item details > * + * {
  margin-top: 8px;
}
.abstract {
  white-space: pre-wrap;
  max-height: 240px;
  overflow: auto;
  font-size: 13px;
  color: var(--muted);
}
.empty {
  border: 1px dashed var(--line-strong);
  border-radius: 8px;
  padding: 16px;
  color: var(--muted);
  font-size: 14px;
}

/* Review */
.review-form {
  border-top: 1px solid var(--line);
  padding-top: 8px;
  font-size: 14px;
}
```

```
.review-form fieldset {
  border: 0;
  padding: 0;
  margin: 0;
}
.review-form legend {
  font-size: 13px;
  color: var(--muted);
  margin-bottom: 4px;
}
.review-form .choice {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  margin-right: 10px;
  color: var(--ink);
  font-size: 14px;
}
.review-form input[type="text"] {
  width: 100%;
  margin: 6px 0;
}
.review-saved {
  font-size: 13px;
}
.review-row {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  align-items: flex-start;
  border-bottom: 1px solid var(--line);
  padding: 10px 0;
  font-size: 14px;
}
.subhead {
  margin-top: 22px;
}
#export-links a {
  display: inline-block;
  border: 1px solid var(--line-strong);
  border-radius: 6px;
  padding: 7px 14px;
  text-decoration: none;
  color: var(--ink);
  font-size: 14px;
}

/* Graph */
.graph-head {
  display: flex;
  justify-content: space-between;
  align-items: flex-end;
  gap: 16px;
  flex-wrap: wrap;
  margin-bottom: 12px;
}
.graph-layout {
  display: grid;
```

```
  977   grid-template-columns: minmax(0, 3fr) minmax(260px, 1fr);
  978   gap: 16px;
  979 }
  980 .legend {
  981   display: flex;
  982   flex-wrap: wrap;
  983   gap: 4px 14px;
  984   font-size: 12px;
  985   color: var(--muted);
  986   margin-bottom: 6px;
  987 }
  988 .legend i {
  989   display: inline-block;
  990   width: 10px;
  991   height: 10px;
  992   border-radius: 2px;
  993   margin-right: 5px;
  994   vertical-align: -1px;
  995 }
  996 #graph {
  997   width: 100%;
  998   height: auto;
  999   min-height: 300px;
 1000   background: var(--surface);
 1001   border: 1px solid var(--line);
 1002   border-radius: 10px;
 1003 }
 1004 #graph line {
 1005   stroke: var(--line-strong);
 1006   stroke-width: 1.4;
 1007   cursor: pointer;
 1008 }
 1009 #graph line:hover,
 1010 #graph line:focus {
 1011   stroke: var(--focus);
 1012   stroke-width: 4;
 1013 }
 1014 #graph .node {
 1015   cursor: pointer;
 1016 }
 1017 #graph .node rect {
 1018   stroke: var(--line-strong);
 1019 }
 1020 #graph .node:hover rect,
 1021 #graph .node:focus rect {
 1022   stroke: var(--focus);
 1023   stroke-width: 3;
 1024 }
 1025 #graph text {
 1026   font: 11px var(--sans);
 1027   fill: #1d2b27;
 1028   pointer-events: none;
 1029 }
 1030 .inspector {
 1031   background: var(--surface);
 1032   border: 1px solid var(--line);
 1033   border-radius: 10px;
 1034   padding: 16px;
```

```
  max-height: 620px;
  overflow: auto;
}
.inspector pre {
  font: 12px/1.5 var(--mono);
  white-space: pre-wrap;
  overflow-wrap: anywhere;
}
#graph-list {
  max-height: 320px;
  overflow: auto;
}
#graph-list button {
  display: block;
  width: 100%;
  text-align: left;
  font-size: 13px;
  margin: 2px 0;
  background: transparent;
  color: var(--ink);
  border-color: var(--line);
  font-weight: 400;
}

/* Evaluation */
.evaluation > * + * {
  margin-top: 14px;
}
.metric-group {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 10px;
  padding: 16px;
}
.metric-group h3 {
  margin-bottom: 4px;
}
.limits {
  background: var(--surface-2);
  border-radius: 10px;
  padding: 14px 16px;
  font-size: 14px;
}
.limits ul {
  margin: 8px 0;
  padding-left: 20px;
}

footer {
  max-width: 1440px;
  margin: 0 auto;
  padding: 16px 24px 28px;
  font-size: 12px;
  color: var(--muted);
  display: flex;
  justify-content: space-between;
  gap: 12px;
  flex-wrap: wrap;
```

```
}


@media (prefers-reduced-motion: reduce) {
  * {
    transition: none !important;
  }
}

.linked-conditions{margin-top:16px;padding:12px 14px;border:1px solid var(--line);border-radius:8px;background:var(--surface-2);font-size:13px}.linked-conditions>summary{cursor:pointer;color:var(--evidence);font-weight:600}.linked-conditions .evidence-item{padding:14px 0}.linked-conditions .evidence-item:last-child{border-bottom:0}
.statement-highlight { background: var(--mark-active); color: var(--ink); box-decoration-break: clone; -webkit-box-decoration-break: clone; }
.comparison-intro { font: 19px/1.5 var(--serif); color: var(--historical); }
.comparison-actions { margin: 16px 0; flex-wrap: wrap; }
.comparison-matrix table { table-layout: fixed; width: 100%; min-width: 610px; }
.comparison-matrix caption { text-align: left; font-size: 12px; color: var(--muted); padding-bottom: 10px; }
.comparison-matrix th:first-child { width: 22%; }
.comparison-matrix thead th { vertical-align: top; background: var(--surface-2); }
.comparison-matrix thead strong { font: 18px var(--serif); }
.comparison-matrix tbody th { vertical-align: top; font-weight: 500; }
.comparison-matrix tbody th .pill { display: table; margin-top: 8px; font-size: 10px; }
.comparison-matrix td { vertical-align: top; }
.comparison-matrix summary { cursor: pointer; color: var(--identity); }
.comparison-preparations { display: block; font-size: 12px; color: var(--muted); margin-top: 5px; }
.comparison-quotation { padding: 12px 0; border-bottom: 1px solid var(--line); }
.comparison-quotation:last-child { border-bottom: 0; }
.comparison-quotation blockquote { margin: 0 0 10px; padding-left: 12px; font: 16px/1.7 var(--serif); }
.comparison-quotation .text-action { white-space: normal; text-align: left; }
.comparison-absence { color: var(--muted); font-size: 12px; }
.comparison-context { padding: 14px 0; border-top: 1px solid var(--line); }
.comparison-context summary { cursor: pointer; font-weight: 600; }
.comparison-context h4 { margin: 18px 0 8px; }
.whatif-publications { margin-top: 14px; }
.whatif-publications summary { cursor: pointer; font-weight: 600; }
.whatif-publications h4 { margin: 16px 0 4px; }
.historical-name-reading { margin-bottom: 20px; padding: 16px; border: 1px solid var(--line); border-radius: 8px; background: var(--surface-2); }
.historical-name-reading h3 { margin: 0 0 8px; font: 23px/1.3 var(--serif); }
.historical-name-source { margin: 16px 0; }
.historical-name-source h4 { margin: 8px 0; }
.historical-name-quote { margin: 14px 0; }
.historical-name-quote blockquote { margin: 0 0 8px; padding-left: 12px; border-left: 3px solid var(--historical); font: 16px/1.65 var(--serif); }
.historical-name-match { color: var(--identity); font-size: 13px; }
.historical-name-chapter { margin: 14px 0; }
.historical-name-chapter>summary { cursor: pointer; font-weight: 600; }
.historical-name-original { white-space: pre-wrap; font: 16px/1.7 var(--serif); max-height: 320px; overflow: auto; padding: 12px; margin-top: 10px; border: 1px solid var(--line); background: var(--surface); }
.identity-pair { margin: 20px 0; padding-top: 16px; border-top: 1px solid var(--line); }
.identity-pair h4 { margin: 0 0 8px; font: 21px/1.3 var(--serif); }
.identity-pair-controls,.identity-pair-grid { display: grid; grid-template-columns: repeat(2,minmax(0,1fr)); gap: 16px; }
.identity-pair-controls label { min-width: 0; font-size: 12px; }
.identity-pair-controls select { display: block; width: 100%; min-width: 0; margin-top: 6px; }
.identity-pair-result { font-weight: 600; color: var(--identity); }
.identity-pair-paper { border-top: 1px solid var(--line); padding: 14px 0; }
.identity-pair-paper>summary,.identity-pair-more>summary,.identity-all-candidates>summary { cursor: pointer; font-weight: 600; }
.identity-pair-grid { margin-top: 12px; }
.identity-pair-grid section { min-width: 0; }
```

```
.identity-pair-grid h5 { margin: 0; font-size: 15px; font-style: italic; }
.identity-pair-grid blockquote { margin: 10px 0; overflow-wrap: anywhere; }
.identity-wording-found { color: var(--identity); font-weight: 600; }
.identity-pair-more,.identity-all-candidates { padding: 12px 0; }

.study-source-comparison { padding: 12px 14px; border: 1px solid var(--line); border-radius: 8px; background: var(--surface-2); }
.study-comparison-actions { margin: 12px 0; }
.study-outcome-definitions { margin: 12px 0; padding: 12px 14px; border-left: 3px solid var(--evidence); background: var(--surface-2); }
.study-outcome-definitions>summary { cursor: pointer; font-weight: 600; }
.study-outcome-definitions section+section { border-top: 1px solid var(--line); padding-top: 12px; }
.study-outcome-definitions blockquote { margin: 12px 0; }
.study-comparison-selection { border: 1px solid var(--line-strong); border-left: 4px solid var(--evidence); border-radius: 8px; padding: 16px; margin: 0 0 18px;
    background: var(--surface); }
.study-comparison-selection h3 { margin: 0; }
.study-comparison-selection ul { padding-left: 20px; }
.study-comparison-selection li { margin: 10px 0; }
.study-comparison-selection li button { margin-left: 12px; }
.study-comparison-dialog { width: min(1220px,calc(100vw - 24px)); max-height: calc(100dvh - 24px); padding: 20px; overflow: auto; border: 1px solid var(--line-
    strong); border-radius: 12px; background: var(--surface); color: var(--ink); }
.study-comparison-dialog::backdrop { background: rgba(22,38,28,.45); }
.study-comparison-heading { display: flex; justify-content: space-between; align-items: start; gap: 16px; position: sticky; top: -20px; padding: 12px 0; background:
    var(--surface); z-index: 2; border-bottom: 1px solid var(--line); }
.study-comparison-heading h2 { margin: 0; font: 25px/1.3 var(--serif); }
.study-comparison-heading button { flex-shrink: 0; }
.study-comparison-question { margin: 18px 0; }
.study-comparison-question blockquote { margin: 12px 0; }
.study-comparison-scroll { overflow-x: auto; margin: 12px 0 20px; border: 1px solid var(--line); }
.study-comparison-scroll table { width: 100%; border-collapse: separate; border-spacing: 0; table-layout: fixed; min-width: 760px; }
.study-comparison-scroll caption { text-align: left; padding: 12px; font-weight: 600; }
.study-comparison-scroll th,.study-comparison-scroll td { padding: 14px; vertical-align: top; text-align: left; border-top: 1px solid var(--line); border-right: 1px
    solid var(--line); overflow-wrap: anywhere; }
.study-comparison-scroll thead th { background: var(--surface-2); font: 17px/1.45 var(--serif); }
.study-comparison-scroll th:first-child { width: 150px; background: var(--surface-2); }
.study-comparison-scroll td { min-width: 260px; }
.study-comparison-scroll td p { margin: 0 0 12px; white-space: pre-line; font-size: 14px; line-height: 1.65; }
.study-comparison-scroll td p:last-child { margin-bottom: 0; }
.study-comparison-scroll summary { cursor: pointer; font-weight: 600; }
.study-comparison-scroll blockquote { margin: 12px 0; }

.statement-reading { margin: 0 0 18px; padding: 16px; border: 1px solid var(--line); border-radius: 8px; background: var(--surface-2); }
.statement-reading label { display: block; font-weight: 600; }
.statement-reading select { display: block; width: 100%; min-width: 0; margin-top: 8px; }
.statement-reading blockquote { margin: 14px 0; padding-left: 12px; border-left: 3px solid var(--historical); font: 17px/1.6 var(--serif); }
.statement-reading .row { flex-wrap: wrap; margin: 12px 0; }
.statement-reading-jump { position: sticky; top: 0; z-index: 3; padding: 10px 12px; border: 1px solid var(--line); border-radius: 6px; background: var(--surface);
    box-shadow: 0 2px 8px #263a2412; }
.statement-reading-jump:not([hidden]) { display: grid; gap: 4px; }
.statement-reading-jump button { text-align: left; }
.study-source-comparison > summary { cursor: pointer; font-weight: 600; color: var(--historical); }
.source-study-grid { display: grid; grid-template-columns: repeat(2,minmax(0,1fr)); gap: 20px; }
.source-study-grid > section { min-width: 0; }
.source-study-grid h4 { margin: 8px 0; }
.source-study-statement + .source-study-statement { margin-top: 16px; padding-top: 12px; border-top: 1px solid var(--line); }
.more-source-statements { margin-top: 14px; }
.more-source-statements > summary { cursor: pointer; color: var(--historical); font-size: 13px; }
.source-study-assessment { font-size: 13px; line-height: 1.6; }
.indication-comparison { padding: 12px 14px; margin: 12px 0; border-left: 3px solid var(--historical); background: var(--surface); }
.indication-comparison>summary { cursor: pointer; font-weight: 600; color: var(--historical); }
```

```
.indication-screen-scope { margin-top: 10px; font-size: 12px; color: var(--muted); }
.indication-screen-scope>summary { cursor: pointer; }
.study-wording { margin: 12px 0; }
.study-wording dt { margin-top: 8px; font-size: 12px; color: var(--muted); }
.study-wording dd { margin: 2px 0 0; font-size: 14px; }
.source-study-grid blockquote { overflow-wrap: anywhere; }
.export-preview { width: min(760px,calc(100vw - 36px)); max-height: calc(100vh - 36px); overflow: auto; padding: 20px; border: 1px solid var(--line-strong); border-radius: 12px; background: var(--surface); color: var(--ink); box-shadow: var(--shadow); }
.export-preview::backdrop { background: rgba(22,38,28,.4); }
.export-preview-head { display: flex; gap: 16px; justify-content: space-between; align-items: start; }
.export-preview-head h2 { margin: 0; font: 23px/1.3 var(--serif); overflow-wrap: anywhere; }
.export-preview textarea { width: 100%; height: min(48vh,480px); min-height: 160px; resize: vertical; font: 13px/1.55 ui-monospace,Consolas,monospace; margin: 12px 0 16px; }


/* Research summary, identity matrix and research map */
.research-summary { margin: 10px 0 12px; }
.research-summary .summary-line { font-size: 1rem; margin: 0 0 8px; }
.digest { list-style: none; margin: 0; padding: 0; display: grid; gap: 6px; }
.digest li { font-size: 0.88rem; }
.digest-topic { font-weight: 600; }
.digest-meta { color: var(--muted); font-size: 0.82rem; }
.identity-matrix { margin-bottom: 18px; }
.identity-matrix h3 { margin: 0 0 4px; }
.matrix-candidate { border: 1px solid var(--line); border-radius: 8px; background: var(--surface); padding: 10px 12px; margin: 8px 0; }
.matrix-candidate summary { cursor: pointer; }
.matrix-row { border-top: 1px solid var(--line); margin-top: 10px; padding-top: 8px; font-size: 0.9rem; }
.matrix-row blockquote { margin: 6px 0; font-size: 0.88rem; }
.identity-references-box { margin: 14px 0; }
.identity-references-box > summary { cursor: pointer; font-weight: 600; }
.map-summary { display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 10px; margin: 14px 0; }
.map-tile { border: 1px solid var(--line); border-radius: 8px; background: var(--surface); padding: 12px; display: grid; gap: 2px; }
.map-tile strong { font-size: 1.5rem; font-family: var(--serif); }
.map-tile span { color: var(--muted); font-size: 0.82rem; }
.map-foresight { margin: 10px 0 4px; max-width: 72ch; }
.map-heading { display: block; }
.research-map td { vertical-align: top; }
.button-link { align-self: end; padding: 8px 12px; border: 1px solid var(--line-strong); border-radius: 8px; color: var(--ink); text-decoration: none; }

.study-list { margin-top: 6px; }
.label-lookup { margin: 18px 0; padding: 16px; border: 1px solid var(--line); border-radius: 10px; }
.label-results { display: grid; gap: 12px; margin-top: 12px; }
.label-result { padding: 14px; border: 1px solid var(--line); border-radius: 8px; min-width: 0; overflow-wrap: anywhere; }
.label-ingredient { margin: 12px 0; padding: 12px; border-left: 3px solid var(--line-strong); background: var(--surface); overflow-wrap: anywhere; }
.label-ingredient p { margin: 6px 0; }
.label-attachment { margin: 12px 0; }
.label-comparison:not(:empty) { margin-top: 18px; padding: 16px; border: 2px solid var(--line-strong); border-radius: 8px; overflow-wrap: anywhere; }
.label-comparison section { padding: 8px 0; border-top: 1px solid var(--line); }
.study-list summary { cursor: pointer; font-size: 13px; color: var(--accent, inherit); }
.study-row { padding: 8px 0; border-top: 1px solid var(--line, rgba(127,127,127,.25)); }
.study-row p { margin: 0; }
.study-row blockquote { margin: 6px 0 0; padding-left: 10px; border-left: 2px solid var(--line, rgba(127,127,127,.35)); font-size: 13px; line-height: 1.45; }
.lead-toggle { display: block; margin-top: 4px; font-size: 0.82rem; }
.lead-row td { background: var(--surface-2); }
.lead-brief { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 14px 24px; padding: 6px 2px; }
.lead-brief h4 { margin: 0 0 4px; font-size: 0.8rem; letter-spacing: 0.04em; text-transform: uppercase; color: var(--muted); }
.lead-brief p { margin: 0 0 6px; }
.lead-brief blockquote { margin: 0 0 8px; font-family: var(--serif); }
```

```
1257
1258 /* Consultation and professional reading: one main task, then its saved result. */
1259 .care-heading { padding: 12px 0 22px; }
1260 .care-heading h1 { max-width: 20ch; margin: 10px 0 14px; font-size: clamp(30px,3.4vw,44px); line-height: 1.12; letter-spacing: -.025em; }
1261 .care-heading .library-lede { max-width: 66ch; font-size: 17px; }
1262 .care-start { display: flex; flex-wrap: wrap; justify-content: space-between; align-items: center; gap: 12px 24px; padding: 16px 0; border-block: 1px solid
     var(--line); }
1263 .care-start > p { max-width: 42ch; color: var(--muted); font-size: 14px; }
1264 .care-jump { display: inline-block; padding-block: 10px; font-size: 14px; text-underline-offset: 3px; }
1265 .care-scope { margin-top: 14px; color: var(--muted); font-size: 13px; max-width: 84ch; }
1266 .care-layout { display: grid; grid-template-columns: minmax(0,1fr) 280px; align-items: start; gap: 24px; }
1267 .care-main { min-width: 0; }
1268 .care-view .handoff-card { margin-block: 20px; padding: 24px; }
1269 .care-view .handoff-card > h2 { margin-bottom: 12px; overflow-wrap: anywhere; }
1270 .care-view p + p { margin-top: 8px; }
1271 .care-view .hint { line-height: 1.6; }
1272 .care-view label { min-width: 0; color: var(--ink); font-size: 14px; line-height: 1.5; }
1273 .care-view input,.care-view select,.care-view textarea { min-width: 0; min-height: 44px; font-size: 16px; }
1274 .care-view button,.care-view .button-link,.care-view summary { min-height: 44px; }
1275 .care-view button { white-space: normal; overflow-wrap: anywhere; }
1276 .care-view button:disabled { cursor: default; }
1277 .care-view [aria-busy="true"] button:disabled { cursor: progress; }
1278 .care-view summary { padding-block: 10px; }
1279 .care-editor > summary { display: list-item; padding: 0 0 0 2px; font-size: 17px; }
1280 .care-editor > summary > .hint { display: block; margin: 4px 0 0 18px; font-weight: 400; }
1281 .care-editor[open] > summary { padding-bottom: 20px; margin-bottom: 22px; border-bottom: 1px solid var(--line); }
1282 .care-form-heading { display: grid; gap: 8px; margin-bottom: 20px; }
1283 .care-form-heading h2 { font-size: 25px; }
1284 .care-field-grid { display: grid; grid-template-columns: repeat(2,minmax(0,1fr)); gap: 0 18px; }
1285 .care-field-grid > legend,.care-field-grid > p,.care-full-width { grid-column: 1 / -1; }
1286 .care-view .care-field-grid label { margin-block: 8px; }
1287 .care-section { padding-top: 24px; margin-top: 24px; border-top: 1px solid var(--line); }
1288 .care-section > h3,.care-section-title { font-family: var(--sans); font-size: 16px; font-weight: 700; line-height: 1.5; }
1289 .care-section-title { margin-top: 24px; }
1290 .care-section > .hint { margin-top: 8px; }
1291 .care-view .consultation-product { padding: 18px; background: var(--surface); }
1292 .care-view .consultation-product legend { padding-inline: 8px; color: var(--identity); font-weight: 700; }
1293 .care-view .consultation-product > .hint { margin-top: 12px; }
1294 .care-view .consultation-product > button:last-child { margin-top: 14px; color: var(--fail); }
1295 .care-view .label-lookup { background: var(--surface-2); padding: 14px 18px; }
1296 .care-view .label-lookup > summary { line-height: 1.5; }
1297 .care-view .label-result { background: var(--surface); }
1298 .care-view .label-result h3 { font-size: 19px; line-height: 1.4; }
1299 .care-view .label-result > p { margin: 8px 0; }
1300 .care-view .label-result button { margin: 8px 8px 0 0; }
1301 .care-view .question-criteria { padding: 16px 18px; margin-top: 24px; }
1302 .care-view .question-criteria legend { padding-inline: 8px; font-weight: 700; font-size: 16px; }
1303 .care-advanced { padding-block: 6px; border-bottom: 1px solid var(--line); }
1304 .care-form-actions { margin-top: 24px; padding-top: 18px; border-top: 1px solid var(--line); display: grid; gap: 10px; }
1305 .care-form-actions button { justify-self: start; padding: 12px 20px; }
1306 .care-context { display: flex; flex-wrap: wrap; align-items: center; gap: 12px 24px; padding: 16px 18px; border: 1px solid var(--line-strong); border-left: 4px solid
      var(--evidence); border-radius: 8px; background: var(--surface); }
1307 .care-context p { flex: 1 1 30ch; font-size: 14px; }
1308 .care-view [role="status"]:not(:empty) { margin-block: 14px; }
1309 #consultation-status:not(:empty),#question-status:not(:empty) { padding: 12px 16px; border: 1px solid var(--line-strong); border-left: 4px solid var(--identity);
     border-radius: 6px; background: var(--surface); line-height: 1.6; overflow-wrap: anywhere; }
1310 #consultation-status.error,#question-status.error { border-left-color: var(--fail); }
1311 .care-output-actions { display: flex; flex-wrap: wrap; gap: 12px; align-items: start; margin-block: 18px; }
```

```
.care-primary-link { display: inline-flex; align-items: center; background: var(--button); border-color: var(--button); color: var(--button-ink); font-weight: 600; padding: 10px 16px; }
.care-downloads { border: 1px solid var(--line-strong); border-radius: 6px; padding-inline: 12px; }
.care-downloads > summary { font-weight: 500; font-size: 14px; }
.care-downloads a { display: block; padding-block: 10px; font-size: 14px; }
.care-view .care-saved { padding: 20px; background: var(--surface-2); }
.care-saved h2 { font-size: 21px; }
.care-saved-item { padding: 14px 0; border-bottom: 1px solid var(--line); }
.care-saved .handoff-list-item { border: none; padding: 4px 0; min-height: 44px; font-weight: 600; color: var(--identity); }
.care-saved .handoff-list-item:hover { text-decoration: underline; text-underline-offset: 3px; }
.care-saved-item p { font-size: 12px; overflow-wrap: anywhere; }
.care-import { margin-top: 18px; }
.care-import > summary { font-size: 14px; }
.care-view input[type="file"] { font-size: 13px; padding: 8px 4px; }
.care-view blockquote { margin: 12px 0; padding-left: 14px; border-left: 3px solid var(--line-strong); }
.care-view .consultation-summary { align-items: start; font-size: 14px; }
.care-view .consultation-product-reading h3 { font-size: 24px; line-height: 1.3; overflow-wrap: anywhere; }
.care-view .consultation-product-reading h4 { margin: 20px 0 10px; }
.care-view .consultation-product-reading > button { margin-block: 8px; }
.care-view .question-branch { padding: 18px; }
.care-view .question-branch h3 { line-height: 1.4; overflow-wrap: anywhere; }
.question-queue { margin-block: 12px; }
.question-queue > summary { border-bottom: 1px solid var(--line); font-size: 14px; }
.care-view .question-study { padding: 16px; }
.care-view .question-study > summary { font-family: var(--serif); font-size: 19px; line-height: 1.5; }
.care-view .question-study > a { display: inline-block; padding-block: 10px; }
.care-view .question-check > strong { font-size: 14px; }
.care-view .question-check > p { margin-top: 6px; font-size: 14px; }
.care-view .question-study form { margin-top: 18px; padding-top: 18px; border-top: 1px solid var(--line); }
.care-view .question-study form h4 { margin: 0; }
.care-view article[tabindex="-1"]:focus { outline: 2px solid var(--identity); outline-offset: 4px; }


/* Consultation clarity: a short path, then progressively disclosed source detail. */
.care-steps{display:flex;gap:.5rem;flex-wrap:wrap;margin:1rem 0 1.5rem}
.care-steps button{flex:1;min-width:90px}
.care-steps [aria-current="step"]{background:var(--green,#245846);color:#fff;border-color:#245846}
.care-step-actions{display:flex;gap:.75rem;justify-content:flex-end;margin-top:1.2rem}
.care-use-questions{display:grid;gap:1rem}
.care-footer{max-width:1440px;margin:2rem auto;padding:1.5rem 3vw;border-top:1px solid #cdd8d0;display:flex;gap:1rem;flex-wrap:wrap;font-size:.85rem;color:#50635a}
.question-overview{margin:1.5rem 0;padding:clamp(1rem,3vw,1.7rem);border:1px solid #b9cec1;border-radius:12px;background:#f1f6f0;color:#213c30}
.question-overview h3{margin-top:.3rem}
.question-finding{background:#fff;padding:1rem 1.2rem;margin:1rem 0;border:1px solid #d3ddd4;border-radius:8px;color:#213c30}
.question-finding h4{font-size:1.1rem;margin:0 0 .5rem}
.question-finding strong{font-size:.85rem}
.question-finding p{margin:.45rem 0 .8rem}
.question-finding details{margin-top:.8rem}
.question-overview a{color:#20583e;overflow-wrap:anywhere}

```

scripts/acquire_care_research.py
SHA-256: f024cc0ff2a025360399b36f21017702588ded0888ddb8e80cca4c33461ca726

```
"""Acquire a bounded, licensed addition to the browser research library.

Only CC BY/CC0 abstracts are redistributed. No full text, figures, health inputs,
or restricted abstracts. Rebuilds use the saved snapshot, never a live query.
"""
from __future__ import annotations
import hashlib
import json
import re
import time
import urllib.parse
import urllib.request
from datetime import datetime, timezone
from html import unescape
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
# A small purpose-built set of human studies and reviews; mixtures remain labelled
# in their original titles/abstracts and are never treated as the person's product.
IDS = ['PMC11133624', 'PMC10221652', 'PMC11053596', 'PMC7847013', 'PMC8746505',
       'PMC13260694', 'PMC12436692', 'PMC10755896', 'PMC3948786']


def plain(text):
    return re.sub(r'\s+', ' ', re.sub(r'<[^>]+>', '', unescape(text or ''))).strip()


def main():
    papers, omitted = [], []
    url = 'https://www.ebi.ac.uk/europepmc/webservices/rest/search?' + urllib.parse.urlencode(
        dict(query=' OR '.join(IDS), resultType='core', format='json', pageSize=50))
    cache = ROOT / '.runtime/care-additions-response.json'
    cache.parent.mkdir(parents=True, exist_ok=True)
    if cache.exists(): raw = cache.read_bytes()
    else:
        with urllib.request.urlopen(url, timeout=45) as response: raw = response.read()
        cache.write_bytes(raw)
    rows = json.loads(raw)['resultList']['result']
    for pmcid in IDS:
        matches = [x for x in rows if x.get('pmcid') == pmcid]
        r = max(matches, key=lambda x: len(x.get('abstractText', '')))
        licence = r.get('license', '').lower()
        if licence not in ('cc by', 'cc0') or not r.get('abstractText'):
            omitted.append(dict(id=pmcid, reason='No abstract or unsupported licence', licence=licence)); continue
        papers.append(dict(id=pmcid, title=plain(r['title']), abstract=plain(r['abstractText']),
                           url='https://europepmc.org/articles/' + pmcid, licence=licence,
                           authors=r.get('authorString', ''), year=r.get('pubYear'), doi=r.get('doi'),
                           acquisition_url=url, response_sha256=hashlib.sha256(raw).hexdigest()))
        print(pmcid, licence, flush=True)
        time.sleep(.5)
    payload = dict(created_at=datetime.now(timezone.utc).isoformat(),
                   selection='Bounded human-study/review additions for cognition, joint symptoms and lipids; not an exhaustive review.',
                   papers=papers, omitted=omitted)
    (ROOT / 'data/demo/care_research_additions.json').write_text(json.dumps(payload, ensure_ascii=False, indent=1) + '\n', encoding='utf-8')

```

```
56
57 if __name__ == '__main__': main()
```

scripts/ai_medicine_groups.py
SHA-256: 55bf87ce06c9d1a62f3cd390c19f8255d85999c94dde7543acb7856be8074cee

```
"""Everyday-language medicine groups for FoliaTrace's on-device AI, and their evaluation.

The browser embeds what a person typed ("my blood pressure pill") with all-MiniLM-L6-v2 and compares
it with each group description below. A group is used only at cosine >= THRESHOLD; otherwise the
medicine stays "not recognised" and goes to the pharmacist. Groups map only to KB classes that a
quoted NIH sentence names, so an AI-understood group can trigger exactly the same NIH statements.

  python scripts/ai_medicine_groups.py      # evaluate on the development and held-out phrase sets
"""
from __future__ import annotations

import json
from datetime import date
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
THRESHOLD = 0.55  # v2: calibrated on expanded development phrases, never on held-out results
MARGIN = 0.06

GROUPS = [
    dict(id='blood_thinner', label='a blood thinner (anticoagulant)', classes=['anticoagulant', 'bleeding'],
         description='A blood thinner (anticoagulant) such as warfarin, Coumadin, Eliquis or Xarelto, taken to prevent blood clots and stroke.'),
    dict(id='aspirin', label='daily aspirin', classes=['aspirin', 'bleeding'],
         description='Daily low-dose aspirin, a baby aspirin taken for the heart.'),
    dict(id='birth_control', label='birth control pills', classes=['oral_contraceptive'],
         description='Birth control pills, oral contraceptives taken to prevent pregnancy.'),
    dict(id='antidepressant', label='an antidepressant', classes=['serotonergic'],
         description='An antidepressant for depression or anxiety, such as an SSRI like Zoloft, Prozac or Lexapro.'),
    dict(id='sleep', label='a sleeping pill or sedative', classes=['sedative'],
         description='A sleeping pill or sedative for insomnia, such as Ambien, or a benzodiazepine like Xanax or Valium.'),
    dict(id='seizure', label='an anti-seizure medicine', classes=['anticonvulsant'],
         description='An anti-seizure medicine for epilepsy, such as Keppra, Dilantin or Lamictal.'),
    dict(id='cholesterol', label='a statin (cholesterol medicine)', classes=['statin'],
         description='A statin to lower cholesterol, such as Lipitor, Crestor or Zocor.'),
    dict(id='diabetes', label='a diabetes medicine', classes=['diabetes_med'],
         description='A diabetes medicine to lower blood sugar, such as metformin, insulin or Ozempic.'),
    dict(id='blood_pressure', label='a blood-pressure medicine', classes=['bp_med'],
         description='A blood pressure medicine for hypertension, such as lisinopril, amlodipine, losartan, a beta blocker or a water pill (diuretic).'),
    dict(id='steroid', label='a steroid (corticosteroid)', classes=['corticosteroid'],
         description='A steroid pill such as prednisone, taken for inflammation, asthma flare-ups or arthritis.'),
    dict(id='thyroid', label='a thyroid hormone medicine', classes=['thyroid_hormone'],
         description='Thyroid hormone pills, such as levothyroxine or Synthroid, for an underactive thyroid.'),
    dict(id='transplant', label='an anti-rejection medicine (immunosuppressant)', classes=['immunosuppressant'],
         description='An anti-rejection medicine after an organ transplant, an immunosuppressant such as tacrolimus or cyclosporine.'),
    dict(id='hiv', label='an HIV medicine', classes=['hiv'], description='HIV medicine, antiretroviral therapy.'),
]

PROTOTYPES = {
    'blood_thinner': ['Blood thinning medicine to prevent blood clots.'],
    'aspirin': ['Aspirin taken daily in a low dose for the heart.'],
    'birth_control': ['Contraceptive medicine to prevent pregnancy.'],
    'antidepressant': ['Antidepressant medicine prescribed for depression and anxiety.'],
    'sleep': ['Prescription sleeping medicine for insomnia.'],
    'seizure': ['Anticonvulsant medicine to prevent seizures and epileptic fits.'],
    'cholesterol': ['Cholesterol lowering medicine, a statin tablet.'],
```

```
    'diabetes': ['Medicine for diabetes to control blood sugar.'],
    'blood_pressure': ['A water pill, a diuretic medicine that helps remove excess fluid.',
                       'Medicine to lower high blood pressure and treat hypertension.'],
    'steroid': ['Corticosteroid tablets to treat inflammation.'],
    'thyroid': ['Thyroid hormone replacement for an underactive thyroid.'],
    'transplant': ['Immunosuppressant medicine to prevent rejection after organ transplant.'],
    'hiv': ['Antiretroviral medicines to treat HIV.'],
}
for group in GROUPS:
    group['prototypes'] = PROTOTYPES[group['id']]

DEV = [
    ('my blood thinner', 'blood_thinner'), ('pill to stop blood clots', 'blood_thinner'), ('the clot medicine after my stroke', 'blood_thinner'),
    ('baby aspirin every morning', 'aspirin'), ('aspirin for my heart', 'aspirin'),
    ('the pill', 'birth_control'), ('my contraceptive', 'birth_control'), ('pill so I don\u2019t get pregnant', 'birth_control'),
    ('my antidepressant', 'antidepressant'), ('pill for depression', 'antidepressant'), ('something for my mood and anxiety', 'antidepressant'),
    ('my sleeping pill', 'sleep'), ('something to help me sleep at night', 'sleep'),
    ('epilepsy medicine', 'seizure'), ('pill for my seizures', 'seizure'),
    ('my cholesterol pill', 'cholesterol'), ('statin', 'cholesterol'),
    ('my sugar pill', 'diabetes'), ('diabetes medication', 'diabetes'), ('insulin shots', 'diabetes'),
    ('my blood pressure pill', 'blood_pressure'), ('water pill', 'blood_pressure'), ('pill for high blood pressure', 'blood_pressure'),
    ('steroids for my arthritis', 'steroid'), ('prednisone pack', 'steroid'),
    ('thyroid pill', 'thyroid'), ('hormone for my thyroid', 'thyroid'),
    ('anti-rejection pills for my kidney transplant', 'transplant'), ('HIV meds', 'hiv'),
    ('painkiller for my back', None), ('antibiotics', None), ('allergy pill', None), ('vitamin D', None), ('antacid for heartburn', None), ('my inhaler', None),
]
DEV += [('my water pill', 'blood_pressure'), ('her water pill', 'blood_pressure'),
        ('the tablet to get rid of extra fluid', 'blood_pressure'),
        ('I take a pill to keep my cholesterol down', 'cholesterol'), ('my statin at night', 'cholesterol'),
        ('a small white pill', None), ('I cannot remember', None), ('medicine', None),
        ('herbal sleeping tea', None), ('fish oil for cholesterol', None), ('ibuprofen', None), ('water', None),
        ('my pain pill', None), ('vitamin C', None), ('not taking any pills', None), ('I stopped taking my statin', None)]
# A bare "the pill" is ambiguous; v2 deliberately leaves it for the pharmacist.
DEV = [(t, None if t == 'the pill' else g) for t, g in DEV]
HELD_OUT = [
    ('the medicine that thins my blood', 'blood_thinner'), ('anticoagulant', 'blood_thinner'), ('stroke prevention pill for my AFib', 'blood_thinner'),
    ('daily aspirin', 'aspirin'), ('low dose aspirin', 'aspirin'),
    ('birth control', 'birth_control'), ('oral contraceptive pill', 'birth_control'),
    ('depression medication', 'antidepressant'), ('SSRI', 'antidepressant'), ('pill my psychiatrist gave me for depression', 'antidepressant'),
    ('sleep aid prescription', 'sleep'), ('pill for insomnia', 'sleep'), ('nerve pill to calm me down', 'sleep'),
    ('anticonvulsant', 'seizure'), ('medicine to prevent fits', 'seizure'),
    ('pill to lower my cholesterol', 'cholesterol'), ('lipid medicine', 'cholesterol'),
    ('blood sugar medicine', 'diabetes'), ('type 2 diabetes pill', 'diabetes'),
    ('hypertension medicine', 'blood_pressure'), ('diuretic', 'blood_pressure'), ('heart pressure pill', 'blood_pressure'),
    ('corticosteroid tablets', 'steroid'), ('steroid pills', 'steroid'),
    ('levothyroxine for hypothyroidism', 'thyroid'), ('underactive thyroid medicine', 'thyroid'),
    ('immunosuppressant after liver transplant', 'transplant'), ('antiretrovirals', 'hiv'),
    ('ibuprofen', None), ('cough syrup', None), ('eye drops', None), ('multivitamin', None), ('medicine for acid reflux', None), ('antihistamine for hay fever',
None),
]


# Frozen v2 synthetic holdout, authored after the prototype design and before
# any v2 scores were inspected. Keep failures; do not tune on these phrases.
HELD_OUT_V2 = [
    ('tablets that prevent my blood from clotting', 'blood_thinner'), ('my anticoagulation medication', 'blood_thinner'), ('a blood thinner for atrial fibrillation',
 'blood_thinner'),
    ('the little aspirin I take each day', 'aspirin'), ('a low-dose aspirin tablet for my heart', 'aspirin'), ('my daily baby aspirin tablet', 'aspirin'),
```

```
    ('the contraception tablets I take daily', 'birth_control'), ('a pill to avoid getting pregnant', 'birth_control'), ('my oral birth control medicine', 'birth_control'),
    ('the medication for my depression', 'antidepressant'), ('an SSRI tablet every morning', 'antidepressant'), ('the antidepressant my doctor prescribed', 'antidepressant'),
    ('my prescribed insomnia medication', 'sleep'), ('a sedative I take before bed', 'sleep'), ('tablets prescribed to help with sleeping', 'sleep'),
    ('tablets for controlling epilepsy', 'seizure'), ('an anti seizure drug every day', 'seizure'), ('my medicine that stops seizures', 'seizure'),
    ('the tablets for high cholesterol', 'cholesterol'), ('a statin my doctor gave me', 'cholesterol'), ('medicine for lowering my lipids', 'cholesterol'),
    ('tablets to manage my type two diabetes', 'diabetes'), ('medication that controls my glucose', 'diabetes'), ('my daily diabetes treatment', 'diabetes'),
    ('my diuretic tablet for fluid buildup', 'blood_pressure'), ('the tablets that bring my blood pressure down', 'blood_pressure'), ('her prescribed water tablets', 'blood_pressure'),
    ('oral corticosteroids prescribed for inflammation', 'steroid'), ('a steroid course for an asthma flare', 'steroid'), ('the prednisone tablets for inflammation', 'steroid'),
    ('my thyroid replacement tablets', 'thyroid'), ('hormone replacement for hypothyroidism', 'thyroid'), ('medicine for a sluggish thyroid gland', 'thyroid'),
    ('the medication to stop my transplanted kidney being rejected', 'transplant'), ('anti rejection treatment after my transplant', 'transplant'), ('immunosuppressive pills for my donor liver', 'transplant'),
    ('antiretroviral treatment for HIV infection', 'hiv'), ('the drugs I take to manage HIV', 'hiv'), ('my HIV antiviral therapy', 'hiv'),
    ('an antibiotic course for an infection', None), ('a pain relief tablet for headaches', None), ('nasal spray for hay fever', None),
    ('tablets for indigestion', None), ('an iron supplement for anemia', None), ('chamomile tea before bed', None),
    ('I no longer take blood thinners', None), ('I used to take cholesterol tablets', None), ('not taking diabetes medication', None),
    ('the tiny round tablet', None), ('something my neighbour recommended', None), ('medicine for nausea on the boat', None),
    ('my calcium and vitamin D', None), ('cough lozenges', None), ('cream for my eczema', None),
]


def embedder():
    import numpy as np
    import onnxruntime as ort
    from tokenizers import Tokenizer
    tok = Tokenizer.from_file(str(ROOT / 'models/minilm/tokenizer.json')); tok.enable_truncation(256)
    sess = ort.InferenceSession(str(ROOT / 'models/minilm/model.onnx')); names = {i.name for i in sess.get_inputs()}
    def emb(text):
        e = tok.encode(text); m = np.array([e.attention_mask], dtype=np.int64)
        feed = {'input_ids': np.array([e.ids], dtype=np.int64), 'attention_mask': m, 'token_type_ids': np.array([e.type_ids], dtype=np.int64)}
        h = sess.run(None, {k: v for k, v in feed.items() if k in names})[0][0]
        v = (h * m[0][:, None]).sum(0) / m.sum(); return v / np.linalg.norm(v)
    return emb


def exclusion(text):
    import re
    s = text.lower().replace('\u2019', "'")
    if re.search(r"\b(not taking|no longer|stopped|used to|don't take|do not take|never take|without medication|not on|none)\b", s): return 'not_current'
    if re.search(r'\b(herbal|tea|fish oil|vitamin|multivitamin|supplement|probiotic)\b', s): return 'not_medicine'
    if re.fullmatch(r'the pill|a pill|pill|medicine|medication|tablets?|a small white pill|i cannot remember', s.strip()): return 'too_vague'
    return None


def score(data, emb, group_vecs):
    rows, c = [], dict(correct_group=0, wrong_group=0, correctly_unassigned=0, missed=0)
    for text, gold in data:
        v = emb(text); sims = {g: max(float(v @ p) for p in gv) for g, gv in group_vecs.items()}
        ranked = sorted(sims, key=sims.get, reverse=True); best = ranked[0]; margin = sims[best] - sims[ranked[1]]
        pred = best if not exclusion(text) and sims[best] >= THRESHOLD and margin >= MARGIN else None
        if pred and pred == gold: c['correct_group'] += 1
        elif pred: c['wrong_group'] += 1
        elif gold is None: c['correctly_unassigned'] += 1
        else: c['missed'] += 1
        rows.append(dict(text=text, gold=gold, predicted=pred, similarity=round(sims[best], 3), margin=round(margin, 3), excluded=exclusion(text)))
```

```
    drugs = sum(1 for _, g in data if g)
    return dict(counts=c, phrases=len(data), drug_phrases=drugs, recall=round(c['correct_group'] / max(1, drugs), 3),
                precision=round(c['correct_group'] / max(1, c['correct_group'] + c['wrong_group']), 3), rows=rows)


def main():
    emb = embedder(); gv = {g['id']: [emb(p) for p in [g['description'], *g['prototypes']]] for g in GROUPS}
    report = dict(created='2026-11-12', version=2, model='sentence-transformers/all-MiniLM-L6-v2 (data/models.lock.json)', threshold=THRESHOLD, margin=MARGIN,
                  method='Cosine similarity between the typed phrase and each group description; below the threshold the medicine stays unrecognised. '
                         'V2 adds short semantic prototypes, conservative exclusions for negated/historical/supplement mentions, and a runner-up margin. '
                         'Thresholds 0.50/0.55/0.60/0.65 and margins 0/0.03/0.06 were explored on development only. '
                         'The original held-out set is now a regression set; fresh held-out phrases were authored before scoring v2. '
                         'These are synthetic English phrases, not a clinical or user validation. Class inferences require pharmacist confirmation.',
                  groups=GROUPS, development=score(DEV, emb, gv), regression=score(HELD_OUT, emb, gv), held_out=score(HELD_OUT_V2, emb, gv))
    out = ROOT / 'reports/ai_medicine_understanding_v2_2026-11-12.json'
    out.write_text(json.dumps(report, ensure_ascii=False, indent=1) + '\n', encoding='utf-8')
    for k in ('development', 'held_out'):
        r = report[k]; print(k, r['counts'], 'recall', r['recall'], 'precision', r['precision'])


if __name__ == '__main__':
    main()
```

scripts/build_browser_inference.mjs
SHA-256: 961712aebcd2d8f9a9e12d3816c1677c1a6eedeca23314fc1a5afa8fd2bfadde

```
// Copy pinned, redistributable assets; no model or question goes to a third-party CDN.
import {copyFile,mkdir,readFile,writeFile} from 'node:fs/promises';
import {createHash} from 'node:crypto';
import {resolve,join} from 'node:path';
const root=resolve(import.meta.dirname,'..'),target=join(root,'sites/foliatrace-research/build/inference');await mkdir(target,{recursive:true});
const lock=JSON.parse(await readFile(join(root,'data/reranker.lock.json'),'utf8'));
for(const file of lock.files){const source=join(root,'models/reranker',file.local);const bytes=await readFile(source);if(createHash('sha256').update(bytes).digest('hex')!==file.sha256)throw Error('Pinned model artifact differs: '+file.local);await copyFile(source,join(target,file.local));}
for(const file of ['ort.wasm.min.mjs','ort-wasm-simd-threaded.mjs','ort-wasm-simd-threaded.wasm'])await copyFile(join(root,'node_modules/onnxruntime-web/dist',file),join(target,file));
await copyFile(join(root,'node_modules/@huggingface/tokenizers/dist/tokenizers.mjs'),join(target,'tokenizers.mjs'));
for(const [name,url] of [['ONNX-RUNTIME-LICENSE.txt','https://raw.githubusercontent.com/microsoft/onnxruntime/main/LICENSE'],['ONNX-THIRD-PARTY-NOTICES.txt','https://raw.githubusercontent.com/microsoft/onnxruntime/main/ThirdPartyNotices.txt'],['MODEL-LICENSE.txt','https://www.apache.org/licenses/LICENSE-2.0.txt']]){
 const response=await fetch(url);if(!response.ok)throw Error('Required licence download failed: '+url);await writeFile(join(target,name),await response.text());
}
await copyFile(join(root,'node_modules/@huggingface/tokenizers/LICENSE'),join(target,'TOKENIZERS-LICENSE.txt'));
await writeFile(join(target,'manifest.json'),JSON.stringify({model:lock.model,revision:lock.revision,licence:lock.licence,model_sha256:lock.files.find(f=>f.local==='model.onnx').sha256,tokenizer_sha256:lock.files.find(f=>f.local==='tokenizer.json').sha256,runtime:'onnxruntime-web@1.31.1',tokenizers:'@huggingface/tokenizers@0.2.0',sequence_length:384,scope:'Relevance ranking only; not evidence entailment or probability.'},null,2));
console.log('Copied pinned browser inference assets to '+target);
```

scripts/build_care_demo.py
SHA-256: 76a36334531faf096e600619332f139259bbffa8e2ecbaa13d50fdbce992c5d9

```
"""Build a privacy-separated, source-only public care demonstration.

No saved user records or local PubMed abstract cache are opened by this build.
The interactive preparation is browser-local; literature cards are labelled replays.
"""
from __future__ import annotations
import hashlib
import json
import shutil
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.common import now
from src.pipeline import Pipeline
from src.graph.store import GraphStore
from src.evidence.consultation import ConsultationInput, prepare_consultation
from src.evidence.product_labels import reference_labels, label_wording
from src.evidence.question_review import ResearchQuestion, run_question, canonical
from src.evidence.care_workspace import study_bridge


def wrapper(packet, origin):
    sha = hashlib.sha256(canonical(packet)).hexdigest()
    return {'id': sha[:24], 'sha256': sha, 'packet': packet, 'origin': origin}


def main():
    runtime = ROOT / '.runtime/public-care-build'
    runtime.mkdir(parents=True, exist_ok=True)
    pipeline = Pipeline(store=GraphStore(runtime / 'isolated.sqlite'), include_local_abstracts=False)
    pipeline.client.offline = True
    distributable = set()
    for name in ('papers.json', 'papers_extended.json'):
        records = json.loads((pipeline.data_dir / name).read_text(encoding='utf-8'))
        distributable.update(p['id'] for p in (records['papers'] if isinstance(records, dict) else records))
    assert all(not p.get('abstract') or p['id'] in distributable for p in pipeline.papers)
    labels = {s['label_id']: s for s in reference_labels()}
    scenarios = [
        ('mint', 'Mint capsules', None, None, 'digestive symptoms', ['abdominal pain', 'irritable bowel syndrome']),
        ('ginger', 'Ginger & nausea', 9936, 'Ginger root Ethanolic Extract', 'nausea', ['nausea', 'vomiting']),
        ('turmeric', 'Turmeric & joint symptoms', 9936, 'Turmeric root Supercritical CO2 Extract', 'joint symptoms', ['osteoarthritis', 'joint pain']),
        ('valerian', 'Valerian & sleep', 216782, 'Valerian', 'sleep', ['sleep quality', 'insomnia']),
        ('chamomile', 'Chamomile & sleep', 216782, 'Chamomile', 'sleep', ['sleep quality', 'insomnia']),
    ]
    cases = []
    for slug, title, label_id, ingredient_name, topic, terms in scenarios:
        if label_id:
            source = labels[label_id]
            product = {'name': source['raw']['fullName'], 'brand': source['raw'].get('brandName') or '',
                       'source_kind': 'PRODUCT_LABEL', 'source_text': label_wording(source['raw']),
                       'label_snapshot': source, 'reason_for_use': 'Fictional question about ' + topic}
        else:
            product = {'name': 'Mint capsules', 'plant_wording': 'Mint',
```

```
                        'source_text': 'Mint capsules; botanical species not stated.',
                        'dose': 'One capsule', 'frequency': 'Occasionally', 'preparation': 'capsules',
                        'reason_for_use': 'Fictional question about digestive symptoms'}
        prep = wrapper(prepare_consultation(pipeline, ConsultationInput(
            visit_label='Fictional public example: ' + title, products=[product])), 'LOCAL_CONSULTATION_PREPARATION')
        item = prep['packet']['products'][0]
        ingredient = next((r for r in item['label_ingredients'] if r['name'] == ingredient_name), None) if label_id else None
        name = (ingredient.get('research_lookup') or ingredient['lookup']) if ingredient else item['name_lookup']
        protocol = ResearchQuestion(question='Which human studies of ' + name + ' report ' + topic + '?',
                                    plant_name=name, source_text=ingredient['review_source_text'] if ingredient else product['source_text'],
                                    outcome_terms=terms, preparation='capsules' if not label_id else '', ranking='auto')
        review = wrapper(run_question(pipeline, protocol), 'LOCAL_QUESTION_REVIEW')
        packet = review['packet']
        rows = {(b['taxon_id'], r['id']): r for b in packet['branches'] for r in b['records']}
        leads = packet['overview']['top_studies']
        # A missing lead remains empty; unrelated papers do not fill the result.
        studies = []
        for lead in leads:
            row = rows[(lead['taxon_id'], lead['record_id'])]
            assert not row['paper'].get('abstract') or row['paper']['id'] in distributable
            bridge = study_bridge(prep, review, item['reported']['item_id'], row['id'], ingredient['key'] if ingredient else None)
            studies.append({'bridge': bridge, 'screening': row['screening'], 'study_types': row['study']['types'],
                            'licence': row['paper']['licence'], 'abstract': row['paper'].get('abstract') or '',
                            'paper_id': row['paper']['id']})
        cases.append({'id': slug, 'title': title, 'question': protocol.question,
                      'reported': item['reported'], 'candidates': ingredient['candidates'] if ingredient else item['candidates'],
                      'ingredient': ingredient_name, 'source_text': protocol.source_text,
                      'overview': {k: v for k, v in packet['overview'].items() if k not in ('selected_readings',)},
                      'coverage': packet['coverage'], 'studies': studies, 'review_sha256': review['sha256'],
                      'source_label': {'id': label_id, 'url': labels[label_id]['source_url'], 'sha256': labels[label_id]['sha256']} if label_id else None})
        print(slug, packet['overview']['endpoint_publications'], 'wording leads;', packet['coverage']['model'], flush=True)
    target = ROOT / 'sites/foliatrace-research/build'
    target.mkdir(parents=True, exist_ok=True)
    payload = {'created_at': now(), 'mode': 'RECORDED_SOURCE_ONLY_CARE_CASES',
               'pipeline_version': __import__('src').__version__, 'code_fingerprint': pipeline.code_fingerprint,
               'snapshot_sha256': pipeline.snapshot, 'local_private_abstracts_loaded': False,
               'abstracts_available': sum(bool(p.get('abstract')) for p in pipeline.papers),
               'cases': cases}
    (target / 'care-cases.json').write_text(json.dumps(payload, ensure_ascii=False, indent=2) + '\n', encoding='utf-8')
    library = [{k: p.get(k, '') for k in ('id', 'title', 'abstract', 'url', 'licence')}
               for p in pipeline.papers if p['id'] in distributable and p.get('abstract')]
    assert len(library) == 415 and len({p['id'] for p in library}) == len(library)
    (target / 'browser-library.json').write_text(json.dumps(library, ensure_ascii=False) + '\n', encoding='utf-8')
    shutil.copy2(ROOT / 'data/NOTICE.md', target / 'DATA-NOTICE.md')
    shutil.copy2(ROOT / 'LICENSE', target / 'CODE-LICENSE.txt')
    shutil.copy2(ROOT / 'reports/retrieval_external.json', target / 'retrieval-evaluation.json')
    from scripts.sync_public_demo import main as sync_app
    sync_app()  # checker app, OCR runtime, QR library, source archive, one-page PDF
    print('Built', len(cases), 'public cases at', target)


if __name__ == '__main__':
    main()
```

scripts/build_code_pdf.py
SHA-256: c701e770191668446d96a8d32d14f0ca0e2ee23e6d7e4373ba795cf19bd7f883

```
"""Render every first-party code file in the allow-listed source ZIP as a code PDF."""
import hashlib
import textwrap
from pathlib import Path
from zipfile import ZipFile
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import landscape, A4
from reportlab.lib import colors
from pypdf import PdfReader

ROOT=Path(__file__).resolve().parents[1]
OUT=ROOT/'output/pdf/foliatrace-code.pdf'
EXT={'.py','.js','.mjs','.ps1','.html','.css','.sql'}


def main():
    archive=ROOT/'dist/dsh-v2/foliatrace-source.zip'
    with ZipFile(archive) as z:
        files=[(n,z.read(n)) for n in sorted(z.namelist()) if Path(n).suffix in EXT]
    OUT.parent.mkdir(parents=True,exist_ok=True)
    c=canvas.Canvas(str(OUT),pagesize=landscape(A4),pageCompression=1)
    c.setTitle('FoliaTrace - complete first-party source listing')
    c.setAuthor('FoliaTrace')
    width,height=landscape(A4);page=0;y=0;current='Source listing'
    def new_page():
        nonlocal page,y
        if page:c.showPage()
        page+=1;c.setFillColor(colors.HexColor('#24513d'));c.setFont('Helvetica-Bold',10)
        c.drawString(30,height-24,'FoliaTrace / source code')
        c.setFont('Helvetica',8);c.drawRightString(width-30,height-24,str(page))
        c.setStrokeColor(colors.HexColor('#bccdbd'));c.line(30,height-32,width-30,height-32)
        c.setFont('Helvetica',8);c.drawString(30,18,current[:150])
        c.setFillColor(colors.black);c.setFont('Courier',7);y=height-47
    new_page()
    intro=[
        'FoliaTrace - complete first-party source listing',
        f'{len(files)} code files from the accompanying allow-listed source archive.',
        'The archive also contains data, evaluation reports, README and AI disclosure.',
        'Third-party runtimes, model binaries, credentials, caches and saved user records are excluded.',
        'Non-ASCII code points are shown as reversible Python-style Unicode escapes.',
        'Long lines wrap visually; the original file bytes and hashes remain in source.zip.',
        'AI-assisted development by the entrant with OpenAI Codex and Anthropic Claude.',
        'Prior EthnoHack work is disclosed in README; this listing includes the complete packaged implementation.',
        f'Source archive SHA-256: {hashlib.sha256(archive.read_bytes()).hexdigest()}',
    ]
    for line in intro:
        c.drawString(30,y,line);y-=14
    for name,data in files:
        current=name.removeprefix('foliatrace/');new_page();c.bookmarkPage(current)
        c.addOutlineEntry(current,current,level=0)
        lines=[current,'SHA-256: '+hashlib.sha256(data).hexdigest(),'']
        text=data.decode('utf-8-sig')
        for number,line in enumerate(text.splitlines(),1):
            encoded=line.expandtabs(4).encode('ascii','backslashreplace').decode('ascii')
            wrapped=textwrap.wrap(encoded,width=165,replace_whitespace=False,drop_whitespace=False) or ['']
```

```
            lines.extend([f'{number:5d} '+wrapped[0],*['      '+s for s in wrapped[1:]]])
        for line in lines:
            if y<33:new_page()
            c.drawString(30,y,line);y-=9
    c.save()
    pages=len(PdfReader(OUT).pages)
    assert pages==page
    print(f'{OUT}: {len(files)} code files, {pages} pages')


if __name__=='__main__':main()
```

scripts/build_product_onepage.py
SHA-256: 3143fe45e163fd416118da4058499f99da51bf4dab396bc29b3d2fe5ca176f40

```
"""Build the current one-page product overview without browser automation."""
from pathlib import Path
from xml.sax.saxutils import escape
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_LEFT
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from product_metrics import release_metrics

ROOT=Path(__file__).resolve().parents[1]
TARGET=ROOT/'output/pdf/foliatrace-one-page.pdf'
GREEN=colors.HexColor('#24513d')
styles={
    'title':ParagraphStyle('title',fontName='Times-Bold',fontSize=34,leading=38,textColor=GREEN,spaceAfter=10),
    'lead':ParagraphStyle('lead',fontName='Helvetica',fontSize=15,leading=20,textColor=GREEN,spaceAfter=16),
    'heading':ParagraphStyle('heading',fontName='Helvetica-Bold',fontSize=12,leading=16,textColor=GREEN,spaceBefore=12,spaceAfter=5),
    'body':ParagraphStyle('body',fontName='Helvetica',fontSize=11,leading=15,textColor=colors.HexColor('#26392f'),spaceAfter=7),
    'small':ParagraphStyle('small',fontName='Helvetica',fontSize=9,leading=12,textColor=colors.HexColor('#526359'),spaceAfter=5),
}


def p(text,style='body'):
    return Paragraph(text,styles[style])


def page(canvas,doc):
    canvas.setFillColor(colors.HexColor('#f8f5ed'));canvas.rect(0,0,*A4,fill=1,stroke=0)
    canvas.setStrokeColor(GREEN);canvas.setLineWidth(2);canvas.line(42,A4[1]-32,A4[0]-42,A4[1]-32)
    canvas.setFont('Helvetica',8);canvas.setFillColor(GREEN)
    canvas.drawString(42,27,'FoliaTrace | v0.8 | 12 November 2026')
    canvas.drawRightString(A4[0]-42,27,'Product overview / 1')


def main():
    m=release_metrics(); h=m['held']; c=h['counts']; kb=m['kb']; hold=m['hold']
    TARGET.parent.mkdir(parents=True,exist_ok=True)
    story=[p('FoliaTrace','title'),p('Bring the bottle. Start a better conversation.','lead'),
        p('A person, a bottle, a medicine they cannot name','heading'),
        p('Mei, 72, takes warfarin and "her water pill". Her daughter buys a memory-and-joint supplement. FoliaTrace uses on-device language AI to suggest the medicine group and neural research ranking to find studies for her bottle and goals. Her pharmacist receives exact NIH warnings, visible uncertainties and source links.'),
        p('Three steps, one brief','heading')]
    cells=[]
    for title,text in [('1. The bottle','Search NIH-listed U.S. labels, paste ingredients, or photograph the panel and check the recognised text.'),('2. Your words','Enter a medicine name or describe it. AI proposes a group, or abstains. Add the purpose of your supplement.'),('3. The brief','NIH quotations, questions, label gaps and goal-specific research. QR handoff; print, download or choose what to share.')]:
        cells.append([p(title,'heading'),p(text)])
    table=Table([[cells[0],cells[1],cells[2]]],colWidths=[170,170,170]);table.setStyle(TableStyle([('VALIGN',(0,0),(-1,-1),'TOP'),('BACKGROUND',(0,0),(-1,-1),colors.HexColor('#eaf0e8')),('BOX',(0,0),(-1,-1),.5,colors.HexColor('#bccdbd')),('LEFTPADDING',(0,0),(-1,-1),10),('RIGHTPADDING',(0,0),(-1,-1),10),('TOPPADDING',(0,0),(-1,-1),4),('BOTTOMPADDING',(0,0),(-1,-1),10)]));story += [table,
        p('AI does useful work; sources keep it accountable','heading'),
        p(f'MiniLM understands descriptions such as "my cholesterol pill"; inferred groups remain labelled AI and need exact-name confirmation. A cross-encoder ranks {m["library"]} open-access abstracts for each herb and each goal. Expand a lead to compare literal population, preparation and comparator wording with the bottle. Low-scoring results stay empty. "Ginseng" and other ambiguous species stay unresolved.'),
```

```
48        p('Evidence you can check','heading'),
49        p(f'<b>Language AI:</b> {c["correct_group"]}/{h["drug_phrases"]} medicine descriptions assigned correctly; <b>{c["wrong_group"]} wrong group assignments</b>; {c["correctly_unassigned"]} other/negated phrases unassigned ({h["phrases"]} fresh synthetic English phrases). Browser and Python decisions agree. <b>Labels:</b> {hold["precision_exact_species"]:.1%} exact-species precision on {hold["labels_fetched"]} unseen NIH labels. <b>Verification:</b> {m["public_checks"]} browser checks on each of the public build and local checker; {m["unit"]["passed"]} behavioral tests. Zero CSP and WCAG A/AA violations in these runs.'),
50        p('One rule: every warning is NIH\'s own words','heading'),
51        p(f'{kb["unique_verified_passages"]} NIH passages verified word-for-word; no generative model writes a warning. Proprietary blends, duplicates and unknowns remain visible. Health inputs and photos stay in the browser; the QR brief travels inside the URL fragment. Free, no account, no paid inference service.'),
52        p('Try: foliatrace-research.civil-bell-5511.chatgpt.site  |  Full local app: scripts/start.ps1','heading'),
53        p(f'Prepares a conversation; does not replace a pharmacist, give doses or certify safety. Covers {kb["herbs"]} herbs. Synthetic language tests and search-ranked labels; research relevance is development-calibrated. No pharmacist review, user study or clinical validation. Fictional examples. NIH quotations public domain; article rights retained. Codex/Claude assisted development; prior EthnoHack work disclosed. MIT code.','small')]
54    SimpleDocTemplate(str(TARGET),pagesize=A4,rightMargin=42,leftMargin=42,topMargin=49,bottomMargin=40,title='FoliaTrace - supplement label to pharmacist brief',author='FoliaTrace').build(story,onFirstPage=page,onLaterPages=page)
55    from pypdf import PdfReader
56    assert len(PdfReader(TARGET).pages)==1,'Product overview must be one page'
57    print(TARGET)
58
59
60 if __name__=='__main__':main()
```

scripts/build_safety_knowledge.py
SHA-256: ceafc6e3c0537294b40e4548e6b0ce7dd54b34529df17eea7c13b983beaa8c6f

```
"""Build FoliaTrace's NIH-sourced botanical safety knowledge base.

Every quotation below is checked against the downloaded NCCIH page text before
it is written. A quote that is not found verbatim stops the build. NCCIH fact
sheets state that they are not copyrighted and are in the public domain.

Usage:
  python scripts/build_safety_knowledge.py            # download pages, verify, write
  python scripts/build_safety_knowledge.py --offline  # use .runtime/nccih/*.html
"""
from __future__ import annotations

import hashlib
import html
import json
import re
import sys
import urllib.request
from datetime import date
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
CACHE = ROOT / '.runtime/nccih'
OUT = ROOT / 'frontend/demo/safety-knowledge.json'
REPORT = ROOT / 'reports/safety_knowledge_verification.json'
BASE = 'https://www.nccih.nih.gov/health/'

# ---------------------------------------------------------------- medicines
# Each entry: canonical name, aliases (generic + common US brands), classes.
# Classes are only those that an NCCIH sentence below names (directly or as a
# named group such as "sedatives" or "medicines for diabetes").
MEDS = [
    # blood thinners
    ('warfarin', ['coumadin', 'jantoven'], ['warfarin', 'anticoagulant', 'bleeding']),
    ('apixaban', ['eliquis'], ['anticoagulant', 'bleeding']),
    ('rivaroxaban', ['xarelto'], ['anticoagulant', 'bleeding']),
    ('dabigatran', ['pradaxa'], ['anticoagulant', 'bleeding']),
    ('edoxaban', ['savaysa'], ['anticoagulant', 'bleeding']),
    ('heparin', [], ['anticoagulant', 'bleeding']),
    ('enoxaparin', ['lovenox'], ['anticoagulant', 'bleeding']),
    ('aspirin', ['acetylsalicylic acid', 'baby aspirin', 'ecotrin', 'bayer aspirin'], ['aspirin', 'bleeding']),
    ('clopidogrel', ['plavix'], ['bleeding']),
    ('prasugrel', ['effient'], ['bleeding']),
    ('ticagrelor', ['brilinta'], ['bleeding']),
    # contraception
    ('birth control pill', ['the pill', 'oral contraceptive', 'combined pill', 'ethinyl estradiol', 'levonorgestrel',
                            'norethindrone', 'norgestimate', 'desogestrel', 'drospirenone', 'yaz', 'yasmin',
                            'loestrin', 'sprintec', 'ortho tri-cyclen', 'slynd', 'seasonique', 'lo loestrin'],
     ['oral_contraceptive']),
    # transplant / immune
    ('cyclosporine', ['ciclosporin', 'neoral', 'sandimmune', 'gengraf'], ['cyclosporine', 'immunosuppressant']),
    ('tacrolimus', ['prograf', 'envarsus', 'astagraf'], ['immunosuppressant']),
    ('mycophenolate', ['cellcept', 'myfortic'], ['immunosuppressant']),
    ('azathioprine', ['imuran'], ['immunosuppressant']),
    ('sirolimus', ['rapamune'], ['immunosuppressant']),
```

```
    ('everolimus', ['zortress'], ['immunosuppressant']),
    # antidepressants / serotonin
    ('amitriptyline', ['elavil'], ['sjw_antidepressant', 'tca', 'serotonergic']),
    ('bupropion', ['wellbutrin', 'zyban'], ['sjw_antidepressant']),
    ('nortriptyline', ['pamelor'], ['tca', 'serotonergic']),
    ('imipramine', ['tofranil'], ['tca', 'serotonergic']),
    ('doxepin', ['silenor'], ['tca', 'serotonergic']),
    ('clomipramine', ['anafranil'], ['tca', 'serotonergic']),
    ('desipramine', ['norpramin'], ['tca', 'serotonergic']),
    ('sertraline', ['zoloft'], ['serotonergic']),
    ('fluoxetine', ['prozac'], ['serotonergic']),
    ('escitalopram', ['lexapro'], ['serotonergic']),
    ('citalopram', ['celexa'], ['serotonergic']),
    ('paroxetine', ['paxil'], ['serotonergic']),
    ('fluvoxamine', ['luvox'], ['serotonergic']),
    ('venlafaxine', ['effexor'], ['serotonergic']),
    ('desvenlafaxine', ['pristiq'], ['serotonergic']),
    ('duloxetine', ['cymbalta'], ['serotonergic']),
    ('trazodone', ['desyrel'], ['serotonergic']),
    ('vortioxetine', ['trintellix'], ['serotonergic']),
    ('tramadol', ['ultram'], ['serotonergic']),
    ('sumatriptan', ['imitrex'], ['serotonergic']),
    ('rizatriptan', ['maxalt'], ['serotonergic']),
    ('phenelzine', ['nardil'], ['maoi', 'serotonergic']),
    ('tranylcypromine', ['parnate'], ['maoi', 'serotonergic']),
    ('isocarboxazid', ['marplan'], ['maoi', 'serotonergic']),
    ('selegiline', ['emsam', 'eldepryl'], ['maoi', 'serotonergic']),
    # seizures
    ('phenytoin', ['dilantin'], ['sjw_anticonvulsant', 'anticonvulsant']),
    ('carbamazepine', ['tegretol'], ['sjw_anticonvulsant', 'anticonvulsant']),
    ('levetiracetam', ['keppra'], ['anticonvulsant']),
    ('lamotrigine', ['lamictal'], ['anticonvulsant']),
    ('valproate', ['valproic acid', 'divalproex', 'depakote'], ['anticonvulsant']),
    ('topiramate', ['topamax'], ['anticonvulsant']),
    ('oxcarbazepine', ['trileptal'], ['anticonvulsant']),
    ('lacosamide', ['vimpat'], ['anticonvulsant']),
    ('phenobarbital', [], ['anticonvulsant', 'sedative']),
    # heart
    ('digoxin', ['lanoxin'], ['digoxin']),
    ('ivabradine', ['corlanor'], ['ivabradine']),
    # HIV
    ('indinavir', ['crixivan'], ['hiv']),
    ('nevirapine', ['viramune'], ['hiv']),
    ('ritonavir', ['norvir'], ['hiv']),
    ('darunavir', ['prezista'], ['hiv']),
    ('atazanavir', ['reyataz'], ['hiv']),
    ('efavirenz', ['sustiva'], ['hiv']),
    ('rilpivirine', ['edurant'], ['hiv']),
    ('lopinavir', ['kaletra'], ['hiv']),
    # cancer
    ('irinotecan', ['camptosar'], ['sjw_cancer']),
    ('imatinib', ['gleevec'], ['sjw_cancer']),
    ('docetaxel', ['taxotere'], ['sjw_cancer']),
    # cholesterol
    ('simvastatin', ['zocor'], ['statin', 'simvastatin']),
    ('atorvastatin', ['lipitor'], ['statin', 'atorvastatin']),
    ('rosuvastatin', ['crestor'], ['statin']),
    ('pravastatin', ['pravachol'], ['statin']),
```

```
    ('lovastatin', ['mevacor', 'altoprev'], ['statin']),
    ('pitavastatin', ['livalo'], ['statin']),
    ('fluvastatin', ['lescol'], ['statin']),
    # diabetes
    ('metformin', ['glucophage', 'glumetza', 'fortamet'], ['metformin', 'diabetes_med']),
    ('insulin', ['lantus', 'humalog', 'novolog', 'levemir', 'tresiba', 'basaglar', 'humulin', 'novolin', 'toujeo'], ['diabetes_med']),
    ('glipizide', ['glucotrol'], ['diabetes_med']),
    ('glyburide', ['glynase', 'diabeta'], ['diabetes_med']),
    ('glimepiride', ['amaryl'], ['diabetes_med']),
    ('sitagliptin', ['januvia'], ['diabetes_med']),
    ('linagliptin', ['tradjenta'], ['diabetes_med']),
    ('empagliflozin', ['jardiance'], ['diabetes_med']),
    ('dapagliflozin', ['farxiga'], ['diabetes_med']),
    ('canagliflozin', ['invokana'], ['diabetes_med']),
    ('semaglutide', ['ozempic', 'rybelsus', 'wegovy'], ['diabetes_med']),
    ('liraglutide', ['victoza'], ['diabetes_med']),
    ('dulaglutide', ['trulicity'], ['diabetes_med']),
    ('tirzepatide', ['mounjaro', 'zepbound'], ['diabetes_med']),
    ('pioglitazone', ['actos'], ['diabetes_med']),
    # blood pressure
    ('lisinopril', ['zestril', 'prinivil'], ['bp_med']),
    ('enalapril', ['vasotec'], ['bp_med']),
    ('ramipril', ['altace'], ['bp_med']),
    ('benazepril', ['lotensin'], ['bp_med']),
    ('losartan', ['cozaar'], ['bp_med']),
    ('valsartan', ['diovan'], ['bp_med']),
    ('olmesartan', ['benicar'], ['bp_med']),
    ('irbesartan', ['avapro'], ['bp_med']),
    ('telmisartan', ['micardis'], ['bp_med']),
    ('amlodipine', ['norvasc'], ['bp_med']),
    ('nifedipine', ['procardia', 'adalat'], ['bp_med']),
    ('diltiazem', ['cardizem'], ['bp_med']),
    ('verapamil', ['calan'], ['bp_med']),
    ('metoprolol', ['lopressor', 'toprol'], ['bp_med']),
    ('atenolol', ['tenormin'], ['bp_med']),
    ('carvedilol', ['coreg'], ['bp_med']),
    ('propranolol', ['inderal'], ['bp_med']),
    ('bisoprolol', ['zebeta'], ['bp_med']),
    ('nadolol', ['corgard'], ['bp_med', 'nadolol']),
    ('hydrochlorothiazide', ['hctz', 'microzide'], ['bp_med']),
    ('chlorthalidone', ['thalitone'], ['bp_med']),
    ('furosemide', ['lasix'], ['bp_med']),
    ('spironolactone', ['aldactone'], ['bp_med']),
    ('clonidine', ['catapres'], ['bp_med']),
    ('hydralazine', [], ['bp_med']),
    # steroids
    ('prednisone', ['deltasone', 'rayos'], ['corticosteroid']),
    ('prednisolone', ['orapred', 'millipred'], ['corticosteroid']),
    ('methylprednisolone', ['medrol', 'solu-medrol'], ['corticosteroid']),
    ('dexamethasone', ['decadron'], ['corticosteroid']),
    ('hydrocortisone', ['cortef'], ['corticosteroid']),
    ('fludrocortisone', ['florinef'], ['corticosteroid']),
    ('budesonide', ['entocort', 'uceris'], ['corticosteroid']),
    # sedatives
    ('alprazolam', ['xanax'], ['benzodiazepine', 'sedative']),
    ('lorazepam', ['ativan'], ['benzodiazepine', 'sedative']),
    ('diazepam', ['valium'], ['benzodiazepine', 'sedative']),
    ('clonazepam', ['klonopin'], ['benzodiazepine', 'sedative', 'anticonvulsant']),
```

```
    ('temazepam', ['restoril'], ['benzodiazepine', 'sedative']),
    ('triazolam', ['halcion'], ['benzodiazepine', 'sedative']),
    ('chlordiazepoxide', ['librium'], ['benzodiazepine', 'sedative']),
    ('zolpidem', ['ambien', 'edluar', 'intermezzo'], ['sedative']),
    ('eszopiclone', ['lunesta'], ['sedative']),
    ('zaleplon', ['sonata'], ['sedative']),
    ('suvorexant', ['belsomra'], ['sedative']),
    # thyroid / bone
    ('levothyroxine', ['synthroid', 'levoxyl', 'unithroid', 'euthyrox', 'tirosint'], ['thyroid_hormone']),
    ('liothyronine', ['cytomel'], ['thyroid_hormone']),
    ('thyroid desiccated', ['armour thyroid', 'np thyroid'], ['thyroid_hormone']),
    ('raloxifene', ['evista'], ['raloxifene']),
]

CLASS_LABELS = {
    'warfarin': 'warfarin', 'anticoagulant': 'an anticoagulant (blood thinner)',
    'bleeding': 'a medicine that affects bleeding', 'aspirin': 'aspirin',
    'oral_contraceptive': 'birth control pills', 'cyclosporine': 'cyclosporine',
    'immunosuppressant': 'an immunosuppressant', 'sjw_antidepressant': 'an antidepressant NIH names',
    'serotonergic': 'a medicine that affects serotonin', 'tca': 'a tricyclic antidepressant',
    'maoi': 'an MAO inhibitor antidepressant', 'sjw_anticonvulsant': 'a seizure medicine NIH names',
    'anticonvulsant': 'an anti-seizure medicine', 'digoxin': 'digoxin', 'ivabradine': 'ivabradine',
    'hiv': 'an HIV medicine', 'sjw_cancer': 'a cancer medicine NIH names', 'statin': 'a statin',
    'simvastatin': 'simvastatin', 'atorvastatin': 'atorvastatin', 'metformin': 'metformin',
    'diabetes_med': 'a diabetes medicine', 'bp_med': 'a blood-pressure medicine', 'nadolol': 'nadolol',
    'corticosteroid': 'a corticosteroid', 'benzodiazepine': 'a benzodiazepine', 'sedative': 'a sedative',
    'thyroid_hormone': 'a thyroid hormone medicine', 'raloxifene': 'raloxifene',
}

CONDITIONS = {
    'pregnancy': 'Pregnant, trying to conceive or breastfeeding',
    'surgery': 'Surgery or a procedure planned',
    'diabetes': 'Diabetes or blood-sugar problems',
    'heart_bp_kidney': 'High blood pressure, heart or kidney condition',
    'liver': 'Liver condition',
    'autoimmune_thyroid': 'Autoimmune or thyroid condition',
    'ragweed': 'Allergic to ragweed, daisies or marigolds',
    'hormone_sensitive': 'Hormone-sensitive condition (e.g. breast, uterine or prostate cancer)',
}

# ---------------------------------------------------------------------- herbs
# level: 'discuss' = NIH names an interaction with this medicine/group;
#        'caution' = NIH describes conflicting/theoretical evidence or a
#                    condition-specific concern;
#        'note'    = product-quality or label caution.
# 'meds' lists class ids. 'conditions' lists condition ids.
HERBS = [
    dict(id='st-johns-wort', name="St. John's wort", page='st-johns-wort',
         aliases=["st john's wort", 'st johns wort', 'saint johns wort', "saint john's wort", 'st. john', 'hypericum', 'hypericum perforatum'],
         rules=[
             dict(level='discuss', meds=['sjw_antidepressant', 'oral_contraceptive', 'cyclosporine', 'sjw_anticonvulsant', 'digoxin', 'ivabradine', 'hiv',
 'sjw_cancer', 'warfarin', 'simvastatin'],
                  quote="St. John\u2019s wort can weaken the effects of many medicines, including crucially important medicines such as:",
                  items={'sjw_antidepressant': 'Some antidepressants, including amitriptyline and bupropion',
                         'oral_contraceptive': 'Birth control pills',
                         'cyclosporine': 'Cyclosporine, which prevents the body from rejecting transplanted organs',
                         'sjw_anticonvulsant': 'Some drugs used to prevent seizures, including phenytoin and carbamazepine',
                         'digoxin': 'Some heart medications, including digoxin and ivabradine',
```

```
                        'ivabradine': 'Some heart medications, including digoxin and ivabradine',
                        'hiv': 'Some HIV drugs, including indinavir and nevirapine',
                        'sjw_cancer': 'Some cancer medications, including irinotecan, imatinib, and docetaxel',
                        'warfarin': 'Warfarin, an anticoagulant (blood thinner)',
                        'simvastatin': 'Certain statins, including simvastatin'},
                 effect='may weaken the medicine'),
            dict(level='discuss', meds=['serotonergic'],
                 quote="taking St. John\u2019s wort with certain antidepressants or other drugs that affect serotonin (a substance produced by some nerve cells) may lead to increased serotonin-related side effects, which can be serious.",
                 effect='serotonin-related side effects'),
            dict(level='discuss', always=True,
                 quote="St. John\u2019s wort can interact in dangerous, sometimes life-threatening ways with a variety of medicines.",
                 effect='interacts with many medicines'),
            dict(level='caution', conditions=['pregnancy'],
                 quote="It may be unsafe to use St. John\u2019s wort during pregnancy because it may increase the risk of birth defects."),
        ]),
    dict(id='ginkgo', name='Ginkgo', page='ginkgo', aliases=['ginkgo', 'ginkgo biloba', 'gingko', 'maidenhair tree'],
         rules=[
            dict(level='discuss', meds=['anticoagulant'],
                 quote='Ginkgo may increase the risk of bleeding in people who are taking anticoagulant drugs, such as warfarin.',
                 effect='bleeding risk'),
            dict(level='caution', conditions=['pregnancy'],
                 quote='It might cause early labor or extra bleeding during delivery if used near that time.'),
        ]),
    dict(id='goldenseal', name='Goldenseal', page='goldenseal', aliases=['goldenseal', 'golden seal', 'hydrastis', 'hydrastis canadensis'],
         rules=[
            dict(level='discuss', meds=['metformin'],
                 quote='For example, a study funded by the National Center for Complementary and Integrative Health (NCCIH) found that levels of metformin\u2014the most commonly prescribed diabetes medicine\u2014decreased about 25 percent in healthy adults who were given goldenseal extract plus metformin. This drop was enough to potentially hinder glucose control in people with type 2 diabetes taking metformin.',
                 effect='may lower metformin levels'),
            dict(level='discuss', always=True, quote='Goldenseal may change the way your body processes many medications.',
                 effect='changes how many medicines are processed'),
            dict(level='caution', conditions=['pregnancy'],
                 quote='Women who are pregnant or breastfeeding should not use goldenseal, and it should not be given to infants.'),
            dict(level='note', quote='Some commercial goldenseal products have been found to contain other herbs or ingredients not listed on the label in addition to or instead of goldenseal.'),
        ]),
    dict(id='red-yeast-rice', name='Red yeast rice', page='red-yeast-rice',
         aliases=['red yeast rice', 'monascus', 'monascus purpureus', 'red rice yeast', 'red mold rice', 'monacolin'],
         identity_note='Not a plant: a mold (Monascus purpureus) fermented on rice.',
         rules=[
            dict(level='discuss', meds=['statin'],
                 quote='Monacolin K is structurally identical to the medicine lovastatin.',
                 effect='same compound as a statin drug'),
            dict(level='discuss', meds=['statin'],
                 quote='Red yeast rice products can have the same types of drug interactions as statin drugs and therefore may interfere with certain medicines or increase the chance for side effects.',
                 effect='statin-type interactions'),
            dict(level='caution', conditions=['pregnancy'],
                 quote='Red yeast rice products are not recommended for those who are pregnant or lactating.'),
            dict(level='note', quote='Levels of monacolin K and lovastatin are not usually included on product labels.'),
        ]),
    dict(id='licorice', name='Licorice root', page='licorice-root',
         aliases=['licorice', 'liquorice', 'licorice root', 'glycyrrhiza', 'glycyrrhiza glabra', 'glycyrrhiza uralensis', 'glycyrrhizin'],
         variants=[dict(pattern=r'deglycyrrhizinated|\bdgl\b', label='Deglycyrrhizinated (DGL)',
                        quote='Oral licorice products that do not contain glycyrrhizin (deglycyrrhizinated licorice) might be safe for up to 4 months.')],
         rules=[
```

```
282             dict(level='discuss', meds=['corticosteroid'], quote='Interactions between licorice and corticosteroids have been reported.',
283                  effect='reported interaction'),
284             dict(level='caution', conditions=['heart_bp_kidney'], meds=['bp_med'],
285                  quote='Even small amounts of glycyrrhizin from licorice root products have been linked to severe adverse effects in people who consume a lot of
    salt, in people with hypertension, and in people with heart or kidney conditions.'),
286         ]),
287     dict(id='garlic', name='Garlic', page='garlic', aliases=['garlic', 'allium sativum', 'aged garlic', 'allicin'],
288         rules=[
289             dict(level='discuss', meds=['bleeding'],
290                  quote='Taking garlic supplements may increase the risk of bleeding. If you take garlic supplements, make sure to tell your health care providers.
    This is especially important if you are going to have surgery or if you take medicines, such as anticoagulants or aspirin, that may also affect bleeding.',
291                  effect='bleeding risk'),
292             dict(level='caution', conditions=['surgery'], quote='Taking garlic supplements may increase the risk of bleeding.'),
293         ]),
294     dict(id='kava', name='Kava', page='kava', aliases=['kava', 'kava kava', 'piper methysticum', 'kavalactones', 'kavalactone'],
295         rules=[
296             dict(level='discuss', meds=['benzodiazepine', 'sedative'],
297                  quote='Kava should not be used together with other substances that have sedative effects, such as benzodiazepines or alcohol.',
298                  effect='added sedation'),
299             dict(level='caution', always=True, conditions=['liver'],
300                  quote='Various kava products have been linked to rare cases of liver injury, some of which have been serious or even fatal.'),
301         ]),
302     dict(id='green-tea', name='Green tea extract', page='green-tea',
303         aliases=['green tea', 'green tea extract', 'camellia sinensis', 'egcg', 'epigallocatechin gallate', 'epigallocatechin'],
304         rules=[
305             dict(level='discuss', meds=['nadolol'],
306                  quote='Green tea at high doses has been shown to reduce blood levels and, therefore, the effectiveness of the drug nadolol, a beta-blocker used for
     high blood pressure and heart problems.',
307                  effect='may lower nadolol levels'),
308             dict(level='discuss', meds=['atorvastatin'], quote='Green tea extract can reduce blood levels of the cholesterol-lowering drug atorvastatin.',
309                  effect='may lower atorvastatin levels'),
310             dict(level='discuss', meds=['raloxifene'],
311                  quote='A study funded by the National Center for Complementary and Integrative Health showed an interaction between green tea and the drug
    raloxifene, which is used to treat osteoporosis.',
312                  effect='reported interaction'),
313             dict(level='caution', conditions=['liver'],
314                  quote='Although uncommon, liver injury has been reported in some people who used green tea products, primarily green tea extracts in tablet or
    capsule form.'),
315         ]),
316     dict(id='asian-ginseng', name='Asian ginseng', page='asian-ginseng',
317         aliases=['asian ginseng', 'panax ginseng', 'korean ginseng', 'chinese ginseng', 'red ginseng', 'white ginseng', 'korean red ginseng'],
318         rules=[
319             dict(level='caution', conditions=['diabetes'], meds=['diabetes_med'],
320                  quote='If you have diabetes, consult your health care provider before using Asian ginseng.'),
321             dict(level='caution', meds=['anticoagulant'], conditions=['autoimmune_thyroid'],
322                  quote='Asian ginseng may worsen autoimmune disorders and interfere with blood clotting.'),
323             dict(level='caution', conditions=['pregnancy'],
324                  quote='Some studies of Asian ginseng suggest it may be unsafe when taken orally during pregnancy.'),
325         ]),
326     dict(id='turmeric', name='Turmeric / curcumin', page='turmeric',
327         aliases=['turmeric', 'curcumin', 'curcuma longa', 'curcuminoids', 'curcuminoid'],
328         variants=[dict(pattern=r'piperine|bioperine|black pepper|piper nigrum|phytosome|meriva|longvida|theracurmin|enhanced absorption|high
    absorption|bioavailab|nano',
329                        label='Absorption-enhanced formulation',
330                        quote='Highly bioavailable formulations of curcumin, which enhance the body\u2019s ability to absorb the curcumin, may harm your liver.',
331                        level='caution')],
332         rules=[
333             dict(level='caution', conditions=['liver'],
```

```
                    quote='Many curcumin products with increased bioavailability are on the market, and liver damage has been reported in some people who have consumed these bioavailable formulations.'),
                dict(level='caution', conditions=['pregnancy'], quote='The use of turmeric supplements during pregnancy may be unsafe.'),
            ]),
        dict(id='valerian', name='Valerian', page='valerian', aliases=['valerian', 'valeriana officinalis', 'valerian root'],
            rules=[
                dict(level='discuss', meds=['sedative'],
                    quote='Because it is possible (though not proven) that valerian might have a sleep-inducing effect, it should not be taken along with alcohol or sedatives.',
                    effect='added sedation'),
            ]),
        dict(id='ashwagandha', name='Ashwagandha', page='ashwagandha',
            aliases=['ashwagandha', 'withania', 'withania somnifera', 'indian ginseng', 'winter cherry'],
            rules=[
                dict(level='discuss', meds=['diabetes_med', 'bp_med', 'immunosuppressant', 'sedative', 'anticonvulsant', 'thyroid_hormone'],
                    quote='There is evidence that ashwagandha might interact with some medications, including those for diabetes and high blood pressure, medicines that decrease the immune system response (immunosuppressants), sedatives, anti-seizure medications (anticonvulsants), and thyroid hormone medications.',
                    effect='may interact'),
                dict(level='caution', conditions=['surgery', 'autoimmune_thyroid'],
                    quote='Ashwagandha is not recommended for people who are about to have surgery, or for those who have autoimmune or thyroid disorders.'),
                dict(level='caution', conditions=['pregnancy'], quote='Ashwagandha should be avoided during pregnancy and should not be used while breastfeeding.'),
                dict(level='caution', conditions=['liver'], quote='Although it is rare, there have been a number of cases that link liver injury to ashwagandha supplements.'),
            ]),
        dict(id='echinacea', name='Echinacea', page='echinacea',
            aliases=['echinacea', 'echinacea purpurea', 'echinacea angustifolia', 'echinacea pallida', 'purple coneflower'],
            rules=[
                dict(level='caution', meds=['immunosuppressant'],
                    quote='There is conflicting evidence about whether echinacea interacts with some drugs metabolized by the liver, and there are theoretical reasons to suspect that echinacea might interact with immunosuppressants or caffeine.'),
                dict(level='caution', conditions=['ragweed'], quote='However, some people have allergic reactions to echinacea, which may be severe.'),
            ]),
        dict(id='black-cohosh', name='Black cohosh', page='black-cohosh',
            aliases=['black cohosh', 'actaea racemosa', 'cimicifuga', 'cimicifuga racemosa', 'black snakeroot'],
            rules=[
                dict(level='caution', conditions=['liver'],
                    quote='Cases of liver damage\u2014some very serious\u2014have been reported in people taking products labeled as black cohosh.'),
                dict(level='caution', conditions=['pregnancy'], quote='Black cohosh may not be safe for use during pregnancy or while breastfeeding.'),
                dict(level='note', quote='Some commercial black cohosh products have been found to contain the wrong herb or to contain mixtures of black cohosh and other herbs that are not listed on the label.'),
            ]),
        dict(id='bitter-orange', name='Bitter orange', page='bitter-orange',
            aliases=['bitter orange', 'citrus aurantium', 'synephrine', 'p-synephrine', 'seville orange'],
            rules=[
                dict(level='caution', conditions=['heart_bp_kidney'], meds=['bp_med'],
                    quote='Cases of serious medical events, including abnormal heart rhythms, heart attacks, and strokes, have been reported in people who had taken bitter orange products, but most of the products contained multiple ingredients rather than bitter orange alone.'),
                dict(level='caution', conditions=['pregnancy'],
                    quote='It may not be safe to use bitter orange during pregnancy or while breastfeeding, and data from animal experiments suggest that substances in bitter orange might decrease milk production.'),
                dict(level='note', quote='Of 23 products with labels that stated the amount of synephrine in the product, only 5 contained amounts of synephrine close to that indicated on the label (between 80 and 120 percent of the stated amount).'),
            ]),
        dict(id='yohimbe', name='Yohimbe', page='yohimbe', aliases=['yohimbe', 'yohimbine', 'pausinystalia', 'pausinystalia johimbe', 'corynanthe johimbe'],
            rules=[
                dict(level='discuss', meds=['maoi', 'tca'],
                    quote='Do not use yohimbe if you are taking types of antidepressant medications called monoamine oxidase inhibitors or tricyclic antidepressants.',
                    effect='do not combine'),
```

```
383             dict(level='caution', always=True, conditions=['heart_bp_kidney'],
384                  quote='Yohimbine, a component of yohimbe, has been associated with cardiac arrhythmia (irregular heartbeat), blood pressure problems, heart attacks, and seizures.'),
385             dict(level='caution', conditions=['pregnancy'], quote='It might be unsafe to use yohimbe orally (taken by mouth) during pregnancy or while breastfeeding.'),
386         ]),
387     dict(id='milk-thistle', name='Milk thistle', page='milk-thistle', aliases=['milk thistle', 'silybum', 'silybum marianum', 'silymarin', 'silibinin'],
388         rules=[
389             dict(level='caution', conditions=['ragweed'],
390                  quote='Milk thistle may cause allergic reactions, particularly among people who are allergic to related plants (for example, ragweed, chrysanthemum, marigold, and daisy).'),
391             dict(level='note', quote='Some products have been found to contain amounts of silymarin substantially different from what\u2019s stated on the label or to be contaminated with pesticides, microorganisms, or mycotoxins (harmful substances produced by molds or other fungi).'),
392         ]),
393     dict(id='ginger', name='Ginger', page='ginger', aliases=['ginger', 'ginger root', 'zingiber', 'zingiber officinale'],
394         rules=[]),
395     dict(id='chamomile', name='Chamomile', page='chamomile',
396         aliases=['chamomile', 'camomile', 'german chamomile', 'matricaria', 'matricaria recutita', 'matricaria chamomilla', 'chamomilla recutita'],
397         rules=[
398             dict(level='discuss', meds=['warfarin'],
399                  quote='Interactions between chamomile and some drugs metabolized by the liver and warfarin (a blood thinner) have been reported, and there are theoretical reasons to suspect that chamomile might interact with other drugs as well, such as sedatives.',
400                  effect='reported interaction'),
401             dict(level='caution', meds=['oral_contraceptive'],
402                  quote='Preliminary studies suggest that taking chamomile along with birth control pills might decrease the effects of birth control pills.'),
403             dict(level='caution', meds=['sedative'],
404                  quote='Interactions between chamomile and some drugs metabolized by the liver and warfarin (a blood thinner) have been reported, and there are theoretical reasons to suspect that chamomile might interact with other drugs as well, such as sedatives.'),
405             dict(level='caution', conditions=['ragweed'],
406                  quote='People are more likely to experience allergic reactions to chamomile if they\u2019re allergic to related plants such as ragweed, chrysanthemums, marigolds, or daisies.'),
407             dict(level='caution', conditions=['hormone_sensitive'],
408                  quote='In addition, chamomile might worsen any condition that is sensitive to exposure to estrogen, such as breast or uterine cancer.'),
409         ]),
410     dict(id='hawthorn', name='Hawthorn', page='hawthorn', aliases=['hawthorn', 'hawthorne', 'crataegus', 'tejocote', 'shanzha'],
411         variants=[dict(pattern=r'tejocote|crataegus mexicana|mexican hawthorn', label='Labeled \u201ctejocote root\u201d',
412                        quote='In January 2024, the FDA published a list of products, including dietary supplements labeled \u201ctejocote root\u201d (C. mexicana), that actually contain yellow oleander, a toxic plant known to cause harm to the nervous system, heart, stomach, and intestines.',
413                        level='discuss')],
414         rules=[
415             dict(level='caution', meds=['digoxin', 'bp_med'], conditions=['heart_bp_kidney'],
416                  quote='It\u2019s unclear why, but one possibility is that hawthorn might have interacted with drugs the patients were taking.'),
417         ]),
418     dict(id='fenugreek', name='Fenugreek', page='fenugreek', aliases=['fenugreek', 'trigonella', 'trigonella foenum-graecum'],
419         rules=[
420             dict(level='caution', meds=['diabetes_med'], conditions=['diabetes'], quote='Large doses may cause a harmful drop in blood sugar.'),
421             dict(level='caution', conditions=['pregnancy'],
422                  quote='Fenugreek is not safe for use during pregnancy in amounts greater than those found in food; its use has been linked to increased risks of birth defects in both animals and people.'),
423         ]),
424     dict(id='cranberry', name='Cranberry', page='cranberry', aliases=['cranberry', 'vaccinium macrocarpon', 'cranberry extract'],
425         rules=[
426             dict(level='caution', meds=['warfarin'],
427                  quote='There is conflicting evidence about whether cranberry interacts with the anticoagulant (blood thinner) warfarin.'),
428         ]),
429     dict(id='passionflower', name='Passionflower', page='passionflower', aliases=['passionflower', 'passion flower', 'passiflora', 'passiflora incarnata', 'maypop'],
430         rules=[
431             dict(level='caution', conditions=['surgery'],
```

```
                quote='Passionflower might interact with some anesthesia drugs, so avoid its use before surgery unless recommended by a health care provider.'),
            dict(level='caution', conditions=['pregnancy'],
                quote='Passionflower should not be used during pregnancy as it may induce uterine contractions and possibly lead to other problems.'),
        ]),
    dict(id='peppermint-oil', name='Peppermint oil', page='peppermint-oil',
        aliases=['peppermint', 'peppermint oil', 'mentha piperita', 'mentha x piperita', 'mentha \xd7 piperita'],
        rules=[
            dict(level='note', quote='Capsules containing peppermint oil are often enteric-coated to reduce the likelihood of heartburn.'),
        ]),
    dict(id='saw-palmetto', name='Saw palmetto', page='saw-palmetto', aliases=['saw palmetto', 'serenoa', 'serenoa repens', 'sabal'],
        rules=[
            dict(level='caution', conditions=['pregnancy'], quote='Saw palmetto may be unsafe for use during pregnancy or while breastfeeding.'),
        ]),
]

GENERAL_QUOTE = 'some herbs and medicines interact in harmful ways.'

# ------------------------------------------------ name ambiguity (identity)
# A bare common name that can denote different species. The checker never
# picks one; it shows each candidate and which NIH page (if any) applies.
AMBIGUOUS = [
    dict(term='ginseng', pattern=r'\bginseng\b',
        qualified=r'(asian|korean|chinese|red|white|american|siberian|indian|brazilian|peruvian)\s+ginseng|panax|eleuthero',
        candidates=[
            dict(species='Panax ginseng', common='Asian / Korean ginseng', herb='asian-ginseng'),
            dict(species='Panax quinquefolius', common='American ginseng', herb=None),
            dict(species='Eleutherococcus senticosus', common='\u201cSiberian ginseng\u201d (eleuthero), a different genus', herb=None),
        ],
        source_quote='Asian ginseng is one of several types of ginseng.', source_page='asian-ginseng'),
    dict(term='mint', pattern=r'\bmint\b', qualified=r'(pepper|spear|corn|water)\s*mint|mentha',
        candidates=[
            dict(species='Mentha \xd7 piperita', common='Peppermint', herb='peppermint-oil'),
            dict(species='Mentha spicata', common='Spearmint', herb=None),
            dict(species='Mentha arvensis', common='Corn mint', herb=None),
        ],
        source_quote=None, source_page=None),
    dict(term='chamomile', pattern=r'\bc(h)?amomile\b', qualified=r'(german|roman)\s+c(h)?amomile|matricaria|chamaemelum|anthemis|chamomilla',
        candidates=[
            dict(species='Matricaria chamomilla', common='German chamomile', herb='chamomile'),
            dict(species='Chamaemelum nobile', common='Roman chamomile', herb=None),
        ],
        source_quote='There are two types of chamomile: German chamomile and Roman chamomile.', source_page='chamomile'),
    dict(term='cohosh', pattern=r'\bcohosh\b', qualified=r'(black|blue)\s+cohosh|actaea|cimicifuga|caulophyllum',
        candidates=[
            dict(species='Actaea racemosa', common='Black cohosh', herb='black-cohosh'),
            dict(species='Caulophyllum thalictroides', common='Blue cohosh \u2014 a different herb', herb=None),
        ],
        source_quote='Blue cohosh may cause adverse effects such as high blood pressure, high blood sugar, and chest pain.', source_page='black-cohosh'),
]

# Botanical words that suggest an herb we have not verified (shown as coverage gaps).
UNVERIFIED_HINTS = ['ashitaba', 'astragalus', 'bacopa', 'boswellia', 'burdock', 'cat\'s claw', 'cayenne', 'cinnamon',
    'dandelion', 'dong quai', 'elderberry', 'eleuthero', 'evening primrose', 'feverfew', 'gotu kola', 'guarana',
    'hops', 'horny goat weed', 'horse chestnut', 'lemon balm', 'maca', 'moringa', 'motherwort', 'nettle',
    'olive leaf', 'oregano', 'pau d\'arco', 'pygeum', 'rhodiola', 'rosemary', 'schisandra', 'skullcap', 'slippery elm',
    'spirulina', 'tribulus', 'uva ursi', 'white willow', 'yarrow', 'yerba mate', 'american ginseng', 'siberian ginseng',
    'panax quinquefolius', 'eleutherococcus', 'eleutherococcus senticosus', 'caulophyllum', 'chamaemelum', 'roman chamomile',
    'mentha spicata', 'mentha arvensis', 'valeriana wallichii',
```

```
    'spearmint', 'blue cohosh', 'roman chamomile', 'berberine', 'quercetin', 'resveratrol', 'aloe', 'senna',
    'cascara', 'chasteberry', 'vitex', 'damiana', 'kratom', 'kanna', 'mucuna', 'black seed', 'nigella', 'garcinia',
    'forskolin', 'coleus', 'gymnema', 'bitter melon', 'holy basil', 'tulsi', 'shatavari', 'triphala', 'amla',
    'lion\'s mane', 'reishi', 'cordyceps', 'chaga', 'turkey tail', 'beetroot', 'grape seed', 'pine bark', 'bilberry',
    'lutein', 'marshmallow root', 'mullein', 'licorice', 'angelica', 'lavender', 'lemon verbena', 'linden',
    'pomegranate', 'sea buckthorn', 'tongkat ali', 'fadogia', 'yohimbine', 'hibiscus', 'butterbur', 'kudzu']


def norm(text: str) -> str:
    text = html.unescape(text)
    text = text.replace('\u2019', "'").replace('\u2018', "'").replace('\u201c', '"').replace('\u201d', '"')
    text = text.replace('\u2014', '-').replace('\u2013', '-').replace('\xa0', ' ')
    return re.sub(r'\s+', ' ', text).strip()


def page_text(raw: str) -> str:
    m = re.search(r'<main.*?</main>', raw, re.S)
    body = m.group(0) if m else raw
    body = re.sub(r'<(script|style)[^>]*>.*?</\1>', '', body, flags=re.S)
    body = re.sub(r'<[^>]+>', ' ', body)
    return norm(body)


def fetch(slug: str, offline: bool) -> str:
    path = CACHE / f'{slug}.html'
    if not offline:
        req = urllib.request.Request(BASE + slug, headers={'User-Agent': 'Mozilla/5.0 (FoliaTrace source verification)'})
        with urllib.request.urlopen(req, timeout=40) as r:
            CACHE.mkdir(parents=True, exist_ok=True)
            path.write_bytes(r.read())
    return path.read_text(encoding='utf-8', errors='ignore')


def names_from(text: str, label: str) -> list[str]:
    m = re.search(label + r':\s*(.*?)\s+(?:Latin Names:|Common Names:|Background\b)', text)
    if not m:
        return []
    raw = m.group(1)
    if label == 'Latin Names':
        # "Curcuma longa , synonym Curcuma domestica; ..." / "X (also known as Y )" -> separate binomials
        raw = re.sub(r'\(\s*(?:synonyms?|also known as)\s*([^)]*)\)', r', \1', raw)
        raw = re.sub(r'\b(?:synonyms?|also known as)\b', ',', raw)
        names = [n.strip(' .') for n in re.split(r'[,;]', raw)]
        return [n for n in names if re.fullmatch(r'[A-Z][a-z]+ (?:x |\xd7 )?[a-z][a-z-]+', n)]
    return [n.strip(' .') for n in re.split(r',\s*', raw) if 1 < len(n.strip()) < 60]


def main() -> None:
    offline = '--offline' in sys.argv
    today = date.today().isoformat()
    pages, checks, failures = {}, 0, []
    for herb in HERBS:
        slug = herb['page']
        raw = fetch(slug, offline)
        text = page_text(raw)
        if 'public domain' not in text:
            failures.append(f'{slug}: public-domain statement not found')
        updated = re.search(r'Last Updated:\s*([A-Za-z]+ \d{4})', text)
```

```
        title = re.search(r'og:title" content="([^"]+)"', raw)
        pages[slug] = dict(url=BASE + slug, title=norm(title.group(1)) if title else herb['name'],
                           updated=updated.group(1) if updated else None, retrieved=today,
                           text_sha256=hashlib.sha256(text.encode()).hexdigest(),
                           common_names=names_from(text, 'Common Names'), latin_names=names_from(text, 'Latin Names'))
        quotes = [r['quote'] for r in herb['rules']] + [v['quote'] for v in herb.get('variants', [])]
        quotes += [item for r in herb['rules'] for item in r.get('items', {}).values()]
        for q in quotes:
            checks += 1
            if norm(q) not in text:
                failures.append(f'{slug}: quote not found verbatim: {q[:90]}')
        pages[slug]['general_advice'] = norm(GENERAL_QUOTE) in text
    for amb in AMBIGUOUS:
        if amb['source_quote']:
            checks += 1
            if norm(amb['source_quote']) not in page_text(fetch(amb['source_page'], True)):
                failures.append(f"ambiguity {amb['term']}: quote not found")
    meds_by_alias = {}
    for name, aliases, classes in MEDS:
        for c in classes:
            if c not in CLASS_LABELS:
                failures.append(f'unknown class {c} for {name}')
        for alias in [name] + aliases:
            key = norm(alias).lower()
            if key in meds_by_alias and meds_by_alias[key] != name:
                failures.append(f'duplicate medicine alias {alias}')
            meds_by_alias[key] = name
    for herb in HERBS:
        for r in herb['rules']:
            for c in r.get('meds', []):
                if c not in CLASS_LABELS:
                    failures.append(f"{herb['id']}: unknown class {c}")
            for c in r.get('conditions', []):
                if c not in CONDITIONS:
                    failures.append(f"{herb['id']}: unknown condition {c}")
    sys.path.insert(0, str(ROOT))
    from scripts.ai_medicine_groups import GROUPS as AI_GROUPS, THRESHOLD as AI_THRESHOLD, MARGIN as AI_MARGIN
    for g in AI_GROUPS:
        for c in g['classes']:
            if c not in CLASS_LABELS:
                failures.append(f"AI medicine group {g['id']}: unknown class {c}")
    if failures:
        print('\n'.join(failures))
        raise SystemExit(f'{len(failures)} verification failure(s); nothing written.')
    herbs_out = []
    for herb in HERBS:
        p = pages[herb['page']]
        alias = sorted({norm(a).lower() for a in herb['aliases'] + p['common_names'] + p['latin_names'] if len(a) > 2}, key=len, reverse=True)
        # Generic words that would over-match are kept out of automatic aliases.
        # NCCIH common names that other species also use ('bearberry' is often uva ursi, 'holy thistle'
        # often blessed thistle, 'golden root' rhodiola) are excluded rather than over-matched.
        alias = [a for a in alias if a not in {'ginseng root', 'ginseng', 'chamomile', 'mint', 'orange', 'tea', 'root',
                                               'bearberry', 'holy thistle', 'yellow root', 'goldenroot', 'coneflower',
                                               'awa', 'kawa', 'sweet root', 'water lemon', 'all-heal'}]
        herbs_out.append(dict(id=herb['id'], name=herb['name'], aliases=alias, latin=p['latin_names'],
                              identity_note=herb.get('identity_note'), source=herb['page'],
                              variants=herb.get('variants', []), rules=herb['rules']))
    passages = {norm(q) for h in HERBS for q in [r['quote'] for r in h['rules']] + [v['quote'] for v in h.get('variants', [])]
```

```
                + [i for r in h['rules'] for i in r.get('items', {}).values()]} | {norm(a['source_quote']) for a in AMBIGUOUS if a['source_quote']}
    payload = dict(
        version='2026.11.12', built=today, verified_passages=len(passages),
        scope='Statements quoted verbatim from NCCIH \u201cUsefulness and Safety\u201d fact sheets (U.S. National Institutes of Health; public domain). '
              'Only medicines or groups that a quoted sentence names are matched. Not finding a statement does not mean a combination is safe.',
        general_quote=GENERAL_QUOTE,
        sources=pages,
        conditions=CONDITIONS,
        class_labels=CLASS_LABELS,
        medicines=[dict(name=n, aliases=a, classes=c) for n, a, c in MEDS],
        herbs=herbs_out,
        ambiguous=AMBIGUOUS,
        ai_medicine_groups=dict(threshold=AI_THRESHOLD, margin=AI_MARGIN, model='sentence-transformers/all-MiniLM-L6-v2',
                                note='AI understanding of how a person describes a medicine. Used only above the threshold; '
                                     'otherwise the medicine stays unrecognised. Evaluation: reports/ai_medicine_understanding_v2_2026-11-12.json',
                                groups=AI_GROUPS),
        unverified_hints=sorted(set(UNVERIFIED_HINTS)),
    )
    OUT.write_text(json.dumps(payload, ensure_ascii=False, indent=1) + '\n', encoding='utf-8')
    REPORT.parent.mkdir(exist_ok=True)
    REPORT.write_text(json.dumps(dict(
        built=today, mode='offline-cache' if offline else 'live-download', pages=len(pages), herbs=len(HERBS),
        medicines=len(MEDS), medicine_aliases=len(meds_by_alias), quote_checks=checks, unique_verified_passages=len(passages), failures=0,
        sources={k: {kk: v[kk] for kk in ('url', 'updated', 'retrieved', 'text_sha256')} for k, v in pages.items()},
        method='Each quotation is normalised only for whitespace and typographic quotes/dashes, then required to occur in the page body text.'),
        indent=1) + '\n', encoding='utf-8')
    print(f'OK {len(HERBS)} herbs, {len(MEDS)} medicines ({len(meds_by_alias)} names), {checks} verbatim checks, {len(passages)} unique passages ->
{OUT.relative_to(ROOT)}')


if __name__ == '__main__':
    main()
```

scripts/evaluate_label_matching.mjs
SHA-256: 7d4a884e765f32d69aebe56e8c3f9ffaef1e746232fa9feefcbfbf1b71618a37

```
// Evaluate FoliaTrace name matching on real NIH DSLD labels.
// Ground truth: DSLD curators' declared Genus/Species on an ingredient row. The matcher sees only the
// ingredient name (+ forms), with the declared Genus/Species removed, then we compare.
// Usage: node scripts/evaluate_label_matching.mjs [--sample dev|holdout] [--per 24]
import {readFile, writeFile, mkdir} from 'node:fs/promises';
import {existsSync} from 'node:fs';
import {indexKnowledge, identify, fromDsldLabel} from '../frontend/demo/safety-engine.js';

const arg = (k, d) => { const i = process.argv.indexOf(k); return i > 0 ? process.argv[i + 1] : d; };
const sample = arg('--sample', 'dev'); const per = Number(arg('--per', '24'));
const offset = sample === 'dev' ? 0 : 60; // the held-out sample uses a later, non-overlapping results page
const kb = JSON.parse(await readFile('frontend/demo/safety-knowledge.json', 'utf8')); const idx = indexKnowledge(kb);
const cacheDir = '.runtime/dsld-cache'; await mkdir(cacheDir, {recursive: true});
const delay = ms => new Promise(r => setTimeout(r, ms));
async function getJson(url, file) {
  const path = `${cacheDir}/${file}.json`;
  if (existsSync(path)) return JSON.parse(await readFile(path, 'utf8'));
  // Polite pacing: NIH rate-limits bursts (HTTP 429). Back off exponentially when told to.
  for (let attempt = 0; attempt < 6; attempt++) {
    try {
      const r = await fetch(url, {headers: {'User-Agent': 'FoliaTrace-evaluation'}, signal: AbortSignal.timeout(15000)});
      if (r.ok) { const j = await r.json(); await writeFile(path, JSON.stringify(j)); await delay(1100); return j; }
      if (r.status === 429) { process.stderr.write(`429, backing off ${15 * 2 ** attempt}s\n`); await delay(15000 * 2 ** attempt); continue; }
    } catch {}
    await delay(1500 * (attempt + 1));
  }
  return null;
}
// Wait until NIH's rate limit has reset before starting (probe every 60 s, up to 2 h).
for (let i = 0; i < 120; i++) {
  try { const r = await fetch('https://api.ods.od.nih.gov/dsld/v9/label/33452', {signal: AbortSignal.timeout(15000)}); if (r.ok) break; process.stderr.write(`probe ${r.status} at ${new Date().toISOString()}\n`); } catch {}
  await delay(60000);
}
let fetchFailures = 0;
// Query terms: every covered herb plus generic botanical queries (so uncovered herbs appear too).
const queries = [...kb.herbs.map(h => h.name.replace(/\s*\/.*$/, '')), 'herbal blend', 'botanical', 'adaptogen', 'sleep support', 'immune support', 'women\'s health', 'liver support', 'energy'];
const ids = new Set();
for (const q of queries) {
  const s = await getJson(`https://api.ods.od.nih.gov/dsld/v9/search-filter?${new URLSearchParams({q, size: String(per), from: String(offset)})}`, `search-${sample}-${per}-${q.replace(/\W+/g, '_')}`);
  for (const h of s?.hits || []) ids.add(h._id);
}
const ownSet = sample === 'holdout' ? new Set() : null;
if (sample === 'holdout') { // drop any label that also appears in the dev sample
  for (const q of queries) { const s = await getJson(`https://api.ods.od.nih.gov/dsld/v9/search-filter?${new URLSearchParams({q, size: String(per), from: '0'})}`, `search-dev-${per}-${q.replace(/\W+/g, '_')}`); for (const h of s?.hits || []) ownSet.add(h._id); }
  for (const id of [...ids]) if (ownSet.has(id)) ids.delete(id);
}
import {levenshtein} from '../frontend/demo/safety-engine.js';
// Each herb's accepted (genus, species) pairs from its NCCIH Latin names.
const latinPairs = new Map([...idx.herbs.values()].map(h => [h.id, h.latin.map(l => l.toLowerCase().replace(/\(.*?\)|synonym/g, ' ').trim().split(/\s+/)).filter(p => p.length >= 2).map(([g, s]) => [g, s])]));
latinPairs.get('asian-ginseng').push(['panax', 'ginseng']);
```

```
const spp = s => !s || /^(spp?|sp\.?|species|x)$/.test(s);
function grade(genus, species, herbId) { // 'exact' | 'genus' | 'wrong'
  const g = genus.toLowerCase(), s = (species || '').toLowerCase();
  const pairs = latinPairs.get(herbId) || [];
  if (pairs.some(([pg, ps]) => pg === g && !spp(s) && levenshtein(ps, s, 2) <= 2)) return 'exact';
  if (pairs.some(([pg]) => pg === g)) return 'genus';
  return 'wrong';
}
function exactTruth(genus, species) { for (const [id] of latinPairs) if (grade(genus, species, id) === 'exact') return id; return null; }
let labels = 0, botanicalLabels = 0, labelsWithCovered = 0, blendLabels = 0, rows = 0, rowsInBlendNoAmount = 0;
const conf = {exact: 0, genus: 0, wrong: 0, missed: 0, correctly_unmatched: 0, ambiguous_with_true_candidate: 0, ambiguous_other: 0}; let graded = 0; const errors =
[]; const perHerb = {};
for (const id of ids) {
  const label = await getJson(`https://api.ods.od.nih.gov/dsld/v9/label/${id}`, `label-${id}`);
  if (!label) { fetchFailures++; if (fetchFailures > Math.max(5, ids.size * 0.02)) throw Error(`too many fetch failures (${fetchFailures}); sample would be biased
\u2014 rerun later`); continue; }
  if (!label.ingredientRows) continue;
  labels++;
  const product = fromDsldLabel(label);
  const botan = product.ingredients.filter(i => i.category === 'botanical');
  if (!botan.length) continue;
  botanicalLabels++;
  if (product.ingredients.some(i => i.blend)) blendLabels++;
  let covered = false;
  for (const ing of botan) {
    rows++; if (ing.blend) rowsInBlendNoAmount++;
    if (!ing.genus) continue; // no curator genus to grade against
    const truth = exactTruth(ing.genus, ing.species);
    if (truth) covered = true;
    const nameOnly = {name: product.name, ingredients: [{text: [ing.name, (ing.text.split(' \xb7 ')[2] || '')].filter(Boolean).join(' \xb7 '), name: ing.name,
category: 'botanical'}]};
    const found = identify(nameOnly, idx);
    const pred = found.find(f => f.herb)?.herb || null; const amb = found.find(f => f.ambiguous);
    graded++;
    const key = truth || '(not covered)'; perHerb[key] ??= {exact: 0, genus: 0, wrong: 0, missed: 0, n: 0}; perHerb[key].n++;
    if (amb && !pred) { const cand = idx.ambiguous.find(a => a.term === amb.ambiguous).candidates.map(c => c.herb); if (truth && cand.includes(truth))
conf.ambiguous_with_true_candidate++; else conf.ambiguous_other++; continue; }
    if (pred) {
      const g = grade(ing.genus, ing.species, pred); conf[g]++; perHerb[key][g]++;
      if (g !== 'exact') errors.push({label: id, ingredient: ing.name, declared: `${ing.genus} ${ing.species || ''}`.trim(), predicted: pred, grade: g});
    } else if (truth) { conf.missed++; perHerb[key].missed++; errors.push({label: id, ingredient: ing.name, declared: `${ing.genus} ${ing.species || ''}`.trim(),
predicted: null, grade: 'missed', truth}); }
    else conf.correctly_unmatched++;
  }
  if (covered) labelsWithCovered++;
}
const predicted = conf.exact + conf.genus + conf.wrong;
const result = {
  sample, created: new Date().toISOString(), knowledge_base: kb.version, labels_selected: ids.size, label_fetch_failures: fetchFailures, labels_fetched: labels,
labels_with_botanicals: botanicalLabels,
  botanical_ingredient_rows: rows, graded_rows_with_curator_genus: graded, confusion: conf,
  precision_exact_species: +(conf.exact / Math.max(1, predicted)).toFixed(4),
  precision_genus_or_better: +((conf.exact + conf.genus) / Math.max(1, predicted)).toFixed(4),
  wrong_genus_rate: +(conf.wrong / Math.max(1, predicted)).toFixed(4),
  recall_of_covered_species: +(conf.exact / Math.max(1, conf.exact + conf.missed)).toFixed(4),
  labels_with_a_covered_herb: labelsWithCovered, labels_with_blend_hidden_amounts: blendLabels,
  botanical_rows_inside_blends_without_own_amount: rowsInBlendNoAmount,
  method: 'Matcher sees ingredient name and forms only; the DSLD curator Genus/Species is removed and used as ground truth. exact = predicted NIH fact sheet lists
```

```
    the declared genus and species (species spelling within 2 edits); genus = same genus but a different or unspecified species; wrong = different genus; missed =
    declared species is covered but nothing was matched. Rows without a curator genus are not graded. ambiguous_with_true_candidate = the matcher refused to choose and
    listed the true species among its candidates.',
103   selection: `DSLD search results (size ${per}, from ${offset}) for each covered herb name plus 8 generic botanical queries${sample === 'holdout' ? '; labels also
    present in the dev sample removed' : ''}. Search-ranked, not a random market sample.`,
104   per_herb: perHerb, errors: errors.slice(0, 80),
105 };
106 await writeFile(`reports/label_matching_${sample}_2026-11-12.json`, JSON.stringify(result, null, 1) + '\n');
107 console.log(JSON.stringify({sample, labels, botanicalLabels, rows, graded, conf, exact: result.precision_exact_species, genusOrBetter:
    result.precision_genus_or_better, wrong: result.wrong_genus_rate, recall: result.recall_of_covered_species, blendLabels, rowsInBlendNoAmount, labelsWithCovered},
    null, 1));
108 for (const e of errors.filter(e => e.grade !== 'genus').slice(0, 30)) console.log(' ', e.grade, '|', e.ingredient, '| declared', e.declared, '| pred', e.predicted);
```

scripts/lib/contracts.mjs
SHA-256: ccd401ecc62cbdaf36351f3c9643b9fe5ba2dc75e915be00bd9603c702935826

```
import Ajv from 'ajv';
import { readJson } from './io.mjs';

export async function validator(schemaPath) {
  const ajv = new Ajv({ allErrors: true, strict: true, strictTypes: false });
  return ajv.compile(await readJson(schemaPath));
}

// Design-contract checks only. This validates references/offsets, not scientific truth.
export function integrityErrors(record) {
  const errors = [];
  const points = Array.from(record.passage.text);
  const checkSpan = span => {
    if (span && (span.end <= span.start || span.end > points.length || points.slice(span.start, span.end).join('') !== span.text)) errors.push('Span differs from original passage');
  };
  const sources = new Map([record.source, ...record.supporting_sources].map(x => [x.id, x]));
  const identifiers = [record.source.id, ...record.supporting_sources.map(x => x.id), record.passage.id, ...record.mentions.map(x => x.id), ...record.historical_claims.map(x => x.id), ...record.modern_hypotheses.map(x => x.id), ...record.evidence.map(x => x.id)];
  const taxa = new Set(record.resolutions.flatMap(x => x.candidates.map(c => c.taxon_id)));
  const ids = new Set([...identifiers, ...taxa]);
  if (new Set(identifiers).size !== identifiers.length) errors.push('Duplicate object ID');
  if (new Set(record.edges.map(x => x.id)).size !== record.edges.length) errors.push('Duplicate edge ID');
  if (record.passage.source_id !== record.source.id) errors.push('Passage references unknown source');
  const mentionIds = new Set(record.mentions.map(x => x.id));
  const hypotheses = new Map(record.modern_hypotheses.map(x => [x.id, x]));
  const checkProvenance = p => {
    const source = sources.get(p.source_id);
    if (!source || p.identifier !== source.identifier) errors.push('Unknown provenance source/identifier');
    if (source && p.licence !== source.licence) errors.push('Provenance licence mismatch');
    if (!Number.isFinite(Date.parse(p.retrieved_at))) errors.push('Invalid provenance timestamp');
  };
  for (const mention of record.mentions) {
    checkSpan(mention.span);
    if (mention.passage_id !== record.passage.id) errors.push('Mention references unknown passage');
  }
  for (const claim of record.historical_claims) {
    if (!mentionIds.has(claim.mention_id)) errors.push('Claim references unknown mention');
    checkSpan(claim.use_span); checkProvenance(claim.provenance);
  }
  for (const resolution of record.resolutions) {
    if (!mentionIds.has(resolution.mention_id)) errors.push('Resolution references unknown mention');
    if (resolution.status === 'UNRESOLVED' && resolution.candidates.length) errors.push('Unresolved record contains candidates');
    if (['RESOLVED', 'GENUS_ONLY', 'AMBIGUOUS'].includes(resolution.status) && !resolution.candidates.length) errors.push('Identity status requires candidates');
    if (resolution.status === 'GENUS_ONLY' && resolution.candidates.some(x => x.rank !== 'GENUS')) errors.push('Genus-only result contains other rank');
    if (resolution.confidence_mode === 'CALIBRATED_PROBABILITY') {
      const sum = resolution.unresolved_probability + resolution.candidates.reduce((a, c) => a + c.probability, 0);
      if (Math.abs(sum - 1) > 1e-8) errors.push('Identity probabilities do not sum to one');
    }
    if (new Set(resolution.candidates.map(c => c.taxon_id)).size !== resolution.candidates.length) errors.push('Duplicate candidate taxon');
    resolution.candidates.forEach(c => checkProvenance(c.provenance));
  }
  for (const hypothesis of record.modern_hypotheses) if (!taxa.has(hypothesis.taxon_id)) errors.push('Hypothesis has no candidate branch');
  for (const evidence of record.evidence) {
    const hypothesis = hypotheses.get(evidence.hypothesis_id);
```

```
    if (!hypothesis || hypothesis.taxon_id !== evidence.taxon_id) errors.push('Evidence has no matching hypothesis/taxon branch');
    if (evidence.citation_status === 'VERIFIED_CLAIM_SUPPORT' && !evidence.supporting_text) errors.push('Verified support requires a supporting passage');
    if (['HISTORICAL_ATTESTATION', 'ETHNOBOTANICAL_OBSERVATION'].some(x => evidence.evidence_types.includes(x)) && evidence.relation === 'SUPPORTS') errors.push('Historical use cannot support modern efficacy');
    checkProvenance(evidence.provenance);
  }
  for (const edge of record.edges) {
    if (!ids.has(edge.subject) || !ids.has(edge.object)) errors.push('Dangling graph edge');
    checkProvenance(edge.provenance);
  }
  if (record.source.source_type === 'SYNTHETIC_CONTRACT' && record.label_tier !== 'WEAK_SYNTHETIC') errors.push('Synthetic fixture cannot be promoted to gold/silver');
  return errors;
}
```

scripts/lib/http.mjs
SHA-256: ffa610c2249800153ea7a76643ce0a0c0cec7c08ba96b6957b63f6174682a55e

```
// Generic bounded GET utility for preparation probes. Not a scientific API adapter.
import { setTimeout as delay } from 'node:timers/promises';

export async function getJson(url, {
  fetchImpl = globalThis.fetch, attempts = 3, timeoutMs = 10000,
  maxBytes = 2_000_000, wait = delay, userAgent = 'EthnoHack-preparation/0.1 (small public API exploration)'
} = {}) {
  const parsed = new URL(url);
  if (parsed.protocol !== 'https:') throw new Error('HTTPS required');
  if (!Number.isInteger(attempts) || attempts < 1 || attempts > 5) throw new Error('Invalid attempt bound');
  if (timeoutMs < 1 || maxBytes < 1) throw new Error('Invalid request bound');
  for (let attempt = 1; attempt <= attempts; attempt++) {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(new Error('Request timeout')), timeoutMs);
    let retryMs = Math.min(1000 * 2 ** (attempt - 1), 10000);
    try {
      const response = await fetchImpl(parsed, {
        method: 'GET', redirect: 'error', signal: controller.signal,
        headers: { 'Accept': 'application/json', 'User-Agent': userAgent }
      });
      if (!response.ok) {
        await response.body?.cancel();
        const retryable = response.status === 429 || response.status >= 500;
        if (!retryable || attempt === attempts) throw Object.assign(new Error(`HTTP ${response.status}`), { terminal: true });
        const header = response.headers.get('retry-after');
        if (header) {
          const value = /^\d+(\.\d+)?$/.test(header) ? Number(header) * 1000 : Date.parse(header) - Date.now();
          if (Number.isFinite(value) && value >= 0) {
            if (value > 30000) throw Object.assign(new Error('Retry-After exceeds probe wait bound; stop'), { terminal: true });
            retryMs = value;
          }
        }
      } else {
        const declared = Number(response.headers.get('content-length'));
        if (declared > maxBytes) {
          await response.body?.cancel();
          throw Object.assign(new Error('Response too large'), { terminal: true });
        }
        const type = response.headers.get('content-type') ?? '';
        if (!type.includes('json')) {
          await response.body?.cancel();
          throw Object.assign(new Error('Expected JSON response'), { terminal: true });
        }
        const reader = response.body.getReader();
        const chunks = [];
        let length = 0;
        try {
          for (;;) {
            const { done, value } = await reader.read();
            if (done) break;
            length += value.byteLength;
            if (length > maxBytes) {
              await reader.cancel();
              throw Object.assign(new Error('Response too large'), { terminal: true });
            }
```

```
            chunks.push(Buffer.from(value));
          }
        } finally { reader.releaseLock(); }
        const body = Buffer.concat(chunks).toString('utf8');
        let data;
        try { data = JSON.parse(body); }
        catch { throw Object.assign(new Error('Invalid JSON response'), { terminal: true }); }
        return { data, body, status: response.status, attempts: attempt, bytes: length };
      }
    } catch (error) {
      if (error.terminal || attempt === attempts) throw error;
    } finally { clearTimeout(timer); }
    await wait(retryMs);
  }
  throw new Error('Request failed');
}
```

scripts/lib/io.mjs
SHA-256: 2673b00aa74c60c720ea495bd41ceec7a2a43feeec53a6060c8c441445c54558

```
 1 import { createHash } from 'node:crypto';
 2 import { mkdir, readFile, writeFile } from 'node:fs/promises';
 3 import { dirname } from 'node:path';
 4
 5 export const sha256 = value => createHash('sha256').update(value).digest('hex');
 6 export const readJson = async path => JSON.parse(await readFile(path, 'utf8'));
 7 export async function writeJson(path, value) {
 8   await mkdir(dirname(path), { recursive: true });
 9   await writeFile(path, JSON.stringify(value, null, 2) + '\n', 'utf8');
10 }
```

scripts/package_dsh_submission.py
SHA-256: 57d0242e2c95c392b4fc3a01c95648d007875e8d10e551dd35ab4c4d32f335b2

```
"""Package the judge-facing FoliaTrace source archive from an explicit allow-list.

Only what is needed to read, run, test and verify v0.8 is included. Internal planning notes,
historical reports, research-era acquisition scripts and screenshots stay out.
"""
from __future__ import annotations

import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED

ROOT = Path(__file__).resolve().parents[1]
TARGET = ROOT / 'dist/dsh-v2/foliatrace-source.zip'

TOP = ['README.md', 'LICENSE', 'package.json', 'package-lock.json', 'requirements.in', 'requirements.lock',
       'app.py', '.gitignore', '.gitattributes', '.env.example']
TREES = ['src', 'frontend', 'data', 'tests']          # whole trees, filtered below
SCRIPTS = ['start.ps1', 'start.mjs', 'validate_data.py', 'build_safety_knowledge.py', 'sync_public_demo.py',
           'build_care_demo.py', 'build_browser_inference.mjs', 'setup_model.py', 'setup_reranker.py',
           'evaluate_label_matching.mjs', 'verify_checker.mjs', 'verify_engine.mjs', 'ai_medicine_groups.py',
           'acquire_care_research.py', 'product_metrics.py', 'sync_product_docs.py',
           'build_product_onepage.py', 'build_code_pdf.py', 'package_dsh_submission.py']
DOCS = ['docs/release_v08_2026-11-12.md', 'docs/ai_disclosure.md', 'docs/assets/foliatrace-v08-brief.png']
REPORTS = ['reports/safety_knowledge_verification.json', 'reports/checker_browser_qa_public_2026-11-12.json',
           'reports/checker_browser_qa_local_2026-11-12.json', 'reports/label_matching_dev_2026-11-12.json',
           'reports/label_matching_holdout_2026-11-12.json', 'reports/retrieval_external.json',
           'reports/ai_medicine_understanding_2026-11-12.json', 'reports/ai_medicine_understanding_v2_2026-11-12.json',
           'reports/ai_research_relevance_2026-11-12.json', 'reports/engine_tests_2026-11-12.json']
OTHER = ['output/pdf/foliatrace-one-page.pdf',
         # checksum-frozen reference files that scripts/validate_data.py verifies at start-up
         'benchmark/historical_transfer/FREEZE.json', 'benchmark/historical_transfer/fernie_reference.json',
         'benchmark/historical_transfer/fernie_ai_audit.json']
SKIP_PARTS = {'__pycache__', 'node_modules', 'raw', 'private', 'local', '.runtime', '.cache', '.git'}
SKIP_PATHS = ('frontend/demo/research/',)            # EthnoHack-era research explorer front end


def wanted(path: Path) -> bool:
    rel = path.relative_to(ROOT).as_posix()
    if any(part in SKIP_PARTS for part in path.relative_to(ROOT).parts) or rel.startswith(SKIP_PATHS):
        return False
    if path.suffix in ('.pyc', '.sqlite', '.db', '.zip'):
        return False
    if rel.startswith('src/') and path.name == 'README.md':   # stale per-package stubs
        return False
    return True


def main() -> None:
    files = [ROOT / f for f in TOP + DOCS + REPORTS + OTHER if (ROOT / f).is_file()]
    files += [ROOT / 'scripts' / f for f in SCRIPTS if (ROOT / 'scripts' / f).is_file()]
    files += [ROOT / 'scripts/lib' / f.name for f in (ROOT / 'scripts/lib').glob('*.mjs')]
    for tree in TREES:
        files += [p for p in (ROOT / tree).rglob('*') if p.is_file() and wanted(p)]
```

```
    missing = [f for f in TOP[:7] + SCRIPTS + DOCS[:2] if not (ROOT / f).exists() and not (ROOT / 'scripts' / f).exists()]
    if missing:
        raise SystemExit(f'Missing required files: {missing}')
    files = sorted(set(files))
    TARGET.parent.mkdir(parents=True, exist_ok=True)
    manifest = {'created_at': datetime.now(timezone.utc).isoformat(), 'product': 'FoliaTrace v0.8',
                'scope': 'Allow-listed source to read, run, test and verify FoliaTrace. No credentials, model binaries, '
                         'runtime caches or saved user records. Data rights: data/NOTICE.md. Start with README.md.',
                'files': {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p in files}}
    with ZipFile(TARGET, 'w', ZIP_DEFLATED) as archive:
        for path in files:
            archive.write(path, 'foliatrace/' + path.relative_to(ROOT).as_posix())
        archive.writestr('foliatrace/SOURCE_MANIFEST.json', json.dumps(manifest, ensure_ascii=False, indent=2) + '\n')
    print(json.dumps({'path': str(TARGET.relative_to(ROOT)), 'files': len(files), 'bytes': TARGET.stat().st_size}, indent=2))


if __name__ == '__main__':
    main()
```

scripts/product_metrics.py
SHA-256: fb2e2ebe164c4b546085fb30294f6570190c97d13280954cb935cf01f363aadd

```
"""One source for release claims: machine-readable reports, never copied numbers."""
import json
from pathlib import Path
ROOT = Path(__file__).resolve().parents[1]


def read(name):
    return json.loads((ROOT / name).read_text(encoding='utf-8'))


def release_metrics():
    med = read('reports/ai_medicine_understanding_v2_2026-11-12.json')
    dev = read('reports/label_matching_dev_2026-11-12.json')
    hold = read('reports/label_matching_holdout_2026-11-12.json')
    kb = read('reports/safety_knowledge_verification.json')
    unit = read('reports/engine_tests_2026-11-12.json')
    public = read('reports/checker_browser_qa_public_2026-11-12.json')
    local = read('reports/checker_browser_qa_local_2026-11-12.json')
    for r in (unit, public, local):
        if r['status'] != 'PASS': raise ValueError('Release metrics require passing verification reports')
    library = read('sites/foliatrace-research/build/browser-library.json')
    botanical = dev['botanical_ingredient_rows'] + hold['botanical_ingredient_rows']
    hidden = dev['botanical_rows_inside_blends_without_own_amount'] + hold['botanical_rows_inside_blends_without_own_amount']
    return dict(med=med, held=med['held_out'], dev=dev, hold=hold, kb=kb, unit=unit, public=public, local=local,
                library=len(library), hidden=hidden, botanical=botanical, blend=f'{hidden / botanical:.0%}',
                labels=dev['labels_fetched'] + hold['labels_fetched'], public_checks=len(public['checks']), local_checks=len(local['checks']))
```

scripts/setup_model.py
SHA-256: ce94d0bd7bb831e8ae0fa7404fa1711edfde0bbd6ee6e8e104b3f7ba3787a95f

```python
"""Install a pinned, open MiniLM ONNX encoder without paid inference APIs."""
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import json
from urllib.request import urlopen
from src.common import ROOT, digest, now, read_json, write_json
from src.ingestion.tls import tls_context

MODEL = "sentence-transformers/all-MiniLM-L6-v2"
LOCK = ROOT / "data" / "models.lock.json"


def main():
    target = ROOT / "models" / "minilm"
    target.mkdir(parents=True, exist_ok=True)
    if LOCK.exists():
        lock = read_json(LOCK)
    else:
        with urlopen(f"https://huggingface.co/api/models/{MODEL}?blobs=true", timeout=40, context=tls_context()) as response:
            data = json.load(response)
        if data.get("cardData", {}).get("license") != "apache-2.0":
            raise ValueError("Unexpected model licence")
        files = {x["rfilename"]: x for x in data["siblings"]}
        model_file = next(x for x in ["onnx/model_quint8_avx2.onnx", "onnx/model_qint8_avx512_vnni.onnx", "onnx/model.onnx"] if x in files)
        lock = {"model": MODEL, "revision": data["sha"], "licence": "Apache-2.0", "observed_at": now(), "purpose": "Sentence embeddings for retrieval, not clinical inference", "files": [{"remote": model_file, "local": "model.onnx", "upstream_sha256": files[model_file].get("lfs", {}).get("sha256")}, {"remote": "tokenizer.json", "local": "tokenizer.json", "upstream_sha256": None}]}
    for entry in lock["files"]:
        dest = target / entry["local"]
        expected = entry.get("sha256") or entry.get("upstream_sha256")
        if dest.exists() and expected and digest(dest.read_bytes()) == expected:
            print("verified:", entry["local"], flush=True)
            continue
        url = f"https://huggingface.co/{lock['model']}/resolve/{lock['revision']}/{entry['remote']}"
        with urlopen(url, timeout=120, context=tls_context()) as response:
            body = response.read(150_000_001)
            if len(body) > 150_000_000:
                raise ValueError("Model artifact exceeds download bound")
        checksum = digest(bytes(body))
        if expected and checksum != expected:
            raise ValueError("Pinned model artifact checksum mismatch")
        dest.write_bytes(body)
        entry.update({"sha256": checksum, "bytes": len(body), "url": url})
        print("installed:", entry["local"], len(body), flush=True)
    write_json(LOCK, lock)


if __name__ == "__main__":
    main()
```

scripts/setup_reranker.py
SHA-256: d305be88e81db061a6bdf7f257401dd44ff390ecb0c98ac9f4a6429fe510e57c

```
"""Download pinned Apache-2.0 MS MARCO cross-encoder ONNX artifacts."""
import json
import sys
from pathlib import Path
from urllib.request import urlopen
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from src.common import ROOT, read_json, write_json, digest, now
from src.ingestion.tls import tls_context


def main():
    lockpath = ROOT / 'data/reranker.lock.json'
    target = ROOT / 'models/reranker'
    target.mkdir(parents=True, exist_ok=True)
    if lockpath.exists():
        lock = read_json(lockpath)
    else:
        model = 'cross-encoder/ms-marco-MiniLM-L6-v2'
        with urlopen(f'https://huggingface.co/api/models/{model}?blobs=true', context=tls_context(), timeout=45) as response:
            data = json.load(response)
        if data.get('cardData', {}).get('license') != 'apache-2.0':
            raise ValueError('Unexpected upstream model rights')
        files = {f['rfilename']: f for f in data['siblings']}
        remote = next((f for f in ('onnx/model_quint8_avx2.onnx', 'onnx/model_qint8_avx512_vnni.onnx', 'onnx/model.onnx') if f in files), None)
        if not remote:
            raise ValueError('No supported upstream ONNX artifact')
        lock = {'model': model, 'revision': data['sha'], 'licence': 'Apache-2.0', 'observed_at': now(), 'purpose': 'Query-document relevance ranking, not scientific entailment or calibrated confidence', 'files': [{'remote': remote, 'local': 'model.onnx', 'sha256': files[remote].get('lfs', {}).get('sha256')}, {'remote': 'tokenizer.json', 'local': 'tokenizer.json'}]}
    for f in lock['files']:
        path = target / f['local']
        if path.exists() and f.get('sha256') == digest(path.read_bytes()):
            print('verified:', f['local'], flush=True)
            continue
        url = f"https://huggingface.co/{lock['model']}/resolve/{lock['revision']}/{f['remote']}"
        with urlopen(url, context=tls_context(), timeout=120) as response:
            data = response.read(150_000_001)
        if len(data) > 150_000_000 or (f.get('sha256') and f['sha256'] != digest(data)):
            raise ValueError('Artifact size/checksum failure')
        path.write_bytes(data)
        f.update({'sha256': digest(data), 'bytes': len(data), 'url': url})
        print('installed:', f['local'], len(data), flush=True)
    write_json(lockpath, lock)


if __name__ == '__main__':
    main()
```

scripts/start.mjs
SHA-256: e0bd3ecc9d61873bb6e11a0955cc1b1a949facef31a363fd7982d640ce667cb1

```
    1 import { spawn } from 'node:child_process';
    2 import { existsSync } from 'node:fs';
    3 const python=process.platform==='win32'?'.venv/Scripts/python.exe':'.venv/bin/python';
    4 const child=spawn(existsSync(python)?python:'python',['app.py'],{stdio:'inherit',windowsHide:true});
    5 child.on('error',error=>{console.error(error.message);process.exitCode=1;});
    6 child.on('exit',code=>{process.exitCode=code??1;});
    7 process.on('SIGINT',()=>child.kill());
```

scripts/start.ps1
SHA-256: f46e7b515e33b5cb5fd6362de1a956035b3bc494b10bab3c50ce047e6090fbf2

```
param([switch]$CheckOnly, [switch]$Offline)
$ErrorActionPreference = 'Stop'
$projectRoot = Split-Path -Parent $PSScriptRoot
Set-Location -LiteralPath $projectRoot
$pythonPath = Join-Path $projectRoot '.venv/Scripts/python.exe'
if (-not (Test-Path -LiteralPath $pythonPath)) {
    python -m venv .venv
    if ($LASTEXITCODE -ne 0) { throw 'Python 3.11 or newer is required.' }
}
& $pythonPath -c "import importlib.util, sys; sys.exit(not all(importlib.util.find_spec(name) for name in 'fastapi uvicorn sklearn rapidfuzz tokenizers
onnxruntime'.split()))"
if ($LASTEXITCODE -ne 0) {
    & $pythonPath -m pip install -r requirements.lock
    if ($LASTEXITCODE -ne 0) { throw 'Dependency installation failed.' }
}
& $pythonPath scripts/validate_data.py
if ($LASTEXITCODE -ne 0) { throw 'Snapshot validation failed.' }
if (-not $CheckOnly) {
    if ($Offline) { $env:FOLIATRACE_OFFLINE = '1' }
    Write-Host 'Opening FoliaTrace at http://127.0.0.1:8000 (Ctrl+C to stop).'
    & $pythonPath app.py
    if ($LASTEXITCODE -ne 0) { throw 'FoliaTrace stopped with an error.' }
}
```

scripts/sync_product_docs.py
SHA-256: 9681b59bc55bc44578f2698e19b90e2586c7506c9ecd820dbe4af5145acd5051

```
"""Synchronize judge-facing descriptions from verified release reports."""
from pathlib import Path
from product_metrics import release_metrics
ROOT=Path(__file__).resolve().parents[1]


def main():
    m=release_metrics(); h=m['held']; c=h['counts']; kb=m['kb']; hold=m['hold']; med=m['med']
    ai=f"{c['correct_group']}/{h['drug_phrases']} medicine descriptions assigned to the correct group ({h['recall']:.1%}); {c['wrong_group']} wrong group assignments, {c['missed']} left unknown; all {c['correctly_unassigned']} other/negated phrases left unassigned ({h['phrases']} synthetic English phrases total)"
    labels=f"{hold['labels_fetched']} unseen NIH labels: {hold['precision_exact_species']:.1%} exact species, {hold['precision_genus_or_better']:.1%} genus-or-better precision, {hold['recall_of_covered_species']:.0%} recall of covered species"
    qa=f"{m['public_checks']}/{m['public_checks']} public-build and {m['local_checks']}/{m['local_checks']} local `/check/` real-browser checks; {m['unit']['passed']}/{m['unit']['tests']} behavioral unit tests"
    parity=m['public']['ai_medicine_parity']; original=parity[0]; v2=parity[1]
    (ROOT/'README.md').write_text(f'''# FoliaTrace

**Bring the bottle. Start a better conversation.**

Can't remember a medicine's name? Describe it. FoliaTrace uses on-device language AI to suggest its group and a second neural model to find research for the person's bottle and goals. It builds a pharmacist brief with exact NIH warnings, explicit uncertainties and source links. No account or paid AI API.

**[Try FoliaTrace](https://foliatrace-research.civil-bell-5511.chatgpt.site)** \u2014 choose Mei, Sam or Ray, or check your own bottle.

Mei, 72, takes warfarin and "her water pill". Her daughter buys a memory-and-joint supplement. Only one in three herb/supplement users disclosed their use to a conventional provider in NHIS 2002 ([Mehta et al. 2008](https://pmc.ncbi.nlm.nih.gov/articles/PMC2787410/)); 57.6% of U.S. adults used supplements in 2017\u201318 ([CDC NCHS](https://www.cdc.gov/nchs/products/databriefs/db399.htm)). The product connects the bottle, the person's own words and evidence a pharmacist can inspect.




## What works

| Task | Behavior |
|---|---|
| Add a bottle | Live NIH DSLD label lookup, pasted ingredients, or on-device photo OCR with editable review |
| Describe medicines | Exact/brand/typo matching first; MiniLM semantic inference when a name is unknown. "My water pill" and "my cholesterol pill" become explicit AI hypotheses, never identified prescriptions. Negated, historical and supplement-only mentions stay unassigned |
| Get the brief | Ingredient identities and medicine groups match verified NIH text. Every warning quotes NIH verbatim. Ambiguous species, unknown medicines, missing amounts and duplicate botanicals remain visible |
| Find personal research | Split compound goals; filter herb candidates and explicit nonclinical/protocol title cues; neural cross-encoder ranks each herb/goal. Low-scoring results stay empty. Expand a live lead to compare literal population, preparation and comparator excerpts with the actual bottle |
| Hand to pharmacist | QR/link carries data in the URL fragment; browser-local print/download and selective sharing. Opening a shared link does not save it unless requested |
| Go deeper | Free-form on-device AI/keyword research and five labelled replays of the local app's seven-dimension study-to-product comparisons |

## Evidence you can reproduce

| Check | Measured result | Source |
|---|---|---|
| Language understanding v2 | {ai} | [Report](reports/ai_medicine_understanding_v2_2026-11-12.json); `python scripts/ai_medicine_groups.py` |
| Browser/Python agreement | Same group decisions on original {original['phrases']} and fresh {v2['phrases']} phrases. Maximum cosine difference {original['max_cosine_delta']:.3f} / {v2['max_cosine_delta']:.3f}; int8 CPU/WASM kernels differ slightly | Browser QA reports |
| Research screening | {m['library']} licensed open-access abstracts; raw-logit floor 2.0 selected on the three development examples. Title screening and source excerpts are deterministic. No held-out relevance accuracy claimed | [Development report](reports/ai_research_relevance_2026-11-12.json) |
| Verbatim warnings | {kb['herbs']} herbs, {kb['unique_verified_passages']} unique NIH passages; build refuses a missing quotation | `python
```

```
   scripts/build_safety_knowledge.py --offline` |
44 | Real-label matching | {labels} | [Held-out report](reports/label_matching_holdout_2026-11-12.json) |
45 | Label uncertainty | {m['hidden']:,}/{m['botanical']:,} botanical rows ({m['blend']}) across {m['labels']:,} labels had no individual amount | Label-matching
   dev/holdout reports |
46 | End-to-end behavior | {qa}; live NIH, photos, QR, inference cancellation, privacy, WCAG A/AA and phone layout | `npm test`; `node scripts/verify_checker.mjs` |
47
48 The language phrases are synthetic English examples, not patient data. The original v1 holdout is now a regression set; v2 uses a fresh frozen set. No pharmacist
   review, user study or clinical outcome is claimed.
49
50 ## AI architecture
51
52 1. **Language understanding:** pinned `sentence-transformers/all-MiniLM-L6-v2`; sentence embeddings compared with short medicine-group prototypes; cosine \u2265
   {med['threshold']:.2f} and runner-up margin \u2265 {med['margin']:.2f}, plus conservative exclusions. The exact medicine still needs confirmation.
53 2. **Research relevance:** pinned `cross-encoder/ms-marco-MiniLM-L6-v2`; up to twenty keyword candidates per herb/goal, reranked on-device. Scores are relevance
   logits, not benefit, safety or applicability probabilities.
54 3. **Photo convenience:** Tesseract LSTM OCR; text is editable before analysis.
55
56 Every warning remains a deterministic match to verified NIH text. No generative model writes a warning. Source-excerpt extraction uses text cues and literal offsets;
    it is not an additional AI model. Models and tokenizers are checksum-verified and self-hosted. Medicines, purposes, conditions and photos stay in the browser; NIH
   receives only the separate label query or record ID.
57
58 ## Run and verify
59
60 Use Python 3.11+ and Node 24. The source archive excludes model binaries; install the pinned assets before building:
61
62 ```powershell
63 npm ci
64 python -m pip install -r requirements.lock
65 python scripts/setup_model.py
66 python scripts/setup_reranker.py
67 node scripts/build_browser_inference.mjs
68 python scripts/build_safety_knowledge.py
69 python scripts/build_care_demo.py
70 python scripts/sync_public_demo.py
71 npm test
72 ```
73
74 Serve `sites/foliatrace-research/build/` for the static product, or run `scripts/start.ps1` for the complete local app at `http://127.0.0.1:8000`; the same checker
   is mounted at `/check/`. First use downloads self-hosted models; once assets are cached, analysis needs no AI service. Live label lookup requires NIH connectivity.
75
76 ```powershell
77 node scripts/verify_checker.mjs http://127.0.0.1:8015 --report=reports/checker_browser_qa_public_2026-11-12.json
78 node scripts/verify_checker.mjs http://127.0.0.1:8000/check --report=reports/checker_browser_qa_local_2026-11-12.json
79 ```
80
81 `--no-live` skips only the live NIH checks. `python scripts/package_dsh_submission.py` creates the allow-listed source archive. [Release
   notes](docs/release_v08_2026-11-12.md) \xb7 [AI disclosure](docs/ai_disclosure.md) \xb7 [Attribution](data/NOTICE.md).
82
83 Submission artifacts: [one-page overview](output/pdf/foliatrace-one-page.pdf) and [complete code PDF](output/pdf/foliatrace-code.pdf). To rebuild them, install
   `reportlab` and `pypdf`, run `build_product_onepage.py`, `package_dsh_submission.py`, then `build_code_pdf.py`. The code PDF lists every first-party code file in the
    source ZIP with its hash; model/vendor binaries stay excluded.
84
85 ## Scope and prior work
86
87 Prepares a pharmacist conversation; does not identify a prescription, recommend doses, replace professional review or certify safety. Coverage is limited to the
   covered herbs, and labels are manufacturer declarations. Both label samples are search-ranked, not representative market samples; curator names can themselves be
   misspelt. Fictional example people and products.
88
```

```
The project began as an EthnoHack herbal-literature workspace and was rebuilt for DSH Hacks V2 around the person-to-pharmacist task. Development was substantially assisted by Codex and Claude. Code MIT; third-party data/models retain their licences. Prior-work eligibility remains the organisers' decision.
''',encoding='utf-8')
    (ROOT/'docs/ai_disclosure.md').write_text(f'''# AI disclosure

## Runtime AI

The product contains three on-device models. Language understanding and personal research ranking are its two core AI tasks; OCR is an optional input aid.

| Role | Model | Limits |
|---|---|---|
| Understand medicine descriptions | `sentence-transformers/all-MiniLM-L6-v2`, pinned in `data/models.lock.json` | Cosine comparison with group descriptions/prototypes, floor {med['threshold']:.2f}, margin {med['margin']:.2f}. Dictionary matches take precedence; negated/historical/supplement mentions are excluded. Results are hypotheses with a pharmacist-confirmation prompt |
| Rank research for each herb and goal | `cross-encoder/ms-marco-MiniLM-L6-v2`, pinned in `data/reranker.lock.json` | {m['library']} open-access abstracts; herb candidates and explicit nonclinical/protocol title cues filtered before neural scoring. Floor 2.0 selected on development examples; at most two leads per herb. No calibrated confidence, clinical entailment or applicability claim |
| Read a label photo | Tesseract English LSTM | Browser-local, editable text; photo never uploaded |

Warnings are deterministic matches to {kb['unique_verified_passages']} verified NIH quotations. No generative model writes, summarises or paraphrases a warning. Excerpts for population, preparation and comparator are extracted by deterministic text cues with literal offsets; absent extracted wording is left empty and is not proof the study omitted it. Recorded seven-dimension comparisons remain explicitly labelled replays.

Model/tokenizer SHA-256 checks run before inference. No paid AI API, hosted LLM or external inference service is used. Health inputs never enter requests; separate NIH label searches do. Worker queues prevent concurrent model initialisation, and input changes/erasure invalidate stale asynchronous results. Shared links do not persist data without the keep action.

## Evaluation and its limits

- Fresh frozen v2 set: **{ai}**. Synthetic English phrases authored for development/testing; no patient/user study.
- The original v1 {original['phrases']}-phrase holdout is retained unchanged and used as a regression set, not presented as fresh v2 testing. V2 adds conservative short prototypes and exclusions; thresholds were chosen using development phrases before scoring the fresh set. Failures remain in the report.
- Actual browser-worker decisions match Python on both sets. Maximum cosine difference: **{original['max_cosine_delta']:.3f}** on the original set, **{v2['max_cosine_delta']:.3f}** on v2. Pinned int8 CPU and WebAssembly kernels can differ numerically; QA tolerances are 0.01 / 0.02 respectively. No threshold was retuned on held-out results.
- Research floor 2.0 and clinical question wording were calibrated on the three fictional development examples after a 2.5 short-query floor missed relevant cognition/joint studies. Explicit animal/bench/protocol title exclusions prevent obvious scope errors. This is development calibration, **not** a held-out research-relevance benchmark or proof a study applies to a person.
- Label matching: {labels}. Search-ranked samples; curator declarations are imperfect ground truth. Engine frozen before the held-out label run.
- {qa}; no uncaught page exceptions, CSP violations or axe WCAG 2 A/AA violations in the recorded runs.

Full methods/results: `reports/ai_medicine_understanding*_2026-11-12.json`, `reports/ai_research_relevance_2026-11-12.json` and browser reports.

## Development assistance

The entrant developed FoliaTrace with substantial assistance from **OpenAI Codex** and **Anthropic Claude (Claude Code)**. Earlier Codex work covered the local FastAPI/SQLite consultation app, versioning, study comparisons and browser reranker. Claude's v0.8 work introduced the NIH checker, language AI, OCR, QR handoff and evaluations. Codex's follow-up corrected browser/Python padding, added conservative medicine prototypes/exclusions, goal-specific research with source excerpts, licensed clinical additions, cancellation/privacy fixes and verification.

AI-assisted choices have **not been reviewed by a pharmacist**: covered herbs, statement-to-medicine mappings, group descriptions/prototypes and fictional scenarios. NIH wording is checked verbatim, which does not validate those mappings or imply NIH endorsement. Three older factual study-reading notes in `data/reading_notes/` retain their Codex labels. No clinical validation is claimed.
''',encoding='utf-8')
    (ROOT/'docs/release_v08_2026-11-12.md').write_text(f'''# FoliaTrace v0.8 \u2014 language to evidence to pharmacist brief

2026-11-12. The browser product now uses language AI in the patient flow and neural evidence ranking in the brief itself.

```

```
## Behavior

- "Her water pill" / "my cholesterol pill" produce visible AI medicine-group hypotheses; the exact name remains a pharmacist question. Known medicine names take precedence. Negated/historical medicine mentions and supplement-only descriptions stay unassigned.
- The person's bottle and separate goals drive live on-device research. Herb filtering, explicit nonclinical/protocol title exclusions and a 2.0 raw-logit floor prevent obvious unrelated fills. Expand a study to read population, preparation and comparator excerpts alongside the bottle label; these excerpts are literal source text, selected by rules.
- {m['library']} open-access abstracts, including a bounded set of licensed cognition/joint/lipid studies. Rights/provenance remain per article; missing or unsupported abstracts are omitted.
- NIH quotations remain deterministic and verified verbatim. No model writes a warning.
- Async worker requests are serialized; input edits and erasure invalidate old briefs. Erasure cannot be undone by an inference completion. Shared-link inference cannot silently save the brief.
- QR, photo input, live NIH lookup, print/download, optional situations and the five recorded local-app comparisons remain available.

## Verification

| Check | Result |
|---|---|
| Knowledge base | {kb['herbs']} herbs; {kb['quote_checks']} checks; {kb['unique_verified_passages']} unique verbatim passages |
| Behavioral tests | {m['unit']['passed']}/{m['unit']['tests']} |
| Public-build browser QA | {m['public_checks']}/{m['public_checks']} |
| Local `/check/` browser QA | {m['local_checks']}/{m['local_checks']} |
| Fresh language holdout | {ai} |
| Browser/Python decisions | Identical on original {original['phrases']} and fresh {v2['phrases']}; max cosine deltas {original['max_cosine_delta']:.3f}/{v2['max_cosine_delta']:.3f} |
| Real-label holdout | {labels} |
| WCAG A/AA / CSP / uncaught errors | zero in both browser reports |

Research relevance is development-calibrated, not independently validated. Language tests are synthetic English; labels are search-ranked. No pharmacist review, usability study or clinical outcome is claimed. Coverage, ambiguity and missing evidence remain explicit. NIH may rate-limit live lookup; paste/photo remain available.

## Reproduce

See README setup. `npm test`, `node scripts/verify_engine.mjs`, `python scripts/ai_medicine_groups.py`, and `node scripts/verify_checker.mjs [base] --report=[path]`. `sync_public_demo.py` derives homepage metrics from reports; `sync_product_docs.py` derives these release claims; `build_product_onepage.py` builds the one-page overview. `package_dsh_submission.py` uses an explicit allow-list and excludes `.runtime`, caches, credentials and model binaries.
''',encoding='utf-8')
    path=ROOT/'docs/dsh_v2_submission.md';s=path.read_text(encoding='utf-8')
    start=s.index('## What it does');end=s.index('## How we built it')
    s=s[:start]+f'''## What it does

FoliaTrace uses **on-device language AI to understand how people describe their medicines**, then **neural research ranking to find evidence for their bottle and goals**. "My cholesterol pill" becomes an explicit statin hypothesis and a pharmacist-confirmation question alongside the relevant NIH red yeast rice quotation. AI understands and retrieves; every warning remains NIH's exact words.

1. **The bottle.** Search NIH DSLD labels, paste ingredients, or photograph the panel. OCR is an optional input aid, and the person reviews its text.
2. **Your words.** Names/brands/typos resolve first. MiniLM compares unknown descriptions with medicine-group prototypes, or abstains. Historical, negated and supplement-only mentions remain unassigned. Suggested groups stay labelled AI and require exact-name confirmation.
3. **The brief.** Verified NIH sentences, source links, questions for the pharmacist, ambiguous species, proprietary blends, duplicates and uncovered ingredients. No generated warning or safety verdict.
4. **Personal research.** For each herb and each goal, a cross-encoder ranks the on-device library of {m['library']} abstracts. Explicit nonclinical/protocol title cues and a relevance floor screen candidates. Expand a lead to inspect literal population, preparation and comparator wording beside the bottle label. A relevance score does not establish benefit or applicability.
5. **The handoff.** A QR/link opens the same brief using browser-local URL-fragment data; print, download and selective sharing are available. The pharmacist can explore more research or inspect five labelled replays of the local app's seven-dimension comparisons.

Mei shows water-pill understanding plus memory/joint research; Sam shows mood-supplement questions; Ray shows cholesterol-pill understanding, red yeast rice warnings
```

```
 and lipid research. People and products are fictional.
169
170 '''+s[end:]
171     a=s.index('- **On-device AI:**');b=s.index('\n- **Serverless handoff:**',a)
172     s=s[:a]+'- **On-device AI:** pinned MiniLM sentence embeddings for medicine language, a pinned MS MARCO cross-encoder for goal-specific research, and optional Tesseract LSTM OCR. Model/tokenizer hashes are verified. No generative model writes a warning.\n'+s[b:]
173     a=s.index('## Evidence');b=s.index('## Challenges',a)
174     s=s[:a]+f'''## Evidence
175
176 - **Language AI, fresh frozen synthetic holdout:** {ai}. The original holdout is retained as a regression set; browser and Python group decisions agree on both. These are synthetic phrases, not patient outcomes.
177 - **Real-label holdout:** {labels}. Both samples are search-ranked and curator names can be misspelt.
178 - **Working product:** {qa}, covering live NIH, photos, QR, inference, research, erasure, privacy, accessibility and layout.
179 - **Research calibration:** three fictional development examples; raw-logit floor 2.0. No held-out relevance accuracy or clinical applicability claimed.
180 - Methods, failures and reports ship with the source. No pharmacist review, usability study or clinical outcome claimed.
181
182 '''+s[b:]
183     s=s.replace('OCR, a neural reranker and a QR encoder all run','Language understanding, purpose-specific research, OCR and a QR encoder all run')
184     s=s.replace('JavaScript \xb7 WebAssembly \xb7 Tesseract.js','JavaScript \xb7 MiniLM \xb7 MS MARCO cross-encoder \xb7 WebAssembly \xb7 Tesseract.js')
185     s=s.replace('two-page brief','printable brief')
186     path.write_text(s,encoding='utf-8')
187     notice=ROOT/'data/NOTICE.md'; text=notice.read_text(encoding='utf-8')
188     marker='## On-device medicine AI and clinical research additions (2026-11-12)'
189     if marker in text: text=text[:text.index(marker)].rstrip()+'\n'
190     additions=__import__('json').loads((ROOT/'data/demo/care_research_additions.json').read_text(encoding='utf-8'))
191     text+='\n'+marker+'\n\n'
192     text+='The pinned `sentence-transformers/all-MiniLM-L6-v2` sentence encoder and tokenizer (`data/models.lock.json`) are Apache-2.0. Self-hosted runtime assets retain `inference/MODEL-LICENSE.txt`; model/tokenizer hashes are checked before use. Group prototypes and synthetic test phrases were authored with AI coding assistance; they are not clinical annotations.\n\n'
193     text+='`data/demo/care_research_additions.json` adds a bounded selection of human studies/reviews to the public browser library. Only abstracts with CC BY/CC0 rights reported by Europe PMC are redistributed; unsupported or missing abstracts are omitted. Authors, article links, acquisition URLs and response SHA-256 are retained. Selection is not an exhaustive evidence review; a study formulation is not the actual bottle. Article-specific licence versions and attribution requirements remain authoritative at the original source. No full articles, images or figures are bundled.\n\n'
194     for p in additions['papers']: text+=f"- **{p['id']}** \u2014 {p['title']} {p['authors']} ({p['year']}). [{p['licence']}]({p['url']}). DOI: {p.get('doi') or 'not listed'}.\n"
195     notice.write_text(text,encoding='utf-8')
196     site=ROOT/'sites/foliatrace-research/README.md'
197     site.write_text(f'''# FoliaTrace \u2014 public browser product
198
199 **Bring the bottle. Start a better conversation.** On-device language AI suggests a group from "my water pill" or "my cholesterol pill"; a neural cross-encoder finds research for the bottle and each goal. Every warning stays a verbatim NIH sentence.
200
201 Live: https://foliatrace-research.civil-bell-5511.chatgpt.site
202
203 This repository contains the static `build/` declared in `.openai/hosting.json`. `ai-worker.js` and `medicine-language.js` handle language inference/exclusions. `model-search-worker.js` and `research-evidence.js` rank evidence and extract literal context. `safety-engine.js` matches verified NIH text. OCR is an optional input aid; QR carries brief data in a URL fragment. All analysis runs locally in the browser.
204
205 Fresh synthetic language holdout: {ai}. Library: {m['library']} licensed abstracts. Verification: {qa}. Development-calibrated research floor, unreviewed medicine mappings, synthetic language tests and other limits are disclosed in `source.zip` \u2192 `README.md` and `docs/ai_disclosure.md`.
206
207 `source.zip` contains the complete local app, source, builders, behavioral tests and evaluation reports. `one-page.pdf` introduces the product; `DATA-NOTICE.md`, `CODE-LICENSE.txt` and `inference/*LICENSE*` retain attribution. Models/tokenizers are pinned and hash-checked. `.gitattributes` preserves exact deployed bytes.
208
209 Fictional people/products. A pharmacist must confirm inferred medicine groups and clinical applicability. No generated medical warning, dose recommendation or safety guarantee.
210 ''',encoding='utf-8')
211     readiness=ROOT/'docs/championship_readiness_2026-11-12.md'
```

```
    s=readiness.read_text(encoding='utf-8')
    tag='## Codex \u63a5\u624b\u5f8c\u7684\u5be6\u6e2c\u66f4\u65b0'
    if tag not in s:
        s=s.replace('## \u4e00\u53e5\u8a71',f'''{tag}

\u76ee\u524d\u6838\u5fc3 AI \u662f**\u4eba\u8a71\u7528\u85e5\u7406\u89e3**\u8207**\u74f6\u5167\u6210\u5206 \xd7
\u500b\u5225\u76ee\u7684\u7684\u7814\u7a76\u6392\u5e8f**\uff1bOCR
\u53ea\u5217\u70ba\u53ef\u9078\u8f38\u5165\u8f14\u52a9\u3002\u9996\u9801\u8207\u7c21\u5831\u76f4\u63a5\u5448\u73fe AI \u85e5\u7269\u7fa4\u7d44\u5047\u8a2d\uff0c\u4ee
5\u53ca\u53ef\u5c55\u958b\u7684\u7814\u7a76\u539f\u53e5\u3002\u4e0b\u65b9\u65e9\u671f\u6539\u9020\u5c0d\u7167\u70ba\u6b77\u53f2\u7d00\u9304\uff1b\u6700\u65b0\u7d50\u
679c\u4ee5\u5831\u544a\u8207\u82f1\u6587 release notes \u70ba\u6e96\u3002

- \u65b0\u7248\u8a9e\u8a00\u4fdd\u7559\u96c6\uff1a{c['correct_group']}/{h['drug_phrases']} \u53e5\u85e5\u7269\u63cf\u8ff0\u6b63\u78ba\uff0c{c['wrong_group']}
\u932f\u8aa4\u5206\u7d44\uff0c{c['missed']} \u4fdd\u7559\u672a\u77e5\uff1b{c['correctly_unassigned']}
\u53e5\u5176\u4ed6\uff0f\u5426\u5b9a\u63cf\u8ff0\u5168\u90e8\u4e0d\u6b78\u985e\u3002\u7e3d\u5171 {h['phrases']}
\u53e5\u5408\u6210\u82f1\u6587\uff0c\u6c92\u6709\u81e8\u5e8a\u6216\u4f7f\u7528\u8005\u9a57\u8b49\u3002
- \u700f\u89bd\u5668\uff0fPython \u5206\u7d44\u4e00\u81f4\uff1b\u539f\u59cb {original['phrases']} \u53e5\u6700\u5927 cosine \u5dee
{original['max_cosine_delta']:.3f}\uff0c\u65b0\u7248 {v2['phrases']} \u53e5 {v2['max_cosine_delta']:.3f}\u3002
- \u516c\u958b build {m['public_checks']} \u9805\u3001\u672c\u6a5f `/check/` {m['local_checks']} \u9805\u700f\u89bd\u5668 QA \u5168\u904e\uff1b{m['unit']['tests']}
\u9805\u884c\u70ba\u6e2c\u8a66\u5168\u904e\u3002
- \u7814\u7a76\u5eab {m['library']} \u7bc7\uff1b\u76f8\u95dc\u6027\u9580\u6abb 2.0 \u50c5\u7531\u958b\u767c\u7bc4\u4f8b\u6821\u6e96\uff0c\u6c92\u6709\u5ba3\u7a31\u73
68\u7acb\u76f8\u95dc\u6027\u6e96\u78ba\u7387\u3002\u690d\u7269\u3001\u660e\u78ba\u975e\u81e8\u5e8a\uff0fprotocol
\u6a19\u984c\u5148\u904e\u6ffe\uff1b\u539f\u53e5\u6458\u9304\u4f7f\u7528\u898f\u5247\uff0c\u6c92\u6709\u5192\u5145\u53e6\u4e00\u500b AI \u6a21\u578b\u3002
- \u4e3b\u8fa6\u8981\u6c42\u4ecd\u542b\u5f71\u7247\u3001\u4e00\u9801 PDF\u3001\u539f\u59cb\u78bc\u4ea4\u4ed8\uff1b\u5f71\u7247\u4f9d\u4f7f\u7528\u8005\u6307\u793a\u6
6ab\u4e0d\u91cd\u9304\u3002\u820a\u4f5c\u6cbf\u7528\u8cc7\u683c\u7531\u4e3b\u8fa6\u5224\u65b7\uff0c\u4e0d\u80fd\u4ee5\u529f\u80fd\u901a\u904e\u53d6\u4ee3\u8cc7\u683c
\u78ba\u8a8d\u3002

## \u4e00\u53e5\u8a71''')
        readiness.write_text(s,encoding='utf-8')
    print('Synchronized README, AI disclosure, release notes and Devpost description from reports.')


if __name__=='__main__': main()
```

scripts/sync_public_demo.py
SHA-256: 023da494f2e01f8a2c321234d9dc3be339cc431368f8395247fe8821d38529e1

```
"""Copy the public browser app (checker, OCR runtime, knowledge base) into the static build.

Runs on its own for front-end iterations, and is called by build_care_demo.py.
The recorded evidence cases and the literature library are produced by build_care_demo.py.
"""
from __future__ import annotations

import hashlib
import json
import shutil
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
TARGET = ROOT / 'sites/foliatrace-research/build'
APP_FILES = ('index.html', 'demo.css', 'demo.js', 'check.js', 'safety-engine.js', 'safety-examples.js',
             'safety-knowledge.json', 'pair-tokenizer.js', 'model-search-worker.js', 'ai-worker.js', 'medicine-language.js', 'research-evidence.js', '_headers')
NM = ROOT / 'node_modules'
OCR_FILES = {
    'tesseract.min.js': NM / 'tesseract.js/dist/tesseract.min.js',
    'worker.min.js': NM / 'tesseract.js/dist/worker.min.js',
    'tesseract-core-lstm.wasm.js': NM / 'tesseract.js-core/tesseract-core-lstm.wasm.js',
    'tesseract-core-simd-lstm.wasm.js': NM / 'tesseract.js-core/tesseract-core-simd-lstm.wasm.js',
    'lang/eng.traineddata.gz': NM / '@tesseract.js-data/eng/4.0.0_best_int/eng.traineddata.gz',
    'LICENSE-tesseract.js.txt': NM / 'tesseract.js/LICENSE.md',
}


VENDOR = {'qrcode.js': NM / 'qrcode-generator/qrcode.js'}


def sha(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def main() -> None:
    TARGET.mkdir(parents=True, exist_ok=True)
    # The EthnoHack-era historical research explorer is not part of the DSH product; keep it out of the site.
    for stale in ('research', 'assets', 'snapshots'):
        path = (TARGET / stale).resolve()
        if path.parent == TARGET.resolve() and path.exists():
            shutil.rmtree(path)
    for name in APP_FILES:
        shutil.copy2(ROOT / 'frontend/demo' / name, TARGET / name)
    # Always start from the original public-domain / CC-licensed corpus. Repeated
    # syncs cannot duplicate the bounded clinical additions.
    papers = []
    for name in ('papers.json', 'papers_extended.json'):
        records = json.loads((ROOT / 'data/demo' / name).read_text(encoding='utf-8'))
        papers.extend(records['papers'] if isinstance(records, dict) else records)
    additions = ROOT / 'data/demo/care_research_additions.json'
    if additions.exists():
        papers.extend(json.loads(additions.read_text(encoding='utf-8'))['papers'])
    library = {p['id']: {k: p.get(k, '') for k in ('id', 'title', 'abstract', 'url', 'licence')}
               for p in papers if p.get('abstract')}
    (TARGET / 'browser-library.json').write_text(json.dumps(list(library.values()), ensure_ascii=False) + '\n', encoding='utf-8')
```

```
    med = json.loads((ROOT / 'reports/ai_medicine_understanding_v2_2026-11-12.json').read_text(encoding='utf-8'))['held_out']
    dev = json.loads((ROOT / 'reports/label_matching_dev_2026-11-12.json').read_text(encoding='utf-8'))
    hold = json.loads((ROOT / 'reports/label_matching_holdout_2026-11-12.json').read_text(encoding='utf-8'))
    metrics = dict(library_count=len(library), medicine_wrong=med['counts']['wrong_group'], medicine_phrases=med['phrases'],
                   medicine_recall=f"{med['recall']:.0%}", medicine_correct=med['counts']['correct_group'], medicine_drugs=med['drug_phrases'],
                   species_precision=f"{hold['precision_exact_species']:.0%}", labels_holdout=hold['labels_fetched'],
                   labels_total=f"{dev['labels_fetched'] + hold['labels_fetched']:,}",
                   blend_percent=f"{(dev['botanical_rows_inside_blends_without_own_amount'] + hold['botanical_rows_inside_blends_without_own_amount']) / (dev['botanical_ingredient_rows'] + hold['botanical_ingredient_rows']):.0%}")
    (TARGET / 'product-metrics.json').write_text(json.dumps(metrics, indent=1) + '\n', encoding='utf-8')
    manifest = {}
    for dest, src in OCR_FILES.items():
        if not src.exists():
            raise SystemExit(f'Missing {src}; run npm ci first.')
        out = TARGET / 'ocr' / dest
        out.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(src, out)
        manifest[dest] = {'bytes': out.stat().st_size, 'sha256': sha(out)}
    # On-device language understanding model (all-MiniLM-L6-v2), checksum-verified in the browser.
    lock = json.loads((ROOT / 'data/models.lock.json').read_text(encoding='utf-8'))
    inf = TARGET / 'inference'; inf.mkdir(exist_ok=True)
    shutil.copy2(ROOT / 'models/minilm/model.onnx', inf / 'embed-model.onnx')
    shutil.copy2(ROOT / 'models/minilm/tokenizer.json', inf / 'embed-tokenizer.json')
    (inf / 'embed-manifest.json').write_text(json.dumps({
        'model': lock['model'], 'revision': lock['revision'], 'licence': lock['licence'],
        'model_sha256': sha(inf / 'embed-model.onnx'), 'tokenizer_sha256': sha(inf / 'embed-tokenizer.json'),
        'purpose': 'Understanding how a person describes a medicine; similarity, not diagnosis.'}, indent=1) + '\n', encoding='utf-8')
    for dest, src in VENDOR.items():
        (TARGET / 'vendor').mkdir(exist_ok=True)
        shutil.copy2(src, TARGET / 'vendor' / dest)
    pkg = json.loads((ROOT / 'package.json').read_text(encoding='utf-8'))
    manifest['_versions'] = {k: v for k, v in pkg.get('dependencies', {}).items() if 'tesseract' in k}
    (TARGET / 'ocr/manifest.json').write_text(json.dumps(manifest, indent=1) + '\n', encoding='utf-8')
    for dest, src in {'source.zip': ROOT / 'dist/dsh-v2/foliatrace-source.zip', 'one-page.pdf': ROOT / 'output/pdf/foliatrace-one-page.pdf', 'code.pdf': ROOT / 'output/pdf/foliatrace-code.pdf',
                      'DATA-NOTICE.md': ROOT / 'data/NOTICE.md', 'CODE-LICENSE.txt': ROOT / 'LICENSE'}.items():
        if src.exists():
            shutil.copy2(src, TARGET / dest)
    print(f'Synced {len(APP_FILES)} app files and {len(OCR_FILES)} OCR files to {TARGET.relative_to(ROOT)}')


if __name__ == '__main__':
    main()
```

scripts/validate_data.py
SHA-256: 121b053e3fd3590a81fbf623d60b8f00d324cdbde589199d8a5b21708213163f

```
"""Offline data setup check, no network or hidden cache required."""
import sys
from pathlib import Path
sys.path.insert(0,str(Path(__file__).resolve().parents[1]))
from src.common import DATA, ROOT, read_json, digest


def validate():
    from src.evidence.product_labels import reference_labels
    reference_labels()  # Canonical hashes and ingredient hierarchy of bounded NIH examples.
    manifest=read_json(DATA/'acquisition.json')
    for name in ['passages.json','taxa.json','papers.json','compounds.json','vernacular_congeners.json']:
        if manifest['files'][name]['sha256']!=digest((DATA/name).read_bytes()):raise ValueError('Checksum mismatch: '+name)
    congeners=read_json(DATA/'vernacular_congeners.json')
    if congeners['taxa_sha256']!=digest((DATA/'taxa.json').read_bytes()):raise ValueError('Open-world congener data was built from a different taxa snapshot')
    freeze=read_json(ROOT/'benchmark'/'historical_transfer'/'FREEZE.json')
    for path,expected in freeze['files'].items():
        if digest((ROOT/path).read_bytes())!=expected:raise ValueError('Frozen historical transfer file changed: '+path)
    papers=read_json(DATA/'papers.json')
    if (DATA/'papers_extended.json').exists():
        papers += read_json(DATA/'papers_extended.json')['papers']
    for lockname in ('mechanisms.lock.json', 'second_corpus.lock.json', 'taxonomy_expansion.lock.json', 'literature_expansion.lock.json'):
        path = ROOT / 'data' / lockname
        if path.exists():
            lock = read_json(path)
            if digest((ROOT / lock['file']).read_bytes()) != lock['sha256']:
                raise ValueError('Optional snapshot checksum mismatch: ' + lockname)
    corpus = ROOT / 'benchmark/direction_corpus'
    if (corpus / 'manifest.json').exists():
        manifest = read_json(corpus / 'manifest.json')
        for split, value in manifest['splits'].items():
            path = corpus / (split + '.json')
            if digest(path.read_bytes()) != value['sha256']:
                raise ValueError('Outcome corpus checksum mismatch: ' + split)
    for paper in papers:
        if paper['licence'].lower() not in ['cc by','cc0','cc-by']:raise ValueError('Paper redistribution licence not approved')
        if not paper['id'].startswith('PMC') or not paper['provenance']['url'].startswith('https://www.ebi.ac.uk/'):raise ValueError('Missing source metadata')
    print('Verified core bundle:',len(read_json(DATA/'passages.json')),'historical sections,',len(read_json(DATA/'taxa.json')),'taxa,',len(papers),'licensed
abstracts. Installed extension checksums also verified; full collection counts are shown in product health.')


if __name__=='__main__':validate()
```

scripts/verify_checker.mjs
SHA-256: 9607e174a00a70ebb9c88b44df7643b5ea8effa64bc9acf5ae31a258112ac63e

```
// Real-browser QA for the public FoliaTrace checker (Chrome/Edge via CDP, isolated profile).
// Usage: node scripts/verify_checker.mjs [baseUrl] [--no-live]
import {spawn} from 'node:child_process';
import {existsSync} from 'node:fs';
import {mkdir, readFile, writeFile, mkdtemp} from 'node:fs/promises';
import {once} from 'node:events';
import {tmpdir} from 'node:os';
import {join} from 'node:path';
import assert from 'node:assert/strict';

const base = process.argv.find(a => a.startsWith('http')) || 'http://127.0.0.1:8015';
const live = !process.argv.includes('--no-live');
const reportPath = process.argv.find(a => a.startsWith('--report='))?.slice(9) || 'reports/checker_browser_qa_2026-11-12.json';
const out = '.runtime/checker-qa'; await mkdir(out, {recursive: true});
const exe = ['C:/Program Files/Google/Chrome/Application/chrome.exe', 'C:/Program Files (x86)/Microsoft/Edge/Application/msedge.exe',
'/usr/bin/chromium'].find(existsSync);
const profile = await mkdtemp(join(tmpdir(), 'foliatrace-qa-'));
const browser = spawn(exe, ['--headless=new', '--remote-debugging-port=0', `--user-data-dir=${profile}`, '--no-first-run', '--no-default-browser-check', '--disable-
extensions', 'about:blank'], {stdio: 'ignore'});
const delay = ms => new Promise(r => setTimeout(r, ms));
async function until(fn, timeout = 30000, label = 'condition') { const end = Date.now() + timeout; let last; while (Date.now() < end) { try { const r = await fn();
if (r) return r; } catch (e) { last = e; } await delay(150); } throw Error('Timed out: ' + label + (last ? ' (' + last.message + ')' : '')); }
const report = {base, live, runner: 'Real Chromium via CDP; isolated profile; fictional data', started: new Date().toISOString(), checks: [], errors: [], requests:
[]};
let ws;
try {
  const port = await until(async () => Number((await readFile(join(profile, 'DevToolsActivePort'), 'utf8')).split('\n')[0]));
  const pages = await (await fetch(`http://127.0.0.1:${port}/json`)).json();
  ws = new WebSocket(pages.find(p => p.type === 'page').webSocketDebuggerUrl); await once(ws, 'open');
  let serial = 0; const pending = new Map();
  ws.addEventListener('message', e => { const m = JSON.parse(e.data); if (m.id && pending.has(m.id)) { const p = pending.get(m.id); pending.delete(m.id); m.error ?
p.reject(Error(JSON.stringify(m.error))) : p.resolve(m.result); }
    else if (m.method === 'Runtime.exceptionThrown') report.errors.push(m.params.exceptionDetails.exception?.description || m.params.exceptionDetails.text);
    else if (m.method === 'Network.requestWillBeSent') report.requests.push({url: m.params.request.url, method: m.params.request.method, body:
m.params.request.postData || ''});
    else if (m.method === 'Target.attachedToTarget') { send('Network.enable', {}, m.params.sessionId).catch(()=>{}); send('Runtime.enable', {},
m.params.sessionId).catch(()=>{}); } });
  const send = (method, params = {}, sessionId) => new Promise((resolve, reject) => { const id = ++serial; pending.set(id, {resolve, reject});
ws.send(JSON.stringify({id, method, params, ...(sessionId ? {sessionId} : {})})); });
  const ev = async x => { const r = await send('Runtime.evaluate', {expression: x, awaitPromise: true, returnByValue: true}); if (r.exceptionDetails) throw
Error(r.exceptionDetails.exception?.description || r.exceptionDetails.text); return r.result.value; };
  const check = (label, ok) => { assert.ok(ok, label); report.checks.push(label); console.log('PASS ' + label); };
  const shot = async name => { const r = await send('Page.captureScreenshot', {format: 'png'}); await writeFile(join(out, name + '.png'), Buffer.from(r.data,
'base64')); };
  const goto = async url => { await send('Page.navigate', {url}); await until(() => ev(`document.documentElement.dataset.ready==='true' &&
location.href===${JSON.stringify(url)}`), 30000, 'app ready'); if (new URL(url).searchParams.has('example')) await
until(()=>ev(`!document.querySelector('#brief').hidden`),120000,'example brief'); };
  await send('Page.enable'); await send('Runtime.enable'); await send('Network.enable');
  await send('Target.setAutoAttach', {autoAttach:true,waitForDebuggerOnStart:false,flatten:true});
  await send('Page.addScriptToEvaluateOnNewDocument', {source: `addEventListener('securitypolicyviolation',e=>{const
v=JSON.parse(sessionStorage.getItem('__csp')||'[]');v.push(e.violatedDirective+' '+e.blockedURI);sessionStorage.setItem('__csp',JSON.stringify(v))})`});
  await send('Emulation.setDeviceMetricsOverride', {width: 1440, height: 1000, deviceScaleFactor: 1, mobile: false});

  await goto(base + '/');
  check('three example people render', await ev(`document.querySelectorAll('#example-list .example').length===3`));
```

```
  check('build button disabled until a product exists', await ev(`document.querySelector('#run-check').disabled`));
  check('knowledge base statistics shown from the verified file', await ev(`/25 herbs \xb7 74 NIH passages \xb7 134 medicines/.test(document.querySelector('#kb-stats').textContent)`));

  // Test the actual browser worker, not a reimplementation of its arithmetic.
  const baseline = JSON.parse(await readFile('reports/ai_medicine_understanding_2026-11-12.json', 'utf8'));
  const current = JSON.parse(await readFile('reports/ai_medicine_understanding_v2_2026-11-12.json', 'utf8'));
  report.ai_medicine_parity = await ev(`(async()=>{
    const kb=await (await fetch('safety-knowledge.json')).json();
    const w=new Worker(new URL('ai-worker.js',location.href),{type:'module'});let seq=0;
    const run=(data)=>new Promise((resolve,reject)=>{const id=++seq;w.onmessage=({data:r})=>{if(r.id!==id)return;r.type==='error'?reject(Error(r.text)):resolve(r.results)};w.postMessage({...data,id,type:'classify'})});
    try { const old=${JSON.stringify(baseline)},v2=${JSON.stringify(current)};
      const sets=[{name:'original_34',ref:old.held_out,threshold:old.threshold,margin:0,groups:kb.ai_medicine_groups.groups.map(({id,description})=>({id,description}))},
        {name:'v2_54',ref:v2.held_out,threshold:v2.threshold,margin:v2.margin,groups:kb.ai_medicine_groups.groups}];
      const output=[];for(const s of sets){const actual=await run({texts:s.ref.rows.map(r=>r.text),threshold:s.threshold,margin:s.margin,groups:s.groups});
        output.push({name:s.name,phrases:actual.length,max_cosine_delta:Math.max(...actual.map((r,i)=>Math.abs(r.score-s.ref.rows[i].similarity))),mismatches:actual.filter((r,i)=>r.group!==s.ref.rows[i].predicted),rows:actual});}
      return output;
    } finally {w.terminate()}
  })()`);
  check('browser AI: original 34 phrases match Python groups, with every cosine difference < 0.01', !report.ai_medicine_parity[0].mismatches.length && report.ai_medicine_parity[0].max_cosine_delta<.01);
  check('browser AI: fresh 54 phrases match Python groups, within 0.02 cosine across int8 runtime kernels', !report.ai_medicine_parity[1].mismatches.length && report.ai_medicine_parity[1].max_cosine_delta<.02);
  await ev(`document.querySelector('#med-input').value='my water pill, ibuprofen';document.querySelector('#med-form').requestSubmit()`);
  await until(()=>ev(`document.querySelector('#med-list .chip.ai')?.textContent.includes('my water pill')`),120000,'AI water pill');
  check('AI: \u201cmy water pill\u201d suggests blood-pressure medicine with an AI tag and confirmation',await ev(`/blood-pressure/.test(document.querySelector('#med-list .chip.ai').textContent)&&/confirm/.test(document.querySelector('#med-list .chip.ai').textContent)`));
  await until(()=>ev(`![...document.querySelectorAll('#med-list .chip')].some(c=>c.classList.contains('pending'))`),120000,'ibuprofen abstention');
  check('AI: ibuprofen remains unrecognised rather than being assigned to a covered group',await ev(`[...document.querySelectorAll('#med-list .chip.unknown')].some(c=>/ibuprofen/.test(c.textContent))`));

  // Example 1: Mei
  await ev(`document.querySelector('[data-example=mei]').click()`);
  await until(() => ev(`!document.querySelector('#brief').hidden`), 120000, 'brief');
  const mei = await ev(`(()=>{const c=[...document.querySelectorAll('#brief .finding.discuss')];return {n:c.length,titles:c.map(x=>x.querySelector('h4').textContent),links:c.map(x=>x.querySelector('.nih-source a').href)}})()`);
  check('Mei: ginkgo \xd7 warfarin and garlic \xd7 warfarin raised first', mei.n === 2 && mei.titles.some(t => /Ginkgo \xd7 Warfarin/.test(t)) && mei.titles.some(t => /Garlic \xd7 Warfarin/.test(t)));
  check('Mei: every raised card links to an NIH NCCIH page', mei.links.every(h => h.startsWith('https://www.nccih.nih.gov/health/')));
  check('Mei: bare \u201cginseng\u201d is an identity question with candidate species', await ev(`[...document.querySelectorAll('.gap.identity h4')].some(h=>/which ginseng/.test(h.textContent)) && document.querySelectorAll('.gap.identity .candidates>li').length===3`));
  check('Mei: proprietary blend reported once with its total', await ev(`[...document.querySelectorAll('.gap-list li')].filter(l=>/Proprietary Wellness Blend/.test(l.textContent)&&/300 mg in total/.test(l.textContent)).length===1`));
  check('Mei: BioPerine flags an absorption-enhanced curcumin form', await ev(`[...document.querySelectorAll('.finding h4')].some(h=>/Absorption-enhanced/.test(h.textContent))`));
  check('Mei: no invented ginger statement', await ev(`![...document.querySelectorAll('.finding h4')].some(h=>/^Ginger/.test(h.textContent))`));
  check('Mei: the AI medicine hypothesis is visible at the start of the brief',await ev(`/her water pill/.test(document.querySelector('#brief .ai-understanding')?.textContent||'')`));
  await until(()=>ev(`/^Ranked/.test(document.querySelector('#ai-research .ai-status')?.textContent||'')`),180000,'Mei research');
  report.research_examples = {mei:await ev(`JSON.parse(document.querySelector('#brief').dataset.research)`)};
  check('Mei: AI routes memory to ginkgo and joint pain to turmeric, with literal study excerpts',await ev(`(()=>{const l=JSON.parse(document.querySelector('#brief').dataset.research);return l.some(x=>x.herb==='Ginkgo'&&x.shown.some(s=>s.goal==='memory'))&&l.some(x=>x.herb==='Turmeric'&&x.shown.some(s=>s.goal==='joint pain'))&&document.querySelectorAll('.lead-context blockquote').length>0})()`));
  await shot('mei-brief');
```

```
  84
  85   // Download / selective sharing / escaping
  86   const dl = await ev(`fetch(document.querySelector('#download-brief').href).then(r=>r.text())`);
  87   check('download contains verbatim NIH sentence and medicine list', dl.includes('Ginkgo may increase the risk of bleeding in people who are taking anticoagulant
     drugs, such as warfarin.') && dl.includes('Medicines I take:') && dl.includes('Warfarin 5 mg'));
  88   await ev(`document.querySelector('#share-meds').click()`);
  89   const dl2 = await ev(`new Promise(r=>setTimeout(r,200)).then(()=>fetch(document.querySelector('#download-brief').href)).then(r=>r.text())`);
  90   check('unticking \u201cMy medicines\u201d removes personal medicine names from lists, AI hypotheses, cards and questions', !dl2.includes('Medicines I take:') &&
     !dl2.includes('Warfarin 5 mg') && !dl2.includes('her water pill'));
  91   await ev(`document.querySelector('#share-meds').click()`);
  92
  93   // Serverless pharmacist handoff: QR + link round trip
  94   await ev(`document.querySelector('#show-handoff').click()`);
  95   await until(() => ev(`document.querySelector('#handoff-qr svg') && document.querySelector('#handoff-link').value.includes('#brief=')`), 10000, 'handoff QR');
  96   const link = await ev(`document.querySelector('#handoff-link').value`);
  97   check('handoff: QR code and link are generated in the browser (brief inside the URL fragment)', link.startsWith(base + '/#brief=') && link.length < 2500);
  98   report.handoff_link_chars = link.length;
  99   const meiTitles = await ev(`[...document.querySelectorAll('#brief .finding h4')].map(h=>h.textContent).join('|')`);
 100   await ev(`localStorage.clear()`);
 101   const before = report.requests.length;
 102   await send('Page.navigate', {url: 'about:blank'}); await delay(300);
 103   await goto(link);
 104   await until(() => ev(`!document.querySelector('#brief').hidden`), 8000, 'shared brief');
 105   check('handoff: a fresh browser opens the identical brief from the link alone', (await ev(`[...document.querySelectorAll('#brief .finding
     h4')].map(h=>h.textContent).join('|')`)) === meiTitles && await ev(`!document.querySelector('#shared-banner').hidden`));
 106   check('handoff: the brief data never appears in any request', report.requests.slice(before).every(r => !r.url.includes('#brief=') && !/warfarin/i.test(r.url +
     r.body)));
 107   check('handoff: opening a shared link does not save it on the device unless asked', await ev(`localStorage.getItem('foliatrace.check.v2')===null`));
 108   await ev(`document.querySelector('#print-root').innerHTML=''`);
 109   await send('Page.navigate', {url: base + '/'}); await until(() => ev(`document.documentElement.dataset.ready==='true'`), 20000, 'home');
 110
 111   // Example 2 & 3
 112   await ev(`document.querySelector('[data-example=sam]').click()`);
 113   await until(() => ev(`document.querySelector('#brief-summary').textContent.length>0 && [...document.querySelectorAll('.finding h4')].some(h=>/St\\.
     John/.test(h.textContent))`), 5000, 'sam');
 114   check("Sam: St. John's wort \xd7 sertraline (serotonin) and \xd7 birth control pill (NIH list item quoted)", await ev(`(()=>{const
     c=[...document.querySelectorAll('.finding.discuss')];return c.some(x=>/Sertraline/.test(x.textContent)&&/serotonin/.test(x.textContent))&&c.some(x=>/Birth control
     pill/.test(x.querySelector('h4').textContent)&&/NIH\u2019s list includes: Birth control pills/.test(x.textContent))})()`));
 115   await until(()=>ev(`/^Ranked/.test(document.querySelector('#ai-research .ai-status')?.textContent||'')`),180000,'Sam research');
 116   report.research_examples.sam=await ev(`JSON.parse(document.querySelector('#brief').dataset.research)`);
 117   check('Sam: patient reading leads exclude quail, rat and irrigation experiments',await ev(`![...document.querySelectorAll('#ai-research
     li>a')].some(a=>/quail|rats|irrigation|foliar/i.test(a.textContent))`));
 118   await ev(`document.querySelector('[data-example=ray]').click()`);
 119   await until(() => ev(`[...document.querySelectorAll('.finding h4')].some(h=>/Red yeast rice/.test(h.textContent))`), 5000, 'ray');
 120   check('Ray: AI suggests a statin for \u201cmy cholesterol pill\u201d; red yeast rice card preserves the NIH lovastatin quote', await
     ev(`[...document.querySelectorAll('.finding.discuss')].some(x=>/Red yeast rice \xd7 my cholesterol pill/.test(x.textContent)&&x.querySelector('.pill-
     ai')&&/structurally identical to the medicine lovastatin/.test(x.textContent))`));
 121   check('Ray: goldenseal \xd7 metformin and duplicate green tea across products', await ev(`[...document.querySelectorAll('.finding.discuss h4')].some(h=>/Goldenseal
      \xd7 Metformin/.test(h.textContent)) && [...document.querySelectorAll('.gap-list li')].some(l=>/Green tea extract.*2 products/.test(l.textContent))`));
 122   await until(()=>ev(`/^Ranked/.test(document.querySelector('#ai-research .ai-status')?.textContent||'')`),180000,'ranked research in brief');
 123   report.research_examples.ray=await ev(`JSON.parse(document.querySelector('#brief').dataset.research)`);
 124   check('brief: research is ranked and unrelated goldenseal studies are withheld',await ev(`document.querySelectorAll('#ai-research .lead-group').length>=3 &&
     [...document.querySelectorAll('#ai-research .lead-group')].some(g=>/Goldenseal/.test(g.querySelector('h4').textContent)&&/No closely matching
     study/.test(g.textContent))`));
 125   check('brief: AI research is included in the downloaded brief', (await ev(`fetch(document.querySelector('#download-
     brief').href).then(r=>r.text())`)).includes('Research leads, ranked by on-device AI'));
 126
 127   // Hostile input is escaped
```

```
  await ev(`document.querySelector('#clear-all').click(); document.querySelector('#tab-text').click(); const f=document.querySelector('#text-form');
f.elements.name.value='<img src=x onerror=window.__pwn=1>'; f.elements.text.value='Kava root 100 mg'; f.requestSubmit(); document.querySelector('#med-
input').value='alprazolam, <b>xx</b>'; document.querySelector('#med-form').requestSubmit(); document.querySelector('#run-check').click();`);
  await until(() => ev(`!document.querySelector('#brief').hidden`), 120000, 'hostile brief');
  check('user text is never interpreted as HTML (screen and download)', await ev(`!window.__pwn && !document.querySelector('#brief img') &&
!document.querySelector('#product-list img')`) && !(await ev(`fetch(document.querySelector('#download-brief').href).then(r=>r.text())`)).includes('<img src=x'));
  check('kava \xd7 alprazolam (benzodiazepine) raised; unknown medicine kept for the pharmacist', await ev(`[...document.querySelectorAll('.finding.discuss
h4')].some(h=>/Kava \xd7 alprazolam/.test(h.textContent)) && /did not recognise the medicine/.test(document.querySelector('#brief').textContent)`));

  // Reload persistence
  await send('Page.reload'); await until(() => ev(`document.documentElement.dataset.ready==='true'`), 20000, 'reload');
  check('products and medicines persist in this browser after reload', await ev(`document.querySelectorAll('.product-card').length===1 &&
document.querySelectorAll('#med-list .chip').length===2`));

  // Clearing during a cold asynchronous medicine request must not resurrect data.
  await ev(`document.querySelector('#med-input').value='the tablets for high cholesterol';document.querySelector('#med-
form').requestSubmit();document.querySelector('#run-check').click();document.querySelector('#clear-all').click()`);
  await delay(2000);
  check('erasing during inference cannot restore a stale brief or saved medicine data',await
ev(`document.querySelector('#brief').hidden&&document.querySelectorAll('#med-list .chip').length===0&&localStorage.getItem('foliatrace.check.v2')===null`));

  // OCR on a rendered label image
  await ev(`document.querySelector('#clear-all').click(); document.querySelector('#tab-photo').click();`);
  await ev(`(async()=>{const c=document.createElement('canvas');c.width=1100;c.height=520;const
g=c.getContext('2d');g.fillStyle='#fff';g.fillRect(0,0,c.width,c.height);g.fillStyle='#111';g.font='bold 46px Arial';g.fillText('Supplement
Facts',40,70);g.font='34px Arial';['Serving Size 1 Capsule','Ginkgo biloba leaf extract 120 mg','Valerian root 200 mg','St. John\\'s Wort extract 300 mg','Other
ingredients: cellulose'].forEach((t,i)=>g.fillText(t,40,140+i*72));const b=await new Promise(r=>c.toBlob(r,'image/png'));const dt=new DataTransfer();dt.items.add(new
 File([b],'label.png',{type:'image/png'}));const i=document.querySelector('#photo-input');i.files=dt.files;i.dispatchEvent(new Event('change'));})()`);
  await until(() => ev(`/Ginkgo/i.test(document.querySelector('#text-form').elements.text.value)`), 120000, 'OCR text');
  const ocrText = await ev(`document.querySelector('#text-form').elements.text.value`);
  check('on-device OCR reads the photographed label into editable text', /valerian/i.test(ocrText) && /john/i.test(ocrText));
  await ev(`document.querySelector('#text-form').requestSubmit(); document.querySelector('#med-input').value='zolpidem, warfarin'; document.querySelector('#med-
form').requestSubmit(); document.querySelector('#run-check').click();`);
  await until(() => ev(`[...document.querySelectorAll('.finding h4')].length>0`), 5000, 'ocr brief');
  check('photo \u2192 brief: valerian \xd7 zolpidem and ginkgo \xd7 warfarin raised; product marked as read from photo', await ev(`(()=>{const
h=[...document.querySelectorAll('.finding.discuss h4')].map(x=>x.textContent).join('|');return /Valerian \xd7 zolpidem/.test(h)&&/Ginkgo \xd7
warfarin/.test(h)&&/Read from your photo/.test(document.querySelector('#product-list').textContent)})()`));
  await shot('ocr-brief');

  // Live NIH label search
  if (live) {
    await ev(`document.querySelector('#clear-all').click(); document.querySelector('#tab-search').click(); document.querySelector('#dsld-q').value='ginkgo biloba';
document.querySelector('#dsld-form').requestSubmit();`);
    await until(() => ev(`document.querySelectorAll('#dsld-results li').length>0`), 20000, 'NIH results');
    check('live NIH DSLD search returns label records', true);
    await ev(`document.querySelector('#dsld-results li button').click()`);
    await until(() => ev(`document.querySelectorAll('.product-card').length===1`), 20000, 'NIH label add');
    check('a live NIH label is added with its record number', await ev(`/NIH label #\\d+/.test(document.querySelector('.product-card').textContent)`));
    await ev(`document.querySelector('#med-input').value='Coumadin'; document.querySelector('#med-form').requestSubmit(); document.querySelector('#run-
check').click();`);
    await until(() => ev(`!document.querySelector('#brief').hidden`), 5000, 'live brief');
    check('live NIH ginkgo label \xd7 Coumadin (brand of warfarin) raises the ginkgo statement', await
ev(`[...document.querySelectorAll('.finding.discuss')].some(x=>/Ginkgo \xd7 Coumadin/.test(x.textContent))`));
    const nihReqs = report.requests.filter(r => r.url.includes('api.ods.od.nih.gov'));
    check('NIH requests carry only the label search or record id \u2014 never medicines', nihReqs.length > 0 && nihReqs.every(r =>
!/coumadin|warfarin|zolpidem|alprazolam/i.test(r.url + r.body)));
  }
  const foreign = report.requests.filter(r => !r.url.startsWith(base) && !r.url.startsWith('data:') && !r.url.startsWith('blob:') &&
```

```
    !r.url.includes('api.ods.od.nih.gov'));
  check('no request leaves for any other host', foreign.length === 0);
  check('no medicine or condition text in any request', report.requests.every(r =>
    !/warfarin|sertraline|zolpidem|alprazolam|metformin|surgery/i.test(r.url.replace(base, '') + r.body)));

  // Pharmacist evidence (recorded) and on-device AI research search, reached from the brief
  await goto(base + '/?example=mei');
  await until(() => ev(`document.querySelectorAll('#case-tabs button').length===5`), 15000, 'recorded cases');
  check('five recorded study-to-product comparisons load (labelled as replays)', await ev(`/replayed/.test(document.querySelector('.recording-note').textContent) &&
    document.querySelectorAll('#case-detail table').length>0`));
  await ev(`document.querySelector('#brief .research').click()`);
  check('\u201cResearch this herb\u201d fills the on-device research question from the brief', await ev(`/Ginkgo clinical trial/.test(document.querySelector('#live-
    form').elements.query.value)`));
  await ev(`document.querySelector('#live-form').elements.query.value='peppermint oil abdominal pain irritable bowel syndrome'; document.querySelector('#live-form
    button[value=ai]').click()`);
  await until(() => ev(`!document.querySelector('#live-results').hidden && document.querySelector('#live-results').dataset.method==='ai'`), 180000, 'on-device AI');
  const ai = await ev(`({n:document.querySelectorAll('#live-results article').length,scores:JSON.parse(document.querySelector('#live-
    results').dataset.scores),status:document.querySelector('#live-status').textContent})`);
  check('on-device neural reranker returns finite scores and keeps the question local', ai.n > 0 && ai.scores.every(Number.isFinite) && /No question was
    sent/.test(ai.status));
  check('no AI request leaves for any other host', report.requests.every(r => r.url.startsWith(base.replace(/\/check$/, '')) || r.url.startsWith('data:') ||
    r.url.startsWith('blob:') || r.url.includes('api.ods.od.nih.gov')));

  // Accessibility & layout
  await goto(base + '/?example=ray');
  await ev(await readFile('node_modules/axe-core/axe.min.js', 'utf8'));
  const axe = await ev(`axe.run(document,{runOnly:{type:'tag',values:['wcag2a','wcag2aa']}}).then(r=>r.violations.map(v=>v.id+':'+v.nodes.length))`);
  report.axe = axe;
  check('axe WCAG 2 A/AA: zero violations with a full brief open', axe.length === 0);
  check('desktop: no horizontal overflow', await ev(`document.documentElement.scrollWidth<=innerWidth`));
  await send('Emulation.setDeviceMetricsOverride', {width: 390, height: 844, deviceScaleFactor: 2, mobile: true});
  await goto(base + '/?example=mei');
  check('390 px phone: no horizontal overflow with a full brief open', await ev(`document.documentElement.scrollWidth<=innerWidth+1`));
  await shot('mobile-brief');
  check('no medicine, purpose or condition text leaves in page or worker requests',
    report.requests.every(r=>!/warfarin|sertraline|zolpidem|alprazolam|metformin|ibuprofen|water(?:%20|\+| )pill|cholesterol(?:%20|\+| )pill|memory(?:%20|\+|
    )and|surgery/i.test(r.url.replace(base,'')+r.body)));
  const csp = await ev(`sessionStorage.getItem('__csp')||'[]'`); report.csp_violations = JSON.parse(csp);
  check('Content-Security-Policy: zero violations on the final page', report.csp_violations.length === 0);
  check('no uncaught page exceptions', report.errors.length === 0);
  report.status = 'PASS';
} catch (e) { report.status = 'FAIL'; report.failure = e.message; console.error('FAIL', e.message); process.exitCode = 1; }
finally {
  report.finished = new Date().toISOString(); report.request_hosts = [...new Set(report.requests.map(r => { try { return new URL(r.url).host; } catch { return
    r.url.slice(0, 20); } }))]; delete report.requests;
  await writeFile(reportPath, JSON.stringify(report, null, 1) + '\n');
  try { ws?.close(); } catch {} browser.kill();
  console.log(report.status, report.checks.length, 'checks', report.errors.length ? 'errors: ' + report.errors.join(' | ') : '');
}
```

scripts/verify_engine.mjs
SHA-256: aff87b5c6faf55a8e8671bb12e737545774b1dbd460c31c4169a5246c9372498

```
    1 // Run the behavioral tests and retain a small, machine-readable release report.
    2 import {spawnSync} from 'node:child_process';
    3 import {writeFile} from 'node:fs/promises';
    4 const result=spawnSync(process.execPath,['--test','--test-
      reporter=tap','tests/safety_engine.test.mjs','tests/research_evidence.test.mjs'],{encoding:'utf8',windowsHide:true});
    5 const count=key=>Number(result.stdout.match(new RegExp('^# '+key+' (\\d+)$','m'))?.[1]||0);
    6 const report={created:new Date().toISOString(),status:result.status===0?'PASS':'FAIL',tests:count('tests'),passed:count('pass'),failed:count('fail'),runner:'node
      --test; medicine matching, exclusions, source quotes and research evidence invariants'};
    7 await writeFile('reports/engine_tests_2026-11-12.json',JSON.stringify(report,null,1)+'\n');
    8 console.log(JSON.stringify(report));process.exitCode=result.status||0;
```

src/__init__.py
SHA-256: 931541660195ff9a9b166ce84ed68e5cd5ed69f8094de5d14ba229695f30deee

```
    1 """FoliaTrace research pipeline."""
    2 __version__ = "0.7.0"
```

src/api/__init__.py
SHA-256: babb59266802b58a095034866c2d3b321435df173a9d2f14198eb426b86a7691

```
    1 """Research API and local product frontend."""
```

src/api/app.py
SHA-256: f445d472d37798e7fdb519fc63a29f0e7b82cb6b911bba8b9702db2e6446650e

```python
from __future__ import annotations

import csv
import io
import json
import re
from threading import Lock
from functools import lru_cache
from collections import Counter
from typing import Literal
from urllib.parse import urlencode
from urllib.parse import quote

from fastapi import FastAPI, HTTPException, Query, UploadFile, File
from fastapi.responses import FileResponse, Response
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, ConfigDict, Field
from starlette.middleware.trustedhost import TrustedHostMiddleware

from src import __version__
from src.common import DATA, ROOT, read_json, write_json, now
from src.nlp.corpus import mine_corpus
from src.pipeline import Pipeline
from src.ingestion.client import ScientificClient
from src.ingestion.pubmed import fetch_records
from src.api.contracts import AnalysisRecord
from src.evidence.classify import conflict_groups
from src.evidence.comparability import not_assessed, summarize
from src.evidence.counterfactual import counterfactuals
from src.evidence.brief import research_brief, brief_markdown
from src.evidence.readings import active_reading_sets
from src.evidence.chemistry import assay_leads
from src.evidence.outcome_model import OutcomeModel
from src.evidence.handoff import HandoffRequest, HandoffPacket, HandoffStore, build_handoff, canonical, handoff_markdown
from src.evidence.question_review import ResearchQuestion, ReadingDecision, QuestionStore, run_question, validate_packet, review_markdown
from src.evidence.question_review import canonical as question_canonical
from src.evidence.consultation import ConsultationInput, ConsultationStore, prepare_consultation, validate_consultation, consultation_markdown
from src.evidence.consultation_summary import summary_markdown, summary_html
from src.evidence.care_workspace import ClarificationAnswer, ShareSelection, add_clarification, review_context, study_bridge, share_preview
from src.evidence.product_labels import LabelSnapshot, search_labels, read_label, label_wording, ingredient_source, compare_labels, summarize as summarize_label,
    ingredients as label_ingredients
from pydantic import ValidationError
import hashlib
from src.retrieval.knowledge import KnowledgeIndex
from src.nlp.extract import normalize

app = FastAPI(title="FoliaTrace consultation and evidence API", version=__version__, description="Prepare herbal-product information and source-linked professional
    evidence reading for a pharmacist conversation.")
app.add_middleware(TrustedHostMiddleware, allowed_hosts=["localhost", "127.0.0.1", "testserver"])
_corpus_lock = Lock()


@lru_cache(maxsize=1)
def pipeline():
    return Pipeline()
```

```

@app.middleware("http")
async def security_headers(request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["Referrer-Policy"] = "no-referrer"
    if request.url.path.startswith("/check"):
        # Browser checker: on-device WASM (OCR, reranker), its own workers, blob downloads, and NIH label search only.
        response.headers.setdefault("Content-Security-Policy", CHECKER_CSP + "; frame-ancestors 'none'")
    response.headers.setdefault("Content-Security-Policy", "default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; connect-src 'self';
object-src 'none'; base-uri 'none'; frame-ancestors 'none'")
    return response


CHECKER_CSP = ("default-src 'self'; script-src 'self' 'wasm-unsafe-eval'; worker-src 'self' blob:; "
               "connect-src 'self' blob: data: https://api.ods.od.nih.gov; img-src 'self' data: blob:; style-src 'self'; "
               "object-src 'none'; base-uri 'none'; form-action 'none'")


class AnalysisRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    text: str | None = Field(None, min_length=1, max_length=50000)
    passage_id: str | None = Field(None, max_length=100)
    source_title: str = Field("Researcher-provided text", max_length=200)
    retrieval: Literal["bm25", "embedding", "hybrid", "rerank"] = "bm25"
    name_to_inspect: str | None = Field(None, min_length=2, max_length=120)


@app.get("/api/health")
def health():
    p = pipeline()
    abstracts = sum(bool(paper.get('abstract')) for paper in p.papers)
    return {"status": "ok", "version": __version__, "snapshot": p.snapshot, "taxa": len(p.taxa), "papers": len(p.papers), "abstracts_available": abstracts,
"literature_coverage": "FULL_LOCAL_ABSTRACT_CACHE" if abstracts == len(p.papers) else "PARTIAL_ABSTRACT_COVERAGE", "offline": p.client.offline, "passages":
len(p.passages) + len(p.extra_passages), "collections": 1 + bool(p.extra_passages), "chembl_assays": len(p.mechanism_records), "outcome_model_trained": (ROOT /
'.cache/outcome_model.joblib').exists(), "encoder_installed": p.index.encoder.available, "reranker_installed": p.index.reranker.available, "study_model_trained":
p.study_model.model is not None, "mode": "LOCAL_LIVE_PIPELINE", "network_required_for_cached_analysis": False}


@app.get("/api/passages")
def passages():
    withdrawn=pipeline().withdrawn()
    return [{k: p.get(k) for k in ("id", "title", "text", "source_id", "licence", "context", "url", "date", "community", "collection")} for p in pipeline().passages
+ pipeline().extra_passages if p['id'] not in withdrawn and p['source_id'] not in withdrawn]


@app.get("/api/search")
def search(q: str = Query(min_length=2, max_length=150), kind: str | None = None):
    return {"results": pipeline().search(q, kind)}


@app.post("/api/analyses", response_model=AnalysisRecord)
def analyze(request: AnalysisRequest):
    p = pipeline()
    source = None
    if request.passage_id:
        source = next((row for row in p.passages + p.extra_passages if row["id"] == request.passage_id), None)
```

```
        if not source:
            raise HTTPException(404, "Historical passage not found")
        text = source["title"] + "\n\n" + source["text"]
        source = {k: v for k, v in source.items() if k != "text"}
    else:
        text = request.text or ""
        source = {"id": "user-provided", "title": request.source_title, "url": None, "licence": "USER_RIGHTS_UNVERIFIED", "context": "Researcher input, not independently verified historical provenance", "community": None}
    if not text.strip():
        raise HTTPException(422, "Supply a passage_id or nonempty text")
    try:
        result = p.analyze(text, source=source, method=request.retrieval, name_to_inspect=request.name_to_inspect)
        result = {**result, "researcher_reviews": p.store.reviews(result["run_id"], result)}
        result["research_brief"] = research_brief(result)
        return result
    except (ValueError, RuntimeError) as error:
        raise HTTPException(422, str(error)) from error


@app.post("/api/uploads/text")
async def upload_text(file: UploadFile = File()):
    body = await file.read(200001)
    if len(body) > 200000:
        raise HTTPException(413, "Text upload exceeds 200KB")
    if not (file.filename or "").lower().endswith(".txt"):
        raise HTTPException(415, "Only UTF-8 .txt files are accepted; no PDF/OCR is claimed")
    try:
        text = body.decode("utf-8-sig")
    except UnicodeDecodeError as error:
        raise HTTPException(422, "Expected UTF-8 text") from error
    if len(text) > 50000:
        raise HTTPException(413, "Passage exceeds 50,000 characters")
    return analyze(AnalysisRequest(text=text, source_title=(file.filename or "Uploaded text")[:200]))


def upgrade_legacy(result):
    """Analyses stored before comparability screening and counterfactuals existed are marked
    as not screened, never silently re-scored with today's rules."""
    for e in result['evidence']:
        e.setdefault('comparability', not_assessed())
        if 'screening' not in e['comparability']:
            # Summary fields added later are derived from the stored checks, not recomputed checks.
            e['comparability'] = summarize(e['comparability']['checks'])
    if 'identity_counterfactuals' not in result:
        result['identity_counterfactuals'] = counterfactuals(result)


def get_run(run_id):
    if not re.fullmatch(r"[0-9a-f]{24}", run_id):
        raise HTTPException(404, "Analysis not found")
    result = pipeline().store.get(run_id)
    if not result:
        raise HTTPException(404, "Analysis not found")
    upgrade_legacy(result)
    p=pipeline();withdrawn=p.withdrawn()
    for key, references in result.get('historical_identity_references', {}).items():
        result['historical_identity_references'][key] = [reference for reference in references if not {reference['source']['id'], reference['source'].get('source_id')} & withdrawn]
```

```
        for reference in result['historical_identity_references'][key]:
            for excerpt in reference['excerpts']:
                excerpt['names'] = [name for name in excerpt['names'] if name['candidate_id'] not in withdrawn]
            reference['excerpts'] = [excerpt for excerpt in reference['excerpts'] if excerpt['names']]
    if result['source']['id'] in withdrawn or result['source'].get('source_id') in withdrawn:
        raise HTTPException(410,'Historical source withdrawn from active views')
    removed_taxa={c['taxon_id'] for r in result['resolutions'] for c in r['candidates'] if c['taxon_id'] in withdrawn}
    if removed_taxa or any(e['paper']['id'] in withdrawn for e in result['evidence']) or 'chembl' in withdrawn or any(a['id'] in withdrawn or a['molecule_chembl_id']
 in withdrawn or a['target']['target_chembl_id'] in withdrawn for a in result.get('compound_assay_leads', [])):
        for resolution in result['resolutions']:
            resolution['candidates']=[c for c in resolution['candidates'] if c['taxon_id'] not in withdrawn]
            if not resolution['candidates']:resolution['status']='UNRESOLVED'
        result['evidence']=[e for e in result['evidence'] if e['paper']['id'] not in withdrawn and e['taxon_id'] not in withdrawn]
        result['compound_assay_leads']=assay_leads(result['evidence'], p.mechanism_records, withdrawn)
        result['graph']=p.make_graph(result)
        result['conflicts']=conflict_groups(result['evidence'])
        result['identity_counterfactuals']=counterfactuals(result)
    result['study_reading_sets'] = active_reading_sets(result.get('study_reading_sets', []), [c['taxon_id'] for r in result['resolutions'] for c in r['candidates']],
 [c['id'] for c in result['conditions']], withdrawn)
    result['researcher_reviews']=p.store.reviews(run_id, result)
    result['research_brief']=research_brief(result)
    return result


class ReviewRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    target_kind: Literal["identity", "evidence"]
    target_id: str = Field(min_length=1, max_length=200)
    decision: Literal["ACCEPT", "REJECT", "INSUFFICIENT_INFORMATION"]
    reason: str = Field(min_length=3, max_length=1000)
    reviewer: str = Field("Local researcher", min_length=1, max_length=80)


def review_targets(result):
    identities = {f"{c['key']}::{b['taxon_id']}" for c in result.get("identity_counterfactuals", []) for b in c["branches"]}
    return {"identity": identities, "evidence": {e["id"] for e in result["evidence"]}}


@app.get("/api/analyses/{run_id}", response_model=AnalysisRecord)
def analysis(run_id: str):
    return get_run(run_id)


@app.get("/api/analyses/{run_id}/reviews")
def list_reviews(run_id: str):
    return {"reviews": get_run(run_id)["researcher_reviews"]}


@app.put("/api/analyses/{run_id}/reviews")
def save_review(run_id: str, review: ReviewRequest):
    result = get_run(run_id)
    if review.target_id not in review_targets(result)[review.target_kind]:
        raise HTTPException(404, "Review target is not part of this analysis")
    pipeline().store.put_review(run_id, review.model_dump())
    active = get_run(run_id)
    return {"reviews": active["researcher_reviews"], "research_brief": active["research_brief"]}


```

```python
@app.delete("/api/analyses/{run_id}/reviews/{target_kind}/{target_id:path}")
def delete_review(run_id: str, target_kind: Literal["identity", "evidence"], target_id: str):
    get_run(run_id)
    pipeline().store.delete_review(run_id, target_kind, target_id)
    active = get_run(run_id)
    return {"reviews": active["researcher_reviews"], "research_brief": active["research_brief"]}


@app.get("/api/graph/{run_id}")
def graph(run_id: str, focus: str | None = None, predicate: str | None = None, source_id: str | None = None, limit: int = Query(100, ge=1, le=300)):
    result=get_run(run_id)
    if pipeline().withdrawn():
        edges=[e for e in result['graph']['edges'] if (not focus or e['subject']==focus or e['object']==focus) and (not predicate or e['predicate']==predicate) and (not source_id or e['provenance']['source_id']==source_id)][:limit]
        ids={e[k] for e in edges for k in ('subject','object')}
        return {'edges':edges,'nodes':[n for n in result['graph']['nodes'] if n['id'] in ids]}
    return pipeline().store.neighborhood(run_id, focus, predicate, source_id, limit)


@app.get("/api/sources/{source_id}")
def source(source_id: str):
    p = pipeline()
    if source_id in p.withdrawn():raise HTTPException(410,'Source withdrawn from active views')
    if source_id in p.papermap:
        return p.papermap[source_id]
    if source_id in p.taxmap:
        return p.taxmap[source_id]
    record = next((x for x in p.passages + p.extra_passages if x["id"] == source_id), None)
    if record:
        if record.get('source_id') in p.withdrawn():raise HTTPException(410,'Source withdrawn from active views')
        return record
    raise HTTPException(404, "Source not in approved local snapshot")


@app.get("/api/benchmarks")
def benchmark_results():
    path = ROOT / "reports" / "benchmark" / "metrics.json"
    result = read_json(path) if path.exists() else {"status": "NOT_RUN"}
    learned = ROOT / 'reports/outcome_model.json'
    if learned.exists():
        result['outcome_model'] = read_json(learned)
    for name in ('retrieval_external', 'use_role_audit', 'product_coverage', 'product_evaluation', 'known_answers', 'unseen_books', 'independent_records', 'expert_review', 'regulator_agreement', 'folklore_blind', 'lead_foresight', 'free_text_blind', 'free_text_blind_round9', 'free_text_blind_round10'):
        path = ROOT / 'reports' / (name + '.json')
        if path.exists():
            result[name] = read_json(path)
    audit = ROOT / 'benchmark/gold/book_statement_audit.json'
    if audit.exists():
        result['book_statement_audit'] = {k: v for k, v in read_json(audit).items() if k != 'items'}
    return result


@app.get('/api/research/chemistry')
def chemical_snapshot():
    p = pipeline()
    path = p.data_dir / 'mechanisms.json'
    if not path.exists():
        return {'records': [], 'mappings': [], 'failures': [], 'publications': {}, 'status': 'NOT_INSTALLED'}
```

```
    result = read_json(path)
    withdrawn = p.withdrawn()
    if 'chembl' in withdrawn:
        raise HTTPException(410, 'ChEMBL records withdrawn from active views')
    result['records'] = [r for r in result['records'] if not {r['id'], r['molecule_chembl_id'], r['target']['target_chembl_id']} & withdrawn]
    result['publications'] = {}
    for c in p.compounds:
        publications = []
        for paper in p.papers:
            if paper['id'] in withdrawn:
                continue
            if match := re.search(r'\b' + re.escape(c['name']) + r'\b', paper['abstract'], re.I):
                publications.append({'paper_id': paper['id'], 'title': paper['title'], 'url': paper['url'], 'licence': paper['licence'], 'supporting_text':
paper['abstract'][max(0, match.start()-100):min(len(paper['abstract']), match.end()+140)], 'scope': 'Text co-mention only. No verified plant-constituent or
therapeutic relation.'})
        result['publications'][str(c['cid'])] = publications
    return result


@app.get('/api/research/corpus')
def historical_corpus(format: Literal['json', 'csv'] = 'json'):
    p = pipeline()
    path = ROOT / 'reports/corpus_mining.json'
    # Local extraction is bounded and offline. Rebuild under one lock, publish
    # atomically, and keep statement-anchor migrations from the previous build.
    with _corpus_lock:
        result = read_json(path) if path.exists() else None
        if result is None or result.get('snapshot_sha256') != p.snapshot or result.get('code_fingerprint') != p.code_fingerprint:
            result = mine_corpus(p, previous=result)
            write_json(path, result)
    result['build_status'] = 'CURRENT'
    withdrawn = p.withdrawn()
    hidden = {s['id'] for s in p.passages + p.extra_passages if s['source_id'] in withdrawn}
    result['records'] = [r for r in result['records'] if r['source_id'] not in withdrawn | hidden]
    result['collections'] = dict(Counter(s.get('collection') or 'Culpeper 1850' for s in p.passages + p.extra_passages if s['id'] not in withdrawn | hidden))
    result['conditions'] = dict(Counter(c['id'] for r in result['records'] for c in r['conditions']))
    result['roles'] = dict(Counter(r.get('statement_role', 'LEGACY_ROLE_NOT_SCREENED') for r in result['records']))
    if format == 'csv':
        output = io.StringIO()
        writer = csv.writer(output)
        writer.writerow(['source_id', 'collection', 'edition', 'plant_wording', 'historical_statement', 'span_start', 'span_end', 'condition_retrieval_aids',
'parts', 'preparations', 'relation_scope', 'negation_flag', 'source_url', 'source_licence', 'human_reviewed', 'statement_role'])
        for r in result['records']:
            values = [r['source_id'], r['collection'], r['date'], '; '.join(n['text'] for n in r['names']), r['statement']['text'], r['statement']['start'],
r['statement']['end'], '; '.join(c['id'] for c in r['conditions']), '; '.join(p['text'] for p in r['parts']), '; '.join(p['text'] for p in r['preparations']),
r['scope'], r['negated'], r['url'], r['licence'], False, r.get('statement_role', 'LEGACY_ROLE_NOT_SCREENED')]
            writer.writerow(["'" + str(v) if str(v).lstrip().startswith(('=', '+', '-', '@')) else str(v) for v in values])
        return Response(output.getvalue(), media_type='text/csv', headers={'Content-Disposition': 'attachment; filename="foliatrace-traditional-uses.csv"'})
    return result


@app.get('/api/research/evidence-map')
def evidence_map(format: Literal['json', 'csv'] = 'json'):
    path = ROOT / 'reports/evidence_map.json'
    if not path.exists():
        raise HTTPException(503, 'Build the research map with scripts/build_evidence_map.py')
    result = read_json(path)
    p = pipeline()
```

```
    result['build_status'] = 'CURRENT' if result.get('snapshot_sha256') == p.snapshot and result.get('code_fingerprint') == p.code_fingerprint else 'PREVIOUS_BUILD'
    if result['build_status'] == 'PREVIOUS_BUILD':
        result['build_warning'] = 'This map was generated by an earlier build. Its study summaries may differ from a new analysis. Rebuild with scripts/build_evidence_map.py.'
    if format == 'csv':
        output = io.StringIO()
        writer = csv.writer(output)
        writer.writerow(['plant', 'topic', 'historical_statements', 'books', 'modern_research_state', 'human_studies', 'syntheses', 'laboratory_studies', 'favourable', 'no_benefit', 'independent_records', 'independent_places', 'recorded_poisonous', 'lead_score', 'key_study', 'key_study_url', 'eligible_source_statements', 'uncertain_source_links', 'priority_components', 'source_statement_ids', 'isolated_plant_studies', 'summary_basis', 'summary_source_ids'])
        for r in result['rows']:
            best = r.get('best_study') or {}
            values = [r['taxon'], r['topic'], r['historical_statements'], '; '.join(r['books']), r['state'], r['human_studies'], r['syntheses'], r['preclinical_studies'], r['favourable'], r['no_benefit'], (r.get('independent_use') or {}).get('records', 0), (r.get('independent_use') or {}).get('places', 0), bool(r.get('recorded_poisonous')), r['lead_score'], best.get('title', ''), best.get('url', ''), r.get('direct_statements', ''), r.get('uncertain_statements', ''), json.dumps(r.get('priority_components', {})), '; '.join(s['source_id'] + ':' + s['claim_id'] for s in r.get('statements', []))]
            values.extend([r.get('isolated_plant_studies', ''), (r.get('assessment_basis') or {}).get('kind', ''), '; '.join((r.get('assessment_basis') or {}).get('paper_ids', []))])
            writer.writerow(["'" + str(v) if str(v).lstrip().startswith(('=', '+', '-', '@')) else str(v) for v in values])
        return Response(output.getvalue(), media_type='text/csv', headers={'Content-Disposition': 'attachment; filename="foliatrace-research-map.csv"'})
    foresight = ROOT / 'reports/lead_foresight.json'
    if foresight.exists():
        # The time-split check of this ranking travels with it, so the order is read with its record.
        result['foresight'] = read_json(foresight)['cutoffs'].get('2010')
    return result


@lru_cache(maxsize=1)
def outcome_model():
    metadata = ROOT / 'reports/outcome_model.json'
    return OutcomeModel.load(ROOT / '.cache/outcome_model.joblib', read_json(metadata) if metadata.exists() else {})


@lru_cache(maxsize=2)
def knowledge_index(snapshot, code_fingerprint, withdrawal_ids):
    return KnowledgeIndex(historical_corpus())


@app.get('/api/research/knowledge-search')
def knowledge_search(q: str = Query(min_length=2, max_length=200), method: Literal['bm25', 'embedding', 'hybrid', 'rerank'] = 'bm25', limit: int = Query(20, ge=1, le=50), collection: str | None = Query(None, max_length=80), condition: str | None = Query(None, max_length=80), role: str | None = Query(None, max_length=80)):
    p = pipeline()
    index = knowledge_index(p.snapshot, p.code_fingerprint, tuple(sorted(p.withdrawn())))
    try:
        return {'query': q, 'method': method, 'results': index.search(q, method=method, limit=limit, collection=collection, condition=condition, role=role), 'scope': 'Ranked historical statements, not answers or evidence of therapeutic benefit. Collection, condition and role filters apply before candidate selection and ranking. Similar wording does not establish medical equivalence.'}
    except (ValueError, RuntimeError) as error:
        raise HTTPException(422, str(error)) from error


class OutcomeRequest(BaseModel):
    model_config = ConfigDict(extra='forbid')
    run_id: str = Field(pattern=r'^[a-f0-9]{24}$')
    paper_id: str = Field(min_length=1, max_length=60)
    evidence: str = Field(min_length=15, max_length=12000)
    outcome: str = Field(min_length=2, max_length=500)
```

```
    intervention: str = Field(min_length=2, max_length=500)
    comparator: str = Field(min_length=2, max_length=500)


class ComparisonRequest(BaseModel):
    model_config = ConfigDict(extra='forbid')
    passage_ids: list[str] = Field(min_length=2, max_length=4)


@app.post('/api/research/compare-sources')
def compare_sources(request: ComparisonRequest):
    if len(set(request.passage_ids)) != len(request.passage_ids):
        raise HTTPException(422, 'Choose different historical passages')
    results = [analyze(AnalysisRequest(passage_id=id, retrieval='bm25')) for id in request.passage_ids]
    taxa = {}
    for result in results:
        for group in result['identity_counterfactuals']:
            for b in group['branches']:
                taxa.setdefault(b['taxon_id'], {'id': b['taxon_id'], 'name': b['name'], 'sources': []})['sources'].append({'source_id': result['source']['id'],
'historical_name': group['label'], 'status': group['status']})
    return {'scope': 'Comparison of recorded uses and candidate branches. Shared candidates or condition terms are research links, not proof of a shared historical
identity or effective treatment.', 'sources': [{'source': r['source'], 'run_id': r['run_id'], 'claims': r['historical_claims'], 'names':
r['identity_counterfactuals'], 'evidence_records': len(r['evidence']), 'abstracts': len({e['paper']['id'] for e in r['evidence']}), 'conditions': r['conditions']}
for r in results], 'candidate_links': list(taxa.values()), 'shared_candidate_ids': [id for id, t in taxa.items() if len({s['source_id'] for s in t['sources']}) > 1],
 'snapshot_sha256': results[0]['snapshot_sha256'], 'code_fingerprint': results[0]['code_fingerprint']}


@app.post('/api/research/outcome-direction')
def predict_outcome(request: OutcomeRequest):
    result = get_run(request.run_id)
    paper = next((e['paper'] for e in result['evidence'] if e['paper']['id'] == request.paper_id), None)
    if not paper:
        raise HTTPException(404, 'Publication is not part of this active analysis')
    if request.evidence not in paper['abstract']:
        raise HTTPException(422, 'Select an exact passage from this publication abstract')
    # A valid quotation does not establish an arbitrary user-entered comparison.
    # Lexical context is a conservative gate, not semantic PICO verification.
    words = set(normalize(paper['abstract']).split())
    stop = {'the', 'and', 'with', 'versus', 'compared', 'to', 'of', 'in', 'a', 'an', 'for', 'group'}
    checks = {}
    for field in ('outcome', 'intervention', 'comparator'):
        terms = {w for w in normalize(getattr(request, field)).split() if len(w) > 2 and w not in stop}
        checks[field] = 'LEXICAL_CONTEXT_PRESENT_NOT_SEMANTICALLY_VERIFIED' if terms and terms & words else 'NOT_ESTABLISHED_IN_ABSTRACT'
    if any(v == 'NOT_ESTABLISHED_IN_ABSTRACT' for v in checks.values()):
        prediction = {'status': 'QUESTION_CONTEXT_NOT_ESTABLISHED', 'direction': None, 'scope': 'The supplied intervention, comparator or outcome could not be linked
 lexically to this abstract. Check the actual study comparison or consult the full text; no direction is proposed.'}
    else:
        prediction = outcome_model().predict(request.evidence, request.outcome, request.intervention, request.comparator)
    prediction['question_context_checks'] = checks
    return {**prediction, 'paper_id': paper['id'], 'supporting_text': request.evidence, 'question': {k: getattr(request, k) for k in ('outcome', 'intervention',
'comparator')}, 'provenance': paper['provenance'], 'licence': paper['licence'], 'stored_as_scientific_fact': False}


@app.get("/api/taxonomy/live")
def taxonomy_live(name: str = Query(min_length=3, max_length=120)):
    try:
        result, provenance = ScientificClient().json("https://api.gbif.org/v1/species/match?" + urlencode({"name": name, "kingdom": "Plantae", "verbose": "true"}))
        return {"result": result, "provenance": provenance, "limitation": "Service match confidence is not historical identity probability; not automatically
```

```
    inserted into local graph."}
425     except Exception as error:
426         raise HTTPException(502, f"Public taxonomy service unavailable: {type(error).__name__}") from error
427 
428 
429 @app.get('/api/taxonomy/verify-scientific')
430 def verify_scientific(name: str = Query(min_length=3, max_length=120)):
431     try:
432         response, provenance = ScientificClient().json('https://verifier.globalnames.org/api/v1/verifications/' + quote(name, safe=''))
433         names = response.get('names', []) if isinstance(response, dict) else response
434         row = names[0] if names else {}
435         candidates = ([row['bestResult']] if row.get('bestResult') else []) + row.get('bestResults', [])
436         return {'query': name, 'match_type': row.get('matchType'), 'results': [{k: c.get(k) for k in ('dataSourceTitleShort', 'matchedName',
    'currentCanonicalSimple', 'taxonomicStatus', 'recordId', 'outlink')} for c in candidates], 'provenance': provenance, 'scope': 'External scientific-name verification,
     not historical vernacular interpretation. No result is automatically attached to the passage or inserted into the local knowledge graph.'}
437     except Exception as error:
438         raise HTTPException(502, 'GNverifier unavailable: ' + type(error).__name__) from error
439 
440 
441 @app.get("/api/citations/{paper_id}/verify-live")
442 def verify_live(paper_id: str):
443     p = pipeline()
444     paper = p.papermap.get(paper_id)
445     if not paper:
446         raise HTTPException(404, "Citation not registered in snapshot")
447     try:
448         if paper_id.startswith('PMID'):
449             row = next(iter(fetch_records(ScientificClient(), [paper['pmid']])), {})
450             checks = {"pmid": row.get("pmid") == paper.get("pmid"), "title": row.get("title") == paper["title"], "doi": row.get("doi") == paper.get("doi"),
    "not_retracted": row.get("retracted") is False}
451             return {"checks": checks, "metadata_match": all(checks.values()), "provenance": paper["provenance"], "entailment": "NOT_ESTABLISHED_BY_METADATA_LOOKUP"}
452         result, provenance = ScientificClient().json("https://www.ebi.ac.uk/europepmc/webservices/rest/search?" + urlencode({"query": f"PMCID:{paper_id}", "format":
    "json", "resultType": "core", "pageSize": 1}))
453         rows = result.get("resultList", {}).get("result", [])
454         row = rows[0] if rows else {}
455         checks = {"pmcid": row.get("pmcid") == paper_id, "title": row.get("title") == paper.get("original_title",paper["title"]), "doi": row.get("doi") ==
    paper.get("doi"), "pmid": row.get("pmid") == paper.get("pmid")}
456         return {"checks": checks, "metadata_match": all(checks.values()), "provenance": provenance, "entailment": "NOT_ESTABLISHED_BY_METADATA_LOOKUP"}
457     except Exception as error:
458         raise HTTPException(502, f"Citation service unavailable: {type(error).__name__}") from error
459 
460 
461 @app.get("/api/exports/{run_id}")
462 def export(run_id: str, format: Literal["json", "csv", "bibtex", "graph", "brief"] = "json"):
463     result = get_run(run_id)
464     if format == "brief":
465         return Response(brief_markdown(result['research_brief']), media_type="text/markdown", headers={"Content-Disposition": f'attachment;
    filename="foliatrace-{run_id}-brief.md"'})
466     if format in ("json", "graph"):
467         value = result if format == "json" else {"graph": result["graph"], "source": result["source"], "researcher_reviews": result["researcher_reviews"],
    "limitations": result["limitations"]}
468         return Response(json.dumps(value, ensure_ascii=False, indent=2), media_type="application/json", headers={"Content-Disposition": f'attachment;
    filename="foliatrace-{run_id}-{format}.json"'})
469     unique = {e["paper"]["id"]: e["paper"] for e in result["evidence"]}
470     if format == "bibtex":
471         def escape(value):
472             return str(value or "").replace("\\", " ").replace("{", "(").replace("}", ")")
473         text = "\n\n".join("@article{" + p["id"] + ",\n" + ",\n".join(f"  {k} = {{{escape(v)}}}" for k, v in [("title", p["title"]), ("author", p["authors"]),
```

```
    ("year", p["year"]), ("doi", p.get("doi")), ("url", p["url"]), ("note", "License: " + p["licence"])]) + "\n}" for p in unique.values())
474         return Response(text, media_type="text/plain", headers={"Content-Disposition": f'attachment; filename="foliatrace-{run_id}.bib"'})
475     output = io.StringIO()
476     writer = csv.writer(output)
477     writer.writerow(["taxon_id", "condition", "identity_qualifier", "identity_review", "paper_id", "title", "evidence_types", "relation", "reported_finding",
    "first_blocking_check", "comparability_screening", "supporting_text", "url", "licence", "evidence_review", "review_reason", "source_rights", "source_id",
    "source_title", "source_url", "source_date", "reviewer", "review_created_at", "review_origin_analysis", "review_retained", "identity_review_notes",
    "historical_statement_comparisons", "candidate_preparation_wording", "study_methods_sources", "author_reported_results", "automated_endpoint_statements",
    "candidate_result_scope"])
478     def safe(value):
479         value = str('' if value is None else value)
480         return "'" + value if value.lstrip().startswith(("=", "+", "-", "@")) else value
481     reviews = {(r["target_kind"], r["target_id"]): r for r in result["researcher_reviews"]}
482     identity_reviews = {}
483     identity_notes = {}
484     for (kind, target), r in reviews.items():
485         if kind == "identity":
486             identity_reviews.setdefault(target.split("::")[-1], []).append(r["decision"] + " (" + target.split("::")[0] + ")")
487             identity_notes.setdefault(target.split('::')[-1], []).append({k: r.get(k) for k in ('target_id', 'decision', 'reason', 'reviewer', 'created_at',
    'origin_run_id', 'carried_forward')})
488     for e in result["evidence"]:
489         review = reviews.get(("evidence", e["id"]))
490         values = [e['taxon_id'], e['condition_id'], e['identity_qualifier'],
491                   '; '.join(identity_reviews.get(e['taxon_id'], [])) or 'UNREVIEWED',
492                   e['paper']['id'], e['paper']['title'], ';'.join(e['study']['types']),
493                   e['relation']['relation'], (e['relation'].get('reported_finding') or {}).get('direction'),
494                   (e.get('comparability') or {}).get('first_blocking_check') or 'NONE',
495                   (e.get('comparability') or {}).get('screening'), e['relation']['supporting_text'],
496                   e['paper']['url'], e['paper']['licence'], review['decision'] if review else 'UNREVIEWED',
497                   review['reason'] if review else '', result['source']['licence']]
498         values += [result['source'].get(k) for k in ('id', 'title', 'url', 'date')]
499         values += [(review or {}).get(k, '') for k in ('reviewer', 'created_at', 'origin_run_id', 'carried_forward')]
500         values.append(json.dumps(identity_notes.get(e['taxon_id'], []), ensure_ascii=False))
501         values.append(json.dumps(e.get('historical_contexts', []), ensure_ascii=False))
502         values.append(json.dumps(e['study'].get('preparation_attribution'), ensure_ascii=False))
503         values.append(json.dumps(e['paper'].get('methods_contexts', []), ensure_ascii=False))
504         values.append(json.dumps(e['paper'].get('reported_results', []), ensure_ascii=False))
505         values.append(json.dumps((e['relation'].get('reported_finding') or {}).get('statements', []), ensure_ascii=False))
506         values.append(json.dumps({k:v for k,v in (e['relation'].get('reported_finding') or {}).items() if k in ('candidate_scope_note','unassigned_statements')},
    ensure_ascii=False))
507         writer.writerow([safe(value) for value in values])
508     return Response(output.getvalue(), media_type="text/csv", headers={"Content-Disposition": f'attachment; filename="foliatrace-{run_id}.csv"'})
509
510
511 def handoff_store():
512     return HandoffStore(pipeline().store.path)
513
514
515 def active_handoff(identifier):
516     record = handoff_store().get(identifier)
517     if not record:
518         raise HTTPException(404, 'Evidence handoff not found')
519     packet = record['packet']
520     withdrawn = pipeline().withdrawn()
521     source = packet['source']
522     references = {source.get('id'), source.get('source_id')} | {s['paper']['id'] for s in packet['studies']} | {c['taxon_id'] for c in packet['candidates']}
523     if references & withdrawn:
524         raise HTTPException(410, 'A handoff source has been withdrawn; create a new reading from current sources')
```

```
    return record


@app.get('/api/handoffs')
def list_handoffs():
    rows = []
    for row in handoff_store().list():
        try:
            active_handoff(row['id'])
            rows.append(row)
        except HTTPException as error:
            if error.status_code != 410:
                raise
    return {'handoffs': rows, 'limit': 100}


@app.post('/api/analyses/{run_id}/handoffs', status_code=201)
def save_handoff(run_id: str, request: HandoffRequest):
    try:
        return handoff_store().put(build_handoff(get_run(run_id), request))
    except (ValueError, KeyError) as error:
        raise HTTPException(422, str(error)) from error


@app.post('/api/handoffs/import', status_code=201)
async def import_handoff(file: UploadFile = File()):
    body = await file.read(20_000_001)
    if len(body) > 20_000_000:
        raise HTTPException(413, 'Handoff exceeds 20 MB')
    try:
        envelope = json.loads(body.decode('utf-8-sig'))
        packet = HandoffPacket.model_validate(envelope['packet']).model_dump()
        if hashlib.sha256(canonical(packet)).hexdigest() != envelope['sha256']:
            raise ValueError('Handoff checksum mismatch; checksum is integrity checking, not authenticity')
        source = packet['source']
        ids = {source.get('id'), source.get('source_id')} | {s['paper']['id'] for s in packet['studies']} | {c['taxon_id'] for c in packet['candidates']}
        if ids & pipeline().withdrawn():
            raise ValueError('Handoff contains withdrawn sources')
    except (UnicodeDecodeError, ValueError, KeyError, TypeError, ValidationError) as error:
        raise HTTPException(422, 'Invalid handoff: '+str(error)) from error
    # Imported packets are separate immutable records, never model output or overwritten notes.
    return handoff_store().put(packet, origin='IMPORTED_UNVERIFIED_RECORD')


@app.get('/api/handoffs/{identifier}')
def read_handoff(identifier: str, format: Literal['json', 'markdown'] = 'json'):
    record = active_handoff(identifier)
    if format == 'markdown':
        prefix = ('Imported record; author and scientific claims are unverified.\n\n' if record['origin'] == 'IMPORTED_UNVERIFIED_RECORD' else '')
        return Response(prefix+handoff_markdown(record['packet']), media_type='text/markdown', headers={'Content-Disposition': f'attachment; filename="foliatrace-
handoff-{record["id"]}.md"'})
    return record


def question_store():
    return QuestionStore(pipeline().store.path)


```

```
def active_question(identifier):
    record = question_store().get(identifier)
    if record is None:
        raise HTTPException(404, 'Research question not found')
    withdrawn = pipeline().withdrawn()
    if any(b['taxon_id'] in withdrawn or any(r['paper']['id'] in withdrawn for r in b['records']) for b in record['packet']['branches']):
        raise HTTPException(410, 'A source in this review was withdrawn; rerun the question with current sources')
    return record


@app.get('/api/research/question-options')
def question_options():
    from src.nlp.extract import CONDITIONS
    return {'topics': [{'id': key, 'label': key.replace('_', ' ')} for key in CONDITIONS],
            'code_fingerprint':pipeline().code_fingerprint,
            'coverage': 'Question-first local search; optional bounded Europe PMC lookup uses public plant names and endpoint phrases only. Specific endpoint wording is a reading check, not diagnostic equivalence.'}


@app.post('/api/research/public-search-plan')
def public_search_plan(protocol:ResearchQuestion):
    from src.evidence.question_search import search_plan
    p=pipeline();resolution=p.resolver.resolve(protocol.plant_name)
    candidates=resolution['candidates'];known={c['taxon_id'] for c in candidates}
    if not set(protocol.candidate_ids)<=known:raise HTTPException(422,'Selected identity is not a candidate for this wording')
    taxa=[p.taxmap[c['taxon_id']] for c in candidates if (not protocol.candidate_ids or c['taxon_id'] in protocol.candidate_ids) and c['taxon_id'] not in p.withdrawn()]
    try:
        return {'queries':search_plan(taxa,protocol.outcome_terms,p.index,protocol.reference_pmids),
                'scope':'Up to 20 PubMed-indexed records per candidate from Europe PMC, plus up to ten supplied reference IDs. No free-text question, product wording or consultation details sent.',
                'offline':p.client.offline,'network_used':False}
    except ValueError as error:
        raise HTTPException(422,str(error)) from error


@app.get('/api/research/questions')
def list_questions():
    # Stored packets were validated on creation/import. With no withdrawals,
    # listing titles need not deserialize every complete literature review.
    if not pipeline().withdrawn():
        return {'reviews':question_store().list(),'limit':100}
    result = []
    for item in question_store().list():
        try:
            active_question(item['id'])
            result.append(item)
        except HTTPException as error:
            if error.status_code != 410:
                raise
    return {'reviews': result, 'limit': 100}


@app.post('/api/research/questions', status_code=201)
def create_question(protocol: ResearchQuestion):
    try:
        return question_store().put(run_question(pipeline(), protocol))
    except (ValueError, RuntimeError) as error:
```

```
        raise HTTPException(422, str(error)) from error


@app.post('/api/research/questions/import', status_code=201)
async def import_question(file: UploadFile = File()):
    raw = await file.read(20_000_001)
    if len(raw) > 20_000_000:
        raise HTTPException(413, 'Research review exceeds 20 MB')
    try:
        record = json.loads(raw.decode('utf-8-sig'))
        payload = {'packet': record['packet'], 'decision_history': record['decision_history']}
        if hashlib.sha256(question_canonical(payload)).hexdigest() != record['sha256']:
            raise ValueError('Research review checksum mismatch')
        validate_packet(record['packet'])
        withdrawn = pipeline().withdrawn()
        if any(b['taxon_id'] in withdrawn or any(r['paper']['id'] in withdrawn for r in b['records']) for b in record['packet']['branches']):
            raise ValueError('Review contains withdrawn sources')
        return question_store().put(record['packet'], origin='IMPORTED_UNVERIFIED_REVIEW', decisions=record['decision_history'])
    except (ValueError, KeyError, TypeError, UnicodeDecodeError) as error:
        raise HTTPException(422, 'Invalid research review: '+str(error)) from error


@app.get('/api/research/questions/{identifier}')
def read_question(identifier: str, format: Literal['json', 'markdown', 'csv'] = 'json'):
    record = active_question(identifier)
    if format == 'markdown':
        return Response(review_markdown(record), media_type='text/markdown', headers={'Content-Disposition': f'attachment; filename="foliatrace-
    question-{record["id"]}.md"'})
    if format == 'csv':
        output = io.StringIO();writer = csv.writer(output)
        writer.writerow(['question', 'plant_wording', 'candidate', 'paper_id', 'title', 'url', 'rights', 'screening', 'checks', 'review_decision', 'review_reason'])
        decisions = {d['record_id']:d for d in record['decision_history']}
        for branch in record['packet']['branches']:
            for row in branch['records']:
                decision = decisions.get(row['id'], {})
                values = [record['packet']['protocol']['question'], record['packet']['protocol']['plant_name'], branch['name'], row['paper']['id'],
    row['paper']['title'], row['paper']['url'], row['paper']['licence'], row['screening']['state'], json.dumps(row['screening']['checks'], ensure_ascii=False),
    decision.get('decision', ''), decision.get('reason', '')]
                writer.writerow(["'"+v if isinstance(v,str) and v.lstrip().startswith(('=','+','-','@')) else v for v in values])
        return Response(output.getvalue(), media_type='text/csv', headers={'Content-Disposition': f'attachment; filename="foliatrace-question-{record["id"]}.csv"'})
    return record


@app.put('/api/research/questions/{identifier}/decisions')
def decide_question(identifier: str, decision: ReadingDecision):
    active_question(identifier)
    try:
        return question_store().decide(identifier, decision)
    except ValueError as error:
        raise HTTPException(422, str(error)) from error


@app.get('/api/product-labels')
def find_product_labels(q:str=Query(min_length=2,max_length=120),mode:Literal['reference','live']='reference',offset:int=Query(default=0,ge=0,le=200),market:Literal[
    'all','on','off']='all'):
    if len(q.strip())<2:raise HTTPException(422,'Enter a product name, brand or barcode')
    try:
        return search_labels(pipeline().client,q.strip(),mode,offset,market)
```

```
    except (ValueError,RuntimeError,OSError) as error:
        raise HTTPException(503,'Label search unavailable: '+str(error)) from error


class LabelComparison(BaseModel):
    model_config=ConfigDict(extra='forbid')
    left:LabelSnapshot
    right:LabelSnapshot


@app.post('/api/product-labels/compare')
def compare_product_labels(request:LabelComparison):
    return compare_labels(request.left.model_dump(),request.right.model_dump())


@app.get('/api/product-labels/{identifier}')
def get_product_label(identifier:int,mode:Literal['reference','live']='reference'):
    if identifier<=0:raise HTTPException(422,'A positive NIH label identifier is required')
    try:
        snap=read_label(pipeline().client,identifier,mode)
        return {'snapshot':snap,'summary':summarize_label(snap['raw']),
                'ingredients':label_ingredients(snap['raw']),'source_text':label_wording(snap['raw'])}
    except (ValueError,RuntimeError,OSError,KeyError,TypeError) as error:
        raise HTTPException(503,'Label record unavailable: '+str(error)) from error


def consultation_store():
    return ConsultationStore(pipeline().store.path)


def active_consultation(identifier):
    record = consultation_store().get(identifier)
    if record is None:
        raise HTTPException(404, 'Consultation preparation not found')
    withdrawn = pipeline().withdrawn()
    for item in record['packet']['products']:
        if any(c['taxon_id'] in withdrawn for c in item['candidates']):
            raise HTTPException(410, 'A name source was withdrawn; prepare a new record')
    for link in record['packet']['linked_evidence']:
        if any(b['taxon_id'] in withdrawn or any(r['paper']['id'] in withdrawn for r in b['records']) for b in link['review']['packet']['branches']):
            raise HTTPException(410, 'A linked evidence source was withdrawn; review current sources')
    return record


@app.get('/api/consultations')
def list_consultations():
    if not pipeline().withdrawn():
        return {'preparations':consultation_store().list(),'limit':100}
    result=[]
    for item in consultation_store().list():
        try:
            active_consultation(item['id']);result.append(item)
        except HTTPException as error:
            if error.status_code != 410:raise
    return {'preparations':result,'limit':100}


@app.post('/api/consultations',status_code=201)
```

```
def create_consultation(request:ConsultationInput):
    try:
        return consultation_store().put(prepare_consultation(pipeline(),request))
    except ValueError as error:
        raise HTTPException(422,str(error)) from error


@app.post('/api/consultations/import',status_code=201)
async def import_consultation(file:UploadFile=File()):
    body=await file.read(20_000_001)
    if len(body)>20_000_000:raise HTTPException(413,'Consultation preparation exceeds 20 MB')
    try:
        record=json.loads(body.decode('utf-8-sig'));packet=record['packet']
        if hashlib.sha256(question_canonical(packet)).hexdigest()!=record['sha256']:raise ValueError('File integrity checksum mismatch')
        validate_consultation(packet)
        withdrawn=pipeline().withdrawn()
        if any(c['taxon_id'] in withdrawn for item in packet['products'] for c in item['candidates']):raise ValueError('A source was withdrawn')
        for link in packet['linked_evidence']:
            if any(b['taxon_id'] in withdrawn or any(r['paper']['id'] in withdrawn for r in b['records']) for b in link['review']['packet']['branches']):raise
ValueError('A linked evidence source was withdrawn')
        return consultation_store().put(packet,origin='IMPORTED_UNVERIFIED_PREPARATION')
    except (ValueError,KeyError,TypeError,UnicodeDecodeError) as error:
        raise HTTPException(422,'Invalid consultation preparation: '+str(error)) from error


class ConsultationEvidenceLink(BaseModel):
    model_config=ConfigDict(extra='forbid')
    item_id:str=Field(pattern=r'^[a-f0-9]{12,32}$')
    review_id:str=Field(pattern=r'^[a-f0-9]{32}$')
    ingredient_key:str|None=Field(default=None,max_length=300)


@app.post('/api/consultations/{identifier}/evidence',status_code=201)
def link_consultation_evidence(identifier:str,request:ConsultationEvidenceLink):
    import copy
    record=active_consultation(identifier);review=active_question(request.review_id)
    try:
        item,ingredient = review_context(record, review, request.item_id, request.ingredient_key)
    except ValueError as error:
        raise HTTPException(422,str(error)) from error
    packet=copy.deepcopy(record['packet'])
    label=item['reported'].get('label_snapshot')
    packet['linked_evidence']=[v for v in packet['linked_evidence'] if not (v['item_id']==request.item_id and v['review']['id']==request.review_id)]
    packet['linked_evidence'].append({'item_id':request.item_id,'source_item_sha256':item['item_sha256'],'review':review,
                                      **({'ingredient_key':request.ingredient_key,'source_label_sha256':label['sha256']} if label else {}),
                                      'scope':'PROFESSIONAL_READING_NOT_PRODUCT_SAFETY_VERIFICATION'})
    packet['created_at']=now()
    return consultation_store().put(packet)


@app.post('/api/consultations/{identifier}/clarifications',status_code=201)
def answer_consultation_clarification(identifier:str,request:ClarificationAnswer):
    try:
        return consultation_store().put(add_clarification(active_consultation(identifier),request))
    except ValueError as error:
        raise HTTPException(422,str(error)) from error


```

```
@app.get('/api/consultations/{identifier}/study-bridge')
def compare_consultation_study(identifier:str,review_id:str=Query(pattern=r'^[a-f0-9]{32}$'),
                               item_id:str=Query(pattern=r'^[a-f0-9]{12,32}$'),
                               record_id:str=Query(pattern=r'^[a-f0-9]{24}$'),
                               ingredient_key:str|None=Query(default=None,max_length=300)):
    try:
        return study_bridge(active_consultation(identifier),active_question(review_id),item_id,record_id,ingredient_key)
    except ValueError as error:
        raise HTTPException(422,str(error)) from error


@app.post('/api/consultations/{identifier}/share-preview')
def preview_consultation_share(identifier:str,request:ShareSelection):
    try:
        return share_preview(active_consultation(identifier),request)
    except ValueError as error:
        raise HTTPException(422,str(error)) from error


@app.get('/api/consultations/{identifier}')
def read_consultation(identifier:str,format:Literal['json','markdown','summary','html']='json'):
    record=active_consultation(identifier)
    if format=='markdown':
        return Response(consultation_markdown(record),media_type='text/markdown',headers={'Content-Disposition':f'attachment; filename="foliatrace-
consultation-{record["id"]}.md"'})
    if format=='summary':
        return Response(summary_markdown(record),media_type='text/markdown',headers={'Content-Disposition':f'attachment; filename="foliatrace-
conversation-{record["id"]}.md"'})
    if format=='html':
        return Response(summary_html(record),media_type='text/html',headers={
            'Content-Disposition':f'inline; filename="foliatrace-conversation-{record["id"]}.html"',
            'Content-Security-Policy':"default-src 'none'; style-src 'unsafe-inline'; base-uri 'none'; form-action 'none'; frame-ancestors 'none'"})
    return record


@app.get("/")
@app.get("/research")
def home():
    return FileResponse(ROOT / "frontend" / "index.html")


app.mount("/assets", StaticFiles(directory=ROOT / "frontend"), name="assets")

# The browser checker (NIH statements x medicines, OCR, pharmacist QR) is the same static app as the
# public demo. Build it with scripts/sync_public_demo.py (and build_care_demo.py for recorded cases).
_CHECKER = ROOT / "sites" / "foliatrace-research" / "build"
if (_CHECKER / "index.html").exists():
    app.mount("/check", StaticFiles(directory=_CHECKER, html=True), name="checker")
```

src/api/contracts.py
SHA-256: 63e83874940bc7c31baa9ee61ca0f210da721f06e38480f55c5abaaaf0cbc145

```python
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field, model_validator


class Span(BaseModel):
    text: str
    start: int = Field(ge=0)
    end: int = Field(gt=0)

    @model_validator(mode='after')
    def ordered(self):
        if self.end <= self.start:raise ValueError('Invalid half-open source span')
        return self


class Mention(Span):
    id: str
    normalized: str
    method: str


class Relation(BaseModel):
    relation: Literal['SUPPORTS','CONTRADICTS','MIXED','IRRELEVANT','INSUFFICIENT']
    supporting_text: str | None
    rationale: str
    directness: str
    mechanistic_relation: str | None = None
    reported_relation: str | None = None


class AnalysisRecord(BaseModel):
    model_config=ConfigDict(extra='forbid')
    run_id: str = Field(pattern=r'^[a-f0-9]{24}$')
    created_at: str
    pipeline_version: str
    snapshot_sha256: str = Field(pattern=r'^[a-f0-9]{64}$')
    knowledge_sha256: str | None = Field(default=None, pattern=r'^[a-f0-9]{64}$')
    code_fingerprint: str
    model_fingerprint: str
    source: dict
    input: str
    mentions: list[Mention]
    historical_claims: list[dict]
    local_names_not_in_any_list: list[str] = []
    resolutions: list[dict]
    conditions: list[dict]
    evidence: list[dict]
    conflicts: list[dict]
    blocked_citations: list[dict]
    cache_hit: bool
    limitations: list[str]
    configuration: dict
    models: dict
    graph: dict
    performance: dict
```

```
    identity_counterfactuals: list[dict] = []
    historical_identity_references: dict[str, list[dict]] = {}
    researcher_reviews: list[dict] = []
    research_brief: dict = {}
    compound_assay_leads: list[dict] = []
    study_reading_sets: list[dict] = []
    language: str = 'en'
    literature_access: dict = {}
    research_digest: list[dict] = []
    plant_literature: dict = {}

    @model_validator(mode='after')
    def integrity(self):
        for mention in self.mentions:
            if self.input[mention.start:mention.end]!=mention.text:raise ValueError('Mention/source span mismatch')
        for evidence in self.evidence:
            Relation.model_validate(evidence['relation'])
            if evidence['identity_qualifier']!='CONDITIONAL_ON_CANDIDATE_IDENTITY':raise ValueError('Identity uncertainty was lost')
            support=evidence['relation']['supporting_text']
            if support and support not in evidence['paper']['abstract']:raise ValueError('Unsupported output span')
            finding = evidence['relation'].get('reported_finding') or {}
            for statement in finding.get('statements', []) + finding.get('unassigned_statements', []):
                start, end = statement['start'], statement['end']
                if not 0 <= start < end <= len(evidence['paper']['abstract']) or evidence['paper']['abstract'][start:end] != statement['supporting_text']:
                    raise ValueError('Endpoint finding/abstract span mismatch')
            if evidence['citation']['status']=='REJECTED':raise ValueError('Rejected citation cannot enter an API response')
            for context in evidence.get('historical_contexts', []):
                span = context['support']
                if self.input[span['start']:span['end']] != span['text']:
                    raise ValueError('Historical statement comparison/source span mismatch')
            attribution = evidence['study'].get('preparation_attribution') or {}
            methods = {c['id']: c for c in evidence['paper'].get('methods_contexts', [])}
            for form in attribution.get('forms', []) + attribution.get('other_preparation_wording', []):
                if form.get('source_kind') == 'FULL_TEXT_METHODS_TABLE':
                    source = methods.get(form.get('source_id'))
                    if not source or source.get('source_kind') != 'FULL_TEXT_METHODS_TABLE' or source['paper_id'] != evidence['paper']['id']:
                        raise ValueError('Preparation table source mismatch')
                    for span in (form, form['name_span'], form['context']):
                        if span.get('source_id') != source['id'] or source['text'][span['start']:span['end']] != span['text']:
                            raise ValueError('Preparation/table transcription span mismatch')
                elif evidence['paper']['abstract'][form['start']:form['end']] != form['text']:
                    raise ValueError('Preparation wording/abstract span mismatch')
            for check in (evidence.get('comparability') or {}).get('checks', []):
                span = check.get('span') or {}
                if span.get('source_kind', '').startswith('FULL_TEXT_METHODS'):
                    context = methods.get(span.get('source_id'))
                    if not context or context.get('source_kind') != span['source_kind'] or context['paper_id'] != evidence['paper']['id'] or
context['text'][span['start']:span['end']] != span['text']:
                        raise ValueError('Methods check/source span mismatch')
        ids={n['id'] for n in self.graph['nodes']}
        for references in self.historical_identity_references.values():
            for reference in references:
                text = reference['input']
                for excerpt in reference['excerpts']:
                    if text[excerpt['start']:excerpt['end']] != excerpt['text']:
                        raise ValueError('Historical name reference/source span mismatch')
                    for name in excerpt['names']:
                        if not (excerpt['start'] <= name['start'] < name['end'] <= excerpt['end']) or text[name['start']:name['end']] != name['text']:
```

```
                            raise ValueError('Historical name reference/name span mismatch')
        if any(e['subject'] not in ids or e['object'] not in ids or not e.get('provenance') for e in self.graph['edges']):raise ValueError('Graph integrity failure')
        return self
```

src/common.py
SHA-256: f10efc63910f70f25705ea3f3704c68ceb787a7edc6538903eb4c4454598baa7

```
from __future__ import annotations

import hashlib
import json
import os
import tempfile
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data" / "demo"


def now() -> str:
    return datetime.now(timezone.utc).isoformat()


def digest(value: str | bytes) -> str:
    return hashlib.sha256(value.encode("utf-8") if isinstance(value, str) else value).hexdigest()


def read_json(path: Path):
    return json.loads(path.read_text(encoding="utf-8"))


def write_json(path: Path, value):
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = None
    try:
        with tempfile.NamedTemporaryFile(mode='w',encoding='utf-8',newline='\n',dir=path.parent,prefix=path.name+'.',suffix='.tmp',delete=False) as handle:
            temporary=Path(handle.name)
            handle.write(json.dumps(value,ensure_ascii=False,indent=2)+'\n')
        os.replace(temporary,path)
    finally:
        if temporary is not None:temporary.unlink(missing_ok=True)
```

src/evidence/__init__.py
SHA-256: dc112d9ace039b17603140da1eccf5c20a1145d1ba39deb8323f3ce99b46373f

```
    1 """Study design, claim-relative relations and source verification."""
```

src/evidence/brief.py
SHA-256: f688cc335f12b5dd4cbdf282889e0728a3c28be015cc51ea03b4aa54fffb5ae1

```
"""Research hand-off derived from active records, never an efficacy recommendation.

No new scientific inference is made here. Every reading-list entry points to an
existing evidence record; review decisions annotate rather than rewrite it.
"""
from collections import Counter
import re
from urllib.parse import urlsplit, quote
from src.evidence.readings import active_reading_sets


def research_brief(result):
    reviews = {(r['target_kind'], r['target_id']): r for r in result.get('researcher_reviews', [])}
    names = []
    for group in result.get('identity_counterfactuals', []):
        candidates = [{
            'taxon_id': b['taxon_id'], 'name': b['name'],
            'review': reviews.get(('identity', f"{group['key']}::{b['taxon_id']}")),
        } for b in group['branches']]
        accepted = [c for c in candidates if (c['review'] or {}).get('decision') == 'ACCEPT']
        state = ('CONFLICTING_ACCEPTS' if len(accepted) > 1 else
                 'ONE_ACCEPT_RECORDED' if accepted else 'IDENTITY_REVIEW_NEEDED')
        ids = {eid for b in group['branches'] for eid in b['evidence_ids']}
        rows = [e for e in result['evidence'] if e['id'] in ids]
        claims = [c for c in result['historical_claims'] if c['id'] in group['claim_ids']]
        actions = []
        if not candidates:
            actions.append({'kind': 'EXPAND_IDENTITY_SEARCH', 'text': 'Look beyond the bundled taxonomy snapshot. No candidate here does not mean the historical plant is unknown.', 'evidence_ids': []})
        elif state == 'CONFLICTING_ACCEPTS':
            actions.append({'kind': 'RECONCILE_IDENTITY_REVIEWS', 'text': 'More than one candidate is accepted. Check whether the source names several plants or whether these reviews conflict; keep the branches separate.', 'evidence_ids': []})
        elif not accepted:
            actions.append({'kind': 'REVIEW_HISTORICAL_IDENTITY', 'text': 'Check the source description, edition and historical context against the candidate names. Record a judgement with a reason before attaching studies to this plant.', 'evidence_ids': []})
        else:
            actions.append({'kind': 'CHECK_RECORDED_IDENTITY', 'text': f"A researcher accepted {accepted[0]['name']}. Read the recorded reason; this judgement has not changed the automated results or established historical identity.", 'evidence_ids': []})
        if group['open_world']:
            actions.append({'kind': 'EXTEND_TAXON_COVERAGE', 'text': 'Investigate the additional GBIF name-sharing species below. Their studies were not retrieved; modern occurrence records do not establish the historical referent.', 'evidence_ids': []})
        uncertain = [c for c in claims if c.get('negated') or c.get('relation_scope') in ('MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION', 'PARAGRAPH_CONTEXT_UNCERTAIN_RELATION')]
        if uncertain:
            actions.append({'kind': 'CHECK_SOURCE_CLAUSE', 'text': 'Re-read the marked clauses and surrounding paragraphs: negation, several plants or a change of subject may prevent assigning this use to one plant. Literature linked through an uncertain subject stays conditional on that reading.', 'claim_ids': [c['id'] for c in uncertain], 'evidence_ids': []})
        needs_identity = [e['id'] for e in rows if not any(c['check'] == 'identity' and c['status'] == 'PASS' for c in (e.get('comparability') or {}).get('checks', []))]
        if needs_identity:
            actions.append({'kind': 'VERIFY_STUDY_SPECIES', 'text': 'Verify the organism in the full text of the linked records. A shared common name or an unscreened record does not establish the study species.', 'evidence_ids': needs_identity})
        named = [e['id'] for e in rows if e['id'] not in needs_identity]
        if named:
            actions.append({'kind': 'APPRAISE_STUDY_SCOPE', 'text': 'Read the species-named studies and check population, intervention, preparation, outcome and
```

```
   study quality. Passing a lexical species check is not evidence of efficacy.', 'evidence_ids': named})
46         if not rows or group['open_world'] or any(not b['evidence_ids'] for b in group['branches']):
47             actions.append({'kind': 'SEARCH_BEYOND_SNAPSHOT', 'text': 'Search additional literature for the empty or uncovered candidate branches. This bundled
   snapshot is not a systematic review; no retrieved record is not a negative result.', 'evidence_ids': []})
48         reading = []
49         for e in rows:
50             checks = (e.get('comparability') or {}).get('checks', [])
51             reading.append({
52                 'evidence_id': e['id'], 'taxon_id': e['taxon_id'],
53                 'taxon_name': next(c['name'] for c in candidates if c['taxon_id'] == e['taxon_id']),
54                 'condition_id': e['condition_id'],
55                 'paper': {k: e['paper'].get(k) for k in ('id', 'title', 'url', 'year', 'licence')},
56                 'species_check': next((c['status'] for c in checks if c['check'] == 'identity'), 'NOT_ASSESSED'),
57                 'failed_checks': [c['check'] for c in checks if c['status'] == 'FAIL'],
58                 'unresolved_checks': [c['check'] for c in checks if c['status'] not in ('PASS', 'FAIL')],
59                 'screening': (e.get('comparability') or {}).get('screening', 'NOT_ASSESSED'),
60                 'final_relation': e['relation']['relation'],
61                 'historical_contexts': e.get('historical_contexts', []),
62                 'study_preparation': (e['study'].get('pico_source_spans') or {}).get('intervention_preparation'),
63                 'preparation_attribution': e['study'].get('preparation_attribution'),
64                 'methods_sources': [context for context in e['paper'].get('methods_contexts', []) if any((c.get('span') or {}).get('source_id') == context['id'] for
   c in checks) or context['id'] in e['study'].get('preparation_attribution', {}).get('methods_context_ids', [])],
65                 'reported_results': e['paper'].get('reported_results', []),
66                 'automated_endpoint_statements': (e['relation'].get('reported_finding') or {}).get('statements', []),
67                 'candidate_result_scope': {k:v for k,v in (e['relation'].get('reported_finding') or {}).items() if k in
   ('candidate_scope_note','unassigned_statements')},
68                 'comparison_notes': (e['relation'].get('reported_finding') or {}).get('comparison_notes', []),
69                 'review': reviews.get(('evidence', e['id'])),
70             })
71         # Triage only: species-named first, rejected last, then date. Not an evidence-quality score.
72         reading.sort(key=lambda r: ((r['review'] or {}).get('decision') == 'REJECT', r['species_check'] != 'PASS', -int(r['paper'].get('year') or 0),
   r['evidence_id']))
73         names.append({
74             'key': group['key'], 'label': group['label'], 'status': group['status'],
75             'source_names': [n['name'] for n in group.get('source_names', [])],
76             'opening_name_clues': [clue for resolution in result.get('resolutions', []) if resolution['mention_id'] in group['mention_ids'] for clue in
   resolution.get('source_common_names', [])],
77             'historical_identity_references': result.get('historical_identity_references', {}).get(group['key'], []),
78             'identity_review_state': state, 'candidates': candidates,
79             'source_claims': [{'id': c['id'], 'text': c['support']['text'], 'start': c['support']['start'], 'end': c['support']['end'], 'preparation':
   c.get('preparation', []), 'part': c.get('part', []), 'route': c.get('route', []), 'relation_scope': c.get('relation_scope'), 'statement_role':
   c.get('statement_role', 'LEGACY_ROLE_NOT_SCREENED'), 'negated': bool(c.get('negated'))} for c in claims],
80             'next_actions': actions, 'reading_list': reading,
81             'trial_readings': active_reading_sets(result.get('study_reading_sets', []), [c['taxon_id'] for c in candidates], group['conditions']),
82             'open_world': group['open_world'], 'single_answer_risk': group['single_answer_view'],
83             'relation_counts': dict(Counter(e['relation']['relation'] for e in rows)),
84             'compound_assays': [a for a in result.get('compound_assay_leads', []) if any(link['evidence_id'] in ids for link in a['linked_evidence'])],
85         })
86     return {
87         'schema_version': '1.4', 'run_id': result['run_id'],
88         'source': result['source'], 'snapshot_sha256': result['snapshot_sha256'], 'knowledge_sha256': result.get('knowledge_sha256'),
89         'code_fingerprint': result['code_fingerprint'],
90         'scope': 'Automated research hand-off from active source, candidate and abstract records. Actions are verification tasks, not treatment advice or validated
   research findings.',
91         'reading_order': 'Species-named records first, researcher-rejected records last, then newest year; not a ranking of scientific quality.',
92         'names': names,
93         'research_assessments': [{k: row.get(k) for k in ('taxon_id', 'taxon', 'condition_id', 'topic', 'state', 'human_studies', 'isolated_plant_studies',
   'assessment_basis')} for row in result.get('research_digest', [])],
```

```
        'limitations': result['limitations'],
    }


def _text(value):
    """Keep source/user text literal in Markdown, including attempted headings/links."""
    return re.sub(r'([\\`*_{}\[\]<>#!|])', r'\\\1', ' '.join(str(value or '').split()))


def _url(value):
    value = str(value or '')
    try:
        parts = urlsplit(value)
        if parts.scheme in ('https', 'http') and parts.netloc and not any(ord(c) < 32 for c in value):
            return quote(value, safe='/:#?&=%+@!$;,~-._')
    except ValueError:
        pass
    return ''


def brief_markdown(brief):
    source = brief['source']
    lines = ['# FoliaTrace research brief', '', brief['scope'], '',
             f"Source: {_text(source.get('title'))}", f"Source ID: {_text(source.get('id'))}",
             f"Edition/date: {_text(source.get('date') or 'not established')}",
             f"Source rights: {_text(source.get('licence'))}"]
    if url := _url(source.get('url')):
        lines.append(f"[Original source]({url})")
    lines += ['', f"Run: {brief['run_id']}", f"Snapshot: {brief['snapshot_sha256']}", f"Code: {brief['code_fingerprint']}", '', brief['reading_order']]
    if not brief['names']:
        lines += ['', 'No plant name recognised in this snapshot. Check the source manually or expand coverage; no historical identity or efficacy conclusion follows.']
    for name in brief['names']:
        lines += ['', f"## {_text(name['label'])}", '', f"Identity state: {_text(name['status'])}", f"Review state: {_text(name['identity_review_state'])}", '', '### Source statements', '']
        if name.get('source_names'):
            lines[-2:-2] = [f"Named in this source: {_text(', '.join(name['source_names']))} (the author's own pairing with this name)", '']
        for claim in name['source_claims']:
            lines += [f"> {_text(claim['text'])}", f"Claim {_text(claim['id'])}; role {_text(claim.get('statement_role', 'LEGACY_ROLE_NOT_SCREENED'))}; scope {_text(claim['relation_scope'])}; negation flagged: {claim['negated']}", '']
            if 'start' in claim:
                lines.append(f"Original source span: {claim['start']}\u2013{claim['end']}. Preparation: {_text(', '.join(p['text'] for p in claim.get('preparation', [])) or 'not stated')}; part: {_text(', '.join(p['text'] for p in claim.get('part', [])) or 'not stated')}; route: {_text(', '.join(p['text'] for p in claim.get('route', [])) or 'not stated')}.")
        if not name['source_claims']:
            lines.append('No use statement extracted for this name.')
        lines += ['', '### Candidate identities', '']
        if name.get('opening_name_clues'):
            lines += ['Source-local opening names; name-list matches remain conditional, not historical identifications.']
            for clue in name['opening_name_clues']:
                lines += [f"> {_text(clue['name'])}", f"Original input span {clue['start']}\u2013{clue['end']}; lookup forms: {_text('; '.join(clue['lookup_forms']))}."]
            lines.append('')
        for c in name['candidates']:
            lines.append(f"- {_text(c['name'])} ({_text(c['taxon_id'])})")
            if r := c['review']:
                lines.append(f"  Researcher review: {_text(r['decision'])}; {_text(r['reason'])}; {_text(r['reviewer'])}; {_text(r['created_at'])}")
                if r.get('carried_forward'):
```

```
                    lines.append(f"  Retained from analysis {_text(r['origin_run_id'])}; exact source wording, name occurrences and candidate identity match. This is
 the original judgement, not a new review.")
        for c in name['open_world']:
            lines.append(f"- Outside snapshot: {_text(c['name'])}; studies not retrieved. GBIF https://www.gbif.org/species/{int(c['gbif_key'])}")
        if references := name.get('historical_identity_references'):
            lines += ['', '### Historical name wording in other sources', '', 'Matching headings are reading links, not evidence that the two authors meant the same
plant. Candidate ranking is unchanged.']
            for reference in references:
                source = reference['source']
                lines += ['', f"{_text(source['title'])}; {_text(source.get('collection'))}; {_text(source.get('date'))}; {_text(source['licence'])}."]
                if url := _url(source.get('url')):
                    lines.append(f'[Original historical source]({url})')
                for excerpt in reference['excerpts']:
                    lines += [f"> {_text(excerpt['text'])}", f"Source input span {excerpt['start']}\u2013{excerpt['end']}; text SHA256
{_text(reference['text_sha256'])}."]
                    for wording in excerpt['names']:
                        lines.append(f"Name wording \u201c{_text(wording['text'])}\u201d; exact {_text(wording['alias_kind'])} spelling for
{_text(wording['candidate_name'])} ({_text(wording['candidate_id'])}) in the bundled taxonomy. Read negation and contrasts in the quote.")
                if not reference['excerpts']:
                    lines.append('Matching chapter heading, but no current candidate scientific-name spelling was found. This does not establish absence of a
historical identification.')
        risk = name['single_answer_risk']
        if risk and (risk['hidden_branches'] or name['open_world']):
            lines += ['', f"Single-answer reconstruction: {len(risk['hidden_papers'])} publications hidden, {len(risk['misattributed_papers'])} potentially
misattributed. This is a reconstruction of choosing one branch, not a measured competitor error rate."]
        for reading_set in name.get('trial_readings', []):
            lines += ['', '### Selected controlled-trial reading comparison', '', _text(reading_set['title']),
                      _text(reading_set['takeaway']), '', _text(reading_set['selection']),
                      _text(reading_set['annotation']['method']), _text(reading_set['annotation']['licence']), _text(reading_set['annotation']['rights_scope']),
                      'Topical reading notes only: not a historical identity decision or an automated efficacy relation.', '']
            for study in reading_set['studies']:
                publication = study['publication']
                url = _url(publication['url'])
                lines.append(f"#### {_text(study['label'])}")
                lines.append(f"[{_text(publication['title'])}]({url})" if url else _text(publication['title']))
                lines.append(f"Authors: {_text(publication['authors'])}. PMID {_text(publication['pmid'])}; DOI {_text(publication['doi'])}.")
                for key, label in [('population', 'Population'), ('design', 'Design'), ('intervention', 'Formulation'), ('comparator', 'Comparator'), ('follow_up',
'Follow-up'), ('primary_outcome', 'Primary endpoint'), ('primary_result', 'Primary result'), ('other_results', 'Other results'), ('reading_question', 'Reading
question')]:
                    lines.append(f"- {label}: {_text(study[key])}")
                lines.append('')
        lines += ['', '### Next research tasks', '']
        for i, a in enumerate(name['next_actions'], 1):
            lines.append(f"{i}. {_text(a['text'])}")
            if a['evidence_ids']:
                lines.append('   Records: ' + ', '.join(_text(eid) for eid in a['evidence_ids']))
        lines += ['', '### Reading and verification queue', '']
        if not name['reading_list']:
            lines.append('No records retrieved in this snapshot.')
        printed_methods = set()
        printed_results = set()
        printed_aims = set()
        for r in name['reading_list']:
            p = r['paper']
            title = _text(re.sub(r'</?(?:i|b|em|strong|sup|sub)>', '', p['title'], flags=re.I))
            citation = f"[{title}]({url})" if (url := _url(p['url'])) else title
            lines += [f"- {citation} ({_text(p['year'])}; {_text(p['id'])}; {_text(p['licence'])})",
                      f"  Record {_text(r['evidence_id'])}; candidate {_text(r['taxon_name'])}; question {_text(r['condition_id'])}.",
```

```
                    f"  Species check: {_text(r['species_check'])}; final relation: {_text(r['final_relation'])}.",
                    f"  Failed checks: {_text(', '.join(r['failed_checks']) or 'none recorded')}; unresolved: {_text(', '.join(r['unresolved_checks']) or 'none recorded')}; screening: {_text(r['screening'])}."]
            if r.get('preparation_attribution'):
                lines.append(f"  Candidate-linked preparation wording: {_text((r.get('study_preparation') or {}).get('text') or 'not established')}; status {_text(r['preparation_attribution']['status'])}. This wording does not confirm the administered intervention or isolate its effect.")
            for note in r.get('comparison_notes', []):
                lines.append('  Comparator reading: ' + _text(note))
            for statement in r.get('automated_endpoint_statements', []):
                lines += [f"  Automated endpoint reading: {_text(statement['direction'])}; abstract span {statement['start']}-{statement['end']}.", f"  > {_text(statement['supporting_text'])}"]
            scope = r.get('candidate_result_scope') or {}
            if scope.get('candidate_scope_note'):
                lines.append('  Candidate linkage: ' + _text(scope['candidate_scope_note']))
            for statement in scope.get('unassigned_statements', []):
                lines += [f"  Source finding not assigned to this candidate (abstract span {statement['start']}-{statement['end']}):", f"  > {_text(statement['supporting_text'])}"]
            if r.get('preparation_attribution'):
                forms = r['preparation_attribution'].get('forms', [])
                if forms:
                    chosen = next((f for f in forms if f['start'] == (r.get('study_preparation') or {}).get('start')), forms[0])
                    kind = 'Methods table transcription' if chosen.get('source_kind') == 'FULL_TEXT_METHODS_TABLE' else 'Abstract'
                    lines.append(f"  {kind} context: \u201c{_text(chosen['context']['text'])}\u201d (local span {chosen['context']['start']}\u2013{chosen['context']['end']}).")
            for context in r.get('reported_results', []):
                if context['id'] in printed_results:
                    lines.append(f"  Author-reported Results already quoted above: {_text(context['id'])}.")
                    continue
                printed_results.add(context['id'])
                url = _url(context['url'])
                lines += [f"  Author-reported Results, for this study's own intervention and population ({_text(context['licence'])}; {_text(context['id'])}):",
                          f"  > {_text(context['text'])}", f"  [Original Results section]({url})" if url else '  Results source URL unavailable.',
                          '  This passage is not a condition-specific relation label or proof that the historical preparation works. Read the design and comparator checks above.']
            if r.get('historical_contexts'):
                for context in r['historical_contexts']:
                    lines.append(f"  Source statement {_text(context['claim_id'])} (span {context['support']['start']}\u2013{context['support']['end']}): {_text(context['preparation_comparison']['detail'])}")
                    if indication := context.get('indication_comparison'):
                        historical = ', '.join(dict.fromkeys(span['text'] for span in indication['historical_wording'])) or 'not established by this word screen'
                        modern = ', '.join(dict.fromkeys(span['text'] for span in indication['study_wording'])) or 'not established by this word screen'
                        lines.append(f"  Indication wording, not diagnostic equivalence: historical {_text(historical)}; study {_text(modern)}. {_text(indication['detail'])}")
                        if (aim := indication.get('study_aim')) and p['id'] not in printed_aims:
                            printed_aims.add(p['id'])
                            lines.append(f"  Stated study aim (original abstract span {aim['start']}\u2013{aim['end']}): {_text(aim['text'])}")
            else:
                lines.append('  No statement-level preparation comparison was stored in this analysis; reanalyze the source to obtain one.')
            for context in r.get('methods_sources', []):
                if context['id'] in printed_methods:
                    lines.append(f"  Methods source already quoted above: {_text(context['id'])}.")
                    continue
                printed_methods.add(context['id'])
                url = _url(context['url'])
                lines += [f"  Methods source: {_text(context['section'])}; {_text(context['licence'])}; {_text(context['id'])}.",
                          f"  > {_text(context['text'])}", f"  [Original Methods section]({url})" if url else '  Methods source URL unavailable.']
            if review := r['review']:
                lines.append(f"  Researcher review: {_text(review['decision'])}; {_text(review['reason'])}; {_text(review['reviewer'])};
```

```
    {_text(review['created_at'])}")
246                 if review.get('carried_forward'):
247                     lines.append(f"  Retained from analysis {_text(review['origin_run_id'])}; exact source wording, species, condition and publication match. This is
     the original judgement, not a new review.")
248         if name.get('compound_assays'):
249             lines += ['', '### Compound assay leads', '', 'ChEMBL-derived assay fields in this section retain CC BY-SA 3.0. Attribution: ChEMBL team, EMBL-EBI and
    the original document IDs below. A publication co-mention is not proof of plant composition; an isolated-molecule assay is not clinical efficacy.', '']
250             for a in name['compound_assays']:
251                 url = _url(a['url'])
252                 lines += [f"- {_text(a['compound'])} against {_text(a['target']['pref_name'])}: {_text(a['standard_type'])} {_text(a['standard_relation'])}
    {_text(a['standard_value'])} {_text(a['standard_units'])}.", f"  {_text(a['assay_id'])}; {_text(a['document_id'])}; {_text(a['licence'])}.", f"  [Measurement
    record]({url})" if url else '  Measurement URL unavailable.']
253     if brief.get('research_assessments'):
254         lines += ['', '## Research summaries and their sources', '']
255         for row in brief['research_assessments']:
256             basis = row.get('assessment_basis') or {}
257             lines += [f"### {_text(row['taxon'])}: {_text(row['topic'])}",
258                       f"Reported state: {_text(row['state'])}. Basis: {_text(basis.get('kind', 'LEGACY_NOT_RECORDED'))}.",
259                       f"Human studies: {_text(row['human_studies'])}; studies without another named plant ingredient: {_text(row.get('isolated_plant_studies'))}.",
260                       _text(basis.get('scope', ''))]
261             for record in basis.get('records', []):
262                 title, url = _text(record['title']), _url(record['url'])
263                 citation = f'[{title}]({url})' if url else title
264                 lines.append(f"- {citation} ({_text(record.get('year'))}; {_text(record['paper_id'])}): {_text(record['direction'])}.")
265             lines.append('')
266     lines += ['', '## Limits', ''] + ['- ' + _text(x) for x in brief['limitations']]
267     return '\n'.join(lines) + '\n'
```

src/evidence/calibration.py
SHA-256: 5a8a03d78f2464730616ff5e42771c68a00aeac22c19975acd4a9b76544d59de

```
"""Held-out study-design calibration; never a historical identity probability."""
import numpy as np
from scipy.optimize import minimize_scalar
from scipy.special import softmax


def temperature_scale(probabilities, temperature):
    return softmax(np.log(np.clip(probabilities,1e-9,1)) / temperature,axis=1)


def fit_temperature(probabilities, expected):
    def loss(t):
        p=temperature_scale(probabilities,t)
        return -np.log(np.clip(p[np.arange(len(expected)),expected],1e-9,1)).mean()
    return float(minimize_scalar(loss,bounds=(.3,5),method='bounded').x)


def calibration_metrics(probabilities, expected, bins=10):
    confidence=probabilities.max(axis=1);correct=probabilities.argmax(axis=1)==expected
    onehot=np.eye(probabilities.shape[1])[expected]
    rows=[];ece=0.0
    for i in range(bins):
        selected=(confidence>=i/bins)&(confidence<(i+1)/bins if i<bins-1 else confidence<=1)
        n=int(selected.sum())
        acc=float(correct[selected].mean()) if n else None
        conf=float(confidence[selected].mean()) if n else None
        if n:ece+=n/len(expected)*abs(acc-conf)
        rows.append({'lower':i/bins,'upper':(i+1)/bins,'n':n,'accuracy':acc,'mean_confidence':conf})
    return {'n':len(expected),'multiclass_brier':float(((probabilities-
   onehot)**2).sum(axis=1).mean()),'ece_10_bins':float(ece),'reliability_bins':rows,'limitation':'Small silver-labelled study-design split; not historical identity,
   clinical efficacy or expert calibration.'}
```

src/evidence/care_workspace.py
SHA-256: d61b8b018cbfdb0b2e5355086082dd209e574507064e603a9b5a8d33a87bdb55

```
"""Source-scoped clarification, product/study comparison and selective sharing.

These views never certify product contents or translate study signals into advice.
"""
from __future__ import annotations

import copy
import hashlib
from html import escape
from typing import Literal
from urllib.parse import urlsplit

from pydantic import BaseModel, ConfigDict, Field, model_validator

from src.common import now
from src.evidence.question_review import canonical
from src.evidence.consultation_summary import handoff_summary, SCOPE
from src.nlp.extract import normalize


class ClarificationAnswer(BaseModel):
    model_config = ConfigDict(extra='forbid')
    item_id: str = Field(pattern=r'^[a-f0-9]{12,32}$')
    field: str = Field(min_length=1, max_length=80)
    status: Literal['ANSWER_RECORDED', 'ASK_PROFESSIONAL']
    note: str = Field(min_length=5, max_length=2000)
    source: Literal['PERSON_REPORTED', 'PACKAGING_REPORTED', 'PROFESSIONAL_NOTE']
    recorded_by: str = Field(min_length=1, max_length=80)

    @model_validator(mode='after')
    def substantive(self):
        if len(self.note.strip()) < 5 or not self.recorded_by.strip():
            raise ValueError('Record an answer or next question and who recorded it')
        return self


def validate_clarifications(packet):
    entries = packet.get('clarification_history', [])
    if not isinstance(entries, list) or len(entries) > 500:
        raise ValueError('Clarification history must contain at most 500 entries')
    products = {i['reported']['item_id']: i for i in packet['products']}
    for entry in entries:
        answer = ClarificationAnswer.model_validate({k: v for k, v in entry.items()
                                                     if k not in ('created_at', 'source_item_sha256')})
        item = products.get(answer.item_id)
        if item is None or entry.get('source_item_sha256') != item['item_sha256']:
            raise ValueError('Clarification belongs to another product version')
        if answer.field not in {q['field'] for q in item['clarification_questions']}:
            raise ValueError('Clarification does not match an original question')
        if not isinstance(entry.get('created_at'), str) or not entry['created_at']:
            raise ValueError('Clarification date is missing')


def add_clarification(record, answer):
    if record['origin'] != 'LOCAL_CONSULTATION_PREPARATION':
```

```
        raise ValueError('Imported records remain unverified; prepare a new local record first')
    packet = copy.deepcopy(record['packet'])
    item = next((i for i in packet['products'] if i['reported']['item_id'] == answer.item_id), None)
    if item is None or answer.field not in {q['field'] for q in item['clarification_questions']}:
        raise ValueError('Choose a clarification from this saved product')
    packet.setdefault('clarification_history', []).append({**answer.model_dump(), 'created_at': now(),
                                                           'source_item_sha256': item['item_sha256']})
    packet['created_at'] = now()
    validate_clarifications(packet)
    return packet


def review_context(record, review, item_id, ingredient_key=None):
    """One source-binding rule shared by comparison and evidence attachment."""
    if record['origin'] != 'LOCAL_CONSULTATION_PREPARATION' or review['origin'] != 'LOCAL_QUESTION_REVIEW':
        raise ValueError('Imported records must be rerun locally before creating a current evidence link')
    item = next((i for i in record['packet']['products'] if i['reported']['item_id'] == item_id), None)
    if item is None:
        raise ValueError('This product is not in the saved preparation')
    label = item['reported'].get('label_snapshot')
    ingredient = None
    if label:
        ingredient = next((r for r in item.get('label_ingredients', []) if r['key'] == ingredient_key), None)
        if ingredient is None:
            raise ValueError('Select the specific label ingredient before linking evidence')
        expected_name, expected_source = ingredient.get('research_lookup') or ingredient['lookup'], ingredient['review_source_text']
        candidates = ingredient['candidates']
    else:
        if ingredient_key is not None:
            raise ValueError('A manually recorded product has no attached label ingredient')
        expected_name, expected_source = item['name_lookup'], item['reported']['source_text']
        candidates = item['candidates']
    protocol = review['packet']['protocol']
    if normalize(protocol['plant_name']) != normalize(expected_name):
        raise ValueError('Evidence question name differs from this product or label ingredient')
    if protocol['source_text'] != expected_source:
        raise ValueError('The evidence question source differs from this product record; open a question from the saved product')
    taxa = {c['taxon_id'] for c in candidates}
    branches = {b['taxon_id'] for b in review['packet']['branches']}
    if not branches or not branches <= taxa:
        raise ValueError('The evidence question does not stay within this product\'s candidate identities')
    return item, ingredient


def study_bridge(record, review, item_id, record_id, ingredient_key=None):
    item, ingredient = review_context(record, review, item_id, ingredient_key)
    selected = [(b, r) for b in review['packet']['branches'] for r in b['records'] if r['id'] == record_id]
    if len(selected) != 1:
        raise ValueError('Choose a study from this evidence question')
    branch, row = selected[0]
    checks = {c['check']: c for c in row['screening']['checks']}
    reported = item['reported']
    protocol = review['packet']['protocol']
    dims = []
    def add(key, label, product, requested, check=None, caution=''):
        found = checks.get(check or key)
        dims.append({'key': key, 'label': label, 'product': product or 'Not provided',
                     'question_requirement': requested or 'Not specified',
```

```
                    'status': found['status'] if found else 'UNKNOWN',
                    'detail': found['detail'] if found else caution or 'Not established in the available study checks.',
                    'caution': caution, 'spans': found.get('spans', []) or ([found['span']] if found and found.get('span') else []) if found else []})
    add('identity', 'Plant identity', ingredient['name'] if ingredient else item['name_lookup'],
        branch['name'], caution='A study naming this candidate does not identify the contents of the bottle.')
    add('preparation', 'Form and preparation', reported['preparation'], protocol['preparation'],
        caution='A capsule is a product form, not proof of its extraction method or equivalence to a trial formulation.')
    add('amount', 'Actual amount and dose', reported['dose'], 'No equivalent-dose inference',
        caution='The reported amount is preserved. Dose, extract strength, serving basis and study dose have not been matched.')
    add('population', 'Study population', 'Individual clinical circumstances not assessed', protocol['population'],
        caution='Human-study signals do not establish applicability to this person.')
    add('single_plant', 'Single plant or mixture', 'Label formula retained' if ingredient else 'Complete formula not verified',
        protocol['intervention'], caution='A constituent finding cannot establish the effect of the whole commercial product.')
    add('comparator', 'Comparison group', 'Not a product characteristic', protocol['comparator'])
    add('requested_outcome', 'Outcome being investigated', reported['reason_for_use'], '; '.join(protocol['outcome_terms']),
        caution='Shared outcome wording is a reading lead, not a finding of benefit.')
    label = reported.get('label_snapshot')
    return {'schema': 'foliatrace-product-study-bridge-1', 'product_name': reported['name'],
            'product_sha256': item['item_sha256'], 'consultation_id': record['id'], 'review_id': review['id'],
            'record_id': row['id'], 'candidate': branch['name'], 'ingredient': ingredient['name'] if ingredient else None,
            'source_label': {'id': label['label_id'], 'url': label['source_url'], 'sha256': label['sha256']} if label else None,
            'paper': {k: row['paper'].get(k) for k in ('id', 'title', 'url')}, 'dimensions': dims,
            'unresolved': sum(d['status'] in ('UNKNOWN', 'PARTIAL') for d in dims),
            'scope_conflicts': [d['label'] for d in dims if d['status'] == 'FAIL'],
            'scope': 'Source comparison for professional reading. Product equivalence and clinical suitability are not established.'}


class ShareSelection(BaseModel):
    model_config = ConfigDict(extra='forbid')
    item_ids: list[str] = Field(default_factory=list, max_length=30)
    include_medicines: bool = False
    include_allergies: bool = False
    include_questions: bool = False
    include_reading: bool = False
    include_clarifications: bool = False


def share_projection(record, selection):
    """Build an allowlisted document. Omitted material never enters its payload."""
    packet = record['packet']
    products = {p['reported']['item_id']: p for p in packet['products']}
    if not selection.item_ids or len(set(selection.item_ids)) != len(selection.item_ids) or not set(selection.item_ids) <= products.keys():
        raise ValueError('Select one or more different products from this saved preparation')
    summary = handoff_summary(record)
    rows = {p['reported']['item_id']: s for p, s in zip(packet['products'], summary['products'])}
    result = {'schema': 'foliatrace-selected-handoff-1', 'title': 'Herbal-product conversation',
              'saved_at': record['saved_at'], 'origin': summary['origin'], 'scope': SCOPE,
              'sharing_scope': 'Selected information only. Omitted products, medicines or allergies must not be interpreted as absent.',
              'products': []}
    for key in selection.item_ids:
        # Explicit allowlist prevents future summary fields from becoming shared.
        fields = ('name','brand','use','actual_use','stated_preparation','reported_reason',
                  'source_text','label','ingredients','other_ingredients','clarifications')
        row = {field: copy.deepcopy(rows[key][field]) for field in fields}
        if selection.include_reading:
            row['reviews'] = copy.deepcopy(rows[key]['reviews'])
        if selection.include_clarifications:
            row['clarification_notes'] = [copy.deepcopy(n) for n in packet.get('clarification_history', []) if n['item_id'] == key]
```

```
        result['products'].append(row)
    if selection.include_medicines:
        result['other_medicines'] = {'status': summary['medicine_status'], 'details': summary['medicines']}
    if selection.include_allergies:
        result['allergies'] = {'status': summary['allergy_status'], 'details': summary['allergies']}
    if selection.include_questions:
        result['questions'] = summary['questions']
        result['next_questions'] = summary['next_questions']
    return result


def selected_html(projection):
    e = lambda v: escape(str(v), quote=True)
    def p(v): return '<p>'+e(v).replace('\n', '<br>')+'</p>'
    def h(v, level=2): return f'<h{level}>'+e(v)+f'</h{level}>'
    def link(url, title):
        try: parsed = urlsplit(url)
        except ValueError: return p(title)
        if parsed.scheme not in ('http', 'https') or not parsed.hostname: return p(title)
        return '<p><a rel="noreferrer noopener" href="'+e(url)+'">'+e(title)+'</a></p>'
    out = ['<!doctype html><html lang="en"><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">',
           '<title>FoliaTrace \xb7 Selected conversation handoff</title>',
           '<style>body{font:16px/1.55 system-ui;color:#193e32;max-width:780px;margin:36px auto;padding:0 24px}h1{font-size:32px}h2{margin-top:30px}p,li{overflow-wrap:anywhere}.scope{padding:16px;border:1px solid #d2ddd4;background:#f1f5ee}section{border-top:1px solid #d2ddd4;margin-top:24px;padding-top:12px}a{color:#17634c}small{color:#516258}@media print{body{font-size:10pt;margin:0}h2,h3{break-after:avoid}section{break-inside:auto}@page{size:A4;margin:15mm}}</style><body><main>',
           h('Your pharmacist conversation',1),p(projection['origin']),'<div class="scope">',p(projection['sharing_scope']),p(projection['scope']),'</div>',
           p('Prepared from saved information dated '+projection['saved_at'])]
    if 'questions' in projection:
        out += [h('Questions to discuss'), p(projection['questions'] or 'No additional question recorded')]
        out += [p(q) for q in projection.get('next_questions', [])]
    for key,title in [('other_medicines','Other medicines'),('allergies','Allergies / previous reactions')]:
        if key in projection: out += [h(title),p(projection[key]['status']),p(projection[key]['details'])]
    for row in projection['products']:
        out += ['<section>',h(row['name']),p(row['use']),p('Brand: '+(row['brand'] or 'Not provided'))]
        out += [p(k.replace('_',' ').capitalize()+': '+v) for k,v in row['actual_use'].items()]
        out += [p('Stated form: '+row['stated_preparation']),p('Reported reason: '+row['reported_reason'])]
        if row['label']:
            out += [link(row['label']['url'],'NIH label '+str(row['label']['id'])+' \xb7 bottle match not established'),h('Declared ingredients \xb7 not actual use',3)]
            for ing in row['ingredients']:
                out += [p(('Within blend: ' if ing['nested'] else '')+ing['name']+': '+('; '.join(ing['amounts']) or 'Individual amount not provided')),p(ing['forms']),p(ing['notes'])]
            other=row.get('other_ingredients') or {}
            out += [p('Other declared ingredients: '+(other.get('text') or '; '.join(i['name'] for i in other.get('ingredients',[])) or 'Not provided'))]
        else: out += [h('Original wording',3),p(row['source_text'] or 'Not provided')]
        out += [h('Still to clarify',3),'<ul>']+['<li>'+e(q)+'</li>' for q in row['clarifications']]+['</ul>']
        for note in row.get('clarification_notes',[]):
            out += [p(note['field']+' \xb7 '+note['status'].replace('_',' ').lower()+': '+note['note']),p('Source: '+note['source']+' \xb7 recorded by '+note['recorded_by']+' \xb7 '+note['created_at'])]
        for reading in row.get('reviews',[]):
            out += [h('Professional reading \xb7 not product approval',3),p(reading['question']),p(reading['coverage_note']),p(str(reading['unreviewed_records'])+' candidate/study records have no recorded human decision.')]
            if not reading['decisions']: out += [p('No human reading decision recorded.')]
            for decision in reading['decisions']:
                out += [link(decision['url'],decision['title']),p(decision['decision']+': '+decision['reason']),p('Recorded by: '+decision['reviewer'])]
        out += ['</section>']
    out += [p('Bring original packaging and the complete medicine list. This selected document is not a portable backup.'),'</main></body></html>']
```

```
    return '\n'.join(out)


def share_preview(record, selection):
    projection = share_projection(record, selection)
    return {'selection': selection.model_dump(), 'projection': projection,
            'sha256': hashlib.sha256(canonical(projection)).hexdigest(), 'html': selected_html(projection)}
```

src/evidence/chemistry.py
SHA-256: 1d04180429ee6271e4021757446eaa8cf5c043d802d5275b58dbf0f7e5eacf30

```
"""Assay leads attached only through publication compound co-mentions."""


def assay_leads(evidence, records, withdrawn=()):
    withdrawn, found = set(withdrawn), {}
    if 'chembl' in withdrawn:
        return []
    for e in evidence:
        for c in e.get('compound_co_mentions', []):
            for a in records:
                if a['cid'] != c['cid'] or {a['id'], a['molecule_chembl_id'], a['target']['target_chembl_id']} & withdrawn:
                    continue
                row = found.setdefault(a['id'], {**a, 'linked_evidence': [], 'link_basis': 'COMPOUND_CO_MENTION_IN_RETRIEVED_PUBLICATION; NOT_VERIFIED_PLANT_CONSTITUENT'})
                row['linked_evidence'].append({'evidence_id': e['id'], 'paper_id': e['paper']['id'], 'candidate_taxon_id': e['taxon_id'], 'supporting_text': c['supporting_text']})
    return list(found.values())
```

src/evidence/classify.py
SHA-256: cec73bc2763a5d7b579b95752f7421f0eb6ef518b2516f553a08697950bcd3a5

```
from __future__ import annotations

import re
from functools import lru_cache
from src.nlp.extract import LATIN_BINOMIAL, LATIN_EPITHET, NOT_GENUS, NOT_EPITHET, RESEARCH_TERMS, research_terms, normalize, sentences, has_research_topic_wording

ANIMAL_WORDS = r'\b(?:rodents?|rats?|mice|mouse|murine|hamsters?|gerbils?|guinea pigs?|zebrafish|fish|trout|carp|tilapia|salmon|broilers?|chickens?|hens?|quail|pigs?|piglets?|swine|sheep|lambs?|ewes?|cattle|calves|cows?|goats?|horses?|rabbits?|bees?|shrimp|dogs?|canine|cats?|feline|drosophila|caenorhabditis elegans)\b'
HUMAN_WORDS = r'\b(?:patients?|humans?|volunteers?|participants?|women|men|adults?|adolescents?|children|infants?|boys|girls)\b'
ANIMAL_PLANT_NAMES = re.compile(r"\b(?:horse[- ]chestnuts?|horse[- ]mints?|cat(?:['\u2019]s|s['\u2019]?)?\s+(?:claws?|whiskers)|goat(?:['\u2019]s|s['\u2019]?)?\s+rue|sheep(?:['\u2019]s|s['\u2019]?)?\s+sorrel|lamb(?:['\u2019]s|s['\u2019]?)?\s+quarters|dog[- ]roses?)\b", re.I)


def animal_signal(text):
    """Animal words inside a plant name do not identify experimental animals."""
    return re.search(ANIMAL_WORDS, ANIMAL_PLANT_NAMES.sub(lambda m:' ' * len(m.group()), text), re.I)


TYPE_RULES = [
    ("META_ANALYSIS", r"\bmeta[- ]analys(?:is|es)\b"),
    ("SYSTEMATIC_REVIEW", r"\bsystematic (?:literature )?review\b"),
    ("REVIEW", r"\b(?:this|a|the present|narrative|comprehensive|scoping) review\b"),
    ("IN_VITRO", r"\bin[- ]vitro\b|\bcell lines?\b|\bcultured cells\b"),
    ("IN_VIVO_ANIMAL", ANIMAL_WORDS),
    ("CLINICAL_TRIAL", r"\brandomi[sz]ed\b|\bdouble[- ]blind\b|\bplacebo[- ]controlled\b|\bclinical trial\b|\bsplit[- ]face (?:clinical )?study\b"),
    ("OBSERVATIONAL_HUMAN", r"\bcohort study\b|\bcase[- ]control study\b|\bcross[- ]sectional study\b"),
]
META_TYPES = {"Randomized Controlled Trial": "CLINICAL_TRIAL", "Controlled Clinical Trial": "CLINICAL_TRIAL", "Clinical Trial": "CLINICAL_TRIAL", "Clinical Trial, Phase II": "CLINICAL_TRIAL", "Clinical Trial, Phase III": "CLINICAL_TRIAL", "Clinical Trial, Phase IV": "CLINICAL_TRIAL", "Observational Study": "OBSERVATIONAL_HUMAN", "Meta-Analysis": "META_ANALYSIS", "Systematic Review": "SYSTEMATIC_REVIEW", "Review": "REVIEW"}


def classify_study(paper, model=None):
    title, abstract = paper["title"], paper["abstract"]
    metadata = {v for k, v in META_TYPES.items() if k in paper.get("publication_types", [])}
    # PubMed indexing occasionally tags a single trial as a meta-analysis; the text must agree.
    if abstract and not re.search(r"meta[- ]?analy|pooled|systematic", title + " " + abstract, re.I):
        metadata -= {"META_ANALYSIS"}
    metadata = sorted(metadata)
    signals = []
    # Do not promote trial/animal terms in a review's background to primary experiments.
    is_review = bool(set(metadata) & {"REVIEW", "SYSTEMATIC_REVIEW", "META_ANALYSIS"}) or bool(re.search(r"\breview\b|meta.analysis", title, re.I))
    for label, pattern in TYPE_RULES:
        for sentence in sentences(title + ". " + abstract):
            matched = animal_signal(sentence['text']) if label == 'IN_VIVO_ANIMAL' else re.search(pattern, sentence['text'], re.I)
            if matched:
                if is_review and label in ("IN_VITRO", "IN_VIVO_ANIMAL", "CLINICAL_TRIAL", "OBSERVATIONAL_HUMAN"):
                    continue
                signals.append({"type": label, "supporting_text": sentence["text"], "method": "TEXT_RULE"})
                break
    types = sorted(set(metadata + [s["type"] for s in signals])) or ["UNKNOWN"]
    animal = bool(animal_signal(title + ' ' + abstract))
    human = bool(re.search(HUMAN_WORDS, title + " " + abstract, re.I))
    methods = re.search(r'\bmethods?\s*:\s*(.*?)(?=\b(?:results?|conclusions?)\s*:|$)', abstract, re.I | re.S)
    method_text = methods.group(1) if methods else ' '.join(s['text'] for s in sentences(abstract) if
```

```
    re.search(r'\b(?:random\w*|allocated|assigned|divided|treated|administered|fed|enrolled|recruited)\b', s['text'], re.I))
    animal_methods = bool(animal_signal(method_text))
    human_methods = bool(re.search(HUMAN_WORDS, method_text, re.I))
    if animal_methods and not human_methods and not is_review:
        # A human disease in Background does not change the experimental population.
        human = False
    split_face = re.search(r'\bsplit[- ]face (?:clinical )?study\b', abstract, re.I) if not animal and not is_review else None
    human = human or bool(split_face)
    if animal and not human and not is_review:
        types = sorted((set(types) - {"UNKNOWN", "CLINICAL_TRIAL", "OBSERVATIONAL_HUMAN"}) | {"IN_VIVO_ANIMAL"})
    if "Clinical Trial Protocol" in paper.get("publication_types", []) or re.search(r"\bprotocol\b", title, re.I):
        types = ["STUDY_PROTOCOL"]
    prediction = model.predict(title + " " + abstract) if model else {"label": "NOT_TRAINED", "used": False}
    # Learned design predictions are triage proposals. A high internal score alone
    # must not promote an otherwise unverified article into human clinical evidence.
    if types == ["UNKNOWN"] and prediction.get("used"):
        prediction = {**prediction, "verification": "MODEL_PROPOSAL_ONLY_REQUIRES_REVIEW"}
    sample = re.search(r"\b(?:n\s*=\s*|(?:a total of\s+)?)(\d{2,5})\s+(patients|participants|subjects|volunteers)\b", abstract, re.I)
    pico = {}
    for key, pattern in [("population", r"\b(?:adult|healthy|older|elderly|female|male|pregnant)?\s*(?:patients|participants|volunteers|rats|mice)\b"),
("intervention_preparation", r"\b(?:aqueous|ethanolic|standardized|standardised)?\s*(?:extract|tea|infusion|oil|capsules?)\b"), ("comparator", r"\b(?:placebo|control
 group|standard care|usual care|active comparator|waiting[- ]list|untreated control)\b")]:
        match = re.search(pattern, abstract, re.I)
        pico[key] = {"text": match.group().strip(), "start": match.start() + len(match.group()) - len(match.group().lstrip()), "end": match.end()} if match else None
    if split_face and not pico['population']:
        pico['population'] = {'text': split_face.group(), 'start': split_face.start(), 'end': split_face.end(), 'reading_scope': 'Within-person design wording;
participant eligibility, sample and allocation require Methods reading. Randomization is not inferred.'}
    return {"types": types, "signals": signals, "metadata_types": metadata, "local_model": prediction, "population_status": "HUMAN_TEXT_SIGNAL" if human else
"ANIMAL_TEXT_SIGNAL" if animal else "UNDETERMINED", "sample_size": int(sample[1]) if sample else None, "sample_size_support": sample.group() if sample else None,
"pico_source_spans": pico, "quality": "NOT_APPRAISED", "limitations": ["Automated design classification; no full clinical risk-of-bias appraisal."]}


# Introductory framing: what a plant "is used for" or "is known to do" is background, not a result.
BACKGROUND = re.compile(r"\b(?:is|are|has been|have been|has long been|have long
been)\s+(?:a\s+|an\s+)?(?:\w+\s+){0,2}?(?:used|known|believed|considered|reputed|reported to|popular|employed)\b|\btraditionally\b|\bfolk
(?:medicine|remed\w*)\b|\bfor centuries\b|\b(?:is|are) an? (?:\w+ ){0,2}(?:herb|plant|remedy|species|spice|shrub|tree)s?\b", re.I)
# Words that mark a sentence as the study's own result, anywhere in the abstract.
RESULT_CUE = re.compile(r"\b(?:results?|conclu\w*|we (?:found|observed|show\w*|demonstrat\w*)|found|showed|shown|demonstrat\w*|revealed|significant\w*|compared
(?:with|to)|versus|than (?:placebo|controls?)|insufficient evidence|no evidence|did not|no difference)\b", re.I)
# Past-tense effect verbs count without an explicit cue only after the opening sentence.
EFFECT_VERB = re.compile(r"\b(?:reduced|improved|increased|decreased|alleviated|ineffective)\b", re.I)
BENEFIT_VERB =
r"reduced|reducing|improved|improving|decreased|decreasing|alleviated|alleviating|relieved|relieving|ameliorated|attenuated|lowered|accelerated|promoted"
# Materials and assay vocabulary: "reduced swelling capacity" is a property, not a symptom.
NOT_A_SYMPTOM = r"(?!\s+(?:capacity|ratio|index|degree|rate|behaviou?r|propert\w+|kinetics))"


AIM_SENTENCE = re.compile(r"^(?:(?:background|objectives?|aims?|purpose|introduction|methods?|design|setting)\s*:|(?:the\s+)?(?:aim|objective|purpose|goal)s?\b|we
(?:aimed|sought|investigated|evaluated|assessed)\b|this (?:study|trial|review) (?:aimed|sought|evaluated|assessed|investigated)\b|to
(?:evaluate|assess|investigate|determine|examine|compare)\b)", re.I)
BENEFIT_STATEMENT =
re.compile(r"\b(?:significant(?:ly)?\s+(?:\w+\s+){0,2}(?:reduc\w*|improv\w*|decreas\w*|lower\w*|alleviat\w*|better|greater|superior|positive|beneficial)|more
effective than|superior to|better than placebo|as effective as|similar (?:effectiveness|efficacy) to|non-?inferior|(?:was|were|is|are|be|appears?(?: to
be)?|seems?(?: to be)?)\s+(?:\w+\s+){0,2}(?:an?\s+)?(?:safe and )?(?:effective|efficacious|beneficial|useful|helpful)|effective
(?:in|for|at|treatment|herb|remedy|option|alternative|agent|therapy)|improved (?:significantly|more)|greater
(?:\w+\s+){0,3}(?:reduction|improvement|decrease)|(?:remarkable|marked|overall|clinically (?:meaningful|relevant)) improvements?|(?:modest|beneficial|positive)
(?:\w+\s+)?(?:effect|activity|benefit)s?|(?:can|may|might|could) (?:\w+\s+)?(?:help )?(?:improve|reduce|relieve|alleviate|lower)|can be considered in the
(?:prevention|treatment)|(?:results?|data|findings|evidence) (?:\w+\s+){0,2}supports?\b|efficacy (?:was|has been) (?:shown|demonstrated|confirmed))\b", re.I)
```

```
NO_EFFECT = r"(?:no|not|without)\s+(?:a\s+)?(?:statistically\s+)?(?:clinically\s+)?(?:significant(?:ly)?|effective|benefit|better|difference|effect|superior|improvement|meaningful|relevant)(?!\s+(?:\w+\s+)?(?:adverse|side|toxic\w*|safety|serious|harm\w*|complication\w*))|(?:did|does|do) not (?:improve|reduce|differ|show|significantly|support|appear to|prevent|relieve|alleviate|lower|yield|provide (?:\w+\s+){0,3}(?:benefit|evidence|relief|indication|improvement)|reveal|demonstrate|result in|affect|change|alter)|cannot (?:reduce|improve|relieve|alleviate|prevent)|ineffective|failed to (?:improve|reduce|show|provide|demonstrate|support)|no better than placebo|no more effective|lack of (?:efficacy|effect|benefit)|not (?:be )?supported|minimal beneficial effects?|no evidence (?:of|for|that) (?:\w+\s+){0,3}(?:effect|efficacy|benefit)"
# "More research is needed" closes most abstracts; it does not unsay the result before it.
FUTURE_WORK = re.compile(r"\b(?:more|further|additional|larger|future)\s+(?:\w+\s+){0,3}(?:studies|research|trials?|investigations?)\b", re.I)
WEAK_EVIDENCE = r"insufficient|inconclusive|limited evidence|not enough|no convincing evidence|conflicting|equivocal|(?:has|have) not been (?:established|proven|proved|confirmed)|remains? (?:unproven|unclear|uncertain)|uncertain whether"
# No inactive arm: "no difference between the groups" then means similar to the active drug.
INACTIVE_ARM = re.compile(r"\bplacebo\b|\bsham\b|\bcontrol group\b|\bcontrols\b|\busual care\b|\bno treatment\b|\buntreated\b", re.I)
GROUP_COMPARISON = re.compile(r"\b(?:between|among)\s+(?:the\s+)?(?:two\s+|both\s+|three\s+)?(?:study\s+|treatment\s+)?groups\b|\bboth groups\b", re.I)
VERDICT_SENTENCES = 3
# A closing sentence states the result in the present tense or a hedge. Earlier in an abstract the
# same wording is background or an aim, so these count only inside the verdict.
VERDICT_BENEFIT = re.compile(r"\b(?:(?:appear(?:s|ed)?|seem(?:s|ed)?) to (?:\w+\s+)?(?:lessen|decrease|reduce|improve|relieve|alleviate)|(?<!\ba )(?<!\bthe )(?:improves?|reduces?|alleviates?|relieves?|decreases?|lessens?|lowers|ameliorates?)\s+(?:\w+\s+){0,4}?(?:symptoms?|severity|scores?|pain|quality|function|intensity|frequency|duration|levels?)|provides? an effective|comparable (?:efficacy|effectiveness)|(?:possess\w*|show\w*|exert\w*|ha[sd]|have|produce[sd]?)\s+(?:\w+\s+){0,3}(?:favou?rable|beneficial|protective|therapeutic|clinically meaningful|positive)\s+(?:\w+\s+){0,2}effects?|(?:may|can|could) be (?:a |an )?(?:\w+\s+){0,3}(?:safe|effective|proper|suitable|appropriate|valuable|useful)\s+(?:\w+\s+){0,3}(?:drug|treatment|option|therapy|remedy|alternative|adjuvant)|supported by (?:sufficient|good|strong|moderate) evidence|promising evidences? for|can help|helps? (?:in )?(?:reliev|reduc|improv|alleviat)\w*)\b", re.I)
PRESENT_BENEFIT_VERB = r"reduces?|improves?|decreases?|alleviates?|relieves?|ameliorates?|attenuates?|lowers"
OPEN_QUESTION = re.compile(r"\b(?:whether|to determine|to assess|to evaluate|to confirm|is needed|are needed|warranted|claims?|viewed with caution)\b", re.I)
# Registration and funding lines close many abstracts and say nothing about the result.
ADMINISTRATIVE = re.compile(r"\b(?:trial registration|registered|registration (?:number|date)|clinicaltrials\.gov|prospero|funding|funded by|copyright|ethic\w* approval)\b|\xa9", re.I)
CONCLUSION_LABEL = re.compile(r"\b(?:conclusions?(?:\s+(?:and|&)\s+(?:relevance|significance))?|interpretation|implications? for practice|authors' conclusions)\s*:", re.I)
ASSOCIATION_BENEFIT = re.compile(r"\bassociated with (?:a )?(?:lower|reduced|decreased) (?:risk|rate|incidence)\b(?!\s+of\s+(?:adverse|side|toxic|harm))", re.I)
# Analysis diagnostics are not treatment outcomes. They can be statistically
# null while the pooled treatment effect is beneficial (or harmful).
ANALYSIS_DIAGNOSTIC = re.compile(r"\b(?:no|not|without)\s+(?:a\s+)?(?:statistically\s+)?significant\s+(?:interaction|heterogeneity|publication bias|association (?:with|between) study|difference (?:between|across|among) (?:the )?subgroups)\b|\b(?:meta[- ]regression|heterogeneity|publication bias|interaction (?:with|between)|subgroup analys\w*)\b", re.I)
SAFETY_OUTCOME = re.compile(r'\b(?:adverse (?:events?|effects?|reactions?)|side effects?|tolerability|well tolerated|safe(?:ty)?|toxic(?:ity)?)\b', re.I)
WITHIN_GROUP = re.compile(r'\b(?:from|relative to|compared (?:with|to)|versus|vs\.?)\s+(?:the\s+)?baseline\b|\bwithin[- ]group\b', re.I)
MISSING_COMPARISON = re.compile(r'\bno\s+(?:between[- ]group|controlled|comparative)\s+(?:\w+\s+){0,2}comparison\b|\bno between[- ]group comparison\b', re.I)
BETWEEN_GROUP = re.compile(r'\b(?:than|versus|compared (?:with|to)|relative to)\s+(?:the\s+)?(?:placebo|controls?)\b|\bbetween[- ]group\b', re.I)


@lru_cache(maxsize=128)
def other_outcomes(terms):
    """Topic vocabulary excluding this endpoint; safety has its own screen."""
    target = set(terms)
    alternatives = {word for values in RESEARCH_TERMS.values() for word in values if word not in target}
    return re.compile(r'\b(?:' + '|'.join(re.escape(w) for w in sorted(alternatives, key=len, reverse=True)) + r')\b', re.I)


def outcome_scope(text, terms):
    if has_research_topic_wording(normalize(text), terms):
        return 'TARGET'
    if SAFETY_OUTCOME.search(text) or other_outcomes(tuple(terms)).search(text):
        return 'OTHER'
    return 'UNSPECIFIED'


```

```
def endpoint_clauses(sentence, terms):
    """Separate coordinated endpoint results without manufacturing new source text."""
    boundaries = list(re.finditer(r'\s+(?:but|whereas|while|however)\s+|[,;]\s*(?:and|but|whereas|while|however)\s+|\s+and\s+(?=(?:had|has|showed|was|were|did|there)\b)', sentence['text'], re.I))
    if not boundaries:
        return [sentence]
    starts = [0] + [m.end() for m in boundaries]
    ends = [m.start() for m in boundaries] + [len(sentence['text'])]
    parts = []
    for left, right in zip(starts, ends):
        segment = sentence['text'][left:right]
        left += len(segment) - len(segment.lstrip())
        right -= len(segment) - len(segment.rstrip())
        if left < right:
            parts.append({'text':sentence['text'][left:right], 'start':sentence['start'] + left, 'end':sentence['start'] + right})
    # An adjective/qualifier fragment belongs to the following endpoint:
    # "blood pressure ..., and had a beneficial, but not significant effect on
    # cognition" must keep "beneficial, but not significant" together.
    index = 0
    while index < len(parts) - 1:
        if outcome_scope(parts[index]['text'], terms) == 'UNSPECIFIED':
            left, right = parts[index]['start'], parts[index + 1]['end']
            parts[index:index + 2] = [{'text':sentence['text'][left - sentence['start']:right - sentence['start']], 'start':left, 'end':right}]
        else:
            index += 1
    # A clause without an endpoint can still refer to the same endpoint. Split
    # only when both sides independently name an outcome (including safety).
    if len(parts) > 1 and all(outcome_scope(p['text'], terms) != 'UNSPECIFIED' for p in parts):
        return parts
    return [sentence]


def botanical_mentions(text, known_genera=None):
    """Recognised genera distinguish binomials from phrases such as 'The number'."""
    return {normalize(n) for n in LATIN_BINOMIAL.findall(text)
            if LATIN_EPITHET.search(normalize(n).split()[-1]) and n.split()[0] not in NOT_GENUS and normalize(n).split()[-1] not in NOT_EPITHET
            and (known_genera is None or normalize(n).split()[0] in known_genera)}


def candidate_result_scope(paper, taxon, findings, known_genera=None):
    """A paper co-mention is not enough to assign another intervention's result."""
    if not taxon.get('name'):
        return findings, None
    scientific = {normalize(taxon['name'])} | {normalize(a['name']) for a in taxon.get('aliases', []) if a['kind'] in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH')}
    genus = normalize(taxon.get('genus') or taxon['name'].split()[0])
    title = normalize(paper['title'])
    titled_species = {normalize(n) for n in LATIN_BINOMIAL.findall(paper['title']) if normalize(n).split()[0] == genus and LATIN_EPITHET.search(normalize(n).split()[-1]) and normalize(n).split()[-1] not in {'extract','extracts','oil','oils','tea','leaves','root','roots','herbal','herbs','species'}}
    if titled_species and not titled_species & scientific:
        return [], 'The title identifies a different species of this genus; a background co-mention cannot assign its result to this candidate.'
    names = scientific | {normalize(a['name']) for a in taxon.get('aliases', []) if a['kind'] == 'VERNACULAR' and len(a['name']) >= 4}
    wording = re.compile(r'\b(?:' + '|'.join(re.escape(n) for n in sorted(names,key=len,reverse=True)) + r')\b', re.I)
    other_titled_plants = botanical_mentions(paper['title'], known_genera) - scientific
    single_named_subject = wording.search(title) and not other_titled_plants
    assigned = []
```

```
    for finding in findings:
        text = finding[1]['text']
        explicit = wording.search(normalize(text))
        other_plants = botanical_mentions(text, known_genera) - scientific
        if explicit or single_named_subject and not other_plants:
            assigned.append(finding)
    if len(assigned) != len(findings):
        return assigned, 'The title does not anchor this candidate. Only result statements explicitly naming the candidate are assigned; generic or other-intervention findings remain unassigned.'
    return findings, None


def study_sentences(text):
    """Preserve decimal statistics inside modern result sentences and exact spans."""
    start = 0
    for boundary in re.finditer(r'\n+|[.!?;]+(?=\s|$)', text):
        # Botanical author initials and prose abbreviations belong to their
        # sentence. Splitting "Ginkgo biloba L. did not..." loses the subject
        # before the result can be assigned to an intervention.
        if boundary.group() == '.' and re.search(r'\b(?:[A-Z]|vs|Dr|Prof|Fig|al|e\.g|i\.e)\.$', text[:boundary.end()]):
            continue
        end = boundary.end()
        left = start + len(text[start:end]) - len(text[start:end].lstrip())
        if left < end and text[left:end].strip():
            yield {'text': text[left:end], 'start': left, 'end': end}
        start = end
    left = start + len(text[start:]) - len(text[start:].lstrip())
    if left < len(text):
        yield {'text': text[left:], 'start': left, 'end': len(text)}


def intervention_clauses(sentence, taxon, known_genera=None):
    """Keep contrasting, explicitly named interventions in separate source spans."""
    if not taxon.get('name'):
        return [sentence]
    names = {taxon['name']} | {a['name'] for a in taxon.get('aliases', []) if a['kind'] in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH', 'VERNACULAR') and len(a['name']) >= 4}
    own = re.compile(r'\b(?:' + '|'.join(re.escape(n) for n in sorted(names, key=len, reverse=True)) + r')\b', re.I)
    boundaries = list(re.finditer(r'(?:[,;]\s*)?\b(?:whereas|while|but|however)\b[,:]?\s*', sentence['text'], re.I))
    if not boundaries:
        return [sentence]
    starts = [0] + [b.end() for b in boundaries]
    ends = [b.start() for b in boundaries] + [len(sentence['text'])]
    parts = []
    for left, right in zip(starts, ends):
        segment = sentence['text'][left:right]
        left += len(segment) - len(segment.lstrip())
        right -= len(segment) - len(segment.rstrip())
        if left < right:
            parts.append({'text': sentence['text'][left:right], 'start': sentence['start'] + left, 'end': sentence['start'] + right})
    def named(text):
        return bool(own.search(text) or botanical_mentions(text, known_genera))
    grouped = []
    for part in parts:
        if named(part['text']) or not grouped:
            grouped.append(part)
        else:
            # A qualifier such as "but was less effective than..." belongs to
```

```
            # the preceding intervention; it must not prevent the plant split.
            left, right = grouped[-1]['start'], part['end']
            grouped[-1] = {'text': sentence['text'][left - sentence['start']:right - sentence['start']], 'start': left, 'end': right}
    return grouped if sum(named(p['text']) for p in grouped) > 1 else [sentence]


def benefit_on_condition(text, terms):
    """A favourable effect verb must act on the condition itself, not anywhere in the sentence."""
    t = "|".join(re.escape(x) for x in sorted(terms, key=len, reverse=True))
    return re.search(rf"\b(?:{BENEFIT_VERB})\b(?:\W+\w+){{0,3}}?\W+(?:{t})\b{NOT_A_SYMPTOM}|\b(?:{t})\b{NOT_A_SYMPTOM}(?:\W+\w+){{0,5}}?\W+(?:was|were|is|are|been)\s+(?:\w+\s+)?(?:{BENEFIT_VERB})\b", text, re.I)


def stated_in_verdict(text, terms):
    """Present-tense or hedged wording of a benefit; only a closing sentence may use it."""
    if OPEN_QUESTION.search(text):
        return None
    t = "|".join(re.escape(x) for x in sorted(terms, key=len, reverse=True))
    return VERDICT_BENEFIT.search(text) or re.search(rf"(?<!\ba )(?<!\bthe )\b(?:{PRESENT_BENEFIT_VERB})\b(?:\W+\w+){{0,3}}?\W+(?:{t})\b{NOT_A_SYMPTOM}", text, re.I)


def reported_findings(relevant, opening_start=0, terms=(), active_comparator=False, verdict_start=None):
    """Result statements in condition-relevant sentences, before any comparability gate."""
    findings = []
    for sentence in relevant:
        text = sentence["text"]
        # Introductory therapeutic claims must not masquerade as the study's finding:
        # background framing is skipped, and the opening sentence needs an explicit cue.
        if BACKGROUND.search(text) or AIM_SENTENCE.search(text):
            continue
        if re.search(r'\b(?:before (?:the )?(?:intervention|treatment|randomi[sz]ation)|at (?:the )?baseline|pre[- ]treatment)\b', text, re.I) and not re.search(r'\b(?:after|following|from baseline|change|changes)\b', text, re.I):
            # Balanced groups before dosing say nothing about treatment effect.
            continue
        if ANALYSIS_DIAGNOSTIC.search(text) and not benefit_on_condition(text, terms) and not re.search(r'\b(?:versus|than|compared (?:with|to))\s+(?:the\s+)?(?:placebo|control)|\b(?:primary (?:outcome|endpoint)|hospitali[sz]ation|mortality|symptoms?)\b', text, re.I):
            continue
        in_verdict = verdict_start is not None and sentence["end"] > verdict_start and sentence["start"] > opening_start
        hedged = stated_in_verdict(text, terms) if in_verdict else None
        closing = in_verdict and (BENEFIT_STATEMENT.search(text) or re.search(NO_EFFECT, text, re.I) or hedged or re.search(WEAK_EVIDENCE, text, re.I))
        if not (closing or RESULT_CUE.search(text) or MISSING_COMPARISON.search(text) or (EFFECT_VERB.search(text) and sentence["start"] > opening_start)):
            continue
        insufficient = re.search(WEAK_EVIDENCE, text, re.I)
        negative = re.search(NO_EFFECT, text, re.I)
        association = ASSOCIATION_BENEFIT.search(text)
        if association and re.search(r'\b(?:not|no(?:\s+\w+){0,4})\s*$', text[:association.start()], re.I):
            association = None
        negative = negative or re.search(r'\bnot associated with (?:a )?(?:lower|reduced|decreased) (?:risk|rate|incidence)\b', text, re.I)
        positive = benefit_on_condition(text, terms) or BENEFIT_STATEMENT.search(text) or hedged or association or re.search(r'\b(?:significant benefit|more beneficial than (?:the )?placebo|(?:show(?:ed|ing)|reported|demonstrated)\s+(?:an?\s+)?improvements?\s+(?:compared|over))\b', text, re.I)
        if FUTURE_WORK.search(text) and not negative and not insufficient and not re.search(r"\b(?:significant|effective|improved|reduced)\b", text, re.I):
            continue
        active_arm_comparison = negative and re.search(r'\b(?:among|between)\s+(?:the\s+)?(?:(?:three|two|four|\d+|different)\s+)?(?:active|herbal)\s+(?:treatments?|mouthwashes|interventions?|groups?|arms?)\b', text, re.I)
        multiple_comparisons = negative and re.search(r'\b(?:whereas|while|but)\b.*\b(?:placebo|control)\b', text, re.I) and re.search(r'\b(?:between|among)\b.*\b(?:arms|groups|treatments|interventions)\b', text, re.I)
        if MISSING_COMPARISON.search(text) or ((negative or positive) and (WITHIN_GROUP.search(text) or
```

```
    re.search(r'\bboth\s+(?:active\s+)?(?:groups|treatments|arms)\b', text, re.I)) and not BETWEEN_GROUP.search(text)):
293             findings.append(('INSUFFICIENT', {**sentence, 'comparison_note': 'A baseline change or an explicitly missing between-group comparison cannot establish a
    treatment-versus-comparator effect.'}))
294         elif negative and active_comparator and (GROUP_COMPARISON.search(text) or re.search(r'\bno\s+(?:statistically\s+)?significant\s+difference\b|\bno
    difference\b', text, re.I)):
295             findings.append(('INSUFFICIENT', {**sentence, 'comparison_note': 'No inactive arm is described; no difference from an active comparator is not a finding
    of no effect.'}))
296         elif active_arm_comparison or multiple_comparisons:
297             sentence = {**sentence, 'comparison_note': 'The sentence compares multiple arms or reports different comparisons. No single candidate-versus-control
    direction is inferred; read the complete result and its comparator.'}
298             findings.append(('INSUFFICIENT', sentence))
299         elif insufficient:
300             findings.append(("INSUFFICIENT", sentence))
301         elif negative and positive and re.search(r"\b(?:but|however|although|though|while|whereas|yet)\b", text, re.I):
302             findings.append(("MIXED", sentence))
303         elif negative:
304             findings.append(("CONTRADICTS", sentence))
305         elif positive:
306             findings.append(("SUPPORTS", sentence))
307     return findings
308 
309 
310 def relation_to_claim(paper, taxon, condition, study, *, taxon_match, require_identity=True, known_genera=None):
311     """Conservative clause classifier. Relations address a modern question, not history."""
312     terms = research_terms(condition)
313     if "STUDY_PROTOCOL" in study["types"]:
314         return {"relation": "INSUFFICIENT", "supporting_text": None, "rationale": "A study protocol is not a completed study result", "directness":
    "PROTOCOL_NO_RESULTS", "reported_finding": {"direction": "NO_RESULT_STATEMENT"}}
315     relevant = []
316     all_sentences = [part for sentence in study_sentences(paper['abstract']) for intervention in intervention_clauses(sentence, taxon, known_genera) for part in
    endpoint_clauses(intervention, terms)]
317     opening = all_sentences[0]["start"] if all_sentences else 0
318     conclusion = CONCLUSION_LABEL.search(paper["abstract"])
319     for sentence in all_sentences:
320         norm = normalize(sentence["text"])
321         if has_research_topic_wording(norm, terms):
322             relevant.append(sentence)
323     # An abstract closes on its verdict, in Conclusions or in its last sentences, often without
324     # repeating the topic words.
325     closing = [s for s in all_sentences if not ADMINISTRATIVE.search(s["text"])]
326     verdict_start = conclusion.start() if conclusion else closing[-VERDICT_SENTENCES]["start"] if len(closing) > VERDICT_SENTENCES else 0
327     if relevant:
328         known = {r["start"] for r in relevant}
329         # Generic conclusions may refer back to this endpoint, but not across a
330         # switch to another endpoint or to safety. A favourable anxiety result
331         # must not reverse a null sleep result merely because it closes the abstract.
332         last_scope = None
333         for sentence in closing:
334             scope = outcome_scope(sentence['text'], terms)
335             if scope != 'UNSPECIFIED':
336                 last_scope = scope
337             if sentence['end'] > verdict_start and sentence['start'] not in known and scope != 'OTHER' and last_scope == 'TARGET':
338                 relevant.append(sentence)
339         relevant.sort(key=lambda s: s["start"])
340     if not relevant:
341         return {"relation": "IRRELEVANT", "supporting_text": None, "rationale": "No condition-specific supporting sentence in the available abstract", "directness":
    "UNDETERMINED", "reported_finding": {"direction": "NO_RESULT_STATEMENT"}}
342     findings = reported_findings(relevant, opening, terms, active_comparator=not INACTIVE_ARM.search(paper["abstract"]), verdict_start=verdict_start)
```

```
    unassigned = findings
    findings, candidate_scope_note = candidate_result_scope(paper, taxon, findings, known_genera)
    # A null/uncertain conclusion does not erase an explicit conflicting result.
    # Retain all endpoint-specific statements and their exact source spans.
    comparative = [f for f in findings if not f[1].get('comparison_note')]
    findings = comparative or findings
    labels = {f[0] for f in findings}
    if "MIXED" in labels or {"SUPPORTS", "CONTRADICTS"} <= labels or {"SUPPORTS", "INSUFFICIENT"} <= labels and not {f[1].get('comparison_note') for f in findings} - {None}:
        reported = "MIXED"
    else:
        reported = "INSUFFICIENT" if "INSUFFICIENT" in labels else findings[-1][0] if findings else "NO_RESULT_STATEMENT"
    # The reported finding is what the abstract says about its own intervention and
    # population. It is kept visible even when it cannot answer the branch question.
    finding = {"direction": reported, "supporting_text": findings[-1][1]["text"], "start": findings[-1][1]["start"], "end": findings[-1][1]["end"]} if findings else {"direction": "NO_RESULT_STATEMENT"}
    comparison_notes = list(dict.fromkeys(s['comparison_note'] for _, s in findings if s.get('comparison_note')))
    if comparison_notes:
        finding['comparison_notes'] = comparison_notes
    if findings:
        finding['statements'] = [{'direction': direction, 'supporting_text': sentence['text'], 'start': sentence['start'], 'end': sentence['end'], **({'comparison_note':sentence['comparison_note']} if sentence.get('comparison_note') else {})} for direction, sentence in findings]
    if candidate_scope_note:
        finding['candidate_scope_note'] = candidate_scope_note
        finding['unassigned_statements'] = [{'direction':direction, 'supporting_text':sentence['text'], 'start':sentence['start'], 'end':sentence['end']} for direction,sentence in unassigned if (direction,sentence) not in findings]
    if require_identity and taxon_match != "SCIENTIFIC_NAME":
        return {"relation": "INSUFFICIENT", "supporting_text": relevant[-1]["text"], "rationale": "Common-name mention does not settle the experimental species for this candidate branch", "directness": "IDENTITY_UNCERTAIN", "reported_finding": finding}
    if not findings:
        return {"relation": "INSUFFICIENT", "supporting_text": relevant[-1]["text"], "rationale": candidate_scope_note or "Relevant wording is present but no explicit condition-specific result safely settles this question", "directness": "INTERVENTION_NOT_LINKED_TO_CANDIDATE" if candidate_scope_note else "ABSTRACT_ONLY", "reported_finding": finding}
    relation = reported
    support = findings[-1][1]["text"]
    if study.get('population_status')=='UNDETERMINED' and relation!='INSUFFICIENT':
        return {'relation':'INSUFFICIENT','reported_relation':relation,'supporting_text':support,'rationale':'An outcome statement is present but human population is not established in the available abstract.','directness':'POPULATION_UNDETERMINED','reported_finding':finding}
    preclinical = bool(set(study["types"]) & {"IN_VITRO", "IN_VIVO_ANIMAL"}) and not bool(set(study["types"]) & {"CLINICAL_TRIAL", "OBSERVATIONAL_HUMAN"})
    if preclinical and relation in ("SUPPORTS", "CONTRADICTS", "MIXED"):
        return {"relation": "INSUFFICIENT", "mechanistic_relation": relation, "supporting_text": support, "rationale": "Preclinical finding retained as indirect evidence; it cannot settle a human efficacy hypothesis", "directness": "PRECLINICAL_ONLY", "reported_finding": finding}
    if not set(study["types"]) & {"CLINICAL_TRIAL", "OBSERVATIONAL_HUMAN", "SYSTEMATIC_REVIEW", "META_ANALYSIS"} and relation != "INSUFFICIENT":
        return {"relation": "INSUFFICIENT", "reported_relation": relation, "supporting_text": support, "rationale": "Relevant findings are retained, but the abstract does not establish an adequately identified human-study design for an efficacy interpretation", "directness": "DESIGN_OR_POPULATION_UNCERTAIN", "reported_finding": finding}
    return {"relation": relation, "supporting_text": support, "rationale": ' '.join(comparison_notes) if comparison_notes else "Automated abstract-level, condition-specific result interpretation; preparation/population/outcome comparability requires human review. No-effect wording does not establish equivalence.", "directness": "ABSTRACT_ONLY_COMPARABILITY_UNREVIEWED", "reported_finding": finding}


def citation_check(paper, text, taxon_match, condition_match, canonical=None):
    errors = []
    if canonical is None:
        errors.append("NOT_IN_RETRIEVED_SNAPSHOT")
    elif any(paper.get(k) != canonical.get(k) for k in ("id", "pmid", "doi", "title", "abstract")):
        errors.append("SNAPSHOT_METADATA_MISMATCH")
    if not re.fullmatch(r"(?:PMC|PMID)\d+", paper.get("id", "")):
```

```
        errors.append("INVALID_PAPER_ID")
    if paper.get("pmid") and not re.fullmatch(r"\d+", str(paper["pmid"])):
        errors.append("INVALID_PMID")
    if paper.get("doi") and not re.fullmatch(r"10\.\d{4,9}/\S+", paper["doi"], re.I):
        errors.append("INVALID_DOI")
    if not paper.get("provenance", {}).get("url", "").startswith(("https://www.ebi.ac.uk/", "https://eutils.ncbi.nlm.nih.gov/")):
        errors.append("UNVERIFIED_METADATA_ORIGIN")
    if text and text not in paper["abstract"]:
        errors.append("SUPPORT_SPAN_NOT_IN_SOURCE")
    if paper.get("retracted"):
        errors.append("RETRACTED_SOURCE")
    return {"status": "REJECTED" if errors else "SOURCE_METADATA_AND_SPAN_CHECKED", "errors": errors, "identifier_present_in_retrieved_snapshot": canonical is not None and not errors, "exact_support_span": bool(text and text in paper["abstract"]), "taxon_scientific_match": taxon_match == "SCIENTIFIC_NAME", "condition_match": bool(condition_match), "entailment_status": "AUTOMATED_HEURISTIC_NOT_EXPERT_VERIFIED", "human_reviewed": False}


def conflict_groups(evidence):
    groups = []
    # Same branch/condition only; even then comparability is not automatically assumed.
    keys = {(e["taxon_id"], e["condition_id"]) for e in evidence}
    for taxon_id, condition_id in sorted(keys):
        rows = [e for e in evidence if e["taxon_id"] == taxon_id and e["condition_id"] == condition_id]
        labels = {e["relation"]["relation"] for e in rows}
        state = "POTENTIAL_CONFLICT_REQUIRES_COMPARABILITY_REVIEW" if {"SUPPORTS", "CONTRADICTS"} <= labels else "MIXED" if "MIXED" in labels else "INSUFFICIENT" if not rows or labels <= {"INSUFFICIENT", "IRRELEVANT"} else "NO_COMPARABLE_CONFLICT_ESTABLISHED"
        groups.append({"taxon_id": taxon_id, "condition_id": condition_id, "state": state, "evidence_ids": [e["id"] for e in rows], "limitation": "Preparation, dose, population and outcome may differ. Opposing abstract wording is a potential conflict, not a validated contradiction."})
    return groups
```

src/evidence/comparability.py
SHA-256: b31f8f3b65865096f0fc54eba1fe90827ce46fced92435efdbd88460c2a2b29e

```
"""Per-record comparability checks between a historical use and a modern abstract.

Each check answers one question a reviewer would ask before letting a paper bear on
"does a specified preparation of taxon X help condition Y in humans?". Checks keep
their own source spans. They explain the conservative relation; they never upgrade it.
"""
from __future__ import annotations

import re
from functools import lru_cache

from src.nlp.extract import research_terms, normalize

PASS, FAIL, PARTIAL, UNKNOWN = "PASS", "FAIL", "PARTIAL", "UNKNOWN"
HUMAN_DESIGNS = {"CLINICAL_TRIAL", "OBSERVATIONAL_HUMAN", "SYSTEMATIC_REVIEW", "META_ANALYSIS"}
PRECLINICAL = {"IN_VITRO", "IN_VIVO_ANIMAL"}
# Coarse preparation families. Matching families is a prompt for review, not equivalence.
PREPARATION_FAMILIES = {
    "aqueous": ["infusion", "tea", "decoction", "boiled", "aqueous", "distilled water", "syrup", "hydrolate", "hydrosol"],
    "oil": ["oil", "essential oil", "ointment"],
    "extract": ["extract", "ethanolic", "tincture", "standardized", "standardised"],
    "solid": ["powder", "capsule", "capsules", "tablet", "tablets", "dried"],
    "fresh": ["juice", "bruised", "fresh"],
}


def family(text):
    value = normalize(text)
    return next((name for name, words in PREPARATION_FAMILIES.items() if any(re.search(r"\b" + re.escape(w) + r"s?\b", value) for w in words)), None)


@lru_cache(maxsize=30_000)
def compiled(pattern):
    # Plant names produce thousands of patterns; re's own small cache would recompile them.
    return re.compile(pattern, re.I)


def span_in(abstract, pattern):
    match = compiled(pattern).search(abstract)
    return {"text": match.group(), "start": match.start(), "end": match.end()} if match else None


def name_pattern(name):
    # A stored scientific alias using the botanical hybrid marker x also matches \xd7.
    tokens = name.split()
    pattern = ''
    for index, token in enumerate(tokens):
        if index:
            pattern += r'\s*' if token.casefold() == 'x' or tokens[index - 1].casefold() == 'x' else r'\s+'
        pattern += '[x\xd7]' if token.casefold() == 'x' else re.escape(token)
    return r'(?<!\w)' + pattern + r'(?!\w)'


def methods_span(context, pattern):
    span = span_in(context['text'], pattern)
```

```
    return {**span, 'source_kind': context.get('source_kind', 'FULL_TEXT_METHODS'), 'source_id': context['id'],
            'url': context['url'], 'licence': context['licence']} if span else None


def check_identity(paper, taxon, taxon_match):
    names = [taxon["name"]] + [a["name"] for a in taxon["aliases"] if a["kind"] in ("SYNONYM", "INPUT_SYNONYM_MATCH")]
    for name in names:
        found = span_in(paper["abstract"], name_pattern(name))
        if found:
            return {"check": "identity", "status": PASS, "detail": "Abstract names the candidate's scientific name", "span": found}
        found = span_in(paper['title'], name_pattern(name))
        if found:
            return {'check': 'identity', 'status': PASS, 'detail': "Title names the candidate's scientific name", 'span': {**found, 'source_kind':
'PUBLICATION_TITLE'}}
    for context in paper.get('methods_contexts', []):
        for name in names:
            found = methods_span(context, name_pattern(name))
            if found:
                return {'check': 'identity', 'status': PASS,
                        'detail': 'A licensed Methods excerpt names this candidate or a stored scientific synonym. This identifies study wording, not the historical
plant.', 'span': found}
    return {"check": "identity", "status": FAIL, "detail": "Only a common name matches; the experimental species is not established for this branch", "span": None}


def check_population(study):
    status = study.get("population_status")
    if status == "HUMAN_TEXT_SIGNAL":
        population = (study.get('pico_source_spans') or {}).get('population')
        if population and population.get('reading_scope'):
            return {'check': 'population', 'status': PASS, 'detail': population['reading_scope'], 'span': population}
        return {"check": "population", "status": PASS, "detail": "Human participants mentioned", "span": (study.get("pico_source_spans") or {}).get("population")}
    if status == "ANIMAL_TEXT_SIGNAL":
        return {"check": "population", "status": FAIL, "detail": "Animal model only", "span": (study.get("pico_source_spans") or {}).get("population")}
    return {"check": "population", "status": UNKNOWN, "detail": "Population not stated in the abstract", "span": None}


def check_design(study):
    types = set(study["types"])
    if "STUDY_PROTOCOL" in types:
        return {"check": "design", "status": FAIL, "detail": "Protocol: no results yet"}
    if types & HUMAN_DESIGNS:
        return {"check": "design", "status": PASS, "detail": ", ".join(sorted(types & HUMAN_DESIGNS)).replace("_", " ").lower()}
    if types & PRECLINICAL:
        return {"check": "design", "status": FAIL, "detail": "Preclinical (" + ", ".join(sorted(types & PRECLINICAL)).replace("_", " ").lower() + ")"}
    if "REVIEW" in types:
        return {"check": "design", "status": PARTIAL, "detail": "Narrative review: secondary, not a systematic synthesis"}
    return {"check": "design", "status": UNKNOWN, "detail": "Study design not established"}


def check_comparator(study, paper=None):
    for context in (paper or {}).get('methods_contexts', []):
        found = methods_span(context, r'\buncontrolled\b|\bsingle[- ]arm\b|\bwithout\s+(?:a\s+)?(?:control|comparator)\b')
        if found:
            return {'check': 'comparator', 'status': FAIL,
                    'detail': 'Methods explicitly describe an uncontrolled or single-arm design. The observed change cannot isolate a treatment effect from time, co-
interventions or placebo effects.', 'span': found}
    for context in (paper or {}).get('methods_contexts', []):
        value = context['text']
```

```
        if re.search(r'\b(?:face[- ]split|split[- ]face) design\b', value, re.I) and re.search(r'\bleft side\b', value, re.I) and re.search(r'\bright side\b', value, re.I):
            found = methods_span(context, r'\bEach participant applied [^.]+\.')
            if found:
                allocation = 'Fixed left/right allocation is explicitly stated; randomized allocation is not established.' if re.search(r'\bfixed\b', value, re.I) else 'Allocation and blinding require source review.'
                return {'check': 'comparator', 'status': PARTIAL,
                        'detail': 'Methods describe a within-person comparison of formulations applied to opposite sides of the face. ' + allocation + ' Formulation and ingredient attribution remain separate questions.', 'span': found}
    comparator = (study.get('pico_source_spans') or {}).get('comparator')
    if not comparator:
        return {'check': 'comparator', 'status': UNKNOWN,
                'detail': 'A comparator is not established from the abstract. Improvement over time alone does not establish a treatment effect.', 'span': None}
    return {'check': 'comparator', 'status': PARTIAL,
            'detail': f"Comparator wording '{comparator['text']}' occurs in the abstract. Its role, allocation, blinding and adequacy have not been appraised.", 'span': comparator}


def check_outcome(relation, condition_id):
    terms = research_terms(condition_id)
    finding = relation.get("reported_finding") or {}
    if finding.get("direction") not in (None, "NO_RESULT_STATEMENT"):
        return {"check": "outcome", "status": PASS, "detail": "A result sentence mentions the condition", "span": {"text": finding["supporting_text"]}}
    if relation.get("supporting_text"):
        return {"check": "outcome", "status": PARTIAL, "detail": "Condition mentioned, but not in a result statement", "span": {"text": relation["supporting_text"]}}
    return {"check": "outcome", "status": FAIL, "detail": "Condition not mentioned in the abstract" if terms else "No outcome terms for this condition"}


def check_preparation(historical, study):
    modern = (study.get("pico_source_spans") or {}).get("intervention_preparation")
    old = [p["text"] for p in historical]
    attribution = study.get('preparation_attribution') or {}
    if attribution.get('status') == 'COMMON_NAME_ONLY_PREPARATION_WORDING':
        return {'check': 'preparation', 'status': UNKNOWN, 'detail': f"The abstract mentions '{modern['text']}' near a common plant name. The study species is not established for this candidate; this is not a verified preparation match.", 'span': modern, 'requires_statement_review': True}
    if attribution.get('status') in ('CANDIDATE_PREPARATION_UNRESOLVED', 'MULTIPLE_CANDIDATE_FORMS'):
        detail = 'Several candidate-linked forms occur in the abstract; no single intervention is assigned.' if attribution['status'] == 'MULTIPLE_CANDIDATE_FORMS' else 'Preparation wording is not locally linked to this candidate species; the first preparation in the abstract is not assigned to it.'
        return {'check': 'preparation', 'status': UNKNOWN, 'detail': detail, 'span': None, 'requires_statement_review': True}
    if not old and not modern:
        return {"check": "preparation", "status": UNKNOWN, "detail": "Neither source states a preparation"}
    if not old:
        return {"check": "preparation", "status": UNKNOWN, "detail": f"This historical statement does not state a preparation; candidate-linked abstract wording is '{modern['text']}'", "span": modern}
    if not modern:
        return {"check": "preparation", "status": UNKNOWN, "detail": f"Historical: {', '.join(sorted(set(old)))}; study preparation not established", "span": None}
    old_families = {family(x) for x in old} - {None}
    new_family = family(modern["text"])
    if new_family and new_family in old_families:
        return {"check": "preparation", "status": PARTIAL, "detail": f"Same broad form ({new_family}): historical {', '.join(sorted(set(old)))} vs study '{modern['text']}'. Preparation process, dose and solvent are not established as equivalent.", "span": modern}
    if not old_families or not new_family:
        return {'check': 'preparation', 'status': UNKNOWN, 'detail': f"Historical {', '.join(sorted(set(old)))} vs study '{modern['text']}': one or both preparation forms are not classified.", 'span': modern}
    return {"check": "preparation", "status": FAIL, "detail": f"Different forms: historical {', '.join(sorted(set(old)))} vs study '{modern['text']}'", "span": modern}


```

```
def comparability(paper, taxon, condition_id, study, relation, taxon_match, historical_preparations, *, preparation_check=None):
    checks = [check_identity(paper, taxon, taxon_match), check_population(study), check_design(study), check_comparator(study, paper), check_outcome(relation, condition_id), preparation_check if preparation_check is not None else check_preparation(historical_preparations, study)]
    return summarize(checks)


def summarize(checks):
    blocking = next((c["check"] for c in checks if c["status"] == FAIL), None) or next((c["check"] for c in checks if c["status"] == UNKNOWN and (c["check"] != "preparation" or c.get('requires_statement_review'))), None)
    unresolved = [c["check"] for c in checks if c["status"] in (PARTIAL, UNKNOWN)]
    screening = "BLOCKED" if blocking else "NO_FAILURE_SOME_UNRESOLVED" if unresolved else "ALL_PASSED"
    return {"checks": checks, "first_blocking_check": blocking, "unresolved_checks": unresolved, "screening": screening, "passed": sum(c["status"] == PASS for c in checks), "total": len(checks), "note": "Automated screening. A PASS on every check would still need human review of dose, comparator and outcome definition."}


def gate_relation(relation, screen):
    """A failed or unknown check withholds a directional relation; checks never upgrade one."""
    blocking = screen["first_blocking_check"]
    if not blocking or relation["relation"] not in ("SUPPORTS", "CONTRADICTS", "MIXED"):
        return relation
    detail = next((c["detail"] for c in screen["checks"] if c["check"] == blocking), "")
    return {**relation, "relation": "INSUFFICIENT", "pre_comparability_relation": relation["relation"], "directness": "COMPARABILITY_BLOCKED", "rationale": f"The abstract-level relation was {relation['relation']}, but the {blocking} check does not pass ({detail}). It cannot answer the modern question for this branch."}


def not_assessed():
    """Marker for analyses stored before comparability screening existed; nothing is inferred."""
    return {"checks": [], "first_blocking_check": "NOT_ASSESSED_LEGACY_RECORD", "unresolved_checks": [], "screening": "NOT_ASSESSED_LEGACY_RECORD", "passed": 0, "total": 0, "note": "Saved before comparability screening existed. Re-run the analysis to screen this record."}
```

src/evidence/consultation.py
SHA-256: 88845eca432c780010280df00d2400430c7e0a1c0df56ab3ea5c87dcf72f3bb5

```python
"""Prepare incomplete herbal-product information for a clinician conversation."""
from __future__ import annotations
import hashlib
import json
import re
import sqlite3
import uuid
from pathlib import Path
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field, model_validator
from src.common import now
from src.nlp.extract import normalize
from src.evidence.question_review import canonical
from src.evidence.overview import product_candidate_order
from src.evidence.product_labels import LabelSnapshot, ingredients, label_wording, ingredient_source, snapshot as label_snapshot


class ProductItem(BaseModel):
    model_config = ConfigDict(extra='forbid')
    item_id: str = Field(default_factory=lambda:uuid.uuid4().hex, pattern=r'^[a-f0-9]{12,32}$')
    name: str = Field(min_length=2, max_length=200)
    source_kind: Literal['PRODUCT_LABEL', 'PERSON_REPORTED'] = 'PERSON_REPORTED'
    use_status: Literal['NOT_ESTABLISHED','REPORTED_CURRENT_USE','REPORTED_PAST_USE'] = 'NOT_ESTABLISHED'
    source_text: str = Field(default='', max_length=5000)
    plant_wording: str = Field(default='', max_length=120)
    brand: str = Field(default='', max_length=200)
    dose: str = Field(default='', max_length=200)
    frequency: str = Field(default='', max_length=200)
    route: str = Field(default='', max_length=200)
    last_taken: str = Field(default='', max_length=200)
    preparation: str = Field(default='', max_length=200)
    reason_for_use: str = Field(default='', max_length=500)
    label_snapshot: LabelSnapshot | None = None

    @model_validator(mode='after')
    def grounded(self):
        if len(self.name.strip()) < 2:
            raise ValueError('Product name must contain substantive text')
        if self.plant_wording and normalize(self.plant_wording) not in normalize(self.name+' '+self.source_text):
            raise ValueError('Plant wording must occur in the product name or its recorded source')
        if self.label_snapshot:
            raw=self.label_snapshot.raw
            if self.source_kind!='PRODUCT_LABEL' or self.name!=raw['fullName'] or self.brand!=(raw.get('brandName') or '') or self.source_text!=label_wording(raw):
                raise ValueError('Attached label wording, name and brand must match the source snapshot; remove it before manual editing')
        return self


class ConsultationInput(BaseModel):
    model_config = ConfigDict(extra='forbid')
    visit_label: str = Field(min_length=2, max_length=160)
    prepared_by: Literal['PATIENT_OR_CARER', 'HEALTHCARE_PROFESSIONAL'] = 'PATIENT_OR_CARER'
    products: list[ProductItem] = Field(min_length=1, max_length=30)
    other_medicines_status: Literal['NOT_PROVIDED', 'NONE_REPORTED', 'LIST_PROVIDED'] = 'NOT_PROVIDED'
    other_medicines: str = Field(default='', max_length=6000)
    allergies_status: Literal['NOT_PROVIDED', 'NONE_REPORTED', 'DETAILS_PROVIDED'] = 'NOT_PROVIDED'
```

```
    allergies: str = Field(default='', max_length=2000)
    questions_for_professional: str = Field(default='', max_length=2000)

    @model_validator(mode='after')
    def complete_states(self):
        if len(self.visit_label.strip()) < 2:
            raise ValueError('A visit label is required; a patient name is not needed')
        if len({p.item_id for p in self.products}) != len(self.products):
            raise ValueError('Product identifiers must be unique; different products must not be merged')
        if self.other_medicines_status == 'LIST_PROVIDED' and not self.other_medicines.strip():
            raise ValueError('Record the medicine list or choose information not provided')
        if self.other_medicines_status != 'LIST_PROVIDED' and self.other_medicines.strip():
            raise ValueError('Medicine details need the list provided status')
        if self.allergies_status == 'DETAILS_PROVIDED' and not self.allergies.strip():
            raise ValueError('Record the reported allergy details or choose information not provided')
        if self.allergies_status != 'DETAILS_PROVIDED' and self.allergies.strip():
            raise ValueError('Allergy details need the details provided status')
        return self


def item_fingerprint(item):
    return hashlib.sha256(canonical(item)).hexdigest()


def prepare_consultation(pipeline, request: ConsultationInput):
    products = []
    for item in request.products:
        original = item.model_dump(exclude_none=True)
        lookup = item.plant_wording or item.name
        resolution = pipeline.resolver.resolve(lookup)
        candidates = []
        label_rows=[]
        if item.label_snapshot:
            for row in ingredients(item.label_snapshot.raw):
                row['candidates']=[]
                row['review_source_text']=ingredient_source(item.label_snapshot.raw,row['key'])
                if row['plant_lookup_available']:
                    resolved=pipeline.resolver.resolve(row['lookup'])
                    explicit_species = ('Genus:' in row['notes'] and 'Species:' in row['notes']) or re.fullmatch(r'[A-Z][a-z]+ [a-z][a-z-]+',row['lookup'])
                    if not resolved['candidates'] and row['lookup'] != row['name'] and not explicit_species:
                        alternative = pipeline.resolver.resolve(row['name'])
                        if alternative['candidates']:
                            row['research_lookup'] = row['name']
                            resolved = alternative
                    row['candidates']=[{'taxon_id':c['taxon_id'],'name':c['name'],'reasons':c['reasons'],
                         'reference_url':'https://www.gbif.org/species/'+c['taxon_id'].removeprefix('gbif:') if c['taxon_id'].startswith('gbif:') else None}
                         for c in product_candidate_order(resolved['candidates'],row['lookup']) if c['taxon_id'] not in pipeline.withdrawn()]
                label_rows.append(row)
            resolution={'status':'LABEL_INGREDIENT_BRANCHES','candidates':[c for row in label_rows for c in row['candidates']]}
        for candidate in product_candidate_order(resolution['candidates'],lookup):
            if candidate['taxon_id'] in pipeline.withdrawn():
                continue
            if any(c['taxon_id']==candidate['taxon_id'] for c in candidates):continue
            candidates.append({'taxon_id': candidate['taxon_id'], 'name': candidate['name'],
                               'reasons': candidate['reasons'],
                               'reference_url': 'https://www.gbif.org/species/'+candidate['taxon_id'].removeprefix('gbif:') if
candidate['taxon_id'].startswith('gbif:') else None})
        missing = [key for key in ('dose','frequency','route','last_taken','preparation') if not getattr(item,key).strip()]
```

```
        tasks = [{'field': key, 'question': {'dose':'What amount is actually used each time? Copy the stated unit.',
                                             'frequency':'How often is this product actually used?',
                                             'route':'How is the product used?',
                                             'last_taken':'When was the last use?',
                                             'preparation':'What form does the label state: tea, oil, extract, capsule or something else?'}[key]} for key in missing]
        if item.use_status=='NOT_ESTABLISHED':
            tasks.insert(0,{'field':'use_status','question':'Is this product actually being used now, used in the past, or only mentioned? Its use status has not been established.'})
        if not item.source_text.strip():
            tasks.insert(0, {'field':'source_text','question':'Bring the bottle or package and copy the exact ingredient label; this record has no source wording yet.'})
        if len(candidates) != 1:
            tasks.insert(0, {'field':'identity','question':'Ask the pharmacist to check the ingredient label and its botanical name. '+('Several plants share this wording.' if candidates else 'No plant candidate was established from this wording.')})
        else:
            tasks.insert(0, {'field':'identity','question':'Check the candidate against the actual ingredient label. A name match does not verify what is in this product.'})
        products.append({'reported': original, 'item_sha256': item_fingerprint(original),
                         'name_lookup': lookup, 'name_status': resolution['status'],
                         'identity_status': 'NOT_VERIFIED_AGAINST_PRODUCT', 'candidates': candidates,
                         **({'label_ingredients':label_rows} if item.label_snapshot else {}),
                         'missing_reported_fields': missing, 'clarification_questions': tasks})
    return {'schema':'foliatrace-consultation-prep-1', 'created_at':now(), 'intake':request.model_dump(),
            'products':products, 'linked_evidence':[], 'code_fingerprint':pipeline.code_fingerprint,
            'snapshot_sha256':pipeline.snapshot,
            'scope':'CONSULTATION_PREPARATION_NOT_COMPLETED_MEDICATION_RECONCILIATION',
            'summary':{'products':len(products), 'products_with_several_candidates':sum(len(p['candidates'])>1 for p in products),
                       'products_without_candidate':sum(not p['candidates'] for p in products),
                       'missing_reported_fields':sum(len(p['missing_reported_fields']) for p in products)},
            'next_visit_questions': (["Provide the full prescription, over-the-counter and supplement list." ] if request.other_medicines_status == 'NOT_PROVIDED' else [])+(["Ask about allergies and previous adverse reactions; no information was provided here."] if request.allergies_status == 'NOT_PROVIDED' else []),
            'limits':['This record preserves reported information; product contents, doses and clinical safety have not been verified.',
                      'No drug interaction check, diagnosis, safety clearance or recommendation to start, stop or change a product is performed.',
                      'Candidate species are hypotheses from a name reference, not identification of the bottle or its ingredients.',
                      'The clinician or pharmacist must review the original products, complete medication list and relevant clinical context.',
                      'Consultation details stay in this local installation. Optional live NIH lookup sends only the separate product search text. Exports can contain health information and publisher-owned study text; choose what to share.'],
            'workflow_sources':[{'title':'NCCIH: Talk with your health care providers','url':'https://www.nccih.nih.gov/health/know-science/how-medications-and-supplements-can-interact/talk-with-your-health-care-providers'},
                                {'title':'AHRQ: Your Medicine, Be Smart. Be Safe.','url':'https://www.ahrq.gov/questions/resources/your-meds/index.html'}]}


def validate_consultation(packet):
    if not isinstance(packet,dict):raise ValueError('Consultation preparation must be an object')
    if packet.get('schema') != 'foliatrace-consultation-prep-1':
        raise ValueError('Unsupported consultation preparation schema')
    intake = ConsultationInput.model_validate(packet['intake'])
    expected = {p.item_id:p.model_dump(exclude_none=True) for p in intake.products}
    for original in packet['intake']['products']:
        if 'use_status' not in original:
            expected[original['item_id']].pop('use_status',None)
    if len(packet['products']) != len(expected):
        raise ValueError('Product count does not match intake')
    seen = set()
    for item in packet['products']:
        key = item['reported']['item_id']
        if key in seen or expected.get(key) != item['reported'] or item_fingerprint(item['reported']) != item['item_sha256']:
            raise ValueError('Product source wording changed or duplicated')
```

```
        if item['identity_status'] != 'NOT_VERIFIED_AGAINST_PRODUCT':
            raise ValueError('Name candidates must not be represented as verified products')
        attachment=expected[key].get('label_snapshot')
        if attachment:
            rows=ingredients(attachment['raw']);actual=item.get('label_ingredients',[])
            if len(rows)!=len(actual) or any({k:v for k,v in a.items() if k not in ('candidates','review_source_text','research_lookup')}!=b or a.get('review_source_text')!=ingredient_source(attachment['raw'],b['key']) for a,b in zip(actual,rows)):
                raise ValueError('Label ingredient projection differs from its original source')
            for a,b in zip(actual,rows):
                if 'research_lookup' in a and (not b['plant_lookup_available'] or a['research_lookup']!=b['name']):
                    raise ValueError('Alternate research wording must be the original botanical ingredient name')
        seen.add(key)
    for link in packet['linked_evidence']:
        if link['item_id'] not in expected:
            raise ValueError('Linked evidence has no product in this record')
        from src.evidence.question_review import validate_packet
        validate_packet(link['review']['packet'])
        if link['source_item_sha256'] != item_fingerprint(expected[link['item_id']]):
            raise ValueError('Linked evidence refers to changed product wording')
        product=next(p for p in packet['products'] if p['reported']['item_id']==link['item_id'])
        if expected[link['item_id']].get('label_snapshot'):
            ingredient=next((r for r in product.get('label_ingredients',[]) if r['key']==link.get('ingredient_key')),None)
            if ingredient is None or link.get('source_label_sha256')!=expected[link['item_id']]['label_snapshot']['sha256']:
                raise ValueError('Linked evidence has no matching ingredient or label version')
            taxa={c['taxon_id'] for c in ingredient['candidates']}
            if not {b['taxon_id'] for b in link['review']['packet']['branches']} <= taxa:
                raise ValueError('Evidence branches do not match the selected label ingredient')
            protocol=link['review']['packet']['protocol']
            if protocol['source_text']!=ingredient['review_source_text'] or normalize(protocol['plant_name'])!=normalize(ingredient.get('research_lookup') or ingredient['lookup']):
                raise ValueError('Evidence source does not match the selected ingredient')
        else:
            protocol = link['review']['packet']['protocol']
            if protocol['source_text'] != product['reported']['source_text'] or normalize(protocol['plant_name']) != normalize(product['name_lookup']):
                raise ValueError('Evidence source does not match the manually recorded product')
            if not {b['taxon_id'] for b in link['review']['packet']['branches']} <= {c['taxon_id'] for c in product['candidates']}:
                raise ValueError('Evidence branches do not match the manually recorded product')
    from src.evidence.care_workspace import validate_clarifications
    validate_clarifications(packet)
    return packet


class ConsultationStore:
    def __init__(self,path:Path):
        self.path=path
        with self.connect() as db:
            db.execute('CREATE TABLE IF NOT EXISTS consultation_preparations(id TEXT PRIMARY KEY,saved_at TEXT NOT NULL,origin TEXT NOT NULL,payload TEXT NOT NULL)')

    def connect(self):
        from contextlib import contextmanager
        @contextmanager
        def opened():
            db=sqlite3.connect(self.path,timeout=20)
            try:
                with db:yield db
            finally:db.close()
        return opened()

```

```
    def put(self,packet,origin='LOCAL_CONSULTATION_PREPARATION'):
        validate_consultation(packet)
        if len(canonical(packet))>19_000_000:raise ValueError('Preparation is too large for portable export; retain fewer evidence reviews per snapshot')
        identifier=uuid.uuid4().hex
        with self.connect() as db:
            db.execute('INSERT INTO consultation_preparations VALUES(?,?,?,?)',(identifier,now(),origin,canonical(packet).decode()))
        return self.get(identifier)

    def get(self,identifier):
        with self.connect() as db:row=db.execute('SELECT id,saved_at,origin,payload FROM consultation_preparations WHERE id=?',(identifier,)).fetchone()
        if not row:return None
        packet=json.loads(row[3]);record={'id':row[0],'saved_at':row[1],'origin':row[2],'packet':packet,'sha256':hashlib.sha256(canonical(packet)).hexdigest()}
        if any(p.get('label_snapshot') and not p['label_snapshot'].get('hash_scheme') for p in packet['intake']['products']):
            import copy
            draft=copy.deepcopy(packet['intake']);detached=False
            for product in draft['products']:
                source=product.get('label_snapshot')
                if not source or source.get('hash_scheme'):continue
                try:
                    upgraded=label_snapshot(source['raw'],source['retrieved_at'],source['access'])
                    product['label_snapshot']=upgraded;product['source_text']=label_wording(upgraded['raw'])
                except ValueError:
                    # Keep the original human-readable wording, never round an unsafe source number.
                    product.pop('label_snapshot');detached=True
            record['draft_intake']=draft
            record['draft_notice']='A new draft uses browser-safe numeric formatting; save it as a new snapshot. The original record and its hashes are
    unchanged.'+(' An unsupported legacy numeric attachment is detached in the draft; original wording and the saved source remain available.' if detached else '')
        return record

    def list(self):
        with self.connect() as db:
            return [dict(zip(('id','saved_at','origin','visit_label'),r)) for r in db.execute("SELECT id,saved_at,origin,json_extract(payload,'$.intake.visit_label')
     FROM consultation_preparations ORDER BY saved_at DESC LIMIT 100")]


def consultation_markdown(record):
    p=record['packet'];q=p['intake'];lines=['# Herbal product consultation preparation',q['visit_label'],'Origin: '+record['origin'],
        'Reported information for a healthcare conversation. This is not a completed medication reconciliation or clinical safety assessment.',
        '## Other medicines',q['other_medicines_status'],q['other_medicines'],'## Allergies and previous reactions',q['allergies_status'],q['allergies']]
    for item in p['products']:
        r=item['reported'];lines+=['## '+r['name'],'Information source: '+r['source_kind'],'Reported use status: '+r.get('use_status','NOT_ESTABLISHED'),'Exact
    recorded wording:',r['source_text'] or 'Not provided']
        for key in ('brand','dose','frequency','route','last_taken','preparation','reason_for_use'):
            lines+=[key.replace('_',' ').capitalize()+': '+(r[key] or 'NOT PROVIDED')]
        if r.get('label_snapshot'):
            label=r['label_snapshot'];raw=label['raw']
            lines+=['### Selected label record (bottle match not established)',label['source_url'],
                'Source SHA256: '+label['sha256'],'Retrieved: '+label['retrieved_at']+' / '+label['access'],
                'Label entered: '+str(raw.get('entryDate'))+'; off-market flag: '+str(raw.get('offMarket')),
                'Label serving sizes (not actual use): '+json.dumps(raw.get('servingSizes',[]),ensure_ascii=False)]
            for ingredient in item.get('label_ingredients',[]):
                lines += [ingredient['key']+' '+ingredient['name'],ingredient['notes']]
                for amount in ingredient['quantities']:
                    lines += ['Individual label amount: NOT PROVIDED (DSLD NP placeholder; not zero).' if amount['amount_status']=='NOT_PROVIDED'
                              else 'Own label amount: '+str(amount.get('operator') or '')+str(amount['quantity'])+' '+amount['unit']+' per
    '+str(amount.get('servingSizeQuantity'))+' '+str(amount.get('servingSizeUnit') or 'label serving')]
            lines+=['Other declared ingredients (source wording): '+json.dumps(raw.get('otheringredients') or {},ensure_ascii=False)]
            lines+=['Full original label JSON, including directions and other ingredients, is included in the portable backup.']
```

```
        lines+=['Candidate names, not verified product ingredients:']+[c['name']+' '+str(c.get('reference_url') or '') for c in item['candidates']]
        if not item['candidates']:lines+=['No candidate established; bring original packaging.']
        lines+=['Clarification questions:']+[t['question'] for t in item['clarification_questions']]
        for link in p['linked_evidence']:
            if link['item_id']==r['item_id']:
                from src.evidence.question_review import review_markdown
                lines+=['### Professional evidence reading', 'Selected ingredient: '+link.get('ingredient_key','manually recorded
    product'),review_markdown(link['review'])]
    lines+=['## Questions for the professional',q['questions_for_professional'],*p['next_visit_questions'],'## Scope and sources',*p['limits']]
    lines += [s['title']+' '+s['url'] for s in p['workflow_sources']]
    return '\n\n'.join(lines)+'\n'
```

src/evidence/consultation_summary.py
SHA-256: 3e6b4918e31faecd2ac29642986a4dd3605b8f63748e62933f698f72b005ed34

```
"""A readable conversation handoff; full source records remain in JSON/Markdown."""
from __future__ import annotations

from html import escape
from urllib.parse import urlsplit

STATUSES = {'NOT_PROVIDED':'Not provided \u2014 ask the person',
            'NONE_REPORTED':'None explicitly reported \u2014 not independently verified',
            'LIST_PROVIDED':'List reported below', 'DETAILS_PROVIDED':'Details reported below'}
USE = {'NOT_ESTABLISHED':'Use not established', 'REPORTED_CURRENT_USE':'Current use reported',
       'REPORTED_PAST_USE':'Past use reported'}
DECISIONS = {'KEEP_FOR_READING':'Keep for further reading',
             'EXCLUDE_FROM_QUESTION':'Exclude from this question', 'NEEDS_FULL_TEXT':'Need full text or more information'}
SCOPE = 'Preparation for a healthcare conversation. Product contents, actual use and clinical safety have not been verified. No interaction check or recommendation to start, stop or change a product.'


def quantity_text(amount):
    if amount['amount_status']=='NOT_PROVIDED':return 'Individual amount not provided (NP; not zero)'
    return str(amount.get('operator') or '')+str(amount['quantity'])+' '+amount['unit']+' per '+str(amount.get('servingSizeQuantity'))+' '+str(amount.get('servingSizeUnit') or 'label serving')


def handoff_summary(record):
    packet=record['packet'];intake=packet['intake'];products=[]
    for item in packet['products']:
        reported=item['reported'];reviews=[]
        for entry in packet['linked_evidence']:
            if entry['item_id']!=reported['item_id']:continue
            review=entry['review'];latest={d['record_id']:d for d in review['decision_history']}
            rows={r['id']:r for b in review['packet']['branches'] for r in b['records']}
            decisions=[]
            for key,decision in latest.items():
                row=rows.get(key)
                if row is None:continue
                decisions.append({'title':row['paper']['title'],'url':row['paper']['url'],
                    'candidate_taxon_id':row['taxon_id'],'decision':DECISIONS[decision['decision']],
                    'reason':decision['reason'],'reviewer':decision['reviewer']})
            ingredient=next((r['name'] for r in item.get('label_ingredients',[]) if r['key']==entry.get('ingredient_key')),None)
            coverage=review['packet']['coverage'];external=coverage.get('public_search')
            note=('Public lookup: '+external['status'].replace('_',' ').lower()+' \xb7 '+external.get('provider','public source')+'. Bounded records only; inspect the full review for exact queries and failures.' if external else 'Local snapshot only; not an exhaustive literature search.')
            reviews.append({'question':review['packet']['protocol']['question'],'ingredient':ingredient,'coverage_note':note,
                            'source_review_id':review['id'],'record_count':len(rows),
                            'unreviewed_records':sum(key not in latest for key in rows),'decisions':decisions})
        label=reported.get('label_snapshot')
        products.append({'name':reported['name'],'brand':reported['brand'],'use':USE.get(reported.get('use_status'),'Use not established'),
            'actual_use':{k:reported[k] or 'Not provided' for k in ('dose','frequency','route','last_taken')},
            'stated_preparation':reported['preparation'] or 'Not provided','reported_reason':reported['reason_for_use'] or 'Not provided',
            'source_text':reported['source_text'],
            'label':{'id':label['label_id'],'url':label['source_url'],'sha256':label['sha256'],
                     'retrieved_at':label['retrieved_at'],'entry_date':label['raw'].get('entryDate'),
                     'market_status':'Off-market record' if label['raw'].get('offMarket')==1 else 'On-market record' if label['raw'].get('offMarket')==0 else 'Not stated'} if label else None,
            'ingredients':[{'name':r['name'],'nested':r['parent_key'] is not None,'notes':r['notes'],
```

```
                           'forms':'; '.join(' '.join(str(f[k]) for k in ('prefix','name') if f.get(k)) for f in r.get('forms',[])),
                           'amounts':[quantity_text(q) for q in r['quantities']]} for r in item.get('label_ingredients',[])],
            'other_ingredients':label['raw'].get('otheringredients') if label else None,
            'clarifications':[q['question'] for q in item['clarification_questions']],
            'clarification_notes':[n for n in packet.get('clarification_history',[]) if n['item_id']==reported['item_id']],
            'reviews':reviews})
    return {'title':intake['visit_label'],'record_id':record['id'],'saved_at':record['saved_at'],
            'origin':'Local reported preparation' if record['origin']=='LOCAL_CONSULTATION_PREPARATION' else 'Imported preparation \u2014 sources and clinical content unverified',
            'scope':SCOPE,'medicine_status':STATUSES[intake['other_medicines_status']], 'medicines':intake['other_medicines'],
            'allergy_status':STATUSES[intake['allergies_status']], 'allergies':intake['allergies'],
            'questions':intake['questions_for_professional'],'next_questions':packet['next_visit_questions'],'products':products}


def summary_markdown(record):
    s=handoff_summary(record)
    lines=['# Pharmacist conversation handoff',s['title'],s['origin'],s['scope'],
           'Record: '+s['record_id']+' \xb7 '+s['saved_at'],
           '## Questions to discuss',s['questions'] or 'No question recorded',*s['next_questions'],
           '## Other medicines',s['medicine_status'],s['medicines'],
           '## Allergies and previous reactions',s['allergy_status'],s['allergies']]
    for product in s['products']:
        lines+=['## '+product['name'],product['use'],'Brand: '+(product['brand'] or 'Not provided')]
        lines += [k.replace('_',' ').capitalize()+': '+v for k,v in product['actual_use'].items()]
        lines += ['Stated preparation: '+product['stated_preparation'],'Reported reason: '+product['reported_reason']]
        if product['label']:
            label=product['label'];lines+=['Selected NIH label '+str(label['id'])+' \u2014 bottle match not established',label['url'],
                'Entered: '+str(label['entry_date'])+' \xb7 '+label['market_status'],'Acquired: '+label['retrieved_at'],
                'Source SHA256: '+label['sha256'],'Declared ingredients \u2014 label quantities, not actual use:']
            for ingredient in product['ingredients']:
                lines+=['- '+('Within blend: ' if ingredient['nested'] else '')+ingredient['name']+': '+('; '.join(ingredient['amounts']) or 'Amount not provided'),ingredient['notes']]
                if ingredient['forms']:lines+=['Declared forms: '+ingredient['forms']]
            other=product['other_ingredients'] or {};lines+=['Other declared ingredients: '+(other.get('text') or '; '.join(r['name'] for r in other.get('ingredients',[])) or 'Not provided')]
        else:lines+=['Original wording:',product['source_text'] or 'Not provided']
        lines+=['Clarify before assessment:']+['- '+q for q in product['clarifications']]
        for note in product['clarification_notes']:
            lines += ['Recorded clarification \u2014 '+note['field']+': '+note['note'],
                      note['status']+' \xb7 '+note['source']+' \xb7 '+note['recorded_by']+' \xb7 '+note['created_at'],
                      'Original reported fields are preserved; this note is not independent verification.']
        for review in product['reviews']:
            lines+=['### Professional reading \u2014 not a clinical conclusion',review['question'],
                'Ingredient: '+str(review['ingredient'] or 'Manually recorded product'),
                review['coverage_note'],
                str(review['record_count'])+' candidate/study records screened; '+str(review['unreviewed_records'])+' have no recorded human reading decision.']
            if not review['decisions']:lines+=['No professional reading decision recorded. Automated screening is not approval.']
            for decision in review['decisions']:
                lines+=[decision['decision']+': '+decision['title'],decision['url'],
                        'Reason: '+decision['reason'],'Recorded by: '+decision['reviewer']]
    lines+=['## Full source record','This summary omits unreviewed abstracts and earlier decision events. The portable JSON and full report preserve original labels, all screened records and decision history.','Bring original packaging and the complete medicine list.']
    return '\n\n'.join(lines)+'\n'


def summary_html(record):
    s=handoff_summary(record)
    def e(v):return escape(str(v),quote=True)
```

```
    def paragraph(text,cls=''):return '<p'+(' class="'+cls+'"' if cls else '')+'>'+e(text).replace('\n','<br>')+'</p>'
    def source(url,title):
        try:parsed=urlsplit(url)
        except ValueError:return paragraph(title+' \u2014 URL not a supported web reference')
        if parsed.scheme not in ('https','http') or not parsed.hostname:return paragraph(title+' \u2014 URL not a supported web reference')
        return '<p><a href="'+e(url)+'" rel="noreferrer noopener">'+e(title)+'</a></p>'
    def heading(text,level=2):return f'<h{level}>'+e(text)+f'</h{level}>'
    out=['<!doctype html><html lang="en"><meta charset="utf-8">',
         '<title>'+e(s['title'])+' \xb7 FoliaTrace handoff</title>',
         '<style>body{font:16px/1.5 system-ui,sans-serif;color:#162b26;background:white;max-width:900px;margin:32px auto;padding:0 20px}h1{font-size:28px}h2{margin-top:28px;font-size:21px}h3{font-size:17px}p,li,dd{overflow-wrap:anywhere}a{color:#155b47}dl{display:grid;grid-template-columns:minmax(100px,1fr) minmax(0,3fr);gap:6px 18px}dt{font-weight:600}dd{margin:0}.notice{border-left:4px solid #8a5d16;background:#fff6e3;padding:12px}.source{font-size:13px}.ingredient{border-left:2px solid #cad7cf;padding-left:12px}.product{border-top:1px solid #cad7cf;margin-top:24px;padding-top:8px}.print-help{color:#52645c}@media print{body{font-size:10pt;max-width:none;margin:0;padding:0}.print-help{display:none}h2,h3{break-after:avoid}a{color:inherit}.notice{background:none;border:1px solid #aaa}a[href]::after{content:" (" attr(href) ")";font-size:8pt}@page{size:A4;margin:15mm}}</style>',
         '<body><main>',heading('Pharmacist conversation handoff',1),paragraph(s['title']),paragraph(s['origin'],'notice'),paragraph(s['scope'],'notice'),
         paragraph('Print or save a PDF using your browser\u2019s print command. This standalone handoff contains reported health information; choose what to share.','print-help'),
         paragraph('Record '+s['record_id']+' \xb7 '+s['saved_at'],'source'),heading('Questions to discuss'),paragraph(s['questions'] or 'No question recorded')]
    out += [paragraph(q) for q in s['next_questions']]
    out += [heading('Other medicines'),paragraph(s['medicine_status']),paragraph(s['medicines']),heading('Allergies / previous reactions'),paragraph(s['allergy_status']),paragraph(s['allergies'])]
    for product in s['products']:
        out+=['<section class="product">',heading(product['name']),paragraph(product['use']),'<dl>']
        facts={'Brand':product['brand'] or 'Not provided',**{k.replace('_',' ').capitalize():v for k,v in product['actual_use'].items()},'Stated preparation':product['stated_preparation'],'Reported reason':product['reported_reason']}
        out += ['<dt>'+e(k)+'</dt><dd>'+e(v)+'</dd>' for k,v in facts.items()];out+=['</dl>']
        if product['label']:
            label=product['label'];out+=[source(label['url'],'NIH label '+str(label['id'])+' \u2014 bottle match not established'),
                paragraph('Entered: '+str(label['entry_date'])+' \xb7 '+label['market_status']+' \xb7 acquired '+label['retrieved_at'],'source'),
                paragraph('Source SHA256: '+label['sha256'],'source'),heading('Declared ingredients \u2014 not actual use',3)]
            for ingredient in product['ingredients']:
                out+=['<div class="ingredient">',paragraph(('Within blend: ' if ingredient['nested'] else '')+ingredient['name']),paragraph('; '.join(ingredient['amounts']) or 'Amount not provided'),paragraph(ingredient['notes'],'source'),'</div>']
                if ingredient['forms']:out+=[paragraph('Declared forms: '+ingredient['forms'])]
            other=product['other_ingredients'] or {};out+=[paragraph('Other ingredients: '+(other.get('text') or '; '.join(r['name'] for r in other.get('ingredients',[])) or 'Not provided'))]
        else:out+=[heading('Original recorded wording',3),paragraph(product['source_text'] or 'Not provided')]
        out+=[heading('Clarify before assessment',3),'<ul>']+['<li>'+e(q)+'</li>' for q in product['clarifications']]+['</ul>']
        for note in product['clarification_notes']:
            out += [heading('Recorded clarification \xb7 '+note['field'],3),paragraph(note['note']),
                    paragraph(note['status']+' \xb7 '+note['source']+' \xb7 '+note['recorded_by']+' \xb7 '+note['created_at'],'source'),
                    paragraph('Original fields remain as reported. This note is not independent verification.','source')]
        for review in product['reviews']:
            out+=[heading('Professional reading \u2014 not a clinical conclusion',3),paragraph(review['question']),paragraph('Ingredient: '+str(review['ingredient'] or 'Manually recorded product')),
                paragraph(review['coverage_note'],'source'),
                paragraph(str(review['record_count'])+' candidate/study records screened; '+str(review['unreviewed_records'])+' have no recorded human reading decision.')]
            if not review['decisions']:out+=[paragraph('No professional reading decision recorded. Automated screening is not approval.')]
            for decision in review['decisions']:
                out+=[source(decision['url'],decision['decision']+': '+decision['title']),paragraph('Reason: '+decision['reason']),paragraph('Recorded by: '+decision['reviewer'],'source')]
        out+=['</section>']
    out+=[heading('Full source record'),paragraph('This concise handoff omits unreviewed abstracts and earlier decision events. Keep the portable JSON or full report for complete original labels, screened records and reading history.'),paragraph('Bring original packaging and the complete medicine list.'),'</main></body></html>']
    return '\n'.join(out)
```

src/evidence/context.py
SHA-256: dff1978c90eb6b7b3659b5371292bc6c760567ff3318561495fe493d6e3a830a

```
"""Keep a study's intervention and each historical statement attached to their source.

These lexical links are reading aids, not botanical identification or causal inference.
In particular, the first preparation in a multi-ingredient abstract is not necessarily
the preparation of the candidate plant.
"""
from copy import deepcopy
import re

from src.evidence.comparability import check_preparation, compiled, family, UNKNOWN, name_pattern


PREPARATIONS = re.compile(
    r"\b(?:(?:aqueous|ethanolic|standardized|standardised|essential)\s+)?"
    r"(?:hydrolates?|hydrosols?|decoctions?|tinctures?|ointments?|emulsions?|"
    r"infusions?|extracts?|capsules?|tablets?|powders?|creams?|teas?|oils?)\b", re.I
)
CLAUSE_BOUNDARY = re.compile(r"[,;:!?/&+]|\n\s*\n|\b(?:and|whereas|while|but|versus|vs)\b", re.I)
UNCERTAIN_SCOPES = {'MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION', 'PARAGRAPH_CONTEXT_UNCERTAIN_RELATION'}

# Bounded word groups for reading the original indication, not disease codes or
# an anatomical ontology. A shared group never establishes a shared endpoint.
SITE_WORDING = {
    'mouth_teeth_gums': r'\b(?:mouthwash(?:es)?|mouth|teeth|tooth|dental|gingiv\w*|periodont\w*|gums)\b',
    'stomach_bowel': r'\b(?:stomach\w*|bowels?|intestin\w*|gastrointestin\w*|abdom\w*|gastric|colonic|ileocolonic)\b',
    'joints_sinews': r'\b(?:joints?|sinews?|tendons?|articular|arthritis|arthritic)\b',
    'skin': r'\b(?:skin|cutaneous|dermal|dermat\w*)\b',
    'lungs': r'\b(?:lungs?|pulmonary|bronch\w*)\b',
    'womb': r'\b(?:womb|uterus|uterine)\b',
    'kidney_bladder': r'\b(?:kidneys?|renal|bladder)\b',
    'eyes': r'\b(?:eyes|ocular|ophthalm\w*)\b',
}


def site_wording(text, source_kind, offset=0):
    """Keep literal source offsets; neither infer a diagnosis nor an outcome."""
    return sorted([
        {'group': group, 'text': match.group(), 'start': offset + match.start(),
         'end': offset + match.end(), 'source_kind': source_kind}
        for group, pattern in SITE_WORDING.items() for match in re.finditer(pattern, text, re.I)
    ], key=lambda span: (span['start'], span['end']))


def compare_indication_wording(context, paper):
    historical = site_wording(context['support']['text'], 'HISTORICAL_INPUT', context['support']['start'])
    aim = next(({'text': match.group(), 'start': match.start(), 'end': match.end(), 'source_kind': 'PUBLICATION_ABSTRACT'}
                for match in re.finditer(r'[^.!?\n]+(?:[.!?]+|$)', paper['abstract'])
                if len(match.group().strip()) >= 40 and re.search(r'\b(?:aim(?:ed|s)?|objectives?)\b', match.group(), re.I)), None)
    # The title and abstract are kept separate so their offsets remain checkable.
    modern = site_wording(paper['title'], 'PUBLICATION_TITLE') + (
        site_wording(aim['text'], 'PUBLICATION_ABSTRACT', aim['start']) if aim else site_wording(paper['abstract'], 'PUBLICATION_ABSTRACT'))
    old_groups, new_groups = {s['group'] for s in historical}, {s['group'] for s in modern}
    shared = old_groups & new_groups
    status = ('SOME_SITE_WORDING_OVERLAPS' if shared else 'DIFFERENT_EXPLICIT_SITE_WORDING') if historical and modern else
   'SITE_WORDING_NOT_ESTABLISHED_ON_BOTH_SIDES'
```

```
    return {
        'status': status, 'historical_wording': historical, 'study_wording': modern,
        'shared_groups': sorted(shared), 'publication_id': paper['id'], 'study_aim': aim,
        'study_wording_scope': 'TITLE_AND_EXPLICIT_ABSTRACT_AIM' if aim else 'TITLE_AND_ABSTRACT; no explicit aim sentence found',
        'detail': ('The source statement and this publication name different site word groups. Read the specific indication and endpoint; a shared search topic does not establish equivalence.'
                   if status == 'DIFFERENT_EXPLICIT_SITE_WORDING' else
                   'Some site words overlap, but the indication, population and measured endpoint still need comparison.' if shared else
                   'This limited word screen does not establish comparable sites on both sides. Read the exact statement and publication.'),
        'method': 'LITERAL_SITE_WORD_GROUPS_NOT_DIAGNOSTIC_OR_OUTCOME_EQUIVALENCE',
        'scope': 'Title/abstract words can describe background, mixtures or secondary outcomes. Absence from these groups does not establish absence from the source. No efficacy relation is changed.',
    }


def occurrences(text, name):
    return list(compiled(name_pattern(name)).finditer(text))


def scope_study_preparation(paper, taxon, study):
    """Prefer nearby candidate-linked wording; withhold a single form if unresolved.

    Common-name-only signals stay separate from scientific-name-grounded signals.
    Links cannot cross coordination or clause boundaries. Every proposed form keeps
    exact abstract offsets and its surrounding source window.
    """
    study = deepcopy(study)
    abstract = paper['abstract']
    scientific = [taxon['name']] + [a['name'] for a in taxon['aliases'] if a['kind'] in ('SYNONYM', 'INPUT_SYNONYM_MATCH')]
    grounded_abstract = any(occurrences(paper['title'] + ' ' + abstract, name) for name in scientific)
    grounded_methods = any(occurrences(context['text'], name) for context in paper.get('methods_contexts', []) for name in scientific)
    grounded = grounded_abstract or grounded_methods
    anchors = []
    names = scientific + [a['name'] for a in taxon['aliases']]
    for name in dict.fromkeys(names):
        for match in occurrences(abstract, name):
            anchors.append((match.start(), match.end(), name, name in scientific))
    proposals, linked = [], []
    for match in PREPARATIONS.finditer(abstract):
        context_start, context_end = max(0, match.start() - 60), min(len(abstract), match.end() + 100)
        proposal = {'text': match.group(), 'start': match.start(), 'end': match.end(),
                    'context': {'text': abstract[context_start:context_end], 'start': context_start, 'end': context_end}}
        proposals.append(proposal)
        nearby = []
        for start, end, name, is_scientific in anchors:
            left, right = (end, match.start()) if end <= match.start() else (match.end(), start)
            if right < left or right - left > 100:
                continue
            between = abstract[left:right]
            if CLAUSE_BOUNDARY.search(between):
                continue
            # Do not jump across a complete sentence to attach an unrelated form.
            if re.search(r'\.\s+[A-Z]', between):
                continue
            nearby.append((not is_scientific, right - left, start, end, name))
        if not nearby:
            continue
        _, distance, start, end, name = min(nearby)
```

```
        context_start = max(0, min(start, match.start()) - 60)
        context_end = min(len(abstract), max(end, match.end()) + 100)
        linked.append({**proposal, 'candidate_wording': name,
                       'name_span': {'text': abstract[start:end], 'start': start, 'end': end},
                       'distance': distance,
                       'context': {'text': abstract[context_start:context_end], 'start': context_start, 'end': context_end}})
    for context in paper.get('methods_contexts', []):
        if context.get('source_kind') != 'FULL_TEXT_METHODS_TABLE' or context.get('reading_role') != 'INTERVENTION_FORMULATIONS':
            continue
        source = {k: context[k] for k in ('id', 'url', 'licence')}
        source = {'source_kind': context['source_kind'], 'source_id': source['id'], 'url': source['url'], 'licence': source['licence']}
        for row in context.get('rows', []):
            anchor = next(((name, match) for name in scientific for match in occurrences(row['text'], name)), None)
            if not anchor:
                continue
            name, name_match = anchor
            for match in PREPARATIONS.finditer(row['text']):
                linked.append({
                    'text': match.group(), 'start': row['start'] + match.start(), 'end': row['start'] + match.end(),
                    **source, 'candidate_wording': name,
                    'name_span': {'text': name_match.group(), 'start': row['start'] + name_match.start(), 'end': row['start'] + name_match.end(), **source},
                    'distance': abs(name_match.start() - match.start()),
                    'context': {'text': row['text'], 'start': row['start'], 'end': row['end'], **source},
                })
    forms = {family(p['text']) or p['text'].casefold() for p in linked}
    status = ('CANDIDATE_LINKED_WORDING' if grounded else 'COMMON_NAME_ONLY_PREPARATION_WORDING') if len(forms) == 1 else 'MULTIPLE_CANDIDATE_FORMS' if forms else
'CANDIDATE_PREPARATION_UNRESOLVED'
    representative = min(linked, key=lambda p: (p.get('source_kind') != 'FULL_TEXT_METHODS_TABLE', p['distance'])) if len(forms) == 1 else None
    study.setdefault('pico_source_spans', {})['intervention_preparation'] = (
        {k: representative[k] for k in ('text', 'start', 'end', 'source_kind', 'source_id', 'url', 'licence') if k in representative} if representative else None
    )
    study['preparation_attribution'] = {
        'status': status, 'candidate_taxon_id': taxon['id'], 'forms': linked,
        'scientific_grounding_scope': 'TITLE_OR_ABSTRACT' if grounded_abstract else 'FULL_TEXT_METHODS' if grounded_methods else 'COMMON_NAME_ONLY',
        'other_preparation_wording': [p for p in proposals if not any(p['start'] == x['start'] and not x.get('source_kind') for x in linked)],
        'method': 'CANDIDATE_NAME_AND_LOCAL_PREPARATION_WORDING_WITH_ATTRIBUTED_FORMULATION_TABLE_ROWS',
        'methods_context_ids': sorted({paragraph['id'] for table in paper.get('methods_contexts', [])
                                       if table.get('source_kind') == 'FULL_TEXT_METHODS_TABLE'
                                       and any(form.get('source_id') == table['id'] for form in linked)
                                       for paragraph in paper.get('methods_contexts', [])
                                       if paragraph['section'] == table.get('referenced_from_methods_section')}),
        'note': 'Lexical association only. A named preparation may be one ingredient of a mixture; dose, intervention and causal attribution need source review.',
    }
    return study


def index_historical_contexts(extracted, resolutions):
    """Join exact stored statements through their mention/candidate/topic records."""
    mentions = {m['id']: m for m in extracted['mentions']}
    candidates = {r['mention_id']: {c['taxon_id'] for c in r['candidates']} for r in resolutions}
    indexed = {}
    for claim in extracted['historical_claims']:
        if claim.get('statement_role') != 'THERAPEUTIC_USE_RECORDED':
            continue
        taxa = {taxon for mid in claim['mention_ids'] for taxon in candidates.get(mid, set())}
        for taxon in taxa:
            for condition in {c['id'] for c in claim['conditions']}:
                indexed.setdefault((taxon, condition), []).append({
```

```
                    'claim_id': claim['id'], 'support': dict(claim['support']),
                    'plant_wording': [dict(mentions[mid]) for mid in claim['mention_ids'] if taxon in candidates.get(mid, set()) and mid in mentions],
                    'condition_wording': [dict(c) for c in claim['conditions'] if c['id'] == condition],
                    'preparation': deepcopy(claim['preparation']), 'part': deepcopy(claim['part']),
                    'route': deepcopy(claim['route']), 'dosage_expressions': deepcopy(claim['dosage_expressions']),
                    'relation_scope': claim['relation_scope'], 'statement_role': claim['statement_role'],
                    'subject_uncertain': claim['relation_scope'] in UNCERTAIN_SCOPES, 'possible_subjects': claim.get('possible_subjects', []),
                    'negated': bool(claim.get('negated')),
                    'link_basis': 'STORED_MENTION_CANDIDATE_AND_LEXICAL_TOPIC; not a confirmed historical identity or diagnostic equivalence',
                })
    return indexed


def compare_historical_contexts(contexts, study, paper):
    return [{**deepcopy(context), 'preparation_comparison': check_preparation(context['preparation'], study),
             'indication_comparison': compare_indication_wording(context, paper)} for context in contexts]


def preparation_across_statements(contexts):
    checks = [c['preparation_comparison'] for c in contexts]
    if not checks:
        return {'check': 'preparation', 'status': UNKNOWN, 'detail': 'No eligible source statement is linked to this candidate and topic.',
    'requires_statement_review': True}
    if len(checks) == 1:
        return {**checks[0], 'claim_ids': [contexts[0]['claim_id']]}
    same = len({(c['status'], c['detail']) for c in checks}) == 1
    if same:
        return {**checks[0], 'claim_ids': [c['claim_id'] for c in contexts]}
    # Do not let one statement's oil stand in for a different statement with no form.
    return {'check': 'preparation', 'status': UNKNOWN,
            'detail': 'The linked historical statements have different or unstated preparations. Inspect each original statement; no pooled preparation comparison is
     assigned.',
            'requires_statement_review': True, 'claim_ids': [c['claim_id'] for c in contexts]}
```

src/evidence/counterfactual.py
SHA-256: ec7db1d4898ec33624bea68a5442a1e89b45af78b992349bc1fd0315e8a2a7c5

```
"""Identity counterfactuals: what changes downstream if a name refers to another taxon.

For every historical name with candidates, each candidate is a branch. Branches are
compared on the evidence they retrieve, the designs and blocking checks behind it,
and the papers that only one branch reaches. A single-answer view is reconstructed so
the reader can see what it would silently drop.
"""
from __future__ import annotations

from collections import Counter

from src.nlp.extract import normalize


def name_groups(result):
    """Collapse repeated mentions of one name; keep every occurrence span."""
    groups = {}
    resolutions = {r["mention_id"]: r for r in result["resolutions"]}
    for mention in result["mentions"]:
        resolution = resolutions.get(mention["id"])
        if not resolution:
            continue
        name = mention.get("heading_name") or mention.get("lookup") or mention["text"]
        key = normalize(name).replace("camomile", "chamomile").replace("featherfew", "feverfew")
        # The names of one list that reach one plant are one question for the reader.
        listed = mention.get("list_id", mention.get("listed_in"))
        if listed is not None and len(resolution["candidates"]) == 1:
            key = f"list-{listed}-{resolution['candidates'][0]['taxon_id']}"
        group = groups.setdefault(key, {"key": key, "label": mention.get("heading_name") or mention["text"], "forms": mention.get("forms", []), "mention_ids": [],
   "occurrences": [], "status": resolution["status"], "candidates": resolution["candidates"], "open_world": resolution.get("open_world_alternatives", [])})
        group["mention_ids"].append(mention["id"])
        group.setdefault("source_names", []).extend(n for n in resolution.get("source_names", []) if n["name"] not in {x["name"] for x in group["source_names"]})
        group["occurrences"].append({"start": mention["start"], "end": mention["end"], "text": mention["text"]})
    for group in groups.values():
        names = list(dict.fromkeys(o["text"] for o in group["occurrences"]))
        if group["key"].startswith("list-") and len(names) > 1:
            group["label"] = f"{names[0]} and {len(names) - 1} more listed name{'s' if len(names) > 2 else ''}"
    claim_conditions = {}
    for claim in result["historical_claims"]:
        if claim.get('statement_role') != 'THERAPEUTIC_USE_RECORDED':
            continue
        for mention_id in claim["mention_ids"]:
            claim_conditions.setdefault(mention_id, set()).update(c["id"] for c in claim["conditions"])
    for group in groups.values():
        group["conditions"] = sorted({c for m in group["mention_ids"] for c in claim_conditions.get(m, set())})
        group["claim_ids"] = [c["id"] for c in result["historical_claims"] if set(c["mention_ids"]) & set(group["mention_ids"])]
    return list(groups.values())


def heading_forms(group):
    return group.get("forms") or []


def species_named(row):
    """Identity check status; None when the record was never screened (legacy analyses)."""
```

```
    check = next((c for c in (row.get("comparability") or {}).get("checks", []) if c["check"] == "identity"), None)
    return None if check is None else check["status"] == "PASS"


def blocking_key(row):
    """No blocking check is not a pass: partial or unknown checks are counted separately."""
    screen = row.get("comparability") or {}
    if screen.get("first_blocking_check"):
        return screen["first_blocking_check"]
    return "NONE_UNRESOLVED" if any(c["status"] != "PASS" for c in screen.get("checks", [])) else "NONE"


def branch_state(rows):
    if not rows:
        return "NO_EVIDENCE_IN_SNAPSHOT"
    if any(species_named(r) is None for r in rows):
        return "COMPARABILITY_NOT_ASSESSED"
    if not any(species_named(r) for r in rows):
        return "COMMON_NAME_ONLY_EVIDENCE"
    if any(r["relation"]["relation"] in ("SUPPORTS", "CONTRADICTS", "MIXED") for r in rows):
        return "SPECIES_SPECIFIC_FINDINGS"
    return "SPECIES_SPECIFIC_BUT_INSUFFICIENT"


def summarize_branch(candidate, rows):
    species = [r for r in rows if species_named(r)]
    return {
        "taxon_id": candidate["taxon_id"],
        "name": candidate["name"],
        "score": candidate["score"],
        "reasons": candidate["reasons"],
        "evidence_ids": [r["id"] for r in rows],
        "papers": sorted({r["paper"]["id"] for r in rows}),
        "species_specific_papers": sorted({r["paper"]["id"] for r in species}),
        "designs": dict(Counter(t for r in {r["paper"]["id"]: r for r in rows}.values() for t in r["study"]["types"])),
        "relations": dict(Counter(r["relation"]["relation"] for r in rows)),
        "blocking_checks": dict(Counter(blocking_key(r) for r in rows)),
        "reported_findings": dict(Counter((r["relation"].get("reported_finding") or {}).get("direction", "NO_RESULT_STATEMENT") for r in rows)),
        "state": branch_state(rows),
    }


def counterfactuals(result):
    out = []
    for group in name_groups(result):
        candidates = group["candidates"]
        if not candidates:
            out.append({**{k: group[k] for k in ("key", "label", "forms", "status", "mention_ids", "occurrences", "conditions", "claim_ids", "open_world",
    "source_names")}, "branches": [], "sensitivity": "NO_CANDIDATE", "single_answer_view": None, "pairs": []})
            continue
        branches = []
        for candidate in candidates:
            rows = [e for e in result["evidence"] if e["taxon_id"] == candidate["taxon_id"] and e["condition_id"] in group["conditions"]]
            branches.append(summarize_branch(candidate, rows))
        pairs = []
        for i, a in enumerate(branches):
            for b in branches[i + 1:]:
                pa, pb = set(a["papers"]), set(b["papers"])
```

```
                union = pa | pb
                pairs.append({"a": a["taxon_id"], "b": b["taxon_id"], "shared_papers": sorted(pa & pb), "only_a": sorted(pa - pb), "only_b": sorted(pb - pa),
    "jaccard": round(len(pa & pb) / len(union), 3) if union else None, "state_changes": a["state"] != b["state"]})
        top = branches[0]
        tie = len(candidates) > 1 and candidates[0]["score"] == candidates[1]["score"]
        hidden = [b for b in branches[1:]]
        union_papers = set().union(*(set(b["papers"]) for b in branches))
        union_species = set().union(*(set(b["species_specific_papers"]) for b in branches))
        other_species = set().union(set(), *(set(b["species_specific_papers"]) for b in hidden))
        # Papers the top branch reaches only through the shared common name, although they name another candidate.
        misattributed = sorted((set(top["papers"]) & other_species) - set(top["species_specific_papers"]))
        states = {b["state"] for b in branches}
        if len(branches) == 1 and not group["open_world"]:
            sensitivity = "SINGLE_CANDIDATE_IN_SNAPSHOT"
        elif len(branches) == 1:
            sensitivity = "OPEN_WORLD_ALTERNATIVES_NOT_RETRIEVED"
        elif len(states) > 1:
            sensitivity = "CONCLUSION_CHANGES_WITH_IDENTITY"
        elif any(p["only_a"] or p["only_b"] for p in pairs):
            sensitivity = "EVIDENCE_SET_CHANGES_WITH_IDENTITY"
        else:
            sensitivity = "STABLE_ACROSS_CANDIDATES"
        single = {
            "taxon_id": top["taxon_id"],
            "name": top["name"],
            "tie_broken_arbitrarily": tie,
            "selection_note": "Top candidate tied with the next; a single-answer system would pick by sort order" if tie else "Highest ranking feature; not a
    probability",
            "hidden_branches": [b["taxon_id"] for b in hidden],
            "hidden_papers": sorted(union_papers - set(top["papers"])),
            "hidden_species_specific_papers": sorted(union_species - set(top["species_specific_papers"]) - set(top["papers"])),
            "misattributed_papers": misattributed,
            "shown_papers": len(top["papers"]),
            "would_state": top["state"],
            "misleading_if": "Shown without the alternative branches" if len(branches) > 1 or group["open_world"] else None,
        }
        out.append({**{k: group[k] for k in ("key", "label", "forms", "status", "mention_ids", "occurrences", "conditions", "claim_ids", "open_world",
    "source_names")}, "branches": branches, "pairs": pairs, "sensitivity": sensitivity, "single_answer_view": single})
    return out
```

src/evidence/digest.py
SHA-256: 6a4d357694f231107476fa10babca665cdd3bdf40278fca7586a17a88d2afbf8

```
"""One readable line per plant and topic: what the human research in the snapshot reports.

Kept apart from the strict statement-level relation, which asks whether a study answers one
historical sentence. This asks what is known about the species and topic at all, so it reads
every paper naming both, not only the ones retrieved for the evidence table.
"""
from __future__ import annotations

import re
from collections import Counter, defaultdict
from functools import lru_cache

from src.evidence.independent_use import independent_use
from src.evidence.regulatory import regulatory
from src.evidence.classify import animal_signal, study_sentences
from src.nlp.extract import LATIN_BINOMIAL, LATIN_EPITHET, RESEARCH_TERMS, SENSE_REQUIRED, normalize, research_terms, topic_pattern

HUMAN = {'CLINICAL_TRIAL', 'META_ANALYSIS', 'SYSTEMATIC_REVIEW', 'OBSERVATIONAL_HUMAN'}
SYNTHESIS = {'META_ANALYSIS', 'SYSTEMATIC_REVIEW'}
PRECLINICAL = {'IN_VITRO', 'IN_VIVO_ANIMAL'}
STATED = ('SUPPORTS', 'CONTRADICTS', 'MIXED', 'INSUFFICIENT')
DISPLAYED_STUDIES = 6
SYNTHESIS_WINDOW = 10

NOT_THE_PLANT = re.compile(r'\b(?:immunotherapy|allergen\w*|pollen allerg\w*|acupoint\w*|acupuncture|moxibustion|polyherbal|medicinal plants|plants?
(?:used|for)|ethnobotan\w*|ethnomedicin\w*|phytochemi\w*|pharmacognos\w*|pre ?clinical|in vitro|animal (?:studies|models?)|\w+ induced
(?:hyperalgesia|allodynia|flare|erythema))\b')
# Trial labels also cover livestock and fish: a randomized trout study is not a human study.
# Whether it harms or interacts is a different question from whether it works.
NOT_EFFICACY = re.compile(r'\b(?:adverse
(?:events?|effects?|reactions?)|safety|toxicity|hepatotoxic\w*|pharmacokinetic\w*|interactions?|allerg\w*|poisoning|intoxication|case reports?)\b')
GENERAL_REVIEW = re.compile(r'\b(?:herbal|herbs?|botanicals?|natural|phytotherap\w*|phytomedicin\w*|nutraceuticals?|dietary
supplements?|complementary|alternative|medicines?|remedies)\b')
# Who was studied, as opposed to what was measured: "dermatitis in patients with breast cancer".
POPULATION = re.compile(r'\b(?:(?:in|among|for|of|with)\s+(?:\w+\s+){0,4}?(?:patients?|women|men|adults|children|infants|subjects|survivors|people|individuals|volunt
eers|participants)\b|(?:during|undergoing|receiving|following)\b|\w+\s+patients\b).*$')
SUPPORTIVE_CARE = re.compile(r'\b(?:dermatitis|mucositis|stomatitis|xerostomia|nausea|vomiting|fatigue|neuropathy|hot
(?:flash|flush)\w*|lymphedema|cachexia|radiation|radiotherapy|irradiation|(?:chemotherapy|radiotherapy|cancer|treatment)[ -](?:induced|related)|side effects)\b')
COMBINATION = re.compile(r'\b(?:plus|combin\w+|compound|formula\w*|mixture|herbal (?:tea|product|preparation|medicine)s?)\b|\+')
PREPARATION_WORDS = 'extracts?|oils?|roots?|leaf|leaves|flowers?|fruits?|berry|berries|seeds?|bark|herb|herbs|tea|tablets?|capsules?|cream|ointment|gel|tincture|syru
p|species|for|in|and|on|as|with|of|the|versus|vs|or|to|preparations?|products?|supplements?|therapy|treatment|use|may|can|is|are|was|were|has|have|improves?|reduces?
|does|did|not|spp'
OUTCOMES = None
HOLDERS = {}


def outcome_pattern():
    """Any topic wording, to tell a title with its own outcome from one naming only a population."""
    global OUTCOMES
    if OUTCOMES is None:
        OUTCOMES = topic_pattern(tuple(sorted({t for terms in RESEARCH_TERMS.values() for t in terms} | {'quality of life', 'cachexia', 'fatigue'})))
    return OUTCOMES


@lru_cache(maxsize=4096)
```

```
def strip_population(text):
    return POPULATION.sub('', text)


def targets_topic(text, pattern):
    """The topic is what the study measured, not only who took part."""
    if not pattern.search(text):
        return False
    remainder = strip_population(text)
    return bool(pattern.search(remainder)) or not outcome_pattern().search(remainder)


def name_pattern(names):
    return re.compile(r'\b(?:' + '|'.join(re.escape(name) for name in sorted(names, key=len, reverse=True)) + r')(?:s|es)?\b') if names else None


def on_subject(row, names, terms):
    """Counts as evidence for this plant and topic only if the study is about both.

    The plant must lead: in the title, or named repeatedly in an abstract whose title gives only a
    product code. The topic must be in the title or in the sentence reporting the result; elsewhere
    in the abstract it may be a secondary measure or a description of the participants.
    """
    title = normalize(row['paper']['title'])
    scientific = {normalize(n) for n in row.get('branch_scientific_names', ())}
    genera = {n.split()[0] for n in scientific}
    named_in_title = {normalize(n) for n in LATIN_BINOMIAL.findall(row['paper']['title']) if normalize(n).split()[0] in genera and
LATIN_EPITHET.search(normalize(n).split()[-1]) and not re.fullmatch(PREPARATION_WORDS, normalize(n).split()[-1])}
    if named_in_title and not named_in_title & scientific:
        # A background mention of this species cannot inherit a study explicitly
        # titled for a different species of the same genus.
        return False
    if NOT_THE_PLANT.search(title):
        return False
    if NOT_EFFICACY.search(title) and not re.search(r'\b(?:efficacy|effectiveness)\b', title):
        return False
    wording = name_pattern(names)
    if not wording:
        return False
    absent = re.compile(r'\b(?:not (?:taking|receiving|using)|without|naive to|no (?:prior )?(?:use of|treatment with))\s+(?:' + '|'.join(re.escape(name) for name in
 names) + r')\b')
    if absent.search(title):
        return False
    finding = (row['relation'].get('reported_finding') or {}).get('supporting_text') or ''
    if not wording.search(title):
        # A title may give only a product code or another name; the abstract then has to name it.
        explicit_result = wording.search(normalize(finding)) and direction_of(row) in STATED
        focused_study = set(row['study']['types']) & ({'CLINICAL_TRIAL'} | SYNTHESIS) and (not GENERAL_REVIEW.search(title) or explicit_result)
        if not focused_study or not wording.search(normalize(row['paper']['abstract'])):
            return False
    pattern = topic_pattern(tuple(terms))
    if 'cancer' in terms and SUPPORTIVE_CARE.search(title):
        return False
    return targets_topic(title, pattern) or targets_topic(normalize(finding), pattern)


def combination(row, names, known):
    """Tested alongside another plant or as a mixture, so the result is not this plant's alone."""
```

```
    title = row['paper']['title']
    if COMBINATION.search(title.casefold()):
        return True
    other = {normalize(name) for name in re.findall(LATIN_BINOMIAL.pattern, title, re.I) if normalize(name) in known and normalize(name) not in names}
    if not other:
        return False
    # A title can name two separately tested ointments. An explicit allocation
    # to named preparation groups is evidence of separate arms, not a mixture.
    prep = r'(?:ointments?|extracts?|oils?|creams?|gels?|teas?|tinctures?|capsules?|tablets?)'
    own_form = re.compile(r'\b(?:' + '|'.join(re.escape(n) for n in names) + r')\s+' + prep + r'\b') if names else None
    other_form = re.compile(r'\b(?:' + '|'.join(re.escape(n) for n in other) + r')\s+' + prep + r'\b')
    allocation =
re.compile(r'\b(?:two|2|three|3|four|4|separate|different)\s+(?:(?:active|case|treatment|intervention)\s+)?(?:groups|arms)\s+(?:including|receiv\w*|treated with)\b')
    for sentence in study_sentences(row['paper'].get('abstract') or ''):
        text = normalize(sentence['text'])
        match = allocation.search(text)
        if match and own_form and own_form.search(text[match.end():]) and other_form.search(text[match.end():]):
            return False
    return True


def direction_of(row):
    return (row['relation'].get('reported_finding') or {}).get('direction', 'NO_RESULT_STATEMENT')


def verdict(directions):
    """Favourable or not when two in three stated results agree; otherwise mixed or inconclusive."""
    positive, negative = directions['SUPPORTS'], directions['CONTRADICTS']
    uncertain = directions['MIXED'] + directions['INSUFFICIENT']
    stated = positive + negative + uncertain
    if not stated:
        return None
    if positive / stated >= 2 / 3:
        return 'HUMAN_FINDINGS_FAVOURABLE'
    if negative / stated >= 2 / 3:
        return 'HUMAN_NO_BENEFIT_REPORTED'
    return 'HUMAN_FINDINGS_MIXED'


def current_syntheses(rows):
    """An updated review replaces its earlier edition, and a decade-old synthesis no longer decides."""
    latest = {}
    for row in sorted(rows, key=lambda r: str(r['paper'].get('year') or 0)):
        latest[normalize(row['paper']['title'])] = row
    years = [int(str(r['paper'].get('year') or 0)[:4] or 0) for r in latest.values()]
    return [r for r, year in zip(latest.values(), years) if year >= max(years, default=0) - SYNTHESIS_WINDOW]


def unique_papers(rows):
    """Count records with the same DOI or PMID once, even across repositories."""
    seen, unique = set(), []
    for row in rows:
        paper = row['paper']
        keys = {('id', paper['id'])}
        doi = re.sub(r'^(?:https?://(?:dx\.)?doi\.org/|doi:\s*)', '', str(paper.get('doi') or '').strip(), flags=re.I).casefold()
        if doi:
            keys.add(('doi', doi))
        pmid = paper.get('pmid') or (re.fullmatch(r'PMID(\d+)', str(paper['id']), re.I) or [None, None])[1]
```

```
        if pmid:
            keys.add(('pmid', str(pmid)))
        duplicate = bool(keys & seen)
        seen.update(keys)
        if not duplicate:
            unique.append(row)
    return unique


def synthesis_verdict(rows):
    """Conflicting reviews do not become independent majority votes for benefit."""
    labels = {direction_of(r) for r in rows} & set(STATED)
    if not labels:
        return None
    if labels == {'SUPPORTS'}:
        return 'HUMAN_FINDINGS_FAVOURABLE'
    if labels == {'CONTRADICTS'}:
        return 'HUMAN_NO_BENEFIT_REPORTED'
    return 'HUMAN_FINDINGS_MIXED'


def state_of(rows, names=(), terms=(), distinctive=(), known=frozenset()):
    """Reported direction across on-subject studies, by the strongest design that states one."""
    rows = unique_papers(rows)
    named = [r for r in rows if identified(r, distinctive)]
    focused = [r for r in named if on_subject(r, names, terms)]
    human = [r for r in focused if is_human(r)]
    isolated = [r for r in human if not combination(r, names, known)]
    # A broad review's explicitly attributed finding is evidence too. Its other
    # interventions have already been excluded by candidate/result screening.
    syntheses = [r for r in isolated if set(r['study']['types']) & SYNTHESIS]
    current = current_syntheses(syntheses)
    directions = Counter(direction_of(r) for r in human)
    isolated_directions = Counter(direction_of(r) for r in isolated)
    preclinical = [r for r in focused if set(r['study']['types']) & PRECLINICAL or animal_signal(r['paper']['title'])]
    basis = current if synthesis_verdict(current) else isolated
    basis_kind = 'CURRENT_SYNTHESES' if synthesis_verdict(current) else 'INDIVIDUAL_PLANT_STUDIES'
    if human:
        synthesised = synthesis_verdict(current)
        state = synthesised or verdict(isolated_directions) or 'HUMAN_STUDIED_RESULT_UNCLEAR'
        # Two small trials are a reason to look further, not a finding: Cochrane calls most such
        # records insufficient. "Favourable" needs a current synthesis or three favourable trials.
        favourable_trials = sum(direction_of(r) == 'SUPPORTS' and 'CLINICAL_TRIAL' in r['study']['types'] for r in isolated)
        if state == 'HUMAN_FINDINGS_FAVOURABLE' and not synthesised and favourable_trials < 3:
            state = 'HUMAN_FINDINGS_EARLY'
        if not isolated:
            basis_kind = 'COMBINATION_ONLY'
            basis = human
    elif preclinical:
        state = 'PRECLINICAL_ONLY'
    elif named:
        state = 'SPECIES_NAMED_NO_TESTED_RESULT'
    elif rows:
        state = 'COMMON_NAME_ONLY'
    else:
        state = 'NO_STUDY_RETRIEVED'
    if not human:
        basis_kind, basis = ('PRECLINICAL_ONLY', preclinical) if preclinical else ('NO_ELIGIBLE_HUMAN_RESULT', [])
```

```
    basis_directions = Counter(direction_of(r) for r in basis)
    assessment = {'kind': basis_kind, 'paper_ids': [r['paper']['id'] for r in basis],
                  'records': [{'paper_id':r['paper']['id'],'title':r['paper']['title'],'year':r['paper'].get('year'),'url':r['paper']['url'],'direction':direction_of(r),'design':r['study']['types']} for r in basis],
                  'directions': dict(basis_directions), 'review_window_years': SYNTHESIS_WINDOW,
                  'conflicting_results': bool(basis_directions['SUPPORTS'] and basis_directions['CONTRADICTS']),
                  'uncertain_results': bool(basis_directions['MIXED'] or basis_directions['INSUFFICIENT']),
                  'scope': 'Automated reading of retrieved abstracts for this species and topic. Reviews may share trials; their number is not a count of independent replications. Preparations, populations and endpoints can differ. Not a risk-of-bias appraisal or pooled effect estimate.'}
    return state, {'human_studies': len(human), 'isolated_plant_studies':len(isolated), 'syntheses': len(syntheses), 'preclinical_studies': len(preclinical), 'favourable': directions['SUPPORTS'], 'no_benefit': directions['CONTRADICTS'], 'mixed': directions['MIXED'] + directions['INSUFFICIENT'], 'publications': len(rows), 'assessment_basis': assessment}


def is_human(row):
    return bool(set(row['study']['types']) & HUMAN) and 'STUDY_PROTOCOL' not in row['study']['types'] and row['study'].get('population_status') != 'ANIMAL_TEXT_SIGNAL' and not animal_signal(row['paper']['title'])


def genus_alone(title, genus):
    """Crataegus extract" means the leading species; "Crataegus pinnatifida" means another one."""
    return bool(re.search(rf'\b{genus}\b(?!\s+(?!(?:{PREPARATION_WORDS})\b)[a-z]{{3,}})', title))


def identified(row, distinctive):
    """A scientific name, or a common name that only this species holds in the snapshot."""
    if row.get('study_identity') == 'SCIENTIFIC_NAME':
        return True
    title = normalize(row['paper']['title'])
    if any(re.search(r'\b' + re.escape(name) + r'(?:s|es)?\b', title) for name in distinctive if ' ' in name or not name.istitle()):
        return True
    return any(genus_alone(row['paper']['title'], name) for name in distinctive if name.istitle())


def study_card(row, names, known):
    finding = row['relation'].get('reported_finding') or {}
    return {'paper_id': row['paper']['id'], 'title': row['paper']['title'], 'year': row['paper'].get('year'), 'url': row['paper']['url'], 'design': row['study']['types'], 'finding': finding.get('supporting_text'), 'finding_statements': finding.get('statements', []), 'direction': finding.get('direction'), 'combination': combination(row, names, known), 'identity': row.get('study_identity')}


def human_order(row):
    """Syntheses first, then trials; a stated result ahead of none; recent ahead of old."""
    types = set(row['study']['types'])
    return (0 if types & SYNTHESIS else 1 if 'CLINICAL_TRIAL' in types else 2, direction_of(row) not in STATED, -int(str(row['paper'].get('year') or 0)[:4] or 0))


def human_studies(rows, names, terms, distinctive):
    return sorted([r for r in unique_papers(rows) if identified(r, distinctive) and on_subject(r, names, terms) and is_human(r)], key=human_order)


def best_study(rows, names=(), terms=(), distinctive=(), known=frozenset()):
    """The strongest on-subject study that states a result; a review without one is not shown first."""
    human = [r for r in human_studies(rows, names, terms, distinctive) if direction_of(r) in STATED]
    if human:
        return study_card(human[0], names, known)
    focused = [r for r in rows if identified(r, distinctive) and on_subject(r, names, terms) and set(r['study']['types']) & PRECLINICAL]
    return study_card(max(focused, key=lambda r: (direction_of(r) in STATED, str(r['paper'].get('year')))), names, known) if focused else None
```

```


def common_name_holders(taxmap):
    """Who holds each English common name, rebuilt when an on-demand plant joins."""
    holders = defaultdict(set)
    for taxon in taxmap.values():
        for alias in taxon.get('aliases', []):
            if alias.get('lang', 'en') == 'en' and alias['kind'] in ('VERNACULAR', 'HISTORICAL_LEXICON'):
                holders[normalize(alias['name'])].add(taxon['id'])
    return holders, frozenset(normalize(t['name']) for t in taxmap.values())


def outside_knowledge(taxon, condition_id):
    """What other sources say beside the studies. None of it can change a stated direction."""
    records = independent_use()
    return {'independent_use': records.reach(taxon, condition_id), 'relatives': records.relatives(taxon, condition_id), 'constituents': records.constituents(taxon,
condition_id), 'recorded_poisonous': records.recorded_poisonous(taxon), 'regulatory': regulatory().conclusion(taxon['id'], condition_id)}


def research_digest(result, taxmap=None, studies=None, literature_size=None):
    """Group studies by candidate and topic for the workspace summary and the evidence map."""
    taxmap = taxmap or {}
    sizes = literature_size or {}
    holders, known = common_name_holders(taxmap)
    genera = {normalize(t.get('name', '').split()[0]) for t in taxmap.values() if t.get('name')}

    def leads_name(taxon_id, name):
        """Literature volume never identifies a species holding an ambiguous name."""
        return holders[name] == {taxon_id}

    def leads_genus(taxon_id):
        # A genus-only paper leaves every species hypothesis open, including
        # genera for which the current snapshot happens to contain one species.
        return False

    def plant_names(taxon_id):
        aliases = (taxmap.get(taxon_id) or {}).get('aliases', [])
        drugs = {normalize(n) for n in (taxmap.get(taxon_id) or {}).get('drug_names', ()) if normalize(n) not in genera}
        genus = {normalize((taxmap.get(taxon_id) or {}).get('name', taxon_id)).split()[0]} if leads_genus(taxon_id) else set()
        return {normalize(a['name']) for a in aliases if a.get('lang', 'en') == 'en' and len(a['name']) >= 4 and a['kind'] != 'INPUT_SYNONYM_MATCH' and
normalize(a['name']) not in genera} | {normalize((taxmap.get(taxon_id) or {}).get('name', taxon_id))} | genus | drugs

    def distinctive_names(taxon_id):
        aliases = (taxmap.get(taxon_id) or {}).get('aliases', [])
        genus = {(taxmap.get(taxon_id) or {}).get('name', taxon_id).split()[0]} if leads_genus(taxon_id) else set()
        return {normalize(a['name']) for a in aliases if a.get('lang', 'en') == 'en' and a['kind'] in ('VERNACULAR', 'HISTORICAL_LEXICON') and len(a['name']) >= 4
and normalize(a['name']) not in SENSE_REQUIRED | genera and leads_name(taxon_id, normalize(a['name']))} | genus | {normalize(n) for n in (taxmap.get(taxon_id) or
{}).get('drug_names', ()) if normalize(n) not in genera}

    names = {c['taxon_id']: c['name'] for r in result['resolutions'] for c in r['candidates']}
    labels = {c['id']: c['label'] for c in result['conditions']}
    statements = Counter()
    for claim in result['historical_claims']:
        if claim['statement_role'] == 'THERAPEUTIC_USE_RECORDED' and not claim.get('negated') and claim.get('practice') != 'NON_PLANT_REMEDY':
            for condition in {c['id'] for c in claim['conditions']}:
                statements[condition] += 1
    rows = []
    for taxon_id in dict.fromkeys(e['taxon_id'] for e in result['evidence']):
```

```
        wording, distinctive = plant_names(taxon_id), distinctive_names(taxon_id)
        for condition_id in dict.fromkeys(e['condition_id'] for e in result['evidence'] if e['taxon_id'] == taxon_id):
            group = studies(taxon_id, condition_id) if studies else [e for e in result['evidence'] if e['taxon_id'] == taxon_id and e['condition_id'] ==
condition_id]
            terms = research_terms(condition_id)
            state, counts = state_of(group, wording, terms, distinctive, known)
            listed = [study_card(r, wording, known) for r in human_studies(group, wording, terms, distinctive)]
            rows.append({'taxon_id': taxon_id, 'taxon': names.get(taxon_id, taxon_id), 'condition_id': condition_id, 'topic': labels.get(condition_id, condition_id),
 'state': state, **outside_knowledge(taxmap.get(taxon_id) or {'id': taxon_id, 'name': names.get(taxon_id, taxon_id)}, condition_id), **counts,
'historical_statements': statements[condition_id], 'best_study': best_study(group, wording, terms, distinctive, known), 'studies': listed[:DISPLAYED_STUDIES],
'combination_only': bool(listed) and all(s['combination'] for s in listed)})
    weight = {'HUMAN_FINDINGS_FAVOURABLE': 0, 'HUMAN_FINDINGS_EARLY': 1, 'HUMAN_FINDINGS_MIXED': 1, 'HUMAN_NO_BENEFIT_REPORTED': 2, 'HUMAN_STUDIED_RESULT_UNCLEAR':
3, 'PRECLINICAL_ONLY': 4, 'SPECIES_NAMED_NO_TESTED_RESULT': 5, 'COMMON_NAME_ONLY': 6}
    rows.sort(key=lambda r: (weight.get(r['state'], 9), -r['human_studies'], -r['historical_statements']))
    return rows


def topic_digest(pipeline, taxon, condition_id):
    """The same reading for a plant and topic outside any chapter, used by evaluation."""
    fake = {'resolutions': [{'candidates': [{'taxon_id': taxon['id'], 'name': taxon['name']}]}], 'conditions': [{'id': condition_id, 'label': condition_id}],
'historical_claims': [], 'evidence': [{'taxon_id': taxon['id'], 'condition_id': condition_id}]}
    return research_digest(fake, pipeline.taxmap, pipeline.topic_studies, pipeline.literature_size)[0]
```

src/evidence/direction.py
SHA-256: f2aff2c8755c4d76dd135ba55acecc9f8bc30765185772b4d16fbcc7d8a6a362

```
"""Conservative outcome-direction baseline; direction is separate from benefit."""
import re
from src.nlp.extract import sentences, normalize


def outcome_direction(evidence, outcome, *, aware=False):
    # XML/annotation hard wraps are formatting, not sentence boundaries.
    evidence = re.sub(r'\s+', ' ', evidence)
    words={w for w in normalize(outcome).split() if len(w)>3 and w not in {'change','improvement','reduction','levels','score','scores','number','total'}}
    ranked=[]
    for s in sentences(evidence):
        overlap=len(words & set(normalize(s['text']).split()))
        ranked.append((overlap,s))
    if not ranked:return {'code':None,'reason':'No evidence sentence'}
    _,sentence=max(ranked,key=lambda pair:(pair[0],pair[1]['start']))
    text=sentence['text']
    if re.search(r'no\s+(?:statistically\s+)?(?:significant|difference)|not\s+(?:statistically\s+)?significant|similar|comparable',text,re.I):code=0
    else:
        up=bool(re.search(r'\bincreas\w*|\bhigher\b|\bgreater\b',text,re.I))
        down=bool(re.search(r'\bdecreas\w*|\breduc\w*|\blower\b|\bless\b',text,re.I))
        code=1 if up and not down else -1 if down and not up else None
        if aware:
            desirable_named = bool(re.search(r'\bimprovement\b|\breduction\b|\bamelioration\b',outcome,re.I))
            explicit_benefit = bool(re.search(r'\badvantage\b|\bamelioration\b|\bimprov\w*',text,re.I))
            if desirable_named and explicit_benefit:code=1
            elif desirable_named and down and not up:code=1
    return {'code':code,'supporting_text':text,'reason':'Lexical outcome-direction baseline; benefit depends on outcome semantics, comparator and intervention. No clinical recommendation.'}
```

src/evidence/handoff.py
SHA-256: c6c9a000d9be32cd001b828fd13d2fa609e949446cd450a16962ce214f9b0351

```python
"""Source-scoped, portable research decisions. These never alter model output."""
from __future__ import annotations

import hashlib
import json
import sqlite3
import uuid
from pathlib import Path
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field, model_validator
from src.common import now


class HandoffRequest(BaseModel):
    model_config = ConfigDict(extra='forbid')
    question: str = Field(min_length=5, max_length=600)
    claim_id: str | None = Field(None, max_length=120)
    candidate_ids: list[str] = Field(default_factory=list, max_length=30)
    decision: Literal['NEXT_READING', 'INSUFFICIENT_INFORMATION', 'EXCLUDE_FROM_THIS_QUESTION']
    reason: str = Field(min_length=5, max_length=3000)
    next_step: str = Field(min_length=5, max_length=2000)
    reviewer: str = Field(default='Local researcher', min_length=1, max_length=80)

    @model_validator(mode='after')
    def substantive(self):
        for key in ('question', 'reason', 'next_step', 'reviewer'):
            if len(getattr(self, key).strip()) < (1 if key == 'reviewer' else 5):
                raise ValueError(f'{key} must contain substantive text')
        if len(set(self.candidate_ids)) != len(self.candidate_ids):
            raise ValueError('Duplicate candidate assumptions')
        return self


class HandoffPacket(BaseModel):
    model_config = ConfigDict(extra='forbid')
    schema_version: Literal['foliatrace-handoff-1'] = 'foliatrace-handoff-1'
    created_at: str
    analysis_id: str = Field(pattern=r'^[a-f0-9]{24}$')
    snapshot_sha256: str = Field(pattern=r'^[a-f0-9]{64}$')
    code_fingerprint: str
    source: dict
    source_input: str = Field(max_length=50000)
    statement: dict | None
    assessment: HandoffRequest
    candidates: list[dict]
    studies: list[dict]
    researcher_reviews: list[dict]
    coverage: dict
    limitations: list[str]

    @model_validator(mode='after')
    def integrity(self):
        for key in ('id', 'title', 'licence'):
            if not isinstance(self.source.get(key), str) or not self.source[key]:
```

```
                raise ValueError('Source metadata missing: '+key)
        if len(self.candidates) > 100 or len(self.studies) > 2000:
            raise ValueError('Handoff record count exceeds limit')
        if not all(isinstance(c.get('taxon_id'), str) and isinstance(c.get('name'), str) for c in self.candidates):
            raise ValueError('Invalid candidate metadata')
        if self.statement:
            span = self.statement['support']
            start, end = span['start'], span['end']
            if not isinstance(start, int) or not isinstance(end, int) or not 0 <= start < end <= len(self.source_input) or self.source_input[start:end] !=
span['text']:
                raise ValueError('Statement/source span mismatch')
            if self.statement['id'] != self.assessment.claim_id:
                raise ValueError('Statement selection mismatch')
        elif self.assessment.claim_id:
            raise ValueError('Selected statement missing')
        ids = {c['taxon_id'] for c in self.candidates}
        if ids != set(self.assessment.candidate_ids):
            raise ValueError('Candidate assumptions mismatch')
        for study in self.studies:
            from src.api.contracts import Relation
            Relation.model_validate(study['relation'])
            paper = study['paper']
            for key in ('id', 'title', 'url', 'licence'):
                if not isinstance(paper.get(key), str):
                    raise ValueError('Invalid study metadata: '+key)
            if study['relation'].get('supporting_text') and study['relation']['supporting_text'] not in (paper.get('abstract') or ''):
                raise ValueError('Study quote is not in the retained abstract')
            for check in study['comparability']['checks']:
                if not isinstance(check.get('check'), str) or check.get('status') not in ('PASS', 'FAIL', 'PARTIAL', 'UNKNOWN'):
                    raise ValueError('Invalid automated study check')
            if study['citation'].get('status') == 'REJECTED':
                raise ValueError('Rejected citation in handoff')
            if study['taxon_id'] not in ids:
                raise ValueError('Study outside selected candidate assumptions')
            if study['identity_qualifier'] != 'CONDITIONAL_ON_CANDIDATE_IDENTITY':
                raise ValueError('Missing conditional identity scope')
            for context in study.get('historical_contexts', []):
                span = context['support']
                if self.source_input[span['start']:span['end']] != span['text']:
                    raise ValueError('Study/source context mismatch')
                if self.statement and span != self.statement['support']:
                    raise ValueError('Study belongs to another source statement')
        return self


def build_handoff(result, request: HandoffRequest):
    claims = {c['id']: c for c in result['historical_claims']}
    if request.claim_id and request.claim_id not in claims:
        raise ValueError('Selected source statement is not part of this analysis')
    statement = claims.get(request.claim_id)
    groups = [g for g in result['identity_counterfactuals']
              if not statement or set(g['mention_ids']) & set(statement['mention_ids'])]
    candidates = {b['taxon_id']: {'taxon_id': b['taxon_id'], 'name': b['name']}
                  for g in groups for b in g['branches']}
    if not set(request.candidate_ids) <= candidates.keys():
        raise ValueError('Candidate does not belong to this source statement')
    selected = [candidates[c] for c in request.candidate_ids]
    studies = []
```

```
    for record in result['evidence']:
        if record['taxon_id'] not in request.candidate_ids:
            continue
        contexts = [c for c in record.get('historical_contexts', [])
                    if not statement or c['support'] == statement['support']]
        if statement and not contexts:
            continue
        # Keep the original machine checks distinct from the question-specific judgement.
        studies.append({**{k: record[k] for k in ('id', 'taxon_id', 'condition_id', 'identity_qualifier', 'paper', 'study', 'relation', 'comparability',
    'citation')}, 'historical_contexts': contexts})
    targets = {s['id'] for s in studies}
    reviews = [r for r in result.get('researcher_reviews', [])
               if (r['target_kind'] == 'evidence' and r['target_id'] in targets)
               or (r['target_kind'] == 'identity' and any(r['target_id'] == g['key']+'::'+c for g in groups for c in request.candidate_ids))]
    packet = HandoffPacket(
        created_at=now(), analysis_id=result['run_id'], snapshot_sha256=result['snapshot_sha256'],
        code_fingerprint=result['code_fingerprint'], source=result['source'], source_input=result['input'],
        statement=statement, assessment=request, candidates=selected, studies=studies,
        researcher_reviews=reviews,
        coverage={'scope': 'SELECTED_SOURCE_STATEMENT' if statement else 'WHOLE_SOURCE_READING',
                  'studies_in_handoff': len(studies), 'publications': len({s['paper']['id'] for s in studies}),
                  'abstracts_available': sum(bool(s['paper'].get('abstract')) for s in studies),
                  'literature_access': result.get('literature_access', {}),
                  'candidate_selection': 'RESEARCHER_ASSUMPTIONS_NOT_IDENTIFICATIONS'},
        limitations=list(dict.fromkeys(result['limitations'] + [
            'This is a research reading decision, not a treatment recommendation or a systematic review.',
            'The recorded question does not change automated retrieval or establish that a study answers it.',
            'No retrieved study is not evidence of no effect. Missing abstracts and unselected candidates remain outside this handoff.',
            'Check publisher and source rights before sharing. User-provided source rights are unverified.',
        ])))
    return packet.model_dump()


def canonical(packet):
    return json.dumps(packet, ensure_ascii=False, sort_keys=True, separators=(',', ':')).encode('utf-8')


class HandoffStore:
    def __init__(self, path: Path):
        self.path = path
        with self.connect() as db:
            db.execute('CREATE TABLE IF NOT EXISTS handoffs(id TEXT PRIMARY KEY, saved_at TEXT NOT NULL, origin TEXT NOT NULL, sha256 TEXT NOT NULL, payload TEXT NOT
     NULL)')

    def connect(self):
        from contextlib import contextmanager
        @contextmanager
        def opened():
            db = sqlite3.connect(self.path, timeout=20)
            try:
                with db:
                    yield db
            finally:
                db.close()
        return opened()

    def put(self, packet, origin='LOCAL_ANALYSIS'):
        packet = HandoffPacket.model_validate(packet).model_dump()
```

```
        identifier = uuid.uuid4().hex
        saved_at = now()
        sha = hashlib.sha256(canonical(packet)).hexdigest()
        with self.connect() as db:
            db.execute('INSERT INTO handoffs VALUES(?,?,?,?,?)', (identifier, saved_at, origin, sha, canonical(packet).decode('utf-8')))
        return self.get(identifier)

    def get(self, identifier):
        with self.connect() as db:
            row = db.execute('SELECT id,saved_at,origin,sha256,payload FROM handoffs WHERE id=?', (identifier,)).fetchone()
        if not row:
            return None
        return dict(zip(('id', 'saved_at', 'origin', 'sha256'), row[:4]), packet=json.loads(row[4]))

    def list(self):
        with self.connect() as db:
            rows = db.execute("SELECT id,saved_at,origin,json_extract(payload,'$.assessment.question'),json_extract(payload,'$.source.title') FROM handoffs ORDER BY saved_at DESC LIMIT 100").fetchall()
        return [dict(zip(('id', 'saved_at', 'origin', 'question', 'source_title'), r)) for r in rows]


def handoff_markdown(packet):
    a = packet['assessment']
    lines = ['# FoliaTrace evidence handoff', '', a['question'], '',
             'Research reading only; not diagnosis, treatment advice or a systematic review.', '',
             '## Researcher decision', a['decision'], a['reason'], 'Next step: '+a['next_step'],
             'Recorded by: '+a['reviewer'], '', '## Source',
             str(packet['source'].get('title')), str(packet['source'].get('url') or 'No source URL supplied'),
             'Rights: '+str(packet['source'].get('licence')), '']
    if packet['statement']:
        span = packet['statement']['support']
        lines += [span['text'], f"Source offsets: {span['start']}\u2013{span['end']} (Unicode code points)"]
    else:
        lines += ['Whole-source reading; no extracted statement selected.', packet['source_input']]
    lines += ['', '## Candidate assumptions, not established identities']
    lines += [c['name']+' \xb7 '+c['taxon_id'] for c in packet['candidates']] or ['No candidate selected; no species conclusion.']
    lines += ['', '## Studies and automatic checks']
    if not packet['studies']:
        lines += ['No matching study included. This does not establish no effect.']
    for s in packet['studies']:
        p = s['paper']
        lines += ['', '### '+p['title'], p['url'], 'Rights: '+str(p.get('licence')), 'Candidate: '+s['taxon_id'],
                  'Conditional on candidate identity. Automated relation: '+s['relation']['relation'],
                  'Automated checks: '+json.dumps(s['comparability'], ensure_ascii=False),
                  'Source comparison: '+json.dumps(s['historical_contexts'], ensure_ascii=False)]
        if s['relation'].get('supporting_text'):
            lines += ['Abstract wording: '+s['relation']['supporting_text']]
        if not p.get('abstract'):
            lines += ['Abstract unavailable; inspect the original publication.']
    lines += ['', '## Prior researcher reviews', json.dumps(packet['researcher_reviews'], ensure_ascii=False, indent=2),
              '', '## Coverage and limits', json.dumps(packet['coverage'], ensure_ascii=False, indent=2)]
    lines += packet['limitations']
    lines += ['', 'Analysis: '+packet['analysis_id'], 'Snapshot: '+packet['snapshot_sha256'],
              'Code: '+packet['code_fingerprint'], 'Recorded: '+packet['created_at']]
    return '\n\n'.join(lines)+'\n'
```

src/evidence/historical_names.py
SHA-256: 812ac644ef0bfbe2571d910edc1fb594074484b7fb34a5b996342fc8a6f511c3

```
"""Literal cross-source identity reading, separate from candidate ranking.

Only matching chapter headings (with two declared spelling variants) are joined.
An author's scientific-name wording is not transferred to another author's plant.
"""
import re
from src.nlp.extract import normalize
from src.common import digest
from src.evidence.comparability import name_pattern
from src.nlp.headings import heading_names


def heading_key(text):
    return normalize(text).replace('camomile', 'chamomile').replace('featherfew', 'feverfew')


def heading_keys(title):
    """Every spelling of every plant a heading announces."""
    return {heading_key(form) for name in heading_names(title) for form in name['forms']}


def historical_name_references(result, passages, taxa, withdrawn=()):
    taxmap = {taxon['id']: taxon for taxon in taxa}
    output = {}
    keys_by_passage = {passage['id']: heading_keys(passage['title']) for passage in passages}
    for group in result['identity_counterfactuals']:
        keys = {heading_key(group['label'])} | {heading_key(form) for form in group.get('forms', [])}
        references = []
        for passage in passages:
            if (passage['id'] == result['source'].get('id') or
                    {passage['id'], passage.get('source_id')} & set(withdrawn) or
                    not (keys_by_passage[passage['id']] & keys)):
                continue
            text = passage['title'] + '\n\n' + passage['text']
            names = []
            for branch in group['branches']:
                taxon = taxmap.get(branch['taxon_id'])
                if not taxon:
                    continue
                for alias in taxon['aliases']:
                    if alias['kind'] not in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH'):
                        continue
                    for match in re.finditer(name_pattern(alias['name']), text, re.I):
                        names.append({'text': match.group(), 'start': match.start(), 'end': match.end(),
                                      'candidate_id': taxon['id'], 'candidate_name': taxon['name'],
                                      'alias_kind': alias['kind'], 'matched_alias': alias['name']})
            unique = {}
            for name in names:
                unique[(name['start'], name['end'], name['candidate_id'])] = name
            names = sorted(unique.values(), key=lambda name: (name['start'], -name['end']))
            windows = []
            for name in names:
                boundary = text.rfind('\n\n', 0, name['start'])
                paragraph_start = boundary + 2 if boundary >= 0 else 0
                paragraph_end = text.find('\n\n', name['end'])
```

```
                if paragraph_end < 0:
                    paragraph_end = len(text)
                start, end = max(paragraph_start, name['start'] - 220), min(paragraph_end, name['end'] + 240)
                if start > paragraph_start:
                    space = text.find(' ', start, name['start'])
                    if space >= 0:
                        start = space + 1
                if end < paragraph_end:
                    space = text.rfind(' ', name['end'], end)
                    if space >= 0:
                        end = space
                if windows and start <= windows[-1][1]:
                    windows[-1][1] = max(windows[-1][1], end)
                else:
                    windows.append([start, end])
            excerpts = [{'text': text[start:end], 'start': start, 'end': end,
                         'names': [name for name in names if start <= name['start'] and name['end'] <= end]}
                        for start, end in windows]
            references.append({
                'source': {field: passage.get(field) for field in ('id', 'title', 'collection', 'date', 'url', 'licence', 'source_id')},
                'input': text, 'text_sha256': digest(text), 'excerpts': excerpts,
                'matched_heading': passage['title'], 'heading_rule': 'NORMALIZED_HEADING_NAME_AND_ORTHOGRAPHIC_FORMS',
                'status': 'SCIENTIFIC_NAME_WORDING_PRESENT' if excerpts else 'MATCHING_HEADING_NO_CANDIDATE_SCIENTIFIC_NAME_FOUND',
                'scope': 'Another source uses this heading and contains the quoted name wording. Read its contrasts and context; it does not identify the plant in
   the current source or change candidate ranks or study relations.',
            })
        output[group['key']] = references
    return output
```

src/evidence/identity_matrix.py
SHA-256: 69fb707028433a9732168914d6ce44907306eec7ca5b1dd8e1b0a6ab48817af1

```
"""Which candidate fits what the author actually wrote?

Adapts the comparison matrix of Lardos et al. (J Ethnopharmacol 2023; PMC7615571): historical
features are scored against modern descriptions (+2 match, -2 conflict, 0 absent) and the total
gives a Strong (>=5), Moderate (3-4) or Weak (<=2) rating. Every point keeps the sentence it came
from. A rating ranks hypotheses for reading; it never establishes an identity.
"""
from __future__ import annotations

import math
import re
from functools import lru_cache

from src.common import DATA, read_json
from src.nlp.extract import normalize, sentences
from src.evidence.comparability import compiled, name_pattern
from src.evidence.independent_use import independent_use

COLOURS = {'yellow': r'yellow|golden|gold-coloured', 'white': r'white|whitish', 'purple': r'purple|purplish|violet-coloured|mauve', 'blue': r'blue|bluish', 'red': r'red|reddish|crimson|scarlet', 'pink': r'pink|pinkish|rose-coloured', 'green': r'green|greenish'}
FLOWER = r'flowers?|flowered|blossoms?|petals?|florets?|corolla|blooms?'
HABITS = {'tree': r'\b(?:a|great|tall|small|large|deciduous|evergreen)\s+(?:\w+\s+)?tree\b|\btree (?:growing|reaching|grows)\b', 'shrub': r'\bshrubs?\b|\b(?:a|as a|forms a)\s+(?:\w+\s+)?bush\b|\bbushy (?:plant|shrub|habit)\b', 'climber': r'\bclimb\w*|\btwin(?:es|ing)\b|\bvines?\b|\bscrambl\w*', 'creeping': r'\bcreep\w*|\bprostrate\b|\btrailing\b|\bmat-forming\b|\bstolon\w*', 'herb': r'\bherbaceous\b|\b(?:perennial|annual|biennial) (?:plant|herb)\b|\bperennial\b|\bannual\b|\bbiennial\b'}
EXCLUSIVE_HABITS = [{'tree', 'herb'}, {'shrub', 'herb'}]
HABITATS = {'woodland': r'woods?\b|woodlands?|forests?|thickets?|copses?|groves?', 'hedgerow': r'hedges?|hedgerows?|hedge-rows?', 'grassland': r'meadows?|pastures?|grasslands?', 'wetland': r'marsh\w*|bogs?\b|fens?\b|ditch\w*|streams?|rivers?|ponds?|watery|water-side|wet places|moist places|damp places|swamps?', 'coast': r'sea-?shores?|sea-?side|coasts?|coastal|maritime|salt-?marsh\w*|by the sea', 'arable': r'corn-?fields?|arable|waste places|waste ground|road-?sides?|way-?sides?|ruderal', 'garden': r'gardens?|cultivated', 'rocky': r'walls?\b|rocks?\b|rocky|stony', 'upland': r'mountains?|hills?\b|hillsides?|alpine|upland', 'heath': r'heaths?\b|moors?\b|moorland'}
HABITAT_SENTENCE = r'\b(?:grows?|growing|found|groweth|native|occurs?|common|wild|habitat|place|inhabit\w*|abounds?|plentiful|thrives?)\b'
MONTHS = ['january', 'february', 'march', 'april', 'may', 'june', 'july', 'august', 'september', 'october', 'november', 'december']
MONTH_PATTERN = r'\b(' + '|'.join(m.capitalize() for m in MONTHS) + r'|' + '|'.join(MONTHS[i] for i in (0, 1, 2, 3, 5, 6, 7, 8, 9, 10, 11)) + r')\b'
SUBJECT_OPENING = re.compile(r'\s*(?:it|they|this (?:herb|plant)|the (?:herb|plant|flowers?|leaves|stalks?|stems?|root|roots))\b', re.I)
ODOURS = {'aromatic': r'aromatic|fragrant|sweet-?scented|sweet smell|odorous', 'foetid': r'f[oe]+tid|stinking|disagreeable (?:odou?r|smell)|unpleasant (?:odou?r|smell)'}


@lru_cache(maxsize=4096)
def scan(text):
    """Description features with the sentence each one came from."""
    found = {'flower_colour': {}, 'habit': {}, 'habitat': {}, 'flowering': {}, 'odour': {}}
    for sentence in sentences(text):
        value = sentence['text']
        for colour, pattern in COLOURS.items():
            if compiled(rf'\b(?:{pattern})\b(?:\W+\w+){{0,4}}?\W+(?:{FLOWER})\b|\b(?:{FLOWER})\b(?:\W+\w+){{0,5}}?\W+(?:{pattern})\b').search(value) and not compiled(rf'\b(?:{pattern})\s+wine\b').search(value):
                found['flower_colour'].setdefault(colour, sentence)
        for habit, pattern in HABITS.items():
            if re.search(pattern, value, re.I):
                found['habit'].setdefault(habit, sentence)
        if re.search(HABITAT_SENTENCE, value, re.I):
            for habitat, pattern in HABITATS.items():
                if compiled(rf'\b(?:{pattern})').search(value):
```

```
                    found['habitat'].setdefault(habitat, sentence)
        if re.search(rf'\b(?:{FLOWER}|flowering|time\b)', value, re.I):
            for month in re.findall(MONTH_PATTERN, value):
                found['flowering'].setdefault(month.casefold(), sentence)
        for odour, pattern in ODOURS.items():
            if re.search(pattern, value, re.I):
                found['odour'].setdefault(odour, sentence)
    return found


def quote(sentence):
    return {'text': sentence['text'][:420], 'start': sentence['start'], 'end': sentence['end']}


def compare_set(feature, old, new, exclusive=True):
    shared = sorted(set(old) & set(new))
    if shared:
        return {'feature': feature, 'status': 'MATCH', 'points': 2, 'values': shared, 'historical': quote(old[shared[0]]), 'modern': quote(new[shared[0]])}
    if old and new and exclusive:
        first, second = next(iter(old)), next(iter(new))
        return {'feature': feature, 'status': 'CONFLICT', 'points': -2, 'values': [first, second], 'historical': quote(old[first]), 'modern': quote(new[second])}
    return None


def compare_months(old, new):
    if not old or not new:
        return None
    index = {m: i for i, m in enumerate(MONTHS)}
    old_months, new_months = {index[m] for m in old}, {index[m] for m in new}
    distance = min(min(abs(a - b), 12 - abs(a - b)) for a in old_months for b in new_months)
    first_old, first_new = sorted(old)[0], sorted(new)[0]
    status, points = ('MATCH', 2) if distance == 0 else ('PARTIAL', 1) if distance == 1 else ('CONFLICT', -2)
    return {'feature': 'flowering time', 'status': status, 'points': points, 'values': [first_old, first_new], 'historical': quote(old[first_old]), 'modern':
quote(new[first_new])}


def habit_conflict(old, new):
    shared = compare_set('growth form', old, new, exclusive=False)
    if shared:
        return shared
    for pair in EXCLUSIVE_HABITS:
        left = next((h for h in pair if h in old), None)
        right = next((h for h in pair if h in new and h != left), None)
        if left and right:
            return {'feature': 'growth form', 'status': 'CONFLICT', 'points': -2, 'values': [left, right], 'historical': quote(old[left]), 'modern':
quote(new[right])}
    return None


def author_named(text, taxon):
    for alias in taxon['aliases']:
        if alias['kind'] not in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH'):
            continue
        match = compiled(name_pattern(alias['name'])).search(text)
        if match:
            start = max(0, match.start() - 110)
            return {'feature': "author's own Latin name", 'status': 'MATCH', 'points': 4, 'values': [alias['name']], 'alias_kind': alias['kind'], 'historical':
{'text': text[start:match.end() + 90], 'start': start, 'end': match.end() + 90}, 'modern': {'text': f"{alias['name']} is a {alias['kind'].replace('_', ' ').lower()}
```

```
    of {taxon['name']}"}}
    return None


def regional(taxon, region):
    count = (taxon.get('occurrence_counts') or {}).get(region)
    if count is None:
        return None
    if count >= 100:
        return {'feature': 'recorded in the source region', 'status': 'MATCH', 'points': 1, 'values': [f'{count} GBIF records in {region}'], 'modern': {'text':
    f'GBIF lists {count} occurrence records for {region}. Modern records do not establish historical presence.'}}
    if count < 10:
        return {'feature': 'recorded in the source region', 'status': 'CONFLICT', 'points': -2, 'values': [f'{count} GBIF records in {region}'], 'modern': {'text':
    f'GBIF lists only {count} occurrence records for {region}. A cultivated or imported plant can still be meant.'}}
    return None


def recorded_use(taxon, topics):
    """The passage's uses against what other communities used this plant for; silence is not conflict."""
    records = independent_use()
    typical = [t for t in sorted(topics) if records.characteristic(taxon, t)]
    if not typical:
        return None
    reach = [records.reach(taxon, t) for t in typical]
    words = ', '.join(t.replace('condition:', '').replace('_', ' ') for t in typical[:3])
    return {'feature': 'use recorded elsewhere', 'status': 'MATCH', 'points': min(len(typical), 2), 'values': [words], 'modern': {'text': f"Dr. Duke's database lists
     {sum(r['records'] for r in reach)} records for {words}, more often than for most plants.", 'url': records.citation['url']}}


def weigh_shared_wording(rated, taxmap):
    """One written binomial reaching several species: the species that holds it as its accepted
    name, or failing that keeps the genus the author wrote, is the plainer reading."""
    named = [(r, row) for r in rated for row in r['rows'] if row['feature'] == "author's own Latin name"]
    if len(named) < 2:
        return
    holders = [r['name'] for r, row in named if row.get('alias_kind') == 'ACCEPTED_NAME']
    for r, row in named:
        if row.get('alias_kind') == 'ACCEPTED_NAME':
            continue
        written = row['values'][0]
        same_genus = [x['name'] for x, _ in named if x['name'].split()[0] == written.split()[0]]
        if holders:
            note = f"The same wording is the accepted name of {holders[0]}; for this species it is a synonym under another author, a homonym."
        elif same_genus and r['name'].split()[0] != written.split()[0]:
            note = f"The source wrote the genus {written.split()[0]}; this synonym places the plant in {r['name'].split()[0]}, while {same_genus[0]} keeps it."
        else:
            continue
        # No points: a well-studied homonym must not outrank the plant the wording names.
        row['points'] = 0
        row['modern'] = {**row['modern'], 'text': row['modern']['text'] + ' ' + note}
        r['score'] = sum(x['points'] for x in r['rows'])
        r['rating'] = rate(r['score'], r['rows'])


@lru_cache(maxsize=1)
def encyclopedia_names():
    path = DATA / 'wikipedia_redirects.json'
    found = {}
```

```
    for row in read_json(path)['names'] if path.exists() else []:
        if row['kind'] == 'VERNACULAR':
            found.setdefault(normalize(row['name']), {})[row['id']] = row
    return found


def recorded_name(taxon, wording):
    """Whether people who name plants for a living give this plant this name.

    A GBIF vernacular list holds every usage anywhere; "eyebright" reaches a Plumbago. An
    encyclopedia redirect and ethnobotanists' field records say which plant the name usually means.
    """
    if not wording:
        return None
    keys = dict.fromkeys(normalize(form) for form in (wording, wording.replace('-', ''), wording.replace('-', ' ')))
    points, notes, values = 0, [], []
    for key in keys:
        article = encyclopedia_names().get(key, {}).get(taxon['id'])
        if article and not values:
            points += 1
            values.append(wording)
            notes.append(f"English Wikipedia sends \"{article['name']}\" to the article on this species ({article['article']}).")
        holders = independent_use().names.get(key, {})
        # Duke may file the plant under a synonym ("Stachys officinalis" for Betonica officinalis).
        synonyms = {normalize(a['name']) for a in taxon['aliases'] if a['kind'] in ('ACCEPTED_NAME', 'SYNONYM')}
        mine = holders.get(taxon['id']) or next((h for h in holders.values() if normalize(h['name']) in synonyms), None)
        # Records from one country say how that country speaks: "ginseng" in Duke is American ginseng.
        if mine and mine['records'] >= 2 and len(mine['places']) >= 2 and mine['records'] == max(h['records'] for h in holders.values()) and not any('Duke' in n for
    n in notes):
            points += 1
            notes.append(f"Dr. Duke's ethnobotanical records use \"{mine['printed']}\" {mine['records']} times for this species, more than for any other.")
    if not points:
        return None
    return {'feature': 'name recorded for this plant', 'status': 'MATCH', 'points': points, 'values': [wording], 'modern': {'text': ' '.join(notes), 'url':
    'https://doi.org/10.15482/USDA.ADC/1239279' if any('Duke' in n for n in notes) else None}}


def rate(score, rows):
    if not rows:
        return 'NOT_ASSESSABLE'
    return 'STRONG' if score >= 5 else 'MODERATE' if score >= 3 else 'WEAK'


def about_this_plant(row, name):
    """Herbals wander: a conflict only counts when the sentence describes the plant in question."""
    if row['status'] != 'CONFLICT' or 'historical' not in row or not name:
        return True
    quoted = row['historical']['text']
    return bool(re.search(rf'\b{re.escape(name)}', quoted, re.I) or SUBJECT_OPENING.match(quoted))


def reading_order(row, prominence):
    """A grounded source name outranks priors; ambiguous readings keep the existing model."""
    points = lambda feature:sum(signal['points'] for signal in row['rows'] if signal['feature'] == feature)
    grounded = bool(row.get('source_name_grounded')) and points("author's own Latin name") > 0
    return (grounded, bool(row.get('source_specific_name')), row['score'] + math.log10(1 + (prominence or {}).get(row['taxon_id'], 0)), points('name recorded for
    this plant'))

```

```

def descriptive_subjects(text, candidates, taxmap, wording):
    """A qualified name can identify the subject of a descriptive sentence.

    A chapter may describe both basil and wild basil. A colour belonging to a
    candidate-specific name is not a colour conflict for every other candidate.
    Shared names remain ambiguous; this does not resolve a historical identity.
    """
    inspected = normalize(wording or '')
    if not inspected:
        return []
    aliases = {}
    for candidate in candidates:
        for alias in (taxmap.get(candidate['taxon_id']) or {}).get('aliases', []):
            name = normalize(alias['name'])
            if alias['kind'] in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH', 'VERNACULAR', 'HISTORICAL_LEXICON') and len(name) > len(inspected):
                aliases.setdefault(name, set()).add(candidate['taxon_id'])
    patterns = [(compiled(name_pattern(name)), ids) for name, ids in aliases.items()]
    scoped = []
    for sentence in sentences(text):
        # Look at the grammatical subject, before its descriptive verb, rather
        # than a different plant introduced later in the sentence.
        subject = re.split(r'\b(?:is|are|was|were|has|have|grows?|bears?|flowers?|belongs?|furnishes?)\b', sentence['text'], maxsplit=1, flags=re.I)[0][:120]
        holders = set().union(*(ids for pattern, ids in patterns if pattern.search(subject)))
        if holders:
            scoped.append((sentence['start'], sentence['end'], holders))
    return scoped


def identity_matrix(text, candidates, taxmap, descriptions, *, region=None, prominence=None, name=None, topics=(), wording=None, english_name=False):
    """Score every candidate of one name against the source text it occurs in."""
    historical = scan(text)
    subjects = descriptive_subjects(text, candidates, taxmap, wording)
    rated = []
    # "The Wild English thyme (Thymus serpyllum) belongs to\u2026": once the entry opens by naming its
    # plant, a Latin name further on ("the garden Thyme, Thymus vulgaris") is another plant.
    defined = any('NAMED_IN_SOURCE_FIRST_SENTENCE' in c.get('reasons', []) for c in candidates)
    for candidate in candidates:
        taxon = taxmap.get(candidate['taxon_id'])
        if not taxon:
            continue
        description = (descriptions or {}).get(candidate['taxon_id'])
        modern = scan(description['text']) if description else None
        excluded = [(start, end) for start, end, holders in subjects if candidate['taxon_id'] not in holders]
        if excluded:
            masked = list(text)
            for start, end in excluded:
                masked[start:end] = ['\n' if ch == '\n' else ' ' for ch in text[start:end]]
            historical = scan(''.join(masked))
        else:
            historical = scan(text)
        rows = []
        named = author_named(text, taxon)
        if named and defined and 'NAMED_IN_SOURCE_FIRST_SENTENCE' not in candidate.get('reasons', []):
            named = {**named, 'points': 0, 'modern': {**named['modern'], 'text': named['modern']['text'] + '. The entry opens by naming another plant; this Latin
    name occurs later, for a plant the author sets beside it.'}}
        if named:
            rows.append(named)
```

```
        specific = candidate.get('source_common_name')
        if specific:
            anchored = specific.get('anchor', True)
            rows.append({'feature':"author's specific common name",'status':'MATCH' if anchored else 'CONTEXT_ONLY','points':2 if anchored else 0,'values':[specific['lookup']],
                         'historical':{'text':specific['name'],'start':specific['start'],'end':specific['end']},
                         'modern':{'text':f"The opening botanical subject uses {specific['name']!r}; {specific['lookup']!r} matches this candidate in the existing name snapshot. " + ("This source-local naming clue does not establish historical identity." if anchored else "The common name spans genera and does not agree with this heading's existing candidates. Retained as an alternative without ranking priority or bonus.")}})
        if modern:
            rows += [row for row in (compare_set('flower colour', historical['flower_colour'], modern['flower_colour']), habit_conflict(historical['habit'], modern['habit']), compare_set('habitat', historical['habitat'], modern['habitat'], exclusive=False), compare_months(historical['flowering'], modern['flowering']), compare_set('smell', historical['odour'], modern['odour'], exclusive=False)) if row]
        rows = [row for row in rows if about_this_plant(row, name)]
        place = regional(taxon, region) if region else None
        # English is not a source location. Only an explicitly supplied region
        # (including a known collection's region) contributes distribution evidence.
        if place:
            rows.append(place)
        used = recorded_use(taxon, topics)
        if used:
            rows.append(used)
        called = recorded_name(taxon, wording)
        if called:
            rows.append(called)
        score = sum(row['points'] for row in rows)
        grounded = bool(named) and ('NAMED_IN_SOURCE_FIRST_SENTENCE' in candidate.get('reasons', []) or any(a['kind'] in ('ACCEPTED_NAME','SYNONYM','INPUT_SYNONYM_MATCH') and normalize(a['name']) == normalize(wording or '') for a in taxon['aliases']))
        rated.append({'taxon_id': candidate['taxon_id'], 'name': candidate['name'], 'score': score, 'rating': rate(score, rows), 'rows': rows, 'source_name_grounded':grounded, 'source_specific_name':bool(specific and specific.get('anchor', True)), 'description_source': {k: description[k] for k in ('url', 'licence', 'title')} if description else None, 'scale': 'Strong >=5, Moderate 3-4, Weak <=2. Ranks hypotheses for reading; not an identification.'})
    weigh_shared_wording(rated, taxmap)
    rated.sort(key=lambda r: reading_order(r, prominence), reverse=True)
    best = rated[0]['score'] if rated else None
    separated = len(rated) > 1 and best - max(r['score'] for r in rated[1:]) >= 2
    return {'candidates': rated, 'leading': rated[0]['taxon_id'] if rated and (separated or len(rated) == 1) and best > 0 else None, 'separation': 'CLEAR' if separated else 'NOT_SEPARATED', 'method': 'DESCRIPTION_MATRIX_ADAPTED_FROM_LARDOS_2023', 'reading_order':"An explicit scientific identification of the inspected name takes priority, followed by the opening subject's specific common-name clues. A relative or later co-mention is not a definition. Other candidates retain the feature-total and publication prior. Source-local clues do not add dictionary aliases or establish historical identity. English alone does not establish a region. Rankings are conditional, not identity probabilities."}
```

src/evidence/independent_use.py
SHA-256: 403b2bc396f578a9d6ca9e297a68c279650487f5091910622a03a01d5a3826ea

```
"""Independent ethnobotanical records from Dr. Duke's database (USDA, CC0).

A use reported in several unrelated places is a stronger research lead than the same use repeated
by one book: herbals copy each other, distant communities mostly do not. A record only says that a
use was reported, so nothing here can raise or lower a verdict about whether a plant works.
"""
from __future__ import annotations

import re
from collections import Counter, defaultdict

from src.common import DATA, read_json
from src.nlp.extract import normalize

PATH = DATA / 'duke_ethnobotany.json'
CHEMISTRY = DATA / 'duke_chemistry.json'
# Duke's activity vocabulary, spelt as he spelt it, under the topics the statement reader uses.
TOPIC_ACTIVITIES = {
    'headache': ['Ache(Head)', 'Headache', 'Cephalalgia'],
    'migraine': ['Migraine', 'Hemicrania'],
    'sleep': ['Insomnia', 'Soporific', 'Hypnotic', 'Sedative'],
    'anxiety': ['Sedative', 'Tranquilizer', 'Anxiety'],
    'depression': ['Depression', 'Melancholy', 'Antidepressant'],
    'digestive_symptoms': ['Stomachic', 'Carminative', 'Ache(Stomach)', 'Dyspepsia', 'Colic', 'Digestive', 'Stomach', 'Gastritis', 'Abdomen', 'Intestine',
  'Flatulence', 'Gastralgia', 'Gastroenteritis', 'Enteritis'],
    'diarrhoea': ['Diarrhea', 'Dysentery', 'Cholera', 'Enteritis'],
    'constipation': ['Purgative', 'Laxative', 'Aperient', 'Cathartic', 'Constipation'],
    'nausea_vomiting': ['Nausea', 'Vomiting', 'Antiemetic', 'Seasickness', 'Hyperemesis'],
    'pain': ['Anodyne', 'Analgesic', 'Neuralgia', 'Bruise', 'Pain', 'Ache(Back)'],
    'joint_gout': ['Rheumatism', 'Arthritis', 'Gout', 'Lumbago', 'Sciatica', 'Ache(Back)'],
    'inflammation': ['Inflammation', 'Swelling', 'Tumor', 'Antiphlogistic', 'Resolvent'],
    'fever': ['Fever', 'Malaria', 'Sudorific', 'Diaphoretic', 'Febrifuge', 'Ague'],
    'cough': ['Cough', 'Expectorant', 'Pertussis', 'Antitussive', 'Hoarseness'],
    'respiratory': ['Cold', 'Asthma', 'Bronchitis', 'Catarrh', 'Flu', 'Tuberculosis', 'Pectoral', 'Chest', 'Lung', 'Pneumonia', 'Phthisis', 'Grippe'],
    'wounds': ['Wound', 'Sore', 'Vulnerary', 'Ulcer', 'Trauma'],
    'burns': ['Burn', 'Scald'],
    'skin': ['Skin', 'Itch', 'Dermatosis', 'Scabies', 'Ringworm', 'Eczema', 'Erysipelas', 'Leprosy', 'Eruption', 'Pimple', 'Wart', 'Boil', 'Abscess', 'Carbuncle',
  'Psoriasis', 'Acne'],
    'bleeding': ['Hemostat', 'Styptic', 'Hemorrhage', 'Hemoptysis', 'Epistaxis', 'Menorrhagia', 'Hematuria'],
    'oral_throat_care': ['Sore(Throat)', 'Throat', 'Stomatitis', 'Gargle', 'Dentifrice', 'Gingivitis', 'Mouthwash', 'Thrush', 'Tonsilitis', 'Pharyngitis',
  'Pyorrhea'],
    'toothache': ['Ache(Tooth)', 'Odontalgia'],
    'eye': ['Eye', 'Ophthalmia', 'Conjunctivitis', 'Collyrium', 'Cataract', 'Blepharitis'],
    'ear': ['Ache(Ear)', 'Otitis', 'Deafness', 'Ear', 'Otalgia'],
    'epilepsy': ['Epilepsy', 'Convulsion'],
    'nervous': ['Spasm', 'Antispasmodic', 'Nervine', 'Nerves', 'Hysteria', 'Paralysis', 'Apoplexy', 'Cramp'],
    'liver_jaundice': ['Jaundice', 'Hepatitis', 'Liver', 'Bilious', 'Cholagogue', 'Gallstone'],
    'urinary': ['Diuretic', 'Dysuria', 'Bladder', 'Cystitis', 'Urogenital', 'Strangury', 'Urethritis'],
    'dropsy_oedema': ['Dropsy', 'Edema'],
    'memory_cognition': ['Memory', 'Amnesia', 'Senility'],
    'kidney_stones': ['Kidney', 'Gravel', 'Calculus', 'Lithontriptic', 'Stone'],
    'menstrual': ['Emmenagogue', 'Dysmenorrhea', 'Amenorrhea', 'Menorrhagia', 'Menopause'],
    'childbirth_lactation': ['Parturition', 'Puerperium', 'Lactogogue', 'Galactagogue'],
    'parasites': ['Vermifuge', 'Anthelminthic', 'Anthelmintic', 'Parasiticide', 'Taenicide', 'Taeniafuge'],
    'heart': ['Cardiotonic', 'Heart', 'Palpitation', 'Angina'],
```

```
    'poison_bites': ['Bite(Snake)', 'Bite(Bug)', 'Bite(Dog)', 'Bite(Scorpion)', 'Antidote', 'Alexiteric', 'Sting'],
    'scurvy': ['Scurvy', 'Antiscorbutic'],
    'haemorrhoids': ['Piles', 'Hemorrhoid', 'Hemorrhoids'],
    'plague_infection': ['Antiseptic', 'Bactericide', 'Plague', 'Infection'],
    'cancer': ['Cancer', 'Tumor'],
    'diabetes': ['Diabetes'],
    'venereal': ['Gonorrhea', 'Syphilis', 'Venereal', 'Leucorrhea', 'Clap'],
    'hypertension': ['Hypertension'],
    'anaemia': ['Anemia', 'Chlorosis'],
}
# Duke's chemical activities under the same topics, kept apart: a report that people used a plant
# for fever and an assay finding a chemical antipyretic are different kinds of knowledge.
COMPOUND_ACTIVITIES = {
    'headache': ['Antimigraine'],
    'migraine': ['Antimigraine', 'Serotoninergic'],
    'sleep': ['Sedative', 'Hypnotic', 'CNS-Depressant', 'Tranquilizer'],
    'anxiety': ['Anxiolytic', 'Tranquilizer', 'Sedative', 'GABA-ergic', 'GABA-Aminotransferase-Inhibitor'],
    'depression': ['Antidepressant', 'MAO-Inhibitor', 'Serotonin-Uptake-Inhibitor'],
    'digestive_symptoms': ['Carminative', 'Antiulcer', 'Spasmolytic', 'Digestive', 'Antigastric', 'Antigastritic', 'Stomachic', 'Antiflatulent'],
    'diarrhoea': ['Antidiarrheic', 'Antidysenteric', 'Amebicide', 'Antiamebic'],
    'constipation': ['Laxative', 'Purgative', 'Cathartic'],
    'nausea_vomiting': ['Antiemetic', 'Antinauseant'],
    'pain': ['Analgesic', 'Antinociceptive', 'Anesthetic', 'Antiallodynic', 'Antineuralgic'],
    'joint_gout': ['Antiarthritic', 'Antirheumatic', 'Antigout', 'Xanthine-Oxidase-Inhibitor', 'Antirheumatalgic', 'Antilumbago'],
    'inflammation': ['Antiinflammatory', 'Antiedemic', 'COX-2-Inhibitor', 'Cyclooxygenase-Inhibitor', 'Lipoxygenase-Inhibitor', 'Antiprostaglandin'],
    'fever': ['Antipyretic', 'Antimalarial', 'Antiplasmodial', 'Febrifuge'],
    'cough': ['Antitussive', 'Expectorant', 'Antipertussal'],
    'respiratory': ['Bronchodilator', 'Antiasthmatic', 'Antibronchitic', 'Antitubercular', 'Antiinfluenza', 'Decongestant', 'Mucolytic', 'Antihistaminic'],
    'wounds': ['Vulnerary', 'Cicatrizant', 'Antiulcer', 'Woundhealer'],
    'burns': ['Antiburn', 'Vulnerary'],
    'skin': ['Antieczemic', 'Antidermatitic', 'Antipsoriac', 'Antiacne', 'Antiseborrheic', 'Antierythemic'],
    'bleeding': ['Hemostat', 'Antihemorrhagic', 'Styptic'],
    'oral_throat_care': ['Antigingivitic', 'Antiplaque', 'Anticariogenic', 'Antipyorrheic', 'Antitonsilitic', 'Antistomatitic'],
    'toothache': ['Antiodontalgic', 'Dentifrice'],
    'eye': ['Antiglaucomic', 'Anticataract', 'Antimyopic', 'Antiophthalmic'],
    'epilepsy': ['Anticonvulsant', 'Antiepileptic'],
    'nervous': ['Antispasmodic', 'Myorelaxant', 'Neuroprotective', 'Antistroke', 'Anticonvulsant'],
    'liver_jaundice': ['Hepatoprotective', 'Cholagogue', 'Choleretic', 'Antihepatotoxic', 'Antihepatitic', 'Hepatotonic', 'Antijaundice', 'Antihepatitic'],
    'urinary': ['Diuretic', 'Antiprostatic', 'Antiprostatitic', 'Antiprostatadenomic', 'Uricosuric', 'Antinephritic', 'Anticystitic'],
    'dropsy_oedema': ['Diuretic', 'Antiedemic', 'Antidropsy'],
    'memory_cognition': ['Anticholinesterase', 'Memory-Enhancer', 'Cognitive', 'Antialzheimeran', 'Nootropic', 'Cholinergic'],
    'kidney_stones': ['Antilithic', 'Lithontriptic', 'Antinephrolytic'],
    'menstrual': ['Emmenagogue', 'Antidysmenorrheic', 'Antimenorrhagic', 'Estrogenic', 'Antimenopausal', 'Antiamenorrheic'],
    'childbirth_lactation': ['Lactagogue', 'Galactagogue', 'Oxytocic', 'Uterotonic'],
    'parasites': ['Anthelmintic', 'Nematicide', 'Vermifuge', 'Antiparasitic', 'Antiascariasis', 'Antischistosomal', 'Taenicide'],
    'heart': ['Cardiotonic', 'Antiarrhythmic', 'Cardioprotective', 'Antianginal', 'Hypocholesterolemic', 'Antiatherosclerotic', 'Antiischemic'],
    'poison_bites': ['Antiophidic', 'Antivenin', 'Antidote', 'Antivenomous'],
    'scurvy': ['Antiscorbutic'],
    'haemorrhoids': ['Antihemorrhoidal', 'Capillarifortificant', 'Vasoprotective'],
    'plague_infection': ['Antibacterial', 'Antiviral', 'Antiseptic', 'Antimicrobial'],
    'cancer': ['Antitumor', 'Cancer-Preventive', 'Antileukemic', 'Anticancer', 'Anticarcinogenic'],
    'diabetes': ['Hypoglycemic', 'Antidiabetic', 'Aldose-Reductase-Inhibitor', 'Insulinotonic', 'Insulin-Sparing'],
    'venereal': ['Antisyphilitic', 'Antigonorrheic', 'Antileukorrheic'],
    'hypertension': ['Hypotensive', 'ACE-Inhibitor', 'Antihypertensive', 'Calcium-Antagonist', 'Vasodilator'],
    'anaemia': ['Antianemic', 'Hematinic', 'Hematopoietic'],
}
ACTIVITY_TOPICS = defaultdict(list)
for topic, activities in TOPIC_ACTIVITIES.items():
```

```
    for activity in activities:
        ACTIVITY_TOPICS[activity].append(topic)
# Common words of English and of the reader's own vocabulary are never read as a local name.
NAME = re.compile(r"^[^\W\d_][^\W\d_' -]{3,}(?:[ '-][^\W\d_]+){0,3}$")


def topics_of(activity):
    """Cancer(Breast)" files under cancer; "Ache(Head)" is listed whole because the root says nothing."""
    return ACTIVITY_TOPICS.get(activity) or ACTIVITY_TOPICS.get(activity.split('(')[0], [])


def place_group(place):
    """Mexico(Seri)" and "Mexico" are one country; the bracket still names who reported it."""
    return place.split('(')[0].strip()


class IndependentUse:
    def __init__(self, path=PATH):
        self.available = path.exists()
        self.by_taxon, self.by_name, self.names = {}, {}, defaultdict(dict)
        self.topic_records, self.total_records = Counter(), 0
        self.genera, self.chemistry = defaultdict(list), None
        self.citation = None
        if not self.available:
            return
        body = read_json(path)
        self.citation = {'doi': body['doi'], 'url': f"https://doi.org/{body['doi']}", 'licence': body['licence']}
        for taxon in body['taxa']:
            topics = defaultdict(list)
            for record in taxon['records']:
                for topic in topics_of(record[0]):
                    topics[topic].append(record)
                name = normalize(record[2])
                if NAME.fullmatch(record[2]):
                    self.names[name].setdefault(taxon['id'], {'id': taxon['id'], 'name': taxon['name'], 'family': taxon['family'], 'records': 0, 'places': set(),
    'printed': record[2], 'reference': record[3]})
                    entry = self.names[name][taxon['id']]
                    entry['records'] += 1
                    entry['places'] |= {record[1]} if record[1] else set()
            self.topic_records.update({topic: len(rows) for topic, rows in topics.items()})
            self.total_records += len(taxon['records'])
            row = {'id': taxon['id'], 'name': taxon['name'], 'records': len(taxon['records']), 'topics': dict(topics), 'poison_records':
    sum(r[0].startswith('Poison') for r in taxon['records'])}
            self.by_taxon[taxon['id']] = row
            self.by_name[taxon['name']] = row
            self.genera[taxon['name'].split()[0]].append(row)

    def plant(self, taxon):
        return self.by_taxon.get(taxon['id']) or self.by_name.get(taxon['name']) if self.available else None

    def reach(self, taxon, condition_id):
        """Records of one plant under one topic. None means the database says nothing either way."""
        row = self.plant(taxon)
        records = (row or {}).get('topics', {}).get(condition_id.replace('condition:', ''))
        if not records:
            return None
        places = Counter(place_group(r[1]) for r in records if r[1])
        return {'records': len(records), 'places': len(places), 'place_names': [p for p, _ in places.most_common(8)], 'activities': [a for a, _ in Counter(r[0] for r
```

```
     in records).most_common(4)], 'references': [r for r, _ in Counter(r[3] for r in records if r[3]).most_common(3)], 'source': self.citation}
167
168     def relatives(self, taxon, condition_id):
169         """Sister species used for the same thing: unrelated peoples converging on one lineage
170         predicts bioactivity better than any single report (Saslis-Lagoudakis et al., PNAS 2012)."""
171         topic = condition_id.replace('condition:', '')
172         name = taxon.get('name', '')
173         kin = [r for r in self.genera.get(name.split()[0] if name else '', ()) if r['name'] != name and r['topics'].get(topic)]
174         if not kin:
175             return None
176         places = {place_group(record[1]) for row in kin for record in row['topics'][topic] if record[1]}
177         return {'species': len(kin), 'places': len(places), 'names': [r['name'] for r in sorted(kin, key=lambda r: -len(r['topics'][topic]))[:3]]}
178
179     def constituents(self, taxon, condition_id):
180         """Distinctive chemicals with a recorded activity on this topic, in any assay and dose."""
181         if self.chemistry is None:
182             self.chemistry = {row['id']: row['topics'] for row in read_json(CHEMISTRY)['taxa']} if CHEMISTRY.exists() else {}
183         found = self.chemistry.get(taxon.get('id'), {}).get(condition_id.replace('condition:', ''))
184         if not found:
185             return None
186         return {'count': found['count'], 'rarest': [{'chemical': c.replace('-', ' ').capitalize(), 'activity': a} for c, a in found['rarest']], 'source':
    self.citation}
187
188     def recorded_poisonous(self, taxon):
189         """Three reports of poisoning are enough to put a safety warning before any research lead."""
190         return (self.plant(taxon) or {}).get('poison_records', 0) >= 3
191
192     def characteristic(self, taxon, condition_id):
193         """Is this use typical of the plant? A much-recorded plant has a record of nearly everything."""
194         row = self.plant(taxon)
195         topic = condition_id.replace('condition:', '')
196         records = len((row or {}).get('topics', {}).get(topic, ()))
197         if records < 2 or not self.topic_records[topic]:
198             return False
199         return records / row['records'] >= 1.5 * self.topic_records[topic] / self.total_records
200
201
202 _shared = None
203
204
205 def independent_use():
206     global _shared
207     if _shared is None:
208         _shared = IndependentUse()
209     return _shared
```

src/evidence/integrity.py
SHA-256: 915f5dbbbf3bd455a444edd680c62f2cd282381235548a005e779add2d57f5f0

```python
"""Stable JSON numbers for newly generated records crossing Python and browser JSON."""
import math

MAX_SAFE_INTEGER=2**53-1


def portable_numbers(value):
    if value is None or isinstance(value,(str,bool)):
        return value
    if isinstance(value,int):
        if abs(value)>MAX_SAFE_INTEGER:raise ValueError('A source number exceeds the exact browser JSON integer range; record its original wording manually')
        return value
    if isinstance(value,float):
        if not math.isfinite(value):raise ValueError('Non-finite source numbers cannot be exported as portable JSON')
        if value.is_integer():
            return portable_numbers(int(value))
        return value
    if isinstance(value,(list,tuple)):return [portable_numbers(v) for v in value]
    if isinstance(value,dict):return {k:portable_numbers(v) for k,v in value.items()}
    raise ValueError('Unsupported portable JSON value')
```

src/evidence/map_support.py
SHA-256: 55929ca741b29a6734af4a848eca1d9ab03df9ff99fe425ce42e51b2d52a4685

```
"""Statement-bound support for the chapter-plant research map.

The map follows the same mention links as the workspace. A chapter title alone
never lends another plant's sentence to every plant listed in that title.
"""
LEAD_WEIGHT = {'PRECLINICAL_ONLY': 3, 'SPECIES_NAMED_NO_TESTED_RESULT': 1, 'NO_STUDY_RETRIEVED': 1}


def chapter_subjects(result):
    mentions = {m['id']: m for m in result['mentions']}
    return {r['mention_id']: r for r in result['resolutions'] if mentions.get(r['mention_id'], {}).get('method') == 'SOURCE_HEADING_SUBJECT' and r['candidates']}


def statement_subjects(claim, result):
    """Primary heading candidates actually reached by this statement's links.

    Primary means the map's stated identity assumption, not an identification.
    Uncertain links remain inspectable but cannot inflate the lead score.
    """
    heading = chapter_subjects(result)
    heading_ids = {r['candidates'][0]['taxon_id'] for r in heading.values()}
    resolutions = {r['mention_id']: r for r in result['resolutions']}
    linked = {identifier: resolutions[identifier] for identifier in claim['mention_ids'] if resolutions.get(identifier, {}).get('candidates')}
    subject_ids = {r['candidates'][0]['taxon_id'] for r in linked.values()}
    direct = claim['relation_scope'] == 'SENTENCE' and len(subject_ids) == 1 and not set(linked) & set(heading)
    single_heading = claim['relation_scope'] == 'SOURCE_SECTION_HEADING' and len({r['candidates'][0]['taxon_id'] for r in heading.values()}) == 1
    attribution = 'EXPLICIT_SINGLE_PLANT' if direct else 'SINGLE_HEADING_SUBJECT' if single_heading else 'UNCERTAIN_SOURCE_LINK'
    return {identifier: {'attribution': attribution, 'eligible_for_priority': direct or single_heading, 'mention_ids': [mid for mid, r in linked.items() if
   r['candidates'][0]['taxon_id'] == identifier], 'identity_candidates': list({c['taxon_id']: {'taxon_id': c['taxon_id'], 'name': c['name']} for r in linked.values() if
    r['candidates'][0]['taxon_id'] == identifier for c in r['candidates']}.values())} for identifier in subject_ids & heading_ids}


def priority(row, state, outside):
    """Expose each actual contribution; uncertain historical links contribute zero."""
    records = row['direct_statements']
    books = len(row['priority_books'])
    components = {'source_statements': records, 'additional_books': 2 * max(0, books - 1), 'independent_places': min((outside.get('independent_use') or
   {}).get('places', 0), 12), 'related_species': min((outside.get('relatives') or {}).get('species', 0), 3), 'research_state_weight': LEAD_WEIGHT.get(state, 0),
   'poison_divisor': 2 if outside.get('recorded_poisonous') else 1}
    base = sum(components[k] for k in ('source_statements', 'additional_books', 'independent_places', 'related_species'))
    score = components['research_state_weight'] * base // components['poison_divisor']
    if not records or state == 'NO_STUDY_RETRIEVED' and records < 3:
        score = 0
    return score, components
```

src/evidence/model.py
SHA-256: af172dcb1c059ecf1e5877777456dfacb91df6c29c3ed15ab273ac114c1b603f

```
from __future__ import annotations

from collections import Counter
import joblib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from src.common import ROOT
from .classify import META_TYPES


def metadata_label(paper):
    types = paper.get("publication_types", [])
    for key in ["Meta-Analysis", "Systematic Review", "Randomized Controlled Trial", "Clinical Trial", "Clinical Trial, Phase III", "Clinical Trial, Phase II", "Review"]:
        if key in types:
            return META_TYPES[key]
    return None


class StudyModel:
    def __init__(self, model=None):
        self.model = model

    def fit(self, papers):
        rows = [(p["title"] + " " + p["abstract"], metadata_label(p)) for p in papers if metadata_label(p)]
        counts = Counter(y for _, y in rows)
        allowed = {k for k, n in counts.items() if n >= 3}
        rows = [(x, y) for x, y in rows if y in allowed]
        if len(allowed) < 2:
            return {"trained": False, "reason": "Fewer than two adequately supported metadata-labelled classes", "counts": dict(counts)}
        self.model = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=1, max_features=15000, sublinear_tf=True), LogisticRegression(max_iter=1500, class_weight="balanced", random_state=20261102))
        self.model.fit([x for x, y in rows], [y for x, y in rows])
        return {"trained": True, "n": len(rows), "counts": dict(Counter(y for _, y in rows)), "label_tier": "SILVER_PUBLICATION_METADATA", "model": "TFIDF_LOGISTIC_REGRESSION", "seed": 20261102}

    def predict(self, text):
        if self.model is None:
            return {"label": "NOT_TRAINED", "used": False}
        values = self.model.predict_proba([text])[0]
        idx = int(values.argmax())
        # No probabilities displayed or claimed calibrated. This threshold is a
        # disclosed abstention heuristic, not an empirical confidence guarantee.
        return {"label": str(self.model.classes_[idx]), "used": bool(values[idx] >= .6), "selection_rule": "internal logistic score >=0.6; uncalibrated; no probability display", "version": "tfidf-logreg-0.2"}

    def save(self, path):
        path.parent.mkdir(parents=True, exist_ok=True)
        temporary = path.with_suffix('.tmp')
        joblib.dump(self.model, temporary)
        temporary.replace(path)

    @classmethod
    def load(cls, path):
```

```
        # Only load application-created local model, never uploaded pickle files.
        return cls(joblib.load(path)) if path.exists() else cls()
```

src/evidence/outcome_model.py
SHA-256: b02d6d0858af566b385baa6bee315f4ad8d8abb14757073503f1e9f574c3d081

```
"""Local supervised outcome-direction proposals, always conditional on a PICO question."""
from pathlib import Path
import joblib
from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression


LABELS = {-1: 'DECREASED', 0: 'NO_SIGNIFICANT_DIFFERENCE', 1: 'INCREASED'}


def model_input(evidence, outcome, intervention='', comparator=''):
    # Explicit fields keep the task relative to the requested outcome and comparison.
    # Word/character models can learn direction language without a runtime LLM.
    return f"OUTCOME: {outcome}\nINTERVENTION: {intervention}\nCOMPARATOR: {comparator}\nEVIDENCE: {evidence}"


def make_model(c=1.0, *, characters=True):
    features = [('words', TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_features=35000, sublinear_tf=True))]
    if characters:
        features.append(('characters', TfidfVectorizer(analyzer='char_wb', ngram_range=(3, 5), min_df=3, max_features=45000, sublinear_tf=True)))
    return Pipeline([('features', FeatureUnion(features)), ('classifier', LogisticRegression(C=c, max_iter=600, class_weight='balanced', random_state=20261021,
solver='liblinear'))])


class OutcomeModel:
    def __init__(self, model=None, metadata=None):
        self.model, self.metadata = model, metadata or {}

    @classmethod
    def load(cls, path, metadata=None):
        path = Path(path)
        return cls(joblib.load(path), metadata) if path.exists() else cls()

    def predict(self, evidence, outcome, intervention, comparator):
        if self.model is None:
            return {'status': 'NOT_TRAINED', 'direction': None, 'scope': 'No learned outcome model installed. Run scripts/train_outcome_model.py.'}
        value = model_input(evidence, outcome, intervention, comparator)
        code = int(self.model.predict([value])[0])
        return {'status': 'MODEL_PROPOSAL_REQUIRES_REVIEW', 'direction_code': code, 'direction': LABELS[code], 'method': self.metadata.get('selected_method',
'TFIDF_LOGISTIC'), 'model_corpus_sha256': self.metadata.get('corpus_sha256'), 'scope': 'Direction of the requested outcome for the specified intervention versus
comparator, on supplied evidence. Direction is not clinical benefit or historical-use validation. No calibrated probability is claimed.'}
```

src/evidence/overview.py
SHA-256: 0598c6cd05af2b6c1ad2a43bf603c9fb67c7486d015fda5038fbbdf5347f41a5

```
"""Readable question results without changing source screening or identity claims."""
from __future__ import annotations

import re
from html import unescape


def publication_text(value):
    return unescape(re.sub(r'<[^>]*>', '', value or ''))


def product_candidate_order(candidates, wording):
    """A disclosed consultation reading order, never an identity confidence score."""
    if re.fullmatch(r'mint(?:\s+(?:product|capsules?))?', wording.strip(), re.I):
        preferred = {'gbif:8707933': 0}  # Peppermint; the bundled supplement trial reading set.
        return sorted(candidates, key=lambda c: preferred.get(c['taxon_id'], 1))
    return list(candidates)


def question_overview(branches, protocol, reading_sets=(), withdrawn=()):
    """Count publications once; retain every candidate-specific check in the packet."""
    grouped = {}
    positions = {}
    def source_priority(pair):
        branch, row = pair
        identity = next((c['status'] for c in row['screening']['checks'] if c['check']=='identity'), 'UNKNOWN')
        return (identity != 'PASS', positions[(branch['taxon_id'],row['id'])], branch['taxon_id'])
    for branch in branches:
        for position, row in enumerate(branch['records'], 1):
            positions[(branch['taxon_id'],row['id'])] = position
            paper = row['paper']
            # Public acquisition uses canonical PubMed IDs. DOI catches mixed-source copies.
            key = ('pmid:' + str(paper['pmid'])) if paper.get('pmid') else (
                'doi:' + paper['doi'].lower().removeprefix('https://doi.org/') if paper.get('doi') else paper['id'])
            grouped.setdefault(key, []).append((branch, row))
    candidates = []
    for rows in grouped.values():
        valid = [(b, r) for b, r in rows if r['screening']['state'] != 'OUTSIDE_REQUESTED_SCOPE']
        if not valid:
            continue
        # BM25 values and cross-encoder logits have different units. Follow the
        # actual branch order, preferring a source-named identity over an
        # unresolved co-mention when a publication has multiple branches.
        b, row = min(valid, key=source_priority)
        checks = {c['check']: c for c in row['screening']['checks']}
        if not checks.get('requested_outcome', {}).get('spans'):
            continue
        trial = 'CLINICAL_TRIAL' in row['study']['types'] and (checks.get('population', {}).get('status') == 'PASS' or row['study'].get('population_status') ==
'HUMAN_TEXT_SIGNAL')
        candidates.append({'paper_id': row['paper']['id'], 'record_id': row['id'],
                           'taxon_id': b['taxon_id'], 'candidate': b['name'],
                           'title': publication_text(row['paper']['title']), 'url': row['paper']['url'],
                           'human_trial_signal': trial, 'screening': row['screening']['state'],
                           'needs_confirmation': [c['check'] for c in checks.values() if c['status'] in ('UNKNOWN', 'PARTIAL')],
                           'score': row['retrieval_score'],
```

```
                           'reading_position': positions[(b['taxon_id'],row['id'])],
                           'scientific_identity_signal': checks.get('identity',{}).get('status')=='PASS'})
    candidates.sort(key=lambda r: (not r['human_trial_signal'], r['screening'] != 'PRIORITY_FOR_READING',
                                   not r['scientific_identity_signal'], r['reading_position'], r['paper_id']))
    overview = {'unique_publications': len(grouped), 'candidate_records': sum(len(b['records']) for b in branches),
                'endpoint_publications': len(candidates),
                'human_trial_signals': sum(r['human_trial_signal'] for r in candidates),
                'top_studies': candidates[:3], 'selected_readings': [],
                'ranking_basis':'Human-trial and scope checks, then scientific-name signals and saved branch reading order. Raw scores from different ranking models
are not compared.'}
    # Only explicit IBS queries activate the selected IBS notes. A general pain query does not.
    query = ' '.join([protocol.question, *protocol.outcome_terms])
    ibs = bool(re.search(r'\birritable bowel syndrome\b|\bIBS\b', query, re.I))
    ids = {b['taxon_id'] for b in branches}
    excluded = set(withdrawn)
    for reading in reading_sets if ibs else ():
        if reading['id'] in excluded or not ids.intersection(reading['taxon_ids']):
            continue
        studies = []
        for note in reading['studies']:
            pub = note['publication']
            if {pub['id'], pub.get('pmid'), pub.get('pmcid')}.intersection(excluded):
                continue
            linked = [(b, r) for rows in grouped.values() for b, r in rows
                      if (r['paper'].get('pmid') == pub.get('pmid') or r['paper']['id'] == pub['id'])
                      and b['taxon_id'] in reading['taxon_ids']
                      and r['screening']['state'] != 'OUTSIDE_REQUESTED_SCOPE'
                      and any(c['check'] == 'requested_outcome' and c.get('spans') for c in r['screening']['checks'])]
            if linked:
                studies.append({**note, 'record_id': linked[0][1]['id'], 'taxon_id': linked[0][0]['taxon_id']})
        if studies:
            overview['selected_readings'].append({'title': reading['title'], 'selection': reading['selection'] + f" Showing {len(studies)} of
{len(reading['studies'])} selected readings linked to this question.",
                'takeaway': reading['takeaway'], 'annotation': reading['annotation'], 'studies': studies})
            if reading['id'] == 'reading-set:peppermint-ibs-trials' and {s['pmid'] for s in studies} == {'26319955','31470006','34319275'}:
                overview['findings_summary'] = ('Among these three selected peppermint-oil trials, one reported a significant advantage on its primary symptom score;
 '
                    'two did not find significant differences on their primary endpoints. Formulations and endpoints differ.')
    return overview
```

src/evidence/product_labels.py
SHA-256: 8a28d655181b5c777f1d125fa7524eb11e3ec2bebae05d94dd9e51a396db4003

```
"""NIH label snapshots. Label claims and quantities are never actual patient use."""
from __future__ import annotations

import hashlib
import json
import re
from pathlib import Path
from typing import Literal
from urllib.parse import urlencode

from pydantic import BaseModel, ConfigDict, Field, model_validator
from src.common import ROOT, now
from src.evidence.question_review import canonical
from src.evidence.integrity import portable_numbers

BASE = 'https://api.ods.od.nih.gov/dsld/v9/'
SOURCE_SCOPE = 'US_LABEL_RECORD_NOT_VERIFIED_AGAINST_BOTTLE'


def source_hash(raw):
    return hashlib.sha256(canonical(raw)).hexdigest()


def ingredients(raw):
    """Flatten with JSON pointers, retaining parenthood and *own* quantities only."""
    result = []
    def walk(rows, path, parent=None, depth=0):
        if not isinstance(rows, list) or depth > 12:
            raise ValueError('Unsupported ingredient hierarchy')
        for i, row in enumerate(rows):
            pointer = path+'/'+str(i)
            if not isinstance(row, dict) or not isinstance(row.get('name'), str):
                raise ValueError('Ingredient name is missing')
            quantities = row.get('quantity') or []
            if not isinstance(quantities, list):
                raise ValueError('Ingredient quantities must be a list')
            if any(not isinstance(q,dict) or not isinstance(q.get('unit',''),(str,type(None))) or not isinstance(q.get('quantity'),(int,float,type(None))) for q in
quantities):
                raise ValueError('Unsupported ingredient quantity')
            # NP = Not Provided in DSLD; its numeric placeholder is not zero dose.
            amount = [dict(q, amount_status='NOT_PROVIDED' if q.get('unit') in ('NP', None, '') or q.get('quantity') is None else 'LABEL_DECLARED') for q in
quantities]
            notes = row.get('notes') or ''
            if not isinstance(notes,str) or not isinstance(row.get('forms') or [],list):
                raise ValueError('Unsupported ingredient notes or forms')
            if any(not isinstance(f,dict) or any(f.get(k) is not None and not isinstance(f[k],str) for k in ('name','prefix')) for f in row.get('forms') or []):
                raise ValueError('Unsupported ingredient form description')
            genus = re.search(r'\bGenus:\s*([A-Z][a-z]+)', notes)
            species = re.search(r'\bSpecies:\s*([a-z][a-z-]+)', notes)
            if not isinstance(row.get('category') or '',str) or not isinstance(row.get('ingredientGroup') or '',str):
                raise ValueError('Unsupported ingredient category or group')
            category = (row.get('category') or '').lower()
            lookup = (genus[1]+' '+species[1]) if genus and species else row.get('ingredientGroup') or row['name']
            result.append({'key':pointer, 'parent_key':parent, 'name':row['name'],
                           'category':category, 'notes':notes, 'forms':row.get('forms') or [],
```

```
                           'quantities':amount, 'lookup':lookup,
                           'plant_lookup_available':category=='botanical'})
            if len(result)>300:
                raise ValueError('This label exceeds the supported 300 ingredient rows')
            walk(row.get('nestedRows') or [],pointer+'/nestedRows',pointer,depth+1)
    walk(raw.get('ingredientRows') or [],'/ingredientRows')
    return result


def label_wording(raw):
    lines = [f"NIH DSLD label {raw['id']}: {raw['fullName']}",
             'Brand: '+str(raw.get('brandName') or 'Not stated'),
             'Label entered: '+str(raw.get('entryDate') or 'Not stated'),
             'Declared ingredients (label quantities, not actual use):']
    for row in ingredients(raw):
        quantities = ['not provided' if q['amount_status']=='NOT_PROVIDED' else f"{q.get('operator') or ''}{q['quantity']} {q['unit']} per
{q.get('servingSizeQuantity')} {q.get('servingSizeUnit') or 'label serving'}" for q in row['quantities']]
        lines.append(row['key']+' '+row['name']+': '+('; '.join(quantities) or 'not provided'))
    lines.append('Full notes, forms, directions and serving data are preserved in the attached source snapshot. Bottle match not established.')
    wording='\n'.join(lines)
    if len(wording)>5000:
        raise ValueError('This label is too long for the consultation form; copy relevant original wording manually')
    return wording


def ingredient_source(raw, key):
    row=next((r for r in ingredients(raw) if r['key']==key),None)
    if row is None:raise ValueError('Ingredient is not in this label')
    return label_wording(raw)+'\nSelected ingredient: '+row['key']+' '+row['name']+'\nExact ingredient notes: '+row['notes']+'\nLabel SHA256: '+source_hash(raw)


class LabelSnapshot(BaseModel):
    model_config = ConfigDict(extra='forbid')
    label_id: int = Field(gt=0)
    sha256: str = Field(pattern=r'^[a-f0-9]{64}$')
    source_url: str
    retrieved_at: str = Field(max_length=80)
    access: str = Field(pattern=r'^(LIVE_OR_CACHED_NIH|BUNDLED_REFERENCE)$')
    raw: dict
    hash_scheme: Literal['NUMERIC_JSON_V1'] | None = None

    @model_validator(mode='after')
    def grounded(self):
        if self.hash_scheme=='NUMERIC_JSON_V1':self.raw=portable_numbers(self.raw)
        if self.source_url != BASE+'label/'+str(self.label_id):
            raise ValueError('Label source must be the fixed NIH endpoint')
        if self.raw.get('id') != self.label_id or not isinstance(self.raw.get('fullName'),str):
            raise ValueError('Label identifier or product name mismatch')
        if len(canonical(self.raw))>1_000_000 or source_hash(self.raw)!=self.sha256:
            raise ValueError('Label source checksum mismatch or size limit exceeded')
        ingredients(self.raw)
        return self


def snapshot(raw, retrieved_at=None, access='LIVE_OR_CACHED_NIH'):
    normal=portable_numbers(raw)
    return LabelSnapshot(label_id=raw['id'],sha256=source_hash(normal),source_url=BASE+'label/'+str(raw['id']),
                         retrieved_at=now() if retrieved_at is None else retrieved_at,access=access,raw=normal,hash_scheme='NUMERIC_JSON_V1').model_dump()
```

```


def summarize(raw):
    return {'label_id':raw['id'], 'name':raw['fullName'], 'brand':raw.get('brandName') or '',
            'entry_date':raw.get('entryDate'), 'off_market':raw.get('offMarket'),
            'upc':raw.get('upcSku'), 'version':raw.get('productVersionCode'),
            'form':(raw.get('physicalState') or {}).get('langualCodeDescription',''),
            'scope':SOURCE_SCOPE}


def reference_labels():
    folder=ROOT/'data'/'product_labels'
    manifest=json.loads((folder/'manifest.json').read_text(encoding='utf-8'))
    result=[]
    for entry in manifest['labels']:
        raw=json.loads((folder/(str(entry['label_id'])+'.json')).read_text(encoding='utf-8'))
        if source_hash(raw)!=entry['sha256']:raise ValueError('Bundled label checksum mismatch')
        result.append(snapshot(raw,manifest['retrieved_at'],'BUNDLED_REFERENCE'))
    return result


def search_labels(client, query, mode='reference', offset=0, market='all'):
    if offset<0 or offset>200 or market not in ('all','on','off'):
        raise ValueError('Unsupported label page or market filter')
    if mode=='reference':
        rows=[s for s in reference_labels() if all(t in (s['raw']['fullName']+' '+str(s['raw'].get('brandName') or '')+' '+str(s['raw'].get('upcSku') or
'')).casefold() for t in query.casefold().split())]
        rows=[s for s in rows if market=='all' or s['raw'].get('offMarket')==({'on':0,'off':1}[market])]
        return {'labels':[summarize(s['raw']) for s in rows[offset:offset+10]], 'access':'BUNDLED_REFERENCE',
                'coverage':'Six fixed US label records for reproducible examples; not a complete or current market search.',
                'total':len(rows),'offset':offset,'has_more':offset+10<len(rows),'market':market}
    if client.offline:
        raise ValueError('Live NIH search is disabled in offline mode. Use the reference examples or manual entry.')
    payload,meta=client.json(BASE+'search-filter?'+urlencode({'q':query,'size':10,'from':offset,
                      'status':{'all':2,'on':1,'off':0}[market],'sort_by':'entryDate','sort_order':'desc'}),max_bytes=2_000_000,patient=False)
    rows=[]
    for hit in payload.get('hits',[]):
        raw=dict(hit['_source'],id=int(hit['_id']))
        rows.append(summarize(raw))
    return {'labels':rows,'access':'LIVE_OR_CACHED_NIH','coverage':'At most 10 NIH matches per page, newest entry first; entry date is not a formulation date or
bottle match. Market status describes the source record.',
            'source':meta,'offset':offset,'has_more':len(rows)==10 and offset<200,'market':market}


def compare_labels(left, right):
    """Compare declared text/quantities without inferring equivalence or potency."""
    left=LabelSnapshot.model_validate(left);right=LabelSnapshot.model_validate(right)
    def rows_by_identity(raw):
        rows=ingredients(raw);paths={};groups={}
        for row in rows:
            # Hierarchical names survive reordering; duplicates are kept as lists.
            parent=paths.get(row['parent_key'],())
            path=parent+(row['name'].strip().casefold(),)
            paths[row['key']]=path
            value={k:row[k] for k in ('name','category','notes','forms','quantities')}
            # Serving order and DV target order are source metadata, not component identity.
            value['quantities']=[{k:v for k,v in q.items() if k!='servingSizeOrder'} for q in value['quantities']]
            groups.setdefault(path,[]).append({'pointer':row['key'],'declared':value})
```

```
        return groups
    a=rows_by_identity(left.raw);b=rows_by_identity(right.raw);changes=[]
    for path in sorted(set(a)|set(b)):
        before=a.get(path,[]);after=b.get(path,[])
        av=sorted([r['declared'] for r in before],key=lambda v:canonical(v))
        bv=sorted([r['declared'] for r in after],key=lambda v:canonical(v))
        if av!=bv:
            changes.append({'ingredient_path':list(path),'status':'ADDED' if not before else 'REMOVED' if not after else 'CHANGED',
                            'before':before,'after':after})
    fields=('fullName','brandName','upcSku','productVersionCode','physicalState','servingSizes','otheringredients')
    metadata=[{'field':f,'before':left.raw.get(f),'after':right.raw.get(f)} for f in fields if left.raw.get(f)!=right.raw.get(f)]
    return {'left':summarize(left.raw),'right':summarize(right.raw),'source_sha256':[left.sha256,right.sha256],
            'ingredient_changes':changes,'label_field_changes':metadata,
            'same_selected_source':left.raw==right.raw,
            'scope':'DECLARED_LABEL_DIFFERENCES_NOT_BOTTLE_VERIFICATION_OR_CLINICAL_EQUIVALENCE',
            'limits':['Changed wording does not by itself prove a formulation change.',
                      'No detected difference does not establish chemical or clinical equivalence.',
                      'Blend totals are not assigned to individual ingredients; NP remains not provided.']}


def read_label(client, identifier, mode='reference'):
    if mode=='reference':
        match=next((s for s in reference_labels() if s['label_id']==identifier),None)
        if match is None:raise ValueError('Label is not in the bounded reference examples')
        return match
    raw,meta=client.json(BASE+'label/'+str(identifier),max_bytes=1_000_000,patient=False)
    return snapshot(raw,meta['retrieved_at'])
```

src/evidence/question_provenance.py
SHA-256: a4be67375746188b758b741fe3b309d7100bf3cc6fee3ed77b8dc3d9dbdf8ece

```
"""Ground every displayed screening quote in its stored publication source."""
from __future__ import annotations


def grounded_span(paper, span):
    if not isinstance(span, dict):
        raise ValueError('A screening quote must be an object')
    start, end, text = span.get('start'), span.get('end'), span.get('text')
    if type(start) is not int or type(end) is not int or not isinstance(text, str) or not text:
        raise ValueError('A screening quote needs exact text and integer offsets')
    kind = span.get('source_kind')
    if kind in ('FULL_TEXT_METHODS', 'FULL_TEXT_METHODS_TABLE'):
        matches = [c for c in paper.get('methods_contexts', []) if c.get('id') == span.get('source_id')]
        if len(matches) != 1:
            raise ValueError('The quoted Methods source is not in this publication')
        context = matches[0]
        if (context.get('source_kind') or 'FULL_TEXT_METHODS') != kind:
            raise ValueError('The quoted Methods source type differs from its stored context')
        source, url, licence = context['text'], context['url'], context.get('licence')
    elif kind in ('PUBLICATION_TITLE', 'PUBLICATION_ABSTRACT', None):
        # Legacy untyped spans were abstract-relative. Retain that meaning when
        # both a title and abstract happen to contain the same quote at an offset.
        choices = [('PUBLICATION_ABSTRACT', paper.get('abstract') or ''),
                   ('PUBLICATION_TITLE', paper['title'])]
        matched = [(k, s) for k, s in choices if (kind is None or kind == k)
                   and 0 <= start < end <= len(s) and s[start:end] == text]
        if not matched:
            raise ValueError('Question evidence/source span mismatch')
        kind, source = matched[0]
        url, licence = paper['url'], paper.get('licence')
    else:
        raise ValueError('Unsupported screening quote source type')
    if not 0 <= start < end <= len(source) or source[start:end] != text:
        raise ValueError('Question evidence/source span mismatch')
    if span.get('url') is not None and span['url'] != url:
        raise ValueError('The quote URL differs from its stored source')
    if span.get('licence') is not None and span['licence'] != licence:
        raise ValueError('The quote rights differ from its stored source')
    return {**span, 'source_kind': kind, 'url': url,
            **({'licence': licence} if licence else {})}


def check_spans(check):
    spans = check.get('spans', [])
    if not isinstance(spans, list):
        raise ValueError('Screening quotes must be a list')
    values = list(spans)
    if check.get('span') is not None and check['span'] not in values:
        values.append(check['span'])
    return values


def ground_checks(paper, screening):
    for check in screening['checks']:
        check['spans'] = [grounded_span(paper, s) for s in check_spans(check)]
```

```
        if check.get('span') is not None:
            check['span'] = grounded_span(paper, check['span'])
    return screening


def validate_checks(paper, screening):
    for check in screening['checks']:
        for span in check_spans(check):
            grounded_span(paper, span)
```

src/evidence/question_review.py
SHA-256: 0f4814edd3ac643809ef209cfc22d169dae9997b26f9379e8e01752816830760

```python
"""Question-driven medicinal-plant evidence screening; never a treatment verdict."""
from __future__ import annotations

import copy
import hashlib
import json
import re
import sqlite3
import uuid
from pathlib import Path
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field, model_validator
from src.common import now
from src.nlp.extract import CONDITIONS, normalize
from src.evidence.comparability import check_identity, check_population, check_design, check_comparator, check_preparation, name_pattern, compiled
from src.evidence.context import scope_study_preparation
from src.evidence.digest import combination
from src.evidence.classify import classify_study
from src.evidence.question_search import search_plan, fetch_public_studies
from src.evidence.integrity import portable_numbers
from src.evidence.overview import product_candidate_order, question_overview, publication_text
from src.evidence.question_provenance import ground_checks, validate_checks


class ResearchQuestion(BaseModel):
    model_config = ConfigDict(extra='forbid')
    question: str = Field(min_length=10, max_length=800)
    plant_name: str = Field(min_length=2, max_length=120)
    condition_id: str | None = Field(default=None,max_length=80)
    retrieval_scope: Literal['question','topic'] = 'question'
    source_mode: Literal['local','public','pubmed'] = 'local'
    reference_pmids: list[str] = Field(default_factory=list,max_length=10)
    population: Literal['human', 'any'] = 'human'
    preparation: str = Field(default='', max_length=120)
    comparator: Literal['controlled', 'placebo', 'any'] = 'controlled'
    intervention: Literal['single_plant', 'any'] = 'single_plant'
    outcome_terms: list[str] = Field(min_length=1, max_length=8)
    source_text: str = Field(default='', max_length=10000)
    source_url: str = Field(default='', max_length=1000)
    candidate_ids: list[str] = Field(default_factory=list, max_length=12)
    ranking: Literal['auto', 'bm25'] = 'auto'

    @model_validator(mode='after')
    def valid(self):
        self.question = self.question.strip()
        self.plant_name = self.plant_name.strip()
        self.condition_id = self.condition_id.removeprefix('condition:') if self.condition_id else None
        if self.condition_id is not None and self.condition_id not in CONDITIONS:
            raise ValueError('Choose a supported health research topic')
        if self.retrieval_scope=='topic' and self.condition_id is None:
            raise ValueError('Choose a broad topic when narrowing retrieval by topic')
        self.reference_pmids=list(dict.fromkeys(self.reference_pmids))
        if any(not re.fullmatch(r'[1-9]\d{0,9}',p) for p in self.reference_pmids):
            raise ValueError('Known PubMed references must be numeric identifiers, at most ten')
```

```
        if self.source_mode=='local' and self.reference_pmids:
            raise ValueError('Choose public literature lookup to retrieve known PubMed references, or clear the reference IDs')
        if len(self.question) < 10 or len(self.plant_name) < 2:
            raise ValueError('A substantive research question and plant name are required')
        self.outcome_terms = list(dict.fromkeys(t.strip() for t in self.outcome_terms))
        if any(not 3 <= len(t) <= 100 for t in self.outcome_terms):
            raise ValueError('Each literal outcome phrase must contain 3\u2013100 characters')
        if self.source_url and not self.source_url.startswith(('https://', 'http://')):
            raise ValueError('Source reference must be an http(s) URL; it is not fetched')
        if len(set(self.candidate_ids)) != len(self.candidate_ids):
            raise ValueError('Duplicate candidate assumptions')
        return self


def canonical(value):
    return json.dumps(value, sort_keys=True, ensure_ascii=False, separators=(',', ':')).encode('utf-8')


def wording_spans(paper, terms):
    """Literal evidence only. Missing wording never becomes an efficacy verdict."""
    found = []
    for field in ('title', 'abstract'):
        text = paper.get(field) or ''
        for term in terms:
            for match in re.finditer(r'(?<!\w)'+re.escape(term)+r'(?!\w)', text, re.I):
                found.append({'source_kind': 'PUBLICATION_'+field.upper(), 'text': match.group(),
                              'start': match.start(), 'end': match.end(), 'url': paper['url']})
    return found


def source_reading(paper, taxon, study, taxon_match, known_names, same_genus_names=None):
    checks = [check_identity(paper, taxon, taxon_match)]
    if checks[0]['status']=='FAIL':
        own={normalize(taxon['name']),*(normalize(a['name']) for a in taxon['aliases'])}
        genus=normalize(taxon['name'].split()[0])+' '
        possible=same_genus_names if same_genus_names is not None else [n for n in known_names if n.startswith(genus)]
        other_title_names=[n for n in possible if n not in own and compiled(name_pattern(n)).search(paper['title'])]
        if other_title_names and not set(study['types']) & {'REVIEW','SYSTEMATIC_REVIEW','META_ANALYSIS'}:
            match=compiled(name_pattern(sorted(other_title_names)[0])).search(paper['title'])
            checks[0]={**checks[0],'detail':'The title explicitly names another species in this genus, not the requested candidate: '+',
  '.join(sorted(other_title_names))+'. Check Methods and synonym interpretation before attributing it to this branch.',
                'spans':[{'source_kind':'PUBLICATION_TITLE','start':match.start(),'end':match.end(),'text':match.group(),'url':paper['url']}]}
            return checks,copy.deepcopy(study),True
        else:
            # A missing binomial is uncertainty; a different explicit intervention name is a conflict.
            checks[0]={**checks[0],'status':'UNKNOWN',
                       'detail':'The available source does not establish this candidate\u2019s scientific identity. A common or genus name needs Methods
  verification; this alone is not a scope exclusion.'}
    study = scope_study_preparation(paper, taxon, copy.deepcopy(study))
    return checks,study,False


def screen_question(protocol, paper, taxon, study, taxon_match, known_names, same_genus_names=None,prepared=None):
    checks,study,conflict=copy.deepcopy(prepared if prepared is not None else source_reading(paper,taxon,study,taxon_match,known_names,same_genus_names))
    if conflict:
        return {'state':'OUTSIDE_REQUESTED_SCOPE','checks':checks,'scope_conflicts':['identity'],
                'note':'Other comparability checks were not applied after this explicit identity conflict. No efficacy conclusion follows.'},study
    if protocol.population == 'human':
```

```
        checks += [check_population(study), check_design(study)]
    if protocol.preparation:
        prep = check_preparation([{'text': protocol.preparation}], study)
        prep['detail'] = prep['detail'].replace('Historical:', 'Question:').replace('historical ', 'requested ')
        checks.append(prep)
    if protocol.comparator != 'any':
        comp = check_comparator(study, paper)
        if not set(study['types']) & {'REVIEW','SYSTEMATIC_REVIEW','META_ANALYSIS'}:
            pattern=r'\b(?:single[- ]arm|(?:uncontrolled|non[- ]controlled)\s+(?:(?:open[-
]label|pilot|clinical|prospective)\s+)*(?:trial|study|design)|without\s+(?:a\s+)?(?:control\s+(?:group|arm)|comparator(?:\s+(?:group|arm))?))\b'
            match=re.search(pattern,paper['title'],re.I)
            field='title'
            if not match:
                methods=re.search(r'\b(?:methods?|study design)\s*:\s*(.*?)(?=\b(?:results?|conclusions?)\s*:|$)',paper.get('abstract') or '',re.I|re.S)
                if methods:
                    found=re.search(pattern,methods[1],re.I)
                    if found:
                        start=methods.start(1)+found.start();end=methods.start(1)+found.end();field='abstract'
                        match=(start,end)
            if match:
                start,end=match if isinstance(match,tuple) else match.span()
                comp={'check':'comparator','status':'FAIL','detail':'The title or explicit Methods describes an uncontrolled/single-arm study. It does not meet this
question\u2019s required comparator.',
                      'spans':[{'source_kind':'PUBLICATION_'+field.upper(),'start':start,'end':end,'text':paper[field][start:end],'url':paper['url']}]}
        if protocol.comparator == 'placebo' and comp['status'] != 'FAIL':
            spans = wording_spans(paper, ['placebo'])
            comp = {'check': 'comparator', 'status': 'PARTIAL' if spans else 'UNKNOWN',
                    'detail': 'Placebo wording found; allocation and actual comparison still need reading.' if spans else 'A placebo comparison is not established
from available text.',
                    'spans': spans}
        checks.append(comp)
    if protocol.intervention == 'single_plant':
        names = {normalize(taxon['name'])} | {normalize(a['name']) for a in taxon.get('aliases', [])}
        mixed = combination({'paper': paper, 'study': study}, names, known_names)
        checks.append({'check': 'single_plant', 'status': 'FAIL' if mixed else 'PARTIAL',
                       'detail': 'Combination or other-plant signal: cannot attribute the intervention to this plant alone.' if mixed else 'No combination signal was
 found; ingredient isolation and study arms still require Methods reading.'})
    spans = wording_spans(paper, protocol.outcome_terms)
    checks.append({'check': 'requested_outcome', 'status': 'PARTIAL' if spans else 'UNKNOWN',
                   'detail': 'Requested literal outcome wording occurs; endpoint definition and result attribution require reading.' if spans else 'None of your
exact outcome phrases was found. Synonyms or full text may still be relevant.',
                   'spans': spans})
    if not paper.get('abstract'):
        checks.append({'check': 'abstract_access', 'status': 'UNKNOWN', 'detail': 'Bibliographic metadata only; no abstract available locally.'})
    failed = [c['check'] for c in checks if c['status'] == 'FAIL']
    state = 'OUTSIDE_REQUESTED_SCOPE' if failed else 'NEEDS_FULL_TEXT' if any(c['status'] == 'UNKNOWN' for c in checks) else 'PRIORITY_FOR_READING'
    return {'state': state, 'checks': checks, 'scope_conflicts': failed,
            'note': 'Reading priority is not eligibility confirmation, a risk-of-bias judgement or evidence of clinical benefit.'}, study


def run_question(pipeline, protocol: ResearchQuestion):
    resolution = pipeline.resolver.resolve(protocol.plant_name)
    all_candidates = product_candidate_order(resolution['candidates'], protocol.plant_name)
    known = {c['taxon_id'] for c in all_candidates}
    if not set(protocol.candidate_ids) <= known:
        raise ValueError('Selected species is not a candidate for this plant wording')
    candidates = [c for c in all_candidates if not protocol.candidate_ids or c['taxon_id'] in protocol.candidate_ids]
    if len(candidates) > 12:
```

```
        raise ValueError('This name has more than 12 candidates; use a more specific source name')
    withdrawn = pipeline.withdrawn()
    candidates = [c for c in candidates if c['taxon_id'] not in withdrawn]
    condition = 'condition:'+protocol.condition_id if protocol.condition_id else None
    index=pipeline.index;external=None;added_ids=set()
    if protocol.source_mode!='local':
        from src.ingestion.client import ScientificClient
        from src.retrieval.index import LiteratureIndex
        # Separate counters, shared durable cache. The original snapshot/index is not mutated.
        client=ScientificClient(cache=pipeline.client.cache,offline=pipeline.client.offline)
        acquired,external=fetch_public_studies(client,search_plan([pipeline.taxmap[c['taxon_id']] for c in
candidates],protocol.outcome_terms,pipeline.index,protocol.reference_pmids))
        by_id={p['id']:p for p in pipeline.papers}
        added_ids={p['id'] for p in acquired if p['id'] not in by_id}
        by_id.update({p['id']:p for p in acquired})
        if acquired:
            index=LiteratureIndex(list(by_id.values()),pipeline.taxa,encoder=pipeline.index.encoder)
            index.reranker=pipeline.index.reranker
    query = ' '.join([protocol.question, protocol.preparation, *protocol.outcome_terms,
                      'human clinical trial' if protocol.population == 'human' else '',
                      protocol.comparator if protocol.comparator != 'any' else ''])
    # The protocol now changes the retrieval query and every record's screening.
    scores = index.bm25(query)
    known_names = {normalize(t['name']) for t in pipeline.taxa}
    knowledge_key=hashlib.sha256(canonical(sorted(known_names))).hexdigest()
    source_cache=getattr(pipeline,'question_source_cache',None)
    if source_cache is None:
        source_cache={};pipeline.question_source_cache=source_cache
    branches = []
    for candidate in candidates:
        taxon = pipeline.taxmap[candidate['taxon_id']]
        taxon_key=hashlib.sha256(canonical({k:taxon.get(k) for k in ('id','name','aliases','drug_names')})).hexdigest()
        same_genus_names=[n for n in known_names if n.startswith(normalize(taxon['name'].split()[0])+' ')]
        if protocol.retrieval_scope=='topic':
            hits=index.matching(taxon,condition,scores=scores,method='QUESTION_BM25',genus=True)
        else:
            # The endpoint question is no longer silently gated by historical topic vocabulary.
            named=dict(index.documents_naming(taxon))
            for i,_ in index.postings.get(normalize(taxon['name'].split()[0]),[]):
                named.setdefault(i,'COMMON_NAME_ONLY_IDENTITY_UNCERTAIN')
            hits=[{'paper':index.papers[i],'retrieval_score':float(scores[i]),'taxon_match':identity}
                  for i,identity in named.items()]
        if len(hits)>2000:raise ValueError('This name has more than 2000 records; narrow by a broad topic or use a more specific identity')
        rows_by_id = {r['paper']['id']: r for r in pipeline.topic_studies(taxon['id'], condition)} if condition else {}
        records = []
        for hit in hits:
            paper = hit['paper']
            if paper['id'] in withdrawn or paper.get('retracted'):
                continue
            original = rows_by_id.get(paper['id'])
            if original and not paper.get('question_acquisition'):
                design=original['study']
            elif not paper.get('question_acquisition'):
                design=pipeline.study_designs.get(paper['id'])
                if design is None:
                    design=classify_study(paper,pipeline.study_model);pipeline.study_designs[paper['id']]=design
            else:
                design=classify_study(paper,pipeline.study_model)
```

```
            paper_key=hashlib.sha256(canonical({k:paper.get(k) for k in ('id','title','abstract','publication_types','methods_contexts','url','licence')})).hexdigest()
            cache_key=(knowledge_key,taxon_key,paper_key,hit['taxon_match'],getattr(pipeline,'model_fingerprint',''))
            prepared=source_cache.get(cache_key)
            if prepared is None:
                prepared=source_reading(paper,taxon,design,hit['taxon_match'],known_names,same_genus_names)
                if len(source_cache)>=2048:source_cache.clear()
                source_cache[cache_key]=prepared
            screening, study = screen_question(protocol, paper, taxon, design, hit['taxon_match'], known_names,same_genus_names,prepared)
            ground_checks(paper, screening)
            records.append({'id': hashlib.sha256((taxon['id']+'|'+paper['id']).encode()).hexdigest()[:24],
                            'taxon_id': taxon['id'], 'paper': paper, 'study': study,
                            'screening': screening, 'topic_finding': original['relation'] if original else None,
                            'source_origin':'PUBLIC_QUESTION_ACQUISITION' if paper.get('question_acquisition') else 'LOCAL_SNAPSHOT',
                            'finding_scope': 'ORIGINAL_BROAD_TOPIC_READING_NOT_A_VERDICT_ON_THE_REQUESTED_ENDPOINT',
                            'retrieval_score': hit['retrieval_score'], 'ranking_method': 'QUESTION_BM25'})
        records.sort(key=lambda r: -r['retrieval_score'])
        reranked = 0
        if protocol.ranking == 'auto' and index.reranker.available and records:
            eligible=[r for r in records if r['screening']['state']!='OUTSIDE_REQUESTED_SCOPE']
            eligible.sort(key=lambda r:0 if any(c['check']=='requested_outcome' and c.get('spans') for c in r['screening']['checks']) else 1)
            subset = eligible[:12]
            ranked = index.reranker.score(query, [r['paper']['title']+' '+(r['paper'].get('abstract') or '') for r in subset]) if subset else []
            for row, score in zip(subset, ranked):
                row['retrieval_score'] = float(score)
                row['ranking_method'] = 'LOCAL_CROSS_ENCODER_WITH_RESEARCH_QUESTION'
            subset.sort(key=lambda r: -r['retrieval_score'])
            selected={r['id'] for r in subset}
            records = subset+[r for r in records if r['id'] not in selected]
            reranked = len(subset)
        order = {'PRIORITY_FOR_READING': 0, 'NEEDS_FULL_TEXT': 1, 'OUTSIDE_REQUESTED_SCOPE': 2}
        # Stable partition retains actual question-sensitive ranking within queues.
        records.sort(key=lambda r: (order[r['screening']['state']],
            0 if any(c['check']=='requested_outcome' and c.get('spans') for c in r['screening']['checks']) else 1))
        for position, row in enumerate(records, 1):
            row['reading_position'] = position
        branches.append({'taxon_id': taxon['id'], 'name': taxon['name'], 'identity_reasons': candidate['reasons'],
                         'records': records, 'reranked_records': reranked,
                         'counts': {state: sum(r['screening']['state'] == state for r in records) for state in order}})
    packet = {'schema': 'foliatrace-question-review-1', 'created_at': now(),
              'protocol': protocol.model_dump(), 'query_used': query,
              'source_reference': {'text': protocol.source_text, 'url': protocol.source_url,
                                   'rights': 'USER_RIGHTS_UNVERIFIED', 'fetched': False},
              'resolution_status': resolution['status'], 'candidate_count_before_selection': len(all_candidates),
              'branches': branches, 'code_fingerprint': pipeline.code_fingerprint,
              'snapshot_sha256': pipeline.snapshot,
              'quote_integrity': 'CHECKED_QUOTE_LOCATIONS_V1',
              'coverage': {'search_scope': 'LOCAL_SNAPSHOT_PLUS_BOUNDED_PUBLIC_SEARCH_NOT_SYSTEMATIC' if external else 'CURRENT_LOCAL_SNAPSHOT_NOT_EXHAUSTIVE_SYSTEMATIC_SEARCH',
                           'papers_in_snapshot': len(pipeline.papers),
                           'papers_searched':len(index.papers),'new_publication_ids':sorted(added_ids),
                           'retrieval_scope':protocol.retrieval_scope,
                           'rerank_limit_per_candidate':12,
                           'abstracts_available': sum(bool(p.get('abstract')) for p in index.papers),
                           'network_used': bool(external and external['network_used']),
                           **({'public_search':external} if external else {}),
                           'model': 'LOCAL_CROSS_ENCODER' if any(b['reranked_records'] for b in branches) else 'BM25_NO_MODEL_USED',
                           'all_matching_records_retained': True},
```

```
              'limitations': ['No treatment recommendation or automatic clinical efficacy verdict.',
                              'Literal endpoint wording is not semantic equivalence; missing wording is not evidence of no effect.',
                              'Species names do not establish the identity of a historical specimen or a product.',
                              'Population, preparation, ingredient isolation, endpoints and comparator allocation require source reading.',
                              'All matching local records, including scope conflicts, remain in the export. This is not an exhaustive systematic review.',
                              'Source and publisher rights apply to all exports, including privately cached abstracts.']}
    packet['overview'] = question_overview(branches, protocol, pipeline.trial_readings, withdrawn)
    packet['numeric_json_schema']='SAFE_NUMBERS_V1'
    packet=portable_numbers(packet)
    validate_packet(packet)
    return packet


def validate_packet(packet):
    if not isinstance(packet, dict):
        raise ValueError('Research review must be an object')
    if packet.get('schema') != 'foliatrace-question-review-1':
        raise ValueError('Unsupported research review schema')
    ResearchQuestion.model_validate(packet['protocol'])
    if not isinstance(packet['branches'], list) or len(packet['branches']) > 12:
        raise ValueError('Too many candidate branches')
    seen = set()
    for branch in packet['branches']:
        if not isinstance(branch['name'], str) or branch['taxon_id'] in seen:
            raise ValueError('Invalid or duplicate species branch')
        seen.add(branch['taxon_id'])
        if len(branch['records']) > 2000:
            raise ValueError('Too many study records')
        for row in branch['records']:
            if not isinstance(row['study'], dict) or not isinstance(row['screening'], dict):
                raise ValueError('Invalid study or screening record')
            if row['screening']['state'] not in ('OUTSIDE_REQUESTED_SCOPE','NEEDS_FULL_TEXT','PRIORITY_FOR_READING'):
                raise ValueError('Invalid reading queue')
            for key in ('id','title','url','licence'):
                if not isinstance(row['paper'][key], str):
                    raise ValueError('Invalid publication metadata')
            if row['taxon_id'] != branch['taxon_id']:
                raise ValueError('A study crossed its candidate identity branch')
            validate_checks(row['paper'], row['screening'])
    return packet


class ReadingDecision(BaseModel):
    model_config = ConfigDict(extra='forbid')
    record_id: str = Field(pattern=r'^[a-f0-9]{24}$')
    decision: Literal['KEEP_FOR_READING', 'EXCLUDE_FROM_QUESTION', 'NEEDS_FULL_TEXT']
    reason: str = Field(min_length=5, max_length=2000)
    reviewer: str = Field(default='Local researcher', min_length=1, max_length=80)

    @model_validator(mode='after')
    def substantive(self):
        if len(self.reason.strip())<5 or not self.reviewer.strip():
            raise ValueError('A substantive reason and recorder name are required')
        return self


class QuestionStore:
    def __init__(self, path: Path):
```

```
        self.path = path
        with self.connect() as db:
            db.executescript('CREATE TABLE IF NOT EXISTS health_reviews(id TEXT PRIMARY KEY, saved_at TEXT NOT NULL, origin TEXT NOT NULL, payload TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS health_review_decisions(seq INTEGER PRIMARY KEY AUTOINCREMENT, review_id TEXT NOT NULL, payload TEXT NOT NULL);')

    def connect(self):
        from contextlib import contextmanager
        @contextmanager
        def opened():
            db = sqlite3.connect(self.path, timeout=20)
            try:
                with db:
                    yield db
            finally:
                db.close()
        return opened()

    def put(self, packet, origin='LOCAL_QUESTION_REVIEW', decisions=()):
        validate_packet(packet)
        if len(canonical({'packet':packet,'decision_history':decisions}))>19_000_000:
            raise ValueError('Review is too large for portable export; narrow the candidate selection')
        identifier = uuid.uuid4().hex
        record_ids = {r['id'] for b in packet['branches'] for r in b['records']}
        for value in decisions:
            parsed = ReadingDecision.model_validate({k:v for k,v in value.items() if k != 'created_at'})
            if parsed.record_id not in record_ids:
                raise ValueError('Decision belongs to a different research review')
        with self.connect() as db:
            db.execute('INSERT INTO health_reviews VALUES(?,?,?,?)', (identifier, now(), origin, canonical(packet).decode()))
            for decision in decisions:
                db.execute('INSERT INTO health_review_decisions(review_id,payload) VALUES(?,?)', (identifier, canonical(decision).decode()))
        return self.get(identifier)

    def get(self, identifier):
        with self.connect() as db:
            row = db.execute('SELECT id,saved_at,origin,payload FROM health_reviews WHERE id=?', (identifier,)).fetchone()
            decisions = [json.loads(v[0]) for v in db.execute('SELECT payload FROM health_review_decisions WHERE review_id=? ORDER BY seq',
(identifier,)).fetchall()]
        if row is None:
            return None
        payload = {'packet': json.loads(row[3]), 'decision_history': decisions}
        return {'id': row[0], 'saved_at': row[1], 'origin': row[2], **payload,
                'sha256': hashlib.sha256(canonical(payload)).hexdigest(),
                'first_result_sha256': hashlib.sha256(canonical(payload['packet'])).hexdigest()}

    def list(self):
        with self.connect() as db:
            return [dict(zip(('id','saved_at','origin','question','plant_name'), row)) for row in db.execute("SELECT
id,saved_at,origin,json_extract(payload,'$.protocol.question'),json_extract(payload,'$.protocol.plant_name') FROM health_reviews ORDER BY saved_at DESC LIMIT 100")]

    def decide(self, identifier, decision):
        record = self.get(identifier)
        if not record:
            raise ValueError('Research review not found')
        if record['origin'] != 'LOCAL_QUESTION_REVIEW':
            raise ValueError('Imported reviews are read-only; run the question locally to make a current decision')
        if decision.record_id not in {r['id'] for b in record['packet']['branches'] for r in b['records']}:
            raise ValueError('Decision target is outside this question')
```

```
        with self.connect() as db:
            db.execute('INSERT INTO health_review_decisions(review_id,payload) VALUES(?,?)', (identifier, canonical({**decision.model_dump(), 'created_at': now()}).decode()))
        return self.get(identifier)


def review_markdown(record):
    p = record['packet'];q = p['protocol']
    lines = ['# FoliaTrace health research review', q['question'],
             'Origin: '+record['origin'], 'Research screening only; not a treatment verdict.',
             '## Protocol fixed before retrieval', json.dumps(q, ensure_ascii=False, indent=2),
             'Actual retrieval query: '+p['query_used'], '## Source reference',
             p['source_reference']['text'], p['source_reference']['url'], 'Rights: USER_RIGHTS_UNVERIFIED']
    overview = p.get('overview') or question_overview(p['branches'], ResearchQuestion.model_validate(q))
    lines += ['## Search overview', f"{overview['unique_publications']} unique publications; {overview['endpoint_publications']} with requested wording and no detected scope conflict; {overview['human_trial_signals']} with human-trial signals.",
              'Study signals are reading leads; eligibility and product applicability require professional review.']
    if overview.get('findings_summary'):
        lines += [overview['findings_summary']]
    for reading in overview['selected_readings']:
        lines += ['### Selected trial findings', reading['selection'], reading['takeaway'], reading['annotation']['method']]
        for note in reading['studies']:
            lines += [note['label'], note['publication']['url'], note['primary_outcome'], note['primary_result'], note['other_results']]
    current = {d['record_id']:d for d in record['decision_history']}
    for branch in p['branches']:
        lines += ['## Candidate assumption: '+branch['name'], branch['taxon_id']]
        for row in branch['records']:
            paper = row['paper'];decision = current.get(row['id'])
            lines += ['### '+publication_text(paper['title']), paper['url'], 'Rights: '+paper['licence'],
                      row['screening']['state'], json.dumps(row['screening'], ensure_ascii=False, indent=2)]
            if decision:
                lines += ['Researcher decision: '+decision['decision'], decision['reason']]
    lines += ['## Coverage', json.dumps(p['coverage'], ensure_ascii=False, indent=2), *p['limitations'],
              'First result SHA256: '+record['first_result_sha256'], '## Decision history', json.dumps(record['decision_history'], ensure_ascii=False, indent=2)]
    return '\n\n'.join(lines)+'\n'
```

src/evidence/question_search.py
SHA-256: 5197e31404c50c2bd1d93c6b9f0a01eb57a45abc443d03fbf0ce6a686e06122e

```
"""Bounded Europe PMC lookup of PubMed-indexed records, using public terms only."""
from __future__ import annotations
import html
import re
import time
from urllib.parse import urlencode
from src.nlp.extract import SENSE_REQUIRED, NEVER_A_PLANT, normalize
from src.retrieval.index import tokens

SEARCH='https://www.ebi.ac.uk/europepmc/webservices/rest/search'


def public_term(value):
    text=' '.join(re.sub(r'[\x00-\x1f"\[\]\\]', ' ', value).split())
    if not text:raise ValueError('A substantive public outcome phrase is required')
    return 'TITLE_ABS:"'+text+'"'


def public_names(taxon,index=None):
    names=[taxon['name']]+[a['name'] for a in taxon['aliases'] if a['kind']=='SYNONYM' and re.fullmatch(r'[A-Z][a-z-]+ [a-z-]+',a['name'])][:3]
    common=[a['name'] for a in taxon['aliases'] if a['kind']=='VERNACULAR' and a.get('lang','en')=='en' and a['name'].isascii() and (len(a['name'])>=7 or ' ' in
a['name']) and normalize(a['name']) not in SENSE_REQUIRED|NEVER_A_PLANT]
    def frequency(name):
        if index is None:return 0
        parts=tokens(name)
        if not parts:return 0
        rows=index.postings.get(min(parts,key=lambda w:index.df.get(w,0)),[])
        pattern=re.compile(r'\b'+re.escape(normalize(name))+r'\b')
        return sum(bool(pattern.search(index.texts[i])) for i,_ in rows)
    scored=[(frequency(n),n) for n in set(common)]
    usable=[n for count,n in sorted(scored,key=lambda v:(-v[0],len(v[1]),v[1])) if count or index is None][:2]
    return list(dict.fromkeys(names+taxon.get('drug_names',[])[:2]+usable))


def search_plan(taxa,outcomes,index=None,reference_pmids=()):
    if len(taxa)>3:raise ValueError('Public lookup supports at most three candidate identities; use an exact scientific name')
    endpoints=' OR '.join(public_term(t) for t in outcomes)
    plans=[]
    for t in taxa:
        names=public_names(t,index)
        plans.append({'taxon_id':t['id'],'name':t['name'],'names_used':names,
            'term':'SRC:MED AND ('+' OR '.join(public_term(n) for n in names)+') AND ('+endpoints+')',
            'orders':[{'sort':'relevance','limit':12},{'sort':'pub_date','limit':8}]})
    if reference_pmids:
        plans.append({'taxon_id':'REFERENCE_PMIDS','name':'Known PubMed reference identifiers','names_used':[],
            'term':'SRC:MED AND ('+' OR '.join('EXT_ID:'+p for p in reference_pmids)+')',
            'orders':[{'sort':'identifiers','limit':len(reference_pmids)}]})
    return plans


def plain(value):
    value=re.sub(r'</h[1-6]>|</label>', ': ', value or '',flags=re.I)
    return ' '.join(html.unescape(re.sub(r'<[^>]*>',' ',value)).split())


```

```
def paper_from_record(row,meta,candidate):
    if not isinstance(row,dict) or not isinstance(row.get('title'),str):raise ValueError('Invalid publication record')
    identifier=str(row['id'])
    if row.get('source')!='MED' or not identifier.isdigit():raise ValueError('Source returned a record outside the PubMed-indexed scope')
    type_list=row.get('pubTypeList') or {}
    if not isinstance(type_list,dict):raise ValueError('Invalid publication types')
    types=type_list.get('pubType',[]);original=row.get('abstractText') or ''
    if not isinstance(types,list) or any(not isinstance(t,str) for t in types) or not isinstance(original,str):
        raise ValueError('Invalid publication types or abstract text')
    return {'id':'PMID'+identifier,'pmid':identifier,'doi':row.get('doi'),'pmcid':row.get('pmcid'),
        'title':plain(row['title']),'original_title':plain(row['title']),'abstract':plain(original),
        'abstract_source_markup':original,'year':int(row['pubYear']) if str(row.get('pubYear','')).isdigit() else None,
        'authors':row.get('authorString') or '','publication_types':types,
        'licence':'EUROPE_PMC_RECORD_ABSTRACT_RIGHTS_WITH_PUBLISHER','source_id':'europe_pmc',
        'url':'https://pubmed.ncbi.nlm.nih.gov/'+identifier+'/',
        'provenance':meta,'abstract_rights':'LOCAL_CACHE_NOT_REDISTRIBUTED',
        'retracted':row.get('isRetracted') in ('Y',True) or any('retract' in t.lower() for t in types),
        'question_acquisition':{'provider':'Europe PMC / PubMed-indexed record','source':meta,'candidate_search':candidate,
             'text_scope':'Plain text rendered from source markup; quoted offsets refer to the stored plain text.'}}


def fetch_public_studies(client,plans):
    papers={};queries=[];errors=[];retracted=[];deadline=time.monotonic()+30;before=client.stats.get('requests',0)
    for plan in plans:
        entry={**plan,'searches':[],'returned_ids':[]};queries.append(entry)
        for order in plan['orders']:
            query=plan['term']+(' sort_date:y' if order['sort']=='pub_date' else '')
            url=SEARCH+'?'+urlencode({'query':query,'format':'json','resultType':'core','pageSize':order['limit']})
            try:
                remaining=deadline-time.monotonic()
                if remaining<=0:raise TimeoutError('The 30-second public lookup budget ended; local and received records remain available')
                body,meta=client.json(url,max_bytes=2_000_000,patient=False,timeout=min(8,remaining))
                if 'errMsg' in body or 'error' in body:raise ValueError('Europe PMC rejected the query')
                rows=body['resultList']['result']
                if not isinstance(rows,list) or len(rows)>order['limit']:raise ValueError('Invalid or unbounded source response')
                ids=[]
                for row in rows:
                    paper=paper_from_record(row,meta,plan['taxon_id']);papers[paper['id']]=paper;ids.append(paper['pmid'])
                    if paper['retracted']:retracted.append(paper['id'])
                entry['searches'].append({'order':order['sort'],'source':meta,'hit_count':body.get('hitCount'),
                                          'returned_ids':ids,'query_translation':body.get('request',{}).get('queryString')})
                entry['returned_ids']+= [k for k in ids if k not in entry['returned_ids']]
                if plan['taxon_id']=='REFERENCE_PMIDS':
                    requested={v for v in re.findall(r'EXT_ID:(\d+)',plan['term'])}
                    missing=requested-set(ids)
                    if missing:errors.append({'taxon_id':plan['taxon_id'],'stage':'reference','detail':'Requested identifiers not returned: '+',
    '.join(sorted(missing))})
            except (OSError,RuntimeError,ValueError,KeyError,TypeError) as error:
                errors.append({'taxon_id':plan['taxon_id'],'stage':'search','detail':str(error)[:300]})
    return list(papers.values()),{'provider':'Europe PMC (PubMed-indexed records)',
        'status':'NO_RESOLVED_CANDIDATES' if not plans else 'PARTIAL' if errors and papers else 'UNAVAILABLE' if errors else 'BOUNDED_SEARCH_COMPLETED',
        'queries':queries,'errors':errors,'received_records':len(papers),'excluded_retracted_ids':sorted(set(retracted)),
        'network_used':client.stats.get('requests',0)>before,
        'scope':'At most 12 relevance and 8 publication-date records per candidate, at most three candidates, plus up to ten explicitly supplied PubMed references.
    Exact title/abstract terms; not an exhaustive systematic search. Cached acquisition dates are retained.',
        'sent_fields':['CURATED_PLANT_NAMES','EXACT_PUBLIC_OUTCOME_PHRASES','EXPLICIT_PUBLIC_REFERENCE_IDS'],
        'not_sent':['FREE_TEXT_QUESTION','PRODUCT_SOURCE_TEXT','CONSULTATION_DETAILS','MEDICINES','ALLERGIES']}
```

src/evidence/readings.py
SHA-256: 0662221fe98ee4995b1d994fc54b16e1fe6e66d8c401fb41cbc5e561f1801725

```
"""Selected factual reading notes, kept separate from automated evidence relations."""
from copy import deepcopy


def active_reading_sets(sets, taxon_ids, condition_ids, withdrawn=()):
    taxa, conditions, excluded = set(taxon_ids), set(condition_ids), set(withdrawn)
    result = []
    for item in sets:
        if item['id'] in excluded or not taxa.intersection(item['taxon_ids']) or not conditions.intersection(item['condition_ids']):
            continue
        item = deepcopy(item)
        item['studies'] = [s for s in item['studies'] if not {s['publication']['id'], s['publication'].get('pmcid'),
s['publication']['pmid']}.intersection(excluded)]
        if item['studies']:
            result.append(item)
    return result
```

src/evidence/regulatory.py
SHA-256: 7cba75234205c966ef85b046b79ff6adb0af0bb00eb9bc476eb3d3dfedeb7d7a

```
"""What the EMA herbal committee concluded for a plant and topic.

Shown beside the reading and never fed into it, so the committee's verdict stays an independent
check on the product. Traditional use is a regulatory class resting on long use; it is not a
finding that a plant does nothing.
"""
from __future__ import annotations

import re
from collections import defaultdict

from src.common import DATA, read_json

PATH = DATA / 'ema_herbal.json'
TOPIC_WORDING = {
    'sleep': r'sleep',
    'anxiety': r'mental stress|nervous tension|anxiety|stress',
    'depression': r'depressi|low mood',
    'digestive_symptoms': r'dyspep|digest|gastrointestinal|flatulen|bloating|stomach|irritable bowel|gastric|spasm',
    'diarrhoea': r'diarrh',
    'constipation': r'constipation',
    'nausea_vomiting': r'nausea|motion sickness|vomit',
    'cough': r'cough|expectorant|catarrh',
    'respiratory': r'\bcolds?\b|bronch|respiratory|sinus',
    'oral_throat_care': r'\bmouth\b|throat|\bgums?\b|\boral\b|pharyng',
    'wounds': r'wound|\bulcers?\b',
    'skin': r'\bskin\b|eczema|dermat|\bboils?\b|acne|itch',
    'burns': r'sunburn|\bburns?\b',
    'urinary': r'urinary|prostat|micturition|bladder',
    'kidney_stones': r'renal gravel|\bgravel\b',
    'menstrual': r'menstrua|menopaus|dysmenorrh|climacteric',
    'heart': r'cardiac|\bheart\b',
    'pain': r'\bpain\b|\baches?\b',
    'joint_gout': r'\bjoint|rheumat|arthr',
    'headache': r'headache',
    'haemorrhoids': r'h(?:a)?emorrhoid',
    'liver_jaundice': r'\bliver\b|\bbile\b|hepat',
    'memory_cognition': r'cognitive|dementia|memory',
}
PATTERNS = {topic: re.compile(wording, re.I) for topic, wording in TOPIC_WORDING.items()}


# "Chronic venous insufficiency, which is characterised by itching and pain": the words after
# "which" describe the condition and are not separate indications.
DESCRIPTION = re.compile(r',?\s+(?:which is|characterised by|such as).*$', re.I)
TOO_BROAD = {'passion', 'arctic', 'cherry', 'lime', 'rose', 'mallow', 'birch', 'wild strawberry'}
DRUG_PART = re.compile(r'\s+(?:leaf|leaves|root|herb|flower|fruit|seed|bark|oil|distillate|rhizome|cone berry|stalks|pods?)(?:\s+and\s+flower)?$', re.I)


def topics_in(indication):
    wording = DESCRIPTION.sub('', indication)
    return [topic for topic, pattern in PATTERNS.items() if pattern.search(wording)]


def drug_name(common):
```

```
    """Trials say "senna" or "psyllium" and not the species: "Psyllium seed" gives the word they use."""
    whole = re.sub(r'^(?:fresh|dried)\s+', '', common.split(',')[0].strip(), flags=re.I)
    short = DRUG_PART.sub('', whole)
    return whole if short.casefold() in TOO_BROAD else short


class Regulatory:
    def __init__(self, path=PATH):
        self.available = path.exists()
        self.by_taxon = defaultdict(dict)
        self.drug_names = defaultdict(set)
        if not self.available:
            return
        for plant in read_json(path)['plants']:
            for species in plant.get('species', []):
                if len(drug_name(plant['common'])) >= 5 and 'combination' not in plant['common'].casefold():
                    self.drug_names[species['id']] |= {drug_name(plant['common']), drug_name(plant['common']).replace('-', ' ')}
                for row in plant['indications']:
                    for topic in topics_in(row['indication']):
                        held = self.by_taxon[species['id']].get(topic)
                        # A monograph may hold both classes; the one needing trials is reported.
                        if not held or (row['class'] == 'WELL_ESTABLISHED' and held['class'] != row['class']):
                            self.by_taxon[species['id']][topic] = {'class': row['class'], 'indication': row['indication'], 'url': plant['url'], 'substance':
plant['latin']}

    def conclusion(self, taxon_id, condition_id):
        return self.by_taxon.get(taxon_id, {}).get(condition_id.replace('condition:', ''))


_shared = None


def regulatory():
    global _shared
    if _shared is None:
        _shared = Regulatory()
    return _shared
```

src/graph/__init__.py
SHA-256: f590e8e17b4745215a5719da052910336c321b98cdae2168b041eb796caeafd1

```
    1 """Queryable provenance graph and analysis audit storage."""
```

src/graph/store.py
SHA-256: 4c6a762f9003af69b84c953d14afef4af24c2c481561e11e7782879f95ba2692

```
from __future__ import annotations

import json
import sqlite3
from contextlib import contextmanager
from pathlib import Path
from src.common import ROOT, now, digest


def review_anchors(analysis):
    """Match a judgement's source and question without using ranked row IDs."""
    source = analysis.get('source', {})
    source_key = digest(json.dumps({
        'source': {k: source.get(k) for k in ('id', 'title', 'url', 'date', 'context')},
        'text': analysis.get('input'),
    }, sort_keys=True, ensure_ascii=False))
    anchors = {}
    for group in analysis.get('identity_counterfactuals', []):
        occurrences = tuple(sorted((o['start'], o['end'], o['text']) for o in group['occurrences']))
        for branch in group['branches']:
            target = ('identity', f"{group['key']}::{branch['taxon_id']}")
            anchors[target] = (source_key, 'identity', group['key'], occurrences, branch['taxon_id'], branch['name'])
    for row in analysis.get('evidence', []):
        paper_key = digest(json.dumps({k: row['paper'].get(k) for k in
                            ('id', 'title', 'abstract', 'doi', 'year', 'licence')},
                            sort_keys=True, ensure_ascii=False))
        anchors[('evidence', row['id'])] = (source_key, 'evidence', row['taxon_id'], row['condition_id'], paper_key)
    return anchors


class GraphStore:
    def __init__(self, path=None):
        self.path = path or ROOT / ".cache" / "foliatrace.sqlite"
        self.path.parent.mkdir(parents=True, exist_ok=True)
        with self.connect() as db:
            db.executescript("""
              CREATE TABLE IF NOT EXISTS runs(id TEXT PRIMARY KEY, created_at TEXT NOT NULL, pipeline TEXT NOT NULL, snapshot TEXT NOT NULL, payload TEXT NOT NULL CHECK(json_valid(payload)));
              CREATE TABLE IF NOT EXISTS graph_nodes(run_id TEXT NOT NULL REFERENCES runs(id) ON DELETE CASCADE, id TEXT NOT NULL, kind TEXT NOT NULL, label TEXT NOT NULL, payload TEXT NOT NULL, PRIMARY KEY(run_id,id));
              CREATE TABLE IF NOT EXISTS graph_edges(run_id TEXT NOT NULL REFERENCES runs(id) ON DELETE CASCADE, id TEXT NOT NULL, subject TEXT NOT NULL, predicate TEXT NOT NULL, object TEXT NOT NULL, source_id TEXT NOT NULL, payload TEXT NOT NULL, PRIMARY KEY(run_id,id));
              CREATE INDEX IF NOT EXISTS edge_subject ON graph_edges(run_id,subject,predicate);
              CREATE INDEX IF NOT EXISTS edge_object ON graph_edges(run_id,object,predicate);
              CREATE TABLE IF NOT EXISTS reviews(run_id TEXT NOT NULL REFERENCES runs(id) ON DELETE CASCADE, target_kind TEXT NOT NULL CHECK(target_kind IN ('identity','evidence')), target_id TEXT NOT NULL, decision TEXT NOT NULL CHECK(decision IN ('ACCEPT','REJECT','INSUFFICIENT_INFORMATION')), reason TEXT NOT NULL, reviewer TEXT NOT NULL, created_at TEXT NOT NULL, PRIMARY KEY(run_id,target_kind,target_id));
            """)

    @contextmanager
    def connect(self):
        db = sqlite3.connect(self.path, timeout=20)
        try:
            db.execute("PRAGMA foreign_keys=ON")
            # sqlite3.Connection.__exit__ commits/rolls back but does not close.
```

```
            # Close deterministically so isolated Windows builds can remove the DB.
            with db:
                yield db
        finally:
            db.close()

    def put(self, analysis):
        with self.connect() as db:
            kept = db.execute("SELECT * FROM reviews WHERE run_id=?", (analysis["run_id"],)).fetchall()
            db.execute("DELETE FROM runs WHERE id=?", (analysis["run_id"],))
            db.execute("INSERT INTO runs VALUES(?,?,?,?,?)", (analysis["run_id"], now(), analysis["pipeline_version"], analysis["snapshot_sha256"],
json.dumps(analysis, ensure_ascii=False)))
            for node in analysis["graph"]["nodes"]:
                db.execute("INSERT INTO graph_nodes VALUES(?,?,?,?,?)", (analysis["run_id"], node["id"], node["kind"], node["label"], json.dumps(node,
ensure_ascii=False)))
            ids = {n["id"] for n in analysis["graph"]["nodes"]}
            for edge in analysis["graph"]["edges"]:
                if edge["subject"] not in ids or edge["object"] not in ids or not edge.get("provenance"):
                    raise ValueError("Graph edge lacks referenced object or provenance")
                db.execute("INSERT INTO graph_edges VALUES(?,?,?,?,?,?,?)", (analysis["run_id"], edge["id"], edge["subject"], edge["predicate"], edge["object"],
edge["provenance"]["source_id"], json.dumps(edge, ensure_ascii=False)))
            db.executemany("INSERT INTO reviews VALUES(?,?,?,?,?,?,?)", kept)

    def get(self, run_id):
        with self.connect() as db:
            row = db.execute("SELECT payload FROM runs WHERE id=?", (run_id,)).fetchone()
        return json.loads(row[0]) if row else None

    def put_review(self, run_id, review):
        # Researcher judgements are kept apart from automated output; one current decision per target.
        with self.connect() as db:
            db.execute("INSERT INTO reviews VALUES(?,?,?,?,?,?,?) ON CONFLICT(run_id,target_kind,target_id) DO UPDATE SET decision=excluded.decision,
reason=excluded.reason, reviewer=excluded.reviewer, created_at=excluded.created_at", (run_id, review["target_kind"], review["target_id"], review["decision"],
review["reason"], review["reviewer"], now()))

    def delete_review(self, run_id, target_kind, target_id):
        analysis = self.get(run_id)
        with self.connect() as db:
            # Clear the judgement for this exact source/target across rankings;
            # an older copy must not reappear immediately after removal.
            matches = self._matching_reviews(db, analysis) if analysis else []
            targets = {(row['run_id'], row['target_kind'], row['target_id']) for target, row in matches
                       if target == (target_kind, target_id)}
            targets.add((run_id, target_kind, target_id))
            db.executemany('DELETE FROM reviews WHERE run_id=? AND target_kind=? AND target_id=?', targets)

    def _matching_reviews(self, db, analysis):
        current = review_anchors(analysis)
        targets = {}
        for target, anchor in current.items():
            targets.setdefault(anchor, []).append(target)
        rows = db.execute("""
            SELECT r.run_id,r.target_kind,r.target_id,r.decision,r.reason,r.reviewer,r.created_at,n.payload
            FROM reviews r JOIN runs n ON n.id=r.run_id
            WHERE json_extract(n.payload,'$.source.id')=? AND json_extract(n.payload,'$.input')=?
            ORDER BY r.created_at DESC,r.run_id
        """, (analysis['source']['id'], analysis['input'])).fetchall()
        origins, matches = {}, []
```

```
        for row in rows:
            origin_id, kind, target_id, decision, reason, reviewer, created_at, payload = row
            if origin_id not in origins:
                origins[origin_id] = current if origin_id == analysis['run_id'] else review_anchors(json.loads(payload))
            anchor = origins[origin_id].get((kind, target_id))
            for target in targets.get(anchor, []):
                matches.append((target, {'run_id': origin_id, 'target_kind': kind, 'target_id': target_id,
                                         'decision': decision, 'reason': reason, 'reviewer': reviewer, 'created_at': created_at}))
        return matches

    def reviews(self, run_id, analysis=None):
        analysis = analysis or self.get(run_id)
        if not analysis:
            return []
        with self.connect() as db:
            matches = self._matching_reviews(db, analysis)
        current = {}
        for target, row in matches:
            if target in current:
                continue
            current[target] = {**{k: row[k] for k in ('decision', 'reason', 'reviewer', 'created_at')},
                               'target_kind': target[0], 'target_id': target[1],
                               'origin_run_id': row['run_id'], 'carried_forward': row['run_id'] != run_id,
                               'match_basis': 'EXACT_SOURCE_AND_RESEARCH_TARGET',
                               'kind': 'RESEARCHER_REVIEW_NOT_SOURCE_OR_MODEL_OUTPUT'}
        return sorted(current.values(), key=lambda row: row['created_at'])

    def neighborhood(self, run_id, focus=None, predicate=None, source_id=None, limit=100):
        where, params = ["run_id=?"], [run_id]
        if focus:
            where.append("(subject=? OR object=?)")
            params.extend([focus, focus])
        if predicate:
            where.append("predicate=?")
            params.append(predicate)
        if source_id:
            where.append("source_id=?")
            params.append(source_id)
        with self.connect() as db:
            rows = db.execute("SELECT payload FROM graph_edges WHERE " + " AND ".join(where) + " ORDER BY id LIMIT ?", (*params, limit)).fetchall()
            edges = [json.loads(r[0]) for r in rows]
            ids = {e[x] for e in edges for x in ("subject", "object")}
            nodes = [json.loads(r[0]) for r in db.execute("SELECT payload FROM graph_nodes WHERE run_id=?", (run_id,)) if json.loads(r[0])["id"] in ids]
        return {"nodes": nodes, "edges": edges}
```

src/ingestion/__init__.py
SHA-256: f944fab86becdbfd5adbabfba67a7195a1467e6f4a0e5dea2f59fbb1dddfbafa

```
    1 """Source acquisition, rights and snapshot manifests."""
```

src/ingestion/client.py
SHA-256: 2e1e5a74dcc40238d81b5d09f3e9b4916e3f24427da92e691557189b478bd0b9

```
from __future__ import annotations

import json
import time
import threading
from pathlib import Path
from urllib.parse import urlparse
from urllib.request import Request, urlopen
from urllib.error import HTTPError, URLError

from src.common import ROOT, digest, now, read_json, write_json
from src.ingestion.tls import tls_context


class ScientificClient:
    """Bounded, persistent GET cache. Cache hits make offline reruns deterministic."""
    _throttle_lock = threading.Lock()
    _last_by_host = {}

    def __init__(self, cache: Path | None = None, offline=False, interval=0.4):
        self.cache = cache or ROOT / ".cache" / "scientific"
        self.cache.mkdir(parents=True, exist_ok=True)
        self.offline, self.interval = offline, interval
        self.last_request = 0.0
        self.stats = {"hits": 0, "requests": 0, "errors": 0}

    def get(self, url: str, *, max_bytes=10_000_000, patient=True, timeout=35) -> tuple[str, dict]:
        allowed = {"www.gutenberg.org", "www.ebi.ac.uk", "api.gbif.org", "pubchem.ncbi.nlm.nih.gov", "huggingface.co", "api.crossref.org", "www.wikidata.org",
   "verifier.globalnames.org", "query.wikidata.org", "en.wikipedia.org", "es.wikipedia.org", "eutils.ncbi.nlm.nih.gov", "api.ods.od.nih.gov"}
        parsed = urlparse(url)
        if parsed.scheme != "https" or parsed.hostname not in allowed:
            raise ValueError("Source URL is not in the scientific acquisition allowlist")
        path = self.cache / f"{digest(url)}.json"
        if path.exists():
            record = read_json(path)
            if digest(record["body"]) != record["sha256"]:
                raise ValueError("Cached response checksum mismatch")
            self.stats["hits"] += 1
            return record["body"], record["metadata"]
        if self.offline:
            raise RuntimeError(f"Offline cache miss: {url}")
        for attempt in range(3):
            with self._throttle_lock:
                last = self._last_by_host.get(parsed.hostname,0)
                time.sleep(max(0,self.interval-(time.monotonic()-last)))
                self._last_by_host[parsed.hostname]=time.monotonic()
            self.stats["requests"] += 1
            try:
                request = Request(url, headers={"User-Agent": "FoliaTrace/0.5 (noncommercial research prototype; bounded requests)", "Accept": "application/sparql-
   results+json, application/json, */*"})
                with urlopen(request, timeout=timeout, context=tls_context()) as response:
                    final = urlparse(response.url)
                    if final.scheme != "https" or final.hostname not in allowed:
                        raise ValueError("Redirect outside scientific source allowlist")
                    raw = response.read(max_bytes + 1)
```

```
                    if len(raw) > max_bytes:
                        raise ValueError("Response exceeds acquisition bound")
                    text = raw.decode("utf-8-sig")
                metadata = {"url": url, "retrieved_at": now(), "sha256": digest(text), "bytes": len(raw)}
                write_json(path, {"body": text, "sha256": digest(text), "metadata": metadata})
                return text, metadata
            except (URLError, TimeoutError) as exc:
                retry_after = exc.headers.get("retry-after", "") if isinstance(exc, HTTPError) else ""
                if retry_after.isdigit() and int(retry_after) > 60:
                    raise RuntimeError("Server requested long retry delay; acquisition stopped") from exc
                if isinstance(exc, HTTPError) and exc.code < 500 and exc.code != 429:
                    self.stats["errors"] += 1
                    raise
                # A reader is waiting: a rate limit ends this route instead of holding the page.
                if attempt == 2 or not patient:
                    self.stats["errors"] += 1
                    raise
                time.sleep(min(int(retry_after) if 'retry_after' in locals() and retry_after.isdigit() else 2 ** attempt, 60))
        raise RuntimeError("Acquisition attempts exhausted")

    def json(self, url, **kwargs):
        text, meta = self.get(url, **kwargs)
        return json.loads(text), meta
```

src/ingestion/pubmed.py
SHA-256: 78e17de16ddd7c78c32b63729933b5c03fdd7182e619184f287924b8da404cbc

```
"""PubMed access shared by the literature snapshot and on-demand lookups for new plants."""
from __future__ import annotations

import re
import xml.etree.ElementTree as ET
from urllib.parse import urlencode

from src.nlp.extract import SENSE_REQUIRED, NEVER_A_PLANT, normalize

ESEARCH = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi'
EFETCH = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi'
HUMAN = ' AND ("randomized controlled trial"[pt] OR "clinical trial"[pt] OR "meta-analysis"[pt] OR "systematic review"[pt])'
PRECLINICAL = ' AND ("animals"[mh:noexp] OR "in vitro techniques"[mh]) NOT "humans"[mh:noexp]'
REVIEW = ' AND "review"[pt] NOT "systematic review"[pt]'


def tiers():
    return [('human', HUMAN, 24), ('review', REVIEW, 5), ('preclinical', PRECLINICAL, 8), ('any', '', 6)]


def name_clause(taxon):
    """Scientific names, plus a distinctive common name when it cannot mean something else."""
    names = [taxon['name']]
    names += [a['name'] for a in taxon['aliases'] if a['kind'] == 'SYNONYM' and re.fullmatch(r'[A-Z][a-z-]+ [a-z-]+', a['name'])][:3]
    common = [a['name'] for a in taxon['aliases'] if a['kind'] == 'VERNACULAR' and a.get('lang', 'en') == 'en']
    usable = [n for n in common if normalize(n) not in SENSE_REQUIRED and normalize(n) not in NEVER_A_PLANT and (' ' in n or len(n) >= 7) and n.isascii()]
    names += taxon.get('drug_names', [])[:2] + sorted(usable, key=len)[:2]
    return ' OR '.join(f'"{n}"[tiab]' for n in dict.fromkeys(names))


def text_of(node):
    return ' '.join(''.join(node.itertext()).split()) if node is not None else ''


def parse_articles(xml_text):
    rows = []
    for item in ET.fromstring(xml_text).findall('PubmedArticle'):
        citation = item.find('MedlineCitation')
        article = citation.find('Article')
        pmid = citation.findtext('PMID')
        abstract = ' '.join(((node.get('Label') + ': ') if node.get('Label') else '') + text_of(node) for node in article.findall('Abstract/AbstractText'))
        types = [text_of(t) for t in article.findall('PublicationTypeList/PublicationType')]
        ids = {i.get('IdType'): i.text for i in item.findall('PubmedData/ArticleIdList/ArticleId')}
        year = citation.findtext('.//PubDate/Year') or (re.search(r'\d{4}', citation.findtext('.//PubDate/MedlineDate') or '') or [None])[0] if article is not None
else None
        authors = ', '.join(f"{a.findtext('LastName')} {a.findtext('Initials') or ''}".strip() for a in article.findall('AuthorList/Author') if
a.findtext('LastName'))[:300]
        rows.append({'pmid_key': pmid, 'id': f'PMID{pmid}', 'pmid': pmid, 'doi': ids.get('doi'), 'pmcid': ids.get('pmc'), 'title':
text_of(article.find('ArticleTitle')), 'original_title': text_of(article.find('ArticleTitle')), 'abstract': abstract, 'year': year, 'authors': authors,
'publication_types': types, 'licence': 'PUBMED_RECORD_RIGHTS_WITH_PUBLISHER', 'source_id': 'pubmed', 'url': f'https://pubmed.ncbi.nlm.nih.gov/{pmid}/', 'provenance':
 {'url': EFETCH + '?' + urlencode({'db': 'pubmed', 'id': pmid})}, 'retracted': any('Retract' in t for t in types), 'abstract_rights':
'LOCAL_CACHE_NOT_REDISTRIBUTED'})
    return rows


```

```
def fetch_records(client, pmids):
    if not pmids:
        return []
    url = EFETCH + '?' + urlencode({'db': 'pubmed', 'id': ','.join(pmids), 'retmode': 'xml'})
    text, _ = client.get(url, max_bytes=60_000_000)
    return parse_articles(text)
```

src/ingestion/tls.py
SHA-256: adc3bf5126a129453ed1acf8a6d6dcafd891684c5ad9fbee5f13c72a7d61677b

```
"""Use operating-system trust stores on Windows; certificate verification stays enabled."""
import os
import ssl


def tls_context():
    if os.name == "nt":
        import truststore
        return truststore.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    return ssl.create_default_context()
```

src/nlp/__init__.py
SHA-256: e7f63367e07cd82ffadab54ce2e14fc21ec9246e7ebcd16df61961eba90508fe

```
    1 from .extract import Extractor, normalize
    2
    3 __all__ = ["Extractor", "normalize"]
```

src/nlp/corpus.py
SHA-256: 0ac04df616e5bea7aa2f9423c5c237cf2c13189e9d8d08a0cd806e2c729c4148

```
"""Reusable knowledge records mined from historical prose, retaining exact spans."""
from collections import Counter
from src.common import now


def mine_corpus(pipeline, previous=None):
    rows, collections = [], Counter()
    withdrawn = pipeline.withdrawn()
    for passage in pipeline.passages + pipeline.extra_passages:
        if passage['id'] in withdrawn or passage['source_id'] in withdrawn:
            continue
        text = passage['title'] + '\n\n' + passage['text']
        extracted = pipeline.extractor.extract(text, section_subject=True, language='en')
        mentions = {m['id']: m for m in extracted['mentions']}
        collection = passage.get('collection') or 'Culpeper 1850'
        collections[collection] += 1
        for claim in extracted['historical_claims']:
            names = [mentions[m] for m in claim['mention_ids']]
            rows.append({'id': passage['id'] + ':' + claim['id'], 'source_id': passage['id'], 'collection': collection, 'source_title': passage['title'], 'date': passage['date'], 'url': passage['url'], 'licence': passage['licence'], 'statement': claim['support'], 'names': [{'text': m.get('heading_name') or m['text'], 'method': m['method'], 'start': m['start'], 'end': m['end']} for m in names], 'conditions': claim['conditions'], 'parts': claim['part'], 'preparations': claim['preparation'], 'routes': claim['route'], 'scope': claim['relation_scope'], 'negated': claim['negated'], 'human_reviewed': False})
            rows[-1]['statement_role'] = claim['statement_role']
            rows[-1]['role_method'] = claim['role_method']
            rows[-1]['dosage_expressions'] = claim['dosage_expressions']
            rows[-1]['practice'] = claim['practice']
            rows[-1]['possible_subjects'] = claim['possible_subjects']
    migrations = {}
    if previous:
        def anchor(row):
            return (row['source_id'], row['collection'], row['date'], row['url'], row['statement']['start'], row['statement']['end'], row['statement']['text'])
        active = {anchor(row): row['id'] for row in rows}
        id_map = {row['id']: active[anchor(row)] for row in previous['records'] if anchor(row) in active}
        migrations[previous['snapshot_sha256']] = id_map
        for migration in previous.get('reading_migrations', []):
            migrations[migration['snapshot_sha256']] = {old_id: id_map[prior_id] for old_id, prior_id in migration['id_map'].items() if prior_id in id_map}
    return {'created_at': now(), 'snapshot_sha256': pipeline.snapshot, 'code_fingerprint': pipeline.code_fingerprint, 'scope': 'Automatically extracted historical statements, including uses, adverse effects and context. Condition mappings are lexical research aids, not diagnostic equivalence. Counts do not establish extraction accuracy, efficacy or historical botanical identity.', 'collections': dict(collections), 'records': rows, 'conditions': dict(Counter(c['id'] for r in rows for c in r['conditions'])), 'roles': dict(Counter(r['statement_role'] for r in rows)), 'extraction_method': pipeline.extractor.version, 'reading_migrations': [{'snapshot_sha256': snapshot, 'id_map': mapping} for snapshot, mapping in migrations.items()]}
```

src/nlp/extract.py
SHA-256: 5281a5ec8ecf4b99ee81490d3e8a1417a857ebe6bfeecc9b5c0fc943c4b9cd01

```
from __future__ import annotations

import bisect
import json
import re
import unicodedata
from functools import lru_cache
from pathlib import Path

from rapidfuzz import fuzz, process

from src.nlp.headings import heading_names


@lru_cache(maxsize=200_000)
def normalize(text: str) -> str:
    text = unicodedata.normalize("NFKD", text).casefold().replace("\xe6", "ae").replace("\u0153", "oe").replace("\xd7", " x ")
    return " ".join(re.sub(r"[^\w]+", " ", "".join(c for c in text if not unicodedata.combining(c))).split())


@lru_cache(maxsize=256)
def topic_pattern(terms):
    parts = []
    for term in terms:
        word = normalize(term)
        tail = r'(?!\s+(?:capacity|capacities|ratio|ratios|behaviour|behavior|kinetics)\b)' if word in ('swelling', 'swellings') else ''
        parts.append(re.escape(word) + tail)
    return re.compile(r'\b(?:' + '|'.join(parts) + r')\b')


def has_research_topic_wording(text, terms):
    """Modern-paper lexical screening; material swelling is not symptom wording.

    Other occurrences remain eligible, and this does not classify diagnoses.
    Historical extraction retains the original words and its separate rules.
    """
    return bool(topic_pattern(tuple(terms)).search(normalize(text)))


def normalized_offsets(text):
    chars, mapping = [], []
    for index, char in enumerate(text):
        value = normalize(char)
        if not value:
            value = " "
        for c in value:
            if c == " " and (not chars or chars[-1] == " "):
                continue
            chars.append(c)
            mapping.append(index)
    return "".join(chars), mapping


# Transparent historical/modern lexical hypotheses. No historical term is declared
# medically equivalent; the mapping is a retrieval aid and is labelled as such.
```

```
CONDITIONS = {
    "headache": ["headache", "head-ache", "head ache", "headaches", "head-ach", "headach", "pains in the head", "pain in the head", "griefs and pains of the head"],
    "migraine": ["migraine", "migraines", "megrim", "hemicrania"],
    "sleep": ["sleep", "sleeping", "insomnia", "sleeplessness", "sleepless", "wakefulness", "provokes sleep", "procures sleep", "cause sleep", "causes sleep", "induce sleep", "induces sleep", "promotes sleep", "to sleep", "poor sleep", "sleep problems", "trouble sleeping"],
    "anxiety": ["anxiety", "anxious", "nervous excitement", "restlessness", "troublesome cares", "wearisomnesse", "weariness"],
    "depression": ["depression", "depressive", "melancholy", "melancholia"],
    "digestive_symptoms": ["digestion", "indigestion", "stomach", "stomacke", "colic", "colick", "cholic", "bowels", "irritable bowel", "dyspepsia", "flatulence", "flatulent", "wind", "griping", "torments of the belly", "heartburn"],
    "diarrhoea": ["diarrhoea", "diarrhea", "diarrhoeas", "flux", "fluxes", "bloody flux", "dysentery", "looseness of the bowels", "lask", "cholera"],
    "constipation": ["constipation", "costiveness", "costive", "opens the belly", "loosens the belly", "purgative", "cathartic", "aperient", "laxative"],
    "nausea_vomiting": ["vomiting", "nausea", "sea-sickness", "seasickness", "provocation to vomit"],
    "pain": ["pain", "pains", "paine", "paines", "aches", "ache", "stitches", "griefs", "bruises"],
    "joint_gout": ["gout", "goute", "gowt", "sciatica", "rheumatism", "rheumatic", "arthritis", "joint-pains", "pains in the joints", "hip-gout", "lumbago"],
    "inflammation": ["inflammation", "inflammations", "swelling", "swellings", "tumours"],
    "fever": ["fever", "fevers", "feuers", "feauers", "ague", "agues", "febrifuge", "tertian", "quartan", "malaria", "malarial", "diaphoretic", "sudorific"],
    "cough": ["cough", "coughs", "coughes", "cough-mixture", "whooping cough", "hoarseness", "expectorant"],
    "respiratory": ["asthma", "shortness of breath", "cold", "influenza", "shortwinded", "wheezing", "wheezings", "consumption", "phthisis", "catarrh", "catarrhal", "colds", "a cold", "bronchitis", "bronchial", "stuffing of the chest", "croup", "pleurisy", "lungs", "pneumonia", "tuberculosis"],
    "wounds": ["wound", "wounds", "ulcer", "ulcers", "sore", "sores", "cuts", "cankers", "abrasions"],
    "burns": ["burn", "burns", "burnings", "scald", "scalds", "scalding", "scaldings"],
    "skin": ["itch", "scab", "scabs", "eczema", "tetter", "tetters", "ringworm", "freckles", "erysipelas", "saint anthony's fire", "st. anthony's fire", "scurf", "leprosy", "boils", "warts", "chilblains", "pimples", "morphew", "rash", "rashes", "skin diseases", "skin complaints", "dermatitis", "psoriasis", "acne"],
    "bleeding": ["bleeding", "stanch", "stanches", "staunch", "spitting of blood", "nose-bleed", "bleeding at the nose", "hemorrhage", "haemorrhage", "fluxes of blood", "bloody"],
    "oral_throat_care": ["gargle", "gargles", "gargled", "gargling", "sore mouth", "sore mouths", "sore throat", "sore throats", "mouthwash", "mouthwashes", "gingival", "gingivitis", "dental plaque", "periodontal", "periodontitis", "quinsy"],
    "toothache": ["toothache", "tooth-ache", "aching teeth", "pain of the teeth", "pains of the teeth", "carious tooth"],
    "eye": ["dimness of the sight", "dimness of sight", "sore eyes", "inflamed eyes", "ophthalmia", "eye-wash", "weak eyes", "the eyes", "pin or web"],
    "ear": ["earache", "ear-ache", "deafness", "pains in the ears", "noise in the ears"],
    "epilepsy": ["epilepsy", "falling sickness", "falling-sickness", "convulsions", "fits"],
    "nervous": ["palsy", "palsies", "hysteria", "hysterical", "cramps", "cramp", "apoplexy", "swoonings", "faintings", "nervous"],
    "liver_jaundice": ["jaundice", "liver", "livers", "liver complaints", "bile", "bilious", "gall-stones", "gallstones", "biliary", "hepatitis"],
    "urinary": ["provokes urine", "provoke urine", "diuretic", "strangury", "stoppage of urine", "bladder", "reins", "urinary", "urethritis", "cystitis", "dysuria"],
    "dropsy_oedema": ["dropsy", "dropsies", "dropsical"],
    "memory_cognition": ["memory", "the senses", "the wits", "weak brain", "forgetfulness", "dementia"],
    "kidney_stones": ["the stone", "stone in the kidneys", "stone in the bladder", "gravel", "kidneys", "kidney stone", "kidney stones", "kidney-stones", "renal calculi", "urinary calculi", "bladder stones"],
    "menstrual": ["courses", "menses", "menstruation", "menstrual", "the mother", "monthly", "suppressed periods", "emmenagogue", "amenorrhoea", "dysmenorrhoea"],
    "childbirth_lactation": ["childbirth", "after-birth", "afterbirth", "travail", "lactation", "nursing mothers", "delivery of women"],
    "parasites": ["worms", "wormes", "worm", "tapeworm", "vermin", "vermifuge", "anthelmintic"],
    "heart": ["palpitation", "palpitations", "heart disease", "the heart"],
    "poison_bites": ["bites", "stings", "venomous", "venom", "poison", "poisons", "snake-bite", "snake-bites", "snakebite", "snakebites", "biting of serpents", "biting of adders", "stinging of", "serpents"],
    "scurvy": ["scurvy", "scorbutic"],
    "haemorrhoids": ["piles", "haemorrhoids", "hemorrhoids"],
    "plague_infection": ["plague", "pestilence", "infection", "contagion", "scarlet fever", "putrid"],
    "cancer": ["cancer", "cancerous"],
    "diabetes": ["diabetes", "diabetic"],
    "venereal": ["syphilis", "syphilitic", "antisyphilitic", "gonorrhoea", "gonorrhea", "venereal", "leucorrhoea", "leucorrhea", "the whites"],
    "hypertension": ["hypertension", "high blood pressure", "blood pressure"],
    "anaemia": ["anaemia", "anemia", "chlorosis"],
}

# What a modern paper calls the same retrieval topic. Kept apart from historical wording:
# "ague" or "the mother" never occur in a trial, and "stomach" in a trial is not a symptom.
RESEARCH_TERMS = {
    "headache": ["headache", "headaches", "tension-type headache", "migraine", "migraines"],
```

```
    "migraine": ["migraine", "migraines"],
    "sleep": ["insomnia", "sleep quality", "sleep disorder", "sleep disorders", "sleep", "sleeping"],
    "anxiety": ["anxiety", "anxiolytic", "perceived stress", "psychological stress"],
    "depression": ["depression", "depressive", "antidepressant"],
    "digestive_symptoms": ["dyspepsia", "irritable bowel", "functional gastrointestinal", "gastrointestinal symptoms", "digestion", "indigestion", "bloating",
"flatulence", "heartburn", "gastritis", "peptic ulcer", "colic", "abdominal pain"],
    "diarrhoea": ["diarrhea", "diarrhoea", "dysentery"],
    "constipation": ["constipation", "laxative"],
    "nausea_vomiting": ["nausea", "vomiting", "emesis", "antiemetic"],
    "pain": ["pain", "analgesic", "analgesia", "antinociceptive", "nociception"],
    "joint_gout": ["gout", "arthritis", "osteoarthritis", "rheumatoid", "sciatica", "low back pain", "joint pain"],
    "inflammation": ["inflammation", "inflammatory", "anti-inflammatory", "swelling", "swellings", "edema"],
    "fever": ["fever", "antipyretic", "malaria", "febrile"],
    "cough": ["cough", "antitussive", "pertussis"],
    "respiratory": ["asthma", "bronchitis", "respiratory", "common cold", "upper respiratory", "influenza", "sinusitis", "tuberculosis", "copd"],
    "wounds": ["wound", "wounds", "wound healing", "ulcer", "ulcers"],
    "burns": ["burn", "burns", "burn wound"],
    "skin": ["dermatitis", "eczema", "psoriasis", "acne", "skin", "warts", "erysipelas"],
    "bleeding": ["bleeding", "hemorrhage", "haemorrhage", "hemostatic", "haemostatic", "menorrhagia", "epistaxis"],
    "oral_throat_care": ["gingivitis", "periodontal", "periodontitis", "dental plaque", "mouthwash", "mouthwashes", "mouth rinse", "sore throat", "pharyngitis",
"tonsillitis", "oral mucositis", "stomatitis", "aphthous", "gingival", "plaque"],
    "toothache": ["toothache", "dental pain", "tooth pain"],
    "eye": ["eye", "eyes", "ocular", "conjunctivitis", "ophthalmic", "blepharitis"],
    "ear": ["otitis", "earache", "tinnitus", "hearing"],
    "epilepsy": ["epilepsy", "seizure", "seizures", "anticonvulsant"],
    "nervous": ["spasm", "spasmolytic", "antispasmodic", "stroke", "neuroprotective", "muscle cramp"],
    "liver_jaundice": ["hepatoprotective", "liver disease", "liver injury", "liver fibrosis", "fatty liver", "hepatic steatosis", "cirrhosis", "jaundice",
"hepatitis", "gallstones", "biliary", "cholestasis"],
    "urinary": ["diuretic", "diuresis", "urinary", "cystitis", "lower urinary tract", "prostatic hyperplasia"],
    "dropsy_oedema": ["edema", "oedema", "fluid retention", "ascites", "heart failure"],
    "memory_cognition": ["memory", "cognition", "cognitive", "dementia", "alzheimer"],
    "kidney_stones": ["kidney stone", "kidney stones", "nephrolithiasis", "urolithiasis", "renal"],
    "menstrual": ["dysmenorrhea", "dysmenorrhoea", "menstrual", "premenstrual", "amenorrhea", "menopause", "menopausal", "hot flashes", "hot flushes",
"climacteric"],
    "childbirth_lactation": ["labor", "labour", "lactation", "breast milk", "postpartum", "childbirth"],
    "parasites": ["anthelmintic", "helminth", "worm", "worms", "nematode", "antiparasitic"],
    "heart": ["heart", "cardiac", "cardiovascular", "cardioprotective", "arrhythmia", "hypertension", "blood pressure", "angina"],
    "poison_bites": ["antivenom", "snake bite", "snakebite", "antidote", "insect bite", "venom"],
    "scurvy": ["scurvy", "vitamin c", "ascorbic"],
    "haemorrhoids": ["hemorrhoids", "haemorrhoids", "hemorrhoidal"],
    "plague_infection": ["antimicrobial", "antibacterial", "antiviral", "infection", "infections"],
    "cancer": ["cancer", "tumor", "tumour", "carcinoma", "antitumor", "anticancer"],
    "diabetes": ["diabetes", "diabetic", "glycemic", "glycaemic", "hyperglycemia", "blood glucose", "insulin resistance"],
    "venereal": ["syphilis", "gonorrhea", "gonorrhoea", "leucorrhea", "leucorrhoea", "vaginitis", "urethritis"],
    "hypertension": ["hypertension", "blood pressure"],
    "anaemia": ["anemia", "anaemia", "hemoglobin", "haemoglobin"],
}


def research_terms(condition):
    key = condition.replace("condition:", "")
    return RESEARCH_TERMS.get(key) or CONDITIONS.get(key) or [key.replace("_", " ")]


PARTS = ["roots", "root", "leaves", "leaf", "flowers", "flower", "bark", "seeds", "seed", "berries", "berry", "fruit", "fruits", "tops", "stalk", "stalks", "petals",
 "bulb", "bulbs", "tubers", "peel", "peels", "rind", "rinds", "gum", "resin", "whole plant", "herb"]
PREPARATIONS = ["decoction", "infusion", "distilled water", "powder", "powdered", "ointment", "essential oil", "oil", "syrup", "tea", "boiled", "bruised", "dried",
"extract", "juice", "tincture", "poultice", "liniment", "lozenge", "lozenges", "conserve", "gargle", "gargles", "lotion", "vinegar", "draught", "plaster", "salve",
```

```
    "fomentation", "electuary", "bath", "clyster", "posset", "cordial", "essence", "wine", "ale", "broth", "jujubes", "balls", "honey", "steeped", "macerated", "wash"]
ROUTES = ["gargle", "gargles", "gargled", "gargling", "outwardly", "inwardly", "applied", "anoint", "anointed", "drank", "drunk", "drink", "swallowed", "worn",
    "bathed", "chewed", "inhaled", "taken"]

# English words that are also plant names. Lower-case use needs a botanical neighbour.
SENSE_REQUIRED = {"date", "dates", "rue", "sage", "balm", "mint", "lime", "pink", "flag", "dock", "rush", "rushes", "rose", "roses", "ash", "bay", "box", "elder",
    "may", "mace", "orange", "violet", "olive", "mustard", "pepper", "broom", "reed", "nut", "plum", "cane", "gum", "tea", "coffee", "corn", "wheat", "oak", "pine",
    "fern", "moss", "grass", "weed", "wort", "bean", "pea", "hop", "hops", "fig", "vine", "cotton", "thrift", "heather", "sorrel", "savory", "hemp", "aloe", "lily",
    "iris", "reeds", "melon", "lemon", "apple", "pear", "peach", "cherry", "rice", "barley", "oats"}
NEVER_A_PLANT = {"drop", "drops", "common", "tree", "plant", "herb", "flower", "wild", "bitter", "sweet", "water", "fire", "sun", "moon", "bread", "butter", "milk",
    "lady", "love", "wood", "field", "stone", "gold", "silver", "star", "bell", "bloom", "blood", "bird", "fish", "cat", "dog", "horse", "bear", "mouse", "rat", "snake",
     "wolf", "goat", "sheep", "pig", "cow", "bull", "lion", "fox", "wool", "soap", "needle", "thorn", "spring", "summer", "winter", "autumn", "cure", "blue", "red",
    "white", "black", "green", "yellow", "purple", "great", "little", "small", "large", "true", "false", "good", "bad", "old", "new", "medicine", "health", "heal",
    "healing", "poison", "grace", "hope", "joy", "life", "death", "night", "day", "mother", "father", "king", "queen", "monk", "bishop", "pope", "cross", "church",
    "dropsy", "colic", "cough", "fever", "gout", "worm", "worms", "staff", "bath", "wine", "beer", "oil", "gall", "salt", "sugar", "honey", "pitch", "tar", "resin",
    "bark", "root", "seed", "leaf", "stem", "fruit", "berry", "nuts", "sand", "clay", "mud", "rock", "hill", "beach", "river", "lake", "marsh", "mountain", "smaller",
    "larger", "medicinal plant", "medicinal plants"}
BOTANICAL_NEIGHBOUR = re.compile(r"\b(?:leaf|leaves|root|roots|oil|tree|trees|juice|flowers?|seeds?|bark|peels?|rinds?|herbs?|plants?|decoction|infusion|syrup|berrie
    s|fruit|ointment|tincture|tops|stalks?|boiled|bruised|gargle|tea|essence|powder|mixed|together)\b", re.I)
GROWING_OR_COOKING = re.compile(r'\b(?:grow(?:s|ing|n)?|garden|cultivat\w*|harvest\w*|soup|stew|salad|recipe|cooking)\b', re.I)
PERSON_PLANT_NAMES = {'basil', 'rosemary', 'rose', 'heather', 'sage', 'olive', 'violet', 'iris', 'lily', 'holly', 'jasmine', 'daisy', 'willow', 'hazel', 'fern',
    'poppy', 'rue'}
PERSON_ROLE =
    r'(?:friend|neighbou?r|colleague|sister|brother|mother|father|daughter|son|wife|husband|aunt|uncle|cousin|nurse|doctor|teacher|student|patient|researcher)'
PLACE_AFTER = re.compile(r'^[-\s]+(?:county|city|province|township|district|street|avenue|road|valley|island|hill|mountain|river)\b', re.I)
PLACE_BEFORE = re.compile(r'\b(?:city|county|province|township|district|island|valley)\s+of\s+(?:the\s+)?$', re.I)
PLACE_PREFIXES = {'san', 'santa', 'santo', 'saint', 'st'}


def person_name_context(before, after):
    """Local grammatical roles outrank nearby tea or symptoms in another phrase."""
    if re.search(r'\b(?:mr|mrs|ms|dr|prof)\.?\s*$|\b(?:told|called|asked|thanked|met|emailed|visited)\s*$|\b(?:my|her|his|our|your)\s+' + PERSON_ROLE + r'\s*$',
    before, re.I):
        return True
    if re.match(r"['\u2019]s\s+(?:" + PERSON_ROLE + r"|headache|cough|appointment|blood pressure|" + '|'.join(sorted(PERSON_PLANT_NAMES)) + r")\b", after, re.I):
        return True
    if re.match(r'\s*,?\s*(?:(?:is|was)\s+)?(?:my|her|his|our|your|an?)\s+' + PERSON_ROLE + r'\b', after, re.I):
        return True
    # A coordinated pair can share the same human predicate.
    predicate = re.sub(r'^\s+and\s+[A-Z][a-z]+\b', '', after)
    if re.match(r'\s+(?:told|called|asked|said|wrote|emailed|visited|attended|discussed|spoke|walked|thanked|replied|reported|explained|studies|studied|lives|lived|r
    ecommends|recommended|slept|arrived|went|and\s+(?:I|he|she|they)\b)', predicate, re.I):
        return True
    if re.match(r'\s+(?:works?|worked)\s+(?:at\s+(?:the\s+)?(?:hospital|school|university|company|shop)|as\s+an?\s+' + PERSON_ROLE + r')\b', predicate, re.I):
        return True
    if re.match(r'\s+(?:gave|gives)\s+(?:me|you|him|her|us|them|my\s+' + PERSON_ROLE + r'|' + '|'.join(sorted(PERSON_PLANT_NAMES)) + r')\b', predicate, re.I):
        return True
    if re.match(r'\s+(?:had|has)\s+(?:trouble sleeping|a headache|an appointment|a
    meeting)\b|\s+(?:was|is)\s+(?:diagnosed|admitted|hospitali[sz]ed)\b|\s+(?:was|is)\s+prescribed\s+(?:medicine|medication|tablets|antibiotics|an?\s+drug)\b', after,
    re.I):
        return True
    # "gave Rosemary a cup" identifies a recipient, not a botanical ingredient.
    return bool(re.search(r'\b(?:gave|give|given|offered|brought|sent)\s*$', before, re.I) and re.match(r'\s+(?:an?|some|her|his|their)\b', after, re.I))
NON_BOTANICAL_SENSE = re.compile(r"\b(?:(?:malate|carbonate|phosphate|oxalate|salts?|muriate|sulphate|citrate|chloride|nitrate|acetate|tannate) of lime|cotton[-
    ]wool|adulterating pepper|lime[- ]water|the elder (?:pliny|cato|brother|son)|elder (?:brother|sister|son|daughter)|(?:rose|rises?)
    (?:up|from|to|above)|(?:he|she|it|they|which|who) rose|(?:that|this|what|recent|early|later|elizabethan|ancient|modern|expected|same|late|earlier) date|to date|up to
     date|of (?:a )?date|in dock|sovereign balm|like balm|in a box|pink[- ](?:white|tinted|tinged)|tinged with pink|french flag|like a flag)\b", re.I)
CUE_STOP = {'mix', 'take', 'boil', 'steep', 'leaves', 'leaf', 'root', 'roots', 'flower', 'flowers', 'seeds', 'seed', 'bark', 'herb', 'plant', 'fruit', 'fruits',
```

```
    'berries', 'syrup', 'juice', 'decoction', 'infusion', 'tincture', 'powder', 'extract', 'plaster', 'plasters', 'ointment', 'poultice', 'headache', 'cough', 'fever',
    'pain', 'pains', 'water', 'herbal', 'this', 'that', 'these', 'those', 'their', 'strong', 'warm', 'cold', 'fresh', 'dried', 'boiled', 'common', 'plant', 'plants',
    'such', 'other', 'some', 'many', 'most', 'each', 'both', 'which', 'large', 'whole', 'wine', 'honey', 'milk', 'sugar', 'salt', 'traditional', 'medicinal', 'local',
    'garden', 'essential', 'green', 'dry', 'young', 'first', 'same', 'itself', 'much', 'several', 'various', 'certain', 'ancient', 'modern', 'people', 'children',
    'women', 'patients', 'informants', 'healers', 'villagers', 'cottagers', 'often', 'also', 'sometimes', 'usually', 'always', 'never', 'there', 'where', 'while',
    'being', 'having', 'made', 'taken', 'given', 'applied', 'used', 'drunk', 'eaten', 'hard', 'soft', 'hairy', 'narrow', 'broad', 'tall', 'long', 'short', 'bruised',
    'powdered', 'infused', 'macerated', 'crushed', 'pounded', 'chewed'}
CUE_NAME = [re.compile(r"\b([A-Z][a-
z]{3,})\s+(?:is|are|was|were)\s+(?:\w+\s+)?(?:applied|used|taken|drunk|given|eaten|chewed|rubbed|bound|boiled|prescribed|employed|administered|smoked|inhaled|steeped
|carried|worn|hung)\b"), re.compile(r"(?<![^\W\d_])([^\W\d_]{4,})\s+(?:decoction|infusion|tincture|tea|root|roots|leaves|leaf|flowers|bark|seeds|paste|juice|ointment
|poultice|extract|syrup|powder|tops|fruit|berries|salve|gel|capsules)\b"),
re.compile(r"\b(?:uses?|used|gives?|gave|drinks?|drank|takes?|took|applies|apply|chews?|prescribes?)\s+((?:[^\W\d_]+\s+){0,2}[^\W\d_]{4,})\s+(?:for|against|to
treat|to relieve)\b"), re.compile(r"\b(?:decoction|infusion|tincture|tea|juice|leaves|root|roots|flowers|bark|seeds|sprig|sprigs|bunch|handful|bundle|fruits?)\s+of\s
+(?:the\s+)?([A-Za-z]{4,})\b")]
# Texts name a plant outright: "commonly known as Yerba Santa", a line opening "American Elm.", a
# closing "(American Mint)". As with every position, GBIF decides whether it is a plant at all.
CUE_NAME += [re.compile(r"\b(?:known|called|named)(?: as)?\s+(?:the\s+)?([A-Z][a-z]+(?:[ -][A-Z][a-z]+|-[a-z]+){0,2})"), re.compile(r"^[ \t]*([A-Z][a-z]+(?:
[A-Z][a-z-]+){1,2})\.(?=\s)", re.M), re.compile(r"\(([A-Z][a-z]+(?: [A-Z][a-z-]+){1,2})\)")]
CONTINUATION = re.compile(r"^\W*(?:as also|also|and also|and)\b", re.I)
INGREDIENT_POSITION = re.compile(r"\b(?:with|and|or|to|like|than|of)\s+(?:\w+\s+){0,2}$", re.I)
PLANT_PART_SUFFIX = re.compile(r"[-\u2013](?:flowers?|leaf|leaves|berry|berries|root|roots|bark|seeds?|tree|tea|juice)\b", re.I)
NOT_A_NAME_START = {'the', 'this', 'that', 'these', 'those', 'their', 'its', 'his', 'her', 'our', 'some', 'many', 'much', 'more', 'most', 'other', 'each', 'both',
    'all', 'any', 'one', 'two', 'three', 'fresh', 'dried', 'dry', 'raw', 'hot', 'cold', 'warm', 'boiled', 'crushed', 'ground', 'herbal', 'plant', 'plants', 'medicine',
    'medicines', 'remedies', 'cough', 'black', 'green', 'white', 'red', 'wild', 'sweet', 'bitter', 'large', 'small'}
CUE_BEFORE = re.compile(r"\b(?:decoction|infusion|tincture|syrup|juice|oil|leaves|leaf|root|roots|flowers?|seeds?|bark|fruits?|berries|tea|extract|powder|ointment|po
ultice|water|essence|conserve|wine|herb)\s+of\s+(?:the\s+)?$", re.I)
# "Rue is good against poison", "Sage is singular good for the head": an herbal states a virtue.
VIRTUE_AFTER = re.compile(r"\s+(?:is|are)\s+(?:(?:very|singular|singularly|also|much|most|exceeding|excellent|especially)\s+)?(?:good|profitable|effectual|available|
excellent|helpful|a remedy|a cure)\s+(?:for|against|to|in)\b", re.I)
CUE_AFTER = re.compile(r"^\s+(?:leaves|leaf|root|roots|flowers?|seeds?|bark|fruits?|berries|tea|oil|juice|ointment|tincture|decoction|infusion|seedes?|leaues|rootes?
|flowres?|barke|juyce|iuyce|oyle)\b", re.I)
LANGUAGE_DATA = json.loads((Path(__file__).resolve().parents[2] / 'data' / 'use_terms_i18n.json').read_text(encoding='utf8'))
SERBIAN_CYRILLIC = dict(zip(['lj', 'nj', 'd\u017e', *'abvgd\u0111e\u017ezijklmnoprst\u0107ufhc\u010d\u0161'], ['\u0459', '\u045a', '\u045f', *'\u0430\u0431\u0432\u04
33\u0434\u0452\u0435\u0436\u0437\u0438\u0458\u043a\u043b\u043c\u043d\u043e\u043f\u0440\u0441\u0442\u045b\u0443\u0444\u0445\u0446\u0447\u0448']))


def serbian_cyrillic(term):
    """Serbian is written in both scripts, letter for letter: "mu\u010dnin" is "\u043c\u0443\u0447\u043d\u0438\u043d"."""
    return re.sub('lj|nj|d\u017e|.', lambda m: SERBIAN_CYRILLIC.get(m.group(), m.group()), term.casefold())


# Serbian Cyrillic text reads the Croatian and Serbian Latin terms, spelled in Cyrillic.
for _table in [*LANGUAGE_DATA['conditions'].values(), LANGUAGE_DATA['preparations']]:
    if 'hr' in _table and 'sr' not in _table:
        _table['sr'] = [serbian_cyrillic(term) for term in _table['hr']]
LOWERCASE_FUNCTION = {'with', 'from', 'into', 'upon', 'over', 'under', 'about', 'after', 'before', 'them', 'they', 'then', 'than', 'when', 'what', 'were', 'have',
    'been', 'will', 'would', 'should', 'could', 'very', 'more', 'less', 'only', 'even', 'just', 'like', 'made', 'good', 'great', 'little', 'small', 'well', 'best',
    'better', 'poor', 'rich'}
ABBREVIATION = re.compile(r"(?:\b(?:St|Dr|Mr|Mrs|Sir|Prof|cf|viz|vol|No|Fig|sp|spp|var|subsp|ssp|e\.g|i\.e|L)|\b[A-Z])$")


OWN_LETTERS = {'pl': r'[\u0105\u0119\u0142\u015b\u017a\u017c]', 'cs': r'[\u0159\u016f\u011b]', 'sk': r'[\u013e\u013a\u0155]', 'hu': r'[\u0151\u0171]', 'ro':
r'[\u0103\u0219\u021b\u0163]', 'tr': r'[\u011f\u0131]'}


# Short sources often have only one function word; these independent lexical
# clues help break ties. Exact name lookup also works across language boundaries.
LANGUAGE_HINTS = {
```

```
    'de': {'hilft', 'getrunken', 'bauchschmerzen', 'kamillentee'},
    'it': {'aiuta', 'dormire', 'camomilla'},
    'pl': {'napar', 'pomaga', 'rumianku', 'rumianek', 'brzucha'},
    'nl': {'kamillethee', 'buikpijn', 'gedronken'},
}


def detect_language(text):
    """Writing system first, then function words. Short or mixed text stays English."""
    letters = [c for c in text if c.isalpha()]
    if not letters:
        return 'en'
    share = lambda test: sum(map(test, letters)) / len(letters)
    if share(lambda c: '\u0370' <= c <= '\u03ff' or '\u1f00' <= c <= '\u1fff') > .3:
        return 'el'
    if share(lambda c: '\u0400' <= c <= '\u04ff') > .3:
        # Letters one language alone writes, then Macedonian function words ("\u0432\u043e", "\u043a\u043e\u0433\u0430").
        # \u044a is everyday Bulgarian ("\u0436\u044a\u043b\u0442"); \u044b and \u044d are Russian only. \u0458, \u0459 and \u045a are Serbian or
        # Macedonian, never Bulgarian; Serbian writes "\u0458\u0435", "\u0443" and "\u043f\u0438\u0458\u0435", Macedonian "\u0435", "\u0432\u043e",
"\u043f\u0438\u0435".
        folded = text.casefold()
        for letters, language in ((r'[\u0453\u045c\u0455]', 'mk'), (r'[\u0452\u045b\u045f]', 'sr'), (r'[\u0456\u0457\u0454\u0491]', 'uk'), (r'[\u044b\u044d]',
'ru')):
            if re.search(letters, folded):
                return language
        if re.search(r'[\u0458\u0459\u045a]', folded) and not re.search(r'[\u044a\u0449]', folded):
            return 'sr' if re.search(r'\b(?:\u0458\u0435|\u0443|\u043d\u0438\u0458\u0435|\u045b\u0435)\b|\u0438\u0458\u0435\b', folded) else 'mk'
        macedonian = sum(word in set(LANGUAGE_DATA['stopwords'].get('mk', ())) for word in re.findall(r"\w+", folded))
        return 'mk' if macedonian >= 1 and not re.search(r'[\u044a\u0449]', folded) else 'bg'
    if share(lambda c: '\u4e00' <= c <= '\u9fff') > .2:
        return 'zh'
    if share(lambda c: '\u0600' <= c <= '\u06ff') > .3:
        return 'ar'
    if share(lambda c: '\u0900' <= c <= '\u097f') > .3:
        return 'hi'
    words = re.findall(r"\w+", text.casefold())
    scores = {language: sum(word in set(terms) for word in words) for language, terms in LANGUAGE_DATA['stopwords'].items()}
    for language, hints in LANGUAGE_HINTS.items():
        scores[language] = scores.get(language, 0) + 2 * len(set(words) & hints)
    best = max(scores, key=scores.get)
    # Polish and Czech share "z", "na" and "do"; \u0142 and \u0159 do not travel. Prose that is not English
    # goes to the one language whose own letters it writes, at least twice.
    marks = {language: len(re.findall(letters, text.casefold())) for language, letters in OWN_LETTERS.items()}
    marked = [language for language, count in marks.items() if count >= 2]
    if len(marked) == 1 and (best != 'en' or scores['en'] < 2) and scores['en'] <= scores[marked[0]] + 1:
        return marked[0]
    return best if scores[best] >= 2 and scores[best] > scores['en'] else 'en'


def condition_sense(condition, original, clause):
    """Reject word senses where the condition word names something else."""
    word = original.casefold()
    if condition == 'digestive_symptoms' and word == 'stomach' and re.search(r'\b(?:over|upon|on an empty|empty|fasting)\s+(?:the\s+)?stomach\b', clause, re.I):
        return False
    if condition == 'digestive_symptoms' and word == 'wind' and not
re.search(r'\b(?:expels?|dispels?|breaks?|dissolves?|helps?|for|against|voids?|ease[sd]?)\b.{0,40}\bwind\b', clause, re.I):
        return False
    if condition == 'burns' and word == 'burn' and not re.search(r'\b(?:a|the|for|any)\s+burn\b', clause, re.I):
```

```
        return False
    if condition == 'plague_infection' and word == 'plague' and not re.search(r'\b(?:the|of|from|against)\s+plague\b', clause, re.I):
        return False
    if condition == 'respiratory' and word == 'cold' and not re.search(r'\b(?:a|the|for|bad|common|of|catch\w*|with)\s+cold\b(?!\s+(?:cause|water|place|ground|climate|weather|bath|infusion|tea|and|nature|quality|stomach))', clause, re.I):
        return False
    if condition == 'wounds' and word == 'sore' and re.search(r'\bsore (?:parts?|throats?|mouths?|eyes?)\b', clause, re.I):
        return False
    if condition == 'kidney_stones' and word == 'the stone' and not re.search(r'\b(?:helps?|expels?|breaks?|against|for|cures?|dissolves?|gravel|cholic|colic|pains? of|voids?|good)\b', clause, re.I):
        return False
    if condition == 'bleeding' and word == 'bloody' and not re.search(r'\bbloody (?:flux|urine|stools?|sputum)\b', clause, re.I):
        return False
    if condition == 'pain' and word == 'griefs' and not re.search(r'griefs? (?:of|and|in|there)', clause, re.I):
        return False
    if condition == 'nervous' and word == 'nervous' and not re.search(r'\bnervous (?:affections?|disorders?|complaints?|debility|exhaustion|headaches?|troubles?|irritab\w+)\b', clause, re.I):
        return False
    if condition == 'heart' and word == 'the heart' and re.search(r'\b(?:heart of|at the heart|by heart)\b', clause, re.I):
        return False
    if condition == 'menstrual' and word == 'courses' and not re.search(r"\b(?:women['\u2019]?s?|her|their|provok\w+|stay\w*|bring\w*|the) courses\b", clause, re.I) or (condition == 'menstrual' and re.search(r'\b(?:wisest|water|river|due) courses\b', clause, re.I)):
        return False
    if condition == 'urinary' and word == 'reins' and re.search(r'\b(?:horse|bridle|take the) reins\b', clause, re.I):
        return False
    if condition == 'cancer' and re.search(r'\b(?:Jupiter|Venus|Mars|Saturn|Mercury|Moon|Sun|sign)\b', clause):
        return False  # Culpeper's zodiac: "an herb of Jupiter, and under Cancer".
    if condition == 'fever' and re.search(r'\bhay[- ]fever\b', clause, re.I):
        return False
    return True


CAUTION = re.compile(r"\b(?:excessive use|continued use will|too (?:habitual|free|frequent|much|large|long)|caution|aggravat\w+|provoke troublesome|injurious|mischievous|fatal|paralyz\w+|paralysis|gangren\w+|(?:a )?large[r]? doses? (?:will|may|of)|has produced|have produced|symptoms? of poison\w*|may be the result|has led to the formation)\b", re.I)
EXPLICIT_USE = re.compile(r"\b(?:(?:is|are|was|were)\s+(?!not\b)(?:\w+\s+){0,2}(?:used|employed|given|applied|administered)\s+(?:[\w-]+\s+){0,5}?(?:in|for|as|to)\b|used so commonly|useful (?:wash|application|remedy|injection)|(?:wash|baths?|poultice|gargle) for\b|of (?:\w+ ){0,2}value (?:as|in|for)\b|possess\w* (?:\w+ ){0,2}value as\b)", re.I)
DOSE_EXCESS = re.compile(r"\b(?:in|by|from) (?:very |too )?(?:large|larger|excessive|over|immoderate) doses?\b|\b(?:an?|too) (?:very |too )?(?:large|larger|excessive|immoderate|full) dose\b|\btoo full doses?\b|\bover-?doses?\b", re.I)
AIMED_TERMS = '|'.join(re.escape(t) for values in CONDITIONS.values() for t in values if ' ' not in t and len(t) > 3)
AIMED = re.compile(r'\b(?:for|against|to (?:cure|ease|help|heal|relieve))\s+(?:\w+\s+){0,4}?(?:' + AIMED_TERMS + r')\b', re.I)
THERAPEUTIC = re.compile(r'\b(?:heal(?:e|es|eth|est|s|ed|ing)?|help\w*|counteract\w*|regulat\w*|(?:will )?corrects?|overcom\w*|aids?|alleviat\w*|cureth|eas(?:e|es|ed|ing)|reviv\w*|remov\w*|assuag\w*|allay\w*|mitigat\w*|expel\w*|abat\w*|drives? away|(?:take|takes|taketh|taken|taking|took) away|resolv\w*|cur(?:e|es|ed|ing|ative|atively)|remed\w*|relie\w*|benefic\w*|profitable (?:for|against|to)|use(?:d|ful|fulness)?|employ\w*|effectual|efficac\w*|medicinal\w*|medicines?\s+(?:for|against|to|in)|(?:taken|given|administered)\s+(?:(?:internally|externally)\s+)?as\s+(?:an?\s+)?medicine|sedative|laxative|astringent|cordial|comfort\w*|mollif\w*|protect\w*|prevent\w*|corrective|purifying|cleansing|cleanses?|purg(?:e|es|ed|ing|eth|est)|administer\w*|prescrib\w*|fomentation\w*|looseness of the bowels|good (?:for|to|against)|given.*for|gave.*for|appl(?:ied|y).*for|the effect is magical|stops?|stays?|stanch\w*|profit\w*|usefully|with (?:benefit|advantage)|wound[- ]herbs?|anti-?scorbutic|anti-?\w+ remedy|breaks? the stone|drieth|filleth|bindeth|cooleth|openeth|send for|obviat\w*|appeas\w*|assist\w*|act as|acts as|effected wonders|applied (?:\w+\s+){0,4}?to|eaten|chewed|voideth|helpeth|taken (?:for|in|inwardly)|strengthen\w*|provok\w*|kills?|virtues?|combat\w*|improv\w*|advis\w*|recommend\w*|promot\w*|make whole|specific (?:for|against|in)|excellent for|of (?:considerable |great |much )?service|serviceable|is good|given to those|anoint\w*|draught|tonic|antidote|dissolv\w*|opens? obstructions?|drink|drunk|drank|gargl\w*)\b', re.I)
ETYMOLOGY = re.compile(r"\b(?:technical|botanical|ordinary|latin|greek|generic|specific) name\b|\bbotanically\b|\bname(?:d)? (?:comes?|is derived|signifies|means)\b|\bcomes from the (?:greek|latin)\b|\bderived from\b|\bsignifies\b|\ba corruption of\b|\bcalled
```

```
    .{0,40}\bbecause\b(?!.*\b(?:employed|given)\b)", re.I)
IN_CONDITION = re.compile(r"^\W*(?:in|for)\s+(?:\w+\W+){0,5}?(?:" + AIMED_TERMS + r")\b", re.I)
DENIED = re.compile(r"\b(?:no benefit|of no avail|ineffectual|useless|no use|not of (?:any )?use|does not seem\b.*\b(?:qualit|medicinal|virtue)|has no (?:real
    )?(?:virtue|quality|effect))", re.I)
DESCRIPTION = re.compile(r'\b(?:rises? up|grows? (?:up|in|on|wild|by|with)|spreads?|creeps?|resembling|leaves like|flowers? (?:in|from|at)
    (?:june|july|august|may|the end)|stalks?\b.*\bleaves\b|gashes)\b', re.I)


def statement_role(text):
    """Conservative discourse triage; all original wording remains inspectable.

    A condition keyword alone is not a medicinal use. This is a transparent lexical
    screen, not historical or biomedical entailment. Adverse language takes priority.
    """
    adverse = re.search(r'\b(?:poison(?:ous|ing)|toxic(?:ity)?|become sick|sometimes they die|deadly to the patient|blistering|disagree(?:s)?|too freely|bad
    property|unquiet sleep)\b', text, re.I)
    adverse_effect = any(
        not re.search(r"(?:\b(?:not|never|without|cannot)|n['\u2019]t)\s*$", text[max(0, match.start()-30):match.start()], re.I)
        for match in re.finditer(r'\b(?:caus(?:e|es|ed|ing)|induc(?:e|es|ed|ing)|produc(?:e|es|ed|ing))\s+(?:(?:severe|violent|great|bad|acute|chronic|unpleasant|pai
    nful|a|an|the|nervous)\s+){0,3}(?:pain|colic|stomach[- ]ache|head[-
    ]ache|headache|swelling|vomiting|nausea|diarrh(?:oea|ea)|sickness|convulsions|rashes|rash|blisters|redness|irritation|fainting|syncope)\b', text, re.I)
    )
    sensory_reaction = re.search(r'\b(?:faint|swoon)(?:s|ed|ing)?\s+(?:at|from|after)\s+(?:the\s+)?(?:smell|odou?r|scent)\b', text, re.I)
    causal_disease = re.search(r'\b(?:humours?|disease|condition|inflammation|swelling|winds?)\s*,?\s*(?:which|that)?\s*(?:cause|causes)\b', text, re.I)
    # "Used for paralysis" names what is treated; the word alone would read as a harm.
    # "In large doses it vomits and purges": the dose makes the action a harm, unless a condition is aimed at.
    # A sentence that first states a use ("medicinal, and in larger doses purgative") keeps it.
    dose = DOSE_EXCESS.search(text)
    excess = dose and not AIMED.search(text) and not re.search(r'\b(?:praised|commended|esteemed)\b', text[:dose.start()], re.I) and not
    THERAPEUTIC.search(text[:dose.start()])
    if sensory_reaction or excess or (adverse_effect and not causal_disease) or ((adverse or CAUTION.search(text)) and not EXPLICIT_USE.search(text)):
        return 'ADVERSE_EFFECT_RECORDED'
    if re.search(r'\bState poison\b|\bfood for\b|\ban excellent food\b|\bso strong a poison\b', text, re.I):
        return 'CONTEXT_ONLY_OR_UNCLEAR'
    if re.search(r'\b(?:symbolis(?:ing|e)|wounds of Christ|sculptured|physician.*famous|sleeping within mine orchard|poor silly countryman)\b', text, re.I) or
    ETYMOLOGY.search(text):
        return 'CONTEXT_ONLY_OR_UNCLEAR'
    aimed = AIMED.search(text)
    if DESCRIPTION.search(text) and not (aimed or re.search(r'\b(?:heal|help|eas|cur|relie|good for|remed)\w*', text, re.I)):
        return 'CONTEXT_ONLY_OR_UNCLEAR'
    return 'THERAPEUTIC_USE_RECORDED' if THERAPEUTIC.search(text) or aimed or IN_CONDITION.search(text) or EXPLICIT_USE.search(text) else 'CONTEXT_ONLY_OR_UNCLEAR'


TEA_WORDS = {normalize(word) for word in ('tea', 'thee', 'tee', 't\xe9', 't\xe8', 'th\xe9', 'ch\xe1', '\u0447\u0430\u0439', '\u0447\u0430\u0458', '\u010daj',
    '\xe7aj', '\xe7ay', '\u03c4\u03c3\u03ac\u03b9', 'ceai')}
FUNCTION_EPITHETS = {'oder', 'aber', '\xfcber', 'unter', 'hinter', 'wieder', 'immer', 'sehr', 'noch', 'schon', 'dieser', 'einer', 'eines', 'einem', 'einen',
    'seiner', 'ihrer', 'unser', 'nicht', 'maar', 'voor', 'door', 'over', 'onder', 'achter', 'weer', 'zeer', 'ook', 'niet'}
OLD_PART = re.compile(r'(?:seed|roote?|leaue|leave|flowre|flower|barke|juyce|iuyce|juice|water|oyle|wine|syrupe|conserve|powder|decoction)e?s?')
MARKED_LETTER = re.compile(r'[\xc0-\xd6\xd8-\xf6\xf8-\u024f]')
MARKED_WORD = re.compile(r'\b[^\W\d_]*[\xc0-\xd6\xd8-\xf6\xf8-\u024f][^\W\d_]*\b')
CASE_LANGUAGES = {'hr', 'sr', 'sq', 'el', 'ro', 'sl', 'pl', 'cs', 'sk', 'mk', 'bg', 'uk', 'hu'}
VOWELS = 'aeiouy\u0430\u0435\u0438\u043e\u0443\u044a\u044f\u044e\u0450\u045d'
# A short consonant-final name takes a case ending ("orah", "oraha") or a possessive adjective ("brin", "brinove").
SHORT_ENDING =
re.compile(r'(?:[aeiouy]{1,2}|om|em|ov[aeiou]?|ev[aeiou]?|in[aeiou]?|\u043e\u0442|\u0442\u0430|\u0442\u043e|\u043e\u0432[\u0430\u0435\u0438\u043e]?)$')


def consonant_marks(word):
```

```
    """Casefolded, with accents on vowels removed and marks on consonants kept: "l\xedpa" -> "lipa", "ho\u0159ec" stays."""
    out = []
    for char in unicodedata.normalize('NFD', word.casefold()):
        if unicodedata.combining(char) and out and out[-1] in 'aeiouy\u0430\u0435\u0438\u043e\u0443\u044b\u044d\u044e\u044f\u0456':
            continue
        out.append(char)
    return unicodedata.normalize('NFC', ''.join(out))


# Romance and Germanic names change a vowel or add a plural, never a consonant: "fromage" is cheese, not "fromager".
UNINFLECTED = {'fr', 'it', 'es', 'pt', 'de', 'nl', 'la'}


def ending_only(word, key, language):
    shared = 0
    while shared < min(len(word), len(key)) and word[shared] == key[shared]:
        shared += 1
    allowed = 'aeiouyhsx' + ('n' if language in ('de', 'nl') else '')
    return all(c in allowed for c in word[shared:] + key[shared:])


ALBANIAN_DEFINITE = {'ri', 'rit', 'ut', 'ni', 'nit'}


def inflects(word, key, language):
    """Is this word an inflected form of a listed name? Endings change, the stem does not.
    "trumz\xeb" is "trumza" and "bokvice" is "bokvica"; "so\u011fuk" (cold) is no form of "s\xf6\u011f\xfct" (willow),
    because a stem ending in a consonant only grows. Croatian "lipe" and "nane" inflect four-letter names,
    and Macedonian "\u043f\u0435\u043b\u0438\u043d\u043e\u0442" is "\u043f\u0435\u043b\u0438\u043d" with its article."""
    if not 0 <= len(word) - len(key) + 2 <= 5:
        return False
    if language in UNINFLECTED and not ending_only(word, key, language):
        return False
    if len(key) >= 7:
        return word[:len(key) - 2] == key[:-2]
    if len(key) >= 5:
        return word[:len(key) - 1] == key[:-1] and (key[-1] in VOWELS or word.startswith(key))
    # Albanian three-letter names take the definite ending after the stem: "bli", "bliri", "\xe7aji i bliut".
    if len(key) == 3 and language == 'sq':
        return word.startswith(key) and word[3:] in ALBANIAN_DEFINITE
    if len(key) != 4 or language not in CASE_LANGUAGES:
        return False
    if key[-1] in VOWELS:
        return word[:3] == key[:3] and word[3] in VOWELS and len(word) <= 6
    return word.startswith(key) and bool(SHORT_ENDING.fullmatch(word[4:]))


# German, Dutch and Hungarian join plant and part in one word: "Baldrianwurzel", "brandnetelthee", "kamillatea".
PLANT_PART = {'de': re.compile(r'(?:wurzeln?|blatter|blatt|bluten|bluthen|blute|kraut|krauter|tee|thee|rinde|samen|ol|saft|beeren|beere|fruchte|frucht|knospen)$'),
              'nl': re.compile(r'(?:thee|wortels?|bladeren|blad|bloemen|bloem|zaden|zaad|olie|bessen|bes|schors|kruid|sap)$'),
              'hu': re.compile(r'(?:tea|teat|teaja|level|levele|levelek|gyoker|gyokere|virag|viraga|olaj|mag|magja|termes|kereg|fozet)$')}
NON_PLANT_REMEDY = re.compile(r"\b(?:snails?|slugs?|egg[- ]?shells?|old eggs|oyster shells|vipers?|bees?\b.{0,30}\bkilled|bee-tea)\b", re.I)


def practice_kind(text):
    """What kind of practice the sentence reports; a filter for readers, not a judgement."""
    if NON_PLANT_REMEDY.search(text):
        return 'NON_PLANT_REMEDY'
```

```
    if re.search(r'\b(?:amulet|charm|worn (?:round|about|in|on|over)|hung|evil eye|evil spirits|anti-magical|ward(?:ing)? off|the devil|witch\w*|spell|superstiti\w*|bag over|frontlets?|protective)\b', text, re.I):
        return 'RITUAL_OR_AMULET'
    if re.search(r"\b(?:horses?|cattle|sheep|farriers?|cows?|swine|dogs?|veterinar\w*)\b(?!['\u2019]s\b|[\s-]+(?:roses?|chestnuts?|radish|tails?|mint|parsley|parsnips?|wood|violets?|mercury|grass|fennel|berr\w+|tooth|bane|sorrel|weed|lichen|camomile|chamomile|slip)\b)", text, re.I):
        return 'VETERINARY'
    if re.search(r'\b(?:hahnemann|homoeopath\w*|homeopath\w*|provers?|law of similars|similars)\b', text, re.I):
        return 'HOMOEOPATHIC'
    return 'MEDICINAL'


def unwrap(text):
    """A line break inside a paragraph is typesetting, not a sentence end. Offsets do not move."""
    letters = list(text)
    for m in re.finditer(r'(?<=[^\n])\n(?=[^\n])', text):
        line = text[text.rfind('\n', 0, m.start()) + 1:m.start()].strip()
        following = text[m.end():m.end() + 8]
        if line.isupper() or LIST_ITEM.match(following):
            continue
        if len(line) >= WRAPPED_LINE or (len(line) >= 30 and following[0].islower()):
            letters[m.start()] = ' '
    return ''.join(letters)


GLOSS_BEFORE = re.compile(r"(?<![\w'\u2019-])([A-Z][\w'\u2019-]{2,} [A-Z][\w'\u2019-]{2,}|[^\W\d_][\w'\u2019-]{2,})(?: (?:" + '|'.join(PARTS) + r"))?(?:\s*,\s*(?:from|of|or|the|i\.\s?e\.)?\s*|\s*\(\s*|\s+(?:from|or|o|ou|oder)\s+)$")


def source_glosses(text, mentions):
    """The name a source itself gives a species: "Kurchi bark, from Holarrhena antidysenterica",
    "El estafiate (Artemisia ludoviciana)". The pairing is the author's, kept whether or not any
    name list knows the local word."""
    spans = [(m['start'], m['end']) for m in mentions]
    found = {}
    for mention in mentions:
        if 'BINOMIAL' not in mention['method']:
            continue
        match = GLOSS_BEFORE.search(text[max(0, mention['start'] - 48):mention['start']])
        if not match:
            continue
        start = mention['start'] - len(match.group(0)) + match.start(1) - match.start(0)
        word = match.group(1)
        key = word.casefold()
        # "in the Philippines (Vitex negundo)", "applied to the skin, Aloe vera": a place or an object.
        if re.search(r"\b(?:in|at|to|into|on|from|of|near|throughout|over)\s+(?:the\s+)?$", text[max(0, start - 20):start], re.I):
            continue
        if (key in NOT_A_NAME_START or key in LOWERCASE_FUNCTION or key in CUE_STOP or key in NEVER_A_PLANT or key.endswith(('aceae', 'ales', 'eae'))
                or any(s < start + len(word) and start < e for s, e in spans if (s, e) != (mention['start'], mention['end']) and 'BINOMIAL' in next(m['method'] for m in mentions if (m['start'], m['end']) == (s, e)))):
            continue
        found.setdefault(mention['id'], []).append({'name': word, 'start': start, 'end': start + len(word), 'binomial': mention['text']})
    return found


def heading_gloss(text, mentions):
    """"BONESET.\\nThe leaves and flowering tops of Eupatorium perfoliatum.": the author names the
    heading's plant in the first sentence after it. Returns (heading mention, binomial mention)."""
```

```
    if not mentions:
        return None
    head = mentions[0]
    line_end = text.find('\n', head['start'])
    # "BONESET. The leaves of ..." runs on in one line; capitals and a stop still close a heading.
    if head['text'].isupper() and text[head['end']:head['end'] + 2] == '. ' and not text[:head['start']].strip():
        line_end = head['end'] + 1
    line = text[text.rfind('\n', 0, head['start']) + 1:line_end if line_end >= 0 else len(text)]
    if 'BINOMIAL' in head['method'] or line_end < 0 or len(line.strip()) > 60 or line.strip().rstrip('.').count('.') > 1:
        return None
    first = re.search(r'[.!?](?:\s|$)', text[line_end:])
    end = line_end + (first.end() if first else len(text) - line_end)
    inside = [m for m in mentions if line_end <= m['start'] < end and 'BINOMIAL' in m['method']]
    if len(inside) == 1 and re.search(r'\b(?:variety of|cultivar of|hybrid of|derived from|related to|compared (?:with|to)|unlike|rather than|instead of|different
from|not to be confused with)\b', text[line_end:inside[0]['start']], re.I):
        # The first binomial may name a relative/comparator, not the heading's
        # own plant: "garden lettuce is a variety of wild lettuce (L. virosa)".
        return None
    return (head, inside[0]) if len(inside) == 1 else None


def heading_common_names(text, mentions):
    """Literal specific names in a library entry's opening botanical subject.

    These are lookup clues, not new dictionary aliases or taxonomic facts.
    A gapped spelling is tried only when it also occurs literally elsewhere
    in the entry, so an arbitrary omitted qualifier cannot invent a name.
    """
    if not mentions or mentions[0]['method'] != 'SOURCE_HEADING_SUBJECT':
        return []
    head = mentions[0]
    line_end = text.find('\n', head['end'])
    if line_end < 0:
        return []
    first = next(sentences(text[line_end:]), None)
    if not first:
        return []
    raw = first['text']
    masked = re.sub(r'\([^()]*\)', lambda m: ' ' * len(m.group()), raw)
    verb = re.search(r'\b(?:is|was|belongs|grows|bears)\b', masked, re.I)
    if not verb:
        return []
    if verb.group().casefold() in ('is', 'was') and not
re.match(r'\s+(?:found\b|native\b|an?\s+(?:\w+\s+){0,5}(?:plant|herb|flower|tree|shrub|vegetable|perennial|annual|variety)\b)', masked[verb.end():], re.I):
        return []
    subject = masked[:verb.start()].split(',', 1)[0]
    article = re.match(r'\s*(?:(?:the|our|this)\s+)?', subject, re.I)
    left = article.end()
    right = len(subject.rstrip())
    subject = subject[left:right]
    heading = normalize(head.get('heading_name') or head['text'])
    if not heading or not re.search(r'(?<!\w)' + re.escape(heading) + r'(?!\w)', normalize(subject)):
        return []
    alternative = list(re.finditer(r'\bor\s+', subject, re.I))
    if alternative:
        left += alternative[-1].end()
        subject = masked[left:right]
    elif normalize(subject) == heading:
```

```
        return []
    if not re.fullmatch(r"[\w'\u2019-]+(?:\s+[\w'\u2019-]+){0,5}", subject.strip()):
        return []
    left += len(subject) - len(subject.lstrip())
    subject = subject.strip()
    start = line_end + first['start'] + left
    forms = [subject]
    words = subject.split()
    if not alternative and len(words) > 2:
        normalized_text = normalize(text)
        for index in range(1, len(words) - 1):
            form = ' '.join(words[:index] + words[index + 1:])
            if re.search(r'(?<!\w)' + re.escape(normalize(form)) + r'(?!\w)', normalized_text):
                forms.append(form)
    return [{'mention_id':head['id'], 'name':subject, 'start':start, 'end':start + len(subject), 'lookup_forms':list(dict.fromkeys(forms)),
    'kind':'OPENING_ALTERNATIVE_NAME' if alternative else 'OPENING_QUALIFIED_NAME'}]


def sentences(text):
    """Return sentence-like clauses with original code-point offsets, no rewritten text."""
    start = 0
    for boundary in re.finditer(r"\n|[.!?;]+(?=[\s\"\u201d\u2019)\]]|$)", text):
        if boundary.group().startswith('.') and ABBREVIATION.search(text[max(0, boundary.start() - 6):boundary.start()]):
            continue
        end = boundary.end()
        raw = text[start:end]
        lead = len(raw) - len(raw.lstrip())
        value = raw.strip()
        if value:
            yield {"text": value, "start": start + lead, "end": start + lead + len(value)}
        start = end
    raw = text[start:]
    value = raw.strip()
    if value:
        lead = len(raw) - len(raw.lstrip())
        yield {"text": value, "start": start + lead, "end": start + lead + len(value)}


def name_list_rows(text):
    """Names an author lists for one plant, each kept at its own place in the source."""
    rows = []
    for number, match in enumerate(m for pattern in NAME_LISTS for m in pattern.finditer(text)):
        begin = match.start('names')
        # "Dh\xe9ras or Dh\xe9nras" is two spellings; italic marks around a language tag are print, not name.
        for order, piece in enumerate(re.finditer(r"(?:(?!\s+or\s)[^,;()])+", match['names'])):
            if re.fullmatch(NAME_TAG, piece.group().strip()):
                continue
            word = piece.group().strip().strip('_*= ').lstrip('. ')
            if re.match(r"or\s", word):
                word = word[3:].strip()
            # "Manga." opens its list and is the name; "Eng." after a name says which language.
            if (order and word.endswith('.') and len(word) <= 7) or LANGUAGE_TAG.fullmatch(word) or LANGUAGE_PAIR.fullmatch(word):
                continue
            word = re.sub(r'^(?:and|or)\s+', '', word.rstrip(' .'))
            start = begin + piece.start() + piece.group().find(word)
            if 3 <= len(word) <= 40 and len(word.split()) <= 4 and word[0].isalpha() and not any(c.isdigit() for c in word):
                rows.append({'text': word, 'start': start, 'end': start + len(word), 'method': 'LOCAL_NAME_PROPOSAL', 'list_id': number})
    return rows
```

```


@lru_cache(maxsize=256)
def term_pattern(terms):
    # Longest alternative first, so "sore throat" wins over "sore" at the same position.
    return re.compile(r"(?<!\w)(?:" + "|".join(re.escape(t) for t in sorted(terms, key=len, reverse=True)) + r")(?!\w)", re.I)


def spans(text, terms):
    if not terms:
        return []
    # Word boundaries and literal punctuation; never execute retrieved instructions.
    return [{"text": m.group(), "start": m.start(), "end": m.end()} for m in term_pattern(tuple(sorted(set(terms)))).finditer(text)]


def stem_spans(text, terms):
    """Word-start stem matching for inflected languages, on accent-insensitive text."""
    folded, offsets = normalized_offsets(text)
    found = []
    for term in sorted(set(terms), key=len, reverse=True):
        key = normalize(term)
        for m in re.finditer(r"(?<!\w)" + re.escape(key) + r"\w*", folded):
            start, end = offsets[m.start()], offsets[m.end() - 1] + 1
            if not any(start < x["end"] and end > x["start"] for x in found):
                found.append({"text": text[start:end], "start": start, "end": end, "stem": len(key)})
    return sorted(found, key=lambda x: x["start"])


HEADING_LINE = re.compile(r"^(?:\[\d+\]\s*)?(?:[A-Z][A-Z'\u2019\-]+(?:[ ,]+[A-Za-z'\u2019\-(),]+){0,8}\.?|Of (?:the )?[A-Z][a-z]+(?: [a-z]+){0,3}\.?|[A-Z][a-z]+(?:[
-][a-z]+)?\.?)$")
# "Black Cohosh." and "Yellow Jasmine or Jessamine."; a dictionary entry is rarely a sentence.
TITLE_LINE = re.compile(r"^[A-Z][\w'\u2019-]+(?: (?:[A-Z][\w'\u2019-]+|of|the|or|and)){0,4}\.?$")
# "Garcinia mangostana, L." and "Cimicifuga Racemosa (L.) Nutt.": a name and its authors, no verb.
BINOMIAL_LINE = re.compile(r"^[A-Z][a-zA-Z]{2,}\s+[A-Za-z][a-zA-Z-]{2,}\b(?:[\s,.&]*(?:\([^)]*\)|[A-Z][\w.'\u2019-]*|var\.|ex|et|and|[a-z]\.))*\s*\.?\s*$")
# "(Ind. Mo-cash)" under a heading belongs to it.
GLOSS_LINE = re.compile(r"^[(\[].*[)\]]\.?$")
# Parts of an entry, never the plant it is about.
ENTRY_PART = re.compile(r"^(?:(?:medicinal |botanical |economic )?(?:uses?|descriptions?|properties|virtues|habitat|habitat and range|history|notes?|references?|dose
|doses|preparations?|constituents|composition|cultivation|collection|prices?|introduction|contents|index|preface|appendix|chapter \w+|part \w+|the plant|the root|the
 leaves)|other (?:common )?names|common names?|synonyms?)\.?$", re.I)
BOLD_ENTRY = re.compile(r"^[ \t]*\d+\.\s+[+=]{1,2}(?P<name>[^+=\n.][^+=]{1,80}?)\.?[+=]{1,2}", re.M)
LEADING_NAME = re.compile(r"^\s*(?:Of (?:the )?)?([A-Z][\w'\u2019-]+(?: [a-z][\w'\u2019-]+)?)\s*[:\u2014\u2013]\s+", re.M)
LATIN_BINOMIAL = re.compile(r"\b[A-Z][a-z]{2,}\s+(?:[x\xd7]\s+)?[a-z]{3,}(?:-[a-z]+)*\b")
# Old books capitalise the epithet: "Euphorbia Tirucalli, L."; a genus or an author vouches for it.
CAPITAL_BINOMIAL = re.compile(r"\b([A-Z][a-zA-Z]{2,})\s+([A-Z][a-zA-Z]{3,}(?:-[A-Za-z]+)*)\b")
AUTHOR_AFTER = re.compile(r"\s*,?\s*\(?(?:[A-Z][a-z]*\.|Linn\b)")
# Editors' emphasis around a name; the marks are blanked so offsets into the source hold.
# Common name lists as books print them, and "American Coffee Berry" with its region set aside.
NAME_LISTS = [re.compile(r"(?:Nom\.\s*Vulg\.|(?:Other\s+)?(?:Common|Local|Native|Vernacular|Popular)\s+Names?)\s*[:.\u2014\u2013-]*\s*(?P<names>[^\n]+)", re.I),
    re.compile(r"\(\s*Ind\.\s+(?P<names>[^)\n]+)\)"), re.compile(r"\b(?:commonly |locally |popularly |generally )?(?:known|called)
    as\s+[\"\u201c]?\s*(?P<names>[A-Z][\w'\u2019-]+(?: [A-Z]?[\w'\u2019-]+){0,3})"),
    re.compile(r"(?:(?:^|[.(\n])\s*)(?:American|Indian|Mexican|Chinese|Japanese|Californian|California|Canadian|Virginian)\s+(?P<names>[A-Z][\w'\u2019-]+(?:
    [A-Z][\w'\u2019-]+){0,2})")]
# "Bhind\xed, R\xe1m-tur\xe1\xed (_Hind._), Bh\xe9nd\xed (_Duk._, _Punj._)": colonial works tag each name with its
# language. Two tagged runs make a list; a tag needs an abbreviation point or a language's own name.
NAME_TAG =
    r"_?(?:[A-Z][a-z]{1,7}\.|Malay|Hindi|Hindustani|Tamil|Telugu|Bengali|Burmese|Sinhalese|Persian|Arabic|Sanskrit|Urdu|Malayalam|Marathi|Gujarati|Kanarese|Nepali)_?"
TAGGED_NAMES = re.compile(r"(?P<names>(?:(?:(?!\n\n)[^()]){2,90}?\(\s*" + NAME_TAG + r"(?:\s*,\s*(?:and\s+)?" + NAME_TAG + r")*\s*\)[,.;]?){2,})")
```

```
NAME_LISTS.append(TAGGED_NAMES)
# "Goldthread; cankerroot; mouthroot; yellowroot.": a paragraph of nothing but names. Each still
# needs a name list or GBIF to count as a plant.
SEMICOLON_NAMES = re.compile(r"(?:^|\n[ \t]*\n)[ \t]*(?P<names>[A-Z][\w'\u2019-]+(?:[ \t]+[\w'\u2019-]+){0,3}(?:;\s+[\w'\u2019-]+(?:[ \t]+[\w'\u2019-]+){0,3}){1,12})\.[ \t]*(?=\n[ \t]*\n|\s*$)")
NAME_LISTS.append(SEMICOLON_NAMES)
LANGUAGE_TAG = re.compile(r"(?:Tag|Vis|Pam|Pamp|Iloc|Sp|Fil|Eng|Indo|Bik|Lat|Mal|Hind|Beng|Fr|Ger)(?:[.-]+(?:Tag|Vis|Pam|Fil|Eng)?)*\.?")
LANGUAGE_PAIR = re.compile(r"[A-Z][a-z]{1,5}\.? and [A-Z][a-z]{1,5}\.?")
LIST_ITEM = re.compile(r"\s{4,}|\s*(?:\d+[.)]|[-*\u2022\u2013])\s")
WRAPPED_LINE = 55
USE_LABEL = re.compile(r"^[
\t]*(?:(?:medicinal|medical|therapeutic)\s+)?(?:uses?|properties|virtues|action(?:\s+and\s+uses)?)\s*\.?\s*(?:--+|\u2014|\u2013|:)\s*(?=\S)", re.I | re.M)
EMPHASIS = re.compile(r"(?<![\w*_=])[_*=]+(?=\S)|(?<=\S)[_*=]+(?![\w*_=])")
LATIN_EPITHET = re.compile(r"(?:a|e|i|o|um|us|is|es|er|or|ur|ans|ens|ix|ex|ax|ox|ys|on)$")
NOT_EPITHET = {'will', 'shall', 'showed', 'would', 'could', 'should', 'must', 'might', 'does', 'made', 'like', 'name', 'same', 'more', 'some', 'here', 'there',
    'where', 'while', 'before', 'after', 'above', 'below', 'alone', 'once', 'twice', 'whole', 'wine', 'water', 'herbe', 'leaves', 'leaf', 'root', 'roots', 'flowers',
    'flower', 'seeds', 'seed', 'bark', 'oil', 'oils', 'extract', 'extracts', 'tea', 'teas', 'decoction', 'infusion', 'tincture', 'paste', 'powder', 'juice', 'syrup',
    'cream', 'tablets', 'capsules', 'fruits', 'fruit', 'berries', 'herb', 'plant', 'plants', 'species', 'was', 'were', 'are', 'has', 'have', 'may', 'can', 'and', 'the',
    'with', 'for', 'from', 'which', 'that', 'this', 'also', 'used', 'taken', 'given', 'applied', 'boiled', 'dried', 'preparations'}
NOT_GENUS = {'Organon', 'Paris', 'London', 'Rome', 'Athens', 'Vienna', 'Berlin', 'Victoria', 'Arabia', 'Persia', 'China', 'India', 'Egypt', 'Greece', 'Italy',
    'France', 'Spain', 'Germany', 'England', 'Scotland', 'Wales', 'Ireland', 'America', 'Africa', 'Europe', 'The', 'For', 'And', 'From', 'With', 'This', 'That', 'These',
     'Those', 'See', 'When', 'Which', 'After', 'Before', 'Under', 'Some', 'Its', 'Their', 'They', 'There', 'Many', 'Most', 'Other', 'Such', 'Each', 'Both', 'Several',
    'Fruits', 'Leaves', 'Roots', 'Flowers', 'Seeds', 'Dioscorides', 'Gerard', 'Culpeper', 'Galen', 'Pliny', 'Aetius', 'Avicenna', 'Hippocrates', 'Parkinson', 'Mattioli',
     'Sir', 'Taken', 'Applied', 'Given', 'Used', 'Boiled', 'Drunk', 'Mixed', 'Made', 'Being', 'Having', 'Your', 'Our', 'His', 'Her', 'Upon', 'Into', 'About', 'Against',
    'Also', 'Another', 'Southern', 'Northern', 'Western', 'Eastern', 'Central', 'Informants', 'Local', 'Traditional', 'Recent', 'Modern', 'Ancient'}


def sections(text):
    """Heading-delimited stretches; a paste of several chapters is not one subject."""
    found = []
    lines = list(re.finditer(r"[^\n]*(?:\n|$)", text))
    for index, line in enumerate(lines):
        value = line.group().strip()
        if not value or len(value) > 70:
            continue
        opens = index == 0 or not lines[index - 1].group().strip() or (found and found[-1]['line'] == index - 1)
        following = lines[index + 1].group().strip() if index + 1 < len(lines) else ''
        after = lines[index + 2].group().strip() if index + 2 < len(lines) else ''
        closes = not following or ((GLOSS_LINE.match(following) or BINOMIAL_LINE.match(following)) and not after)
        shaped = HEADING_LINE.match(value) or TITLE_LINE.match(value) or BINOMIAL_LINE.match(value)
        if opens and closes and shaped and not ENTRY_PART.match(value) and index + 2 < len(lines):
            begin = line.start() + (len(line.group()) - len(line.group().lstrip()))
            found.append({'heading': value, 'start': begin, 'end': begin + len(value), 'line': index})
    for index, section in enumerate(found):
        section['body_end'] = found[index + 1]['start'] if index + 1 < len(found) else len(text)
    return found


# A symptom or a preparation is never a plant, whatever a vernacular database records.
NEVER_A_PLANT |= {normalize(term) for terms in CONDITIONS.values() for term in terms} | {normalize(term) for term in PREPARATIONS + PARTS} | {'balsam', 'wool'}


def albanian_forms(key):
    """Albanian lists a noun as written in the record; prose uses the other form. A feminine name
    is definite in -a and indefinite in -\xeb or bare ("d\xebllinja", "d\xebllinj\xeb"; "san\xebza", "san\xebz")."""
    head, _, rest = key.partition(' ')
    forms = {key}
    if len(head) >= 5 and head.endswith('a'):
```

```
        forms |= {f'{head[:-1]}{ending}{" " + rest if rest else ""}' for ending in ('e', '')}
    elif len(head) >= 5 and head.endswith('e'):
        forms.add(f'{head[:-1]}a{" " + rest if rest else ""}')
    return forms


class Extractor:
    version = "gazetteer-span-0.6-short-text-safe"

    def __init__(self, taxa):
        self.taxa = taxa
        self.aliases = {}
        self.foreign_aliases = {}
        # Local names written with letters English lacks ("san\xebz", "\xe7aj mali"), found in English prose too.
        self.marked_aliases = {}
        self.foreign_written = {}
        self.genus_names = {}
        for taxon in taxa:
            gazetteer_only = taxon.get('gazetteer_only')
            for alias in taxon["aliases"]:
                language = alias.get('lang', 'en')
                if '(' in alias['name']:
                    continue
                if language not in ('en', 'la') and alias['kind'] == 'VERNACULAR':
                    self.foreign_aliases.setdefault(language, {}).setdefault(normalize(alias['name']), set()).add(taxon['id'])
                    self.foreign_written.setdefault((language, normalize(alias['name'])), set()).add(unicodedata.normalize('NFC', alias['name'].casefold()))
                    if MARKED_LETTER.search(alias['name']) and len(normalize(alias['name'])) >= 4:
                        key = normalize(alias['name'])
                        for form in albanian_forms(key) if language == 'sq' else (key,):
                            self.marked_aliases.setdefault(form.split()[0], set()).add((form, key, language))
                    continue
                key = normalize(alias['name'])
                if key in NEVER_A_PLANT or len(alias["name"]) < (3 if alias['kind'] == 'HISTORICAL_LEXICON' else 4):
                    continue
                if gazetteer_only and alias['kind'] == 'VERNACULAR' and ' ' not in key and len(key) < 5:
                    continue
                self.aliases.setdefault(alias["name"], set()).add(taxon["id"])
            if not gazetteer_only:
                self.aliases.setdefault(taxon["genus"], set()).add(taxon["id"])
            elif len(taxon['genus']) >= 5:
                self.genus_names.setdefault(taxon['genus'], set()).add(taxon['id'])
        # Orthographic transformations, not new identity assertions.
        for name, ids in list(self.aliases.items()):
            if "feverfew" in name.lower():
                self.aliases.setdefault(re.sub("feverfew", "featherfew", name, flags=re.I), set()).update(ids)
            if "chamomile" in name.lower():
                self.aliases.setdefault(re.sub("chamomile", "camomile", name, flags=re.I), set()).update(ids)
            if "john" in name.lower():
                for transformed in [name.replace("\u2019", "'"), name.replace("'", "\u2019")]:
                    self.aliases.setdefault(transformed, set()).update(ids)
        # Index aliases by their first normalized token. Recompiling a thousand
        # whole-document regexes per chapter scales poorly and thrashes re's cache.
        self.alias_prefixes = {}
        self.alias_taxa = {}
        for name, identifiers in self.aliases.items():
            self.alias_taxa.setdefault(normalize(name), set()).update(identifiers)
        for key in self.alias_taxa:
            if key:
```

```
                self.alias_prefixes.setdefault(key.split()[0], []).append(key)
        self.known_genera = {normalize(t['genus']) for t in taxa}
        self.foreign_exact = {}
        for language, table in self.foreign_aliases.items():
            for key in table:
                if len(key) >= 4 and key not in TEA_WORDS:
                    self.foreign_exact.setdefault(key.split()[0], []).append((key, language))
        self.single_word_vernaculars = sorted(k for k in self.alias_taxa if ' ' not in k and len(k) >= 6)

    def sense_ok(self, text, start, end, key):
        """Is this word used as a plant here? Capitalised, or next to botanical wording."""
        window = text[max(0, start - 48):min(len(text), end + 32)]
        if NON_BOTANICAL_SENSE.search(window):
            return False
        if key in SENSE_REQUIRED or key in PERSON_PLANT_NAMES:
            # A geographic role is local to this occurrence: "Orange trees in
            # Orange County" still contains one real plant mention.
            if PLACE_AFTER.match(text[end:end + 48]) or PLACE_BEFORE.search(text[max(0,start - 48):start]):
                return False
        if key in PLACE_PREFIXES:
            # Exact cross-language fallback can otherwise turn "in Santa
            # Barbara" into a catnip alias. Keep a botanical continuation such
            # as "Santa Maria leaves" rather than blacklisting the name itself.
            following = text[end:end + 72]
            named = re.match(r'\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)?', following)
            if named and not CUE_AFTER.match(following[named.end():]) and (re.search(r'\b(?:in|near|from|to|within|around|across)\s*$',text[max(0,start -
    32):start],re.I) or PLACE_AFTER.match(following[named.end():])):
                return False
        # A person's conversation can mention tea or pain without naming a plant.
        if key in PERSON_PLANT_NAMES:
            before, after = text[max(0, start - 56):start], text[end:end + 96]
            if person_name_context(before, after):
                return False
        if key not in SENSE_REQUIRED:
            return True
        raw = text[start:end]
        sentence_start = start == 0 or bool(re.search(r"[.!?;:\n\"\u201c]\s*$", text[max(0, start - 3):start]))
        if raw[:1].isupper() and not sentence_start:
            return True
        before, after = text[max(0, start - 28):start], text[end:end + 32]
        return bool(CUE_BEFORE.search(before) or CUE_AFTER.match(after) or raw[:1].isupper() and VIRTUE_AFTER.match(text, end) or BOTANICAL_NEIGHBOUR.search(before +
     ' ' + after) or GROWING_OR_COOKING.search(before + ' ' + after))

    def same_letters(self, word, language, key):
        """An inflected guess keeps the writer's letters: "hore\u010dce" (fever) is no form of "ho\u0159ec" (gentian).
        A writer who types no diacritics at all ("horecce") is matched on plain letters."""
        word = consonant_marks(word)
        if word == normalize(word):
            return True
        # Vowel length alternates ("l\xedpa", "lipov\xfd"); a marked consonant is another letter.
        return any(inflects(word, consonant_marks(written), language) for written in self.foreign_written.get((language, key), ()) if ' ' not in written)

    def foreign_mentions(self, text, language):
        table = self.foreign_aliases.get(language, {})
        if not table:
            return []
        hits = []
        folded, offsets = normalized_offsets(text)
```

```
        if language == 'zh':
            for key in sorted(table, key=len, reverse=True):
                for m in re.finditer(re.escape(key), folded):
                    hits.append((m.start(), m.end(), key))
        else:
            by_first = {}
            for key in table:
                by_first.setdefault(key[:3], []).append(key)
            for token in re.finditer(r'\w+', folded):
                part = PLANT_PART[language].search(token.group()) if language in PLANT_PART else None
                if part and part.start() >= 4 and token.group() not in table:
                    stem = token.group()[:part.start()]
                    key = next((k for k in (stem, stem + 'e', stem[:-1], stem[:-2]) if ' ' not in k and len(k) >= 4 and k in table), None)
                    if key:
                        hits.append((token.start(), token.end(), key))
                        continue
                for key in by_first.get(token.group()[:3], []):
                    value = folded[token.start():token.start() + len(key)]
                    if value == key and (token.start() + len(key) == len(folded) or not folded[token.start() + len(key)].isalnum()):
                        hits.append((token.start(), token.start() + len(key), key))
                    # A short name may change only its last letter ("trumza", "trumz\xeb"); "manita" is no form of "manies".
                    elif ' ' not in key and inflects(token.group(), key, language) and self.same_letters(text[offsets[token.start()]:offsets[token.end() - 1] + 1],
    language, key):
                        hits.append((token.start(), token.end(), key))
        rows = []
        # The word as written beats an inflected guess: "manzanilla" is chamomile, "manzanillo" the manchineel.
        for begin, finish, key in sorted(hits, key=lambda h: (-(h[1] - h[0]), folded[h[0]:h[1]] != h[2], h[0])):
            if key in TEA_WORDS:
                continue
            start, end = offsets[begin], offsets[finish - 1] + 1
            if not any(start < x['end'] and end > x['start'] for x in rows):
                rows.append({'text': text[start:end], 'start': start, 'end': end, 'lookup': key, 'method': 'MULTILINGUAL_NAME_PROPOSAL', 'language': language})
        return rows

    def marked_mentions(self, text):
        """Local names inside English prose: "women collect \xe7aj mali". Only a word with letters English
        lacks is looked up, and only as written, so English words never turn into foreign plant names."""
        rows = []
        folded, offsets = normalized_offsets(text)
        for word in MARKED_WORD.finditer(text):
            begin = bisect.bisect_left(offsets, word.start())
            if begin >= len(offsets):
                continue
            for form, key, language in sorted(self.marked_aliases.get(normalize(word.group()), ()), key=lambda k: (-len(k[0]), k[0] != k[1])):
                finish = begin + len(form)
                if folded.startswith(form, begin) and (finish == len(folded) or not folded[finish].isalnum()):
                    start, end = offsets[begin], offsets[finish - 1] + 1
                    if not any(start < x['end'] and end > x['start'] for x in rows):
                        rows.append({'text': text[start:end], 'start': start, 'end': end, 'lookup': key, 'method': 'MULTILINGUAL_NAME_PROPOSAL', 'language':
    language})
                    break
        return rows

    def mentions(self, text, language='en', section_subject=False):
        normalized, offsets = normalized_offsets(text)
        hits = []
        proposals = []
        for token in re.finditer(r'\w+', normalized):
```

```
            for key in self.alias_prefixes.get(token.group(), []):
                end = token.start() + len(key)
                if normalized.startswith(key, token.start()) and (end == len(normalized) or not re.match(r'\w', normalized[end])):
                    proposals.append((token.start(), end, key))
            # An English plural still explicitly names its subject: "Raw
            # Onions" must not fall back to a chapter about garlic, leek and onion.
            # Only exact -s forms are recovered; no plural-to-species fuzzy guess.
            singular = token.group()[:-1] if language == 'en' and token.group().endswith('s') else None
            if singular and singular in self.alias_taxa and len(singular) >= 4:
                proposals.append((token.start(), token.end(), singular))
        words = list(re.finditer(r'\w+', normalized))
        for first, second in zip(words, words[1:]):
            joined = first.group() + second.group()
            if second.start() - first.end() == 1 and joined in self.alias_taxa and len(joined) >= 7:
                proposals.append((first.start(), second.end(), joined))
        for begin, finish, key in sorted(set(proposals), key=lambda x: (-(x[1]-x[0]), x[0])):
            start, end = offsets[begin], offsets[finish-1]+1
            # A known word inside an unknown hyphenated plant name is not the
            # same entity (e.g. willow-herb versus willow; wood-sage versus sage).
            compound = (end < len(text)-1 and text[end] in "-\u2013" and text[end+1].isalpha()) or (start > 1 and text[start-1] in "-\u2013" and
    text[start-2].isalpha())
            if compound and not PLANT_PART_SUFFIX.match(text[end:end + 12]):
                continue
            if not self.sense_ok(text, start, end, key):
                continue
            if not any(start < x["end"] and end > x["start"] for x in hits):
                hits.append({"text": text[start:end], "start": start, "end": end, **({'lookup': key} if ' ' not in key and key != normalize(text[start:end]) else
    {})})
        if language != 'en':
            for row in self.foreign_mentions(text, language):
                if not any(row['start'] < x['end'] and row['end'] > x['start'] for x in hits):
                    hits.append(row)
        else:
            for row in self.marked_mentions(text):
                if not any(row['start'] < x['end'] and row['end'] > x['start'] for x in hits):
                    hits.append(row)
        # Latin binomials can be proposed even outside the snapshot and then abstained.
        # Exact fallback has no cross-language fuzzy or inflection guessing.
        for token in words if not hits else ():
            for key, alias_language in self.foreign_exact.get(token.group(), ()):
                finish = token.start() + len(key)
                if normalized.startswith(key, token.start()) and (finish == len(normalized) or not normalized[finish].isalnum()):
                    start, end = offsets[token.start()], offsets[finish - 1] + 1
                    if self.sense_ok(text, start, end, key) and not any(start < x['end'] and end > x['start'] for x in hits):
                        hits.append({'text': text[start:end], 'start': start, 'end': end, 'lookup': key, 'method': 'CROSS_LANGUAGE_EXACT_NAME', 'language':
    alias_language})
        for m, candidate in self.binomials(text):
            genus, epithet = candidate.split()[0], candidate.split()[-1]
            if normalize(genus) in PERSON_PLANT_NAMES and not self.sense_ok(text, m.start(), m.start() + len(genus), normalize(genus)):
                continue
            known = normalize(genus) in self.known_genera
            plural_word = epithet.endswith('s') and not epithet.endswith(('us', 'is', 'ans', 'ens', 'ides', 'ys', 'os', 'as'))
            if genus in NOT_GENUS or epithet in NOT_EPITHET or plural_word or not LATIN_EPITHET.search(epithet):
                continue
            # "Fennell seede", "Sage leaues": an unknown genus followed by a part of a plant is English.
            if not known and OLD_PART.fullmatch(epithet.casefold()):
                continue
            # "Mio fratello", "Mijn broer": outside English a capital opening a sentence is no genus,
```

```
            # and German capitalises every noun ("Morgen zur Schule"): an unknown epithet needs four letters.
            if not known and language != 'en' and (re.search(r'(?:^|[.!?;:]\s*)$', text[:m.start()]) or len(epithet) < 4 or epithet.casefold() in FUNCTION_EPITHETS):
                continue
            # A recognised genus must not swallow an unknown full binomial.
            contained = [x for x in hits if m.start() <= x['start'] and x['end'] <= m.end()]
            if contained and any(x['start'] < m.start() or x['end'] > m.end() for x in hits if m.start() < x['end'] and m.end() > x['start']):
                continue
            # "Listovi bokvice" (plantain leaves) and "Horehound drunke" have a binomial's shape; an unknown genus keeps the known name.
            if not known and (contained or normalize(epithet) in self.known_genera):
                continue
            hits = [x for x in hits if x not in contained]
            if not any(m.start() < x['end'] and m.end() > x['start'] for x in hits):
                hits.append({'text': m.group(), 'start': m.start(), 'end': m.end(), 'method': 'SCIENTIFIC_BINOMIAL_PROPOSAL' if known else
    'UNVERIFIED_BINOMIAL_PROPOSAL', **({'lookup': candidate} if candidate != m.group() else {})})
        # Genus spelling is case-insensitive; a lower-case genus also needs
        # botanical context. It proposes the group and does not settle a species.
        genera = {normalize(name): name for name in self.genus_names}
        for m in re.finditer(r"\b[A-Za-z]{5,}\b", text):
            key = normalize(m.group())
            if key in genera and genera[key] not in NOT_GENUS and (m.group()[0].isupper() or BOTANICAL_NEIGHBOUR.search(text[max(0, m.start()-40):m.end()+32])) and
    self.sense_ok(text, m.start(), m.end(), key) and not any(m.start() < x['end'] and m.end() > x['start'] for x in hits):
                hits.append({'text': m.group(), 'start': m.start(), 'end': m.end(), 'lookup': genera[key], 'method': 'GENUS_NAME_PROPOSAL'})
        # Abbreviated genus must be resolved against candidate genera, never silently expanded.
        for m in re.finditer(r"\b[A-Z]\.\s+[a-z]{3,}\b", text):
            if m.group().split()[-1] not in NOT_EPITHET and not any(m.start() < x["end"] and m.end() > x["start"] for x in hits):
                hits.append({"text": m.group(), "start": m.start(), "end": m.end()})
        # Propose parenthesised Latin-looking binomials outside the 24-taxon list.
        # They undergo exact reconciliation only, never fuzzy identity assignment.
        for m in re.finditer(r"\(\s*([A-Z][a-z]{2,}\s+(?:[x\xd7]\s+)?[a-z]{3,}(?:-[a-z]+)*)\s*\)", text):
            start, end = m.span(1)
            if m[1].split()[0] in NOT_GENUS:
                continue
            if not any(start < h['end'] and end > h['start'] for h in hits):
                hits.append({'text': m[1], 'start': start, 'end': end, 'method': 'PARENTHETICAL_BINOMIAL_PROPOSAL'})
        # "\u0427\u0430\u0439 \u043e\u0442 \u043b\u0430\u0439\u043a\u0430", "ch\xe1 de erva-cidreira", "Der Thee von Pfefferminze": the drink, made of the other
    plant.
        drinks = [x for x in hits if normalize(x['text']) in TEA_WORDS]
        for drink in drinks:
            if not re.match(r'\s+(?:plant|bush|shrub|tree)\b', text[drink['end']:], re.I):
                hits.remove(drink)
        # "Brinove jagode" is juniper berries: after a plant's possessive adjective the noun is its part.
        if language in CASE_LANGUAGES:
            adjectives = [x for x in hits if x.get('method') == 'MULTILINGUAL_NAME_PROPOSAL' and re.search(r'(?:ov|ev|in)[aeiou]?$', normalize(x['text']))]
            hits = [x for x in hits if not (x.get('method') == 'MULTILINGUAL_NAME_PROPOSAL' and any(a is not x and a['end'] < x['start'] and not
    text[a['end']:x['start']].strip() for a in adjectives))]
        hits += self.spelling_proposals(text, hits) if language == 'en' else []
        hits += self.position_proposals(text, hits) if language == 'en' and not section_subject else []
        hits += self.list_proposals(text, hits) if not section_subject else []
        # Proposal routes obey the same sense rules as the main gazetteer.
        # Otherwise a rejected person name can reappear via spelling recovery.
        hits = [x for x in hits if normalize(x['text']) not in {'herbs', 'plants', 'flowers', 'trees'} and self.sense_ok(text, x['start'], x['end'],
    normalize(x.get('lookup') or x['text'])) and (normalize(x['text']) not in TEA_WORDS or re.match(r'\s+(?:plant|bush|shrub|tree)\b', text[x['end']:], re.I))]
        return [{"id": f"mention-{i}", "method": x.get('method', self.version), **{k: v for k, v in x.items() if k != 'method'}, "normalized":
    normalize(x.get('lookup') or x["text"])} for i, x in enumerate(sorted(hits, key=lambda x: x["start"]))]

    def binomials(self, text):
        for m in LATIN_BINOMIAL.finditer(text):
            yield m, m.group()
```

```
        for m in CAPITAL_BINOMIAL.finditer(text):
            alone = re.fullmatch(r'[ \t]*', text[text.rfind('\n', 0, m.start()) + 1:m.start()]) and re.match(r'[ \t]*\.?[ \t]*(?:\n|$)', text[m.end():])
            if normalize(m[1]) in self.known_genera or alone or AUTHOR_AFTER.match(text, m.end()):
                yield m, f'{m[1].capitalize()} {m[2].lower()}'

    def list_proposals(self, text, hits):
        """Listed names that no known name already covers."""
        rows = []
        for row in sorted(name_list_rows(text), key=lambda r: r['start']):
            same = next((x for x in hits if (x['start'], x['end']) == (row['start'], row['end'])), None)
            if same is not None:
                # A known name inside a list still belongs to that entry when the reader is shown it.
                same.setdefault('listed_in', row['list_id'])
            if not any(row['start'] < x['end'] and row['end'] > x['start'] for x in hits + rows):
                rows.append(row)
        return rows

    def labelled_entries(self, text, mentions, blocks):
        """A "Uses.--" paragraph speaks of the plant the entry opened with."""
        found = []
        floor = 0
        for label in USE_LABEL.finditer(text):
            if any(b['start'] <= label.start() < b['body_end'] for b in blocks):
                continue
            head = text.rfind('\n\n', floor, max(label.start() - 1, 0))
            head = max(text.rfind('\n\n', floor, head) if head > floor else head, floor)
            names = [m for m in mentions if head <= m['start'] < label.start()]
            # The first Latin name is the entry; those after it are the author's synonyms.
            first = next((m for m in names if 'BINOMIAL' in m['method']), None)
            subjects = ([first] if first else []) + [m for m in names if 'BINOMIAL' not in m['method']]
            floor = text.find('\n\n', label.end())
            floor = len(text) if floor < 0 else floor
            if subjects:
                found.append({'heading': subjects[0]['text'], 'start': subjects[0]['start'], 'end': subjects[0]['end'], 'subjects': subjects, 'labelled': True,
    'names': {m['normalized'] for m in subjects}, 'taxa': set().union(set(), *(self.alias_taxa.get(normalize(f), set()) for m in subjects for f in m.get('forms',
    [m['normalized']])))})
        for index, block in enumerate(found):
            block['body_end'] = found[index + 1]['start'] if index + 1 < len(found) else len(text)
        return found

    def position_proposals(self, text, hits):
        """Unknown words in a plant position; only GBIF can turn one into a candidate."""
        rows = []
        for pattern in CUE_NAME:
            for m in pattern.finditer(text):
                word = m.group(1)
                key = word.casefold()
                last = key.split()[-1]
                if key in CUE_STOP or last in CUE_STOP or key in NEVER_A_PLANT or last in NEVER_A_PLANT or key in SENSE_REQUIRED or word in NOT_GENUS or key in
    LOWERCASE_FUNCTION or key.endswith(('ing', 'ed', 'ly')) or key.split()[0] in NOT_A_NAME_START:
                    continue
                if not any(m.start(1) < x['end'] and m.end(1) > x['start'] for x in hits + rows):
                    rows.append({'text': word, 'start': m.start(1), 'end': m.end(1), 'method': 'PLANT_POSITION_PROPOSAL'})
        return rows

    def spelling_proposals(self, text, hits):
        """Early-modern spelling: only a word in a plant position may be matched approximately."""
        rows = []
```

```
        for m in re.finditer(r"\b[A-Za-z]{5,}\b", text):
            if any(m.start() < x['end'] and m.end() > x['start'] for x in hits + rows):
                continue
            before, after = text[max(0, m.start() - 26):m.start()], text[m.end():m.end() + 16]
            capitalised = m.group()[0].isupper() and not re.search(r"(?:^|[.!?;:\n\"\u201c]\s*)$", before)
            if not (CUE_BEFORE.search(before) or CUE_AFTER.match(after) or capitalised):
                continue
            key = normalize(m.group())
            if key in CUE_STOP or key in LOWERCASE_FUNCTION or key in NEVER_A_PLANT or key in SENSE_REQUIRED or m.group() in NOT_GENUS or key.endswith(('ed',
    'ing')):
                continue
            match = process.extractOne(key, self.single_word_vernaculars, scorer=fuzz.ratio, score_cutoff=84)
            if match and match[0][0] == key[0]:
                rows.append({'text': m.group(), 'start': m.start(), 'end': m.end(), 'lookup': match[0], 'method': 'APPROXIMATE_SPELLING_PROPOSAL'})
        return rows

    def _taxa(self):
        # Supplied by Pipeline after construction; kept explicitly for unknown-binomial proposals.
        return getattr(self, "taxa", [])

    def heading_mentions(self, text, section):
        """Plant names a heading announces, each spanning its own heading wording."""
        rows = []
        line = text[section['start']:section['end']]
        names = heading_names(line)
        for name in names:
            specific = any(other is not name and len(other['name'].split()) > len(name['name'].split()) and
    other['name'].casefold().endswith(name['name'].casefold()) for other in names)
            if specific:
                continue
            start = section['start'] + max(name['start'], 0)
            end = min(section['end'], start + len(name['name']))
            if not text[start:end].strip() or any(start < r['end'] and end > r['start'] for r in rows):
                free = next(((m.start() + section['start'], m.end() + section['start']) for m in re.finditer(r'[^\s,().]+', line) if not any(m.start() +
    section['start'] < r['end'] and m.end() + section['start'] > r['start'] for r in rows)), None)
                if not free:
                    continue
                start, end = free
            rows.append({'text': text[start:end], 'start': start, 'end': end, 'normalized': normalize(name['name']), 'heading_name': name['name'], 'forms':
    name['forms'], 'method': 'SOURCE_HEADING_SUBJECT'})
        return rows

    def condition_rows(self, clause, language, fallback_languages=()):
        rows = []
        if language == 'en':
            for condition, terms in CONDITIONS.items():
                for span in spans(clause["text"], terms):
                    if condition_sense(condition, span['text'], clause['text']):
                        rows.append((condition, span))
        else:
            found = [(condition, span) for condition, table in LANGUAGE_DATA['conditions'].items() for span in stem_spans(clause["text"], table.get(language, []))]
            # "Sore throat" also starts with the word for pain; the longer wording decides, then the
            # longer stem: "\u03c0\u03bf\u03bd\u03bf\u03ba\u03ad\u03c6\u03b1\u03bb\u03bf" is a headache before it is a "\u03c0\u03cc\u03bd\u03bf".
            for condition, span in sorted(found, key=lambda row: (row[1]['start'] - row[1]['end'], -row[1]['stem'])):
                if not any(span['start'] < other['end'] and span['end'] > other['start'] for _, other in rows):
                    rows.append((condition, span))
        if not rows:
            for fallback in dict.fromkeys(fallback_languages):
```

```
                if fallback != language:
                    found = self.condition_rows(clause, fallback)
                    if found:
                        return found
        return [{"id": f"condition:{condition}", "label": condition.replace("_", " "), "mapping_status": "LEXICAL_RETRIEVAL_AID_NOT_DIAGNOSTIC_EQUIVALENCE",
    "original": span["text"], "start": span["start"] + clause["start"], "end": span["end"] + clause["start"]} for condition, span in rows]

    def extract(self, text, **options):
        result = self.read_text(unwrap(EMPHASIS.sub(lambda m: ' ' * len(m.group()), text)), **options)
        for claim in result['historical_claims']:
            claim['support'] = {**claim['support'], 'text': text[claim['support']['start']:claim['support']['end']]}
        return result

    def read_text(self, text, *, section_subject=False, requested_name=None, language=None):
        language = language or detect_language(text)
        mentions = self.mentions(text, language, section_subject)
        if section_subject:
            # Without GBIF at hand a Latin-looking pair needs a known genus or the author's brackets.
            mentions = [m for m in mentions if m['method'] != 'UNVERIFIED_BINOMIAL_PROPOSAL']
        if requested_name is not None:
            requested_name = requested_name.strip()
            if not 2 <= len(requested_name) <= 120:
                raise ValueError('Choose a name of 2\u2013120 characters from the source')
            matches = list(re.finditer(r'(?<!\w)' + re.escape(requested_name) + r'(?!\w)', text, re.I))
            if not matches:
                raise ValueError('This name does not occur in the source. Enter the original spelling, including spaces and punctuation.')
            # User-selected source wording is explicit provenance, not an alias
            # learned by the extractor or a verified botanical identification.
            mentions = [m for m in mentions if not any(hit.start() < m['end'] and hit.end() > m['start'] for hit in matches)]
            mentions += [{'text': hit.group(), 'start': hit.start(), 'end': hit.end(), 'normalized': normalize(hit.group()), 'method': 'RESEARCHER_SELECTED_NAME'}
    for hit in matches]
        heading_end = text.find('\n')
        if section_subject:
            blocks = [{'heading': text[:heading_end], 'start': 0, 'end': heading_end, 'body_end': len(text)}] if heading_end > 0 else []
        else:
            blocks = sections(text)
            # "Chamaemelum: the flowers are heating" announces its subject inline.
            for match in LEADING_NAME.finditer(text):
                if match[1].split()[0] not in NOT_GENUS:
                    blocks.append({'heading': match[1], 'start': match.start(1), 'end': match.end(1), 'inline': True})
            # "258. +N\xedm Tree or Margosa.+ The leaves...": a numbered entry opens with its name in bold.
            for match in BOLD_ENTRY.finditer(text):
                if not any(b['start'] <= match.start('name') < b['end'] for b in blocks):
                    blocks.append({'heading': match['name'], 'start': match.start('name'), 'end': match.end('name'), 'inline': True})
            blocks.sort(key=lambda b: b['start'])
            for index, block in enumerate(blocks):
                block['body_end'] = blocks[index + 1]['start'] if index + 1 < len(blocks) else len(text)
        for block in blocks:
            announced = self.heading_mentions(text, block)
            if not section_subject:
                # An inferred heading keeps recognised names; unknown words are not promoted.
                inside = [m for m in mentions if block['start'] <= m['start'] < block['end']]
                known = [m for m in announced if any(normalize(f) in self.alias_taxa for f in m['forms'])]
                if BINOMIAL_LINE.match(text[block['start']:block['end']]) and any('BINOMIAL' in m['method'] for m in inside):
                    announced = inside
                elif known:
                    announced = known
                else:
```

```
                    # "May-Apple." may only be known to GBIF; there a rejected word leaves no trace.
                    wording = text[block['start']:block['end']].rstrip(' .,')
                    whole = [] if block.get('inline') or wording.casefold() in CUE_STOP or any(m['start'] == block['start'] and m['end'] >= block['start'] + len(wording) for m in inside) else [{'id': 'heading', 'text': wording, 'start': block['start'], 'end': block['start'] + len(wording), 'normalized': normalize(wording), 'method': 'PLANT_POSITION_PROPOSAL', 'heading_line': True}]
                    announced = whole + inside
            mentions = [m for m in mentions if not (block['start'] <= m['start'] < block['end'])] + announced
        mentions = [{**m, 'id': f'mention-{i}'} for i, m in enumerate(sorted(mentions, key=lambda m: m['start']))]
        for index, block in enumerate(blocks):
            block['subjects'] = [m for m in mentions if block['start'] <= m['start'] < block['end']]
            # A common name, then its Latin name on the next line: one entry, one subject.
            if index and not text[blocks[index - 1]['end']:block['start']].strip():
                block['subjects'] = blocks[index - 1]['subjects'] + block['subjects']
            block['names'] = {m['normalized'] for m in block['subjects']}
            block['taxa'] = set().union(set(), *(self.alias_taxa.get(normalize(f), set()) for m in block['subjects'] for f in m.get('forms', [m['normalized']])))
        use_paragraphs = []
        if not section_subject:
            blocks = sorted(blocks + self.labelled_entries(text, mentions, blocks), key=lambda b: b['start'])
            # Under an author's own "Uses" label a topic word is a use, whatever the verb.
            use_paragraphs = [(m.start(), (text.find(chr(10) * 2, m.end()) + 1 or len(text) + 1) - 1) for m in USE_LABEL.finditer(text)]
        paragraphs, begin = [], 0
        for boundary in list(re.finditer(r'\n\s*\n', text)) + [None]:
            end = boundary.start() if boundary else len(text)
            paragraphs.append((begin, end))
            begin = boundary.end() if boundary else len(text)
        # "Bhind\xed, R\xe1m-tur\xe1\xed (_Hind._) \u2026" on its own, then a paragraph of uses that never repeats
        # the name: a short list that records no use is the entry the next paragraph describes.
        listed = {}
        for before, after in zip(paragraphs, paragraphs[1:]):
            names = [m for m in mentions if before[0] <= m['start'] < before[1]]
            residue = list(text[before[0]:before[1]])
            for m in names:
                residue[m['start'] - before[0]:m['end'] - before[0]] = ' ' * (m['end'] - m['start'])
            # Only names, language tags and punctuation: "Sage grows in every garden" is not a list.
            residue = re.sub(r'_[^_]{1,12}_|\([^)]{0,30}\)|\b(?:or|and)\b', ' ', ''.join(residue))
            if names and before[1] - before[0] <= 400 and len(re.findall(r'\w{3,}', residue)) <= 2 * len(names) \
                    and not re.search(r'\b(?:the|an?|is|are|was|were|be|in|of|for|with|by|to|at|on|it|this|that|which|from|grows?|grown|used|called|known)\b', residue, re.I) \
                    and not any(after[0] <= m['start'] < after[1] for m in mentions):
                listed[after] = names
        result, recent = [], []
        for clause in sentences(text):
            block = next((b for b in reversed(blocks) if b['start'] <= clause['start'] < b['body_end']), None)
            subject = block['subjects'] if block else []
            paragraph = next((p for p in paragraphs if p[0] <= clause['start'] <= p[1]), None)
            nearby = [m for m in mentions if m["start"] < clause["end"] and m["end"] > clause["start"]]
            named = [m for m in nearby if m['method'] != 'SOURCE_HEADING_SUBJECT']
            explicit_recent = [m for start, group in recent if paragraph and start >= paragraph[0] for m in group][-4:]
            conditions = self.condition_rows(clause, language, [m['language'] for m in mentions if m.get('language') and m['start'] < clause['end'] and m['end'] > clause['start']])
            if named:
                recent = (recent + [(clause['start'], named)])[-2:]
            if not conditions:
                continue
            scope = "SENTENCE"
            possible = []
            if not nearby and subject:
                # Coreference window: only a plant named in the last two sentences of this
```

```
                # paragraph competes with the chapter subject. A distant mention does not.
                rival = [m for m in explicit_recent if m['normalized'] not in block['names'] and not (self.alias_taxa.get(m['normalized'], set()) & block['taxa'])]
                nearby = subject
                scope = "PARAGRAPH_CONTEXT_UNCERTAIN_RELATION" if rival else "SOURCE_SECTION_HEADING"
                possible = sorted({m['text'] for m in rival})
            elif subject and nearby and not any(m in subject for m in nearby) and all(INGREDIENT_POSITION.search(text[max(0, m['start'] - 30):m['start']]) for m in nearby):
                nearby = subject + nearby
                scope = "MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION"
            elif not nearby and explicit_recent:
                nearby = explicit_recent[-1:]
                scope = "PRECEDING_SENTENCE_SUBJECT"
            elif not nearby and paragraph in listed:
                nearby = listed[paragraph]
                scope = "PRECEDING_NAME_LIST"
            if not nearby:
                continue
            # Explicitly retain multi-plant ambiguity instead of attaching the whole clause to all.
            if scope == "SENTENCE" and len({m["normalized"] for m in nearby}) > 1:
                scope = "MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION"
            fields = {}
            preparation_terms = PREPARATIONS if language == 'en' else LANGUAGE_DATA['preparations'].get(language, [])
            for field, terms in [("part", PARTS if language == 'en' else []), ("preparation", preparation_terms), ("route", ROUTES if language == 'en' else [])]:
                values = spans(clause["text"], terms) if language == 'en' else stem_spans(clause["text"], terms)
                fields[field] = [{"text": v["text"], "start": v["start"] + clause["start"], "end": v["end"] + clause["start"]} for v in values]
            dosage = []
            for m in re.finditer(r"\b(?:half|one|two|three|four|five|six|ten|twenty|thirty|\d+)\s+(?:to\s+\w+\s+)?(?:drams?|drachms?|ounces?|grains?|drops?|spoonfuls?|pints?|teaspoonfuls?|tablespoonfuls?)\b", clause["text"], re.I):
                dosage.append({"text": m.group(), "start": clause["start"] + m.start(), "end": clause["start"] + m.end()})
            role = statement_role(clause['text']) if language == 'en' else 'THERAPEUTIC_USE_RECORDED'
            if role == 'CONTEXT_ONLY_OR_UNCLEAR' and any(begin <= clause['start'] < end for begin, end in use_paragraphs):
                role = 'THERAPEUTIC_USE_RECORDED'
            if language == 'en' and role == 'CONTEXT_ONLY_OR_UNCLEAR' and CONTINUATION.match(clause['text']) and result and result[-1]['statement_role'] == 'THERAPEUTIC_USE_RECORDED' and clause['start'] - result[-1]['support']['end'] < 4:
                role = 'THERAPEUTIC_USE_RECORDED'
            result.append({"id": f"historical-claim-{len(result)}", "type": "HISTORICAL_ATTESTATION", "statement_role": role, "role_method": "LEXICAL_DISCOURSE_SCREEN_NOT_ENTAILMENT" if language == 'en' else "LANGUAGE_LEXICON_SCREEN_NOT_ENTAILMENT", "practice": practice_kind(clause['text']) if language == 'en' else 'MEDICINAL', "mention_ids": [m["id"] for m in nearby], "relation_scope": scope, "possible_subjects": possible, "support": clause, "conditions": conditions, **fields, "dosage_expressions": dosage, "negated": bool(DENIED.search(clause["text"])), "human_reviewed": False})
        return {"mentions": mentions, "historical_claims": result, "language": language}
```

src/nlp/headings.py
SHA-256: 0ea8b3179f2e6543ffca34cd3c5dabc1685f1e1935bd5d1dc8a4da9376df30b6

```
"""Split a herbal chapter heading into the plant names it actually announces.

Headings pack several shapes into one line: page markers, botanical family glosses,
synonyms, qualifiers and several plants. Every returned name keeps the span of the
heading wording it came from, so a resolved name never loses its source position.
"""
from __future__ import annotations

import re

QUALIFIERS = {'white', 'black', 'greater', 'lesser', 'great', 'little', 'small', 'horse', 'sweet', 'meadow', 'cultivated', 'common', 'wood', 'wild', 'ground',
   'garden', 'field', 'marsh', 'water', 'sea', 'red', 'yellow', 'deadly', 'english', 'dwarf', 'stinking', 'bitter', 'true', 'dog', "dog's"}
NOT_PLANT = {'other', 'the', 'and', 'or', 'also', 'called', 'see', 'a', 'an', 'of', 'tree', 'plants', 'flowers', 'root', 'radix', 'herb', 'herbs', 'sea', 'water'}
DROP_GLOSS = re.compile(r'^(?:[a-z]+(?:aceoe|aceae|iferae|iferoe|ceoe|acoe)|leguminous|labiatoe|compositoe|umbelliferoe|radix.*|a root|see also.*|the|other)$', re.I)
NOT_A_PLANT_NAME = re.compile(r'^(?:flowers? of|sea plants|sea weeds|water plants)$', re.I)


def singular(word):
    lower = word.casefold()
    if lower.endswith('ies') and len(lower) > 5:
        return word[:-3] + 'y'
    if lower.endswith(('ches', 'shes', 'sses', 'xes')) and len(lower) > 5:
        return word[:-2]
    if lower.endswith('s') and not lower.endswith(('ss', 'us', 'is')) and len(lower) > 4:
        return word[:-1]
    return word


def forms(name):
    """Orthographic lookup variants; none of them asserts an identity."""
    clean = ' '.join(name.replace('\u2019', "'").split())
    out = [clean, clean.replace('-', ' '), clean.replace('-', ''), clean.replace(' ', ''), re.sub(r"^(?:the|a|an)\s+", '', clean, flags=re.I)]
    words = clean.split()
    out.append(' '.join(words[:-1] + [singular(words[-1])]))
    out.append(' '.join(out[-1].split()).replace('-', ' '))
    out.append(re.sub(r"\s+tree$", '', clean, flags=re.I))
    out.append(re.sub(r"^st\.?\s+", "st. ", clean, flags=re.I))
    if re.match(r"john'?s wort", clean, re.I):
        out += ["st. john's wort", "st john's wort"]
    seen, unique = set(), []
    for value in out:
        key = value.casefold().strip()
        if key and key not in seen:
            seen.add(key)
            unique.append(value.strip())
    return unique


def _add(found, text, start, name):
    name = re.sub(r"^(?:the|a|an)\s+", '', ' '.join(name.split()), flags=re.I).strip(" ,.;:\u2014-")
    name = re.sub(r"^other\s+", '', name, flags=re.I)
    if len(name) < 3 or name.casefold() in NOT_PLANT or NOT_A_PLANT_NAME.fullmatch(name):
        return
    if any(name.casefold() == row['name'].casefold() for row in found):
        return
```

```
    found.append({'name': name, 'start': start, 'end': min(len(text), start + len(name)), 'forms': forms(name)})


def heading_names(heading):
    """Return plant names announced by a heading, in reading order."""
    text = heading
    line = re.sub(r'\[\d+\]', lambda m: ' ' * len(m.group()), heading).rstrip()
    line = line.rstrip('. ')
    found = []
    groups = list(re.finditer(r'\(([^)]*)\)', line))
    base = re.sub(r'\([^)]*\)', lambda m: ' ' * len(m.group()), line)
    head_offset = len(base) - len(base.lstrip())
    base_clean = base.strip().rstrip(' .,\u2014-')
    parts = [p for p in re.split(r'\s*(?:,|\u2014|\band\b|\bAND\b|\bor\b|\bOR\b|\bcalled\b|\bCALLED\b)\s*', base_clean) if p.strip()]
    # "NIGHT SHADE, DEADLY" and "IVY, Common": a trailing qualifier belongs to the plant before it.
    qualified = []
    for part in parts:
        part = re.sub(r'^the\s+', '', part, flags=re.I)
        if qualified and part.casefold().split()[-1] in QUALIFIERS and len(part.split()) == 1:
            qualified[-1] = f'{part} {qualified[-1]}'
        else:
            qualified.append(part)
    # "MERCURY-DOG'S" inverts the plant and its qualifier.
    inverted = []
    for part in qualified:
        match = re.fullmatch(r"([A-Za-z]+)-([A-Za-z]+'[sS])", part)
        inverted.append(f'{match[2]} {match[1]}' if match else part)
    head = inverted[0] if inverted else ''
    for part in inverted:
        index = text.casefold().find(part.split()[-1].casefold())
        _add(found, text, max(index, head_offset), part)
    for group in groups:
        items = [p.strip() for p in re.split(r'\s*(?:,|\band\b|\bor\b|\bAND\b|\bOR\b)\s*', group[1]) if p.strip()]
        for item in items:
            item = re.sub(r'^(?:also called|called|the|The)\s+', '', item).strip(' .')
            if not item or DROP_GLOSS.fullmatch(item):
                continue
            start = text.find(item)
            words = item.casefold().split()
            if len(words) == 1 and words[0] in QUALIFIERS:
                for row in list(found):
                    if len(row['name'].split()) == 1:
                        _add(found, text, max(start, 0), f"{item} {row['name']}")
                        # A qualified entry is more specific than its bare head.
                continue
            _add(found, text, max(start, 0), item)
    return found
```

src/pipeline.py
SHA-256: dfedee439265d5c50de41534505e77cdb51dceb7e14da65ae8d8c15e630d889f

```python
from __future__ import annotations

from collections import Counter, defaultdict
import json
import re
import os
import time
from urllib.parse import quote
from src import __version__
from src.common import DATA, ROOT, digest, now, read_json
from src.nlp import Extractor
from src.nlp.extract import heading_gloss, heading_common_names, normalize, sentences, source_glosses
from src.taxonomy import Resolver
from src.retrieval.index import LiteratureIndex, design_rank
from src.evidence.classify import classify_study, relation_to_claim, citation_check, conflict_groups
from src.evidence.model import StudyModel
from src.evidence.comparability import comparability, gate_relation, summarize
from src.evidence.context import scope_study_preparation, index_historical_contexts, compare_historical_contexts, preparation_across_statements
from src.evidence.historical_names import historical_name_references
from src.evidence.counterfactual import counterfactuals
from src.evidence.chemistry import assay_leads
from src.evidence.identity_matrix import identity_matrix
from src.evidence.digest import research_digest
from src.evidence.regulatory import regulatory
from src.evidence.independent_use import independent_use
from src.ingestion.client import ScientificClient
from src.taxonomy.lexicon import fold, local_names
from src.taxonomy.open_world import reconcile_common_name, scientific_readings, on_demand_literature, taxon_from_record
from src.evidence.readings import active_reading_sets
from src.graph.store import GraphStore


def mention_topics(extracted, mention):
    """What the text says this named plant was used for."""
    return {c['id'] for claim in extracted['historical_claims'] if mention['id'] in claim['mention_ids'] and claim['statement_role'] == 'THERAPEUTIC_USE_RECORDED'
for c in claim['conditions']}


class Pipeline:
    def __init__(self, *, data_dir=DATA, store=None, use_lexicon=True, include_local_abstracts=True):
        self.data_dir = data_dir
        # Public/demo builds must never inherit a developer's private abstract cache.
        self.include_local_abstracts = include_local_abstracts
        self.use_lexicon = use_lexicon
        self.manifest = read_json(data_dir / "acquisition.json")
        self.snapshot = digest((data_dir / "acquisition.json").read_bytes())
        for file in ("passages.json", "taxa.json", "papers.json", "compounds.json"):
            if self.manifest.get("files", {}).get(file, {}).get("sha256") != digest((data_dir / file).read_bytes()):
                raise ValueError(f"Snapshot checksum absent or mismatched: {file}")
        self.passages = read_json(data_dir / "passages.json")
        self.taxa = read_json(data_dir / "taxa.json")
        self.base_taxa_count = len(self.taxa)
        self.taxonomy_observations = []
        expansion_path = data_dir / 'taxa_extended.json'
        if expansion_path.exists():
```

```
            lock = read_json(ROOT / 'data/taxonomy_expansion.lock.json')
            if digest(expansion_path.read_bytes()) != lock['sha256']:
                raise ValueError('Extended scientific taxonomy snapshot checksum mismatch')
            expansion = read_json(expansion_path)
            original_ids = {t['id'] for t in self.taxa}
            self.taxa += [t for t in expansion['taxa'] if t['id'] not in original_ids]
            self.taxonomy_observations = expansion['observations']
            self.snapshot = digest(self.snapshot + lock['sha256'])
        self.load_core_taxa(data_dir)
        self.literature_size = {}
        self.papers = read_json(data_dir / "papers.json")
        extended_papers = data_dir / 'papers_extended.json'
        if extended_papers.exists():
            lock = read_json(ROOT / 'data/literature_expansion.lock.json')
            if digest(extended_papers.read_bytes()) != lock['sha256']:
                raise ValueError('Extended literature snapshot checksum mismatch')
            ids = {p['id'] for p in self.papers}
            self.papers += [p for p in read_json(extended_papers)['papers'] if p['id'] not in ids]
            self.snapshot = digest(self.snapshot + lock['sha256'])
        self.load_pubmed(data_dir)
        methods_path = data_dir / 'study_methods.json'
        if methods_path.exists():
            methods_lock = read_json(ROOT / 'data/study_methods.lock.json')
            if digest(methods_path.read_bytes()) != methods_lock['sha256']:
                raise ValueError('Study Methods snapshot checksum mismatch')
            methods = read_json(methods_path)['papers']
            for paper in self.papers:
                if paper['id'] in methods:
                    paper['methods_contexts'] = methods[paper['id']]['contexts']
                    paper['reported_results'] = methods[paper['id']].get('reported_results', [])
            self.snapshot = digest(self.snapshot + methods_lock['sha256'])
        # Name lists and independent records change what a reader is told.
        for lock in ('duke_ethnobotany', 'duke_chemistry', 'merrill_1903_names', 'ema_herbal', 'local_names_wikidata'):
            path = ROOT / f'data/{lock}.lock.json'
            if path.exists():
                self.snapshot = digest(self.snapshot + read_json(path)['sha256'])
        self.trial_readings = []
        readings_path = data_dir / 'trial_readings.json'
        if readings_path.exists():
            readings_lock = read_json(ROOT / 'data/trial_readings.lock.json')
            if digest(readings_path.read_bytes()) != readings_lock['sha256']:
                raise ValueError('Trial reading notes snapshot checksum mismatch')
            self.trial_readings = read_json(readings_path)['sets']
            self.snapshot = digest(self.snapshot + readings_lock['sha256'])
        self.compounds = read_json(data_dir / "compounds.json")
        self.taxmap = {t["id"]: t for t in self.taxa}
        self.scientific_genera = {normalize(name).split()[0] for t in self.taxa for name in [t['name']] + [a['name'] for a in t.get('aliases', []) if a['kind'] in
    ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH')] if name.strip()}
        for taxon in self.taxa:
            taxon['drug_names'] = sorted(regulatory().drug_names.get(taxon['id'], ()))
        self.papermap = {p["id"]: p for p in self.papers}
        self.extra_passages, self.mechanism_records = [], []
        for filename, lockname in [('fernie_passages.json', 'second_corpus.lock.json'), ('mechanisms.json', 'mechanisms.lock.json')]:
            path = data_dir / filename
            if path.exists():
                lock = read_json(ROOT / 'data' / lockname)
                if digest(path.read_bytes()) != lock['sha256']:
                    raise ValueError('Optional snapshot checksum mismatch: ' + filename)
```

```
                self.snapshot = digest(self.snapshot + lock['sha256'])
                if filename == 'fernie_passages.json':
                    self.extra_passages = read_json(path)
                else:
                    self.mechanism_records = read_json(path)['records']
        congeners_file = data_dir / "vernacular_congeners.json"
        self.congeners = None
        if congeners_file.exists():
            if self.manifest["files"].get("vernacular_congeners.json", {}).get("sha256") != digest(congeners_file.read_bytes()):
                raise ValueError("Snapshot checksum absent or mismatched: vernacular_congeners.json")
            self.congeners = read_json(congeners_file)
        self.extractor = Extractor(self.taxa)
        self.study_designs = {}
        self.client = ScientificClient(offline=os.environ.get('FOLIATRACE_OFFLINE', '').lower() in {'1', 'true', 'yes'})
        self.descriptions = {}
        descriptions_path = data_dir / 'taxon_descriptions.json'
        if descriptions_path.exists():
            self.descriptions = read_json(descriptions_path)['descriptions']
            self.snapshot = digest(self.snapshot + digest(descriptions_path.read_bytes()))
        self.resolver = Resolver(self.taxa, self.congeners)
        self.index = LiteratureIndex(self.papers, self.taxa)
        self.topic_cache = {}
        self.study_model = StudyModel.load(ROOT / ".cache" / "study_model.joblib")
        # The reader's vocabulary is code too: a changed use-term table must not return a cached reading.
        self.code_fingerprint = digest(b"".join(path.read_bytes() for path in sorted((ROOT / "src").rglob("*.py"))) + (ROOT /
    'data/use_terms_i18n.json').read_bytes())
        model_path = ROOT / ".cache" / "study_model.joblib"
        encoder_manifest=ROOT/'data'/'models.lock.json'
        self.model_fingerprint = digest((model_path.read_bytes() if model_path.exists() else b'NOT_TRAINED')+(encoder_manifest.read_bytes() if
    encoder_manifest.exists() else b'NO_ENCODER_MANIFEST'))
        reranker_manifest = ROOT / 'data/reranker.lock.json'
        if reranker_manifest.exists():
            self.model_fingerprint = digest(self.model_fingerprint.encode() + reranker_manifest.read_bytes())
        self.withdrawals_path = ROOT / "data" / "withdrawals.json"
        self.store = store or GraphStore()

    def load_core_taxa(self, data_dir, derived=True):
        """Heading candidates, archaic names and the multilingual gazetteer, in that priority."""
        by_id = {t['id']: t for t in self.taxa}
        by_name = {t['name']: t for t in self.taxa}
        core_path = data_dir / 'taxa_core.json'
        if core_path.exists():
            lock = read_json(ROOT / 'data/core_taxa.lock.json')
            if digest(core_path.read_bytes()) != lock['sha256']:
                raise ValueError('Core taxonomy snapshot checksum mismatch')
            self.snapshot = digest(self.snapshot + lock['sha256'])
            for row in read_json(core_path)['taxa']:
                known = by_id.get(row['id']) or by_name.get(row['name'])
                if known:
                    names = {normalize(a['name']) for a in known['aliases']}
                    known['aliases'] = known['aliases'] + [a for a in row['aliases'] if normalize(a['name']) not in names]
                    known['occurrence_counts'] = row['occurrence_counts']
                else:
                    self.taxa.append(row)
                    by_id[row['id']] = by_name[row['name']] = row
        lexicon_path = ROOT / 'data/historical_names.json'
        if lexicon_path.exists() and self.use_lexicon:
            self.snapshot = digest(self.snapshot + digest(lexicon_path.read_bytes()))
```

```
        # Wikidata labels: species tagged as medicinal plants, then every EU monograph species and core species, then the
        # same species in more European languages; last, names field studies printed beside an identification.
        for name in ('taxa_gazetteer', 'regulator_names', 'core_species_names', 'more_language_names', 'field_study_names'):
            gazetteer_path, lock_path = data_dir / f'{name}.json', ROOT / 'data' / ('gazetteer.lock.json' if name == 'taxa_gazetteer' else f'{name}.lock.json')
            if not gazetteer_path.exists():
                continue
            lock = read_json(lock_path)
            if digest(gazetteer_path.read_bytes()) != lock['sha256']:
                raise ValueError(f'{gazetteer_path.name} snapshot checksum mismatch')
            self.snapshot = digest(self.snapshot + lock['sha256'])
            for row in read_json(gazetteer_path)['taxa']:
                url = f"https://www.wikidata.org/wiki/{row['wikidata']}" if row.get('wikidata') else f"https://www.gbif.org/species/{row['id'].split(':', 1)[1]}"
                names = [{'name': n['name'], 'kind': 'VERNACULAR', 'lang': n['lang'], 'url': n.get('url', url), **({'sources': n['sources']} if 'sources' in n else
    {})} for n in row['names'] if not n.get('everyday')]
                known = by_name.get(row['name']) or by_id.get(row['id'])
                if known:
                    present = {normalize(a['name']) for a in known['aliases']}
                    known['aliases'] = known['aliases'] + [n for n in names if normalize(n['name']) not in present]
                    continue
                if row['name'] in by_name:
                    continue
                taxon = {'id': row['id'], 'name': row['name'], 'scientific_name': row['name'], 'rank': 'SPECIES', 'family': 'Not in snapshot', 'genus':
    row['name'].split()[0], 'status': 'NOT_CHECKED', 'aliases': [{'name': row['name'], 'kind': 'ACCEPTED_NAME', 'url': url}] + names, 'licence': 'CC0-1.0' if
    row.get('wikidata') else 'Names from CC BY field studies; GBIF backbone', 'provenance': {'url': url}, 'gazetteer_only': True, 'scope': 'Wikidata medicinal-plant
    record; names are retrieval hypotheses.' if row.get('wikidata') else 'Local names a field study recorded; names are retrieval hypotheses.'}
                self.taxa.append(taxon)
                by_id[row['id']] = by_name[row['name']] = taxon
        redirects_path = data_dir / 'wikipedia_redirects.json'
        if redirects_path.exists():
            lock = read_json(ROOT / 'data/wikipedia_redirects.lock.json')
            if digest(redirects_path.read_bytes()) != lock['sha256']:
                raise ValueError('Wikipedia redirect snapshot checksum mismatch')
            self.snapshot = digest(self.snapshot + lock['sha256'])
            genera = defaultdict(set)
            for genus in {t['genus'] for t in self.taxa} | set(independent_use().genera):
                genera[fold(genus)].add(genus)
            for row in read_json(redirects_path)['names']:
                taxon = by_id.get(row['id'])
                # An everyday word ("Burgundy", a cultivar; "Robin") would turn prose into plants,
                # and a cultivar spelt like another genus ("Arnika", a sunflower) would misname it.
                if not taxon or row.get('everyday') or genera.get(fold(row['name']), {taxon['genus']}) - {taxon['genus']} or normalize(row['name']) in
    {normalize(a['name']) for a in taxon['aliases']}:
                    continue
                url = 'https://en.wikipedia.org/wiki/' + quote(row['article'].replace(' ', '_'))
                taxon['aliases'] = taxon['aliases'] + [{'name': row['name'], 'kind': row['kind'], 'lang': 'en', 'url': url}]
        everyday = ROOT / 'data/plant_names_i18n.json'
        if everyday.exists():
            self.snapshot = digest(self.snapshot + digest(everyday.read_bytes()))
            lexicon = read_json(everyday)
            for species, table in lexicon['names'].items():
                taxon = by_name.get(species)
                if not taxon and species in lexicon.get('gbif_keys', {}):
                    key = lexicon['gbif_keys'][species]
                    url = f'https://www.gbif.org/species/{key}'
                    taxon = {'id': f'gbif:{key}', 'name': species, 'scientific_name': species, 'rank': 'SPECIES', 'family': 'Not in snapshot', 'genus':
    species.split()[0], 'status': 'NOT_CHECKED', 'aliases': [{'name': species, 'kind': 'ACCEPTED_NAME', 'url': url}], 'licence': 'CC-BY-4.0', 'provenance': {'url': url},
     'gazetteer_only': True, 'scope': 'Everyday-name table; names are retrieval hypotheses.'}
                    self.taxa.append(taxon)
```

```
                    by_id[taxon['id']] = by_name[species] = taxon
                if not taxon:
                    continue
                present = {(normalize(a['name']), a.get('lang')) for a in taxon['aliases']}
                taxon['aliases'] = taxon['aliases'] + [{'name': name, 'kind': 'VERNACULAR', 'lang': language, 'url': 'data/plant_names_i18n.json'} for language,
    names in table.items() for name in names if (normalize(name), language) not in present]
        short_path = data_dir / 'short_names.json'
        if derived and short_path.exists():
            lock = read_json(ROOT / 'data/short_names.lock.json')
            if digest(short_path.read_bytes()) != lock['sha256']:
                raise ValueError('Short-name snapshot checksum mismatch')
            self.snapshot = digest(self.snapshot + lock['sha256'])
            for row in read_json(short_path)['names']:
                taxon = by_id.get(row['id'])
                if taxon and not row['everyday']:
                    taxon['aliases'] = taxon['aliases'] + [{'name': row['name'], 'kind': 'VERNACULAR', 'lang': row['lang'], 'url': row['url'], 'short_form_of':
    row['short_form_of']}]
        # Apply after every taxon source is loaded: historical trade names can
        # refer to species introduced by the global gazetteer, such as Cinchona.
        if lexicon_path.exists() and self.use_lexicon:
            for name, entry in read_json(lexicon_path)['names'].items():
                for species in entry['species']:
                    taxon = by_name.get(species)
                    if taxon and normalize(name) not in {normalize(a['name']) for a in taxon['aliases']}:
                        taxon['aliases'] = taxon['aliases'] + [{'name': name, 'kind': 'HISTORICAL_LEXICON', 'url': 'data/historical_names.json'}]

    def load_pubmed(self, data_dir):
        """Bibliographic records from the repository, abstracts from the local cache."""
        path = data_dir / 'papers_pubmed.json'
        if not path.exists():
            return
        lock = read_json(ROOT / 'data/pubmed.lock.json')
        if digest(path.read_bytes()) != lock['sha256']:
            raise ValueError('PubMed snapshot checksum mismatch')
        self.snapshot = digest(self.snapshot + lock['sha256'])
        local = ROOT / '.cache/literature/pubmed_abstracts.json'
        use_local = self.include_local_abstracts and local.exists()
        abstracts = read_json(local) if use_local else {}
        # Abstract availability/content changes the findings, even when the
        # bibliographic snapshot is unchanged. Never reuse a bibliographic-only
        # analysis after full abstracts have become available (or vice versa).
        self.snapshot = digest(self.snapshot + (digest(local.read_bytes()) if use_local else 'NO_LOCAL_PUBMED_ABSTRACTS'))
        present = {str(p.get('pmid')) for p in self.papers}
        snapshot = read_json(path)
        self.papers += [{**p, 'abstract': abstracts.get(p['pmid'], '')} for p in snapshot['papers'] if p['pmid'] not in present and not p.get('retracted')]
        self.literature_size = {row['taxon_id']: sum(tier['count'] for tier in row['tiers'].values()) for row in snapshot['searches']}

    def register_taxon(self, taxon):
        before = digest(json.dumps(self.taxmap.get(taxon['id']), sort_keys=True, ensure_ascii=False))
        if taxon['id'] not in self.taxmap:
            taxon['drug_names'] = sorted(regulatory().drug_names.get(taxon['id'], ()))
            self.taxa.append(taxon)
            self.taxmap[taxon['id']] = taxon
            self.resolver = Resolver(self.taxa, self.congeners)
        else:
            # An older name for a plant already held: the plant stays, the name is learned.
            known = self.taxmap[taxon['id']]
            held = {normalize(a['name']) for a in known['aliases']}
```

```
            older = [a for a in taxon['aliases'] if a['kind'] == 'SYNONYM' and normalize(a['name']) not in held]
            if older:
                known['aliases'] = known['aliases'] + older
                self.resolver = Resolver(self.taxa, self.congeners)
        held = self.taxmap[taxon['id']]
        self.scientific_genera.update(normalize(name).split()[0] for name in [held['name']] + [a['name'] for a in held.get('aliases', []) if a['kind'] in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH')] if name.strip())
        # The gazetteer files some plants under a GBIF synonym key ("Hibiscus tiliaceus"); once the
        # accepted species is known, that entry's names join it and the duplicate leaves the lists.
        # Only a GBIF synonym record's own key counts: a same-named accepted species is a homonym, not a twin.
        usages = {a['usage'] for a in held['aliases'] if a['kind'] == 'SYNONYM' and a.get('usage')}
        twins = [t for t in self.taxa if t.get('gazetteer_only') and t['id'] != held['id'] and t['id'] in usages]
        for twin in twins:
            present = {normalize(a['name']) for a in held['aliases']}
            held['aliases'] = held['aliases'] + [a for a in twin['aliases'] if a['kind'] == 'VERNACULAR' and normalize(a['name']) not in present]
            self.taxa.remove(twin)
            self.taxmap[twin['id']] = held
        if twins:
            self.resolver = Resolver(self.taxa, self.congeners)
        if before != digest(json.dumps(held, sort_keys=True, ensure_ascii=False)):
            self.record_knowledge('taxon:' + held['id'], held)
        for twin in twins:
            self.record_knowledge('taxon:' + twin['id'], held)
        return held

    def record_knowledge(self, key, value):
        """Track live additions separately from the immutable bundled snapshot."""
        if not hasattr(self, '_knowledge_updates'):
            self._knowledge_updates = {}
        self._knowledge_updates[key] = digest(json.dumps(value, sort_keys=True, ensure_ascii=False))

    def knowledge_fingerprint(self):
        return digest(self.snapshot + json.dumps(getattr(self, '_knowledge_updates', {}), sort_keys=True))

    def candidate_for(self, taxon, wording):
        """A GBIF match for a word the snapshot lacks; labelled, ranked like any common-name match."""
        alias = next((a for a in taxon['aliases'] if normalize(a['name']) == normalize(wording)), {'name': wording, 'kind': 'VERNACULAR', 'url': taxon['provenance']['url']})
        return {'taxon_id': taxon['id'], 'name': taxon['name'], 'scientific_name': taxon['scientific_name'], 'rank': taxon['rank'], 'family': taxon['family'], 'score': .9, 'score_meaning': 'RANKING_FEATURE_NOT_PROBABILITY', 'confidence': 'LOW', 'reasons': [taxon.get('open_world', 'GBIF_LOOKUP')], 'matched_alias': alias, 'provenance': taxon['provenance'], 'licence': taxon['licence'], 'gazetteer_only': False}

    def read_name_lists(self, mentions, resolutions):
        """Names listed together describe one plant, so they are weighed together.

        One local name often covers several plants; five names of one entry rarely agree by chance.
        """
        lists = defaultdict(list)
        for mention, resolution in zip(mentions, resolutions):
            if mention.get('list_id') is not None or mention['method'] == 'RESEARCHER_SELECTED_NAME':
                lists[mention.get('list_id', 'selected')].append((mention, resolution, local_names().lookup(mention['text'])))
        for rows in lists.values():
            agreement = Counter(entry['id'] for _, _, found in rows for entry in found)
            agreement.update(c['taxon_id'] for _, resolution, _ in rows for c in resolution['candidates'])
            best = max(agreement.values(), default=0)
            for mention, resolution, found in rows:
                kept = [e for e in found if best < 2 or agreement[e['id']] == best][:5]
                known = {c['taxon_id'] for c in resolution['candidates']}
```

```
                for entry in kept:
                    taxon = self.register_taxon(taxon_from_record(entry, mention['text']))
                    if taxon['id'] not in known:
                        candidate = self.candidate_for(taxon, mention['text'])
                        candidate['reasons'] = ['RECORDED_LOCAL_NAME'] + candidate['reasons'][1:]
                        candidate['name_record'] = {k: entry.get(k) for k in ('source', 'printed', 'where', 'reference')}
                        resolution['candidates'].append(candidate)
                if resolution['candidates']:
                    resolution['candidates'].sort(key=lambda c: -agreement[c['taxon_id']])
                    resolution.update({'status': 'OPEN_WORLD_AMBIGUOUS' if len(resolution['candidates']) > 1 else 'PROVISIONAL', 'mention_id': mention['id'],
    'list_agreement': {'names_agreeing': best, 'names_listed': len(rows)}})

    def widen_candidates(self, mentions, resolutions, use_synonyms):
        """Binomials the snapshot does not hold are checked with GBIF; non-plants are dropped."""
        rejected, looked_up = set(), 0
        for mention, resolution in zip(mentions, resolutions):
            if mention['method'] == 'APPROXIMATE_SPELLING_PROPOSAL' and not resolution['candidates']:
                rejected.add(mention['id'])
                continue
            # A snapshot common name can hide the plant a text means, so for a proposal or the
            # plant the text opens with ("boneset" is comfrey in the snapshot) it does not end the search.
            held = resolution['candidates']
            proposal = mention['method'] in ('PLANT_POSITION_PROPOSAL', 'LOCAL_NAME_PROPOSAL') or (bool(held) and mention is mentions[0])
            if proposal and all(c['reasons'][0] == 'EXACT_VERNACULAR' for c in held):
                # Position alone proves nothing: without a GBIF plant the word is dropped.
                taxa = reconcile_common_name(self.client, mention['text']) if looked_up < 8 else None
                looked_up += 1
                if not taxa:
                    if not held:
                        rejected.add(mention['id'])
                    continue
                for taxon in taxa:
                    self.register_taxon(taxon)
                known = {c['taxon_id'] for c in held}
                refreshed = held + [self.candidate_for(t, mention['text']) for t in taxa if t['id'] not in known]
                resolution.update({'status': 'OPEN_WORLD_AMBIGUOUS' if len(refreshed) > 1 else 'PROVISIONAL', 'candidates': refreshed, 'mention_id': mention['id']})
                if mention.get('heading_line'):
                    # "American Hellebore" as a whole is a plant; "hellebore" inside it is not a second one.
                    rejected |= {m['id'] for m in mentions if m is not mention and mention['start'] <= m['start'] and m['end'] <= mention['end']}
                continue
            # A binomial held only as a synonym may be a homonym of a species the snapshot lacks; one
            # held only by the gazetteer may be a GBIF synonym key ("Hibiscus tiliaceus", Talipariti).
            synonym_only = bool(held) and all(c['reasons'][0] == 'EXACT_SYNONYM' or c.get('gazetteer_only') for c in held)
            if (held and not synonym_only) or mention['method'] not in ('UNVERIFIED_BINOMIAL_PROPOSAL', 'PARENTHETICAL_BINOMIAL_PROPOSAL',
    'SCIENTIFIC_BINOMIAL_PROPOSAL'):
                continue
            try:
                taxa = scientific_readings(self.client, mention.get('lookup') or mention['text'])
                looked_up += 1
            except Exception:
                resolution['open_world_lookup'] = 'NETWORK_UNAVAILABLE'
                continue
            if not taxa:
                if mention['method'] == 'UNVERIFIED_BINOMIAL_PROPOSAL' and not held:
                    rejected.add(mention['id'])
                continue
            for taxon in taxa:
                self.register_taxon(taxon)
```

```
            refreshed = self.resolver.resolve(mention.get('lookup') or mention['text'], use_synonyms=use_synonyms, method='exact')
            resolution.update({**refreshed, 'mention_id': mention['id']})
        return rejected, looked_up

    def plant_literature(self, taxon_ids):
        """What is known about a plant regardless of topic, so an unmatched topic is no dead end."""
        output = {}
        for taxon_id in taxon_ids:
            taxon = self.taxmap.get(taxon_id)
            if not taxon:
                continue
            rows = [self.index.papers[i] for i, label in self.index.documents_naming(taxon).items() if label == 'SCIENTIFIC_NAME']
            rows.sort(key=lambda row: (design_rank(row), -int(str(row.get('year') or 0)[:4] or 0)))
            output[taxon_id] = {'name': taxon['name'], 'publications': len(rows), 'human_studies': sum(design_rank(row) <= 2 for row in rows), 'leading': [{'id':
row['id'], 'title': row['title'], 'year': row.get('year'), 'url': row['url'], 'types': row.get('publication_types', [])[:3]} for row in rows[:3]]}
        return output

    def topic_studies(self, taxon_id, condition_id):
        """All snapshot papers for one plant and topic, read the same way as a retrieved study."""
        key = (taxon_id, condition_id, len(self.papers))
        if key not in self.topic_cache:
            taxon, rows = self.taxmap[taxon_id], []
            for hit in self.index.matching(taxon, condition_id, genus=True):
                paper = hit['paper']
                design = self.study_designs.get(paper['id']) or self.study_designs.setdefault(paper['id'], classify_study(paper, self.study_model))
                rows.append({'paper': paper, 'study': design, 'study_identity': hit['taxon_match'], 'branch_scientific_names': [taxon['name']] + [a['name'] for a in
taxon['aliases'] if a['kind'] in ('ACCEPTED_NAME', 'SYNONYM', 'INPUT_SYNONYM_MATCH')], 'relation': relation_to_claim(paper, taxon, condition_id, design,
taxon_match=hit['taxon_match'], known_genera=self.scientific_genera)})
            self.topic_cache[key] = rows
        return self.topic_cache[key]

    def ensure_literature(self, candidate_ids, conditions=None):
        """A plant with no indexed paper is read from PubMed once; the response stays cached."""
        fetched, failed = [], False
        for taxon_id in candidate_ids[:8]:
            taxon = self.taxmap.get(taxon_id)
            if not taxon:
                continue
            topics = (conditions or {}).get(taxon_id, ())
            title_name = normalize(taxon['name'])
            untested = [c for c in topics if sum(design_rank(hit['paper']) <= 2 and title_name in normalize(hit['paper']['title']) for hit in
self.index.matching(taxon, c)) < 3]
            if (sum(label == 'SCIENTIFIC_NAME' for label in self.index.documents_naming(taxon).values()) >= 3 and not untested):
                continue
            try:
                rows = [row for row in on_demand_literature(self.client, taxon, sorted(untested or topics)) if not row.get('retracted') and row['id'] not in
self.papermap]
            except Exception:
                failed = True
                break
            self.papers += rows
            self.papermap.update({row['id']: row for row in rows})
            self.index.add(rows)
            for row in rows:
                self.record_knowledge('paper:' + row['id'], row)
            self.literature_size[taxon_id] = max(self.literature_size.get(taxon_id, 0), len(rows))
            fetched.append({'taxon_id': taxon_id, 'name': taxon['name'], 'publications': len(rows)})
        return {'plants_read_on_demand': fetched, 'status': 'NETWORK_UNAVAILABLE' if failed else 'OK'}
```

```

    def withdrawn(self):
        return set(read_json(self.withdrawals_path)["withdrawn_source_ids"]) if self.withdrawals_path.exists() else set()

    def analyze(self, text, *, source=None, method="hybrid", use_synonyms=True, citation_guard=True, branch_propagation=True, persist=True, name_to_inspect=None):
        if not text.strip() or len(text) > 50000:
            raise ValueError("Input must contain 1\u201350,000 characters")
        source = source or {"id": "user-provided", "title": "Researcher-provided text", "licence": "USER_RIGHTS_UNVERIFIED", "context": "User-provided passage;
provenance and rights not independently verified", "url": None, "community": None}
        withdrawn = self.withdrawn()
        if source["id"] in withdrawn or source.get("source_id") in withdrawn:
            raise ValueError("Source is withdrawn from active analysis")
        def analysis_key():
            return digest(text + self.knowledge_fingerprint() + self.code_fingerprint + self.model_fingerprint + str(self.index.encoder.available) + repr((source,
method, use_synonyms, citation_guard, branch_propagation, sorted(withdrawn), name_to_inspect)))[:24]
        cache_key = analysis_key()
        if persist:
            previous = self.store.get(cache_key)
            if previous:
                previous["cache_hit"] = True
                return previous
        start = time.perf_counter()
        section = source.get("id", "").startswith(("culpeper-", "fernie-section-", "fernie-subsection-"))
        extracted = self.extractor.extract(text, section_subject=section, requested_name=name_to_inspect)
        resolutions = []
        for mention in extracted["mentions"]:
            exact_proposal = mention['method'] in ('SOURCE_HEADING_SUBJECT', 'PARENTHETICAL_BINOMIAL_PROPOSAL', 'SCIENTIFIC_BINOMIAL_PROPOSAL',
'UNVERIFIED_BINOMIAL_PROPOSAL', 'RESEARCHER_SELECTED_NAME', 'MULTILINGUAL_NAME_PROPOSAL', 'CROSS_LANGUAGE_EXACT_NAME', 'APPROXIMATE_SPELLING_PROPOSAL',
'PLANT_POSITION_PROPOSAL')
            if mention['method'] == 'GENUS_NAME_PROPOSAL':
                exact_proposal = False
            resolution = self.resolver.resolve(mention.get('heading_name') or mention.get('lookup') or mention["text"], use_synonyms=use_synonyms, method='exact' if
exact_proposal else 'full', forms=mention.get('forms'))
            resolution['candidates']=[c for c in resolution['candidates'] if c['taxon_id'] not in withdrawn]
            if not resolution['candidates'] and mention['method'] == 'SOURCE_HEADING_SUBJECT':
                # A heading may be a genus or a near spelling; candidates stay labelled as such.
                resolution = self.resolver.resolve(mention.get('heading_name') or mention['text'], use_synonyms=use_synonyms, method='full',
forms=mention.get('forms'))
                resolution['candidates'] = [c for c in resolution['candidates'] if c['taxon_id'] not in withdrawn]
            if not resolution['candidates']:resolution['status']='UNRESOLVED'
            resolution["mention_id"] = mention["id"]
            resolutions.append(resolution)
        # Chapters are analysed offline; their binomials were reconciled when the snapshot was built.
        if not section:
            self.read_name_lists(extracted['mentions'], resolutions)
        rejected, gbif_lookups = (set(), 0) if section else self.widen_candidates(extracted['mentions'], resolutions, use_synonyms)
        extracted['local_names_not_in_any_list'] = list(dict.fromkeys(m['text'] for m in extracted['mentions'] if m['id'] in rejected and m.get('list_id') is not
None))
        if rejected:
            extracted['mentions'] = [m for m in extracted['mentions'] if m['id'] not in rejected]
            resolutions = [r for r in resolutions if r['mention_id'] not in rejected]
            for claim in extracted['historical_claims']:
                claim['mention_ids'] = [m for m in claim['mention_ids'] if m not in rejected]
                # "Der Thee von Pfefferminze": once the rejected word is gone, one plant is left.
                if claim['relation_scope'] == 'MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION' and len(claim['mention_ids']) == 1 and any(m['id'] == claim['mention_ids'][0]
and claim['support']['start'] <= m['start'] < claim['support']['end'] for m in extracted['mentions']):
                    claim['relation_scope'] = 'SENTENCE'
            extracted['historical_claims'] = [c for c in extracted['historical_claims'] if c['mention_ids']]
```

```
        glosses = source_glosses(text, extracted['mentions'])
        if not section:
            # "Juniper berries (Juniperus communis)": the author names one plant twice, so one card carries both.
            for binomial_id, names in glosses.items():
                if not any(r.get('mention_id') == binomial_id and r['candidates'] for r in resolutions):
                    continue
                for gloss in names:
                    twins = [m for m in extracted['mentions'] if m['id'] != binomial_id and 'BINOMIAL' not in m['method'] and m['start'] < gloss['end'] and gloss['start'] < m['end']]
                    if not twins:
                        continue
                    gloss.update({'name': twins[0]['text'], 'start': twins[0]['start'], 'end': twins[0]['end']})
                    merged = {m['id'] for m in twins}
                    extracted['mentions'] = [m for m in extracted['mentions'] if m['id'] not in merged]
                    resolutions = [r for r in resolutions if r.get('mention_id') not in merged]
                    for claim in extracted['historical_claims']:
                        claim['mention_ids'] = list(dict.fromkeys(binomial_id if m in merged else m for m in claim['mention_ids']))
                        if claim['relation_scope'] == 'MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION' and len(claim['mention_ids']) == 1:
                            claim['relation_scope'] = 'SENTENCE'
        # A pasted entry names its heading's plant in the first sentence; the heading takes that identity.
        glossed = heading_gloss(text, extracted['mentions'])
        if glossed:
            head, binomial = glossed
            target = next((r for r in resolutions if r.get('mention_id') == binomial['id'] and r['candidates']), None)
            named = next((r for r in resolutions if r.get('mention_id') == head['id']), None)
            if target and named:
                gloss = [{**c, 'reasons': ['NAMED_IN_SOURCE_FIRST_SENTENCE'] + c['reasons']} for c in target['candidates']]
                # In a book chapter the heading's own reading stays beside the author's binomial:
                # Fernie's "Common Primrose (Primula veris)" uses an older concept than today's name.
                kept = [c for c in named['candidates'] if c['taxon_id'] not in {g['taxon_id'] for g in gloss}] if section else []
                named.update({'status': 'AMBIGUOUS' if kept else target['status'], 'candidates': gloss + kept, 'glossed_by': binomial['text'], **({} if section else {'open_world_alternatives': []})})
                glosses.setdefault(binomial['id'], []).append({'name': head['text'], 'start': head['start'], 'end': head['end'], 'binomial': binomial['text']})
        if section:
            for clue in heading_common_names(text, extracted['mentions']):
                named = next((r for r in resolutions if r['mention_id'] == clue['mention_id']), None)
                if not named:
                    continue
                additions = {}
                for form in clue['lookup_forms']:
                    found = self.resolver.resolve(form, method='exact', use_synonyms=use_synonyms)
                    for candidate in found['candidates']:
                        if candidate['taxon_id'] in withdrawn:
                            continue
                        additions.setdefault(candidate['taxon_id'], {**candidate, 'reasons':['OPENING_SOURCE_COMMON_NAME'] + candidate['reasons'], 'source_common_name':{**clue, 'lookup':form}})
                if additions:
                    # Keep the generic heading's readings. This is local source
                    # context, never a learned alias affecting another passage.
                    current = {c['taxon_id']:c for c in named['candidates']}
                    genera = {c['name'].split()[0] for c in list(current.values()) + list(additions.values())}
                    for identifier, candidate in additions.items():
                        # A globally shared common name is not enough to jump
                        # between genera. Agreement with the heading, a unique
                        # lookup, or refinement within one genus can rank first;
                        # other readings stay visible without a score bonus.
                        candidate['source_common_name']['anchor'] = identifier in current or len(additions) == 1 or len(genera) == 1
                        if identifier in current:
```

```
                            current[identifier] = {**current[identifier], 'source_common_name':candidate['source_common_name'],
'reasons':list(dict.fromkeys(candidate['reasons'] + current[identifier]['reasons']))}
                        else:
                            current[identifier] = candidate
                    named['candidates'] = list(current.values())
                    named['source_common_names'] = [clue]
                    if len(current) > 1:
                        named['status'] = 'AMBIGUOUS'
        for resolution in resolutions:
            if resolution.get('mention_id') in glosses:
                resolution['source_names'] = glosses[resolution['mention_id']]
            if len(resolution['candidates']) >= 1:
                mention = next(m for m in extracted['mentions'] if m['id'] == resolution['mention_id'])
                matrix = identity_matrix(text, resolution['candidates'], self.taxmap, self.descriptions, region='GB' if section else None,
prominence=self.literature_size, topics=mention_topics(extracted, mention), name=(mention.get('heading_name') or mention['text']).split()[0], wording=None if
'BINOMIAL' in mention['method'] else mention.get('heading_name') or mention.get('lookup') or mention['text'], english_name=extracted.get('language') == 'en' and
'MULTILINGUAL' not in mention['method'])
                resolution['identity_matrix'] = matrix
                order = {row['taxon_id']: index for index, row in enumerate(matrix['candidates'])}
                ratings = {row['taxon_id']: row for row in matrix['candidates']}
                for candidate in resolution['candidates']:
                    candidate['identity_rating'] = ratings.get(candidate['taxon_id'], {}).get('rating', 'NOT_ASSESSABLE')
                    candidate['identity_score'] = ratings.get(candidate['taxon_id'], {}).get('score')
                resolution['candidates'].sort(key=lambda c: order.get(c['taxon_id'], 99))
        conditions = {c["id"]: c for claim in extracted["historical_claims"] for c in claim["conditions"]}
        resolution_map = {r["mention_id"]: r for r in resolutions}
        branch_conditions = {}
        source_contexts = index_historical_contexts(extracted, resolutions)
        for claim in extracted["historical_claims"]:
            if claim.get('statement_role') != 'THERAPEUTIC_USE_RECORDED' or claim.get('practice') == 'NON_PLANT_REMEDY':
                continue
            # An uncertain subject stays labelled on the statement; it no longer hides the literature.
            for mention_id in claim["mention_ids"]:
                for c in resolution_map.get(mention_id, {}).get("candidates", []):
                    branch_conditions.setdefault(c["taxon_id"], set()).update(x["id"] for x in claim["conditions"])
        candidate_ids = list(branch_conditions)
        named = list(dict.fromkeys(c['taxon_id'] for r in resolutions for c in r['candidates']))
        # Chapters are served from the snapshot; only a researcher's own text reads PubMed live.
        literature_access = {'plants_read_on_demand': [], 'status': 'SNAPSHOT'} if section else self.ensure_literature(list(dict.fromkeys(candidate_ids + named)),
branch_conditions)
        if not branch_propagation:
            candidate_ids = candidate_ids[:1]
        evidence, blocked = [], []
        for taxon_id in candidate_ids:
            taxon = self.taxmap[taxon_id]
            for condition_id in sorted(branch_conditions[taxon_id])[:14]:
                for hit in self.index.search(taxon, condition_id, method=method, limit=4):
                    paper = hit["paper"]
                    if paper["id"] in withdrawn:
                        continue
                    design = self.study_designs.get(paper["id"]) or self.study_designs.setdefault(paper["id"], classify_study(paper, self.study_model))
                    study = scope_study_preparation(paper, taxon, design)
                    relation = relation_to_claim(paper, taxon, condition_id, study, taxon_match=hit["taxon_match"], known_genera=self.scientific_genera)
                    citation = citation_check(paper, relation["supporting_text"], hit["taxon_match"], hit["condition_match"], self.papermap.get(paper["id"]))
                    if citation_guard and citation["status"] == "REJECTED":
                        blocked.append({"paper_id": paper["id"], "reasons": citation["errors"]})
                        continue
                    eid = f"evidence-{len(evidence)}"
```

```
                    safety = []
                    for sentence in sentences(paper["abstract"]):
                        if re.search(r"\badverse\b|\binteraction\b|\bcontraindicat|\bhepatotoxic|\btoxicity\b", sentence["text"], re.I):
                            safety.append({"supporting_text": sentence["text"], "interpretation": "PUBLISHED_ABSTRACT_SIGNAL_REQUIRES_REVIEW",
"not_personalized_advice": True})
                    compound_mentions = [{**c,"supporting_text":match.group(),"support_start":match.start(),"support_end":match.end()} for c in self.compounds if
(match:=re.search(r"\b" + re.escape(c["name"]) + r"\b", paper["abstract"], re.I))]
                    isolated_compound = any(c["name"].casefold() in paper["title"].casefold() for c in compound_mentions) and normalize(taxon["name"]) not in
normalize(paper["title"])
                    if isolated_compound:
                        relation = {**relation, "relation": "INSUFFICIENT", "directness": "COMPOUND_STUDY_NOT_PLANT_PREPARATION", "rationale": "Title identifies an
isolated compound intervention. Plant co-mention does not make this evidence for a historical whole-plant preparation."}
                    historical_contexts = compare_historical_contexts(source_contexts.get((taxon_id, condition_id), []), study, paper)
                    checks = comparability(paper, taxon, condition_id, study, relation, hit["taxon_match"], [],
preparation_check=preparation_across_statements(historical_contexts))
                    if isolated_compound:
                        checks["checks"].append({"check": "intervention", "status": "FAIL", "detail": "Isolated compound, not a plant preparation"})
                        checks = summarize(checks["checks"])
                    relation = gate_relation(relation, checks)
                    evidence.append({"id": eid, "taxon_id": taxon_id, "condition_id": condition_id, "modern_hypothesis": {"id":
f"hypothesis:{taxon_id}:{condition_id}", "question": f"Does a specified preparation of {taxon['name']} improve {conditions[condition_id]['label']} in humans?",
"population": "HUMAN_HYPOTHESIS; study population must be checked", "preparation": "SOURCE_STATEMENTS_COMPARED_SEPARATELY; see historical_contexts",
"historical_use_is_not_efficacy": True}, "identity_qualifier": "CONDITIONAL_ON_CANDIDATE_IDENTITY", "study_identity": hit["taxon_match"], "paper": paper, "study":
study, "relation": relation, "comparability": checks, "historical_contexts": historical_contexts, "citation": citation, "retrieval": {k:v for k,v in hit.items() if k
 != "paper"}, "safety_signals": safety, "compound_co_mentions": compound_mentions, "strength_components": {"design": study["types"], "quality": "NOT_APPRAISED",
"directness": relation["directness"], "identity": hit["taxon_match"], "sample_size_available": study["sample_size"] is not None, "citation": citation["status"],
"replication": "NOT_ASSESSED"}})
        result = {"run_id": cache_key, "created_at": now(), "pipeline_version": __version__, "snapshot_sha256": self.snapshot, "source": source, "input": text,
**extracted, "resolutions": resolutions, "conditions": list(conditions.values()), "evidence": evidence, "conflicts": conflict_groups(evidence), "blocked_citations":
blocked, "cache_hit": False, "limitations": ["A modern-name match is not proof of a historical referent.", "Evidence is conditional on candidate taxon identity;
common-name-only study identity remains uncertain.", "Abstract-only automated relations require review of intervention, population and outcome comparability.",
"Retrieval is bounded by PubMed relevance order; no retrieved evidence is not evidence of no effect.", "Research use only. No clinical validation or treatment
recommendations."], "configuration": {"retrieval": method, "synonyms": use_synonyms, "citation_guard": citation_guard, "branch_propagation": branch_propagation},
"models": {"encoder": "MiniLM ONNX" if self.index.encoder.available else "NOT_INSTALLED", "study_classifier": "TFIDF_LOGREG" if self.study_model.model is not None
else "NOT_TRAINED"}}
        # Resolve/enrich may have added taxonomy or abstracts after cache lookup.
        # Store under the resulting knowledge state, so the next request can hit
        # it and older analyses cannot conceal newly acquired evidence.
        result['run_id'] = analysis_key()
        result['knowledge_sha256'] = self.knowledge_fingerprint()
        result['configuration']['requested_name'] = name_to_inspect
        result['literature_access'] = literature_access
        result['research_digest'] = research_digest(result, self.taxmap, self.topic_studies, self.literature_size)
        result['plant_literature'] = self.plant_literature(named)
        result["identity_counterfactuals"] = counterfactuals(result)
        result['historical_identity_references'] = historical_name_references(result, self.passages + self.extra_passages, self.taxa, withdrawn)
        result['study_reading_sets'] = active_reading_sets(self.trial_readings, candidate_ids, conditions, withdrawn)
        result['models']['reranker'] = 'MS_MARCO_MINILM_L6_ONNX_CPU' if self.index.reranker.available else 'NOT_INSTALLED'
        result["compound_assay_leads"] = assay_leads(evidence, self.mechanism_records, withdrawn)
        result["graph"] = self.make_graph(result)
        result["code_fingerprint"] = self.code_fingerprint
        result["model_fingerprint"] = self.model_fingerprint
        result["performance"] = {"latency_ms": round((time.perf_counter() - start) * 1000, 1), "network_requests_during_analysis": gbif_lookups +
len(literature_access['plants_read_on_demand'])}
        if persist:
            self.store.put(result)
        return result

```

```
    def make_graph(self, result):
        nodes, edges = {}, []
        def node(id, kind, label, **metadata):
            nodes.setdefault(id, {"id": id, "kind": kind, "label": label, **metadata})
        def edge(a, p, b, provenance):
            edges.append({"id": f"edge-{len(edges)}", "subject": a, "predicate": p, "object": b, "provenance": provenance})
        source = result["source"]
        sp = {"source_id": source["id"], "url": source.get("url"), "licence": source["licence"], "location": source.get("title"), "method": self.extractor.version,
    "human_reviewed": False}
        node(source["id"], "HistoricalSource", source["title"], provenance=sp)
        for mention in result["mentions"]:
            node(mention["id"], "PlantMention", mention["text"], span=mention)
            edge(mention["id"], "MENTIONED_IN", source["id"], {**sp, "support": mention["text"], "start": mention["start"], "end": mention["end"]})
        for claim in result["historical_claims"]:
            node(claim["id"], "HistoricalClaim", claim["support"]["text"][:90], supporting_text=claim["support"]["text"], statement_role=claim.get('statement_role'))
            for mention in claim["mention_ids"]:
                predicate = {'THERAPEUTIC_USE_RECORDED': 'ATTESTED_USE', 'ADVERSE_EFFECT_RECORDED': 'REPORTED_ADVERSE_EFFECT'}.get(claim.get('statement_role'),
    'CONTEXT_MENTION')
                if claim['relation_scope'] in ('MULTI_PLANT_CLAUSE_UNCERTAIN_RELATION', 'PARAGRAPH_CONTEXT_UNCERTAIN_RELATION'):
                    predicate = 'POSSIBLE_SUBJECT_OF'
                edge(mention, predicate, claim["id"], {**sp, "support": claim["support"]["text"], "relation_scope": claim["relation_scope"], 'statement_role':
    claim.get('statement_role')})
            for condition in claim["conditions"]:
                node(condition["id"], "Condition", condition["label"])
                edge(claim["id"], "LEXICAL_CONDITION_MAPPING", condition["id"], {**sp, "support": condition["original"], "mapping_status":
    condition["mapping_status"]})
            for field, kind, predicate in [("part", "PlantPart", "USES_PART"), ("preparation", "Preparation", "PREPARED_AS"), ("route", "Route",
    "ROUTE_REPORTED_AS")]:
                for value in claim[field]:
                    identifier = kind.lower() + ':' + normalize(value["text"])
                    node(identifier, kind, value["text"])
                    edge(claim["id"], predicate, identifier, {**sp, "support": value["text"], "start": value["start"], "end": value["end"]})
        for resolution in result["resolutions"]:
            for c in resolution["candidates"]:
                node(c["taxon_id"], "ModernTaxon", c["name"])
                edge(resolution["mention_id"], "CANDIDATE_IDENTITY", c["taxon_id"], {"source_id": c["taxon_id"], **c["provenance"], "licence": c["licence"],
    "matched_alias": c["matched_alias"], "reasons": c["reasons"], "method": self.resolver.version, "historical_identity_validated": False})
        question_source_links = set()
        for e in result["evidence"]:
            paper = e["paper"]
            node(paper["id"], "Publication", paper["title"], url=paper["url"], evidence_types=e["study"]["types"], methods_sources=paper.get('methods_contexts', []),
     reported_results=paper.get('reported_results', []))
            node(e["modern_hypothesis"]["id"], "ModernHypothesis", e["modern_hypothesis"]["question"])
            for context in e.get('historical_contexts', []):
                key = (context['claim_id'], e['modern_hypothesis']['id'])
                if key not in question_source_links:
                    edge(context['claim_id'], 'MOTIVATES_QUESTION_UNDER_CANDIDATE_IDENTITY', e['modern_hypothesis']['id'], {
                        **sp, 'support': context['support']['text'], 'start': context['support']['start'], 'end': context['support']['end'],
                        'candidate_taxon_id': e['taxon_id'], 'condition_id': e['condition_id'], 'link_basis': context['link_basis'],
                        'historical_use_is_not_efficacy': True,
                    })
                    question_source_links.add(key)
            ep = {"source_id": paper["id"], **paper["provenance"], "licence": paper["licence"], "support": e["relation"]["supporting_text"], "method": "abstract-
    relation-rules-0.2", "human_reviewed": False, "identity_qualifier": e["identity_qualifier"], "comparability": "UNREVIEWED"}
            edge(e["taxon_id"], "HAS_MODERN_QUESTION", e["modern_hypothesis"]["id"], ep)
            edge(e["modern_hypothesis"]["id"], e["relation"]["relation"], paper["id"], ep)
            for i, safety in enumerate(e["safety_signals"]):
                sid = f"safety:{paper['id']}:{i}"
```

```
                node(sid, "SafetySignal", safety["supporting_text"][:75])
                edge(paper["id"], "REPORTS_SAFETY_WORDING", sid, {**ep, "support": safety["supporting_text"], "interpretation": safety["interpretation"]})
            for compound in e["compound_co_mentions"]:
                cid = f"pubchem:{compound['cid']}"
                node(cid, "Compound", compound["name"], url=compound["url"])
                edge(paper["id"], "CO_MENTIONS_COMPOUND", cid, {**ep, "support": compound["supporting_text"], "start": compound["support_start"], "end":
compound["support_end"], "does_not_assert_plant_contains_compound": True})
        for a in result.get('compound_assay_leads', []):
            cid, tid = f"pubchem:{a['cid']}", a['target']['target_chembl_id']
            node(cid, 'Compound', a['compound'], url=f"https://pubchem.ncbi.nlm.nih.gov/compound/{a['cid']}")
            node(a['id'], 'AssayMeasurement', f"{a['standard_type']} {a['standard_value']} {a['standard_units']}", url=a['url'], licence=a['licence'])
            node(tid, 'MolecularTarget', a['target']['pref_name'], organism=a['target']['organism'], target_type=a['target']['target_type'])
            ap = {'source_id': a['id'], 'url': a['url'], 'licence': a['licence'], 'method': 'exact-InChIKey-database-join', 'retrieved_at':
a['provenance']['retrieved_at'], 'document_id': a['document_id'], 'support': a['assay_description'], 'scope': a['scope'], 'link_basis': a['link_basis']}
            edge(cid, 'HAS_REPORTED_ASSAY', a['id'], ap)
            edge(a['id'], 'MEASURED_AGAINST', tid, ap)
        return {"nodes": list(nodes.values()), "edges": edges}

    def search(self, query, kind=None):
        q = normalize(query)
        withdrawn=self.withdrawn()
        groups = [("HistoricalSource", self.passages + self.extra_passages, lambda x: x["title"] + " " + x["text"]), ("ModernTaxon", self.taxa, lambda x: x["name"] +
 " " + " ".join(a["name"] for a in x["aliases"])), ("Publication", self.papers, lambda x: x["title"] + " " + x["abstract"]), ("Compound", self.compounds, lambda x:
x["name"])]
        hits = []
        for group, rows, text in groups:
            if kind and group != kind:
                continue
            for row in rows:
                if row.get("id") in withdrawn or row.get("source_id") in withdrawn:
                    continue
                if q in normalize(text(row)):
                    hits.append({"kind": group, "id": row.get("id", f"pubchem:{row.get('cid')}"), "label": row.get("title") or row.get("name"), "url":
row.get("url")})
        return hits[:100]
```

src/retrieval/__init__.py
SHA-256: c7b52c21a8e7beb77c5c585a34a17c10db17abb9b7580d5abc6b57eaec6d41e5

```
    1 """Lexical, local neural and hybrid branch-specific retrieval."""
```

src/retrieval/index.py
SHA-256: e32465e2fd0419fa5c1665ce61c2f9094a6ae02f03223c71ee128efd5c768fcf

```
from __future__ import annotations

import math
import re
from collections import Counter
from pathlib import Path

import numpy as np
from src.common import ROOT, read_json
from src.nlp.extract import SENSE_REQUIRED, normalize, topic_pattern, research_terms
from src.retrieval.reranker import LocalReranker


def tokens(text):
    return re.findall(r"[a-z]{2,}", normalize(text))


def design_rank(paper):
    types = set(paper.get("publication_types", []))
    title = paper["title"].casefold()
    if types & {"Meta-Analysis", "Systematic Review"} or "meta-analysis" in title or "systematic review" in title:
        return 0
    if types & {"Randomized Controlled Trial", "Controlled Clinical Trial"} or "randomized" in title or "randomised" in title:
        return 1
    if any(t.startswith("Clinical Trial") for t in types) or "clinical trial" in title:
        return 2
    if "Review" in types or re.search(r"\breview\b", title):
        return 3
    return 4


class LocalEncoder:
    """Pinned MiniLM ONNX encoder. Absence is explicit, never silently substituted."""
    def __init__(self, model_dir=None):
        self.path = model_dir or ROOT / "models" / "minilm"
        self.available = (self.path / "model.onnx").exists() and (self.path / "tokenizer.json").exists()
        self.session = None

    def encode(self, texts):
        if not self.available:
            raise RuntimeError("Local encoder is not installed. Run scripts/setup_model.py, or select bm25 explicitly.")
        if self.session is None:
            from src.common import digest
            lock = read_json(ROOT / "data" / "models.lock.json")
            for entry in lock["files"]:
                if digest((self.path / entry["local"]).read_bytes()) != entry["sha256"]:
                    raise ValueError("Local encoder artifact checksum differs from pinned manifest")
            import onnxruntime as ort
            from tokenizers import Tokenizer
            options = ort.SessionOptions()
            options.intra_op_num_threads = 4
            self.session = ort.InferenceSession(str(self.path / "model.onnx"), sess_options=options, providers=["CPUExecutionProvider"])
            self.tokenizer = Tokenizer.from_file(str(self.path / "tokenizer.json"))
            self.tokenizer.enable_truncation(max_length=256)
            self.tokenizer.enable_padding()
```

```
        outputs = []
        for start in range(0, len(texts), 16):
            encodings = self.tokenizer.encode_batch(texts[start:start + 16])
            ids = np.asarray([e.ids for e in encodings], dtype=np.int64)
            mask = np.asarray([e.attention_mask for e in encodings], dtype=np.int64)
            feed = {"input_ids": ids, "attention_mask": mask, "token_type_ids": np.zeros_like(ids)}
            values = self.session.run(None, {x.name: feed[x.name] for x in self.session.get_inputs()})[0]
            if values.ndim == 3:
                weights = mask[..., None].astype(np.float32)
                values = (values * weights).sum(axis=1) / np.maximum(weights.sum(axis=1), 1)
            values = values / np.maximum(np.linalg.norm(values, axis=1, keepdims=True), 1e-12)
            outputs.append(values)
        return np.concatenate(outputs) if outputs else np.empty((0, 384))


class LiteratureIndex:
    def __init__(self, papers, taxa, *, encoder=None):
        self.papers, self.taxa = [], taxa
        self.docs, self.counts, self.texts, self.lengths = [], [], [], []
        self.postings = {}
        self.encoder = encoder or LocalEncoder()
        self.reranker = LocalReranker()
        self.embeddings = None
        self.naming_cache = {}
        self.add(papers)

    def add(self, papers):
        """Incremental: a plant looked up on demand joins the index without rebuilding it."""
        for paper in papers:
            index = len(self.papers)
            self.papers.append(paper)
            document = tokens(paper["title"] + " " + paper["abstract"])
            counts = Counter(document)
            self.docs.append(document)
            self.counts.append(counts)
            self.lengths.append(len(document))
            self.texts.append(normalize(paper["title"] + " " + paper["abstract"]))
            for word, count in counts.items():
                self.postings.setdefault(word, []).append((index, count))
        self.df = {word: len(rows) for word, rows in self.postings.items()}
        self.avg_len = sum(self.lengths) / max(len(self.lengths), 1)
        # Encoding nine thousand abstracts takes minutes; a looked-up plant's papers are appended instead.
        if self.embeddings is not None and papers:
            self.embeddings = np.concatenate([self.embeddings, self.encoder.encode([p["title"] + " " + p["abstract"] for p in papers])])

    def bm25(self, query):
        scores = np.zeros(len(self.papers))
        total = len(self.docs)
        for word in set(tokens(query)):
            rows = self.postings.get(word)
            if not rows:
                continue
            df = len(rows)
            idf = math.log(1 + (total - df + .5) / (df + .5))
            for i, tf in rows:
                scores[i] += idf * tf * 2.5 / (tf + 1.5 * (.25 + .75 * self.lengths[i] / max(self.avg_len, 1)))
        return scores

```

```
    def documents_naming(self, taxon):
        """Papers whose text carries a scientific name or a usable common name of this taxon."""
        key = (taxon["id"], len(self.papers))
        if key not in self.naming_cache:
            self.naming_cache[key] = self.find_naming(taxon)
        return self.naming_cache[key]

    def find_naming(self, taxon):
        scientific = [taxon["name"]] + [a["name"] for a in taxon["aliases"] if a["kind"] in ("SYNONYM", "INPUT_SYNONYM_MATCH")]
        common = [a["name"] for a in taxon["aliases"] if a["kind"] in ("VERNACULAR", "HISTORICAL_LEXICON") and a.get('lang', 'en') == 'en' and len(a["name"]) >= 5
    and normalize(a["name"]) not in SENSE_REQUIRED]
        found = {}
        common += taxon.get('drug_names', [])
        for names, label in ((scientific, "SCIENTIFIC_NAME"), (common, "COMMON_NAME_ONLY_IDENTITY_UNCERTAIN")):
            for name in names:
                key = normalize(name)
                parts = tokens(name)
                if not parts:
                    continue
                pattern = re.compile(r"\b" + re.escape(key) + r"\b")
                for i, _ in self.postings.get(min(parts, key=lambda w: self.df.get(w, 0)), ()):
                    if i not in found and pattern.search(self.texts[i]):
                        found[i] = label
        return found

    def matching(self, taxon, condition, *, scores=None, method="DIRECT_LOOKUP", genus=False):
        """Every paper naming the plant and the topic, unranked; the digest reads all of them.

        With genus=True, papers naming only the genus are included as identity-uncertain rows.
        """
        pattern = topic_pattern(tuple(research_terms(condition)))
        results = []
        named = dict(self.documents_naming(taxon))
        if genus:
            for i, _ in self.postings.get(normalize(taxon["name"].split()[0]), ()):
                named.setdefault(i, "COMMON_NAME_ONLY_IDENTITY_UNCERTAIN")
        for i, label in named.items():
            if not pattern.search(self.texts[i]):
                continue
            paper = self.papers[i]
            results.append({"paper": paper, "retrieval_score": round(float(scores[i]), 5) if scores is not None else 0, "retrieval_method": method, "taxon_match":
    label, "condition_match": True, "branch_taxon_id": taxon["id"], "design_rank": design_rank(paper), "topic_in_title":
    bool(pattern.search(normalize(paper["title"])))})
        return results

    def search(self, taxon, condition, *, method="hybrid", limit=6):
        terms = research_terms(condition)
        query = taxon["name"] + " " + " ".join(terms)
        lexical = self.bm25(query)
        semantic = None
        if method in ("embedding", "hybrid", "rerank"):
            if not self.encoder.available:
                if method == "embedding":
                    raise RuntimeError("Embedding retrieval requested but local model is absent")
                effective = "BM25_ONLY_ENCODER_NOT_INSTALLED"
            else:
                if self.embeddings is None:
                    self.embeddings = self.encoder.encode([p["title"] + " " + p["abstract"] for p in self.papers])
```

```
                semantic = self.embeddings @ self.encoder.encode([query])[0]
                effective = method.upper()
        else:
            effective = "BM25"
        if method == "embedding" and semantic is not None:
            scores = semantic
        elif semantic is not None:
            scores = np.zeros(len(self.papers))
            for component in [lexical, semantic]:
                for rank, i in enumerate(np.argsort(-component)):
                    scores[i] += 1 / (60 + rank + 1)
        else:
            scores = lexical
        results = self.matching(taxon, condition, scores=scores, method=effective)
        # Human evidence first: a reader of one source statement should not meet cell assays
        # before the trial that tests the plant. Ties keep the retrieval order.
        results.sort(key=lambda r: (r["taxon_match"] != "SCIENTIFIC_NAME", not r["topic_in_title"], r["design_rank"], -r["retrieval_score"]))
        results = results[:40 if method == 'rerank' else limit]
        if method == 'rerank' and results:
            reranked = self.reranker.score(query, [r['paper']['title'] + ' ' + r['paper']['abstract'] for r in results])
            for row, score in zip(results, reranked):
                row['retrieval_score'] = round(float(score), 5)
                row['retrieval_method'] = 'RERANK_CROSS_ENCODER_AFTER_IDENTITY_CONDITION_FILTER'
            results.sort(key=lambda r: (r["design_rank"], -r['retrieval_score']))
            results = results[:limit]
        return results
```

src/retrieval/knowledge.py
SHA-256: 94676febfca1876aa40e76845f2ff9349085ae44a32564e173bf91bbb95f1c06

```
"""Local semantic search over source-attested use records, not answer generation."""
import re
import tempfile
import numpy as np
from src.retrieval.index import LocalEncoder, tokens
from collections import Counter
from src.common import ROOT, digest
from src.retrieval.reranker import LocalReranker


class TextIndex:
    """Shared ranking implementation used by the app and external retrieval evaluation."""
    def __init__(self, texts, *, cache_dir=None):
        self.texts = texts
        self.encoder = LocalEncoder()
        self.reranker = LocalReranker()
        self.embeddings = None
        self.cache_dir = cache_dir or ROOT / '.cache/knowledge-index'
        self.cache_key = digest('text-index-v1-384-rerank-40' + '\n'.join(self.texts))
        self.documents = [Counter(tokens(t)) for t in self.texts]
        self.df = Counter(word for doc in self.documents for word in doc)
        self.avg_length = sum(sum(d.values()) for d in self.documents) / max(len(self.documents), 1)

    def lexical(self, query):
        scores = np.zeros(len(self.texts))
        for term in set(tokens(query)):
            df = self.df[term]
            idf = np.log(1 + (len(self.texts) - df + .5) / (df + .5))
            for i, doc in enumerate(self.documents):
                tf = doc[term]
                if tf:
                    scores[i] += idf * tf * 2.2 / (tf + 1.2 * (.25 + .75 * sum(doc.values()) / max(self.avg_length, 1)))
        return scores

    def rank(self, query, *, method='bm25', limit=20, allowed=None):
        if method not in ('bm25', 'embedding', 'hybrid', 'rerank'):
            raise ValueError('Unknown knowledge search method')
        if not query.strip() or allowed == set():
            return []
        lexical = self.lexical(query)
        if method == 'bm25':
            scores = lexical
        else:
            if not self.encoder.available:
                raise RuntimeError('Install the pinned local encoder with scripts/setup_model.py; semantic search has no silent keyword fallback')
            if self.embeddings is None:
                self.cache_dir.mkdir(parents=True, exist_ok=True)
                path = self.cache_dir / (self.cache_key + '.npy')
                if path.exists():
                    values = np.load(path, allow_pickle=False)
                    if values.shape != (len(self.texts), 384) or not np.isfinite(values).all():
                        raise ValueError('Invalid local knowledge embeddings; rebuild this cache')
                    self.embeddings = values
                else:
                    self.embeddings = self.encoder.encode(self.texts)
```

```
                    with tempfile.NamedTemporaryFile(dir=self.cache_dir, suffix='.npy', delete=False) as temp:
                        temporary = type(path)(temp.name)
                        np.save(temp, self.embeddings, allow_pickle=False)
                    try:
                        temporary.replace(path)
                    finally:
                        temporary.unlink(missing_ok=True)
            semantic = self.embeddings @ self.encoder.encode([query])[0]
            if method == 'embedding':
                scores = semantic
            else:
                scores = np.zeros(len(self.texts))
                for component in (lexical, semantic):
                    for rank, i in enumerate(np.argsort(-component, kind='stable')):
                        if component is lexical and component[i] <= 0:
                            continue
                        scores[i] += 1 / (60 + rank + 1)
        order = np.argsort(-scores, kind='stable')
        if allowed is not None:
            order = np.asarray([i for i in order if int(i) in allowed], dtype=int)
        if method == 'rerank':
            pool = order[:max(40, limit)]
            reranked = self.reranker.score(query, [self.texts[int(i)] for i in pool])
            return [(int(pool[i]), float(reranked[i])) for i in np.argsort(-reranked, kind='stable')[:limit]]
        return [(int(i), float(scores[i])) for i in order[:limit] if method != 'bm25' or scores[i] > 0]


class KnowledgeIndex(TextIndex):
    def __init__(self, corpus, *, cache_dir=None):
        self.rows = corpus['records']
        texts = [' '.join([r['source_title'], r['statement']['text'], ' '.join(n['text'] for n in r['names'])]) for r in self.rows]
        super().__init__(texts, cache_dir=cache_dir)

    def search(self, query, *, method='bm25', limit=20, collection=None, condition=None, role=None):
        allowed = None
        if collection or condition or role:
            allowed = {i for i, r in enumerate(self.rows) if (not collection or r['collection'] == collection) and (not condition or any(c['id'] == condition for c
in r['conditions'])) and (not role or r.get('statement_role') == role)}
        return [{'record': self.rows[i], 'score': round(score, 6), 'score_meaning': 'RETRIEVAL_RANKING_NOT_PROBABILITY', 'method': method} for i, score in
self.rank(query, method=method, limit=limit, allowed=allowed)]
```

src/retrieval/reranker.py
SHA-256: d9f03a299cb486b44be9577405db965bf2a74b3523d2393ffe91a104f51d6bc6

```python
"""Pinned query-document cross-encoder. Scores are relevance logits, not evidence."""
import threading
import numpy as np
from src.common import ROOT, read_json, digest


class LocalReranker:
    def __init__(self, model_dir=None):
        self.path = model_dir or ROOT / 'models/reranker'
        self.available = all((self.path / f).exists() for f in ('model.onnx', 'tokenizer.json'))
        self.session = None
        self.lock = threading.RLock()

    def score(self, query, documents):
        if not self.available:
            raise RuntimeError('Install the pinned local reranker with scripts/setup_reranker.py; no silent ranking fallback')
        if not documents:
            return np.empty(0)
        with self.lock:
            if self.session is None:
                import onnxruntime as ort
                from tokenizers import Tokenizer
                manifest = read_json(ROOT / 'data/reranker.lock.json')
                for artifact in manifest['files']:
                    if digest((self.path / artifact['local']).read_bytes()) != artifact['sha256']:
                        raise ValueError('Reranker artifact differs from pinned checksum')
                options = ort.SessionOptions()
                options.intra_op_num_threads = 4
                self.session = ort.InferenceSession(str(self.path / 'model.onnx'), sess_options=options, providers=['CPUExecutionProvider'])
                self.tokenizer = Tokenizer.from_file(str(self.path / 'tokenizer.json'))
                self.tokenizer.enable_truncation(max_length=384, strategy='longest_first')
                self.tokenizer.enable_padding()
            values = []
            for start in range(0, len(documents), 8):
                batch = self.tokenizer.encode_batch([(query, text) for text in documents[start:start+8]])
                feed = {'input_ids': np.array([b.ids for b in batch], dtype=np.int64), 'attention_mask': np.array([b.attention_mask for b in batch], dtype=np.int64),
    'token_type_ids': np.array([b.type_ids for b in batch], dtype=np.int64)}
                logits = self.session.run(None, {f.name: feed[f.name] for f in self.session.get_inputs()})[0]
                values.extend(logits.reshape(-1).tolist())
            scores = np.asarray(values)
            if scores.shape != (len(documents),) or not np.isfinite(scores).all():
                raise ValueError('Invalid query-document ranking output')
            return scores
```

src/taxonomy/__init__.py
SHA-256: 5e2cee2f8d632c04ecb03b556ed4cf2ccdd1725bf5bc432077b78aebb67f1be1

```
    1 from .resolve import Resolver
    2
    3 __all__ = ["Resolver"]
```

src/taxonomy/lexicon.py
SHA-256: 488d7a8ac5b102c67818913ecd4399fe3cb55ef24c792bbf7e30ca5aaa5c0636

```
"""Local plant names from published lists. A recorded name is a hypothesis, never an identification.

Sources: Dr. Duke's ethnobotanical records (USDA, CC0), Merrill's 1903 dictionary of Philippine
plant names and Dalziel's 1916 Hausa vocabulary (public domain), and Wikidata labels (CC0). All work offline, so a rate-limited service never changes an answer.
"""
from __future__ import annotations

import re
from collections import Counter, defaultdict

from src.common import DATA, read_json
from src.evidence.independent_use import independent_use
from src.nlp.extract import normalize

MERRILL = DATA / 'merrill_1903_names.json'
WIDER = DATA / 'local_names_wikidata.json'
GAZETTEER = DATA / 'taxa_gazetteer.json'
SOUNDS = str.maketrans('eocjvzy', 'iukhbsi')


def fold(word):
    """Old orthographies of one sound: "Gomamila" and "gumamela", "cayanga" and "kayanga"."""
    key = normalize(word).replace('-', '').replace(' ', '').replace('qu', 'k').translate(SOUNDS)
    return re.sub(r'(.)\1+', r'\1', key)


# Devanagari, Bengali, Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada and Malayalam share one
# layout, so one table reads all nine: offset within the block to its usual English-style spelling.
BRAHMIC = range(0x0900, 0x0D80)
VOWELS = {0x05: 'a', 0x06: 'aa', 0x07: 'i', 0x08: 'ii', 0x09: 'u', 0x0A: 'uu', 0x0B: 'ri', 0x0E: 'e', 0x0F: 'e', 0x10: 'ai', 0x12: 'o', 0x13: 'o', 0x14: 'au'}
SIGNS = {0x3E: 'aa', 0x3F: 'i', 0x40: 'ii', 0x41: 'u', 0x42: 'uu', 0x43: 'ri', 0x46: 'e', 0x47: 'e', 0x48: 'ai', 0x4A: 'o', 0x4B: 'o', 0x4C: 'au'}
CONSONANTS = dict(zip(range(0x15, 0x3A), ['k', 'kh', 'g', 'gh', 'n', 'ch', 'chh', 'j', 'jh', 'n', 't', 'th', 'd', 'dh', 'n', 't', 'th', 'd', 'dh', 'n', 'n', 'p',
   'ph', 'b', 'bh', 'm', 'y', 'r', 'r', 'l', 'l', 'zh', 'v', 'sh', 'sh', 's', 'h']))
MARKS = {0x01: 'n', 0x02: 'n', 0x03: 'h'}
CHILLU = {0x0D7A: 'n', 0x0D7B: 'n', 0x0D7C: 'r', 0x0D7D: 'l', 0x0D7E: 'l', 0x0D7F: 'k', 0x09CE: 't'}


def romanize(word):
    """Indian-script spelling in Latin letters, as nineteenth-century authors wrote it: \u092d\u093f\u0902\u0921\u0940 "bhindi"."""
    out, pending = [], False
    for char in word:
        code = ord(char)
        if code in CHILLU:
            out.append(CHILLU[code])
            pending = False
            continue
        if code not in BRAHMIC:
            if pending:
                out.append('a')
            out.append(char)
            pending = False
            continue
        offset = (code - 0x0900) % 0x80
        if offset in CONSONANTS:
            if pending:
```

```
                out.append('a')
            out.append(CONSONANTS[offset])
            pending = True
        elif offset in SIGNS:
            out.append(SIGNS[offset])
            pending = False
        elif offset == 0x4D:
            pending = False
        elif offset in VOWELS:
            if pending:
                out.append('a')
            out.append(VOWELS[offset])
            pending = False
        elif offset in MARKS:
            if pending:
                out.append('a')
            out.append(MARKS[offset])
            pending = False
    if pending and len(out) <= 1:
        out.append('a')
    # Hindi and its neighbours drop a closing inherent vowel: \u0928\u0940\u092e is "nim".
    return ''.join(out)


def sounded(word):
    """One key for the ways colonial authors and modern scripts spell an Indian name: aspiration,
    voicing, vowel length and hyphens fall away. "Vendaik-k\xe1y" and \u0bb5\u0bc6\u0ba3\u0bcd\u0b9f\u0bc8\u0b95\u0bcd\u0b95\u0bbe\u0baf\u0bcd meet."""
    key = re.sub(r'[^a-z]', '', normalize(romanize(word)))
    key = key.replace('sh', 's').replace('ch', 'c').replace('w', 'v').replace('z', 'j').replace('ee', 'i').replace('oo', 'u')
    key = key.replace('h', '').translate(str.maketrans('gjdbeoy', 'kctpiui'))
    key = re.sub(r'(.)\1+', r'\1', key)
    return key[:-1] if len(key) > 4 and key.endswith('a') else key


SERBIAN = dict(zip('\u0430\u0431\u0432\u0433\u0434\u0452\u0435\u0436\u0437\u0438\u0458\u043a\u043b\u0459\u043c\u043d\u045a\u043e\u043f\u0440\u0441\u0442\u045b\u0443\
u0444\u0445\u0446\u0447\u045f\u0448', ['a', 'b', 'v', 'g', 'd', '\u0111', 'e', '\u017e', 'z', 'i', 'j', 'k', 'l', 'lj', 'm', 'n', 'nj', 'o', 'p', 'r', 's', 't',
'\u0107', 'u', 'f', 'h', 'c', '\u010d', 'd\u017e', '\u0161']))
# Where the noun stands in a two-word name: "Echter Baldrian", "Crna bazga", "Kara M\xfcrver" end with it;
# "Guimauve officinale", "piantaggine maggiore" start with it, unless an adjective leads ("Grand plantain").
HEAD_LAST = {'de', 'hr', 'sr', 'bg', 'tr', 'nl', 'hu', 'sl', 'mk'}
# Polish, Czech, Slovak and Ukrainian botanical names put the noun first: "Pokrzywa zwyczajna".
HEAD_FIRST = {'fr', 'it', 'es', 'pt', 'ro', 'pl', 'cs', 'sk', 'uk'}
LEADING_ADJECTIVES = {'grand', 'grande', 'gros', 'grosse', 'petit', 'petite', 'vrai', 'vraie', 'faux', 'fausse', 'bon', 'bonne', 'gran', 'falso', 'falsa', 'vero',
'vera'}
# "li\u015b\u0107 bobkowy" is bay leaf: a part of a plant, or its tea, is never the plant's short name.
PART_WORDS = {normalize(w) for w in ('li\u015b\u0107', 'li\u015bcie', 'kwiat', 'korze\u0144', 'nasiona', 'owoc', 'ziele', 'herbata', 'list', 'listovi', 'cvijet',
'cvet', 'koren', 'korijen', 'seme', 'sjeme',
             'plod', '\u010daj', 'kora', 'k\u016fra', 'kv\u011bt', 'ko\u0159en', 'semeno', 'cvet', 'thee', 'tea', 'blad', 'wortel', 'bloem', 'zaad', 'vrucht',
'lev\xe9l', 'gy\xf6k\xe9r', 'vir\xe1g', 'mag', 'term\xe9s',
             'feuille', 'fleur', 'racine', 'graine', 'fruit', 'foglia', 'fiore', 'radice', 'seme', 'frutto', 'hoja', 'flor', 'ra\xedz', 'semilla', 'fruto', 'folha',
 'raiz', 'semente',
             'blatt', 'bl\xfcte', 'wurzel', 'samen', 'frucht', 'frunz\u0103', 'floare', 'r\u0103d\u0103cin\u0103', 's\u0103m\xe2n\u021b\u0103', 'fruct', 'yaprak',
'\xe7i\xe7ek', 'k\xf6k', 'tohum', 'meyve', 'gjethe', 'lule', 'rr\xebnj\xeb',
             'fara', '\u043b\u0438\u0441\u0442', '\u0446\u0432\u0435\u0442', '\u043a\u043e\u0440\u0435\u043d', '\u0441\u0435\u043c\u0435',
'\u043f\u043b\u043e\u0434', '\u0447\u0430\u0439', '\u0447\u0430\u0458', '\u043b\u0438\u0441\u0442\u044f', '\u043a\u0432\u0456\u0442\u043a\u0430',
'\u043a\u043e\u0440\u0456\u043d\u044c', '\u03c6\u03cd\u03bb\u03bb\u03bf', '\u03ac\u03bd\u03b8\u03bf\u03c2', '\u03c1\u03af\u03b6\u03b1',
'\u03c3\u03c0\u03cc\u03c1\u03bf\u03c2', '\u03ba\u03b1\u03c1\u03c0\u03cc\u03c2', '\u03c4\u03c3\u03ac\u03b9')}
LINKING_WORDS = {'de', 'des', 'du', 'd', '\xe0', 'a', 'aux', 'au', 'di', 'del', 'della', 'dei', 'degli', 'da', 'dos', 'das', 'do', 'la', 'le', 'les', 'el', 'los',
```

```
    'e', 'y', 'et', 'und', 'von', 'der', 'od', 'i', 'ili', 'van', 'het', 'en', 'z', 'ze', 'in', '\xe9s', '\u0438', '\u043e\u0434'}


def serbian_latin(name):
    """Serbian is written in both scripts; field notes from Kosovo and Bosnia are mostly in Latin."""
    out = ''.join(SERBIAN.get(c.lower(), c) if c.lower() in SERBIAN else c for c in name)
    return out[:1].upper() + out[1:] if name[:1].isupper() else out


def short_forms(taxa):
    """The noun people write without its qualifier: "Baldrian" for "Echter Baldrian", "guimauve" for
    "Guimauve officinale", "bokvica" for "\u0416\u0435\u043d\u0441\u043a\u0430 \u0431\u043e\u043a\u0432\u0438\u0446\u0430". A short form joins only when every plant
    that carries it shares one genus, or most do ("\u0432\u043e\u0434\u0435\u043d\u0430 \u0431\u043e\u043a\u0432\u0438\u0446\u0430" is water plantain), so it narrows
    to a group of species. A frequent word is screened out at build time (scripts/derive_short_names.py)."""
    found, written, spelled = defaultdict(set), defaultdict(set), set()
    genera = {normalize(t['genus']) for t in taxa}
    for taxon in taxa:
        names = [(a.get('lang'), a['name'], a.get('url')) for a in taxon['aliases'] if a['kind'] == 'VERNACULAR' and a.get('lang') not in (None, 'en', 'la')]
        for lang, name, _ in names:
            written[(lang, normalize(name))].add((taxon['id'], taxon['genus']))
        latin = [('hr', serbian_latin(name), url) for lang, name, url in names if lang == 'sr' and re.search('[\u0430-\u0448]', name.lower())]
        for lang, name, url in names + latin:
            words = re.findall(r"[^\W\d_]+", name)
            forms = [name] if (lang, name, url) in latin else []
            # A Latin binomial filed as a local name ("Scirpus setaceus") has no local noun in it.
            if words and normalize(words[0]) in genera:
                continue
            # "Feuilleea julibrissin" is a Latin synonym under a French label: the epithet gives it away.
            if len(words) == 2 and words[1] == taxon['name'].split()[-1]:
                continue
            if lang in HEAD_LAST and len(words) == 2 and (lang != 'de' or words[1][:1].isupper()):
                forms.append(words[1])
            elif lang == 'de' and len(words) == 1 and '-' in name:
                forms.append(name.rsplit('-', 1)[1])
            elif lang in HEAD_FIRST and len(words) == 2 and not set(map(str.casefold, words)) & LINKING_WORDS:
                forms.append(words[1] if words[0].casefold() in LEADING_ADJECTIVES else words[0])
            for form in forms:
                # Four letters is a whole Slavic plant name ("rman", yarrow; "brin", juniper).
                if len(normalize(form)) >= 4 and normalize(form) not in PART_WORDS:
                    found[(lang, normalize(form))].add((taxon['id'], taxon['genus'], form, name, url))
                    if form is name and (lang, name, url) in latin:
                        # A Serbian name in Latin letters is the recorded name, not a shortening: "\u0161ipak" is
                        # rose hip in Serbian though Croatian "\u0161ipak" is pomegranate, so both readings stay.
                        spelled.add((lang, normalize(form), taxon['id']))
    by_id = {t['id']: t for t in taxa}
    out = []
    for (lang, key), rows in sorted(found.items()):
        holders = Counter(genus for _, genus in {row[:2] for row in rows} | written.get((lang, key), set()))
        genus, count = holders.most_common(1)[0]
        kept = [row for row in rows if row[1] == genus and count * 2 > sum(holders.values()) or (lang, key, row[0]) in spelled]
        for identifier, _, form, source, url in sorted(kept, key=str):
            if (key, lang) not in {(normalize(a['name']), a.get('lang')) for a in by_id[identifier]['aliases']} and not any(r['id'] == identifier and r['lang'] ==
lang and normalize(r['name']) == key for r in out[-20:]):
                out.append({'name': form, 'lang': lang, 'id': identifier, 'accepted': by_id[identifier]['name'], 'short_form_of': source, 'url': url})
    return out


class LocalNames:
```

```
    def __init__(self):
        self.exact, self.folded, self.sounds = defaultdict(dict), defaultdict(dict), defaultdict(dict)
        self.citations = {}
        if MERRILL.exists():
            body = read_json(MERRILL)
            self.citations['Merrill 1903'] = {'citation': body['citation'], 'url': body['url'], 'licence': body['licence']}
            for row in body['names']:
                self.add(row['name'], {'id': row['id'], 'name': row['accepted'], 'family': row['family'], 'records': 1, 'source': 'Merrill 1903', 'printed': row['name'], 'where': ', '.join(row['languages'])})
        if WIDER.exists():
            for row in read_json(WIDER)['names']:
                self.add(row['name'], {'id': row['id'], 'name': row['accepted'], 'family': row['family'], 'records': 1, 'source': row['source'], 'printed': row['name'], 'where': row['language']})
        if GAZETTEER.exists():
            # Hindi names stay out of prose matching; inside an author's name list they may be read.
            for taxon in read_json(GAZETTEER)['taxa']:
                for name in taxon['names']:
                    if name['lang'] == 'hi':
                        self.add(name['name'], {'id': taxon['id'], 'name': taxon['name'], 'family': taxon.get('family', 'Unknown'), 'records': 1, 'source': 'Wikidata', 'printed': name['name'], 'where': 'Hindi'})
        duke = independent_use()
        if duke.available:
            self.citations['Duke'] = {'citation': "Duke JA. Dr. Duke's Phytochemical and Ethnobotanical Databases. USDA Agricultural Research Service; 2016.", 'url': duke.citation['url'], 'licence': duke.citation['licence']}
            for key, plants in duke.names.items():
                for entry in plants.values():
                    self.add(entry['printed'], {'id': entry['id'], 'name': entry['name'], 'family': entry['family'], 'records': entry['records'], 'source': 'Duke', 'printed': entry['printed'], 'where': ', '.join(sorted(entry['places'])[:3]), 'reference': entry['reference']})

    def add(self, wording, entry):
        self.exact[normalize(wording)].setdefault(entry['id'], entry)
        key = fold(wording)
        if len(key) >= 5:
            self.folded[key].setdefault(entry['id'], entry)
        sound = sounded(wording)
        if len(sound) >= 3:
            self.sounds[sound].setdefault(entry['id'], entry)

    def lookup(self, wording):
        """Exact spelling first, then a folded spelling, then the spoken form of an Indian name."""
        found = self.exact.get(normalize(wording)) or self.folded.get(fold(wording)) or self.sounds.get(sounded(wording)) or {}
        return sorted(found.values(), key=lambda e: (-e['records'], e['name']))


_shared = None


def local_names():
    global _shared
    if _shared is None:
        _shared = LocalNames()
    return _shared
```

src/taxonomy/open_world.py
SHA-256: 2988a6d8cfd385559d24963d0726418fb4735a1e0d5b79ce978bee6be5b5ddd9

```
"""Names outside the bundled snapshot: reconcile with GBIF, then read PubMed on demand.

Every response is cached by the scientific client, so a later offline run reproduces the
same result. Without a network the mention stays visible and unresolved; nothing is guessed.
"""
from __future__ import annotations

import re
from urllib.parse import urlencode

from src.ingestion.pubmed import ESEARCH, HUMAN, fetch_records, name_clause
from src.nlp.extract import normalize, research_terms

BACKBONE = 'd7dddbf4-2cf0-4f39-9b2a-bb099caae36c'
GBIF = 'https://api.gbif.org/v1/'


def taxon_from_record(entry, wording):
    """A plant known from a name list, built offline from the list's own GBIF reconciliation."""
    page = f"https://www.gbif.org/species/{entry['id'].split(':')[1]}"
    return {'id': entry['id'], 'name': entry['name'], 'scientific_name': entry['name'], 'rank': 'SPECIES', 'family': entry['family'], 'genus': entry['name'].split()[0], 'status': 'ACCEPTED', 'aliases': [{'name': entry['name'], 'kind': 'ACCEPTED_NAME', 'url': page}, {'name': wording, 'kind': 'VERNACULAR', 'url': page}], 'licence': 'CC-BY-4.0', 'provenance': {'source': entry['source'], 'url': page}, 'open_world': 'PUBLISHED_NAME_LIST', 'scope': 'Named in a published list of local names; a name match is not a historical identification.'}


def taxon_from_key(client, key, *, via):
    record, provenance = client.json(f'{GBIF}species/{key}')
    if record.get('rank') != 'SPECIES' or record.get('kingdom') != 'Plantae':
        return None
    common, _ = client.json(f'{GBIF}species/{key}/vernacularNames?limit=100')
    names = sorted({v['vernacularName'] for v in common.get('results', []) if v.get('language') in ('eng', 'en') and 4 <= len(v.get('vernacularName', '')) <= 40})
    aliases = [{'name': record['canonicalName'], 'kind': 'ACCEPTED_NAME', 'url': f'https://www.gbif.org/species/{key}'}]
    aliases += [{'name': n, 'kind': 'VERNACULAR', 'url': f'{GBIF}species/{key}/vernacularNames?limit=100'} for n in names]
    return {'id': f'gbif:{key}', 'name': record['canonicalName'], 'scientific_name': record['scientificName'], 'rank': 'SPECIES', 'family': record.get('family', 'Unknown'), 'genus': record.get('genus', record['canonicalName'].split()[0]), 'status': record.get('taxonomicStatus', 'ACCEPTED'), 'aliases': aliases, 'licence': 'CC-BY-4.0', 'provenance': provenance, 'open_world': via, 'scope': 'Reconciled on demand with the GBIF backbone; a name match is not a historical identification.'}


def reconcile_scientific(client, name):
    """Exact Plantae species match. None means GBIF does not know this as a plant species."""
    match, _ = client.json(f'{GBIF}species/match?' + urlencode({'name': name, 'kingdom': 'Plantae', 'strict': 'true'}))
    if match.get('matchType') != 'EXACT' or match.get('kingdom') != 'Plantae' or match.get('rank') != 'SPECIES':
        return None
    key = match.get('acceptedUsageKey') or match.get('usageKey')
    taxon = taxon_from_key(client, key, via='GBIF_EXACT_SCIENTIFIC_NAME')
    if taxon and normalize(name) != normalize(taxon['name']):
        # The text keeps its own older name; it stays findable as a synonym of the accepted one.
        taxon['aliases'].append({'name': name, 'kind': 'SYNONYM', 'url': f'{GBIF}species/{key}', 'usage': f"gbif:{match['usageKey']}"})
    return taxon


def scientific_readings(client, name):
    """Every species a written binomial can mean, the accepted holder of the name first.

```

```
    "Artemisia ludoviciana" is Nuttall's sagebrush and also Besser's synonym of A. indica;
    "Holarrhena antidysenterica" is a synonym of two species, so GBIF's match gives up at family.
    """
    taxon = reconcile_scientific(client, name)
    body, _ = client.json(f'{GBIF}species/search?' + urlencode({'q': name, 'rank': 'SPECIES', 'datasetKey': BACKBONE, 'limit': 20}))
    rows = [r for r in body.get('results', []) if r.get('kingdom') == 'Plantae' and normalize(r.get('canonicalName', '')) == normalize(name)]
    keys = [r['key'] for r in rows if r.get('taxonomicStatus') == 'ACCEPTED'] + [r['acceptedKey'] for r in rows if r.get('acceptedKey')]
    found = [taxon] if taxon else []
    for key in dict.fromkeys(keys):
        if len(found) >= 3:
            break
        if f'gbif:{key}' in {t['id'] for t in found}:
            continue
        other = taxon_from_key(client, key, via='GBIF_SCIENTIFIC_NAME_HOMONYM')
        if other:
            if normalize(other['name']) != normalize(name):
                usage = next((r['key'] for r in rows if r.get('acceptedKey') == key), None)
                other['aliases'].append({'name': name, 'kind': 'SYNONYM', 'url': f'{GBIF}species/search?' + urlencode({'q': name, 'datasetKey': BACKBONE}), 'usage':
f'gbif:{usage}' if usage else None})
            found.append(other)
    return found


def genus_members(client, word):
    match, _ = client.json(f'{GBIF}species/match?' + urlencode({'name': word, 'kingdom': 'Plantae', 'rank': 'GENUS', 'strict': 'true'}))
    if not (match.get('rank') == 'GENUS' and match.get('matchType') == 'EXACT' and match.get('kingdom') == 'Plantae'):
        return None
    species, _ = client.json(f'{GBIF}species/search?' + urlencode({'highertaxonKey': match['usageKey'], 'rank': 'SPECIES', 'status': 'ACCEPTED', 'datasetKey':
BACKBONE, 'limit': 300}))
    rows = sorted(species.get('results', []), key=lambda r: -len(r.get('vernacularNames', [])))
    return [t for t in (taxon_from_key(client, r['key'], via='GBIF_GENUS_MEMBER') for r in rows[:3]) if t]


def reconcile_word(client, word):
    """A capitalised word may be a genus; otherwise an exact English common name. None means no plant."""
    if word[0].isupper():
        members = genus_members(client, word)
        if members:
            return members
    # Old print hyphenates what GBIF spells apart: "Couch-Grass".
    for wording in dict.fromkeys((word, word.replace('-', ' '))):
        body, _ = client.json(f'{GBIF}species/search?' + urlencode({'q': wording, 'qField': 'VERNACULAR', 'rank': 'SPECIES', 'highertaxonKey': 6, 'datasetKey':
BACKBONE, 'status': 'ACCEPTED', 'limit': 40}))
        key = normalize(wording)
        exact = []
        # The most widely named species first: the search order alone hides the familiar plant.
        for row in sorted(body.get('results', []), key=lambda r: -len(r.get('vernacularNames', []))):
            names = {normalize(v.get('vernacularName', '')) for v in row.get('vernacularNames', []) if v.get('language') in ('eng', 'en', '')}
            if key in names or f'common {key}' in names:
                exact.append(row['key'])
        if exact:
            return [t for t in (taxon_from_key(client, k, via='GBIF_EXACT_COMMON_NAME') for k in exact[:3]) if t] or None
    return None


def reconcile_common_name(client, word):
    """Every plant a name can mean: the one an encyclopedia means first, then GBIF's holders.

```

```
    GBIF alone answers "couch grass" with Bermuda grass, an Australian usage; the plant a reader
    of an old herbal expects comes from the encyclopedia. None means neither route knows it.
    """
    found = {}
    for route in (reconcile_encyclopedia, reconcile_word):
        try:
            found.update({t['id']: t for t in route(client, word) or () if t['id'] not in found})
        except Exception:
            continue
    return list(found.values())[:5] or None


def reconcile_encyclopedia(client, word, languages=('en', 'es')):
    """An everyday name GBIF lacks: a Wikipedia redirect leads to the item, Wikidata to its taxon.

    "Ashwagandha" reaches Withania somnifera this way. GBIF still has the last word on the species.
    """
    # Wikipedia titles are sentence case and spelt apart; old print is neither.
    plain = word.replace('-', ' ')
    wordings = dict.fromkeys((word, plain, plain.capitalize()))
    for language in languages:
        for wording in wordings:
            body, _ = client.json(f'https://{language}.wikipedia.org/w/api.php?' + urlencode({'action': 'query', 'format': 'json', 'redirects': 1, 'prop':
'pageprops', 'ppprop': 'wikibase_item', 'titles': wording}), patient=False)
            for page in body.get('query', {}).get('pages', {}).values():
                item = page.get('pageprops', {}).get('wikibase_item')
                if not item:
                    continue
                # An article titled by its binomial needs no second source.
                title = page.get('title', '')
                for name in [title] if looks_scientific(title) else taxon_names(client, item):
                    taxon = reconcile_scientific(client, name) if looks_scientific(name) else None
                    if taxon:
                        taxon['open_world'] = 'WIKIPEDIA_REDIRECT_TO_GBIF_SPECIES'
                        taxon['aliases'].append({'name': word, 'kind': 'VERNACULAR', 'url': f'https://www.wikidata.org/wiki/{item}'})
                        return [taxon]
                    # "Gum plant" is a genus: its best-known species stand for the name.
                    members = genus_members(client, name) if re.fullmatch(r'[A-Z][a-z-]+', name) else None
                    if members:
                        return members
    return None


def taxon_names(client, item):
    """The taxon an item is, or the taxon its product comes from (kava the drink, its plant)."""
    def claims(entity):
        # One request per item: Wikidata limits rate, and a reader is waiting on this.
        body, _ = client.json('https://www.wikidata.org/w/api.php?' + urlencode({'action': 'wbgetentities', 'format': 'json', 'props': 'claims', 'ids': entity}),
patient=False)
        found = body.get('entities', {}).get(entity, {}).get('claims', {})
        return lambda prop: [c['mainsnak']['datavalue']['value'] for c in found.get(prop, []) if 'datavalue' in c['mainsnak']]
    held = claims(item)
    names = held('P225')
    if names:
        return names
    for prop in ('P1582', 'P186'):
        for source in held(prop):
            names += claims(source['id'])('P225')
```

```
    return names


def on_demand_literature(client, taxon, conditions=(), *, limit=14):
    """Human studies first, then the recorded topics, then any study naming the plant."""
    clause = name_clause(taxon)
    identifiers = []
    words = list(dict.fromkeys(term for condition in conditions for term in research_terms(condition)[:4]))[:14]
    topical = ' AND (' + ' OR '.join(f'"{word}"[tiab]' for word in words) + ')' if words else ''
    # Trials often name only the genus ("Harpagophytum extract"), so the focused search keeps it.
    genus = f'{clause} OR "{taxon["name"].split()[0]}"[tiab]'
    for names, suffix, count in ((genus, topical + HUMAN, limit if words else 0), (clause, HUMAN, limit), (clause, topical, 10 if words else 0), (clause, '', 8)):
        if not count:
            continue
        body, _ = client.json(ESEARCH + '?' + urlencode({'db': 'pubmed', 'term': f'({names}){suffix}', 'retmode': 'json', 'retmax': count, 'sort': 'relevance'}))
        identifiers += [pmid for pmid in body.get('esearchresult', {}).get('idlist', []) if pmid not in identifiers]
    rows = fetch_records(client, identifiers)
    for row in rows:
        row.pop('pmid_key', None)
        row['taxon_ids'] = [taxon['id']]
    return rows


def looks_scientific(name):
    return bool(re.fullmatch(r'[A-Z][a-z-]+ [a-z-]+', name))
```

src/taxonomy/resolve.py
SHA-256: 4ba22838f0df67cbefb8898a8c9a5285458fb585f13df2f3b688b5b944c12447

```
from __future__ import annotations

import re
from rapidfuzz import fuzz, process
from src.nlp.extract import normalize


def spelling(value):
    return normalize(value).replace("camomile", "chamomile").replace("featherfew", "feverfew")


class Resolver:
    version = "candidate-ranker-0.4"

    def __init__(self, taxa, congeners=None):
        self.taxa = taxa
        self.taxon_ids = {t['id'] for t in taxa}
        self.by_id = {t['id']: t for t in taxa}
        # GBIF-derived name collisions outside the snapshot (data/demo/vernacular_congeners.json).
        # Absent data means the open-world check is reported as not run, never as passed.
        self.congeners = (congeners or {}).get("aliases")
        self.congener_rule = (congeners or {}).get("rule")
        self.alias_index, self.genus_vernacular, self.genus_index = {}, {}, {}
        for taxon in taxa:
            self.genus_index.setdefault(normalize(taxon['genus']), []).append(taxon)
            for alias in taxon['aliases']:
                # A common name learned from one reader's text must not narrow the next reader's:
                # those stay with GBIF, which lists every plant holding the name.
                if taxon.get('open_world') and alias['kind'] == 'VERNACULAR':
                    continue
                key = spelling(alias['name'])
                if alias['kind'] == 'GENUS_VERNACULAR':
                    self.genus_vernacular.setdefault(key, []).append((taxon, alias))
                self.alias_index.setdefault(key, []).append((taxon, alias))
        self.alias_keys = list(self.alias_index)

    def open_world(self, candidate):
        # Only common-name matches are exposed to name collisions; scientific names are not.
        if self.congeners is None or candidate["reasons"][0] not in ("EXACT_VERNACULAR", "FUZZY_NAME"):
            return []
        if candidate["matched_alias"].get("kind") != "VERNACULAR":
            return []
        entry = self.congeners.get(candidate["taxon_id"], {}).get(normalize(candidate["matched_alias"]["name"]))
        return [{**c, "via_alias": candidate["matched_alias"]["name"], "in_snapshot": False, "evidence_retrieved": False} for c in (entry or
{}).get("congeners_in_region", []) if f"gbif:{c['gbif_key']}" not in self.taxon_ids]

    def scored(self, keys, *, method, use_synonyms):
        """Best alias per taxon among name forms; forms are spellings of one source name."""
        best = {}
        for key in keys:
            for taxon, alias in self.alias_index.get(key, ()):
                if not use_synonyms and alias['kind'] != 'ACCEPTED_NAME':
                    continue
                reason = 'EXACT_' + alias['kind']
                if taxon['id'] not in best or (best[taxon['id']][2].get('kind') == 'VERNACULAR' and alias['kind'] != 'VERNACULAR'):
```

```
                    best[taxon['id']] = (1.0, reason, alias)
        # A name accepted for one species and listed as a synonym of another is a homonym:
        # the accepted holder leads, the other stays in view rather than being swapped in.
        # Only a GBIF-checked holder counts: the gazetteer keeps some synonym keys under their own name.
        if any(reason == 'EXACT_ACCEPTED_NAME' and not self.by_id[k].get('gazetteer_only') and self.by_id[k].get('status', 'ACCEPTED') == 'ACCEPTED' for k, (_,
   reason, _) in best.items()):
            best = {k: (.95, 'EXACT_SYNONYM_HOMONYM', alias) if reason == 'EXACT_SYNONYM' else (score, reason, alias) for k, (score, reason, alias) in best.items()}
        if best or method != 'full':
            return best
        for key in keys[:1]:
            if len(key) < 5:
                continue
            for match, ratio, _ in process.extract(key, self.alias_keys, scorer=fuzz.ratio, score_cutoff=86, limit=8):
                if match[0] != key[0]:
                    continue
                for taxon, alias in self.alias_index[match]:
                    if use_synonyms or alias['kind'] == 'ACCEPTED_NAME':
                        score = ratio / 100 * 0.87
                        if taxon['id'] not in best or best[taxon['id']][0] < score:
                            best[taxon['id']] = (score, 'FUZZY_NAME', alias)
        return best

    def resolve(self, name, *, method="full", geography=None, use_synonyms=True, forms=None):
        keys = list(dict.fromkeys(spelling(value) for value in [*(forms or []), name] if value))
        normalized = keys[0]
        # "Figs" is listed only for a Bornean fig; the reader means the fig. Plural and singular both count.
        if re.fullmatch(r"[a-z]{2,}[^su]s", normalized):
            keys.append(normalized[:-1])
        abbreviation = re.fullmatch(r"([A-Z])\.\s+([a-z]+)", name)
        matches = {}
        if normalized in self.genus_vernacular and use_synonyms:
            matches = {taxon['id']: (.80, 'GENUS_VERNACULAR_ONLY', alias) for taxon, alias in self.genus_vernacular[normalized]}
        elif abbreviation:
            for taxon in self.taxa:
                if taxon['genus'].startswith(abbreviation[1]) and normalize(taxon['name']).split()[-1] == abbreviation[2]:
                    matches[taxon['id']] = (.96, 'ABBREVIATED_GENUS', {'name': taxon['name'], 'kind': 'ACCEPTED_NAME', 'url': taxon['provenance']['url']})
        else:
            matches = self.scored(keys, method=method, use_synonyms=use_synonyms)
            singular = {taxon['id'] for taxon, _ in self.alias_index.get(keys[-1], ())} if keys[-1] != normalized and keys[-1] == normalized[:-1] else set()
            if singular:
                # A plural listed as a name is rarer evidence than the singular everyone uses.
                matches = {k: (score * .95, reason, alias) if k not in singular and spelling(alias['name']) == normalized else (score, reason, alias) for k, (score,
   reason, alias) in matches.items()}
            if not matches and method != 'exact':
                for key in keys:
                    # Curated species first, then the most widely named; a genus never implies one species.
                    ranked = sorted(self.genus_index.get(key, ()), key=lambda t: (bool(t.get('gazetteer_only')), -len(t['aliases'])))
                    for taxon in ranked[:5]:
                        matches[taxon['id']] = (.80, 'GENUS_CONTEXT_ONLY', {'name': taxon['genus'], 'url': taxon['provenance']['url']})
        candidates = []
        for identifier, (best, reason, matched_alias) in matches.items():
            taxon = self.by_id[identifier]
            geo_note = None
            if geography:
                countries = taxon.get("verified_countries", [])
                geo_note = "NO_VERIFIED_RANGE_IN_SNAPSHOT" if not countries else "RANGE_MATCH" if geography in countries else "RANGE_CONFLICT_REQUIRES_REVIEW"
                if countries and geography not in countries and method == "full":
                    best *= 0.65
```

```
            candidates.append({"taxon_id": identifier, "name": taxon["name"], "scientific_name": taxon["scientific_name"], "rank": taxon["rank"], "family": taxon["family"], "score": round(best, 4), "score_meaning": "RANKING_FEATURE_NOT_PROBABILITY", "confidence": "HIGH" if reason in ("EXACT_ACCEPTED_NAME", "EXACT_SYNONYM", "EXACT_INPUT_SYNONYM_MATCH") else "MEDIUM" if reason.startswith("EXACT_") else "LOW", "reasons": [reason] + ([geo_note] if geo_note else []), "matched_alias": matched_alias, "provenance": taxon["provenance"], "licence": taxon["licence"], "regional_records": taxon.get('occurrence_counts'), "gazetteer_only": bool(taxon.get('gazetteer_only'))})
        # Curated and regionally recorded candidates first; equal name evidence is never a tie-break for identity.
        candidates.sort(key=lambda c: (-c["score"], c['gazetteer_only'], -((c.get('regional_records') or {}).get('GB', 0)), -len(self.by_id[c['taxon_id']]['aliases']), c["name"]))
        if method == "full" and candidates:
            best = candidates[0]["score"]
            candidates = [c for c in candidates if c["score"] >= best - 0.10]
        candidates = candidates[:5]
        alternatives = []
        for c in candidates:
            if c['matched_alias'].get('kind') == 'VERNACULAR' and (self.congeners is None or c['taxon_id'] not in self.congeners):
                c['reasons'].append('OPEN_WORLD_CHECK_NOT_AVAILABLE')
            found = self.open_world(c)
            if found:
                c["reasons"].append("OPEN_WORLD_CONGENERS_IN_REGION")
                alternatives.extend(found)
        if not candidates:
            status = "UNRESOLVED"
        elif all(any(reason.startswith("GENUS_") for reason in c["reasons"]) for c in candidates):
            status = "GENUS_ONLY"
        elif len(candidates) > 1 and candidates[0]["score"] - candidates[1]["score"] < .08:
            status = "AMBIGUOUS"
        elif alternatives:
            # The snapshot holds one species, but GBIF lists regional congeners with this name.
            status = "OPEN_WORLD_AMBIGUOUS"
        elif candidates[0]["confidence"] == "HIGH":
            status = "RESOLVED_MODERN_NAME"
        else:
            status = "PROVISIONAL"
        unique = list({a["gbif_key"]: a for a in alternatives}.values())
        coverage = any('OPEN_WORLD_CHECK_NOT_AVAILABLE' in c['reasons'] for c in candidates)
        check = ({"status": "PARTIAL" if coverage else "RUN", **(self.congener_rule or {})} if self.congeners is not None else {"status": "NOT_RUN"})
        return {"query": name, "status": status, "confidence_mode": "QUALITATIVE", "candidates": candidates, "open_world_alternatives": unique, "open_world_check": check, "unresolved_possible": status != "RESOLVED_MODERN_NAME", "limitation": "Modern-name reconciliation is not proof of historical botanical identity. Candidates limited to the local snapshot; open-world alternatives are names only, with no evidence retrieved. Expanded common names may lack regional collision data; see each candidate's reasons.", "method": method, "version": self.version}
```

tests/research_evidence.test.mjs
SHA-256: 37391d750b7d737be73d1717b1c0642f9d6e522fdb4ac47be43791972ae030b8

```
import test from 'node:test';
import assert from 'node:assert/strict';
import {sourceContext, researchQueries, excludedFromPatientLeads} from '../frontend/demo/research-evidence.js';
test('multiple patient goals remain separate research questions',()=>{
  const queries=researchQueries('Turmeric','memory and joint pain');
  assert.deepEqual(queries.map(q=>q.goal),['memory','joint pain']);
  assert.ok(queries.every(q=>q.query.includes('in people')));
});
test('abstract excerpts preserve exact source characters and absent dimensions stay empty',()=>{
  const abstract='Background: A small study. Forty adult participants received extract 500 mg daily. The trial was randomized against placebo.';
  const rows=sourceContext({abstract});
  for(const row of rows){assert.ok(row.span);assert.equal(abstract.slice(row.span.start,row.span.end),row.span.text)}
  assert.ok(sourceContext({abstract:'No relevant information provided.'}).every(r=>r.span===null));
});
test('obvious nonclinical and protocol titles stay out of patient research leads',()=>{
  for(const title of ['Valerian stress in Japanese quails','Goldenseal in a rat burn wound model','Foliar application during drought','Curcumin trial protocol','Lipids in HepG2 cells'])assert.equal(excludedFromPatientLeads({title}),true);
  assert.equal(excludedFromPatientLeads({title:'Turmeric for knee osteoarthritis: a randomized trial'}),false);
});
```

tests/safety_engine.test.mjs
SHA-256: 872442a9e8a329eccd3f27840a4188659922de4aeb2aac171f4e493a63017d06

```
// node --test tests/safety_engine.test.mjs
import test from 'node:test';
import assert from 'node:assert/strict';
import {readFileSync, existsSync} from 'node:fs';
import {indexKnowledge, parseLabelText, fromDsldLabel, analyze, resolveMedicines, identify} from '../frontend/demo/safety-engine.js';
import {EXAMPLES} from '../frontend/demo/safety-examples.js';

const kb = JSON.parse(readFileSync(new URL('../frontend/demo/safety-knowledge.json', import.meta.url)));
const idx = indexKnowledge(kb);
const Q = f => f.quotes.map(q => q.text + ' ' + q.items.join(' ')).join(' ');
// What the on-device AI returns for the examples' everyday phrases (see reports/ai_medicine_understanding_*.json).
const AI_REPORT = JSON.parse(readFileSync(new URL('../reports/ai_medicine_understanding_v2_2026-11-12.json', import.meta.url)));
const AI_FIXTURE = Object.fromEntries(AI_REPORT.development.rows.filter(r=>r.predicted).map(r=>[r.text,{group:r.predicted,score:r.similarity}]));
const run = ex => analyze({products: ex.products.map(p => parseLabelText(p.text, p.name)), medicines: ex.medicines, conditions: ex.conditions, aiMedicines:
AI_FIXTURE}, idx);
// Page text cache written by `python scripts/build_safety_knowledge.py` (downloads the NIH pages).
const pagesUrl = new URL('../.runtime/nccih/pages.json', import.meta.url);
const pages = existsSync(pagesUrl) ? JSON.parse(readFileSync(pagesUrl, 'utf8')) : null;
const clean = s => s.normalize('NFKD').replace(/[\u2018\u2019]/g, "'").replace(/[\u201c\u201d]/g, '"').replace(/[\u2013\u2014]/g, '-').replace(/\s+/g, ' ');

test('Mei: ginkgo and garlic are flagged against warfarin, with NIH quotes', () => {
  const r = run(EXAMPLES[0]);
  const discuss = r.findings.filter(f => f.level === 'discuss' && !f.conditional);
  assert.ok(discuss.some(f => f.herb === 'ginkgo' && f.medicines.some(m => m.name === 'warfarin')));
  assert.ok(discuss.some(f => f.herb === 'garlic' && f.medicines.some(m => m.name === 'warfarin')));
  assert.ok(r.findings.some(f => f.herb === 'turmeric' && f.because === 'variant'), 'BioPerine marks a bioavailable curcumin formulation');
  assert.ok(r.gaps.some(g => g.type === 'identity' && g.term === 'ginseng'), 'bare ginseng stays ambiguous');
  assert.ok(r.gaps.some(g => g.type === 'blend' && g.ingredients.some(i => /garlic/i.test(i)) && g.blend === 'Proprietary Wellness Blend'), 'blend amounts flagged');
  assert.ok(r.findings.every(f => f.source.url.startsWith('https://www.nccih.nih.gov/health/')));
  assert.ok(!r.findings.some(f => f.herb === 'ginger' && f.level !== 'note'), 'no invented ginger interaction');
});

test('Sam: St. John\'s wort flags sertraline (serotonin) and birth control (weakened effect)', () => {
  const r = run(EXAMPLES[1]);
  const sjw = r.findings.filter(f => f.herb === 'st-johns-wort' && f.because === 'medicine');
  assert.ok(sjw.some(f => f.medicines.some(m => m.name === 'sertraline') && /serotonin/.test(Q(f))));
  assert.ok(sjw.some(f => f.medicines.some(m => m.name === 'birth control pill') && /weaken/.test(Q(f)) && /Birth control pills/.test(Q(f))));
  assert.ok(r.gaps.some(g => g.type === 'identity' && g.term === 'chamomile'), 'chamomile species unspecified');
  const cham = r.gaps.find(g => g.type === 'identity' && g.term === 'chamomile');
  assert.ok(cham.candidates.find(c => c.herb === 'chamomile').findings.some(f => f.medicines.some(m => m.name === 'birth control pill')), 'German chamomile statement
 is nested under the identity question');
  assert.ok(!r.findings.some(f => f.herb === 'chamomile'), 'not promoted to the main list');
});

test('Ray: inferred statin triggers the NIH red yeast rice quote; goldenseal vs metformin; duplicate green tea', () => {
  const r = run(EXAMPLES[2]);
  assert.ok(r.findings.some(f => f.herb === 'red-yeast-rice' && /lovastatin/.test(Q(f)) && f.medicines.some(m => m.input === 'my cholesterol pill' && m.inferred)),
'AI-understood statin triggers the red yeast rice statement');
  assert.ok(!r.findings.some(f => f.herb === 'green-tea' && /atorvastatin/.test(Q(f))), 'an atorvastatin-specific sentence is not applied to an unnamed statin');
  assert.ok(r.findings.some(f => f.herb === 'goldenseal' && f.medicines.some(m => m.name === 'metformin')));
  assert.ok(r.gaps.some(g => g.type === 'duplicate' && g.herb === 'green-tea'));
  const ginseng = r.gaps.find(g => g.type === 'identity' && g.term === 'ginseng');
  assert.ok(ginseng && ginseng.candidates.find(c => c.herb === 'asian-ginseng').flagged, 'diabetes caution only for the Asian ginseng candidate');
  assert.ok(!r.findings.some(f => f.herb === 'asian-ginseng'), 'Asian ginseng statements stay conditional');
  assert.ok(r.gaps.some(g => g.type === 'duplicate' && g.herb === 'green-tea' && g.products.length === 2), 'green tea flagged across both products');
```

```
  assert.equal(r.findings.find(f => f.herb === 'red-yeast-rice' && f.because === 'medicine').quotes.length, 2, 'two statin statements in one card');
});

test('Siberian ginseng is not treated as Asian ginseng', () => {
  const p = parseLabelText('Siberian ginseng (Eleutherococcus senticosus) root 500 mg', 'X');
  const f = identify(p, idx);
  assert.ok(!f.some(x => x.herb === 'asian-ginseng'));
  assert.ok(f.some(x => x.unverified));
});

test('medicine resolution: brands, doses, typos and unknowns', () => {
  const r = resolveMedicines(['Coumadin 5mg', 'warfrin', 'Zoloft 50 mg daily', 'Synthroid', 'mystery pill'], idx);
  assert.equal(r[0].name, 'warfarin'); assert.equal(r[1].name, 'warfarin'); assert.ok(r[1].corrected);
  assert.equal(r[2].name, 'sertraline'); assert.equal(r[3].name, 'levothyroxine'); assert.equal(r[4].known, false);
});

test('no medicines and no conditions: no medicine-specific flag is invented', () => {
  const r = analyze({products: [parseLabelText('Ginkgo biloba 120 mg', 'G')], medicines: [], conditions: []}, idx);
  assert.equal(r.findings.filter(f => f.because === 'medicine').length, 0);
  assert.equal(r.checked.length, 1);
});

test('OCR noise: fuzzy matching recovers misspelt herb names only in OCR mode', () => {
  const p = {...parseLabelText('Gink9o biloba 120 mg\nValerlan root 200 mg', 'OCR'), source: 'ocr'};
  const r = analyze({products: [p], medicines: ['zolpidem'], conditions: []}, idx);
  assert.ok(r.checked.some(c => c.herb === 'valerian' && c.basis === 'fuzzy'));
  assert.ok(r.findings.some(f => f.herb === 'valerian' && f.medicines.some(m => m.name === 'zolpidem')));
  const strict = analyze({products: [parseLabelText('Valerlan root 200 mg', 'T')], medicines: [], conditions: []}, idx);
  assert.equal(strict.checked.length, 0);
});

test('DSLD record: declared genus/species resolves identity; blends propagate', () => {
  const label = {id: 1, fullName: 'Test', brandName: 'B', ingredientRows: [
    {name: 'Ginseng', notes: 'PlantPart: root Genus: Panax Species: ginseng', category: 'botanical', quantity: [{quantity: 100, unit: 'mg'}]},
    {name: 'Herbal Blend', category: 'blend', quantity: [{quantity: 400, unit: 'mg'}], nestedRows: [
      {name: 'Ginkgo', notes: 'Genus: Ginkgo Species: biloba', category: 'botanical', quantity: []},
      {name: 'Astragalus', notes: 'Genus: Astragalus Species: membranaceus', category: 'botanical', quantity: []}]}]};
  const p = fromDsldLabel(label);
  const r = analyze({products: [p], medicines: ['warfarin'], conditions: []}, idx);
  assert.ok(r.checked.some(c => c.herb === 'asian-ginseng' && c.basis === 'label_species'));
  assert.ok(!r.gaps.some(g => g.type === 'identity'), 'label species removes the ambiguity');
  assert.ok(r.gaps.some(g => g.type === 'blend' && g.ingredients.includes('Ginkgo') && g.ingredients.includes('Astragalus') && g.total === '400 mg'));
  assert.ok(r.gaps.some(g => g.type === 'unverified' && /Astragalus/.test(g.name)));
  assert.ok(r.findings.some(f => f.herb === 'ginkgo' && f.level === 'discuss'));
});

test('every quote in the knowledge base occurs verbatim in the cached NIH page', {skip: pages ? false : 'run `python scripts/build_safety_knowledge.py` first to download the NIH pages'}, () => {
  let n = 0;
  for (const h of kb.herbs) {
    const text = clean(pages[h.source].text);
    for (const q of [...h.rules.map(r => r.quote), ...(h.variants || []).map(v => v.quote)]) { n++; assert.ok(text.includes(clean(q)), h.id + ': ' + q.slice(0, 60)); }
  }
  assert.ok(n >= 60);
});

test('regression: a qualified non-covered species is reported, never silently dropped', () => {
```

```
  const r = analyze({products: [parseLabelText('Ginseng (Panax quinquefolius) root 200 mg\nVitamin C 500 mg', 'X')], medicines: ['warfarin'], conditions: []}, idx);
  assert.ok(r.gaps.some(g => g.type === 'unverified' && /quinquefolius/.test(g.name)));
  assert.ok(r.unmatched.some(u => /Vitamin C/.test(u.ingredient)), 'non-herb lines are listed as not checked');
  assert.ok(!r.findings.some(f => f.herb === 'asian-ginseng'));
});

test('regression: a condition-specific caution is not repeated as a general caution', () => {
  const r = analyze({products: [parseLabelText('Kava root 100 mg', 'K')], medicines: [], conditions: ['liver']}, idx);
  const liver = r.findings.filter(f => f.herb === 'kava' && f.quotes.some(q => /liver injury/.test(q.text)));
  assert.equal(liver.length, 1);
  assert.equal(liver[0].because, 'condition');
});

test('AI-understood medicine: a lay phrase triggers the same NIH statement, marked as inferred', () => {
  const phrase = 'the clot medicine after my stroke';
  const products = [parseLabelText('Ginkgo biloba leaf extract 120 mg', 'G')];
  const withAi = analyze({products, medicines: [phrase], conditions: [], aiMedicines: {[phrase]: {group: 'blood_thinner', score: 0.62}}}, idx);
  const f = withAi.findings.find(x => x.herb === 'ginkgo' && x.because === 'medicine');
  assert.ok(f && f.medicines[0].inferred && f.medicines[0].label === 'a blood thinner (anticoagulant)');
  assert.ok(/which I think is a blood thinner/.test(f.question) && /exactly/.test(f.question));
  assert.ok(withAi.gaps.some(g => g.type === 'inferred_medicine' && g.input === phrase));
  assert.ok(!withAi.gaps.some(g => g.type === 'unknown_medicine'));
  const noAi = analyze({products, medicines: [phrase], conditions: []}, idx);
  assert.ok(!noAi.findings.some(x => x.because === 'medicine') && noAi.gaps.some(g => g.type === 'unknown_medicine'));
  const unsure = analyze({products, medicines: [phrase], conditions: [], aiMedicines: {[phrase]: {group: null, best: 'blood_thinner', score: 0.44}}}, idx);
  assert.ok(!unsure.findings.some(x => x.because === 'medicine'), 'below the threshold nothing is inferred');
});

test('dictionary beats AI: a known medicine name is never overridden by an AI reading', () => {
  const r = resolveMedicines(['warfarin'], idx, {warfarin: {group: 'cholesterol', score: 0.9}});
  assert.equal(r[0].name, 'warfarin'); assert.ok(!r[0].inferred);
});

test('negated and historical names never become current medicine matches, even with supplied AI metadata', () => {
  const phrases = ['I stopped taking warfarin', 'not taking my statin', 'herbal sleeping tea'];
  const r = analyze({products:[parseLabelText('Ginkgo biloba 120 mg\nRed yeast rice 100 mg\nValerian 200 mg','X')],medicines:phrases,
    aiMedicines:Object.fromEntries(phrases.map(t=>[t,{group:'blood_thinner',score:.99}]))},idx);
  assert.equal(r.findings.filter(f=>f.because==='medicine').length,0);
  assert.equal(r.gaps.filter(g=>g.type==='unknown_medicine').length,phrases.length);
});

test('persisted or shared AI metadata below the active threshold cannot trigger a warning', () => {
  const phrase='my cholesterol pill';
  for(const score of [.54,NaN,Infinity,undefined]) {
    const r=analyze({products:[parseLabelText('Red yeast rice 1200 mg','X')],medicines:[phrase],aiMedicines:{[phrase]:{group:'cholesterol',score}}},idx);
    assert.ok(!r.findings.some(f=>f.because==='medicine'));
  }
});
```